O'REILLY®

Full Stack Testing

A Practical Guide for Delivering High Quality Software

Compliments of

harness



Gayathri Mohan

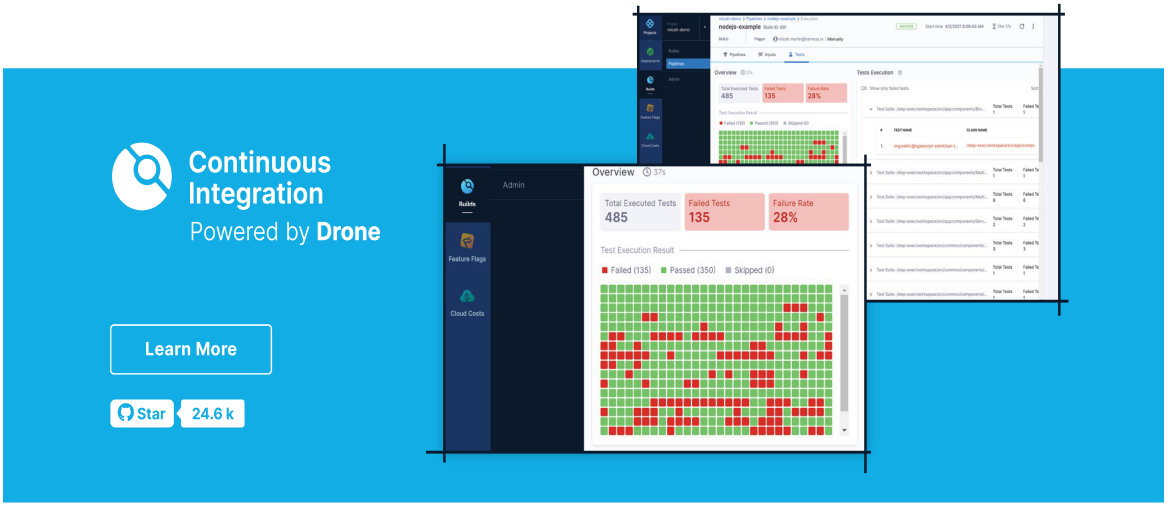
Foreword by Dr. Rebecca Parsons

Harness

harness

Test 98% Faster.
Scale with Confidence.

The intelligent CI solution
built by – and for – developers.



Harness — Your Intelligent Multi-cloud Software Delivery Platform

Fully integrated modules with one pipeline for all. No more scripts, plugins, version dependencies, toil, downtime, or frustrated developers.

Continuous | Continuous | Cloud Cost


Integration


Delivery

Management


Feature Flags


Service Reliability Management


Security Testing Orchestration

Praise for *Full Stack Testing*

From manual exploratory testing to creating test strategies across various quality dimensions and working with emerging technologies, this book covers a lot of ground for beginner as well as experienced quality analysts. Gayathri has done a phenomenal job of distilling just enough theory to introduce the topic and follow it with practical examples so you can apply them in your projects with existing tools and frameworks.

—Bharani Subramaniam, head of technology for Thoughtworks India

An expansive survey of testing strategies and patterns that covers its subject in both breadth and depth. The theoretical underpinnings of various forms of testing are backed by practical, hands-on examples in several chapters. Gayathri's book should find its way to the desktops of people who write (and, therefore, are bound to test) software.

—Saleem Siddiqui, author of *Learning Test-Driven Development*

This book provides a bird's-eye view of full stack testing and will help you learn about testing and enhance corporate processes related to software testing. I would recommend the book to quality assurance engineers, technical project managers, and software architects. The book gives a railway map of different paths and approaches that can be applied and investigated depending on the application scope, budget, and time frames.

—Nigar Akif Movsumova, software engineer at EPAM Systems

The term full stack development refers to additional skills a developer should have to carry out their job. Full stack testing pertains to the software being tested, and it encompasses all technologies, processes, people skills, and various types of testing that are to be performed to make software better. Full Stack Testing by Gayathri Mohan insightfully covers these multifaceted topics, empowering readers to deliver high-quality software.

—Srinivasan Desikan, adjunct professor and author of *Software Testing: Principles and Practices*

Like the proverbial blindfolded team members trying to individually feel their way to understanding an elephant, Gayathri's book provides the necessary perspective for teams to understand a holistic view of testing. While individual testing yields positive results, understanding the full stack enables better whole-project outcomes.

—Neal Ford, director/software architect/meme wrangler at Thoughtworks and author of *Software Architecture: The Hard Parts*

Full Stack Testing

A Practical Guide for Delivering High Quality Software

Gayathri Mohan

Full Stack Testing

by Gayathri Mohan

Copyright © 2022 Gayathri Mohan. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Melissa Duffield

Development Editor: Jill Leonard

Production Editor: Jonathon Owen

Copyeditor: Rachel Head

Proofreader: Liz Wheeler

Indexer: nSight, Inc.

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

June 2022: First Edition

Revision History for the First Edition

- 2022-06-03: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781098108137* for release details.

The O’Reilly logo is a registered trademark of O’Reilly Media, Inc. *Full Stack Testing*, the cover image, and related trade dress are trademarks of O’Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher’s views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O’Reilly and Harness. See our statement of editorial independence.

978-1-098-13317-7

[LSI]

Foreword

The term *shift left*, which refers to performing an activity earlier or to the left along a timeline, is becoming increasingly common. We hear about why it is important to shift left design, security, and, most relevant here, testing. Bringing testing forward in the software development life cycle decreases the cost and complexity of fixing bugs because they are found closer to when they were created, which establishes more of a context for what could have caused things to go wrong. When we think about things like performance testing, we can start looking for trends before we start actually worrying about the specific values. This again allows us to spot when performance worsens significantly. We can then explore if this means that we've hit something that is fundamentally less performant or if perhaps we just made a mistake that caused the performance degradation.

While shifting testing left means tests are run against software that is known to be incomplete and subject to change, the enhanced ability to fix issues that arise far outweighs the costs of continuous testing, particularly when a significant portion of the test suite is automated. While some tests and some styles of testing, like exploratory testing, need to be done manually, tests that can be automated should be.

There are all kinds of testing to be done, and the title of Gayathri's book is apt. Full stack testing gives a comprehensive overview of testing across the entire stack, looking at performance, UI, contract, end-to-end functional, unit testing, and even accessibility testing. The question for many involved in testing revolves around knowing how to do testing across the full stack. That's where this book comes in. While there are many books about testing and even about agile testing, which does advocate for shifting testing left, Gayathri's book looks in-depth at each aspect of testing a modern application. The book describes the issues that arise in each aspect of testing and looks at principles and strategies that apply to that aspect of testing.

Each of these sections then includes a set of hands-on exercises that demonstrate concretely how to actually do such testing. Now, I recognize the specific exercises and the tools included in the exercises may change and evolve in time. However, these exercises are of value even if the tools do change, because they show how to use tools to construct the right kinds of tests. The exercises make the testing approach concrete; the tools provide the ability to experiment on tests of that nature. The tools will inevitably continue to evolve, but the testing strategies you will learn will have a much longer shelf life.

The range of testing approaches in Gayathri's book is broad, encompassing static analysis, data testing strategies, and even exploratory testing. Given the growing complexity of our software systems, the role of exploratory testing becomes increasingly important. In addition, security testing is given its own chapter, as we all know how much more vulnerable our systems are to hackers. Accessibility testing also has its chapter, describing how we can make our systems easier to use, even for those who are disabled.

Each aspect of testing requires looking at what kinds of things might go wrong and then creating a testing strategy to uncover things that have gone wrong. A properly constructed test suite across the range of test types provides the safety net that allows us to evolve our software systems with confidence. Gayathri's book, based on her experience testing different types of systems, guides software professionals in creating the proper testing strategies and suites.

Dr. Rebecca Parsons
Chief Technology Officer at Thoughtworks, coauthor of Building Evolutionary Architectures

Preface

If you're in the software industry, it is highly unlikely that you won't have worn the testing hat at least once, irrespective of your role. That's because testing is such an integral aspect of software engineering, woven into every stage of the software delivery cycle. With the exponential adoption of digitization today, where various web and mobile applications have become so enmeshed in people's daily lives, testing along various quality dimensions has become imperative.

When we look at testing as a software discipline, we can see how it has undergone its own trajectory of evolution over the many decades of its existence, growing to incorporate new practices, frameworks, methodologies, and tools. Manual testing has evolved into manual exploratory testing, and remains a fundamental part of the testing discipline today. In the meantime, the rise of automated testing combined with continuous integration and continuous deployment (CI/CD) practices has caused the value derived from testing to skyrocket. Moving beyond functional use cases, automated testing of cross-functional requirements such as performance, security, and reliability to receive holistic feedback and continuously deliver high-quality software is the critical need of the hour. This is why full stack testing is viewed as a desirable specialization today in the industry. I presume you're here because you want to transpile into a full stack tester so you can deliver high-quality software at work—first, kudos to your commitment, and second, welcome aboard!

Why I Wrote This Book

I would like to humbly tell you that many testing experts before me could have written this book, and it didn't need to be me. Perhaps

their responsibilities did not allow them the time, or they lacked the inclination; whatever the reason, the opportunity has fallen to me, and I am grateful for it! (Although if some other expert had written this book back when I was a beginner in testing, it would have saved me a lot of effort: I had to rummage through hundreds of blogs and try out dozens of tools myself to acquire the testing skills I've accumulated over many years.)

Through my experience consulting with clients in my day-to-day job, I've observed that the teams that have implemented a wise testing strategy have mostly succeeded, while most of those that didn't failed miserably. For instance, I have seen client teams that relied exclusively on UI-driven end-to-end tests and burned themselves out with maintenance tasks, or that only did manual testing and faced a lot of production defects. Some teams only did functional testing, failing to uncover critical non-functional issues. Overall, such teams were characterized by poor software quality, an unhappy team, and a lack of competitive edge. It's a surprise to me that such a skew in the understanding of testing practices still exists today, when testing as a discipline has been around for decades. I can only assume that this is largely due to a lack of testing talent in the industry, and with the ongoing cold war among the software companies to plunder the best talent, it is only fair to share and spread the knowledge widely.

Although there are several testing tutorials on individual tools out there, there isn't a coherent narrative on how to upskill oneself on the current testing trends with practical examples using different tools. And for many niche skills like security and accessibility testing, consumable materials for beginners to read are not widely available. This book aims to be a comprehensive resource that will enable a beginner in testing to upskill themselves to an advanced beginner level in all of the skills essential for web and mobile application testing today.

If you're wondering what I mean by advanced beginner, I'm referring to the Dreyfus model of skill acquisition, which elaborates five stages

through which an individual progresses as they gain a skill: novice, advanced beginner, competent, proficient, and expert. This book is written with the ambitious goal of catapulting its readers through the first two stages across 10 different testing skills, with practical examples. Given that the third stage is competent, which can be achieved only with extensive practice, I believe the book takes its readers as far as it can!

Who Should Read This Book?

This book is primarily tailored for beginners in software testing and existing software testing professionals who want to expand their breadth of knowledge. That said, any software role whose responsibilities overlap with testing, such as an application developer or DevOps engineer, could benefit from the book. In all cases, a fundamental requirement is to possess some coding knowledge, especially in Java, as the book has hands-on exercises in Java and, in some places, JavaScript. Also, if you are a reader who is new to the software industry, I would recommend doing a preliminary read on software development processes such as the Agile and waterfall methodologies before diving into this book.

Navigating This Book

The book starts with an introduction to full stack testing and elaborates on the 10 testing skills that are essential to delivering high-quality web and mobile applications. Once the foundations are established, there are 10 independent skill development chapters. Each of these chapters contains the following structural elements:

- The topics essential for context setting are grouped under the “Building Blocks” heading. If you are new to the skill, this section will give insight into what it involves and why and where the skill needs to be applied.

- This is followed by a strategy section, which elaborates on how to apply the skill in a given situation.
- Then there are exercises that guide the readers with step-by-step instructions on executing the skill using multiple tools.
- There is also an “Explore More Tools” section in some chapters, where parallel tools that are similar to the ones discussed in the exercises section, or other tools that may add value at some point for the readers during their practice, are discussed further to enrich the reader’s grasp of the skill.
- Lastly, you will find my perspectives, based on personal observations and experience, in some of the chapters, followed by key takeaways, which are a concise overview of the lessons learned in each chapter.

After the 10 skill development chapters, the book talks about how to move further in testing with the help of first principles and individual soft skills. There is also a bonus chapter for enthusiastic readers that serves as an introduction to testing in emerging technologies. It presents a brief on testing in four emerging technologies—AI/ML, blockchain, IoT, and AR/VR—with the intention of assisting readers in kick-starting their learning in those areas as well.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names,

databases, data types, environment variables, statements, and keywords.

`Constant width bold`

Shows commands or other text that should be typed literally by the user.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

> **TIP**
>
> This element signifies a tip or suggestion.

> **NOTE**
>
> This element signifies a general note.

> **WARNING**
>
> This element indicates a warning or caution.

O’Reilly Online Learning

> **NOTE**
>
> For more than 40 years, *O’Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-998-9938 (in the United States or Canada)

707-829-0515 (international or local)

707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/full-stack-testing*.

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly-media*

Follow us on Twitter: *https://twitter.com/oreillymedia*

Watch us on YouTube: *https://www.youtube.com/oreillymedia*

Acknowledgments

Earlier in my career, not in my wildest imagination would I have thought about writing a full-fledged technical book—and that too for O'Reilly! It was the inspiration, motivation, and nurturing environment provided by Thoughtworks that led me along this path, and I am incredibly thankful for being associated with such a lovely group of passionate technologists and encouraging leaders. In no particular order, I would like to express my appreciation to and acknowledge the support I have received from some of the amazing folks at Thoughtworks: Prasanna Pendse, who nudges everyone around to set high goals and, when I set myself up for this, saw to it that I got the appropriate support till the end; Bharani Subramanian, who worked closely with me until the book's completion, sharing his illuminating ideas that led to the shaping of each of the chapters; and Pallavi Vadlamani, a close friend more than a colleague, who also worked closely with me right from the early stages and reviewed every chapter. Satish Viswanathan, Kief Morris, Sriram Narayan, Neal Ford, and Sudhir Tiwari are few others who have extended their support throughout various stages of this book's development; truly, it is invaluable to have such knowledgeable folks share their wise and timely directions! I would also like to specially thank Dr. Rebecca Parsons, Thoughtworks' CTO and my role model, who wrote the foreword and was kind enough to volunteer to review the chapters from the draft stages. What more support could I really ask for from an organization?!

My sincere gratitude to the O'Reilly crew: especially Jill Leonard and Melissa Duffield, for setting the appropriate space for the book to be launched successfully, and the technical reviewers, Chris Northwood, Alexander Tarlinder, Srinivasan Desikan, Saleem

Siddiqui, Ian Molyneaux, and Nigar Movsumova, who have provided feedback on every granular detail and got the book to the state it is in today.

I also want to register my exquisite appreciation of and gratitude to my long-term mentor, Dhivya Arunagiri, who has spent several years boosting my confidence and helping me shape my career, and my friends, who have been a solid source of comfort whenever I got exhausted from writing alongside work and family commitments amidst a pandemic. I also take this opportunity to express my heartfelt love and appreciation to my ever encouraging and supporting parents.

Last but not the least, a special callout to my dear husband, Manoj Mahalingam, who is an inspiration, a friend, and a guide and without whom this book wouldn't exist today. I would like to dedicate this book to him and my lovely daughter, Magathi Manoj, for allowing me the much-needed mind space and time through several nights, weekends, and holidays for more than a year as I worked on this project.

Indeed, as I write this, I am thinking how blessed I am to be surrounded by such an amazing bunch of family, friends, and colleagues. Thank you very much, everyone! I am forever grateful.

Chapter 1. Introduction to Full Stack Testing

In today's world, digitalization is required to sustain and grow any business. Many businesses are leading the way in this aspect, while some are in the early phases of modernizing their existing digital platforms.

Digitalization is key to broadening a business's reach from a local community to a global scale, translating to more adoption and more revenue. Almost all small and large-scale enterprises in various sectors, like health care, retail, travel, academics, social media, banking, and entertainment, devise plans to advance their digital strategies as a critical measure to reach new customer segments and yield higher profits.

In this journey toward digitalization and modernization, innovation becomes a crucial driver. The businesses that innovate constantly continue to stay relevant and thrive over many decades. Netflix is a classic example: it started as an online DVD rental portal in the 1990s, then ventured into online streaming in 2007, cannibalizing its own DVD rental business. It later started producing original content, called *Netflix Originals*. As of the end of 2021, Netflix was the largest online streaming service with well over 200 million global subscribers.

The technology space has been evolving in parallel with these innovative businesses to accommodate their increasingly advanced needs. Gone are the days where people were willing to wait in line to buy movie tickets, drive to a store in a remote location to buy a specialty product, or carry around a handwritten shopping list searching for specifics. Technology eases such everyday tasks. We can sit at home and stream our favorite entertainment programs at

the touch of a button, try on a new dress virtually, schedule regular delivery of the items on our grocery lists, brew a coffee with voice command, and more.

With the rapid pace of evolution in technology, product strategies must be versatile, catering to different customer needs to fend off competition in the sector at hand. It's no longer enough to just build a website; horizons must be broadened. Consider ride-hailing companies like Uber and Lyft, which provide varying ways of accessing their services: the web, Android and iOS mobile platforms, even a WhatsApp chatbot. This kind of versatile product strategy has helped these companies expand across the globe and outgrow their competitors.

Innovation and versatility help businesses be successful in acquiring a critical mass of customers. But the challenge then is to thrive further, earning more revenue and gaining more customers. We've seen how industry giants like Amazon leverage their existing customer base to cross-sell services and products as a strategy for expansion. Amazon, which started as an online bookstore, now cross-sells products ranging from fresh pantry items to electronics to apparel, jewelry, and more, meeting customer demand for consumer goods in nearly every market segment.

Why are we discussing these details in a book on software testing? Because today's software industry caters to all these business needs, providing the tools to innovate new product ideas, bring them to life, and scale them to reach new customer segments across the globe. Unquestionably, software development teams are standing on the edge, especially when the need of the hour is to deliver *with high quality*! Indeed, software quality has become a nonnegotiable criterion in today's competitive market. Compromising on software quality is equivalent to running a race knowing that you will lose it. This has been reinforced by many real-world examples. For instance, in October 2014 Indian ecommerce giants Snapdeal and Flipkart went head to head on their seasonal sale after months of

marketing. Unfortunately, Flipkart's website crashed multiple times during the "Big Billion Day" sale due to overwhelming demand, causing it to lose many customers and a great deal of revenue to Snapdeal. Similarly, Yahoo! failed to keep pace with its competitors (in spite of being one of the first of its kind in the market), in part because it failed to pay attention to the quality of its search product and in part because of damage to the brand caused by poor security measures, which led to the biggest data breach in history, exposing 3 billion user accounts in 2013. Such is the impact of software quality today!

There are many similar examples across the globe that reinforce the observation that businesses, despite whatever novel product ideas they might have, face a steep and slippery slope when quality is compromised, as customers quickly move on to more reliable competitors. At times, businesses may be forced to choose time-to-market over software quality, but they should be aware that they have only created a debt for themselves that needs to be resolved before their competitors use it to their advantage. Thus, we can firmly say that quality is quintessential for sustaining a business in the long term—and high quality can only be achieved through a combination of skillful development and meticulous testing, paying attention to every aspect of the application throughout its stack. To get you started on this path, this chapter will introduce what's involved in full stack testing for a typical web or mobile application.

Full Stack Testing for High Quality

To begin with, let us come together on a common understanding of *software quality*. Software quality was once equated to a bug-free application—but anyone in the software world today will agree that it's not just that anymore. If you ask end users to define quality, you will hear them speak of ease of use, look and feel, data privacy, swiftness in rendering information, and 24/7 availability of services. If

you ask businesses to define quality, you will hear about return on investment, real-time analytics, zero downtime, no vendor lock-in, scalable infrastructure, data security, legal compliance, and more. All of these are aspects of what makes an application high-quality software today. Failings in any of these areas will affect the quality in some way or another, which is why we need to test for them meticulously!

Though the list of quality requirements looks tall, we have tools and methodologies to cater to most of these needs. So, the bridge to high quality is made up of knowledge of those tools and, more importantly, the skill to apply them in a given context, both from a development and a testing perspective. This book aims to help you build that bridge, teaching you the testing skills you need to deliver high-quality web and mobile applications.

Testing, in a nutshell, is a practice to validate that the behavior of the application is as expected throughout. For testing to be successful, it needs to be practiced at the micro and macro levels. It has to be entwined with the granular aspects of the application, such as testing every method in a class, every input data value, log message, error code, and so on. Similarly, it has to focus on macro aspects such as testing features, integrations between features, and end-to-end workflows. But we cannot stop testing there! We need to further test the holistic quality aspects of the application—security, performance, accessibility, usability, and more—to achieve the end goal of delivering high-quality software. We can encapsulate all of that by saying we need to do *full stack testing*! As represented in Figure 1-1, full stack testing involves testing different aspects of the application's quality in each layer (database, services, and UI), and the application as a whole.

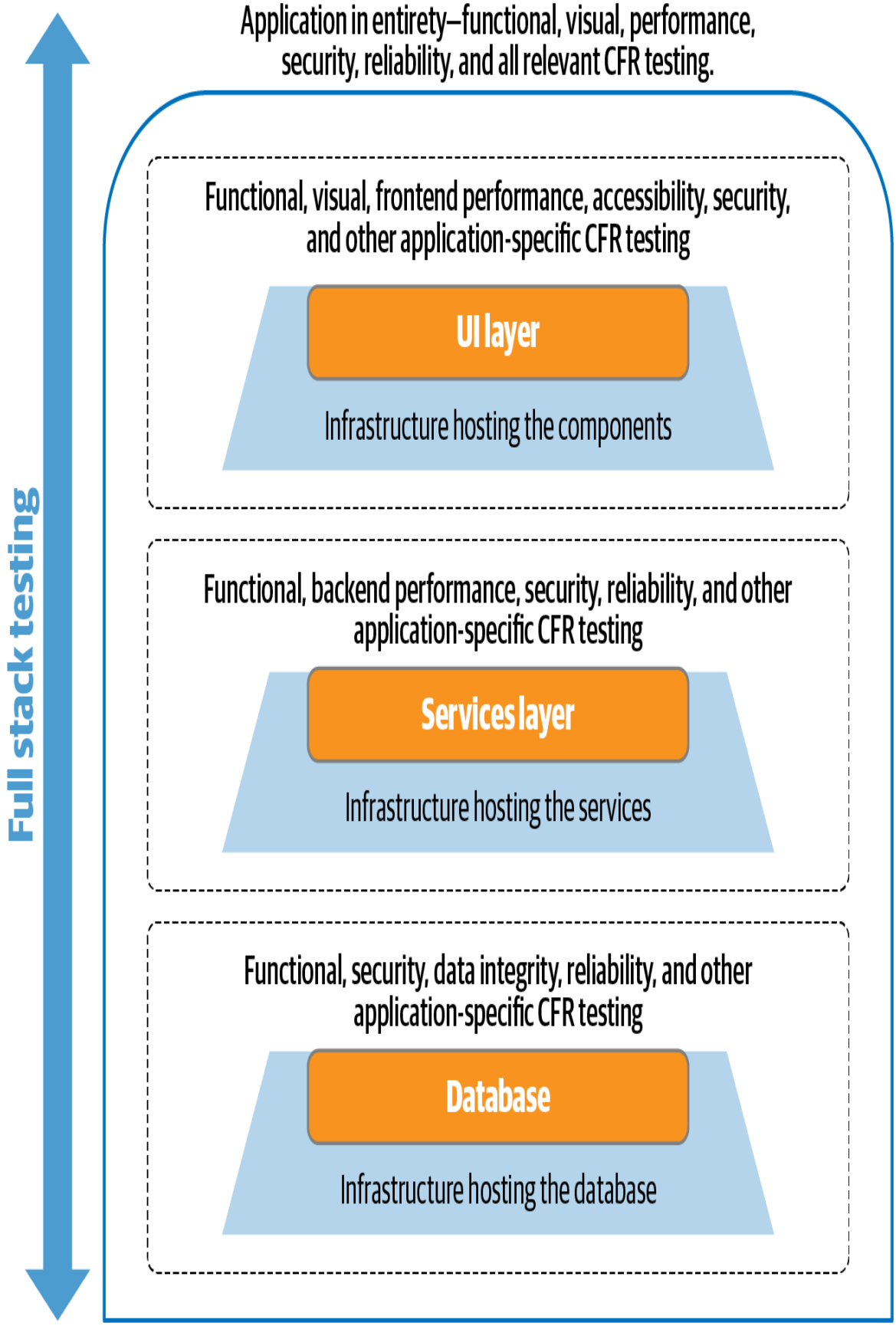

Application in entirety—functional, visual, performance, security, reliability, and all relevant CFR testing.
Full stack testing
Functional, visual, frontend performance, accessibility, security, and other application-specific CFR testing
UI layer
Infrastructure hosting the components
Functional, backend performance, security, reliability, and other application-specific CFR testing
Services layer
Infrastructure hosting the services
Functional, security, data integrity, reliability, and other application-specific CFR testing
Database
Infrastructure hosting the database

Figure 1-1. A representation of full stack testing

Indeed, full stack testing and development should be inseparable, like the two rails of a railway track. We must advance along both rails simultaneously to build quality into the product; otherwise, we are guaranteed to derail. For example, suppose we are writing a small block of code to calculate the total order amount for an ecommerce application. We need to test whether the code is computing the right amount and whether it is secure in parallel. If we don't do this, we might wind up with gaps in the railway line, and if we continue to develop on top of this fractured line we will have poor integration and suboptimal functionality. To ingrain testing at such an elementary level, teams need to stop thinking of it as a siloed post-development activity, as it was done traditionally. Full stack testing needs to begin in parallel with development and be practiced throughout the delivery cycle, giving faster feedback. The practice of starting testing early in the delivery cycle is referred to as *shift-left testing,* and it's a key principle to follow for full stack testing to yield the right results.

Shift-Left Testing

If we were to write down the sequence of activities in a traditional software development lifecycle, it would read *requirements analysis*, *design*, *development*, and *testing*, with testing coming at the end. As seen in Figure 1-2, shift-left testing suggests shifting the testing activities to the beginning of the cycle instead to produce high-quality results.

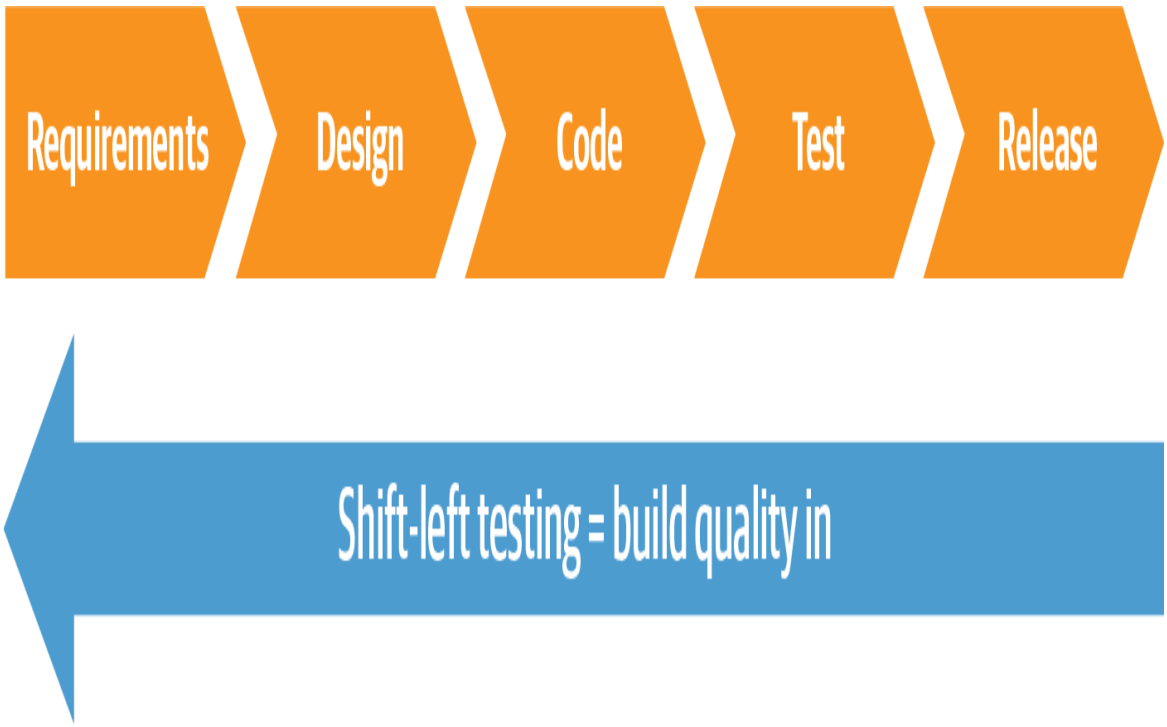


Figure 1-2. Shift-left testing

Let's consider an analogy to better illustrate this concept. Imagine your team is building a house. Does it seem sensible to complete the construction fully and only then check for quality? What if you find out the rooms are not of the correct sizes, or the interior walls are not strong enough to bear the load? Those are the kinds of issues shift-left testing tries to avoid, by implementing quality checks right from the planning stage and continuing them throughout the development phase. This allows for the end product to be of the highest possible quality.

Continuing quality checks throughout the development phase means repeating them iteratively for every small chunk of work, so that the needed changes can be incorporated smoothly. In the house construction analogy, it means performing these checks as each wall is built so that any issues are corrected immediately. To perform such extensive tests, shift-left testing relies heavily on automated testing and CI/CD practices, where the quality checks are automated at the micro and macro levels and continuously run against every small chunk of work in the CI server. This ensures the application is

continuously tested, with minimal cost and effort compared to manually testing every small chunk of work for multiple quality aspects.

To see what this means in a software context, let's break down shift-left testing into day-to-day activities. Consider a software team that follows an iterative development cycle, such as in Agile development. Some of the quality checks they may do in different phases of delivery to shift testing to the left are captured in Figure 1-3.

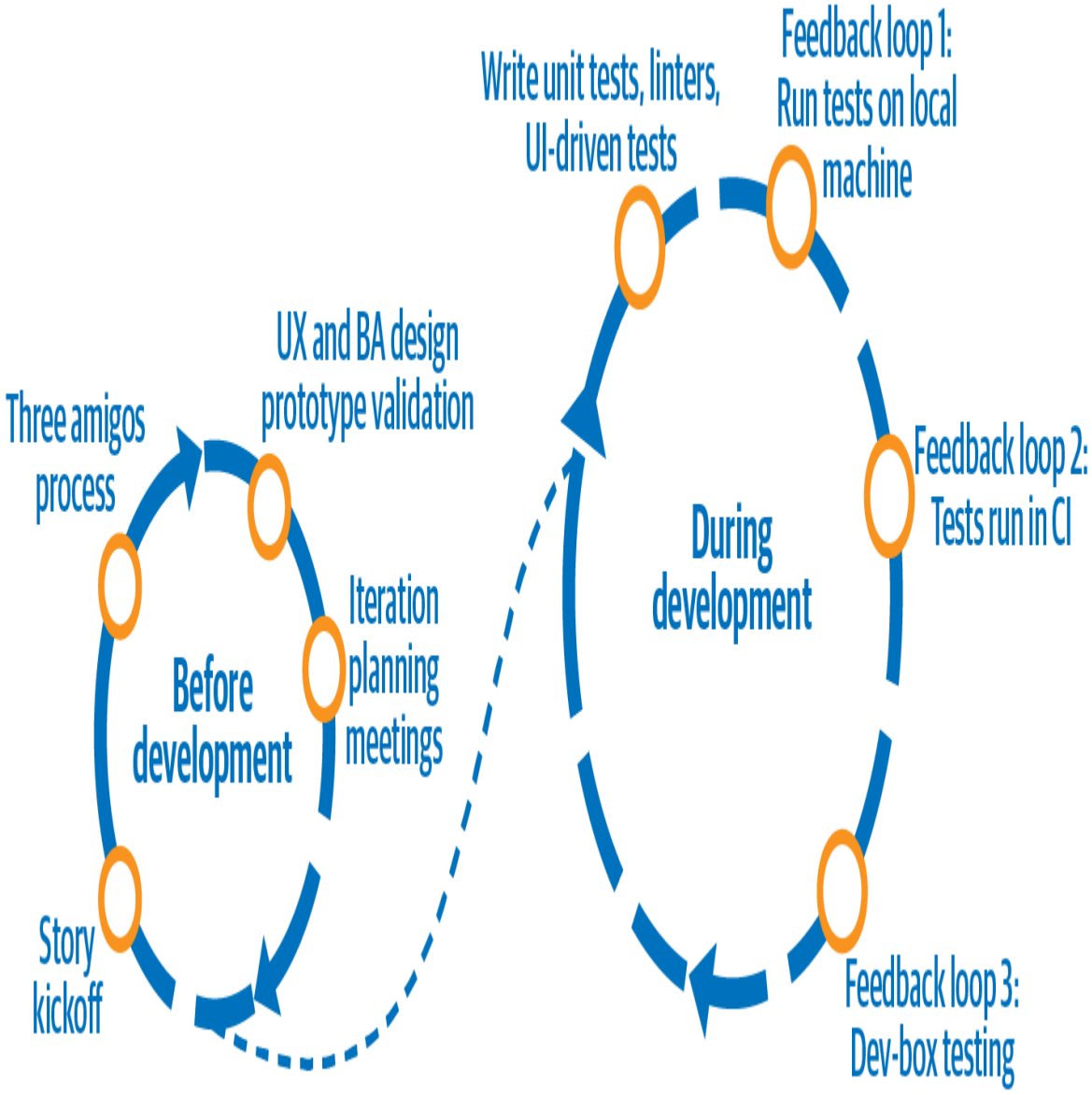


Figure 1-3. A set of quality checks shifted left

Reading Figure 1-3 from the left, it begins with highlighting a set of quality checks that are carried out by the team before a user story is considered ready for development:

- A ceremony called the *three amigos process* is conducted in the analysis phase. Here the business representatives, developers, and testers gather briefly to mull over the feature thoroughly. The process aims to collect all three roles' perspectives so that integrations, edge cases, and other business requirements don't

get overlooked. This is the first step in shifting left, where the requirements of a feature are validated to begin with.

- In parallel, the business representative on the team works with the user experience (UX) designer to validate and improve the application design.
- Once these two steps are completed, an *iteration planning meeting* (IPM) is conducted at the beginning of the iteration/sprint to discuss the user stories of that iteration in detail. This provides an open space for the team to collectively validate the requirements once again.
- During the iteration, just before a user story is picked up for development, a *story kickoff* happens. The story kickoff is a minified version of the three amigos process where the focus of discussion gets deeper into that particular user story's requirements and edge cases. By this stage, we can fairly say that the team has tested/validated the requirements diligently.

Similarly, while developing a user story, the following quality checks are implanted and utilized to get fast feedback:

- Developers write unit tests as part of each story and integrate them with CI. They also add linting tools and plug-ins for static code analysis and integrate them with CI to get continuous feedback.
- In some teams, developers also write the UI-driven functional tests as part of user story development and integrate those with CI. In other teams, testers write them post-development; both are common practices.
- Before committing the latest changes, developers run a set of automated tests on their local machines to get the first level of feedback.

- The second level of feedback is obtained from the suite of automated tests (unit, service, UI, etc.) that are run during CI for every commit.
- The third level of feedback is received from a process called *dev-box testing*, where the testers and the business representatives do a quick round of manual exploratory testing on a developer's machine to quickly verify the newly developed functionality.

With such rigorous focus on providing faster feedback, the team will get almost half of the feedback that would have otherwise been gained through manual testing post-development before the user story even gets to the testing phase itself. In other words, the team just shifted testing to the left, in the process giving the testers on the team the liberty to fully explore the user story for various quality aspects rather than just verifying the expected functional behaviors.

Thus, shift-left testing both enables defect prevention (by having multiple rounds of validation on the requirements) and assists in catching any defects that do creep in early, either on a local developer's machine or in CI. In addition, it ensures that high-quality software is delivered by giving testers the space to explore various quality aspects in depth.

> **NOTE**
>
> Extreme Programming (XP) is a flavor of Agile software development framework that incorporates shift-left testing. If you'd like to delve further into XP methodologies and practices, Kent Beck's *Extreme Programming Explained* (Addison-Wesley Professional) is a recommended read.

This concept of incorporating testing earlier in the delivery cycle is not restricted to functional application testing. It can be applied to

testing in general, including security testing, performance testing, and more. For example, one of the many ways to shift security testing to the left is to use a pre-commit scanning tool like Talisman, which scans the commit for secrets and alerts even before checking in the code. In each of the upcoming chapters, you will see practical approaches to shift-left testing.

Overall, this approach embodies the aphorism “Quality is the team’s responsibility,” as performing quality checks at every phase of the software development life cycle—validating application design prototypes, requirements, and so on, as discussed earlier—has to be owned by different team members. So, we can say that building the relevant testing skills to perform various quality checks is crucial for all the roles in a team to deliver high-quality software successfully!

Ten Full Stack Testing Skills

When we think of testing skills, we tend to consolidate them into two broad skills—manual and automated testing. But technology has evolved over the course of the last several decades, and these broad terms mask the essential new skills that one has to learn to perform various quality checks and deliver high-quality web and mobile applications. Figure 1-4 shows the 10 full stack testing skills that will enable us to perform full stack testing efficiently.

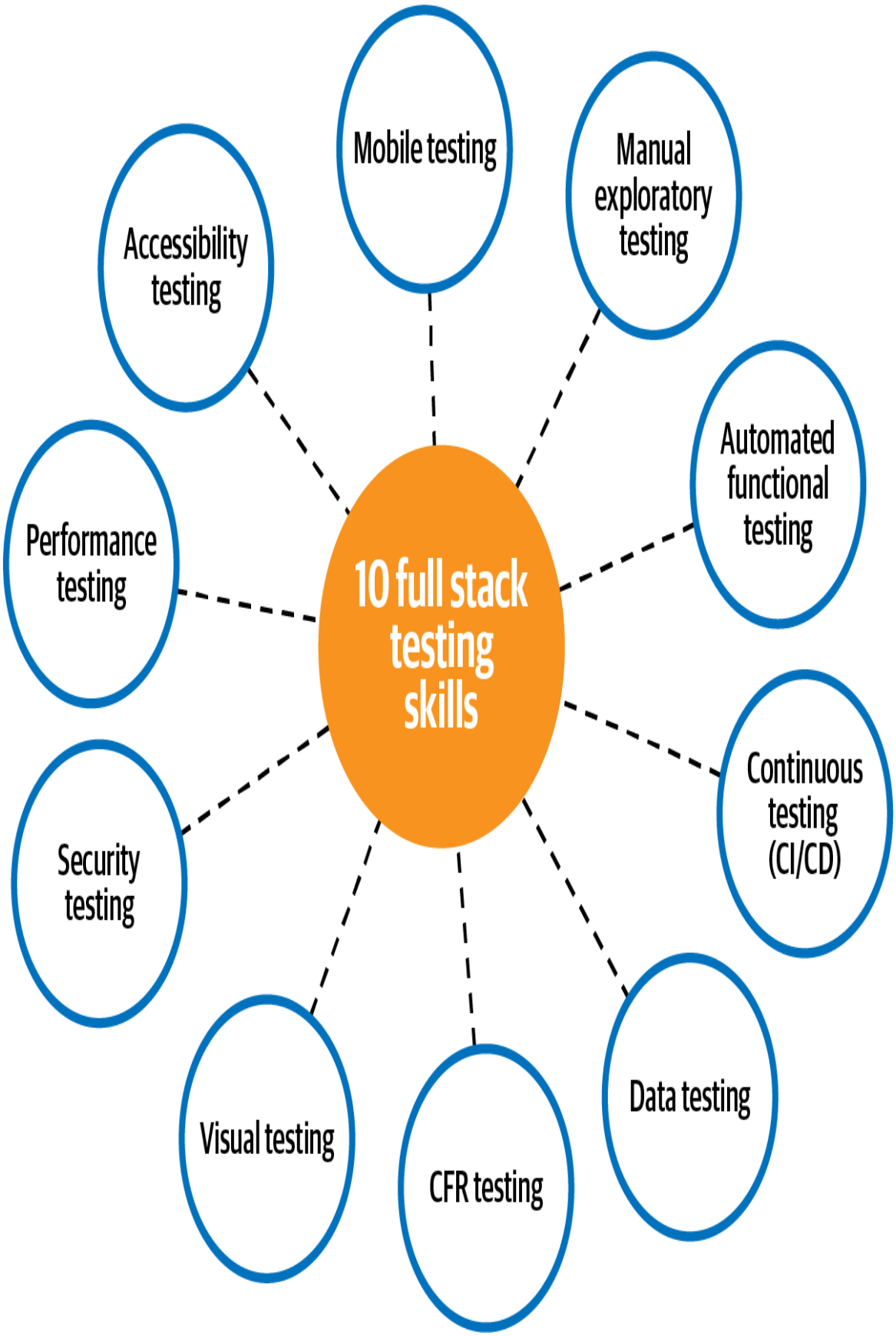

10 full stack testing skills
Mobile testing
Manual exploratory testing
Accessibility testing
Automated functional testing
Performance testing
Continuous testing (CI/CD)
Security testing
Data testing
Visual testing
CFR testing

Figure 1-4. Ten full stack testing skills needed for delivering high-quality web and mobile applications

Let's explore these 10 skills, and why you should care about learning them:

Manual exploratory testing

> Firstly, to clarify, manual exploratory testing is different from manual testing. The latter refers to verifying a given list of requirements and doesn't necessarily demand an analytical mindset. In contrast, manual exploratory testing is the skill of delving into the application details, coming up with different real-life scenarios apart from what is documented in user stories, simulating them in a test environment, and observing the application's behavior. It demands a logical and analytical mindset and is the first and foremost skill required to create a bug-free application. There are various methodologies and approaches that can be learned to structure these exploration sessions, which we shall discuss in Chapter 2.

Automated functional testing

> This is one of the core skills for shift-left testing, as discussed earlier. Doing automated testing also significantly reduces manual testing effort, especially when the application grows to include more features. In simple terms, the skill here is to write code to test feature requirements automatically, without human intervention. To do this we need tools, and therefore knowledge of different tools that can be used to write tests at different application layers is required to acquire this skill. It doesn't stop there, however; one also needs to know what antipatterns to look for in automated testing and how to stay clear of them. We'll discuss the different aspects of this skill in Chapter 3.

Continuous testing

Continuous delivery is a practice where features are delivered incrementally to end users in short cycles, instead of through a single big-bang release. By continuously delivering, the business earns profits early and can assess and retune its product strategy quickly based on end user feedback. To power continuous delivery, we have to test the application continuously so that it is always in a ready-to-be-released state. As obvious as it may seem, the wise way to do this is to automate and integrate quality checks into your CI/CD pipelines and run them frequently to ease the testing process. The skill of continuous testing involves determining which types of automated tests should be run at each stage of the delivery cycle so that the team can get faster feedback and integrate them effectively into the CI/CD pipelines. These essentials are discussed in detail in Chapter 4.

Data testing

You may have heard the sayings “Data is money” and “Data is the new oil.” These ideas highlight how important testing for data integrity is today. When users’ data is lost, or the application shows the wrong data to end users, they lose trust in the application itself. The skill of data testing requires knowledge about the different types of data storage and processing systems typically used in web and mobile applications (databases, caches, event streams, etc.) and the ability to derive appropriate test cases. Chapter 5 discusses these topics, and how data flow between the application components creates new test cases apart from the functional flows.

Visual testing

The look and feel of the application is a major contributor to the business’s brand value, and especially when it comes to big business-to-customer (B2C) products used by millions, low visual quality can impact brand value instantly. Therefore, it is essential to validate that end users have a harmonious and pleasant visual

experience by conducting visual testing of the application. Visual testing requires an understanding of how the UI components interact with each other and with the browser, for web applications. Such checks can be automated too, using tools that are different from those used for automated functional testing. We will talk about this skill, and about the stark differences between these two types of automation, in Chapter 6.

Security testing

Security breaches have become all too prevalent in today's world, and not even giants like Facebook and Twitter are excluded from such attacks. Security issues have a heavy cost for both the end users and the business in terms of loss or exposure of sensitive information, legal penalties, and brand reputation. So far, security testing has been viewed as a niche skill in the industry, with qualified penetration testers typically engaged only toward the end of the development cycle to look for security issues. But with the lack of available professional security testing talent and growing incidence of security breaches, software teams are well advised to incorporate basic security testing as part of their day-to-day work. We will discuss how to think like a hacker and seek out security issues in application functionality in Chapter 7, along with tools to automate security scans.

Performance testing

Even a slight drop in application performance can lead to huge financial and reputational losses for a business—recall the Flipkart example discussed earlier. The skill of performance testing involves measuring a set of key performance indicators at different application layers. Performance tests can also be automated and integrated with CI pipelines to get continuous feedback. We will discuss a shift-left performance testing strategy along with relevant tools in Chapter 8.

Accessibility testing

Web and mobile applications have become everyday commodities. Making them accessible to people with permanent or temporary disabilities is not only mandated by legal regulations in many countries, but also ethically the right thing to do. In order to acquire the accessibility testing skill, we must first understand the accessibility standards required by law. We can then use both manual and automated accessibility auditing tools to validate whether those standards are met. We will discuss this skill, and why incorporating accessibility features may even be a lucrative option for businesses, in Chapter 9.

Cross-functional requirements testing

We've seen that end users and businesses have a tall list of quality requirements, such as availability, scalability, maintainability, observability, and so on, apart from just needing bug-free functionality. These are called the *cross-functional requirements* (CFRs) of an application. Although functional requirements generally grab the most attention, it is the CFRs that imbue quality into the application, and failing to test these will lead to unsatisfied business or software teams, end users, or both. Therefore, CFR testing skill is a fundamental testing skill. We will discuss the testing methodologies and tools for validating different CFRs in Chapter 10.

> **NOTE**
>
> CFRs are also referred to as *non-functional requirements* (NFRs) by many in the industry. We will discuss the subtle differences between these two terms in Chapter 10.

Mobile testing

The sheer number of apps available on the leading app stores (Google Play and the Apple App Store) in 2021 may come as a surprise—a total of 5.7 million. The explosion in the number of mobile apps stems mainly from our increased usage of mobile devices. Indeed, the web analytics company Global Stats announced in 2016 that their data showed mobile and tablet internet usage across the globe had surpassed desktop usage. So, the ability to test mobile applications and the compatibility of websites across mobile devices is a critical skill today.

Although all of the previously mentioned skills are required for testing mobile applications, it requires a change in mindset too. Additionally, a whole set of mobile-specific testing tools have to be learned in order to perform various quality checks on mobile applications. Therefore, mobile testing is carved out as a separate skill here. We will traverse the nuances of the mobile landscape in Chapter 11.

Together, these 10 full stack testing skills will enable you to test the full scope of holistic quality aspects of web and mobile applications. As mentioned earlier, it's important for every role in the team to acquire some degree of competency in each of these skills. The book will show you how, skill by skill, with practical examples.

Key Takeaways

Here are the key takeaways from this chapter:

- Software quality cannot be equated to just bug-free functionality anymore. An application can be deemed suboptimal in quality if its holistic quality dimensions (security, performance, visual quality, etc.) are not on par.
- Full stack testing refers to testing all the quality dimensions of an application holistically at every layer, thereby delivering high-

quality software.

- For full stack testing to meet its goal of delivering high-quality software, teams should shift testing to the left, so that it begins in parallel with analysis and continues throughout the delivery cycle.
- Shift-left testing embodies the aphorism “Quality is the team’s responsibility,” as it demands that every role in the team take ownership of performing certain quality checks at different phases of delivery. This requires all team members to upskill themselves, acquiring relevant testing skills at varied competency levels.
- The two classic monolithic categories of testing skills, manual and automated, mask a vast set of new testing skills required to perform full stack testing efficiently. This chapter introduced 10 different testing skills that are essential for delivering high-quality web and mobile applications today, which we will explore over the course of the following chapters.

Chapter 2. Manual Exploratory Testing

Not all those who wander are lost.

—J.R.R. Tolkien

Manual exploratory testing is an intense activity where you exercise the test application with the objective to explore and understand its behaviors in various situations that are not articulated explicitly anywhere—be it in the requirements document or user stories. As a result of the exploration, often new user flows that were not envisaged during the analysis or development phase and bugs in the existing user flows will be discovered. When such discoveries happen, it is refreshingly joyous for the individual who found them, as it showcases their complex analytical and keen observation skills!

Typically, manual exploratory testing is carried out in a testing environment, where the entire application is deployed. The testers take the liberty to meddle with the various application components, such as the database, services, or background processes, as they please, in order to simulate different real-time scenarios and observe the application's behavior. This exploratory style of testing differs from traditional manual testing, which refers to the task of manually executing a particular set of actions described as acceptance criteria in user stories or in the requirements document and verifying whether the stated expectations are met successfully. In other words, manual testing doesn't necessarily exercise any analytical skills, whereas exploratory testing lays a green field in front of the testers, inviting them to go above and beyond what is documented, and even beyond what is *known* so far about the application!

Given the overlap between manual testing and exploratory testing, some teams, even today, underestimate the value of exploratory testing. Also there is often a perception that the amount of analysis carried out as part of user story elaboration and development is enough to go live, especially when supplemented with automated testing (Chapter 3 discusses automated testing in detail). However, this belief overlooks the fact that the analysis carried out during user story creation is usually primarily from the business's point of view, and during development, the developers may focus on the current scope of functionality and confine their thinking to that small piece. This leaves an obvious gap, where the application is not explored from an end user's perspective and with a big-picture lens in a deployed environment. That gap may leave space for integration issues and missed end user flows—which is why teams need a manual exploratory testing phase post-development.

> **NOTE**
>
> Exploratory testing brings all the three angles—the business's requirements, technical implementation details, and the end user's needs—together, and challenges everything that is thought of as *true* from all these angles. A good practice is to call a functionality complete only after the new user flows and test cases discovered through exploratory testing are automated as well.

It may not be necessary to assign a separate individual to conduct this post-development exploratory testing, although that approach might yield better results due to accumulation of application knowledge and because the task demands someone with sharp observation and analysis skills. If cost or availability concerns prevent this, the existing team members should take on the responsibility of performing exploratory testing in a round-robin fashion during each iteration. Indeed, developing exploratory testing skills may help every role to perform better.

If you are one such team member looking to develop your exploratory testing skills, this chapter is for you. We shall discuss the existing frameworks in the industry that can assist with exploratory testing, and a strategy to approach this task. The exercises in the chapter focus on performing exploratory testing of web UIs and APIs, specifically. We will also examine a set of useful practices to maintain test environment hygiene, as a fully deployed test environment plays a pivotal role in the success of manual exploratory testing.

COMMONLY USED TERMS

The following are some commonly used terms that you will see in this chapter:

- A *feature* or *functionality* is how the application provides value to its end users. For example, login is a feature that provides security to end users.
- A *user flow* is a set of actions the end user performs in the application to achieve the value provided by the functionality. For example, in order to log in, the end user has to enter their credentials and sign in; this is the login user flow.
- A *test case* is a set of actions that validate that the functionality is working as expected. For example, entering a valid username and password and verifying that the login is successful is a test case. Similarly, entering an invalid username and verifying that there is an error message is also a test case. The former is a positive test case, as it allows the end user to achieve the value provided by the functionality successfully, whereas the latter is a negative test case, as it doesn't allow the value to be achieved. To explore a functionality completely, both the positive and negative test cases have to be simulated and observed.
- An *edge case* is a negative test case that occurs very rarely.

Building Blocks

Let's begin by taking a look at eight exploratory testing frameworks, with practical examples of their use. Later we will practice exploring a functionality.

Exploratory Testing Frameworks

The goal of exploratory testing frameworks is to help us form mental models that can be intuitively applied to relevant contexts in the application. They aim to narrow the scope of testing by giving clarity and structure to a piece of functionality. For example, numeric input fields are common in applications. Instead of randomly testing all possible numeric values to test such a field, frameworks lend us structures to logically compartmentalize the inputs into sample sets. Similarly, there are frameworks that try to structure business rules and thereby help us see the different user flows and test cases. Let's dive into them one by one, with examples.

As our first example, let's take a web page that asks for the user's income as input, as seen in Figure 2-1, and displays the amount of tax owed by the user as output. Figure 2-1 also shows the different tax brackets used for computation on the right side.

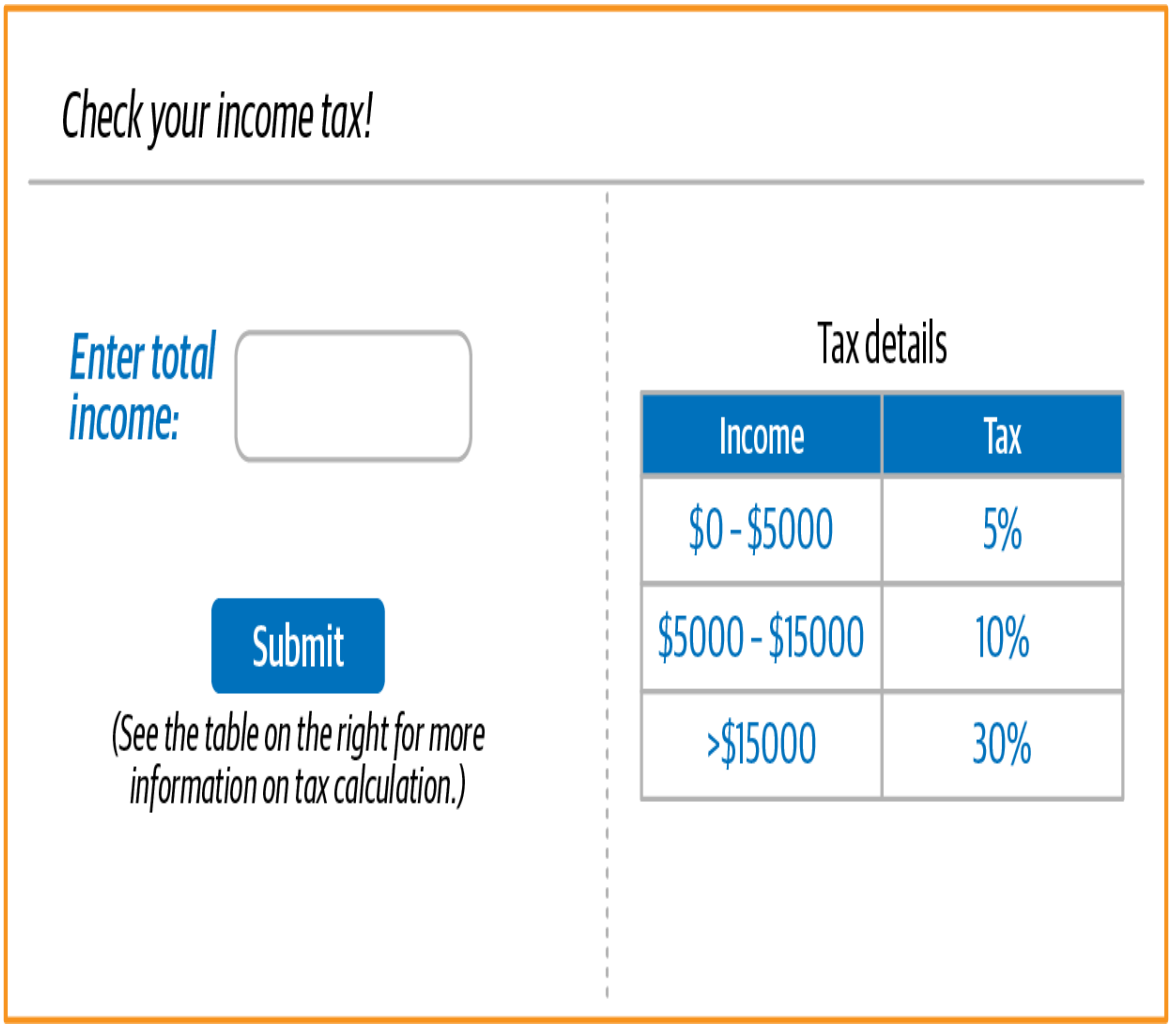


Figure 2-1. A simple tax calculator example

To test whether the tax calculation logic works as expected, we need to identify positive and negative test cases to try as inputs. A point to note here is that income is a continuous numeric value ranging from 0 to infinity. To arrive at our positive and negative test cases, we need to logically narrow down the right set of numeric input values and verify the output. There are two frameworks that can help us do this: equivalence class partitioning and boundary value analysis.

Equivalence class partitioning

The equivalence class partitioning framework suggests that we split the inputs that result in the same output or undergo similar processing into partition classes, and that it is sufficient to pick just one sample input from each partition to test the functionality entirely.

Applying this suggestion to the tax calculator example, the first set of partition classes will be the tax brackets themselves: [0 – 5000], [5001 – 15000], and [>15000]. These three classes can be considered *equivalence classes*, as every input within each class

will be subjected to the same rules, and to validate the positive test cases it's enough to test with three input data points, one from each class. For instance, unless you're bored and want to try more, testing with the inputs 2,000, 10,000, and 20,000 will be sufficient to validate the positive test cases. Next, the same framework can be applied to derive the negative test cases. The classes of inputs that should result in an error are [*negative values*], [*letters*], [*symbols*], and so on. Again, one value from each class is enough to test the negative test cases.

In addition to manual exploratory testing, this framework is helpful in unit testing (discussed in Chapter 3). It can also be applied to any other relevant context in the application, such as testing time-based outcomes (before and after an event), internal states of the system, and so on.

Boundary value analysis

Boundary value analysis extends the equivalence class partitioning method by explicitly checking the boundary conditions in each of the classes. This is helpful in finding errors, as the boundary conditions are often vaguely defined and improperly implemented. For instance, in our simple tax calculator example, the requirements for the tax brackets may have been stated as "5% tax for income below $5,000, 10% for income between $5,000 and $15,000, and 30% tax for income over $15,000." However, this fails to clearly define the boundary conditions; i.e., which classes the values 5000 and 15000 should fall into. The boundary value analysis framework draws attention to such issues and helps resolve them by testing the boundary values in each of the equivalence classes, in addition to picking an input within the range of the class.

Let's apply this framework to the tax calculator example by analyzing the boundary values in each of the equivalence classes we found earlier. The first class, [0 – 5000] has 0 and 5,000 as boundary values. But logically, when the income is 0, there shouldn't be any taxes. So, this forms new equivalence classes: [0] and [1 – 5000]. The boundary values that we should test to cover all the positive test cases are therefore [0, 1, 5000, 5001, 15000, 15001], as seen in Figure 2-2.

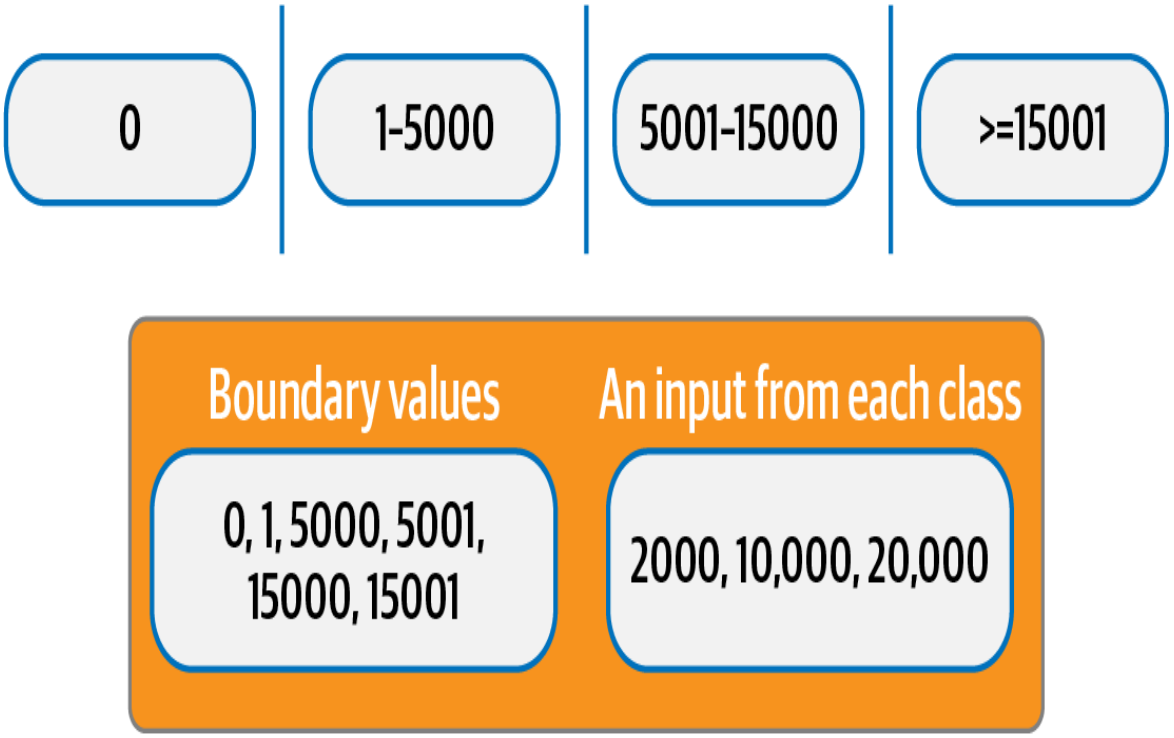


Figure 2-2. Equivalence classes with boundary conditions

As this example illustrates, although this is a testing framework, it yields maximum benefits for the team when applied in all the delivery phases, starting from analysis.

Having discussed two frameworks to help structure the input values of a single field and translate them into a minimal set of positive and negative test cases, it's time to move on to frameworks that deal with slightly more complex scenarios where multiple input combinations result in different outputs. To help us with this discussion, let's take the classic example of a login page that takes two inputs, an email address and password, and discuss how the state transition, decision table, and cause-effect frameworks help in visualizing the different test cases.

State transition

The state transition framework is helpful in deriving test cases in situations where the application's behavior changes based on the history of inputs. For example, our login page might show an error message the first and second time the user enters an incorrect password, but the account might get locked the third time. In such scenarios we can draw a transition tree, as seen in Figure 2-3, to derive test cases. In the transition tree, you can observe that each state of the application is depicted as a node. The possible outcomes of an action are shown as subnodes, with the actions/events triggering the outcomes noted as the branch labels.

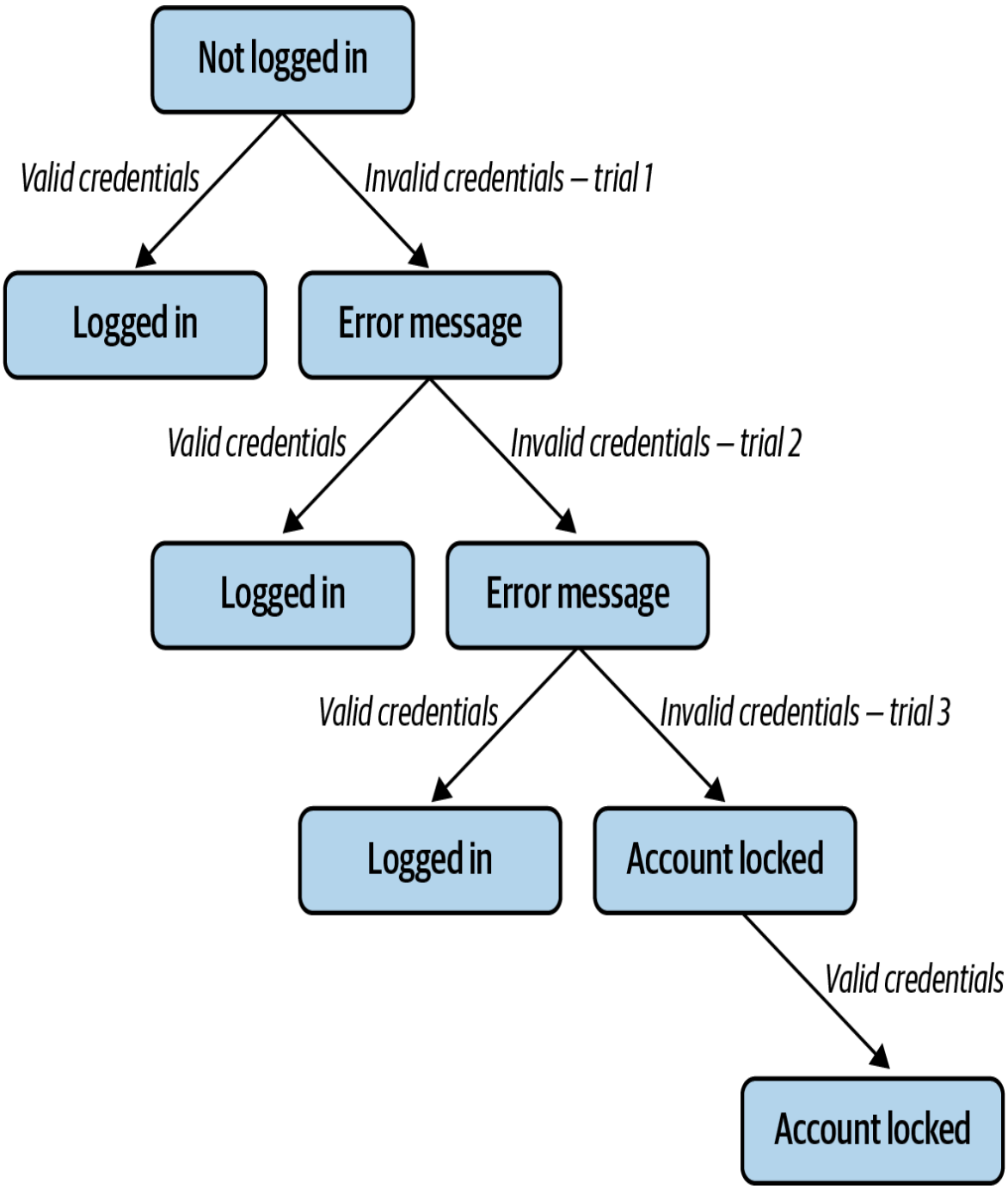


Figure 2-3. State transition tree for invalid login scenario

This tree gives a clear picture of each test case with its starting state, the action that changes the application's state, and the expected outcomes to be validated. The visualization also gives us a realistic estimate of the amount of effort required to test a feature by clarifying the number of states and transitions, which helps in the planning phase.

State transitions can be much more complicated than this, such as in an order management system where orders go through states like payment complete, pending, shipped, canceled, fulfilled, and so on. In such cases, visualizing each state as a node and the actions that take the order to each possible next state will give a clear overview of the feature itself.

Decision table

When inputs are logically bound (`AND`, `OR`, etc.) to produce outcomes, decision tables can be used for deriving test cases. This can save a lot of time during testing, as you have all the possible input combinations and expected outputs clearly marked in the table ahead of time. In the login example, the email and password are logically bound by the `AND` operator; that is, both the email and password have to be right for a successful login. Table 2-1 shows the decision table we can create for this scenario.

Table 2-1. Decision table for login scenario

Decision table	Test case 1	Test case 2	Test case 3	Test case 4	
Conditions	Email	True	False	False	True
	Password	False	True	False	True
Actions	Login	-	-	-	True
	Error message	True	True	True	-

The method can also save time by allowing us to eliminate certain unneeded test cases. For example, in the login scenario, Test case 3 where both inputs are incorrect can be eliminated as the login fails if even one of the inputs is wrong.

Cause-effect graphing

Cause-effect graphing is another way of visualizing logically bound inputs and their possible outcomes. The framework helps to view the big picture of a feature and hence is particularly useful in the analysis phase. Once you've created the graph, you can translate it into a decision table to derive detailed test cases. Figure 2-4 shows the cause-effect diagram for the same login example.

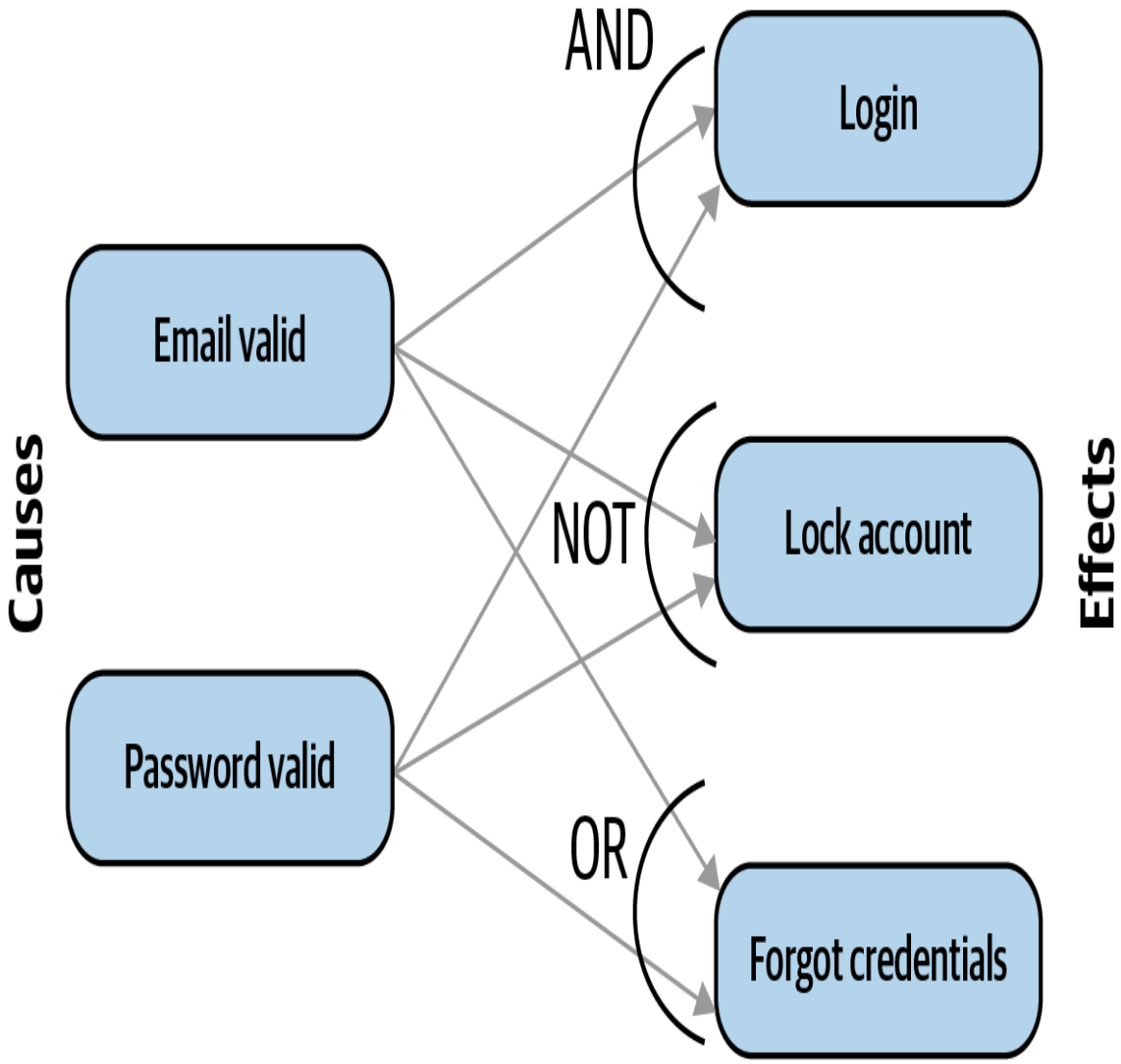


Figure 2-4. Cause-effect graphing for login scenario

Causes are listed on one side and effects on the other, and the navigation paths between them are laid out with the help of logical operators.

The frameworks we've seen so far are useful for structuring inputs that are related to each other. Next, we'll look at two frameworks that can help us deal with multiple independent variables and large datasets.

Pairwise testing

We often have to deal with more than one input value in applications, and it can be a struggle to manage their variations and derive test cases. Pairwise testing, also known as all-pairs testing, is a framework that assists in condensing the test cases to a minimum when multiple such independent variables/inputs drive the outcomes. Let's work through a small exercise to illustrate how it works.

Consider a form that takes three independent inputs: operating system (OS) type, device manufacturer, and resolution. The OS field can take two values: Android or Windows. The device field can take three values: Samsung, Google, or Oppo. Finally, the resolution field can take Small, Medium, and Large as values. So, when we are testing this form, we have 2 * 3 * 3 = 18 input combinations, as seen in Table 2-2.

Table 2-2. Example test cases without applying t

h
e

p
a
i
r
w
i
s
e

t
e
s
ti
n
g

m
e
t
h
o
d

Test cases	Device	Resolution	OS
1	Samsung	Small	Android
2	Samsung	Medium	Android
3	Samsung	Large	Android
4	Google	Small	Android
5	Google	Medium	Android
6	Google	Large	Android
7	Oppo	Small	Android
8	Oppo	Medium	Android
9	Oppo	Large	Android
10	Samsung	Small	Windows
11	Samsung	Medium	Windows
12	Samsung	Large	Windows
13	Google	Small	Windows
14	Google	Medium	Windows

15	Google	Large	Windows
16	Oppo	Small	Windows
17	Oppo	Medium	Windows
18	Oppo	Large	Windows

Pairwise testing suggests that testing any given pair of inputs once is enough, as they are independent variables. This will reduce our list of test cases to just nine, as seen in Table 2-3.

Table 2-3. Reduced test cases when we apply the

pairwise testing method

Test cases	Device	Resolution	OS
1	Oppo	Small	Android
2	Samsung	Small	Windows
3	Google	Small	Android
4	Oppo	Medium	Windows
5	Samsung	Medium	Android
6	Google	Medium	Windows
7	Oppo	Large	Android
8	Samsung	Large	Windows
9	Google	Large	Android/Windows

The new condensed table has cut down the repetition of several pairs. For example, the [Google, Medium] and [Google, Windows] pairs each occur only once now.

Sampling

So far, we have dealt with inputs that are small and consumable by the human brain without the help of tools. But what if we have to test large datasets? For example, let's say a legacy insurance system has been migrated to a new system, and we have to test whether the existing insurance details have been transferred correctly into the new system. There could be millions of users in the legacy system, and we cannot apply the frameworks discussed so far to derive test cases. For example, we can't determine equivalence classes as each user will have their own variations in terms of age, premiums, length of contract, scheme type, etc., and we can't apply pairwise testing as there are too many variables to identify and eliminate the recurring pairs. In such cases, sampling is a useful technique.

Sampling, in general, can be applied to any input that is continuous and large in nature. It involves selecting a subset of the values to use for testing, as seen in Figure 2-5, usually using one of the following techniques: random sampling or criteria-specific sampling.

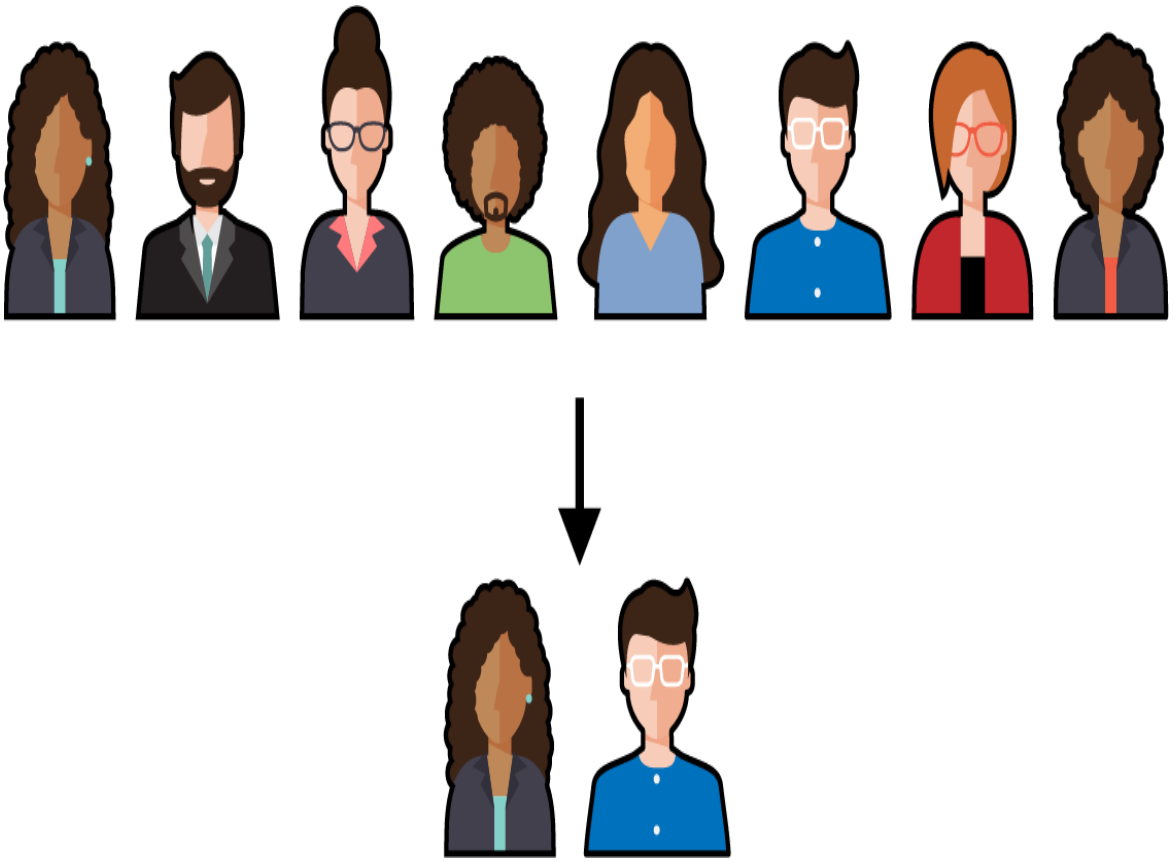


Figure 2-5. Sampling based on criteria or random sampling

Random sampling is where we pick any data sample from the dataset and verify the results. For example, if there are 1,000 users, we can choose 50–100 users randomly from the legacy system and compare their data in that system against the data stored in the new system. Criteria-specific sampling is where we pick the samples by identifying some common characteristics in the dataset. For instance, in the insurance

system, we could sample based on user-specific criteria like age, length of contract (number of years subscribed), mode of payment, profession, etc., and insurance policy–specific criteria like payment intervals, price, etc. We could further refine the technique by making the sample count for each criterion proportional to the actual distribution of the values in the dataset. This would form a representational mini-dataset and likely cover all kinds of test cases.

That brings us to the last framework, which is more about exercising our analytical and logical thinking skills than a set of fixed guidelines. Let's get to that now.

Error guessing method

Error guessing involves predicting possible failures based on past experience. These might include common problems with integration, input validation, boundary cases, and more. Although past experience plays a critical role in predicting probable error cases, you can also use your understanding of the technology and logical reasoning. Indeed, promoting this kind of thinking boosts your exploratory testing skills across the board.

Here are a few types of errors that crop up regularly, in my experience:

- Missing validations for invalid/blank input values and lack of appropriate error messages directing the user to correct the input
- Unclear HTTP status codes returned for data validation, technical, and business errors (we'll take a look at some of these in "API Testing")
- Unhandled boundary conditions specific to the domain, data types, states, etc.
- Technical errors such as the server being down, responses timing out, etc. unhandled on the UI side
- UI issues (such as jerks and residues) during transitions, data refreshes, and navigation
- The SQL keywords *like* and *equals* used interchangeably, changing the results entirely
- Uncleared caches and undefined session timeouts
- Reposting a request when the user clicks the back button in the browser
- Missing file format validation when uploading files from different OS platforms

You can use this pack of eight exploratory testing frameworks to structure your thought process around exploring a functionality and deriving meaningful test cases. Note that these frameworks can be applied to any relevant context in the application, and not necessarily only to input data. Now that you're equipped with these tools, before we get to some practical exercises let's take a closer look at what's involved in exploring a functionality.

Exploring a Functionality

Suppose you are asked to perform exploratory testing on the order creation functionality of an ecommerce application. What discovery paths should you start with? This section answers that question by throwing light on four essential paths that must be explored in any given application, as illustrated in Figure 2-6.

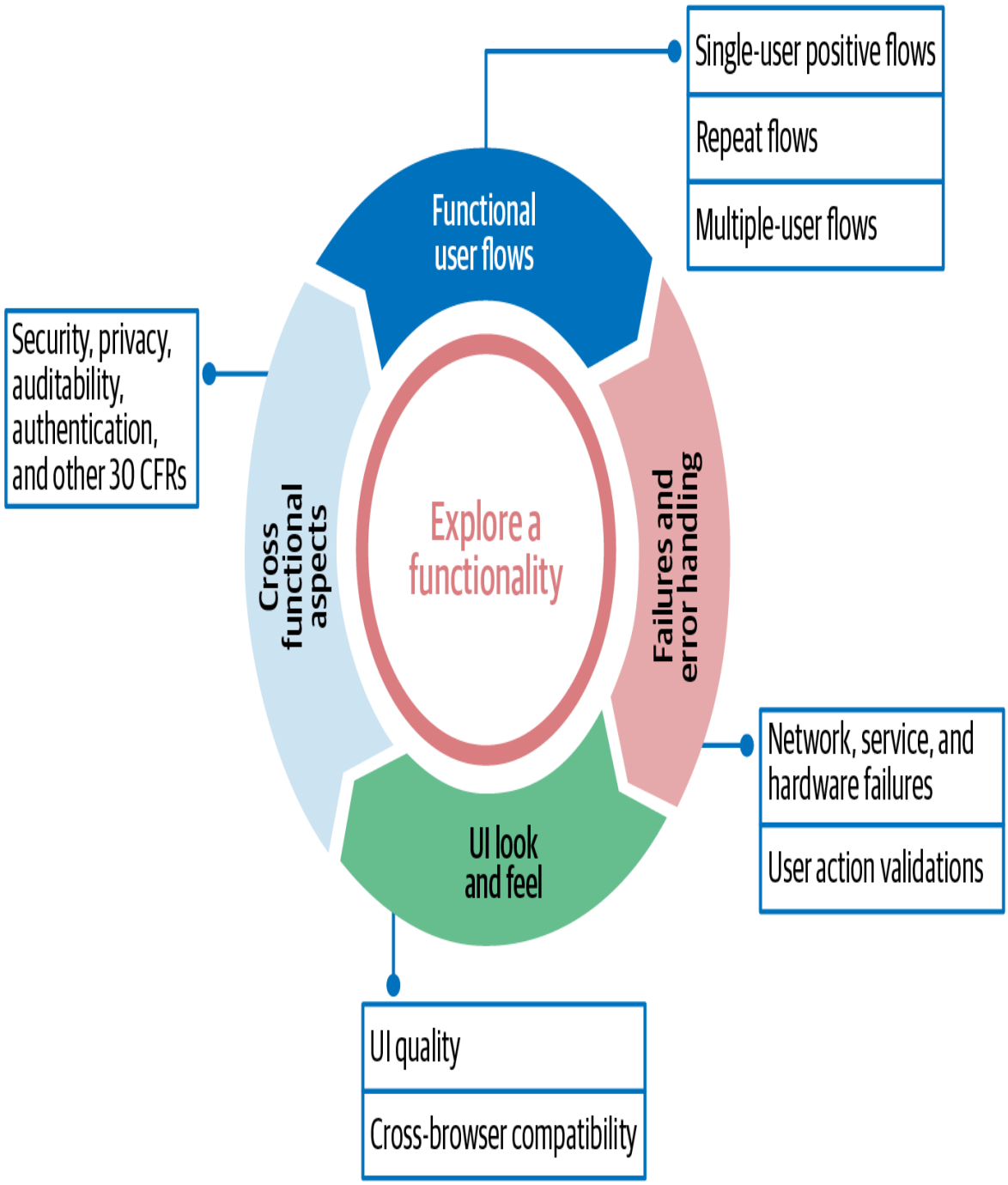


Figure 2-6. Four essential discovery paths while exploring a functionality

Functional user flows

The functional user flows of an application refer to the journeys an end user takes while using the application, such as logging in, searching for a product, adding it to the shopping cart, providing a shipping address, choosing a delivery option, paying for the order, and finally getting an order confirmation. This is a single-user positive flow, which is what you should validate first. You will have to explore the positive flow with different shipping addresses, payment and delivery methods, and item combinations to ensure it works entirely.

While exploring, you may find yourself using some of the exploratory testing frameworks discussed earlier. For example, you may use the equivalence class partitioning and boundary value analysis methods to verify whether the application adds the right amount of tax to the total price. Similarly, you can use a transition tree to derive test cases around the shipping address and delivery methods available to that address combination. Once you ensure that this single-user positive flow works perfectly, you should start exploring the two other kinds of flows:

Repeat flows

> End users often repeat the same flow (or parts of it) multiple times in the application, like searching for different products and adding them to the shopping cart. But commonly, a user flow is tested once, and if it works the repeat flows are ignored under the assumption that the behavior will remain the same. Practically, however, this assumption may or may not be true. For example, if the user tries to add the same item to their shopping cart again, the UI may show a message saying the product has already been added and checking if they want to increase the quantity. Repeat flows must be tested as well.

Multiple-user flows

> A functionality may work perfectly from a single user's point of view, but that may not be the case if several users interact with the application simultaneously in real time. Exploring possible collision scenarios, where one user's actions impact another, is therefore important. For instance, what happens when two different users add the last available product to their shopping carts at the same instant?

In short, functional user flows are often chosen as the first discovery path to explore in an application, but within that path, there could be several sub-branches to explore. A few essential sub-branches are single-user positive flows, repeat flows, and multiple-user flows.

Failures and error handling

As called out at the beginning of the chapter, exploratory testing is carried out in a test environment and involves meddling with the application components to simulate real-time scenarios and observe the application behavior. There are two phrases in that

statement that require your attention, as they form the core of exploratory testing: *meddling with the application* and *real-time scenarios*. When considering real-time scenarios, you should also think of all possible failures, as failures are practically unavoidable. For instance, there could be a network failure between the application components, preventing it from sending a response to the user, or the network could be slow between the end user and the application server, resulting in delays, or the application services could be down due to a hardware failure. All these failures and more must be anticipated during exploratory testing and simulated in the test environment.

In addition to the aforementioned network, service, and hardware failures, there could also be errors due to invalid user actions. A functionality can be deemed complete only if it has built-in validations to handle such cases. In the order creation functionality, several instances call for exploring these validations. For example, as discussed earlier, the login page needs to have validations on the email and password, the search text entered for searching for a product needs to be validated for invalid inputs, availability of the item, and so on. The other places are the shipping address and payment details, adding items to the shopping cart, etc.

Exploratory testing should place a large emphasis on identifying possible failures and handling errors. As part of error handling, the functionality should advise the users of their mistakes and suggest possible remediations through meaningful error text.

The UI look and feel

The UI is what the end user sees, and there can't be obvious problems with its quality. Its look and feel is therefore another important discovery path to explore. To give just a few examples, UI quality–related test cases in the order creation functionality might include ensuring that an appropriate amount of space is provided for shipping addresses (not so little that there isn't enough room to display a long street name, or so much that there's a huge amount of blank space when part of the address is short) and that product images are displayed with the appropriate quality. End users should be able to seamlessly operate the application from their preferred browsers, and there should be a loading icon when there are delays. A structured approach to UI quality testing is discussed in detail in Chapter 6.

Cross-functional aspects

There may be several cross-functional aspects in any given functionality, such as security, performance, accessibility, authentication, authorization, auditability, privacy, and so on, that require specific focus during exploratory testing. Many of these aspects have an entire chapter of this book dedicated to them because of their importance. Briefly, here are a few cross-functional requirements that we would want to examine from the point of view of exploring the order creation functionality:

Security

In the order creation user flow, an abusive user could enter SQL queries in the UI input fields and try to hack the application. The application should have validations in place to handle such attempts. Similarly, the users' credit card details should not be stored in plain text in the application database and should not be logged in plain text in the application logs, so that they are secure even in the event of a potential breach. All of these security testing aspects and more are discussed in detail in Chapter 7.

Privacy

Users' private data, such as credit card details and shipping addresses, should not be stored in the application database without their consent. Also, users should be informed of the ways in which their data might be used for analytics or whether it will be sent to third-party services for processing in advance. Several data privacy clauses are also enforced by legal regulations; we'll discuss these issues in Chapter 10.

Authentication/authorization

Most websites have user authentication functionality, which calls for exploring authentication-related test cases such as single sign-on, two-factor authentication, session expiry, account locking, unlocking, etc. In the ecommerce application, end users may be allowed to view the product catalog without logging in, but not to place an order.

Similarly, there could be roles (e.g., admin, customer executive) and permissions (e.g., editing an order) assigned to different users, which requires exploring authorization-related test cases such as multiple overriding roles, new permissions being added to existing roles, observing the application behavior when an operation is executed without the right permissions, etc.

Again, these are just a few examples; around 30 different cross-functional aspects and ways to test them are discussed in Chapter 10.

The four discovery paths described here should lead to well-rounded testing coverage on any given functionality. Note that while exploring each of these paths, you may think of new ideas and test cases that don't belong to that path. It's important to jot these down so that you can come back to them later, or use them to kick-start your exploration of another path.

Manual Exploratory Testing Strategy

The manual exploratory testing strategy depicted in Figure 2-7 ties together all that we've discussed so far, and additionally bundles in the team processes. This will give

you a practical outline for performing exploratory testing in your day-to-day project work. Let's begin with the outer semicircle and move inward.

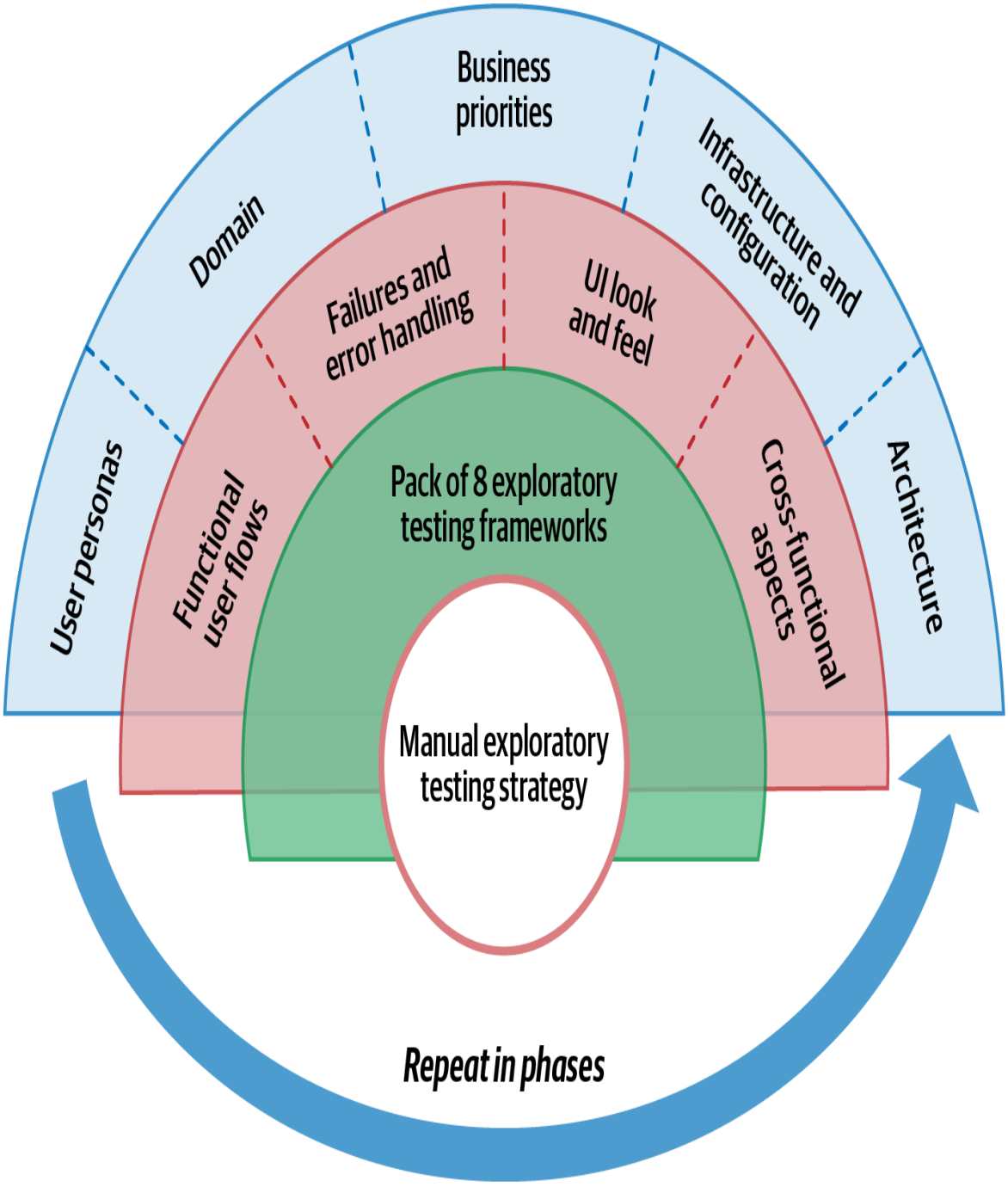


Figure 2-7. The manual exploratory testing strategy

Understand the Application

The outer semicircle emphasizes understanding the application details and points to five broad application areas that you should focus on. Gathering details about these will

help you get started with exploratory testing, but as mentioned earlier, you will certainly find new information about the application during the exploration.

> **NOTE**
>
> At times, exploratory testing is confused with *monkey testing*, an approach where the application is tested with random inputs and with zilch knowledge about the functionality. It is important to note that in exploratory testing you should have a precise understanding of the functionality being tested, and test with the mindset of exploring the unknowns.

Here is a brief on the five broad areas that you may focus on while trying to establish an understanding of the application:

User personas

A persona is a character that represents a set of end users with similar attributes. In software teams, such user personas are created at the beginning of the project so that their specific needs can be imbued into all the stages of the delivery lifecycle, starting from design. An example of user personas impacting the features of an application is in a social networking site, where young adults may expect an extravagant experience while seniors may expect a clean and clear interaction. Testing is all about wearing the end user's hat, so knowing the set of user personas the application intends to serve and exploring how each persona will perceive and interact with the application is vital.

Domain

Every domain—social networking, transport, health, etc.—has a tailored workflow, process, and set of terminology or jargon that needs to be understood to kick-start exploration. Ecommerce is a perfect example where domain knowledge becomes critical in testing. For instance, an order, once created, goes through a defined workflow: capture, promise, confirm, and so on. The order fulfillment flow has to interact with numerous parties, such as the warehouse that stores the items, the shipping partner that transports the items from the warehouse to the customer, and the vendors that replenish the items regularly. So, while observing the application's behavior during exploratory testing, you need to know how to go about exploring all the application's paths. You may find it hard to do that without basic domain knowledge.

Business priorities

Consider the scenario where the business priority is to design the solution as a platform for extensibility and scalability purposes. In such cases, just testing the functional user flow from the UI may not be sufficient. It needs to be explored from a "platform" point of view, observing whether the UI and web services are tightly

coupled or if the web services are independent and can be integrated with other systems, and other similar angles.

Infrastructure and configuration

As discussed earlier, exploratory testing involves meddling with the test environment to simulate real-time scenarios, including failure cases. Having information about which application components are deployed where and the configurable levers will provide critical hints for finding new discovery paths. For example, web services may be configured with the maximum number of hits they can serve within a time period, known as *rate limiting*; you may need to observe the application behavior when the rate limit is exceeded. Gathering some basic information about the infrastructure and configuration, such as how the services and database are deployed (on a single machine or spread across multiple machines), rate limiting settings, API gateway settings, and the like will help you uncover important test cases.

Application architecture

Knowledge of the application architecture will add branches to your discovery paths in an exploratory testing session. For example, if the architecture involves web services, you may need to perform exploratory testing of the API (discussed in "API Testing") instead of just exploring the UI. Similarly, if the application involves event streams (discussed in Chapter 5), exploring the cases around asynchronous communication becomes important. Understanding the architecture at a high level will help you carve out the discovery pathways in terms of internal component integrations, data flow between components, third-party integrations, and error handling. Several of these aspects are discussed throughout the book as well.

Once you have gathered sufficient information about these five application areas, you are ready to dive into the actual exploratory testing phase.

If this all sounds a little overwhelming—particularly the parts about architecture and infrastructure—don't worry too much about these details now. It's perfectly fine to approach exploratory testing from a functional point of view and gradually learn to ask more questions along these lines.

Explore in Parts

The next semicircle in the manual exploratory testing strategy diagram in Figure 2-7 points to exploring in parts.

In his 2003 paper "Exploratory Testing Explained", James Bach defines the practice as "simultaneous learning, test design, and test execution." To this day, this is still one of the most widely used definitions of exploratory testing. To elaborate, exploratory testing is performing a series of actions in the application while observing the behavior, and

thereby learning more about the application and exploring it incrementally. Such a process demands that our brains be alert all the time and that we do *in-depth analysis.* As humans, we are best at paying intense attention and really getting into the depths of something when we focus on smaller scopes of work. This is why we should explore individual parts of the application at a time! Those parts could be any of the previously discussed discovery paths, or a sub-branch in the path, such as a user flow, a feature, or a cross-functional aspect like security.

Keeping track of all these paths and sub-branches while exploring in depth can be difficult. One strategy for dealing with this is to use a mind map,[1] like the one in Figure 2-8. This can be shared with the whole team.

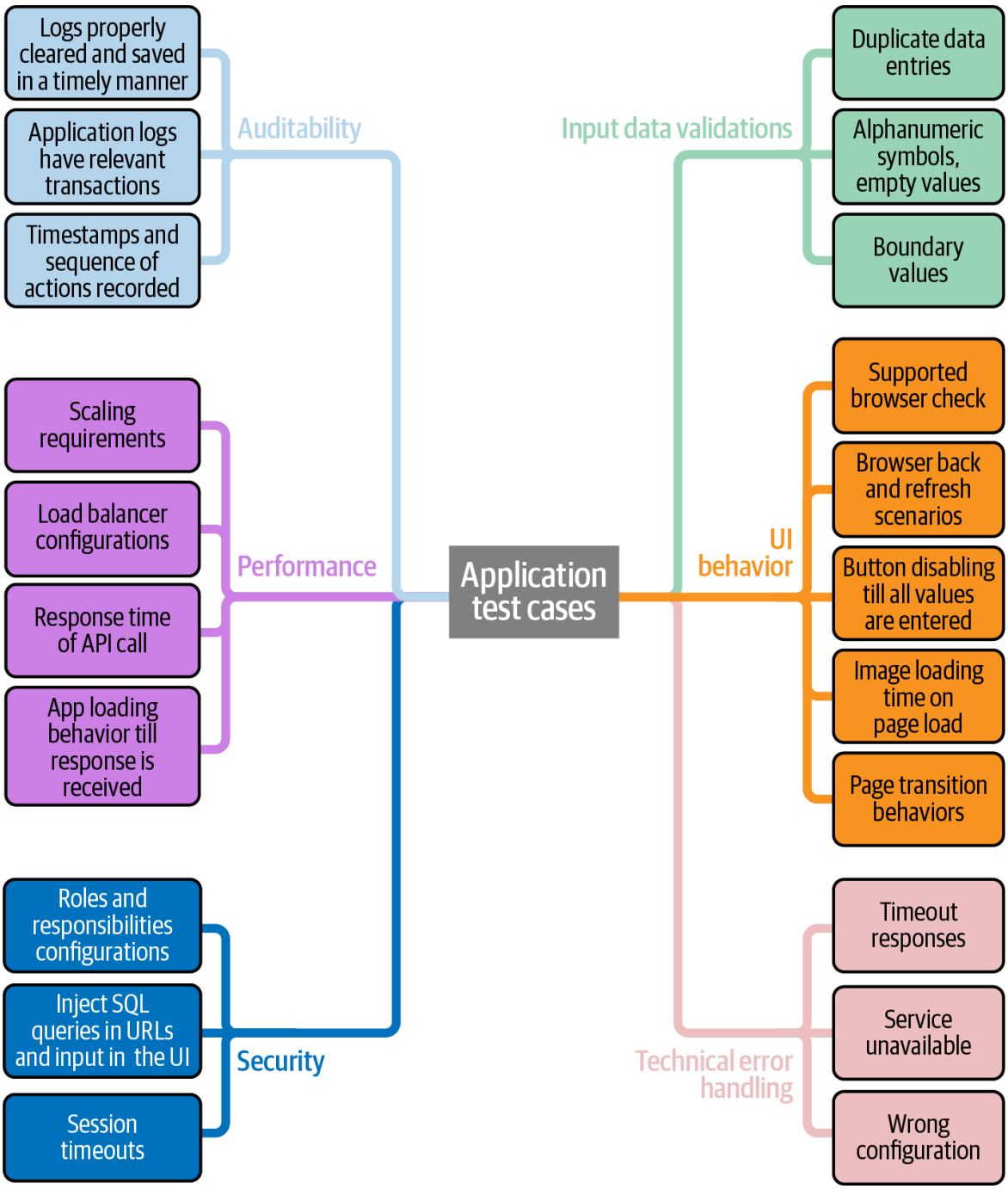


Figure 2-8. An exploratory testing mind map

During this phase, you may also need the pack of eight testing frameworks, represented as the inner semicircle in Figure 2-7.

Repeat Exploratory Testing in Phases

Exploratory testing cannot be a one-time activity. The team will continuously be adding new code, new features, and new integrations, resulting in changes to the application

behavior and thereby calling for exploration again. Considering exploratory testing as a continuous process will allow you to structure the scope that should be explored in depth at a particular time or phase. For example, some Agile teams practice *dev-box testing*, where business representatives and testers perform time-bounded exploratory testing of the user story that was just developed on the developer's own machine. Here you can restrict the scope to only the positive user flows, validations, and UI look and feel. The next phase conducive for exploration is the user story testing phase post-development. Here, you can expand the scope of exploration to include some of the cross-browser and cross-functional aspects. In addition, some Agile teams conduct regular *bug bashes*, where all the team members get together and explore the application features developed so far. And finally, in the release testing phase, you can focus on cross-functional aspects such as performance, reliability, and scalability in depth, and explore the positive user flows and integrations at a slightly higher level. Planning your exploratory testing phases ahead of time will help the team get continuous feedback, and therefore allow space for continuous improvement.

> **TIP**
>
> Exploratory testing is organic in nature. Hence, you may discover new pathways that you didn't plan for and that may consume your allotted time in an iteration. This is to be expected. A tip here is to consider whether a pathway can be included in the next user story testing phase or in bug bashes, and if so note that down and move on.

To recap the strategy, when you are starting exploratory testing, first get to know the application details, then jot down the individual paths to explore. Then, continue your exploration of the different pathways throughout the phases of the delivery cycle in order to provide continuous feedback to the team.

Exercises

We have discussed a lot of theory about frameworks and strategies so far. To apply these to explore the discovery paths of an application, you may need to learn some relevant tools, such as SQL for exploring the database (discussed in Chapter 5), Postman for exploring the APIs, and so on. This book covers several such tools throughout. In this section, we will discuss API and web UI exploratory testing tools.

API Testing

An application programming interface, or API, provides a way for systems to interact with each other. APIs essentially abstract away the underlying complexities of a system and simplify the exchange of information over the network as XML, JSON, or plain text using the HTTP protocol. To standardize information exchange, protocols like SOAP and specifications like REST were invented. These days RESTful APIs are more

prevalent than SOAP, and even the legacy systems that use SOAP are being rewritten with REST specifications. To help you understand REST APIs, let's consider a basic ecommerce application like the one in Figure 2-9, with three REST services (the order, authentication, and customer services), a UI, and a database.

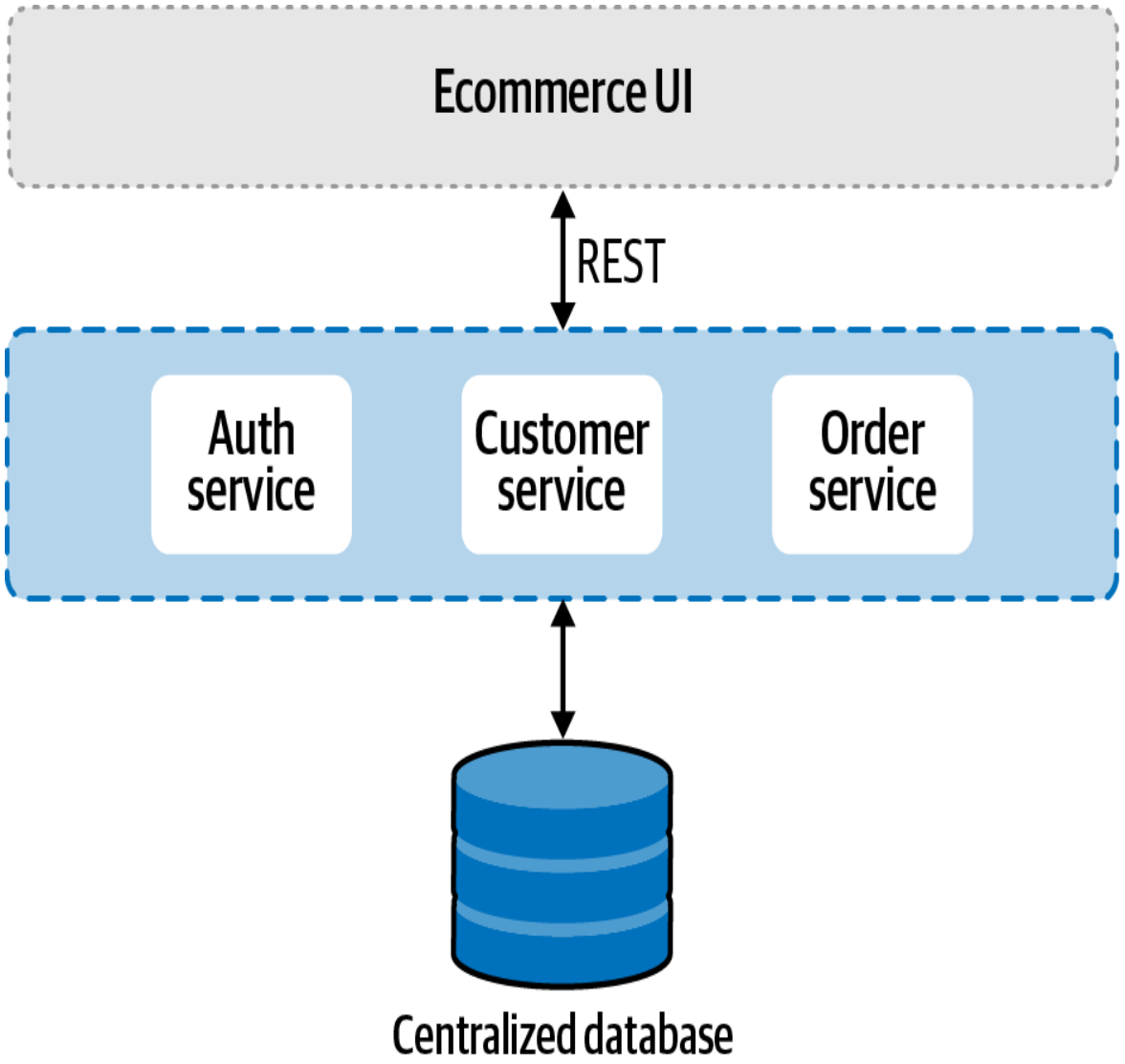


Figure 2-9. A sample ecommerce application with a services-based architecture

A web service is a component that serves an independent purpose within an application. For example, the order service in our sample ecommerce application might have the responsibility of managing (creating, updating, and deleting) orders, while the customer service is responsible for maintaining the customers' details. This eases information exchange, as the other components in the application, such as the UI or other services, can access the relevant service's API to get the information they need.

> **NOTE**
>
> A *service-oriented architecture* is one where the core application functionalities are written as web services, as in Figure 2-9.

To give an example, suppose an end user completes paying for an order via the ecommerce UI. As shown in Example 2-1, the UI will immediately send an order creation request to the order service with the order details, and the order service will process the request and send a response back to the UI. The ecommerce UI is called a *client* in this context.

Example 2-1. Sample REST request and response

```
// Request

POST method: http://eCommerce.com/orders/new
{
  "name":"V-Neck Tshirt",
  "sku":"ABCD1234",
  "color":"Red",
  "size":"M"
}

// Response

Status Code: 200 OK
Response Body:
{
   "Msg": "successfully created",
   "ID": "Order1234227891"
}
```

If you look at the request in Example 2-1, you'll see that it hits the API `/orders/new` using the POST HTTP method. Typically, POST is used for creating or adding new information, and GET is used for retrieving information—for instance, getting a list of the orders made by a customer. There are PUT and DELETE methods too, which are used for update and delete operations, respectively. The request body packages up the order details—in this case, the item's name, stock-keeping unit (SKU), color, and size—as a JSON object. This entire structure is referred to as the *contract*. If the client doesn't stick to this contract, the service will not process the request.

Similarly, the response also sticks to a contract: it will include a status code indicating the success or failure of the operation and may also have a response body that gives more information regarding the operation. In Example 2-1, the response status code is *200 OK*, which indicates success, and the response body has a message saying "successfully created" along with the order ID generated by the order service. On receiving this response, the ecommerce UI will take the user to the order confirmation page and display the order ID on that page. Note that all these actions will happen

synchronously—in other words, the ecommerce UI will wait until it has received a response before moving on to the order confirmation page.

Now, given this basic understanding of how APIs work, you may have a question: why do we need to explore the APIs separately when we can test the order creation functionality from the web UI? The concise answer is that today, APIs have become products themselves! A slightly more elaborate answer is that APIs encompass all the business logic and validations, making them standalone products that other internal and external components can reuse. For example, our hypothetical ecommerce business could build a new mobile shopping app and reuse the same order creation API, or build a customer support portal with the same customer service APIs. They could even branch out into an entirely new domain, and reuse the authentication service's APIs to build the login functionality in that new product. So, exploring the APIs as standalone products is very important in today's digital world.

Here are some of the discovery paths, apart from the core business logic, to pay attention to when you are exploring APIs:

Validation of the request contract

Validation should be performed so that, for example, if a fraudulent client creates a new order with invalid data formats, the order service rejects the request.

Authentication

Most of the time APIs are protected with some authentication mechanisms for security reasons, such as sending a token (a long encrypted string) in the request header. This is an important discovery path for exploration.

Permissions

APIs may have restrictions on operations they can perform for their clients. For example, an admin may be permitted to edit an existing order, but a customer executive may be restricted to just viewing the order.

Backward compatibility

Sometimes, as the product evolves, API contracts may also need to change. But since there could be existing clients using the APIs, new versions may need to be created and maintained in parallel alongside the old ones. The application must be tested with both API versions.

HTTP status codes

The status codes returned for technical and business failures should be relevant. Table 2-4 lists the most common status codes.

Table 2-4. HTTP status code sand the

ir

m

e

a

n

i

n

g

Status code	Meaning
200 OK	Indicates success for GET, PUT, or POST requests
201 Created	Indicates a new object, such as a new order, has been created
400 Bad Request	Indicates the request was malformed
401 Unauthorized	Indicates the client is not allowed to access the requested resource and should reissue the request with the required credentials
403 Forbidden	Indicates that the request is valid and the client is authenticated, but the client is not allowed access to the requested page or resource for some reason
404 Not Found	Indicates that the requested resource is not available now
500 Internal Server Error	Indicates that the request is valid but the server is unable to handle it, possibly due to internal bugs
503 Service Unavailable	Indicates the server is down (for example, when undergoing maintenance)

To explore all these discovery paths you need tools, and in the next sections I'll introduce a few.

Postman

Postman is a common tool for API testing. The installation binaries for the desktop version are available for free, and a web version is also available to try. Here, I'll give you a quick introduction to the desktop application:

1. Download the installation binary for your OS from the official site.
2. Open Postman and select New → HTTP Request. This will take you to the new request window.
3. Do a Google search for "exploratory testing" in your browser, then copy the URL and paste it in the URL field in Postman's new request window, as seen in Figure 2-10. Notice that the HTTP method is automatically set to GET in the drop-down next to this field.
4. You will see the parameters of the Google search request automatically being populated in the Params tab. The query parameter `q` will show `exploratory+testing`. You can change this to any other keyword to search for it.
5. Hit the Send button to complete the request.
6. You will receive the response status code, headers, body, and cookies in the bottom panel, as shown in Figure 2-10.

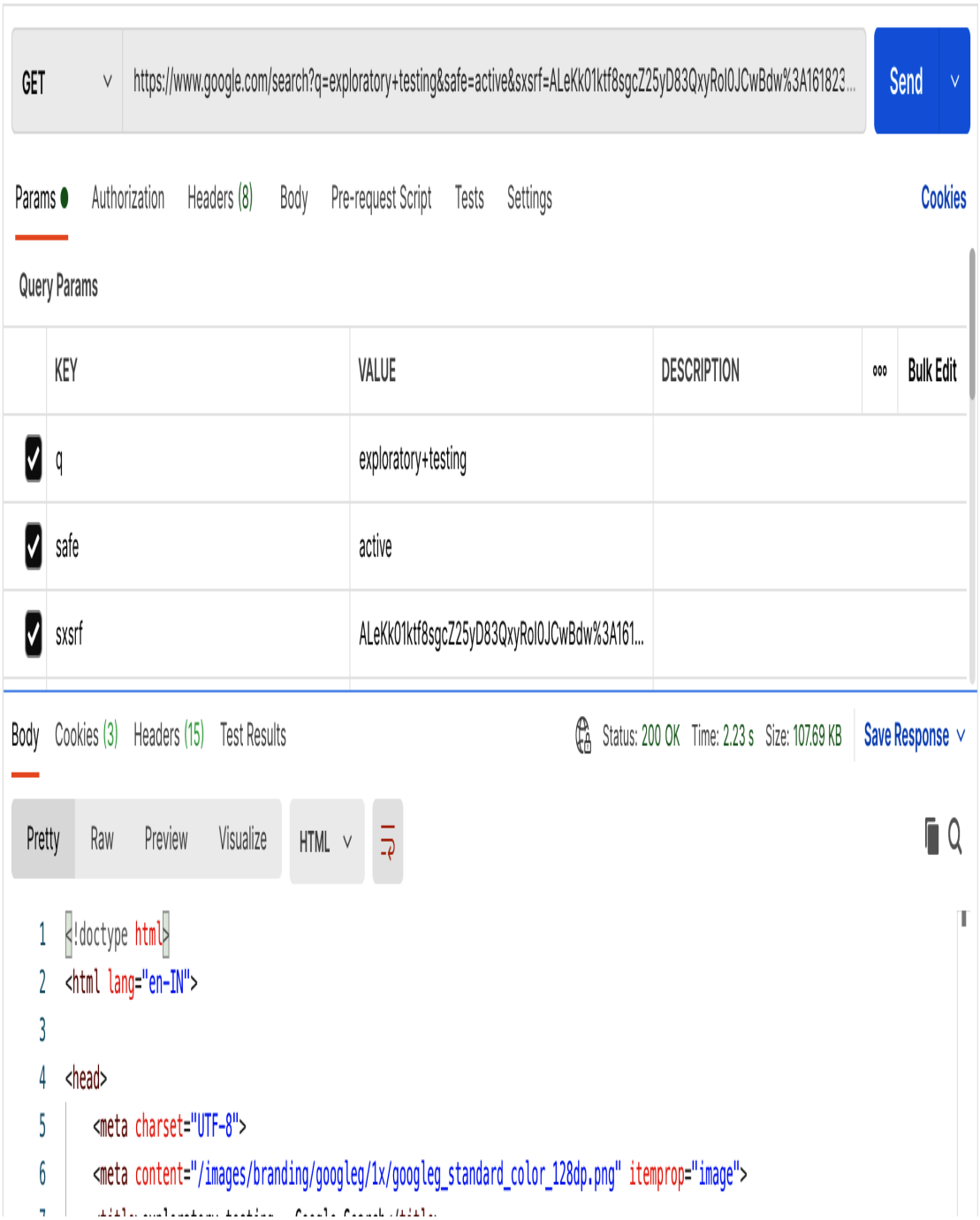


Figure 2-10. Creating a new request and verifying the response with Postman

In this case, the response is HTML. If you click the Preview button, you will see the exact same page of search results displayed in your browser. In many cases the response will instead be a JSON object, as we saw in Example 2-1; the UI will parse the JSON and display the right information.

The Google search was a GET request, but the steps for creating a POST request are similar: choose POST as the HTTP method in the drop-down, enter the API request in

the URL field and the request body on the Body tab, and hit Send to see the response. If you'd like to practice some more, Any API has a consolidated list of 1,400 publicly hosted REST APIs that you can try.

> **WARNING**
>
> Postman, by default, saves your workspace in the cloud under your Postman account. This feature is helpful to sync your work on a new machine. However, ensure that syncing to Postman's cloud doesn't violate any non-disclosure agreements (NDAs) with clients or your internal IT policy.

Postman provides several other facilities for exploring APIs. A few commonly exploited provisions are listed here:

- The token sent for authentication purposes can be sent along with the request by adding it on the Authorization tab. You can add invalid strings there to verify that the request fails.
- Similarly, when cookies are sent along with the request, they can be added on the Cookies tab (the tab is below the Send button).
- Postman captures the time taken to receive the response, as seen next to the status code in Figure 2-10. This is helpful to quickly explore if the performance degrades for different inputs.
- Instead of creating requests manually, you can directly import the API specifications from Swagger, OpenAPI, etc., with their links.

Postman also provides support for testing GraphQL and SOAP services, in addition to REST services.

WireMock

WireMock is a tool for creating and altering *stubs*, which are software components that emulate another component's behaviors. Stubs are especially useful when developing and testing complex applications with multiple integrations, where not all the integrating services are ready yet. The teams agree on a service contract and are able to continue development by creating stubs of the integrating services. Stubs are created by explicitly programming them to respond to particular requests with a defined output. This feature can be used in exploratory testing to set up different positive and negative integration test cases. Of course, it's imperative that you test the end-to-end functionality once again with the actual components once they are ready.

> **NOTE**
>
> Setting up the stub server and configuring the application to point to the stub may be taken care of by a DevOps engineer or the developers on the team. However, the testers need to know how to alter the stubs in order to simulate the test cases. This exercise is specifically included for that purpose.

To illustrate the use of WireMock, let's go back to our example ecommerce application. Suppose we don't have an actual payment service to integrate yet, but we know the request and response contract of the `/makePayment` endpoint, which the ecommerce UI uses to send payments. To explore the different test cases of this integration, we need to set up a stub of the `/makePayment` endpoint with positive and negative responses. Here are the steps:

1. Download the standalone WireMock JAR from the official website.
2. Open the terminal and run the following command with the JAR version that you downloaded:

   ```
   $ java -jar wiremock-jre8-standalone-x.x.x.jar
   ```

   The command will start a WireMock server on port 8080.
3. To create a new stub, construct the `/makePayment` API contract, as seen in Example 2-2, and send a POST request to the *http://localhost:8080/__admin/mappings/new* endpoint using Postman. (That is, in Postman's new request window, set the HTTP method to POST in the drop-down, enter the URL in the URL field and the JSON in Example 2-2 in the Body → raw section, and hit Send.)

   Example 2-2. Sample stub using WireMock

   ```
   {
       "request": {
           "method": "POST",
           "url": "/makePayment"
       },
       "response": {
           "status": 200,
           "body": "Payment Successful"
       }
   }
   ```
4. Now check that the stub works by creating another POST request to hit the *http://localhost:8080/makePayment* URL. You should receive a response with status code 200 OK and the message "Payment Successful," as described in the stub. Our imaginary ecommerce UI should show an order confirmation page on receiving this response.

5. Now, to alter the stub to return a failure response, change the response body in Example 2-2 to the following and POST it to the same `/mappings/new` endpoint:

```
"response": {
    "status": 401,
    "body": "Payment Unauthorized"
}
```

   On receiving this response, the ecommerce UI should show an error message.

Similarly, you can set up other test cases (invalid requests, service unavailable scenarios, etc.) with appropriate status codes in the response body and observe whether the UI handles them appropriately. As you can see, stubs lend a helpful hand in API exploratory testing when actual integrating services are not available to test.

We can now move on to web UI exploratory testing tools.

Web UI Testing

This section throws light on three basic web UI testing tools: browsers, Bug Magnet, and Chrome DevTools.

Browsers

The first and foremost tool for exploring a web UI is the browser. A best practice is to cover at least 85% of your application's user base while testing. At the time of writing, the most recent stats on the global distribution of browser usage from gs.statcounter.com showed that Chrome has about a 64.5% share, followed by Safari at 18.8%, then Edge at 4.05%, Firefox at 3.4%, and Samsung Internet at 2.8%. These statistics indicate that you need to include Chrome and Safari in your testing, but the third place oscillates between Edge and Firefox frequently, so it is advisable to include both to explore the UI quality. You can download any of these browsers onto your local machine, for any OS.

> **NOTE**
>
> It is sometimes necessary to test on older browsers like Internet Explorer 11 or Edge Legacy, though Microsoft has officially ended support for these versions. One way to do this is to download the Windows VM onto your machine.

Alternatively, cloud-hosted testing platforms like BrowserStack and Sauce Labs absolve you of the need to install different versions of browsers and OSs on local machines. They provide virtual access to web browsers on different OSs, for a cost. The process is simple: pay for a subscription (free trials are also available), log in to the

portal, pick a combination of browser version and OS (as seen in Figure 2-11), and test your application.

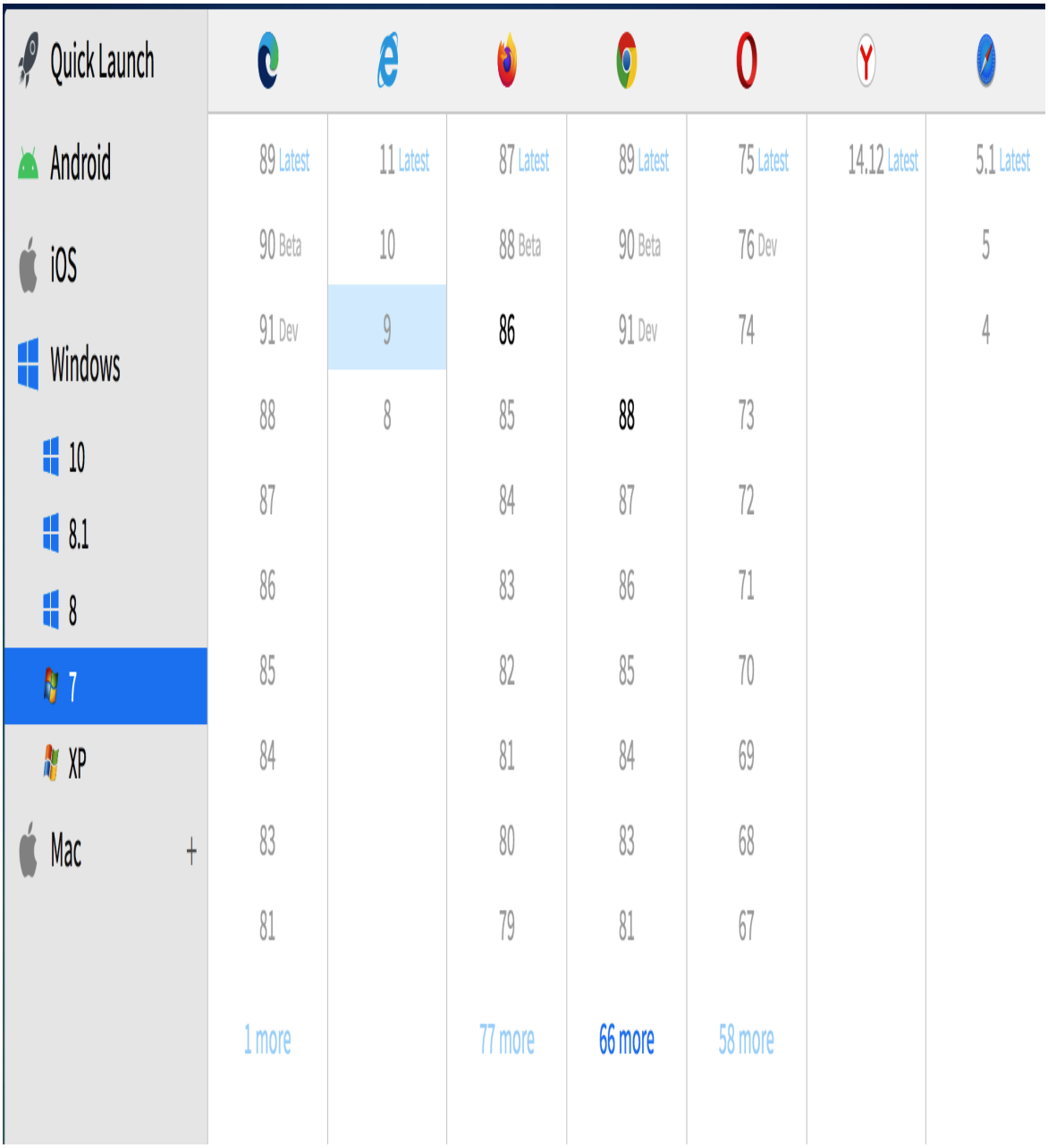


Figure 2-11. BrowserStack and similar services allow you to test various combinations of OS and browser versions.

BrowserStack also enables local testing of private applications, hosted in quality assurance (QA) or staging environments. Depending on your testing needs, subscribing to such a service may be worthwhile—they can be valuable, especially when you need to test on a wide range of older browsers.

Bug Magnet

Bug Magnet is a browser plug-in available for Chrome and Firefox that enables testing edge cases in an application. It provides a list of common test cases and appropriate values to be entered in the editable elements of the application for each test case. The tool mainly helps as a checklist for exploratory testing. To try it out:

1. Install the plug-in in your Chrome browser.
2. Open Google search and right-click on the search text field.
3. You will find Bug Magnet in the right-click menu, as seen in Figure 2-12. As you can see, it suggests a lot of edge cases, from which you can select one. For instance, choose Names → Name Length and select the first name. The long name will be populated in the Google search text box. If there are validations in your application for input string length, then an appropriate error message should appear.

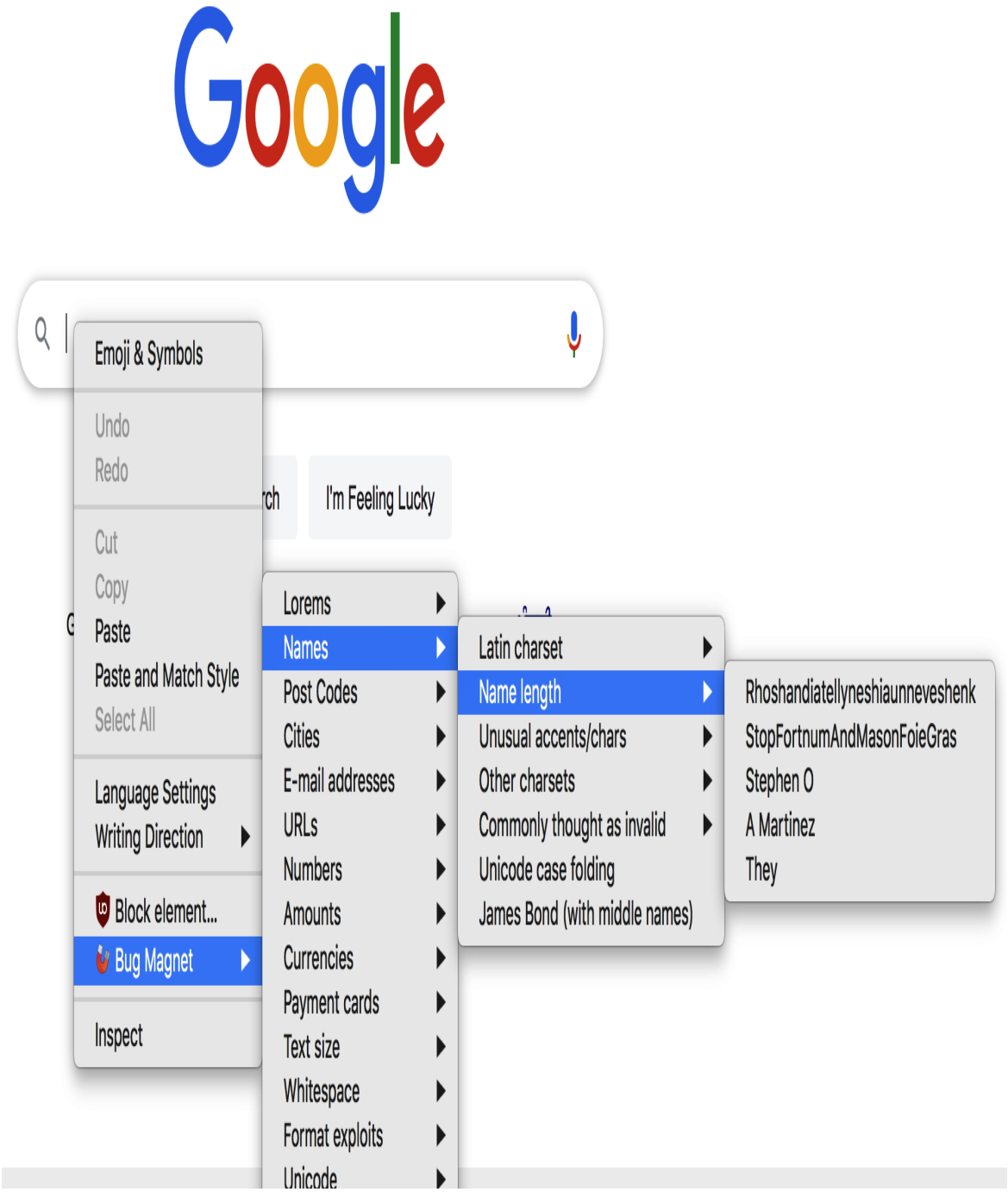


Figure 2-12. You can use the Bug Magnet plug-in as a guide during manual exploratory testing.

TIP

In addition to Bug Magnet, there are also several exploratory testing heuristics cheat sheets that you can use to ensure that you are not missing test cases. These are especially helpful for beginners.

Chrome DevTools

Chrome DevTools is as versatile as a Swiss army knife. It comes with a multitude of provisions that assist in exploratory testing, security testing, performance testing, and more. Throughout the book, you will see this tool pop up in different places. To get an idea of what it offers:

1. Open your Chrome browser and search for “exploratory testing.”
2. Right-click on the search results page and select the Inspect option. The DevTools will open immediately. You can also use the shortcut keys Cmd-Option-C or Cmd-Option-I on macOS or Shift-Ctrl-J on Windows to open DevTools.

From there, you can explore an array of things such as the following:

Page errors

As seen in Figure 2-13, the Console tab shows the errors on the web page. A web page should stay clear of any errors as a general practice, so it’s a good idea to check this tab as you land on each new page of the test application. Errors reported in this tab can also help with debugging any issues you find in a web page. For example, if you see that an image is missing, you can check the Console tab and include the error reported here with your bug report.

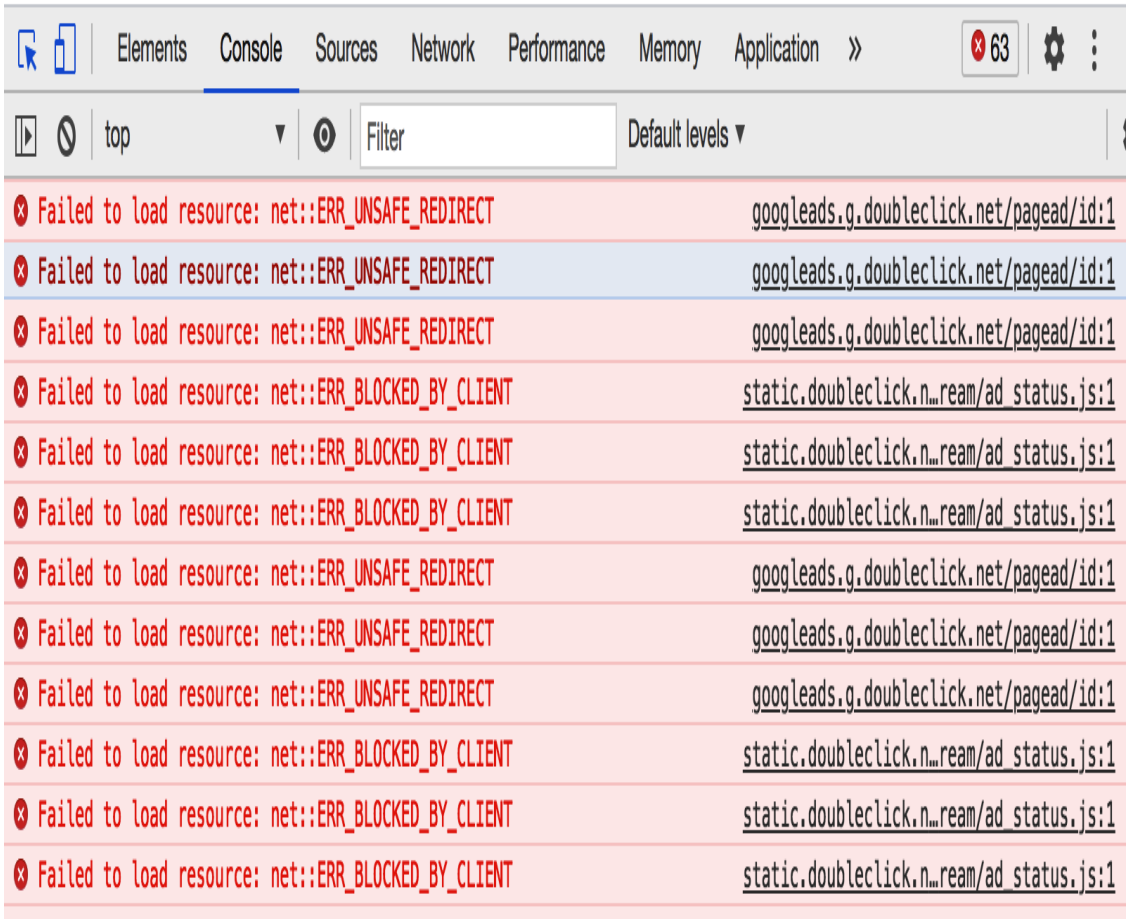


Figure 2-13. The Console tab showing the errors on a web page

Number of requests made from a page

Sometimes, due to errors in the application logic, a web page may make many unwanted API calls and become slow. You can catch such issues in the Network tab, which shows the total count of requests sent from that page at the bottom left.

First-time user behavior

When you test the same application repeatedly, some of the resources (such as images on the web pages) will get cached. So, if an image is changed during development, this may go unnoticed. Use the “Disable cache” checkbox on the Network tab to clear the cache, and view the page again. Also bear in mind that the cache works similarly for end users, so this provision helps you to explore the first-time user experience of the application.

UI behavior on slow networks

To explore the experience of an end user with limited network bandwidth, you can throttle the network from the Network tab and observe the UI’s behavior. As seen in Figure 2-14, there is a drop-down next to the “Disable cache” checkbox that allows

you to simulate 2G, 3G, and 4G network conditions. Select an option from the drop-down, clear the browser cache, and reload the page. DevTools will display a series of screenshots, as seen in Figure 2-14, that tell the story of how the application gradually loaded at the specified bandwidth. Progressive web apps (discussed in Chapter 11) work even when offline, and the network throttling drop-down also includes an option to go offline to verify this behavior.

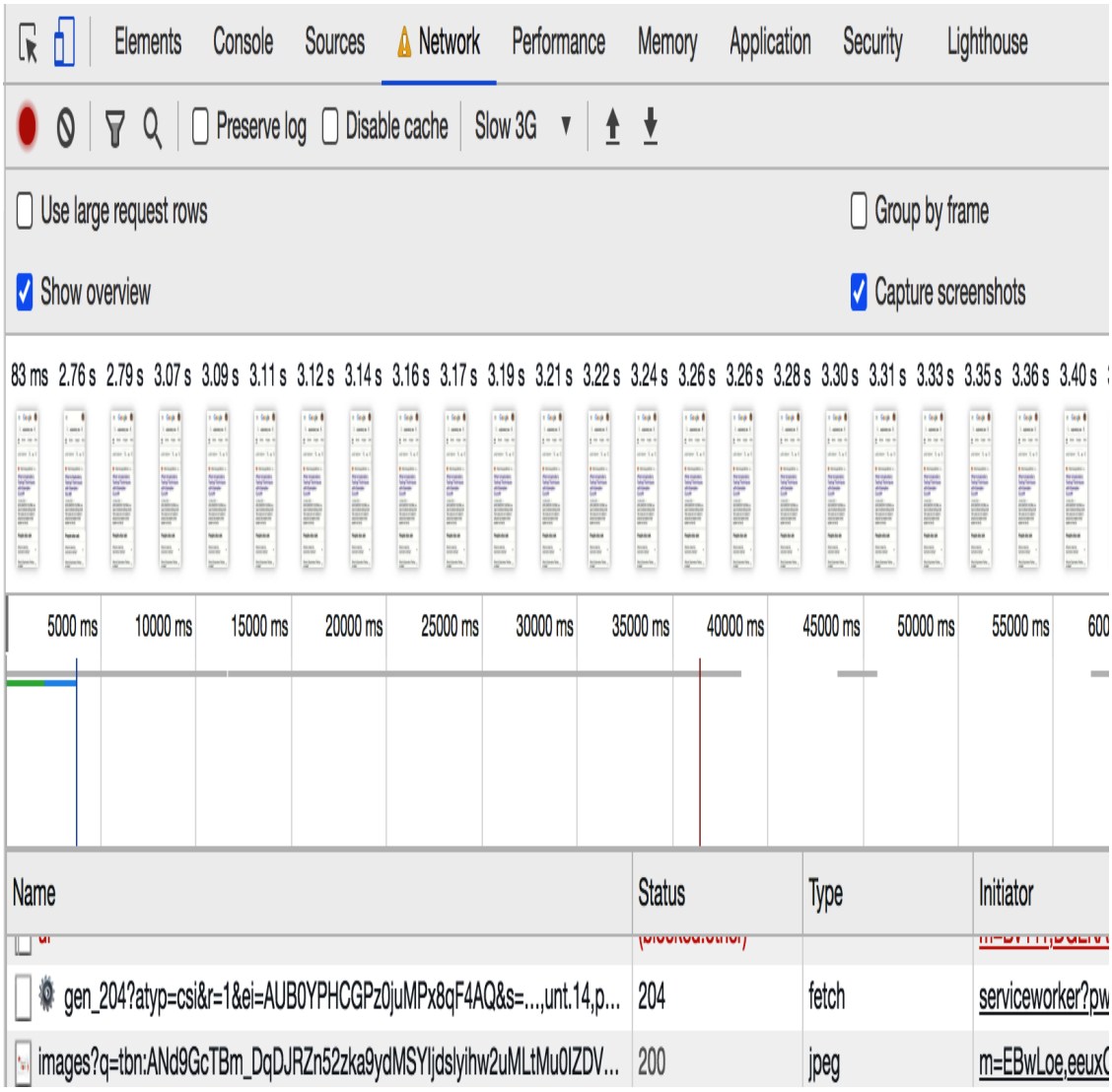


Figure 2-14. Network throttling using Chrome DevTools

UI and API integration

The Network tab captures all the network calls on the web page, including any calls to web services from the UI. It records the request and response headers (including authentication tokens), query parameters, responses, and other useful information about every request made, as seen in Figure 2-15.

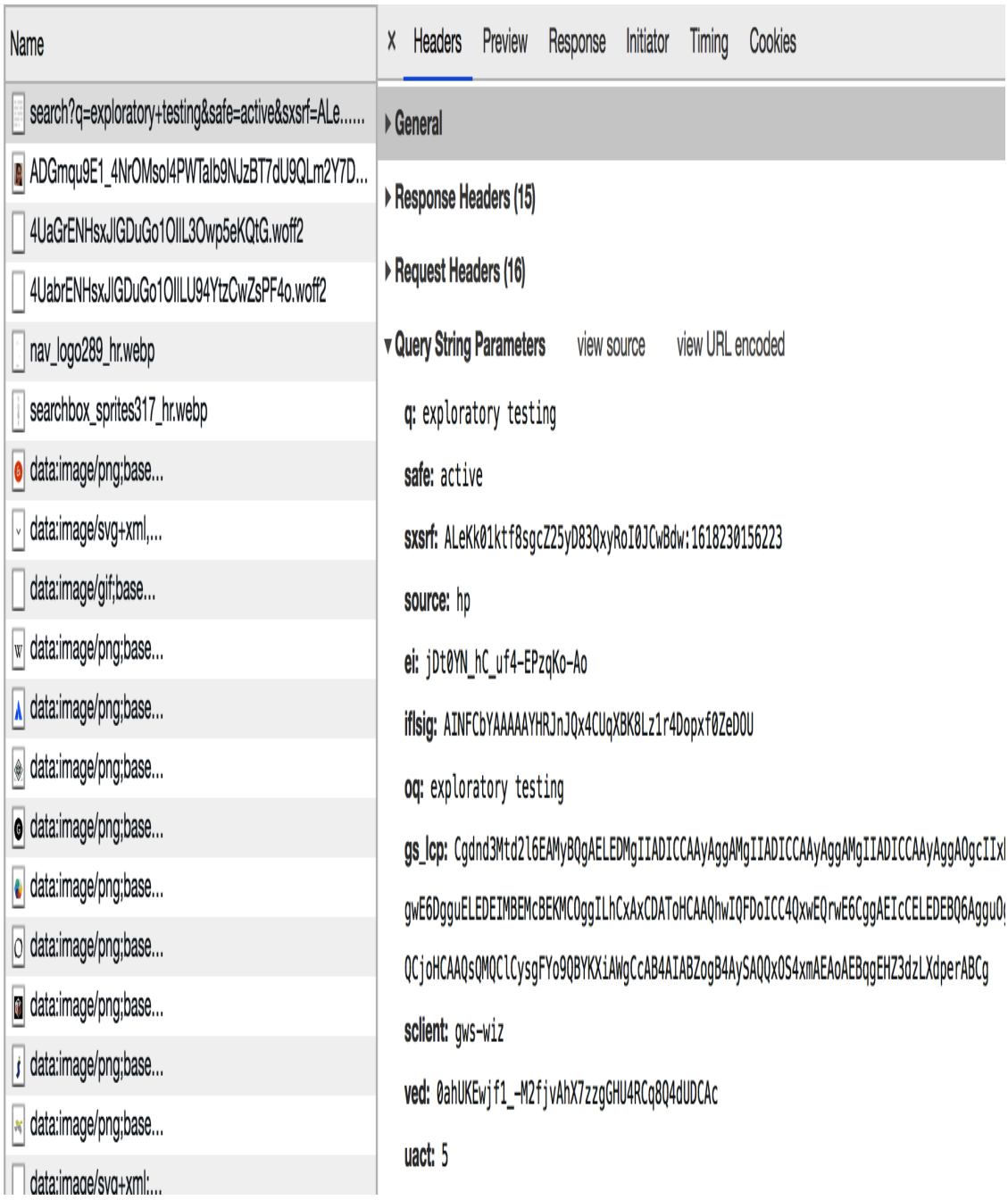


Figure 2-15. Request and response details on the Network tab

This request/response information can be used to explore UI/API integrations. For example, you can check if the UI passes the correct query parameters as entered by the end user to the right endpoint. Also, you can observe the UI behavior for different responses from the service. For instance, the UI should show an “Item Unavailable” error message when the item availability API returns a 404 status code.

Service down behaviors

When you have to simulate request failure test cases, you can block the specific request from the Network tab and observe the UI behavior. For example, find the first image-loading URL in the “exploratory testing” Google search results on the Network tab, right-click and choose “Block Request URL” from the menu, and reload the page. The respective image will be blocked from loading in the UI. This feature can be used to test a “service down” scenario without actually bringing it down.

Cookies

Cookies are primarily used to store the session information in an application. The Application tab shows the list of cookies stored and their details, as seen in Figure 2-16. You can also edit or delete cookie values from here during exploration and observe the application’s behavior.

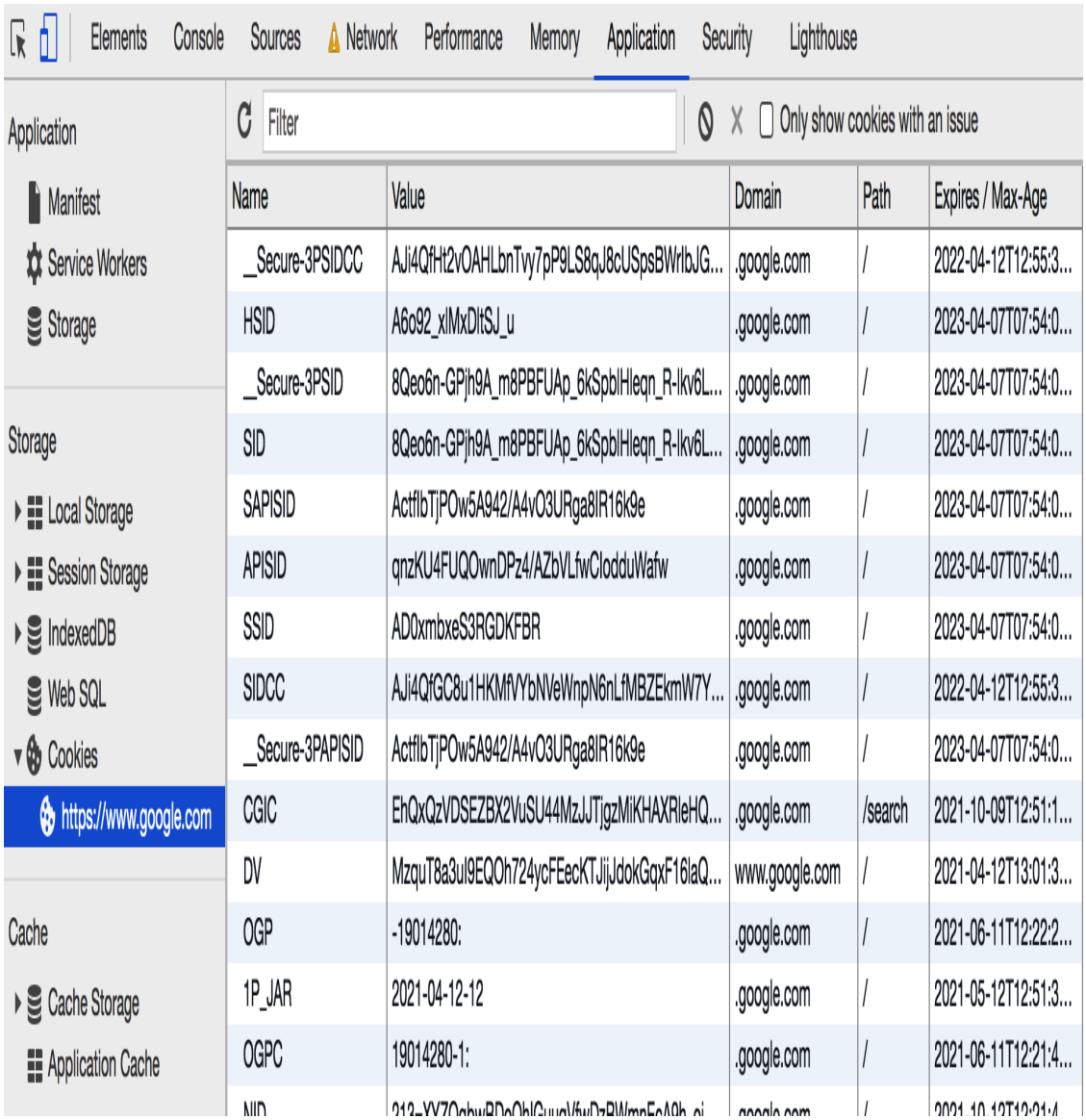


Figure 2-16. Cookies can be edited or deleted from the Application tab.

To read more about the fine-grained details of all the Chrome DevTools features, check out the official site.

You're now equipped with a handful of tools to get started with exploring APIs and web UIs. But there is one more key topic left on the pathway to achieving the goals of exploratory testing: maintaining good hygiene in the test environment. We'll discuss that next.

Perspectives: Test Environment Hygiene

The test environment is the actual playground where the testers apply their exploratory testing skills, and when it is poorly maintained, it directly affects the testers and their

outcomes. The following list describes a few maintenance smells you might encounter, their impact, and remedies to overcome them:

Shared versus dedicated test environments

In large teams a single test environment is often shared among multiple subteams, and this places heavy restrictions on the ability of individual teams' testers to meddle with the environment in the way dictated by their discovery paths. For example, if they need to bring a service down momentarily, they need to get consent from other teams. Worse, they need to coordinate with other teams or wait for the next scheduled deployment, which may be once a day or once a week, to explore the latest code. Having a dedicated test environment, at least with the components that fall under the remit of an individual subteam, will give its testers the liberty to explore adventurously.

Deployment hygiene

Once the team has its own dedicated environment, a suitable approach is to have a manual trigger for new deployments instead of automated deployments through the continuous integration pipeline, as those could alter the tester's configurations in the environment without warning. Also, the build should be made available for deployment in the CI pipeline only when the automated test run stage has passed. This is to ensure that there are no open defects in the latest code, which can block exploratory testing. You'll learn more about CI and deployment strategies in Chapter 4.

Additionally, the test environment should be set up as closely as possible to the production environment, with firewalls, separated tiers/components, rate limit configurations, etc., as part of the deployment. Only then can the failure test cases discussed earlier be explored thoroughly.

Test data hygiene

Test data comes under the purview of the testers, and they should follow certain practices to ensure that they don't unintentionally derail their own exploration. In particular, be wary of stale data and configurations when continuing to test a new feature in the same deployment. A recommendation to avoid such complications is to make it a point to deploy a new build whenever starting a new user story (assuming that a new deployment will clear out the old data and configurations and restore the application back to a fresh state). Another option is to create a new set of test data for every user story, such as a new user, instead of exploring with existing users, which could be in different states.

Test data creation can be complex when there are hundreds of linked tables. An option then is to have the new deployment delete the old data and replace it with a standard set of test data, or have a SQL script create appropriate new test data as part of the deployment. Another possibility is to anonymize the production data and

use it in the testing environment, although that may raise concerns from a security perspective if due diligence is not paid.

Autonomous teams

More often than not, access to the test environment is restricted. Team members may not have login credentials, or the required permissions to update configurations, look at application logs, or set up stubs; to perform actions like these, they need to request assistance from the DevOps or system maintenance team. This is especially frustrating during exploratory testing, where the testers may need access to all the application components. Ensuring the team is autonomous and has access to everything it needs will cut down on delays due to external dependencies and enable smooth delivery.

Thirty-party services setup

Usually third-party services are left out of the test environment setup, with the assumption that the integrations can be tested directly in production. This may result in unwanted blockages, especially when problems are discovered too late in the delivery cycle. Hence, when setting up the test environment it's important to ensure there's some way to explore the integrations with third-party services, either by employing stubs or by paying for limited access to those services.

Now that we have discussed the what, why, and how of manual exploratory testing, it is important to emphasize that this remains an art, which draws its energy from the analytical and observational skills of the individual. And due to this individualistic nature, there isn't really a set way to validate the outcomes of exploratory testing. In other words, today your analytical brain could spark a discovery path that leads to uncovering bugs, but it may not do the same tomorrow. Because of this unpredictability, it's important to maintain a disciplined approach toward exploratory testing by following the concepts mentioned in this chapter.

Key Takeaways

Here are the key takeaways from this chapter:

- Manual exploratory testing involves wandering through the test application with the intention to explore and understand the application's behavior, which may eventually lead to discovering new user flows and bugs in the existing user flows.
- Manual exploratory testing differs from manual testing in that the latter is about checking a list of specifications, while the former relies on an individual's analysis and keen observational skills.
- Exploratory testing brings together the business needs, the technical implementation, and the end user's perspective while challenging what is known to

be true from all these angles.

- We discussed a pack of eight exploratory testing frameworks that can assist in structuring the tester's thought processes and deriving meaningful test cases.
- The manual exploratory testing strategy emphasizes understanding the application details in five broad areas and then kick-starting the exploration of four essential pathways: the functional user flows, failures and error handling, UI look and feel, and cross-functional aspects.
- Exploratory testing has to be a continuous process. It can be planned to repeat in different phases of the delivery lifecycle, such as in dev-box testing, user story testing, bug bashes, and release testing.
- To explore the different discovery paths in an application, you may need to learn to use new tools. This chapter discussed relevant API and web UI exploratory testing tools such as Postman, WireMock, Bug Magnet, and Chrome DevTools.
- The test environment is the playground for manual exploratory testing, and maintaining its hygiene is critical to achieving the goals of exploratory testing. We discussed some common issues in test environment maintenance and remedies to overcome them.
- Manual exploratory testing is a highly individual process, relying on analytical and observational skills. Structuring the approach toward exploratory testing is vital to streamline the outcomes.

---

1 A mind map is a visualization technique where the main ideas are captured along with their branches. Tools like Coggle and XMind can be used for drawing them.

Chapter 3. Automated Functional Testing

Bring aboard your autopilot!

Automated testing is the practice of using tools instead of humans to perform user-like actions on an application and verify its expected behavior. The practice has been around since the 1970s, and the techniques and tools in this space have continuously evolved alongside software. To cite a few examples, in the 1970s software applications were predominantly written with FORTRAN and the RXVP tool was used to do automated testing. In the 1980s, when PCs evolved, AutoTester was introduced for automated testing. In the 1990s, when the World Wide Web boomed, test automation tools like Mercury Interactive and QuickTest became popular, and the automated load testing tool Apache JMeter was invented. With the continuous advancement of the web, the 2000s saw the birth of Selenium, and the number of automated testing tools has been growing ever since. Today, we even have AI/ML-powered automated testing tools that enrich the overall test automation experience.

This innovation has been driven by a few key observations: automated testing significantly reduces the cost of testing and enables software teams to get faster feedback on application quality than they would with manual testing. To show why this is the case, let’s consider a scenario where you only perform manual testing throughout your application development cycle, and see how automated testing compares in the same situation. Let’s say, on average, each feature in your application has 20 test cases, and you take 2 minutes per test case to execute them, or 40 minutes to test one feature manually. Whenever a new feature is developed, you need to test its integration with the existing features and also ensure

the existing features are not broken due to the new changes—a practice referred to as *regression testing*. The risk of not doing regression testing early enough is that you will find integration bugs only during release testing, which is very late in the cycle and might delay the release timeline. So, in our example, regression testing along with new feature testing will take 80 minutes when there is a second feature, 120 minutes when there is a third feature, and so on.

Soon enough, when your application has to go live with 15 features, you will have to plan for 600 minutes of testing time. To make matters worse, sometimes, a mature application has to work across different versions of services. Your testing time will increase proportionally to the number of versions that need to be supported. For example, if a service has two versions, your application testing time will become 1,200 minutes for every release. Additionally, if you find bugs, depending upon their nature (e.g., a bug that requires a change in DB schema) you might end up spending another 1,200 minutes testing the application before going live! This cycle will continue with an increase in testing time as new features get added to every release.

Businesses that don't invest enough in automated testing combat this problem by increasing their manual testing capacity—but they still get slower feedback than they would with automated testing. For example, for our hypothetical application, even if there are 12 people testing in parallel it will still take them 100 minutes, whereas automated tests in the right layers can run much faster and give quicker feedback. It's also important not to forget that if you have automated tests, you don't have to assemble your 12 teammates at midnight to test an urgent production defect fix before releasing. And even if you dare to, manual testing can be error-prone as it depends heavily on the quality of the test cases' documentation and execution.

Of course, there is a cost to creating automated tests and running them regularly. However, it's a cost that needs to be weighed against

the value of delivering the product quickly and frequently to market, the expense of manual testing (in terms of time and capacity), and the confidence it gives the team during development and while fixing production issues.

In summary, a recommendation for businesses is that you need both manual and automated testing to deliver a high-quality product, and a wise strategy is to balance them—choosing one or the other is not an option. In simple words, the strategy could be: use your manual testing capacity to perform manual *exploratory* testing to discover new test cases, and automate them to cater to regression testing.

We discussed manual exploratory testing in Chapter 2; this chapter's goal is to enable you to perform effective automated functional testing of web applications across all application layers. I'll introduce an automated functional testing strategy that can give your team faster feedback, and show you how to use automation tools and set up frameworks at different application layers. The chapter also provides an overview of AI/ML tools in the automated testing space, and presents antipatterns in automated testing that you should watch out for and tips to alleviate them in the early stages. Are you ready? Let's dive in!

Building Blocks

To begin with, let me recall the discussion from Chapter 1 where we spoke about how testing has to be practiced at both the micro and macro levels of the application to deliver high quality. This extends to automated functional testing as well.

When it comes to implementing such testing, some organizations focus solely on the macro-level tests in the higher layers of the application, adding more and more UI-driven end-to-end functional tests, and entirely miss out the micro-level tests in the lower layers of the application. For example, one team I consulted with had 200+ UI-driven end-to-end functional tests; the suite took 8 hours to run

every day, only to fail at the end due to the inherently brittle nature of the macro-level tests. This clearly is an antipattern as it not only runs counter to the goal of getting fast feedback by performing automated testing, but also doesn't provide stable feedback. This is why teams need to include both micro- and macro-level tests as part of their automated functional testing efforts: the micro-level tests run faster and are more stable.

Let's start with an introduction to the different types of micro- and macro-level tests. Later, we'll walk through some exercises that will guide you in implementing them.

Introduction to Micro and Macro Test Types

As we discuss the different test types, observe four of their traits: the scope at which they operate, the purpose they fulfill, their swiftness in giving feedback, and the amount of effort needed to create and maintain them. This fundamental understanding will enable you to tailor the automated testing efforts for your project appropriately (that is, you can choose which ones to use based on your project's needs). To explain the different test types, we'll again use our hypothetical ecommerce application from Chapter 2.

As shown in Figure 3-1, the application has three layers: the ecommerce UI, the RESTful services (authentication, customer, and order services), and the database (DB). Briefly, the UI interacts with the services to process information, and the services communicate with the database to store/retrieve relevant information. The application also integrates with an external product information management (PIM) service and downstream systems (the warehouse management system and so on) in order to fulfill the orders. A typical user flow in the application would be as follows: the user enters their credentials in the ecommerce UI, the credentials are sent to the auth service for verification, and on successful login, the user searches for products and places orders via the ecommerce

UI. The responsibility of the order service is to receive the orders placed by the user, validate the product information against the external vendor PIM service, and pass it on to the warehouse management system to trigger the delivery processes.

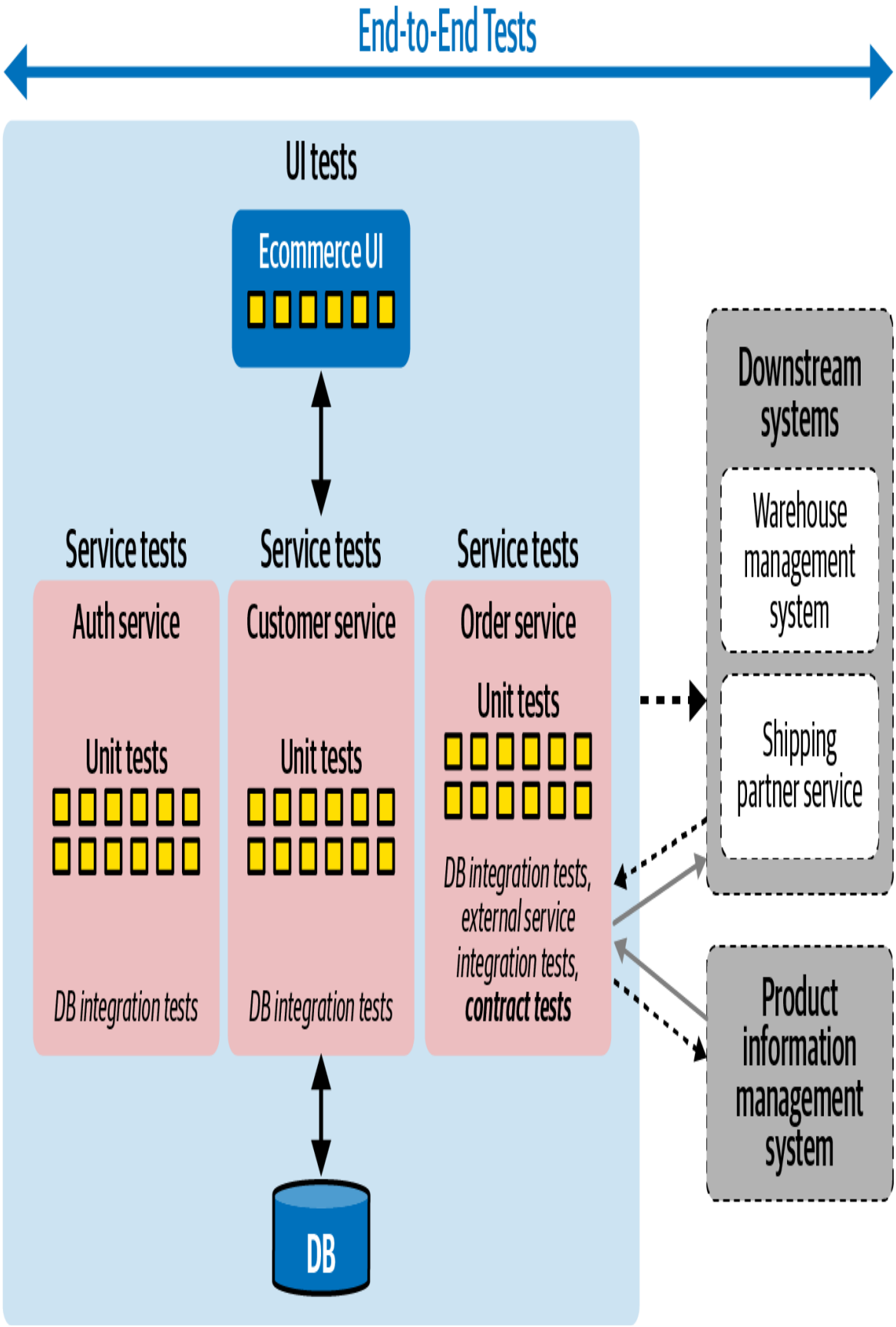

End-to-End Tests
UI tests
Ecommerce UI
Service tests
Auth service
Unit tests
DB integration tests
Service tests
Customer service
Unit tests
DB integration tests
Service tests
Order service
Unit tests
DB integration tests, external service integration tests, contract tests
DB
Downstream systems
Warehouse management system
Shipping partner service
Product information management system

Figure 3-1. Micro and macro tests at appropriate layers of an example service-oriented ecommerce application

Figure 3-1 shows the different micro and macro-level tests required at appropriate layers to meet the automated functional testing needs for this application holistically. Let's unfold them one by one.

Unit tests

You will find unit tests in all of the services and also in the UI layer of our example application. These tests aim to create safety nets at the micro level. They validate the smallest portions of an application's functionality; for example, a unit test might verify a method's behavior in a class. This is the level at which you will want to add automated tests for most of the basic input validation.

Let's say we have a method called `return_order_total(item_prices)` in the order service of the ecommerce application, which returns the total order amount. The following are some of the unit tests that can be added to verify its behavior:

- Return the total amount when `item_prices` has a negative value due to discounts.
- Return the total amount when `item_prices` is empty.
- Return the total amount when `item_prices` contains an invalid value (e.g., a value including letters, symbols, etc.).
- Return the total amount when item prices are sent with different currency symbols and separators if the application supports localization.
- Return a properly rounded total amount with fixed decimal values.

Unit tests reside within the application code base and are written by developers. In teams that follow test-driven development (TDD),

developers write unit tests before the application code, make them fail, and then add just enough application code to make them pass. This practice helps avoid unwanted and untested logic in the code. JUnit, TestNG, and NUnit are some commonly adopted unit testing frameworks in the backend. Jest, Mocha, and Jasmine are popular frontend unit testing frameworks.

Unit tests are the fastest to run. Since they reside inside the application code base, they are easy to create and maintain. They are usually run as part of the application build stage on a local developer's machine, achieving the goals of shift-left testing and providing quick and early feedback.

Integration tests

In most medium and large web applications there are quite a few integration points with internal or external components such as services, the UI, databases, caches, filesystems, and so on, which may be distributed across network and infrastructure boundaries. In order to test that all these integration points work as expected, you need to write integration tests that run against the actual integrating systems. The focus of such tests should be to verify the positive and negative integration flows, not the detailed end-to-end functionality. As a result, they should ideally be as small as unit tests.

In the ecommerce application example, the order service integrates with internal components such as the ecommerce UI, the database, and other services to exchange information. It also integrates with the external vendor PIM service and the downstream systems. We need to write integration tests for each service to verify whether it can properly communicate with other dependent services and with the DB, and to see if the order service integration tests should be added to verify the integration with those external systems and services in particular.

Integration tests can be written using unit testing frameworks like those mentioned in the previous section, along with specific tools to

simulate the integration—for example, JUnit can be used with Spring Data JPA to write DB integration tests. These tests also reside with the application code, making them relatively easy for developers to create and maintain. Their swiftness depends on the time taken by the external system to respond; therefore, they can be slower than unit tests, which run in complete isolation.

Contract tests

Integration tests may not be feasible if the integrating services are also under development. This is usually the case mostly in large-scale application development, where multiple teams work independently on different services. In such projects, teams agree on a standard contract for every service and work with stubs of the dependent services until they are ready. However, when stubs are used, there is a caveat—you won't know if the actual integrating services' contracts have changed! If this happens, you'll continue to build new features on top of broken contracts until you figure that out during actual integration testing with real services, at the end of the development cycle. This is one of the primary reasons to have contract tests.

Contract tests are written to validate the stubs against the actual contracts of the integrating services and to provide feedback continuously to both teams as they progress with development. Contract tests don't necessarily check for the exact data returned by the integrating service, but rather focus on the contract structure itself. In our example ecommerce application, contract tests can be added to validate the external vendor PIM service's contract so that whenever it changes, we can change the order service features accordingly. Contract tests can also be written for the integrations between the ecommerce UI and the services, if the development happens in parallel. The end-to-end workflow of contract testing involves collaboration between teams and is discussed in detail later in the chapter. Tools like Postman and Pact enable automation of this workflow.

Contract tests, in general, run very fast as their scope is small (simply verifying the contract structure). They reside with the application codebase and hence are relatively easy for developers to create and maintain, although not as simple as unit tests. The additional complexity is because of the end-to-end setup that requires collaboration between teams.

Service tests

As discussed in Chapter 2, APIs need to be treated as products themselves and tested thoroughly, independent of the UI behavior. This is the focus of service tests.

Services essentially handle all the domain-specific logic such as the business rules, error criteria, retry mechanisms, data storage, and so on. They reject invalid requests after validating their structure and value format. This is where macro-level testing begins, as service tests cover integrations, domain workflows, and so on. For example, here are some service tests we might add to our ecommerce application for the order service:

- Verify that only an authenticated user can create a new order.
- Verify that an order is created only if the items are available at the point of creation.
- Verify that the right HTTP status codes are returned for positive and negative inputs.

Similarly, every service needs to have service tests for all its endpoints.

Service tests sometimes reside in a separate code base, but to get fast feedback it's best to keep them as part of the service components themselves. They're slightly more complex to create and maintain than unit tests, as they involve a real test data setup in the DB. Usually, the testers in the team own these tests. They run faster than the UI-driven end-to-end tests and slightly slower than

the previous three micro-level tests (unit, integration, and contract tests). Tools like REST Assured, Karate, and Postman can be used to automate API tests.

> **NOTE**
>
> Any entity that is well encapsulated and can be independently reused or replaced, such as a service, is called a component. When you hear the term *component tests*, you can think of service tests as one example.

UI functional tests

UI-driven functional tests are run on an actual browser and mimic the user actions in the application. These tests give us feedback on the integration between multiple components, such as services, the UI, and the DB. These macro-level tests should focus on validating all the critical user flows. One example of a critical user flow in the ecommerce application is searching for a product, adding the product to the cart, paying for the product, and getting an order confirmation. This can be added as a UI functional test. When writing such tests, avoid validating the same details covered as part of the lower-level micro tests again, as this will be redundant and increase their execution time. For instance, verifying the order totals for different combinations of item prices should be covered by unit tests and needn't be validated again as part of a UI functional test.

UI functional tests are usually kept apart from the application code as a separate code base. They mainly come under the tester's purview, although they may be jointly owned with developers. These tests take longer to run and tend to be brittle, as they depend on the entire application stack's behavior, including the infrastructure, network, and so on, being stable. Additionally, they require considerable maintenance effort compared to other types of tests as failures could happen anywhere across the entire application—for

example, a change in an element ID, a delay in page load, or unavailability of services due to environment issues.

Tools like Selenium and Cypress are popularly adopted to write automated UI tests. You'll find exercises for both of them later in the chapter.

> **TIP**
>
> Every time you think about adding a UI functional test, first question the intent of the test (e.g., validating input, service-level business rules, etc.) and see if you can write lower-level micro tests to achieve the same goal.

End-to-end tests

As the name suggests, end-to-end tests should validate the entire breadth of your domain workflow, including downstream systems. In the ecommerce application, after an order is placed on the website, the downstream systems (such as the warehouse management system, third-party shipping partner services, and so on) actually fulfill the order. This end-to-end domain flow needs to be tested for proper integration.

Depending on the application context, the UI functional tests often tend to become end-to-end tests. If not, create separate end-to-end tests using a combination of UI, service, and DB testing tools to cover the entire integration flow. Obviously, these tests take the longest time to run and require more care in maintaining, as they need a stable environment and test data setup across various systems. The intent of these tests is to determine whether all the components are integrated properly end to end, and not to test the components' functionalities. So, you can just have a few tests that will activate all your components.

> **NOTE**
>
> A commonly adopted practice is for the developers to write all the micro-level tests during development and testers to write the macro-level tests as part of the testing phase. However, this is heavily dependent on the skill sets available in the team and can vary from team to team.

With that, we've traversed through all the micro and macro test types, and you should have an understanding of their four essential traits. The next section discusses an automated functional testing strategy that is widely adopted by software teams. You can use this as a foundation to define a strategy that suits your project's specific needs.

Automated Functional Testing Strategy

A one-liner strategy that can be applied to automated testing is: *add tests to validate the right scope of the functionality in the right layers of the application such that they give the fastest feedback to the team*! Mike Cohn crystallized this nicely with a visual cue in his 2009 book *Succeeding with Agile* (Addison-Wesley Professional) as the *test pyramid* concept. The test pyramid recommends having a broad bucket of micro-level tests and gradually reducing the number of macro-level tests as their scope increases. For example, if you have 10*x* unit and integration tests, you should have 5*x* service tests and only *x* UI-driven tests. If you layer these one on top of another, with the unit tests at the bottom, they form a pyramid. The obvious reason for such a recommendation is that as the scope of the tests increases, they take more time to run and cost more to write and maintain.

> **NOTE**
>
> There are other automation test shapes apart from the pyramid, such as the honeycomb and the test trophy. Essentially, they all emphasize the same principle: that micro-level tests are easier to write and run than macro-level tests. If you explore those test shapes, pay attention to what they define as the scope of each of the test types. The shapes change as the scope of the tests changes.

A typical test pyramid for a service-oriented web application such as our ecommerce example might look like Figure 3-2.

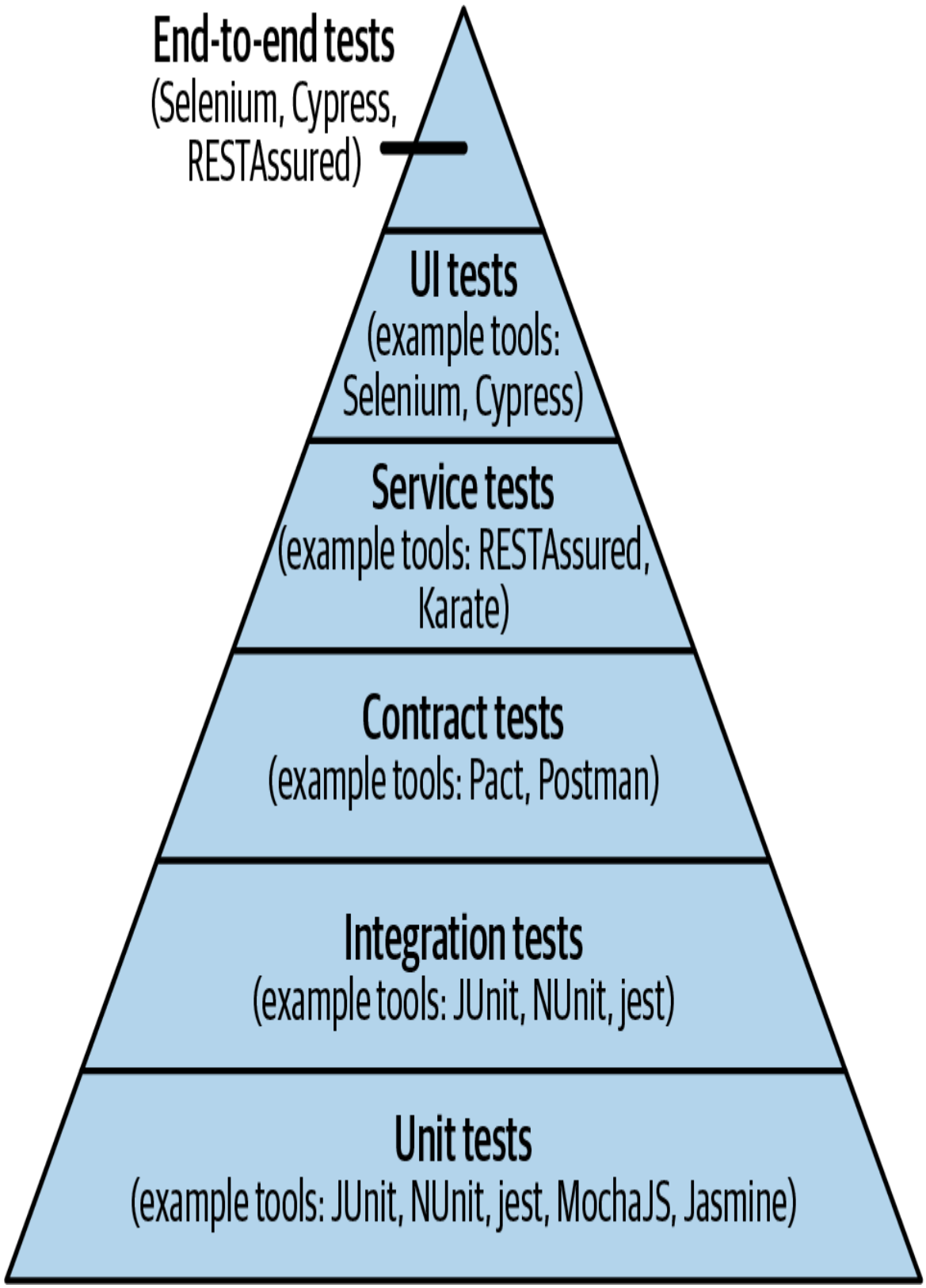


Figure 3-2. Test pyramid for a service-oriented web application

I have seen the test pyramid work in practice, and so have many other testing practitioners. One noteworthy example I can cite is that after we transformed the aforementioned project that had 200+ UI-driven end-to-end tests to adhere to the test pyramid, the team was able to get feedback within 35 minutes of code commit with ~470 tests!

> **NOTE**
>
> Although the test pyramid can be thought of as the ideal to pursue, there may be situations where it is not possible to achieve a true pyramid shape. This can be due to practical pitfalls, such as lack of a fully deployed test environment to support end-to-end tests, or lack of tools to automate certain kinds of functionalities (such as barcode scanning), or, bluntly, lack of skills. In such cases, the team should be mindful of the trade-offs they are making and choose the quantity and types of tests in such a way that they can still achieve the goal of getting fast feedback, despite these constraints.

Another part of the automation strategy should be to have a way to track the automation coverage in order to ensure there is no backlog. Test management tools like TestRail, project management tools like Jira, or something as simple as an Excel sheet can be adopted for this purpose. Tracking automation coverage is essential. For various reasons, many teams omit (or set aside) the automation efforts from the user story's scope, leading to delayed and incomplete feedback. This can cause them to lose confidence in the automation suite itself. Tracking all the test cases and ensuring they are automated can help avoid this. An ideal practice, and one that many Agile teams follow, is to call a user story "done" only if all its micro- and macro-level tests are automated!

Exercises

Having explored all of the test types, it's time to flex those coding muscles. The exercises in this section will get you started with setting up a functional testing framework in three of the application layers: I'll show you how to implement UI-driven functional tests using both Selenium and Cypress, service tests using REST Assured, and unit tests with JUnit. Let's get started!

THE TEST AUTOMATION TECH STACK

Here are a few pointers to consider when pinning down your automation tech stack:

- It's best to keep it similar to the development tech stack so that your team members don't have to learn an entirely new set of tools. I've observed that when teams have different tech stacks for development and testing, there is a natural resistance among the developers to owning the tests, hindering the objectives of shift-left testing and getting faster feedback.
- Avoid the urge to pull the tests from all layers into a common code base. Keep the tests in each layer within their respective components, so that the tests are shipped along when the components are reused. This implies that the tech stack choices in each layer will depend on the respective component's development tech stack.

UI Functional Tests

Selenium WebDriver and Cypress are two popular tools that aid in creating UI-driven functional automated tests. You can write Selenium WebDriver tests with many programming languages, such as Java, C#, Python, JavaScript, etc. Although Cypress tests can be written only in JavaScript, this tool is packed with many benefits, as

you will see. I'll show you how to use both, so based on your team's programming language preferences and development tech stack you can choose the appropriate tool.

Java–Selenium WebDriver framework

Let's start with an exercise to create an automation framework using Java and Selenium WebDriver.

Prerequisites

As prerequisites, you'll need to install the following essential tools:

- The latest version of Java
- The integrated development environment (IDE) of your choice—IntelliJ is a commonly used IDE for Java
- The Chrome browser

You will also need to install a few other tools, as described in the following sections.

Maven

Apache Maven is a build automation tool. Build automation tools essentially help in standardizing dependency management processes and project build steps. To elaborate further, in many projects the use of a few third-party libraries and plug-ins is required to create new functionalities. It is crucial for all team members to use the same versions of the libraries and plug-ins, and to follow the same sequence of steps to create application artifacts (i.e., build the project, run the tests, etc.). Build automation tools help achieve these goals. Maven and Gradle are two popular choices for building Java applications. For this exercise we will be using Maven; download it now and follow the installation instructions on the website.

To get a quick understanding of how Maven works, let's take a look at the Project Object Model (POM) XML file, *pom.xml*. This is where you define all the dependent libraries, plug-ins, and their versions, as seen in Example 3-1, and Maven takes it forward from there.

Example 3-1. A sample pom.xml file

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
              http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>SeleniumJavaExample</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>15</maven.compiler.source>
        <maven.compiler.target>15</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.0.0</version>
        </dependency>
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.4.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Here are the important things to notice in this file:

- The `groupID`, `artifactID`, and `version` attributes define the project coordinates that allow Maven to keep track of the project

over time. When creating a new project in IntelliJ, you can add these details.

- The properties section declares the Java version to be used for compilation. You can also include project-specific variables here that you want to reuse in other sections of the *pom.xml* file or application code.
- The dependencies section is where you list the dependent libraries and their versions. Note that the file refers to Selenium 4.0 as a dependency. Maven maintains a central repository of all the libraries and their different versions, from which it pulls them to your machine based on the configuration in this section of the *pom.xml* file. This centralized way of managing the libraries ensures that all team members have the exact same binaries of all the libraries. To add a dependency for a library in your *pom.xml* file, search for the library in the Maven Repository and copy its coordinate values into the dependencies section.

Similarly, you can configure plug-ins, environments, etc. by declaring them as attributes appropriately in your *pom.xml* file, as described in the Maven documentation. You'll be using the file in Example 3-1 to write your Selenium WebDriver tests.

Maven also provides build lifecycle commands that are needed to create application artifacts. Commands that you will use frequently include:

`mvn compile`

Compiles the project code

`mvn clean`

Cleans up (i.e., removes) the previously created artifacts

`mvn test`

Runs the tests written using testing frameworks (we'll set up one of these next)

There are also other Maven commands to install, deploy, and so on, to complete the entire lifecycle of application artifact creation.

TestNG

TestNG is a testing framework, also referred to as a *test runner*. JUnit is another popular testing framework in Java. Testing frameworks, in general, provide the ability to create tests, add assertions, add setup and teardown tasks, organize tests into groups, execute tests, and present a test run summary. TestNG can be used for all types of tests: unit, integration, and end-to-end. To install it, you can simply add it as a dependency in your *pom.xml* file, as seen in Example 3-1.

A few prominent features in TestNG that you might use regularly are as follows:

- `@Test`: An annotation to indicate the method in a class as a test method for TestNG to run them. So every test should be preceded with this annotation.
- `@BeforeClass`, `@AfterClass`, `@BeforeMethod`, `@AfterMethod`, `@BeforeSuite`, `@AfterSuite`: As their names suggest, these run before or after test classes, test methods, or the entire suite. Your test's setup and teardown methods can be annotated with these tags.
- `assertEquals()`, `assertTrue()`, and other assertion methods to perform validations within the testsIntelliJ guides you in the syntax of these methods while you're creating tests.

Selenium WebDriver

Jason Huggins originally invented Selenium, a popular open source test automation tool, in 2004, and it has gone through many

incarnations since then. You can read about the fascinating history and evolution of the tool on its website, and an extremely vibrant open source community still supports it today.

> **WHY IS IT CALLED SELENIUM?**
>
> Selenium, the chemical element, is used as an antidote to mercury poisoning. Before Selenium, the automated testing tool, the most popular tool used for automated testing was called Mercury. Get the joke?

Selenium WebDriver mainly facilitates interaction with the web application rendered in the browser. It doesn't serve any other purpose, such as assertions, report generation, etc., which is why we need other tools like TestNG and Maven to complete the automation framework.

Selenium WebDriver has three basic components:

APIs

These are the methods that let you interact with the application elements in the browser (clicking, typing in fields, etc.).

Client library

The Selenium WebDriver client library bundles the APIs for us to use in our test suite. Client libraries are available in many programming languages.

Driver

This is the component that instructs the browser to take the actions dictated by the API. The drivers are usually created and maintained by the respective browsers themselves and are not part of the Selenium distribution package. For example, if you want to run the tests against Chrome, you have to download the

ChromeDriver separately and include it in your automation scripts.

Let's first get to know the different APIs provided by Selenium WebDriver. Example 3-2 lists some of the commonly used WebDriver methods to find different elements in the application. Selenium identifies elements on a web page based on their HTML attribute values, such as `id`, `className`, `cssSelector`, etc. You can select the Inspect option from the right-click menu to get these values in Chrome. Try inspecting the Amazon search text box, and you will find the ID to be `"twotabsearchtextbox"`.

Example 3-2. A few commonly used WebDriver methods for finding elements

```
// find element by ID
driver.findElement(By.id("login"))

// find element by CSS selector
driver.findElement(By.cssSelector("#login"));

// find element by class name
driver.findElement(By.className("login-card"));

// find element by XPath
driver.findElement(By.XPath("//@login"));

// find multiple elements
driver.findElements(By.cssSelector("#username li"));
```

> **TIP**
>
> The `id` is unique to each element on a page. Hence, it is the preferred locator type to keep your tests stable. CSS selectors and XPath locators tend to break when the application undergoes frequent changes.

Selenium WebDriver also provides advanced ways to find elements on the page using relative locators, such as specifying that they are

`above`, `below`, or `toLeftOf` another element.

Once we find elements, we need to interact with them. Example 3-3 lists a few frequently used Selenium WebDriver methods for performing different actions on elements.

Example 3-3. A few commonly used WebDriver methods for interacting with web elements

```
// click an element
driver.findElement(By.id("submit")).click();

// type text into an input box
driver.findElement(By.cssSelector("#username")).sendKeys(user
name);
```

You can also use the Actions class in WebDriver for more advanced interactions like keyDown, contextClick, and dragAndDrop.

Apart from methods to interact with application elements, WebDriver also provides methods to manage browser behavior such as opening a URL, going back, closing the browser, setting the browser window size, setting cookies in the browser, switching between multiple tabs, and so on. Example 3-4 shows a few of these commonly used browser manipulation methods.

Example 3-4. A few commonly used WebDriver methods for manipulating browser behavior

```
// open a URL
driver.get("https://example.com");

// browser back, forward, and refresh
driver.navigate().back();
driver.navigate().forward();
driver.navigate().refresh();

// open browser in iPad size
driver.manage().window().setSize(new Dimension(768, 1024));

// close browser
driver.close();
```

```
// quit the driver session
driver.quit();
```

When navigating across pages, you have to make the test wait for the page to load or for an element to be visible after the page loads. Some teams use hardcoded sleep statements to make the test wait, but they make the tests fragile as the page load times could be different in different environments. WebDriver offers a few inbuilt waiting strategies to overcome this issue:

- The *implicit* wait strategy makes WebDriver poll the Document Object Model (DOM), which represents the entire content of an HTML document, for *x* seconds, waiting for the element to appear. The default WebDriver behavior is to wait for 0 seconds; you can change this and use the implicit wait during the driver initialization stage to set a standard wait time.
- The *explicit* wait strategy makes WebDriver wait for up to *x* seconds for an expected condition to become true.
- The *fluent* wait option gives more flexibility in defining a wait strategy. It makes WebDriver wait a maximum of *x* seconds for an expected condition to become true, checking whether the condition is true every *y* seconds.

Example 3-5 shows these different wait methods.

Example 3-5. WebDriver wait strategies

```
// Implicit wait for 10 seconds before timeout exception
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(
10));

// Explicit wait for 10 seconds until the submit button
becomes clickable
WebElement submitButton = new WebDriverWait(driver,
Duration.ofSeconds(10)).

until(ExpectedConditions.elementToBeClickable(By.id("submit")
));
```

```
// Fluent wait, which polls every 1 second up to a maximum of
3 seconds and waits
// for the spinner to disappear
FluentWait wait = new FluentWait(driver)
                .withTimeout(Duration.ofSeconds(3))
                .pollingEvery(Duration.ofSeconds(1))
                .ignoring(NoSuchElementException.class);
wait.until(ExpectedConditions.invisibilityOf(driver.findEleme
nt(By.id("spinner"))));
```

These are the most frequently used WebDriver methods to assist in writing day-to-day tests. WebDriver also offers many more advanced interactions, such as listening to events and taking different actions based on the event type, interacting with modal windows, and almost anything else you might want to test in a browser. Selenium 4 also allows mocking the server responses and debugging using the Chrome DevTools protocol. If you want to exploit such advanced capabilities, see the website for details.

Page Object Model

The Page Object Model is the most commonly adopted design pattern for a UI-driven automation framework. It involves re-creating the application structure just as it is in the automation framework; i.e., you create a page class for every page in your application and define the elements and actions on the page in that class. The pattern has proven fruitful as it allows abstraction and encapsulation, and hence makes it easier to fix issues or add new changes. For example, when an element ID changes, you know where to find the element (in its page class) and fix it. If you don't have such an abstraction, you will have to make the ID change in all the tests explicitly.

Example 3-6 shows a sample `LoginPage` class with three elements: a username field, a password field, and a signin button. It also has a `login(email, password)` method to perform the login action on the page. We will refer to this `LoginPage` class later when creating a test.

Example 3-6. The `LoginPage` class using the Page Object Model

```
// LoginPage.java

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    private WebDriver driver;
    private By emailID = By.id("user_email");
    private By passwordField = By.id("user_password");
    private By signInButton =
        By.cssSelector("input.gr-button.gr-button--large");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public HomePage login(String email, String password){
        driver.findElement(emailID).sendKeys(email);

driver.findElement(passwordField).sendKeys(password);
        driver.findElement(signInButton).click();
        return new HomePage(driver);
    }
}
```

Similarly, your automation framework should page classes representing all of your application pages.

Setup and workflow

Having explored all the components needed for a typical Java–Selenium WebDriver UI automation framework, the next step is to put them together and write the first test for a simple user flow: logging in to your favorite ecommerce application (choose one where you have an account) and asserting on the home page title. Follow the steps here to create this test:

1. Open IntelliJ and create a new Maven project by selecting File → New → Project → Maven.
2. Select the Java version you downloaded.
3. Move on to the next window to enter your project name, location, `groupID`, and `artifactID`, as seen in Figure 3-3.

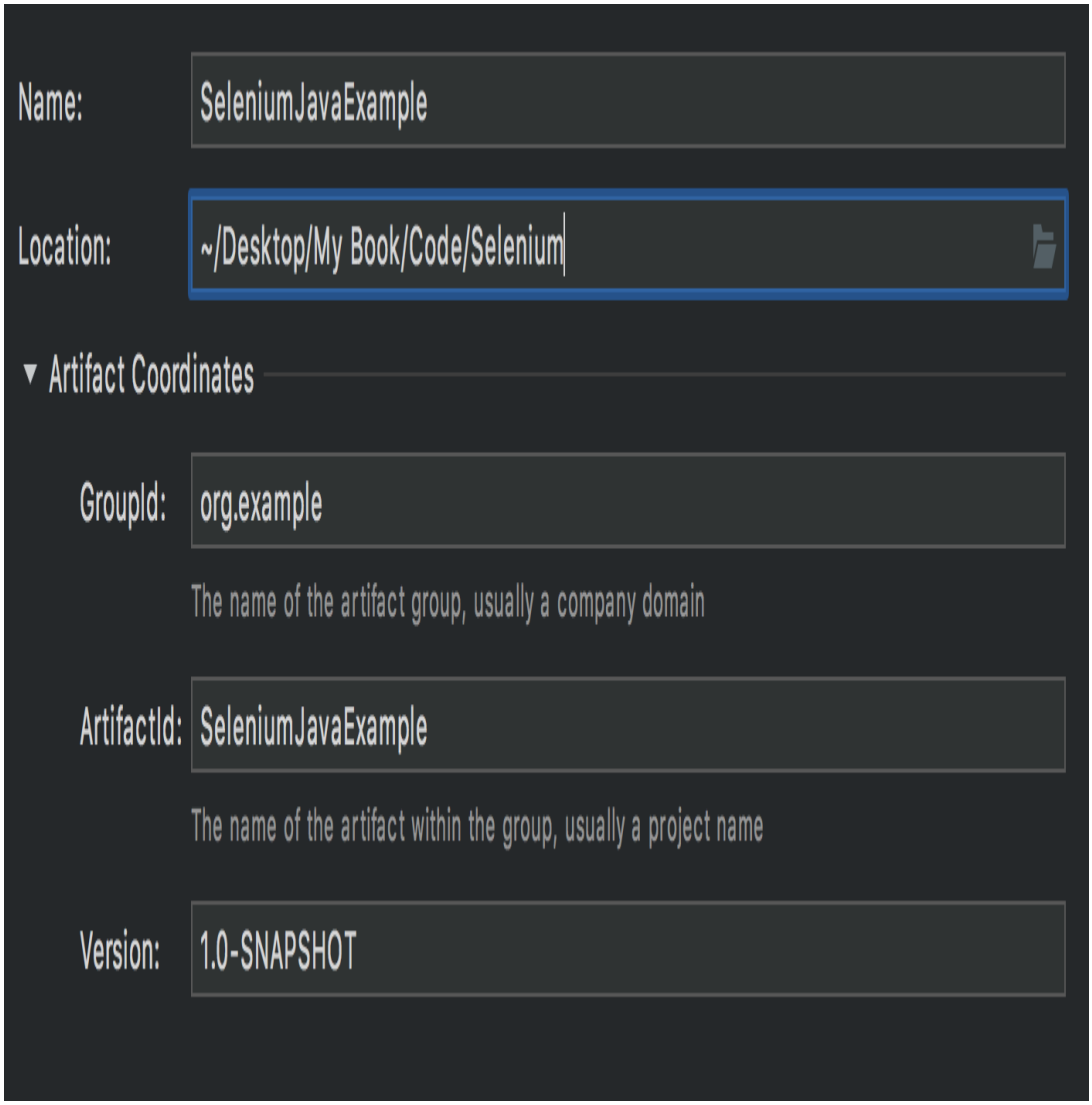


Figure 3-3. Creating a new Maven project in IntelliJ

These three steps will set up an initial project structure, as shown in Example 3-7.

Example 3-7. Initial Maven project structure

```
├── SeleniumJavaExample.iml
├── pom.xml
└── src
    ├── main
    │   ├── Java
    │   └── resources
    └── test
        └── Java
```

4. Download the ChromeDriver executable compatible with your local Chrome browser version (to view your Chrome version, select Chrome → About Chrome).
5. Place the executable file under the *src/main/resources* folder inside your project.
6. Add the project dependencies, Selenium, Java, and the TestNG library, as in Example 3-1. In IntelliJ, the Maven panel is on the right side; you can use this to refresh and download the libraries immediately.
7. Create a package called `base` under *src/test/java*.
8. Add a new class file called *BaseTests.java*, where you can define the WebDriver setup as seen in Example 3-8.

Example 3-8. The `BaseTests` class

```
// BaseTests.java

package base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

import java.time.Duration;

public class BaseTests {

    protected WebDriver driver;
```

```
    @BeforeMethod
    public void setUp(){
        System.setProperty("webdriver.chrome.driver",
            "src/main/resources/chromedriver");
        driver = new ChromeDriver();

driver.manage().timeouts().implicitlyWait(Duration.ofSeco
nds(10));
        driver.get("http://eCommerce.com/sign_in");
    }

    @AfterMethod
    public void teardown(){
        driver.quit();
    }
}
```

The `setUp()` method does a few things: it provides the ChromeDriver executable path, instantiates the `ChromeDriver` object, defines the implicit wait as 10 seconds, and opens the application URL using the `driver` object. The `teardown()` method does one job: it quits the browser session after the test run. Note the TestNG annotations `@BeforeMethod` and `@AfterMethod` used to create and delete a new driver session, which will run for every test.

9. Next, create a new package called `tests` under *src/test/java* and add your first test class—for example, `LoginTest`, as in Example 3-9. You can see the `@Test` annotation and the `assertEquals()` method provided by TestNG.

Example 3-9. The `LoginTest` class with the first test

```
// LoginTest.java

package tests;

import base.BaseTests;
import org.testng.annotations.Test;
import pages.LoginPage;
import static org.testng.Assert.*;
```

```
public class LoginTest extends BaseTests {

    @Test
    public void verifySuccessfulLogin(){
        LoginPage loginPage = new LoginPage(driver);

assertEquals(loginPage.login("example@gmail.com",
            "Admin123").getTitle(), "Home page");
    }
}
```

10. After creating the tests, you'll need to create your page classes. Create a new package called `pages` under *src/main/java* and add your page classes there. Refer to Example 3-6 for the `LoginPage` class and Example 3-10 for the `HomePage` class.

Example 3-10. The `HomePage` class

```
// HomePage.java

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import
org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

public class HomePage {

    private WebDriver driver;
    private By searchField =
By.cssSelector("input.searchBox");

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }

    public String getTitle(){
        WebDriverWait wait = new WebDriverWait(driver,
           Duration.ofSeconds(10));
        wait.until(ExpectedConditions.
           presenceOfElementLocated(searchField));
        return driver.getTitle();
```

```
    }
}
```

The page classes will have the Selenium WebDriver methods we discussed earlier to find and interact with the elements in these classes. Also, note how the page classes are chained to return the other pages' objects. For example, the `login()` method in the `LoginPage` class returns a `HomePage` object along with the `driver` object. Also remember that assertions don't belong in the page classes!

11. Now you can run the test from the IDE itself by right-clicking on the green triangle next to the @Test tag, or run it from the command line using Maven as follows:

```
$ mvn clean test
```

This command will open the Chrome browser and run your test. It also creates an HTML report at *target/surefire-reports/index.html*, as seen in Figure 3-4.

1 suite

All suites

Command line suite

Info

- [unset file name]
- 1 test
- 0 groups
- Times
- Reporter output
- Ignored methods
- Chronological view

Results

- 1 method, 1 passed
- Passed methods (hide)
  - verifySuccessfulLogin

tests.LoginTest

verifySuccessfulLogin

Figure 3-4. HTML report generated by Maven Surefire plug-in

Congratulations, you've successfully created and run your first test!

This test passed, but when there are failures in CI, having screenshots of those failures will be very useful for debugging. To take screenshots on failures, alter your `teardown()` method as shown in Example 3-11. Create a folder called *screenshots* under *src/main/resources*, and the failure screenshots will be placed there.

Example 3-11. Taking screenshots on failures

```
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.testng.ITestResult;
import java.io.File;
import java.io.IOException;
import com.google.common.io.Files;

@AfterMethod
    public void teardown(ITestResult result){
        if(ITestResult.FAILURE == result.getStatus()) {
            var camera = (TakesScreenshot) driver;
            File screenshot =
camera.getScreenshotAs(OutputType.FILE);
            try {
                Files.move(screenshot,
                    new
File("src/main/resources/screenshots/" +
                    result.getName() + ".png"));
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
        driver.quit();
    }
```

You can add more capabilities to your automation framework based on specific project needs. For example, you can run your tests in parallel using TestNG or Selenium Grid capabilities, group the tests and run across multiple browsers using TestNG capabilities, include a behavior-driven development (BDD) framework like Cucumber,

and so on. However, always remember to keep your UI tests to a minimum.

BEHAVIOR-DRIVEN DEVELOPMENT

BDD is a software development practice that intends to bring the business and the technical team members closer. For example, BDD frameworks like Cucumber provide facilities to write tests in natural language, resembling a typical user story with the Given, When, Then structure. This enables the business folks to pass on requirements as failing tests and the technical folks to start building features by fixing the failing tests.

JavaScript–Cypress Framework

Cypress was released in 2014, 10 years after Selenium was first introduced, and has gained widespread adoption as an end-to-end UI automation tool. Cypress tests can be written only in JavaScript, unlike Selenium tests. Despite that limitation, it has become popular due to some of the following salient features:

- Cypress's architecture is such that it doesn't execute commands over the network like Selenium does, but executes them within the same run-loop as the application. This makes it much faster.
- Cypress is bundled with all the tools necessary to write end-to-end UI automation tests, so you don't need to set up additional tools like TestNG, Cucumber, etc. It incorporates existing, proven tools to do their respective tasks. For example, by default Cypress uses Mocha as a testing framework and Chai for assertions.
- Since Cypress is embedded within the application, it enables the creation of varied test cases such as stubbing application functions, simulating server-down scenarios by altering requests, setting up predefined application states, and more.

- Cypress addresses test flakiness due to incorrect adoption of wait strategies by automatically waiting for a page to load and elements to be visible or clickable.
- Cypress makes debugging test failures much simpler as it provides screenshots, logs, and videos for every command that the test has executed. It also allows you to inspect errors on the application page in their predefined state as part of the test flow using Chrome DevTools.

Cypress has good open source community support, with new plug-ins frequently added to cater to advanced requirements. So, let's see how to set up a UI automation framework using Cypress and the Page Object Model.

> **NOTE**
>
> The Cypress community advocates for using the Application Actions Model rather than the Page Object Model. If you'd like to explore this, check out Gleb Bahmutov's blog post.

Prerequisites

The following tools are required to set up an automation framework in JavaScript:

- Node.js 12 or above
- The IDE of your choice—Visual Studio Code is popular for JavaScript projects
- A browser—Cypress works with Chrome, Chromium, Edge, Electron, and Firefox

Cypress

Once you have the prerequisites installed, follow these five steps to quickly get acquainted with how Cypress works:

1. Create a project directory. Install Cypress by running the following command from the project folder in your terminal:

```
$ npm install cypress --save-dev
```

2. Create a *package. json* file in this folder with the contents shown in Example 3-12.

   Example 3-12. The package.json file

```
{
  "name": "functional-tests",
  "version": "1.0.0",
  "description": "UI Driven End-to-End Tests",
  "main": "index.js",
  "devDependencies": {
    "cypress": "^9.2.0"
  },
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

3. Run the following command, which will open the Cypress application as seen in Figure 3-5. It also will set up a bootstrap automation framework structure with example Cypress tests for a sample Todo web application.

```
$ node_modules/.bin/cypress open
```

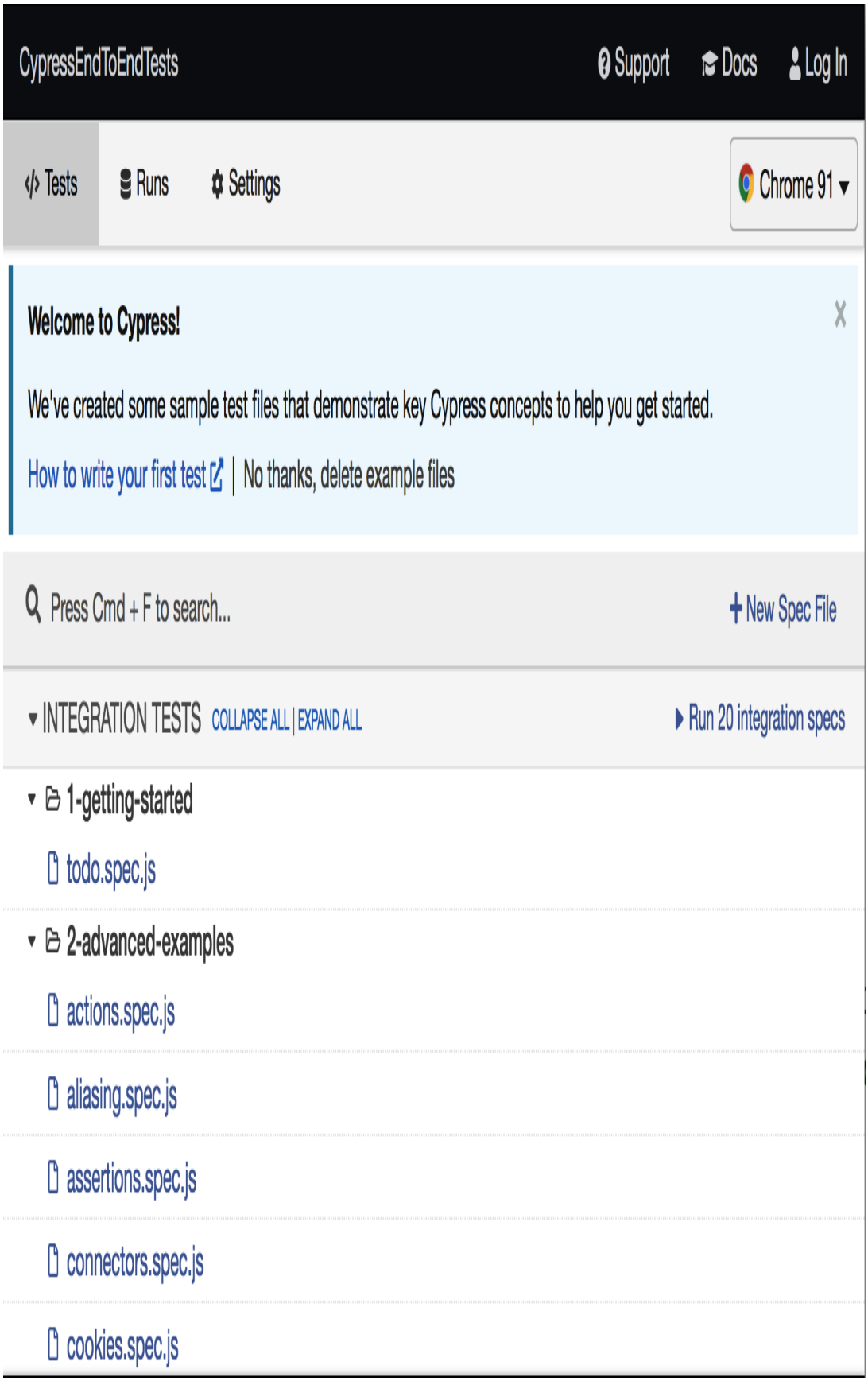

CypressEndToEndTests
Support
Docs
Log In
Tests
Runs
Settings
Chrome 91
Welcome to Cypress!
We've created some sample test files that demonstrate key Cypress concepts to help you get started.
How to write your first test
No thanks, delete example files
Press Cmd + F to search...
New Spec File
INTEGRATION TESTS
COLLAPSE ALL | EXPAND ALL
Run 20 integration specs
1-getting-started
todo.spec.js
2-advanced-examples
actions.spec.js
aliasing.spec.js
assertions.spec.js
connectors.spec.js
cookies.spec.js

Figure 3-5. Cypress application with test files

4. Once the setup is done, you can try running the existing tests to get an idea of how easy it is to work with Cypress before setting up your application-specific page object framework. Select your preferred browser from the drop-down in the top-right corner of the Cypress application, and click on any test (*.spec.js*) file. Cypress will open the browser, run the tests within the file, and show you a report like the one in Figure 3-6.

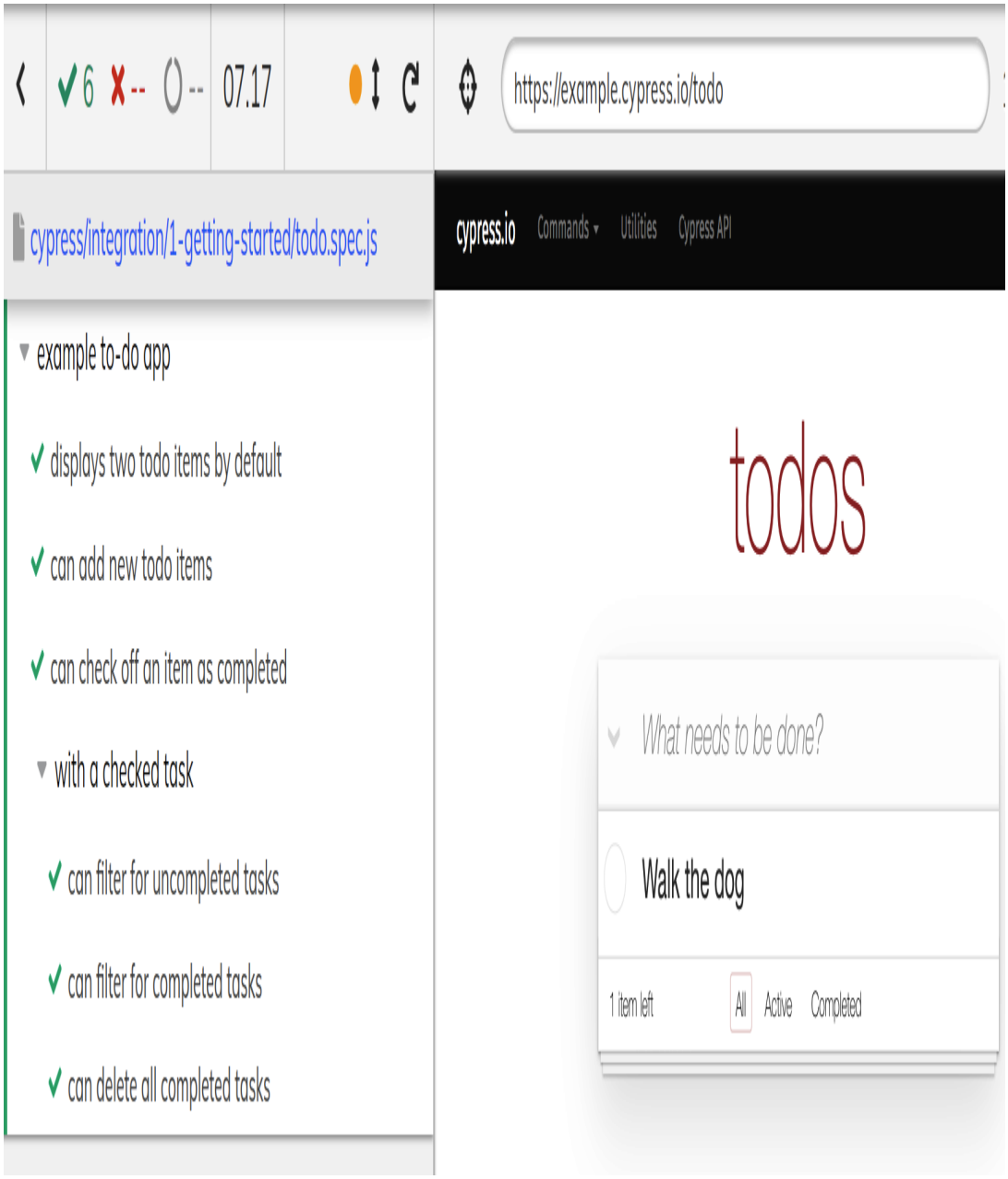


Figure 3-6. Cypress test run report

To further explore, click on one of the tests. You'll see a list of commands executed in that test, and when you hover over each command the application's state as it was when it executed the command will be displayed on the right, as seen in Figure 3-7. What more support could we ask for for debugging?

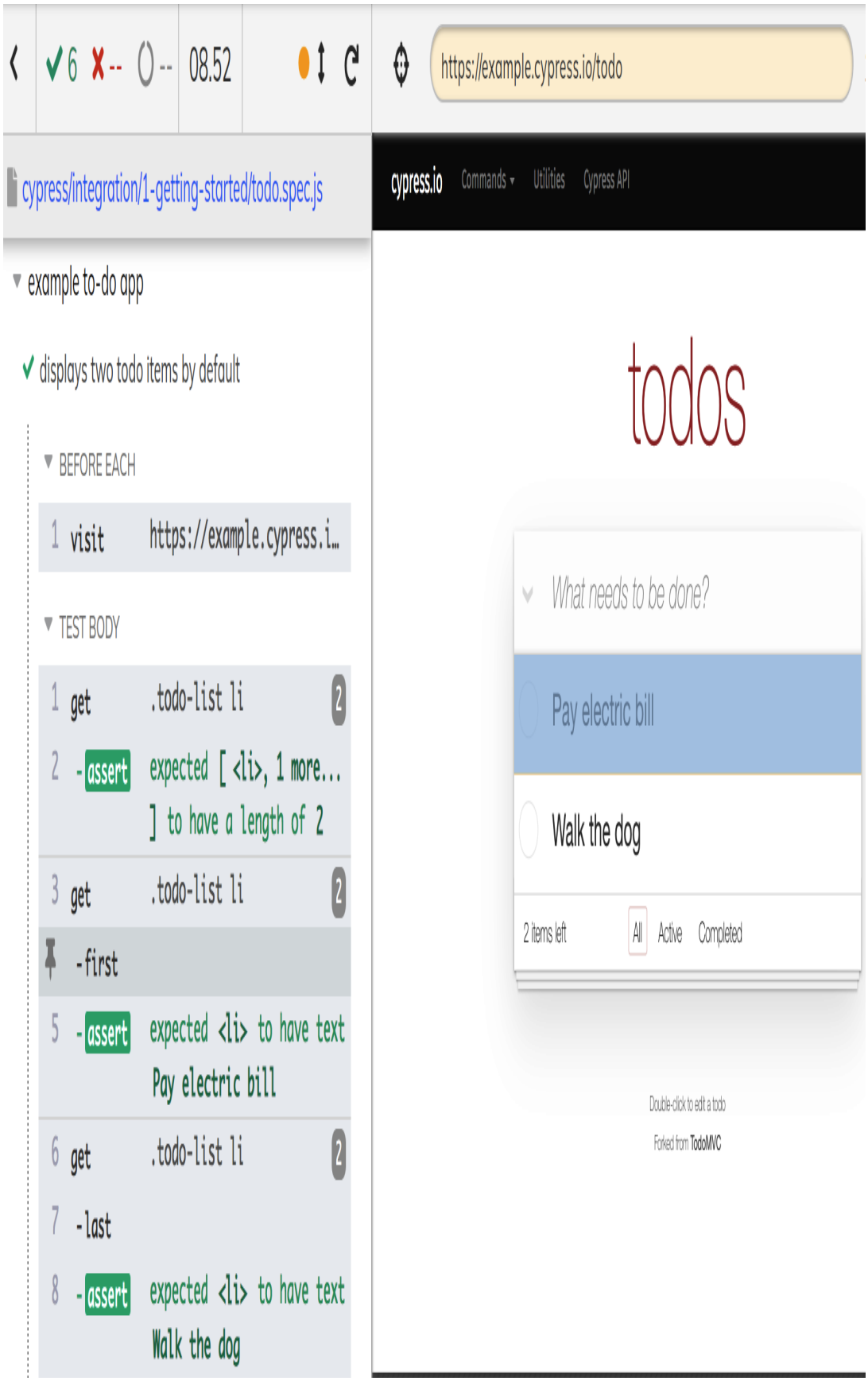

6
--
--
08.52
https://example.cypress.io/todo
cypress/integration/1-getting-started/todo.spec.js
cypress.io
Commands
Utilities
Cypress API
example to-do app
displays two todo items by default
BEFORE EACH
1 visit https://example.cypress.i…
TEST BODY
1 get .todo-list li 2
2 -assert expected [, 1 more...] to have a length of 2
3 get .todo-list li 2
-first
5 -assert expected <li> to have text Pay electric bill
6 get .todo-list li 2
7 -last
8 -assert expected <li> to have text Walk the dog
todos
What needs to be done?
Pay electric bill
Walk the dog
2 items left
All
Active
Completed
Double-click to edit a todo
Forked from TodoMVC

Figure 3-7. Debugging with Cypress

5. To run tests from the command line, add the following code to your *package.json* file and start the test run using the `npm test` command. You will notice the tests run in headless mode and a *videos* folder is created in your project directory with recordings of your test runs:

```
"scripts": {
    "test": "cypress run"
  }
```

Now that you've had an introduction to the way Cypress works, we'll briefly look at the methods it provides to interact with and navigate your web application. Open any test file, and you will notice some commonly used methods like the following:

- `get(element_locator)` gets the web element from the DOM after automatically waiting for it to be available. The Cypress application has a tool to inspect the element and get the locator, as seen in Figure 3-8. You can use this when creating your page modules.



Figure 3-8. Finding element locators using Cypress

- `get(element_locator)`.click() clicks on the chosen element.
- `title()` returns the page title.

- `get(select_locator).select(option)` selects options from a drop-down.
- `get(element_locator).rightclick`() performs a right-click on the chosen element.

Other advanced methods can be found in the tool's detailed documentation.

Setup and workflow

It takes just a few steps to create an automation framework using Cypress and the Page Object Model. You can create the same test as in the Selenium section to open an ecommerce application, log in, and assert on the home page title by following these steps:

1. Create a new tests folder under *cypress/integration*, say *ecommerce-e2e-tests*, and create a test file, say *login_tests.spec.js*, under it.
2. Now create a new */page-objects* folder outside the */integration* folder and create your page modules inside them: *login-page.js* and *home-page.js*, as seen in Example 3-13.
3. You can run the test either directly from the Cypress application or by using the `npm test` command.

Example 3-13. Cypress page object framework

```
// page-objects/login-page.js

/// <reference types="cypress" />

export class LoginPage {

    login(email, password){
        cy.get('[id=user_email]').type(email)
        cy.get('[id=user_password]').type(password)
        cy.get('.submitPara > .gr-button').click()
    }
}
```

```
// page-objects/home-page.js

/// <reference types="cypress" />

export class HomePage {

    getTitle(){
        return cy.title()
    }
}

// integration/eCommerce-e2e-tests/login_tests.spec.js

/// <reference types="cypress" />

import {LoginPage} from '../../page-objects/login-page'
import {HomePage} from '../../page-objects/home-page'

describe('example to-do app', () => {
    const loginPage = new LoginPage()
    const homePage = new HomePage()

    beforeEach(() => {
      cy.visit('https://example.com')
    })

    it('should log in and land on home page', () => {
       loginPage.login('example@gmail.com', 'Admin123')
       homePage.getTitle().should('have.string', 'Home
Page')
    })
})
```

Note the use of the `beforeEach()` method provided by the Mocha testing framework (similar to `@beforeMethod` in TestNG) to open the application URL before every test run and the `should('have.string', string)` assertion method from the Chai framework—these are bundled with Cypress by default.

Cypress runs your tests automatically each time you save new changes to them. This eases test creation, as you can quickly verify that your new code works as expected. You'll also see how to do visual testing with Cypress in Chapter 7. In summary, if you can

cross the hurdle of learning JavaScript (which is not that tough), you will benefit greatly from Cypress.

Service Tests

Now let's move on to service tests. In this section we will set up a test automation framework using the REST Assured Java library to validate a sample REST API. If you're new to APIs, refer to "API Testing" for an introduction.

Prerequisites

First, make sure you have the following prerequisites installed:

- The latest version of Java.
- The IDE of your choice—IntelliJ is a common choice for Java
- Maven

Java–REST Assured Framework

REST Assured is the go-to Java library for performing automated testing of REST APIs. It offers a domain-specific language (DSL) with Gherkin syntax (Given, When, Then) to create readable and maintainable API tests, and uses hamcrest matchers for assertions. REST Assured can work with any testing framework, like JUnit or TestNG.

Suppose we have the following GET `/items` API in our hypothetical order service, which returns a list of items and their details:

```
GET: https://eCommerce.com/items

Response:

Status Code: 200
[
    {
        "SKU": "984058981",
```

```
        "Color": "Green",
        "Size": "M"
    }
]
```

Then the REST Assured DSL to call the GET API and assert the status code will look like the following:

```
given().
        when().
        get("https://eCommerce.com/items").
        then().
        assertThat().statusCode(200);
```

Isn't that simple? Similarly, you have DSL for POST, PUT, and all other API testing–related methods.

Let's set up an API automation testing framework now and write a test to validate the same GET `/items` endpoint. You can bring that endpoint up as a stub on your machine by following the steps in Chapter 2.

> **TIP**
>
> If you need sample APIs to practice with, the Any API site has a consolidated list of 1,400 publicly hosted REST APIs to choose from.

Setup and workflow

As we saw in the UI automation framework setup, the three basic components of an automated testing framework are the dependency manager (Maven, in our case), a library to perform the required type of testing (REST Assured for APIs), and a testing framework to create and run the tests (we'll use TestNG). Create your framework by bringing those three components together as follows:

1. Create a new Maven project using IntelliJ (or your IDE of choice). Refer to "Java–Selenium WebDriver framework" for details.
2. Add the TestNG and REST Assured dependencies in your *pom.xml* file. You can find the required dependency parameters in the Maven Central repository, as discussed earlier.
3. Create a new package called `tests` under the */src/test/java* folder and a new test class called `ItemsTest`.
4. Example 3-14 shows a sample test for verifying the GET `/items` endpoint.

Example 3-14. The `ItemsTest` class with an API test for the GET `/items` endpoint

```
// ItemsTest.java

package apitests;

import org.testng.annotations.Test;

import static io.restassured.RestAssured.given;

public class ItemsTest {

    @Test
    public void
verifyGetItemsEndpointReturnsSuccessStatusCode(){
        given().
                when().
                get("http://localhost:1000/items").
                then().
                assertThat().statusCode(200);
    }
}
```

You can run the test either from the IDE or by running the `mvn clean test` command from your terminal.

Once you get the basic setup working, you can add a test to verify a POST `/items` endpoint. Let's say the POST endpoint takes the same item details as JSON in the request body and returns a 201 HTTP response on successfully adding the item to a new order. Create a stub on your machine, following the steps in Chapter 2.

To pass a JSON body to POST requests, a cleaner method is to create a `dataObject` class and serialize it using a JSON serialization library—for example, the `jackson-databind` library. Let's add that to our framework:

1. Add the `jackson-databind` library in your *pom.xml* file.
2. Now create a new `dataObjects` package under */src/main/java* and add a new `dataObject` class, say *ItemDetails.java*. Example 3-15 shows the `ItemDetails` class representing the JSON body for the POST request.

   Example 3-15. The `ItemDetails` class as a `dataObject`

```
// ItemDetails.java

package dataobjects;

import com.fasterxml.jackson.annotation.JsonProperty;
import
com.fasterxml.jackson.annotation.JsonPropertyOrder;

@JsonPropertyOrder({"sku", "color", "size"})
public class ItemDetails {

    private String sku;
    private String color;
    private String size;

    public ItemDetails(String sku, String color, String
size){
        this.sku = sku;
        this.color = color;
        this.size = size;
    }
```

```
        @JsonProperty("sku")
        public String getSku(){
            return sku;
        }

        @JsonProperty("color")
        public String getColor(){
            return color;
        }

        @JsonProperty("size")
        public String getSize(){
            return size;
        }
    }
```

Note how the `jackson-databind` library allows defining the expected JSON structure at the beginning with the `@JSONPropertyOrder` annotation.

3. In your test class, you can use the `ItemDetails` object as the POST request body. Example 3-16 shows the POST `/items` endpoint test.

Example 3-16. API test for the POST `/items` endpoint

```
@Test
    public void
verifyPostItemsEndpointReturnsSuccessStatusCode(){

        ItemDetails greenShirt = new ItemDetails("98765490",
"Green", "M");

        given().
                contentType(ContentType.JSON).
                body(greenShirt).
                log().body().
                when().
                    post("http://localhost:1000/items").
                then().
                assertThat().
                statusCode(200);
    }
```

When you run the test, the `log().body()` method will log the request body for you to check the serialization. We have only asserted on the `statusCode` of the response here. REST Assured offers more flexibility to find the required fields in the response body, as detailed in the official documentation, and assert them appropriately.

Unit Tests

Since unit tests are tightly integrated with the application code, the testing framework you use should be compatible with the application programming language, such as JUnit or TestNG for Java, NUnit for .NET, Jest or Mocha for JavaScript, RSpec for Ruby, and so on. We will look at the JUnit setup here. The prerequisites for JUnit are the same as mentioned for API tests.

> **NOTE**
>
> Although unit tests are written only by developers, it is important for testers to understand their basic structure so that they can plan the application's testing strategy wisely. The intention of including this as an exercise is to give that experience to testers, and hence this section will be kept simple.

JUnit

JUnit is a very popular unit testing framework created by Kent Beck and Erich Gamma in 1997. It has catered to the full spectrum of unit testing needs since then and continues to be the de facto unit testing framework for Java today. JUnit offers test creation, assertion, organization, running, and reporting capabilities. TestNG, another popular framework, was created to address some of the gaps in JUnit, but JUnit has upgraded its features in its latest editions to fill those gaps.

Some of the basic JUnit features are as follows:

- Test and lifecycle annotations such as `@Test` for marking the test methods, and `@BeforeEach`, `@BeforeAll`, `@AfterEach`, and `@AfterAll` for setup and teardown activities
- The `@DisplayName` annotation to show a readable name for each test
- Custom tagging annotations such as `@Tag("smoke")`, which can be used to run only a subset of tests when needed
- Assertion APIs such as `assertTrue()`, `assertEquals()`, `assertAll()`, and so on

Setup and workflow

Let's write a couple of simple unit tests for the customer service in our ecommerce application. Create a new Java project and add a `CustomerManagement` class, as seen in Example 3-17. This sample class has two methods that add and return customer details. We'll add unit tests for these two methods next.

Example 3-17. The `CustomerManagement` class

```
// CustomerManagement.java

package Customers;

import java.util.ArrayList;
import java.util.List;

public class CustomerManagement {

    private String firstName;
    private String lastName;
    private String age;

    private List<List<String>> customers = new
ArrayList<List<String>>();

    public List<List<String>> getCustomers(){
```

```
        return customers;
    }

 // if the customer name is empty, throw an exception; else
add the customer
    public void addCustomers(List<String> customerDetails){
        if (customerDetails.get(0).isEmpty())
            throw new IllegalArgumentException();
        customers.add(customerDetails);
    }
 }
```

To add the unit tests:

1. Add the following dependencies in your *pom.xml* file:

```
    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-api</artifactId>
            <version>5.7.2</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-engine</artifactId>
            <version>5.7.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

2. Create a new test class in the file *CustomerManagementTests.java*, under the */src/main/test* folder.
3. Use the JUnit annotations and assertions to create your tests, as seen in Example 3-18.

   Example 3-18. CustomerManagementTests.java with JUnit tests

```
package customersUnitTests;

import Customers.CustomerManagement;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

import java.util.ArrayList;
import java.util.List;

@DisplayName("When managing new customers")
public class CustomerManagementTests {

    @Test
    @DisplayName("should return empty when there are no
customers")
    public void
shouldReturnEmptyWhenThereAreNoCustomers(){
        CustomerManagement customer  = new
CustomerManagement();
        List<List<String>> customers =
customer.getCustomers();

        assertTrue(customers.isEmpty(), "Error: Customers
exists");
    }

    @Test
    @DisplayName("should throw exception when customer
name is invalid")
    public void shouldThrowExceptionForInvalidInput(){
        List<String> newCustomer = new ArrayList<>();
        newCustomer.add("");
        newCustomer.add("Jackson");
        newCustomer.add("20");

        CustomerManagement customer  = new
CustomerManagement();
        IllegalArgumentException err =
            assertThrows(IllegalArgumentException.class,
() ->
            customer.addCustomers(newCustomer));

    }
}
```

You can see the `@DisplayName` tag includes readable test descriptions. As these explain, the first test checks whether the `getCustomers()` method returns an empty value when there are no existing customers, and the second test asserts that the `addCustomers()` method returns an `IllegalArgumentException` when a customer with an empty first name is added. Also note the different assertion methods to assert exceptions and return values.

You can run these tests from the IDE or the command line, using the `mvn clean test` command. You'll see the test run results with their display names, as shown in Figure 3-9.

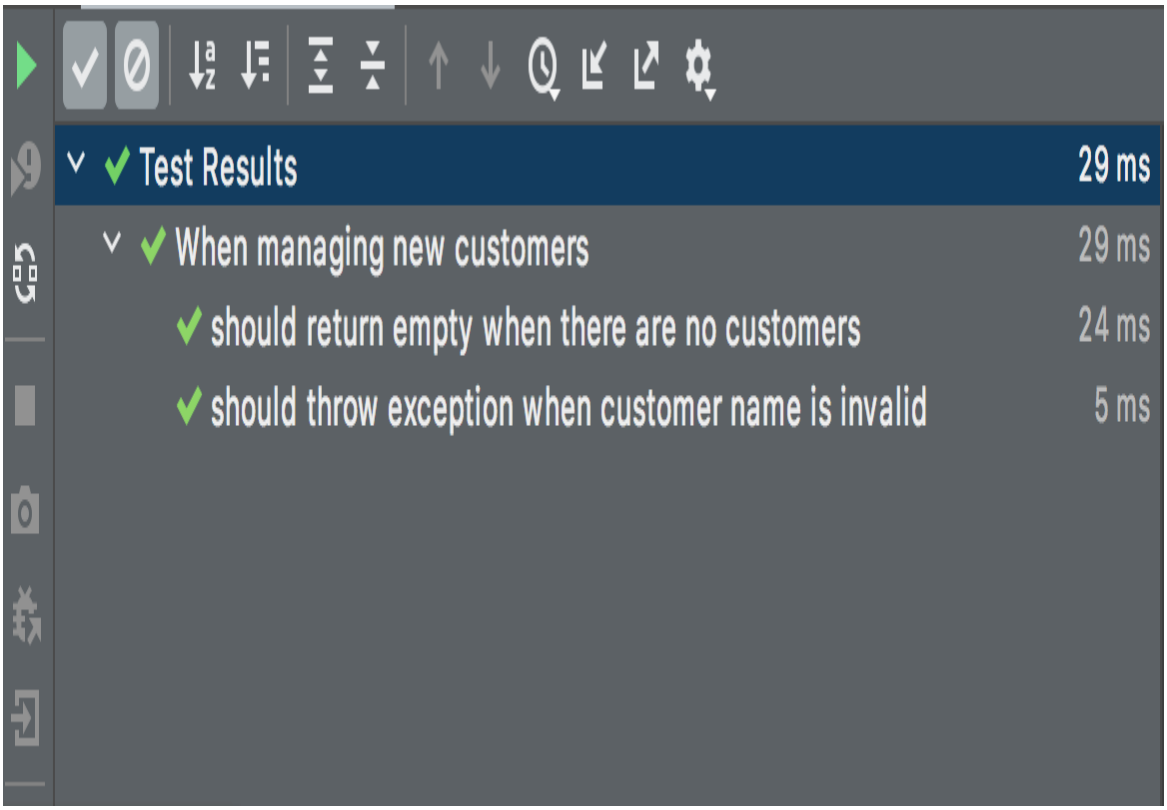


Figure 3-9. JUnit test run results in IntelliJ, which shows their readable display names

Apart from JUnit, depending upon the test case you are unit-testing, you might require additional capabilities from the application development framework (e.g., Spring Boot) and external libraries

such as Mockito for mocking the service call, `jackson-databinder` for data binding, and so on. When such additional capabilities are used to access an external system such as a database, the unit test becomes an integration test.

CHARACTERISTICS OF GOOD TESTS

The characteristics listed here apply to all the types of tests we've discussed up until now. Tests without these characteristics can easily turn into a maintenance hazard:

- Tests should be readable with proper method and variable names expressing the intent appropriately. Follow the Arrange, Act, and Assert (AAA) pattern, which suggests that you first arrange the prerequisites of the test case, then perform the actions required for the test case, and finally assert the expected behavior.
- Each test should verify only one behavior so that it is fast and expresses the right intentions when it fails.
- Tests should be independent of each other. Remember, chaining tests will also lead to chained errors. Having a proper setup and teardown for every test will help keep your tests independent and facilitate parallel execution.
- Tests should be environment-agnostic. For example, tests should not depend on static data in a particular environment.
- Automate the test building and running processes so that any team member can check out the code and run a single command to trigger the tests without having to manage the dependencies manually.

Additional Testing Tools

We will explore a few more test automation tools in this section: Pact, a contract testing tool; Karate, a BDD tool for creating service tests; and some of the AI/ML test automation tools that are currently booming in the market. This will give you a broader understanding of the tools in the functional test automation space and thereby help you make wise choices where necessary.

Pact

Pact is a popular tool for creating contract tests in Java. The tests can also be written in Python, JavaScript, Go, Scala, and other languages. Pact specifically is used for *consumer-driven contract testing*.

A *consumer* is an application (e.g., service or web UI) that receives information from another application (e.g., service or message queue). Obviously, the application that provides the required information is the *provider*. To give an example, the order service in the ecommerce application receives the vendor's item details from the PIM service, so this makes the order service a consumer and the PIM service the provider. Note that many other consumers could consume the PIM service apart from the order service. Also, each consumer may need different information from the PIM service. For example, the order service may require the SKU of each item as part of the item's details but may not use its manufacturer's address, which may be a requirement for another consumer.

Given that the requirements are consumer-driven, the PIM service may get pushed into a situation where it needs to change its contracts to cater to a new consumer or a new requirement, which will pose a risk to the order service and other consumer teams. They need a mechanism to continuously verify that the PIM service's contracts—especially the attributes relevant to them—are intact to avoid integration issues later. Testers or developers can write service or integration tests to mitigate the risk, but these tests might be brittle

and slow due to the dependencies on both applications, as well as being costly to set up and maintain. Tangentially, sometimes the provider and the consumer may be undergoing parallel development, which means you can't even write end-to-end integration or service tests. Consumer-driven contract tests then become key to sort out these entanglements.

As seen in Figure 3-10, with consumer-driven contract tests each consumer team writes tests against the stubbed version of the provider's agreed contracts. The tests specifically assert the attributes expected by that consumer and not the entire contract. These tests are then passed on to the provider's team, which runs all of them against the actual provider APIs, ensuring it serves its consumers' needs as expected. When deviations are found, the provider's team can at least caution the respective consumer to expect the change.

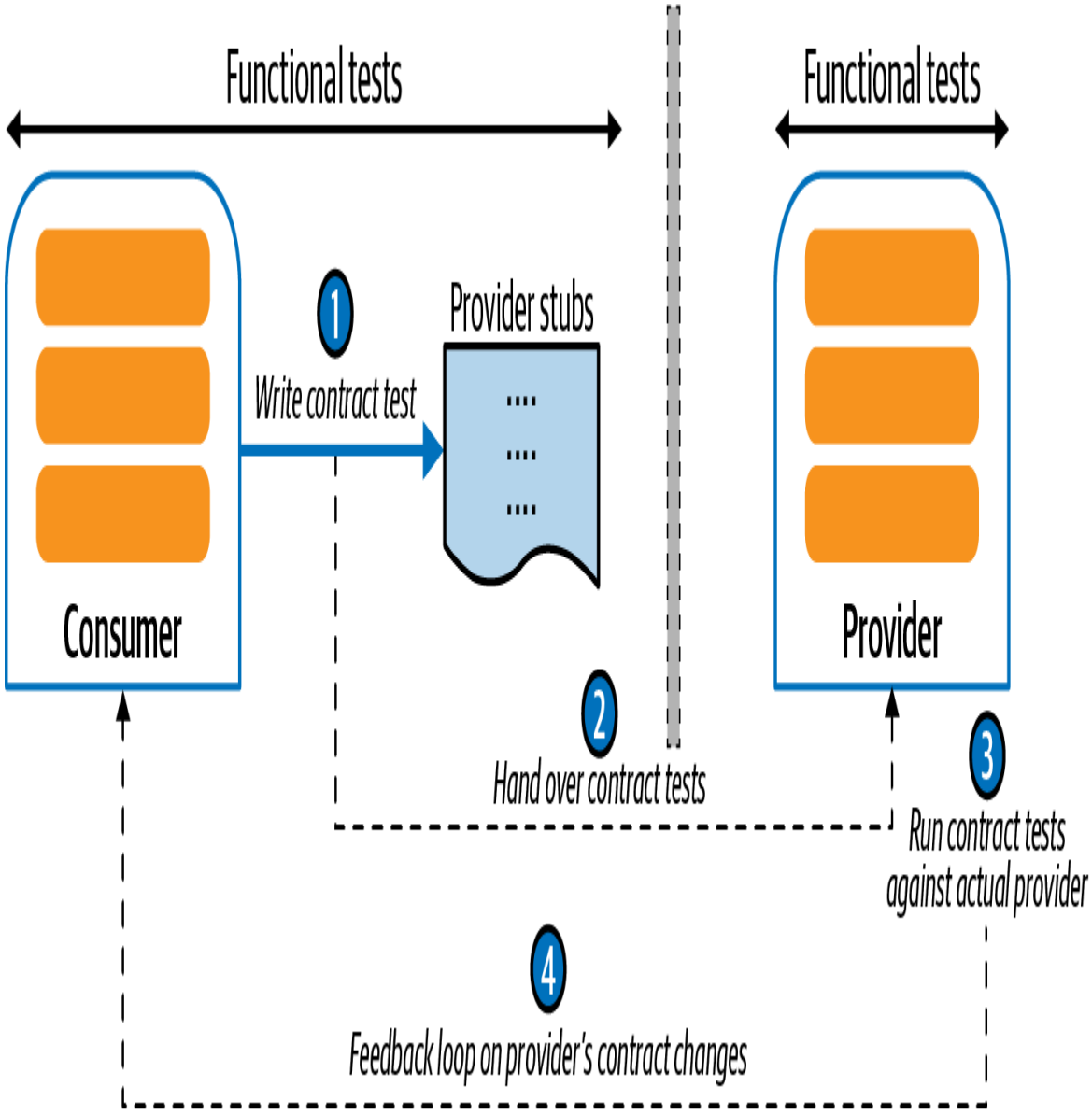


Figure 3-10. Consumer-driven contract testing flow

Basically, this type of contract testing splits the end-to-end integration testing into parts, as follows:

- Each consumer writes micro- and macro-level tests to validate its functional behavior by stubbing the provider, as seen in Figure 3-10.
- Each consumer also writes contract tests against the provider's stub, and the provider runs them continuously.

- The provider writes micro- and macro-level tests to validate its functional behavior.

This alleviates some of the pain points in writing end-to-end integration/service tests as the scope of the contract tests is smaller, and eliminates dependencies.

Pact enables the contract testing process to be fully automated. To get an idea of its workflow, we'll use the same order and PIM service example. Let's say the order service is integrating with the GET `/items` endpoint in the external PIM service in order to retrieve the item details, specifically the available sizes, SKU, and colors. The two teams' Pact workflow will be the following:

1. As the first step, the order service team collates all the integration test cases. For example, some of the integration test cases will be that the `/items` endpoint should return the item details as expected when the item exists, should return an empty array when the item doesn't exist, and should return appropriate error codes (404, 500, etc.) on invalid requests.
2. The order service team creates stubs for these test cases using Pact.
3. The order service team writes consumer contract tests using Pact against these stubs, asserting on the specific attributes: status codes, SKU, available sizes, and colors. These tests, when run, automatically produce a *pact file*. This file captures the different requests to the `/items` endpoint and assertions on the expected attributes in the responses.
4. The pact file is passed on to the PIM team automatically via an open source provision called the Pact Broker, which needs to be set up and maintained by both the consumer and provider teams. The Pact team also offers a paid service called Pactflow, which eliminates the need to set up and maintain the Pact

Broker. To make it simpler, the files can also be shared via folders.

5. On the PIM service side, the team writes a provider contract test to receive the pact file from the Pact Broker and set up the test data in different states as per the consumer tests' requirements. When the provider test runs, Pact will make appropriate requests as described in the pact file against the actual PIM service and verify the actual responses.
6. The provider test's results are available to the consumer via the Pact Broker, completing the full feedback loop without intervention.
7. Both the consumer and the provider's Pact tests are integrated to the CI pipeline so that the teams can receive feedback continuously.

Example 3-19 shows a sample Pact consumer test with a `pactMethod`. First, the `pactMethod` establishes the state of the `/items` endpoint as expected by the Pact consumer test. As you can see, the `given()` method describes this state and will be referred to by the provider test to initiate the appropriate test data setup. The Pact consumer test then brings up the `/items` stub as described by the `pactMethod` and asserts on the item details response.

Example 3-19. A sample consumer test using Pact

```
@ExtendWith(PactConsumerTestExt.class)
public class ItemsPactConsumerTest {

    @Pact(consumer = "Order service", provider =
"PIMService")
    RequestResponsePact
getAvailableItemDetails(PactDslWithProvider builder) {
        return builder.given("items are available")
            .uponReceiving("get item details")
            .method("GET")
            .path("/items")
```

```
            .willRespondWith()
            .status(200)
            .headers(Map.of("Content-Type",
"application/json; charset=utf-8"))
            .body(newJsonArrayMinLike(2, array ->
                array.object(object -> {
                    object.stringType("SKU", "A091897654");
                    object.stringType("Color", "Green");
                    object.stringType("Size", "S");
                })
            ).build())
            .toPact();
    }

    @Test
    @PactTestFor(pactMethod = "getAvailableItemDetails")
    void getItemDetailsWhenItemsAreAvailable(MockServer
mockServer) {

        // brings up the PIM /items endpoint stub as
described by
        // the pact method above
        RestTemplate restTemplate = new RestTemplateBuilder()
                .rootUri(mockServer.getUrl())
                .build();

        List<Item> items = new
PIMService(restTemplate).getAvailableItemDetails();

        Item item1 = new Item("A091897654","Green","S");
        Item item2 = new Item("A091897654","Green","S");
        List<Item> expectedItems = List.of(item1, item2);
        assertEquals(expectedItems, items);
    }
```

This test will generate a pact file and is shared with the provider via a folder. The pact provider test in Example 3-20 receives the pact file, does the test data setup per the `@State` annotated method, hits the actual `/items` endpoint as instructed by the pact file, and asserts that the actual response has the same item details format as in Example 3-19.

Example 3-20. A sample provider test using Pact

```
@Provider("PIMService")
@PactFolder("pacts")
@ExtendWith(SpringExtension.class)
@SpringBootTest(webEnvironment =
SpringBootTest.WebEnvironment.RANDOM_PORT)

public class ItemsPactProviderTest {

    @LocalServerPort
    int port;

    @MockBean
    private ItemRepository itemRepository;

    @BeforeEach
    void setUp(PactVerificationContext context) {
        context.setTarget(new HttpTestTarget("localhost",
port));
    }

    @TestTemplate

@ExtendWith(PactVerificationInvocationContextProvider.class)
    void verifyPact(PactVerificationContext context,
HttpRequest request) {
        context.verifyInteraction();
    }

    @State("items are available")
    void setItemsAvailableState() {
        when(itemRepository.getItems()).thenReturn(
            List.of(new Item("A091897654", "Green", "S"),
                new Item("A091897654","Green","S")));
    }
```

Pact generates HTML reports that can be integrated with CI. Pact tests are generally tightly tied to the application code, and knowledge of the application development frameworks in use (e.g., Spring Boot) may be required to create and debug them.

Karate

Karate grabs attention mainly because of its unique way of aiding service test creation. It offers predefined Gherkin statements (similar to Cucumber) to write tests, eliminating the need to code. The tool is not restricted to API testing—it aims to support end-to-end automated UI testing, contract testing, mock server setups, and so on—but it makes writing service tests much more straightforward than you might imagine. Example 3-21 shows the same test we used to assert on the GET `/items` endpoint with REST Assured written using Karate.

Example 3-21. A test for the GET `/items` endpoint using the Karate DSL

```
Feature: Order service should return item details

  Scenario: verify GET items endpoint
     Given url 'http://localhost:1000/items'
     When method get
     Then status 200
```

That's all there is to it—three lines of predefined Gherkin statements. You can find a full list of these statements on the Karate GitHub page. Installing the tool is as simple as importing a Maven archetype during project creation in IntelliJ.

AI/ML Tools in Automated Functional Testing

We've discussed quite a few tools in this chapter, and they suffice for your functional test automation needs at all the application layers. However, artificial intelligence and machine learning technologies have given rise to newer tools to provide improved assistance with some day-to-day test automation efforts, such as test authoring, test maintenance, test report analysis, and test governance. In this section, I'll provide a quick overview of the currently available tools.

Test authoring

AI/ML leaping to assist with test authoring is a significant milestone in the testing space, as it enables folks without coding skills to author UI-driven functional tests easily. Test.ai, Functionize, Appvance, Testim, and TestCraft are some of the paid tools that offer this functionality.

To author tests using these tools, you will have to navigate the user flow on the website manually, while the “ML-backed” recorder in the tool identifies the elements and actions performed at every step and creates the tests in the background. An advantage of the ML-backed recorder is that it identifies the elements not just by their locators, but also by their structural and visual aspects. Some of these tools also assist in test maintenance and root cause analysis, which significantly lightens the load in the test automation space. They can be plugged into CI too, to get continuous feedback.

Test maintenance

Have you ever faced a situation where there were a huge number of UI functional test failures due to a single element’s ID changing? Most often, only the ID of the element will have changed, and its functionality, look, and feel will have remained unaltered. Still, the UI functional tests will fail as they primarily rely on the element’s locator values. I certainly have, and I wondered if there might be tools that could autocorrect such small changes and save me time.

This autocorrection functionality, termed *self-healing*, is now available as part of AI/ML-powered test automation tools like test.ai and Functionize. As mentioned earlier, the ML-backed recorder captures the structural and visual aspects of UI elements along with their locators; when an element’s locator value changes, the element is still identified as the same, and the tools just get our approval to update the locator value in the test scripts.

Test report analysis

As mentioned earlier, I have seen large enterprise projects that had hundreds of UI-driven automated tests that ran all night, with a dedicated automation team analyzing the test results in the morning. The team would spend hours trying to figure out the root causes of test failures. Most often, they were one of the three things: defects, new feature changes, or environment issues. Once the root causes were found, the team made bug reports for defects, fixed the test scripts for new feature changes, and followed up with the infrastructure team to get the environment issues resolved. They were kept busy with such repetitive tasks every day. ReportPortal, an open source test report analysis tool, could have come in handy to them!

ReportPortal has an ML-based auto-analyzer functionality that reads the test failure logs and categorizes them into defects, test script issues, and environment issues. The ML algorithm learns from the log data of the previously analyzed test failures. This requires some preliminary manual effort to analyze the previous test failures and tag them appropriately. Once there is enough test failure analysis data, the auto-analyzer learns from it and starts identifying the test failures accurately, saving the team a lot of time.

Test governance

Proper test coverage in the appropriate application layers is a pertinent issue for all teams. A team might rejoice over a huge percentage of functional or unit test coverage, only to be ignorant of a module having no tests at all. Test governance is all about ensuring the right tests are at the right layers and injecting quality gates at every layer. This requires data from all layers, including the functionalities that are untested. SeaLights is an AI/ML-powered test governance tool for just this purpose: it presents metrics about test coverage across all layers, identifies the areas of code with poor test coverage, identifies quality risks by correlating the test execution data and test coverage, and provides many other quality governance–related features.

Such are the enhancements provided by AI/ML technologies in the test automation space. In summary, their assistance is starting to become significant, and they continue to evolve as well. Where possible, teams should leverage such tools to offload repetitive tasks so they can focus their brainpower on higher-order tasks like planning, innovation, security, performance, and so on.

Perspectives

We have delved deeply and broadly into the functional test automation space, but before closing the chapter, I would like to draw your attention to a few more key topics: antipatterns in automated functional testing, automation test coverage, and specifically what it means to have 100% automation coverage.

Antipatterns to Overcome

Even after you've spent heaps of time and effort crafting the right automated functional testing strategy and implementing the testing frameworks in the right layers, it is essential to realize that your automated functional testing efforts have just begun. Throughout the delivery timeline, you should continue to watch for antipatterns in automated functional testing as the team progresses with developing more and more tests. In my observation, it is easy to fall prey to these antipatterns with the delivery buzz, and thus being watchful for the early symptoms becomes crucial. In this section we'll discuss a few common antipatterns—the ice cream cone and the cupcake, as seen in Figure 3-11—along with their symptoms and tips to overcome them.

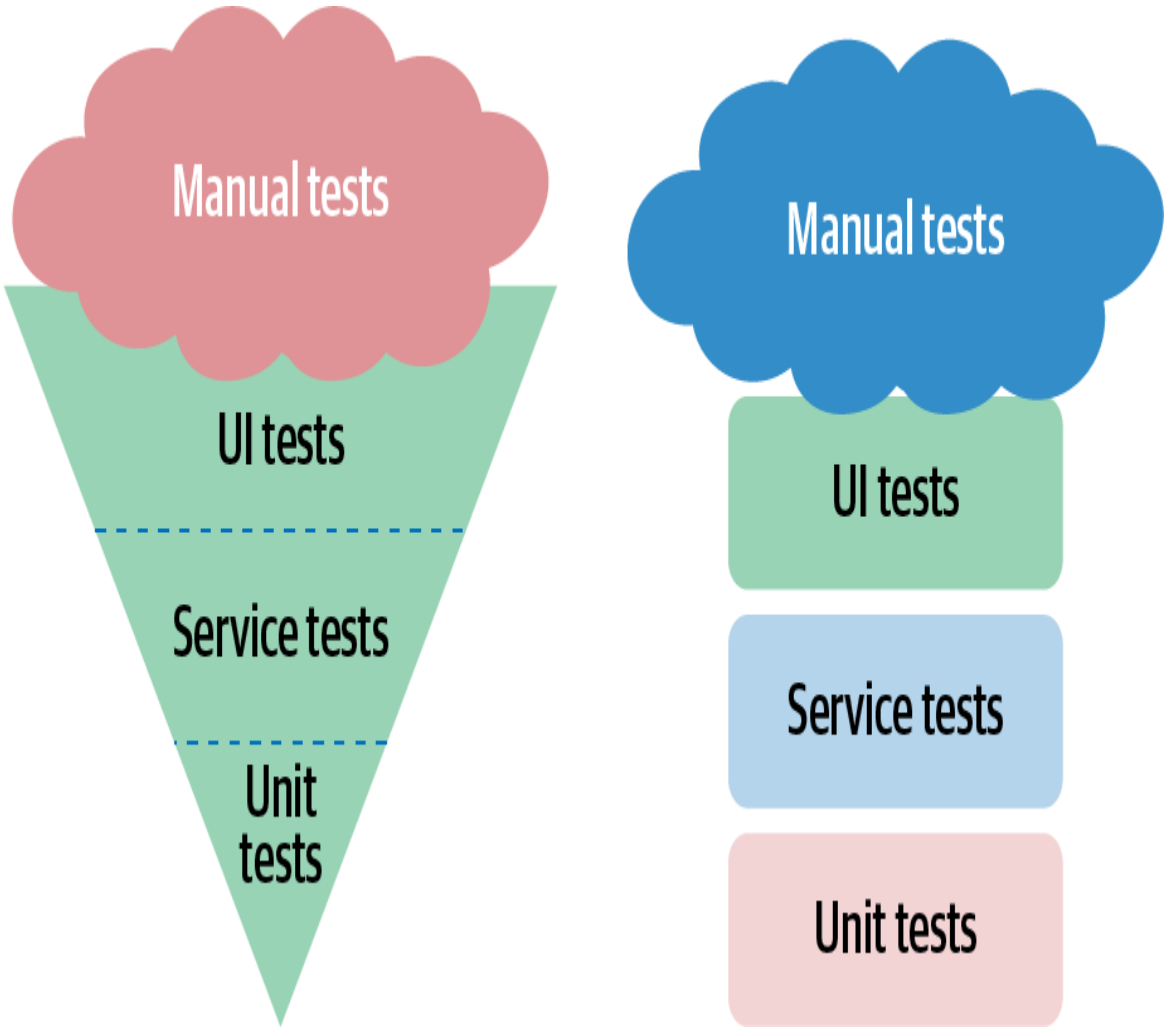


Figure 3-11. Antipatterns in automated functional testing

The ice cream cone

When you invert the test pyramid, it looks like a cone. This is referred to as the ice cream cone antipattern, where there are more macro-level UI-driven tests and very few micro-level tests. You can sense the ice cream cone antipattern when you observe some of these symptoms in the project:

- Waiting for a long period to get feedback from the tests run
- Catching defects later in the cycle, sometimes only during the release testing stage

- Elaborate manual testing required to give feedback despite having automated tests
- Frustration in the team with the automated tests as the diligent efforts in automating the UI flows have not been fruitful in giving the right results

> **TIP**
>
> The earliest sign at which you can prevent your team from drifting steeply toward this antipattern is when you find regression defects during manual story testing. Do a root cause analysis immediately, and fix your team practices early.

The cupcake

When you duplicate tests in multiple layers, instead of a test pyramid you wind up with a wide bottom layer, a wide middle, and an even wider top—overall, it looks like a cupcake. This kind of disorganization generally happens when you have siloed teams of developers and automated testers. For example, the developers will have added unit tests to verify all the invalid login inputs, and the testers will add the same tests in the UI layer.

You can sense this antipattern when your team takes a long time to release even a tiny feature. Also, you might notice blame games, as one role will expect the other role to have added appropriate tests whenever there is a bug.

TIP

A simple way to avoid this antipattern is to have a short discussion among the relevant roles in a team to determine which tests are expected to be written in each layer. The right avenue for such a discussion could be the user story kickoff meeting, which should be followed by documenting the results of the discussion in the user story cards.

100% Automation Coverage!

Teams usually track the automation coverage percentage as a metric, and a high percentage is often considered a validation of their good software development practices. The automation coverage percentage is calculated by capturing all the application test cases, marking them as automated or not, and using simple mathematics to derive a percentage. Teams often set themselves the goal of achieving 100% automation coverage, with good intentions—but when doing so, it’s important to keep a few pointers in mind.

CODE COVERAGE AND MUTATION TESTING

The traditional code coverage metric is different from the automation test coverage metric. Code coverage tells you whether there are lines of code that will not be executed by the existing unit tests. In other words, it identifies untested lines of code. Code coverage tools like JaCoCo and Cobertura can be integrated in the CI build pipeline and cause the build to fail when the code coverage percentage is below a certain threshold, to prevent the untested code from percolating any further than the build stage. However, high code coverage doesn't necessarily indicate that all the test cases are automated.

To find the missed test cases in unit testing, a technique called *mutation testing* is employed. Mutation testing changes the application's code and checks if the tests fail. For example, when there are void method calls, it removes the calls in the code and runs the unit tests again. The mutation is said to be "killed" if the tests fail and to have "survived" if not. PIT is a popular mutation testing tool that can be added as a Maven dependency and executed from the command line. It lists the test cases that survived along with an overall mutation score for the application. Mutation testing, though very effective, is time-consuming; hence, it has to be used wisely.

The first point I'd like to make about the automation coverage percentage is that even if you have 100% coverage, that doesn't guarantee a bug-free application! The percentage is simply a measure of how many *known* test cases are automated—you will probably discover presently unknown cases later. It's important to point this out to business stakeholders and your team, as otherwise it may lead them to question the reliability of the automation test suite and the value of the effort spent on it when a critical bug is found. It is also crucial to make them understand that the expected

outcome from tracking this metric is to disclose the automation backlog (ideally, there won't be one) and plan capacity in the upcoming iterations to complete these tasks. You can also use the tracking wisely to observe whether your team is drifting toward one of the antipatterns mentioned in the previous section.

The second pointer is that when you are tracking automation coverage, you should observe whether all the areas of the application have automation coverage. Especially when you are developing large-scale applications with different teams working on various components, your coverage percentage may still be high (say, >80%) even if one module has zero tests, as long as the other modules have high test coverage percentages.

The penultimate pointer on 100% automation coverage is that you should include both functional and cross-functional test cases while calculating this metric. Most often, cross-functional test cases don't contribute to the percentage, resulting in bugs later (you will learn more about cross-functional test case automation in the upcoming chapters).

And finally, while you should aim to automate all the test cases, depending upon the nature of the application, environments, automation costs, etc., it may be impossible to achieve 100% automation coverage. In such cases, you should track the non-automated test cases properly and add them to your manual testing list. That said, you should not have a tall manual test case list—you want to avoid the 1,200 minutes of release testing I warned about at the beginning of the chapter!

The greatest benefits of all this meticulous tracking and ensuring proper automation coverage will start to show as the project grows, especially when it extends over a few years. As they say, code outlives people, and often the automated tests end up being the only trustable living documentation of the application's functionalities. Thus, your efforts in writing good automated tests will prove to be a

worthwhile investment, not only for the project but for you and your future teammates.

Key Takeaways

Here are the key takeaways from this chapter:

- Automated testing is the practice of using tools to verify the expected behavior of the application in order to receive fast feedback during software development.
- A wise way to balance the testing capacity in a project is to perform manual exploratory testing to find new test cases and automate them to aid in regression testing.
- When it comes to automated functional testing, its scope expands beyond just the commonly adopted UI-driven functional tests. Unit, integration, contract, service, UI functional, and end-to-end tests are the different micro- and macro-level test types that, when woven together appropriately, provide swift feedback.
- The test pyramid is the ideal goal while crafting your automated functional testing strategy. Adding a broad base of micro-level tests and gradually decreasing the number of macro-level tests as their scope widens is the best way to reduce test creation and running time.
- Several tools, including AI/ML tools, have evolved to ease automated functional test authoring, maintenance, and analysis efforts.
- Although you may have put a lot of effort into creating your automation frameworks in different layers, the work doesn’t stop there. You need to keep watching for signs of antipatterns, like the ice cream cone and the cupcake.

- It is vital to track automation coverage to ensure the automation efforts are not sidelined amidst the delivery buzz. Also, be aware that a high automation coverage percentage can lead your team into a false sense of security; it’s important to look beyond the number to ensure you have coverage in all areas of the application.

Chapter 4. Continuous Testing

Your fast feedback efforts are in limbo without continuous feedback!

In the previous chapter, we discussed how adding tests in the right layers of the application accelerates feedback cycles. It is imperative to receive such fast feedback *continuously* and not just in random bursts to seamlessly regulate the application's quality throughout the development cycle. This chapter is dedicated to the elaboration of such a continuous testing practice.

Continuous testing (CT) is the process of validating application quality using both manual and automated testing methods after every incremental change, and alerting the team when the change causes deviation from the intended quality outcomes. For example, when a piece of functionality deviates from the expected application performance numbers, the CT process immediately notifies the team by means of failing performance tests. This gives the team an opportunity to fix issues as early as possible, when they are still relatively small and manageable. A lack of such a continuous feedback loop might leave the issues unnoticed for an extended period, allowing them to cascade to deeper levels of the code over time and increasing the effort required to remedy them.

The CT process relies heavily on the practice of *continuous integration* (CI) to perform automated testing against every change. Adopting CI together with CT allows the team to do *continuous delivery* (CD). Ultimately, the trio of CI, CD, and CT make the team a high-performing one, as measured by the four key metrics, *lead time, deployment frequency, mean time to restore,* and c*hange fail percentage*. These metrics, which we'll look at toward the end of this chapter, provide insights about the quality of the team's delivery practices.

This chapter will equip you with the skills required to establish a CT process for your team. You will learn about CI/CD/CT processes and strategies to achieve multiple feedback loops on various quality dimensions. A guided exercise to set up a CI server and integrate the automated tests is included too.

Building Blocks

As a foundation for the continuous testing skill, this section will introduce you to the terminology and the overall CI/CD/CT process. You will also learn the fundamental principles and etiquette that should be carefully imbued within the team to make the process successful. Let's begin with an introduction to CI.

Introduction to Continuous Integration

Martin Fowler, author of a half dozen books including *Refactoring: Improving the Design of Existing Code* (Addison Wesley) and Chief Scientist at Thoughtworks, describes continuous integration as "a software development practice where members of a team integrate their work frequently, usually each person integrates at least daily—leading to multiple integrations per day." Let's consider an example to illustrate the benefits of following such a practice.

Two teammates, Allie and Bob, independently started developing a login and home page. Work started in the morning, and by noon Allie had finished a basic login flow and Bob had completed a basic home page structure. They both tested their respective functionalities on their local machines and continued work. By the end of the day, Allie had completed the login functionality by making the application land on an empty home page after successful login, since the home page wasn't available to her yet. Similarly, Bob completed the home page functionality by hardcoding the username in the welcome message, since the user information from the login was not available to him.

The following day, they both reported their functionalities to be “done”! But are they really done? Which of the two developers is responsible for integrating the pages? Should they create a separate integration user story for every integration scenario across the application? If so, will they be ready for the expense of the duplicated testing efforts involved in testing the integration story? Or should they delay testing until the integration is done? These are the kinds of questions that get addressed implicitly with continuous integration.

When CI is followed, Allie and Bob will share their work progress throughout the day (after all, both had a basic skeleton of their functionalities ready by noon). Bob will be able to add the necessary integration code to abstract the username after login (e.g., from a JSON or JWT token), and Allie will be able to make the application land on the actual home page after successful login. The application will really be usable and testable then!

It may seem like a small additional cost to integrate the two pages the next day in this example. However, when code is accrued and integrated later in the development cycle, integration testing becomes costly and time-consuming. Furthermore, the more testing is delayed, the more likely they are to find entangled issues that are hard to fix, sometimes even warranting a rewrite of a major chunk of the software. This subsequently will create a general fear of integration among team members—often an unspoken accompaniment of delayed integration!

The practice of continuous integration essentially tries to reduce such integration risks and save the team from ad hoc rewriting and patches. It doesn’t entirely eliminate integration defects, but makes it easier to find and fix them early, when they are just budding.

The CI/CT/CD Process

Let’s start by looking at the continuous integration and testing processes in detail. Later, we’ll see how they connect to form the continuous delivery process.

The CI/CT process relies on four individual components:

- The *version control system* (VCS), which holds the entire application code base and serves as a central repository from which all team members can pull the latest version of the code and where they can integrate their work continuously
- The automated functional and cross-functional tests that validate the application
- The CI server, which automatically executes the automated tests against the latest version of the application code for every additional change
- The infrastructure that hosts the CI server and the application

The continuous integration and testing workflow begins with the developer, who, as soon as they finish a small portion of functionality, pushes their changes into a common version control system (e.g., Git, SVN). The VCS tracks every change submitted to it. The changes are then sent through the continuous testing process, where the application code is fully built and automated tests are executed against it by a CI server (e.g., Jenkins, GoCD). When all the tests pass, the new changes are considered fully integrated. When there are failures, the respective code’s owner fixes the issues as quickly as possible. Sometimes, changes are reverted back from the VCS until the issues are resolved. This is mainly to prevent others from pulling the code with issues and integrating their work on top of it.

BENEFITS OF VCSS

Ever think about how teams shared their code before VCSs? Some teams used shared drives, and others directly patched their code to a central server that hosted the entire code base! Such was the pain that led to the development of the first ever VCS in the 1960s, called the Source Code Control System (SCCS). Since then, VCSs have gotten richer, with new features that took away a lot of pain points and offered tremendous benefits for work integration.

A few significant benefits are as follows:

- A VCS keeps track of every version of code pushed into it, be it an addition, deletion, or modification of code, in a separate database. This serves as a long-term history of changes and hence significantly eases root cause analysis of issues.
- Since the versions are maintained independently, a VCS allows teams to roll back to a previously working version of the application when there are issues.
- Changes in the VCS can be tied to a user story or a defect card. This gives the team the ability to trace the changes back to a user story and understand the context behind the code written and the evolution of a feature over time.
- Sometimes, teams may have to work on a common area of code to build their features. A VCS allows team members to create branches of the main code base, build on top of them, and merge them into the main code base a little later. However, a long-living feature branch is an antipattern.

As Figure 4-1 shows, Allie pushes her code for the basic login functionality along with the login tests into the common version

control system before noon, as part of commit C_n.

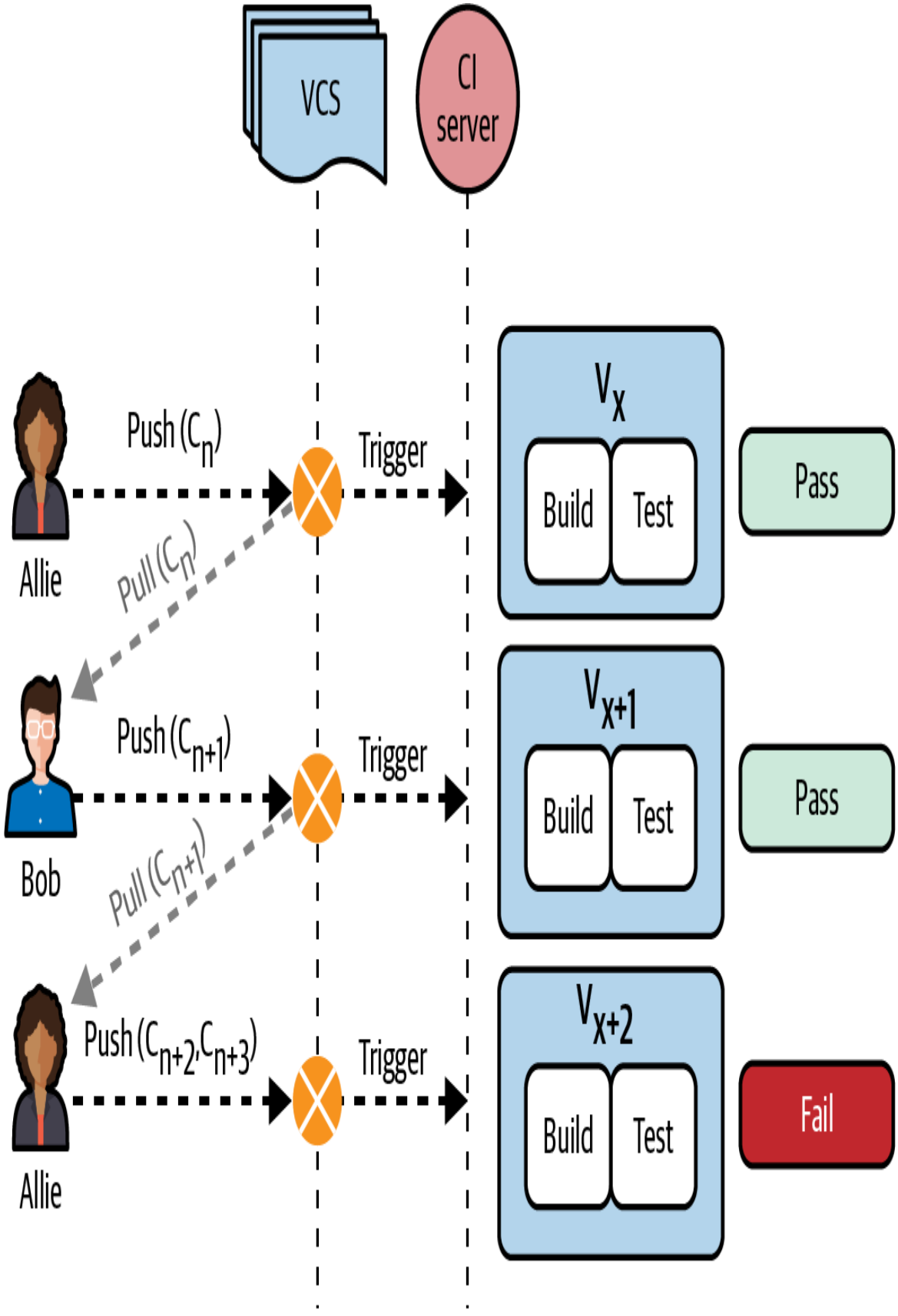

VCS
CI server
Push (C_n)
Trigger
Allie
Pull (C_n)
V_x
Build
Test
Pass
Push (C_{n+1})
Trigger
Bob
Pull (C_{n+1})
V_{x+1}
Build
Test
Pass
Push (C_{n+2},C_{n+3})
Trigger
Allie
V_{x+2}
Build
Test
Fail

Figure 4-1. Components in a continuous integration and testing process

> **NOTE**
>
> In the Git VCS, a *commit* is a snapshot of the entire code base at a given point in time. When practicing continuous integration, it is recommended that small incremental changes are saved as independent commits on the local machine. When the functionality reaches a logical state, such as completing a basic login functionality, the commits should be pushed to the common VCS repository. Only on pushing the changes to the VCS do the CI and testing processes begin.

The new change, C_n, triggers a separate pipeline in the CI server. Each pipeline is composed of many sequential stages. The first is the *build and test* stage, which builds the application and runs automated tests against it. These include all the micro- and macro-level tests discussed in Chapter 3 and tests that assert on the application's quality dimensions (performance, security, etc.), which we will discuss in the upcoming chapters. Once this stage is complete, the test results are indicated to Allie. In this case, Allie's code has been successfully integrated, and she proceeds with her login functionality.

Later in the day, Bob pushes commit C_{n+1} for the home page feature after pulling the latest changes (C_n) from the common VCS. C_{n+1} is thus a snapshot of the application code base including both Allie's and Bob's new changes. This triggers the build and test stage in the CI process. The tests when run against C_{n+1} ensure that Bob's new changes haven't broken any of the previous functionalities, including Allie's latest commit, as she has also added the login tests. Luckily, Bob hasn't. However, we see in Figure 4-1 that Allie's changes as part of commits C_{n+2} and C_{n+3} have broken the integration, and the tests have failed. She needs to fix them before proceeding any further with her work, as she has introduced a bug into the common

VCS. She can push her fix as another commit, and the process will continue.

Imagine the same workflow in a large distributed team, and you can understand how much easier CI makes it for all team members to share their progress and integrate their work seamlessly. Also, in large-scale applications, there are typically several interdependent components that warrant exhaustive integration testing, and the continuous testing process provides the much-needed confidence in the finesse of their integration!

With that kind of confidence gained from the fully automated integration and testing processes, the team is placed in a privileged spot to push their code to production whenever the business demands it. In other words, the team is equipped to do continuous delivery.

Continuous delivery depends upon following continuous integration and testing processes so that the application is production-ready at all times. Additionally, it dictates having an automated deployment mechanism that can be triggered with a single click to deploy to any environment, be it QA or production. Figure 4-2 shows the continuous delivery process.

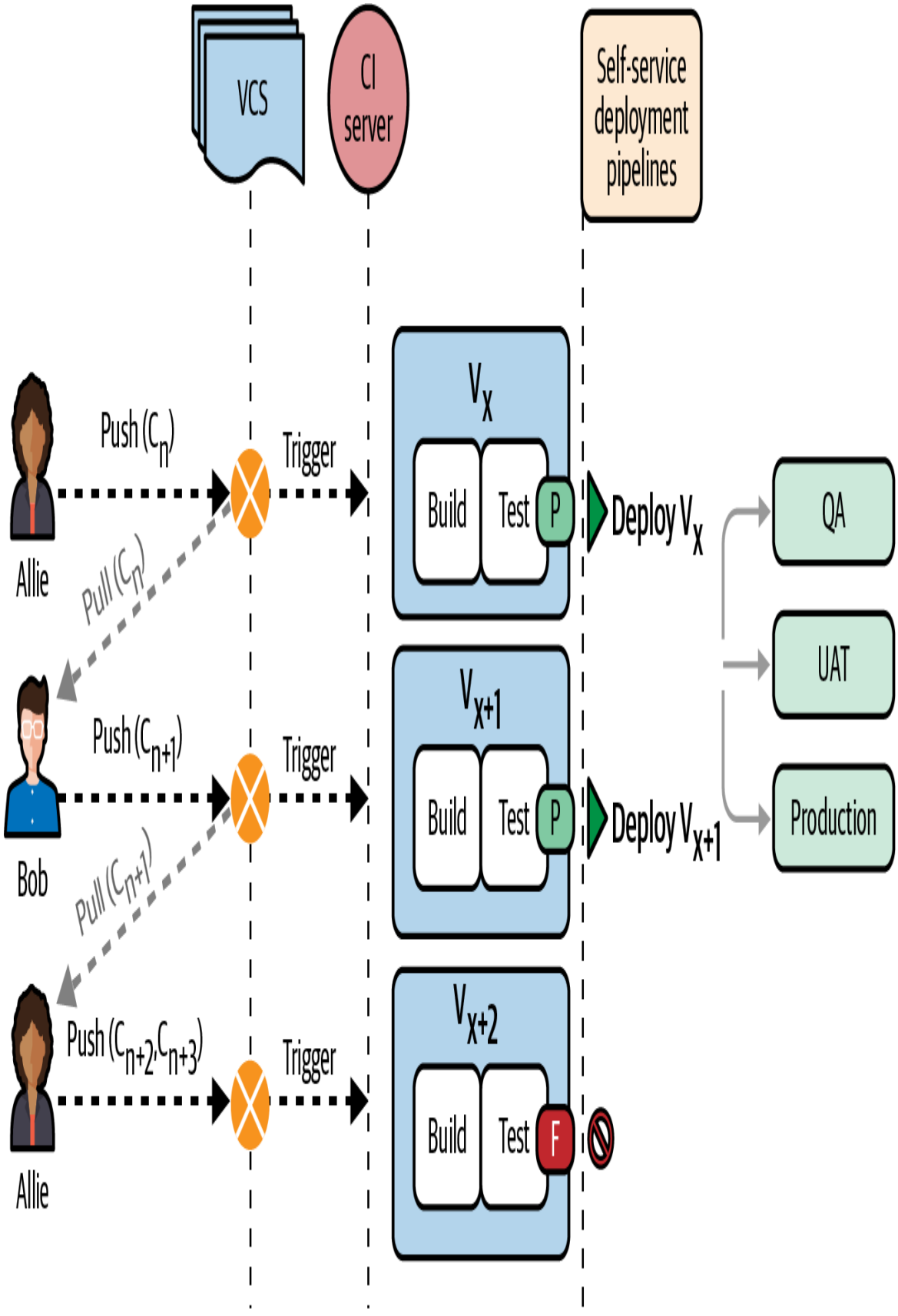

VCS
CI server
Self-service deployment pipelines
Allie
Push (C_n)
Pull (C_n)
Trigger
V_x
Build
Test
P
Deploy V_x
QA
UAT
Production
Bob
Push (C_{n+1})
Pull (C_{n+1})
Trigger
V_{x+1}
Build
Test
P
Deploy V_{x+1}
Allie
Push (C_{n+2},C_{n+3})
Trigger
V_{x+2}
Build
Test
F

Figure 4-2. Continuous delivery process with CI, CT, and deployment pipelines

As you can see, the continuous delivery process encompasses the CI/CT processes along with the self-service deployment pipelines. These pipelines are stages configured in the CI server as well; they perform the task of deploying the "chosen" version of the application artifacts to the required environment.

The CI server lists all the commits with their test results status. Only if all the tests have passed for a commit (or a set of commits) does it offer the option to deploy that particular application version (V). For example, let's say Allie's team wants to receive feedback from the business on the basic login functionality pushed as part of commit C_n. They can push the Deploy V_x button, as seen in Figure 4-2, and choose the user acceptance testing (UAT) environment. This will deploy only the changes made up to that point to the UAT environment—that is, Bob's C_{n+1} and later commits will not be deployed. As you can see, the commits C_{n+2} and C_{n+3} are not available for deployment as the tests have failed.

This kind of continuous delivery setup solves many critical issues, but one of the most important benefits it provides is the ability to launch product features to the market at the right time. Often delays in feature releases result in loss of revenue and loss of customers to competitors. Additionally, from the team's point of view, the deployment process becomes fully automated, reducing the dependency on certain individuals to do their magic on the day of deployment; anyone is free to make a hassle-free deployment to any environment at any time. Automating deployments also reduces the risk of incompatible libraries, missing or incorrect configurations, and insufficient documentation.

CONTINUOUS DEPLOYMENT VERSUS CONTINUOUS DELIVERY

Continuous deployment is different from continuous delivery. Continuous deployment involves having automated deployment pipelines that push every commit to production automatically after the continuous testing process. In other words, the feature you committed just now is available to real end users in production immediately. In contrast, practicing continuous delivery is about being ready at any time to push the application to production with a self-service deployment option. Continuous delivery is suitable in cases where businesses have set launch dates for features. Sometimes, companies even make public announcements on feature inauguration.

Principles and Etiquette

Now that we have discussed the CI/CD/CT processes, it is important to call out that these processes can reach fruition only if all the team members follow a set of well-defined principles and etiquette. After all, it is an automated way to collaborate on their work—be it automated tests, application code, or infrastructure configurations. The team should establish these principles at the beginning of their delivery cycle and keep reinforcing them throughout. Here is a minimum set of principles and etiquette a team will have to respect to be successful:

Do frequent code commits

> Team members should make frequent code commits and push them to the VCS as soon as they finish each small piece of functionality, so that it is tested and made available for others to build on top of it.

Always commit self-tested code

Whenever a new piece of code is committed, it should be accompanied by automated tests in the same commit. Martin Fowler calls this practice *self-testing code*. For example, as we saw earlier, Allie committed her login functionality along with login tests. This ensured that her commit was not broken when Bob committed his code next.

Adhere to the Continuous Integration Certification Test

Each team member should ensure their commit passes the continuous testing process before moving on to the next set of tasks. If the tests fail, they need to fix them immediately. According to Martin Fowler's Continuous Integration Certification Test, a broken build and test stage should be repaired within 10 minutes. If this is not possible, the broken commit should be reverted, leaving the code stable (or green).

Do not ignore/comment out the failing tests

In the rush to make the build and test stage pass, team members should not comment out and ignore the failing tests. As evident as the reasons for why this should not be done are, it's a common practice.

Do not push to a broken build

The team should not push their code when the build and test stage is broken (or red). Pushing work on top of an already broken code base will lead to tests failing again. This will further burden the team with the additional task of finding which changes originally broke the build.

Take ownership of all failures

When tests fail in an area of code that someone didn't work on, but it fails because of their changes, the responsibility of fixing the build is still on them. If necessary, they can pair with someone

who has the required knowledge to fix it, but ultimately getting it fixed before moving on to their next task is a fundamental requisite. This practice is essential because often the responsibility of fixing the failed tests is tossed around, causing a delay in resolving the issues. Sometimes the tests are eliminated from running in the CI for days as the issue is not fixed. This results in the continuous testing process giving incomplete or false feedback for the changes pushed during that open window.

Many teams also adopt stricter practices for their own benefit, such as mandating that all the micro- and macro-level tests pass on local machines before pushing the commit to the VCS, failing the build and test stage if a commit does not meet the code coverage threshold, publishing the commit's status (pass or fail) with the name of the individual who made the commit to everyone on a communication channel such as Slack, playing loud music in the team area whenever a build is broken from a dedicated CI monitor, and so on. Also, as a tester on the team, I keep an eye on the status of the tests in the CI and see to it that they get fixed on time. Fundamentally, all these measures are taken to streamline the team's practices around CI/CT processes and thereby yield the right benefits—although the foremost measure that always seems to work best is empowering the team with knowledge of not only the "how" but also the "why" behind the process!

Continuous Testing Strategy

Now that you know the processes and principles, the next step is to create and apply strategies custom to your project needs.

In the previous section, the continuous testing process was demonstrated with a single build and test stage that runs all the tests and gives feedback in a single loop. You can also accelerate the feedback cycle with two independent feedback loops: one that runs

the tests against the static application code (e.g., all the micro-level tests), and the other that runs the macro-level tests against the deployed application. This, in a way, is a slight shift left where we leverage the micro-level (unit, integration, contract) tests' ability to run faster than the macro-level (API, UI, end-to-end) tests to get faster feedback.

Figure 4-3 shows a CT process with two stages. As you can see here, a common practice is to combine the application compilation with the micro-level tests as a single stage in CI. This is traditionally called the *build and test* stage. When the team adheres to the test pyramid, like we discussed in Chapter 3, the micro-level tests will end up validating a broad range of application functionalities. As a result, this stage helps them get extensive feedback on the commit quickly. The build and test stage should be swift enough to finish execution within a few minutes so that, per the recommended principles and etiquette, the team will wait for it to complete before moving on to the next task. If it takes longer, the team should find ways to improve it—for example, parallelizing the build and test stage for each component instead of having a single stage for the entire code base.[1]

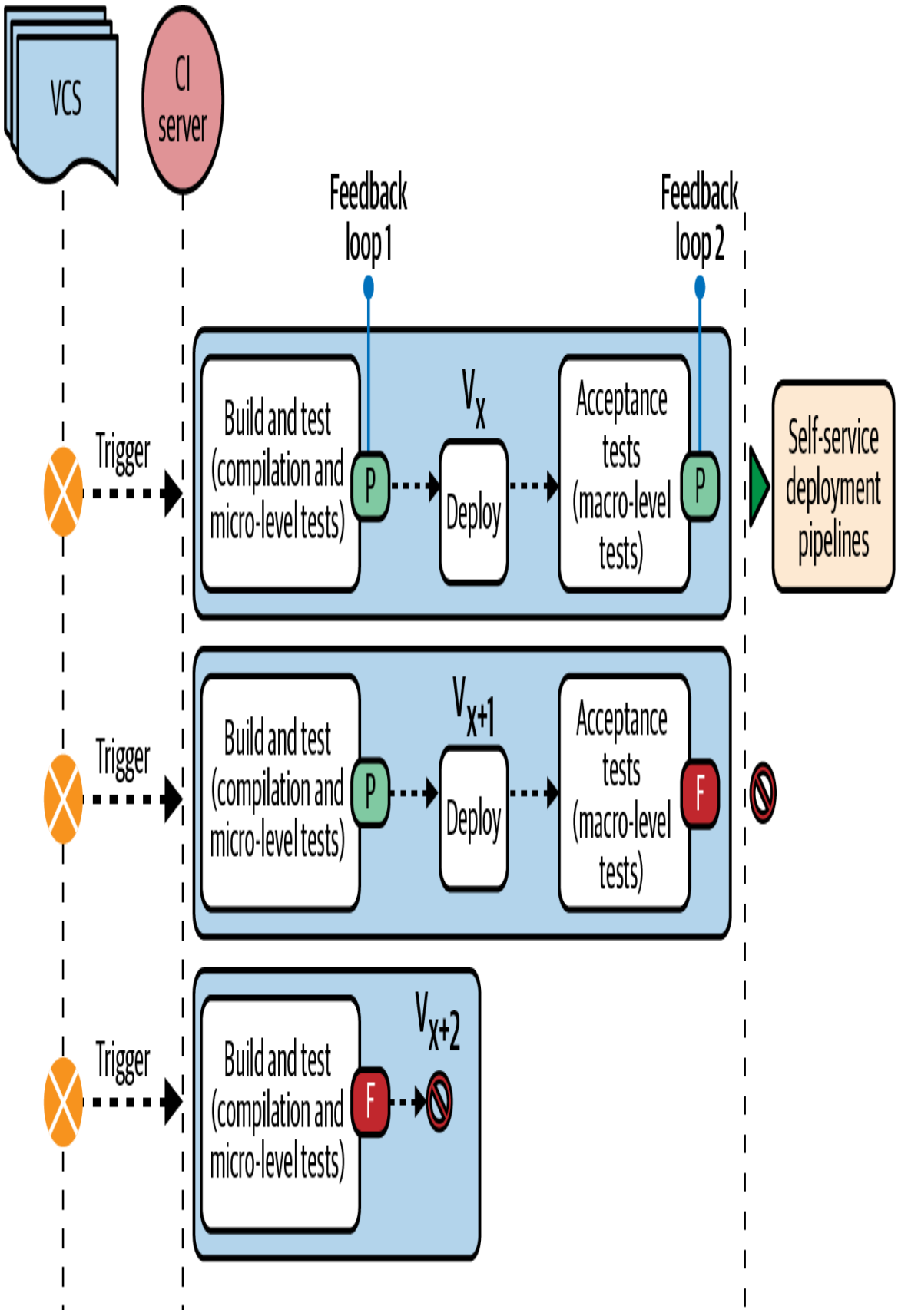

VCS
CI server
Feedback loop 1
Feedback loop 2
Trigger
Build and test (compilation and micro-level tests)
P
V_X
Deploy
Acceptance tests (macro-level tests)
P
Self-service deployment pipelines
Trigger
Build and test (compilation and micro-level tests)
P
V_{X+1}
Deploy
Acceptance tests (macro-level tests)
F
Trigger
Build and test (compilation and micro-level tests)
F
V_{X+2}

Figure 4-3. The continuous testing process with two feedback loops

> **NOTE**
>
> In their book *Continuous Delivery* (Addison-Wesley Professional), Jez Humble and David Farley suggest that the build and test stage should be short enough that it takes "around the amount of time you can devote to making a cup of tea, a quick chat, checking your email, or stretching your muscles."

As soon as the build and test stage passes, the *deploy* stage pushes the application artifacts to a CI environment (sometimes called the dev environment). The next stage, called the *functional testing stage* or *acceptance testing stage*, runs the macro-level tests against the deployed application in the CI environment. Only when this stage passes is the application ready for self-service deployments to other higher-level environments such as QA, UAT, and production.

The feedback from this stage might take longer as the acceptance tests take longer to run, and the stage is triggered after the application deployment, which takes time too. But when teams properly implement the test pyramid, the two feedback loops should take less than an hour to complete. The example I gave in Chapter 3 corroborates this: when the team had ~200 macro-level tests it took them 8 hours to get feedback, but when they reimplemented their testing structure to conform to the test pyramid, it took them only about 35 minutes from commit to being ready for self-service deployment with ~470 micro- and macro-level tests.

Another consideration is that when the feedback loop is short, team members can still prioritize fixing the issues found in the continuous testing process even if they've picked up a new task shortly after the build and test stage. If it takes several hours, they may be tempted to ignore the failing tests and track them as defect cards to fix later. This is harmful, as it means they are integrating their new code on

top of defects, and the new code is not thoroughly tested either as the failing tests are ignored. Hence, the team should continue to monitor and adopt ways to quicken the two feedback loops using techniques like parallelizing the test run, implementing the test pyramid, removing duplicate tests, and refactoring the tests to remove waits and abstract common functionalities.[2]

This continuous testing process can be further extended to receive cross-functional feedback, as depicted in Figure 4-4. Teams can run automated performance, security, and accessibility tests as part of the two existing feedback loops or configure separate stages subsequent to the acceptance testing stage in the CI server, achieving the goal of receiving continuous fast feedback on the application's quality holistically. You will learn shift-left strategies for cross-functional testing in the upcoming chapters.

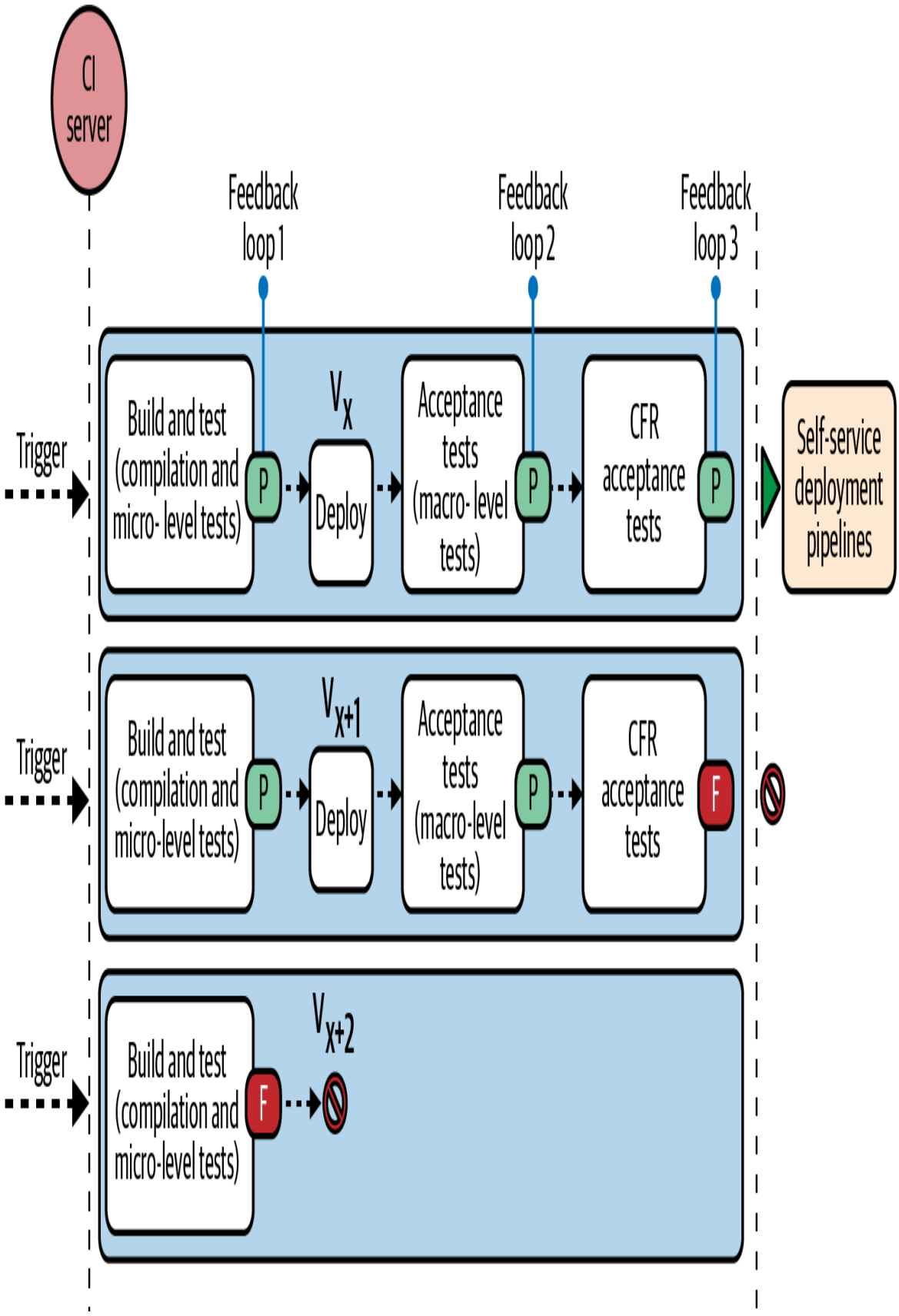

CI server
Feedback loop 1
Feedback loop 2
Feedback loop 3
Trigger
Build and test (compilation and micro- level tests)
P
Deploy
V_X
Acceptance tests (macro- level tests)
P
CFR acceptance tests
P
Self-service deployment pipelines
Trigger
Build and test (compilation and micro-level tests)
P
Deploy
V_{X+1}
Acceptance tests (macro-level tests)
P
CFR acceptance tests
F
Trigger
Build and test (compilation and micro-level tests)
F
V_{X+2}

Figure 4-4. The continuous testing process with three feedback loops

CONTINUOUS INTEGRATION VERSUS CONTINUOUS TESTING

As the name implies, the process of continuous integration ends with the build and test stage. That is, a commit is considered integrated only when it passes the micro-level tests (at least the unit tests).[3]

The continuous testing process encompasses validating the holistic application behavior, including its functional and cross-functional aspects, *for every commit*, with the goal of ensuring that it is ready for continuous delivery. In fact, continuous testing doesn't stop with executing automated tests; it includes the manual exploratory testing efforts for every commit after self-service deployment. The CT process also requires the team to automate the scenarios found during exploratory testing in order to call the functionality or commit "done."

At this point, when you run all the tests in a chained pipeline fashion it may take ample time and resources to finish all the stages. A way to strategize the CT process in this case is to split the tests into *smoke tests* and *nightly regression* tests, as seen in Figure 4-5.

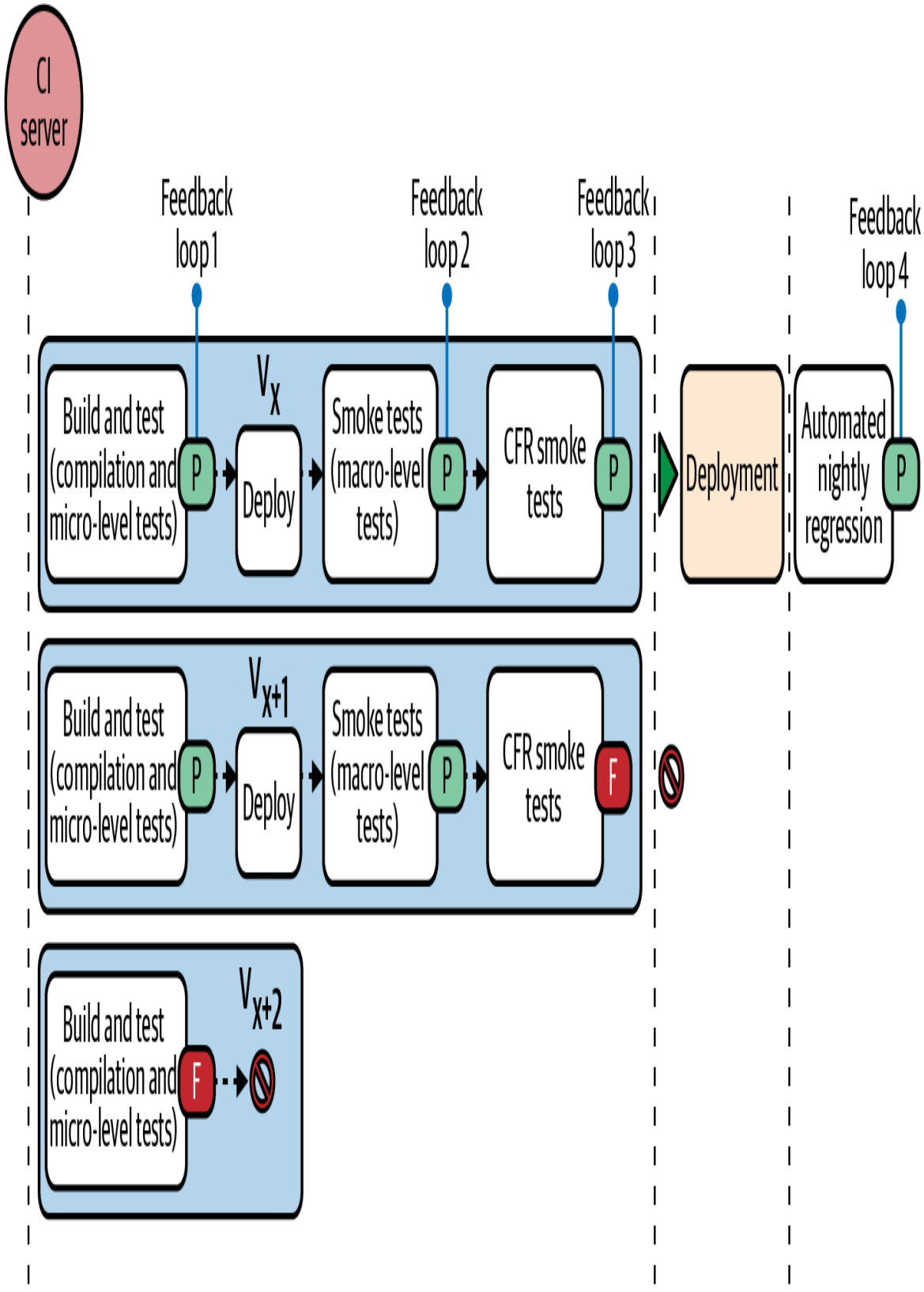


Figure 4-5. The continuous testing process with four feedback loops

Smoke testing is a term borrowed from the electrical engineering world, where electricity is passed after the circuit is completed to assess the end-to-end flow. When there are issues in the circuit, there will be smoke (hence the name). Similarly, you can choose the tests that cover the end-to-end flow of every feature in the application to form the smoke test pack and only run them as part of the acceptance testing stage. This way, you can get a high-level signal on the status of every commit quickly. As seen in Figure 4-5, the commit is ready for self-service deployment after the smoke test stage.

When you choose to perform smoke testing, you have to complement it with nightly regression. The nightly regression stage is configured in the CI server to run the entire test suite once every day when the team is off work (e.g., it may be scheduled to run at 7 p.m. every day). The tests are run against the latest code base with all the day's commits. The team must make a habit of analyzing the nightly regression results first thing the next day and prioritize fixing defects and environment failures. Sometimes this may require test script changes, and that has to be prioritized for the day as well so that the continuous testing process gives the right feedback for the upcoming commits.

You can apply these two strategies to split both the functional and cross-functional tests. For example, you can choose to run the performance load test for a single critical endpoint as part of every commit and run the remaining performance tests as part of nightly regression (performance tests are discussed in Chapter 8). Similarly, you can run the static code security scanning tests as part of the build and test stage and run the functional security scanning tests (discussed in Chapter 7) as part of the nightly regression stage. As obvious as it may be, the caveat with such an approach is that the feedback is delayed by a day. Consequently, there is a delay in fixing the feedback as well; the issues are tracked as defects and fixed later. As a result, you should be careful while choosing the types of

tests you run as part of the smoke test and nightly regression stages. Also, note that only macro-level and cross-functional tests should be categorized as smoke tests; all the micro-level tests should still be run as part of the build and test stage.

Most often, when the application is young, you can forgo these strategies and enjoy the privilege of running all the tests for every commit. Then, as the application starts to grow (along with the number of tests), you can implement the various CI runtime optimization methods, then eventually go the smoke test and nightly regression way.

Benefits

If you're wondering whether all that effort to undertake a continuous testing process will turn out to bear worthwhile fruit, Figure 4-6 showcases some benefits to get you and your team motivated.

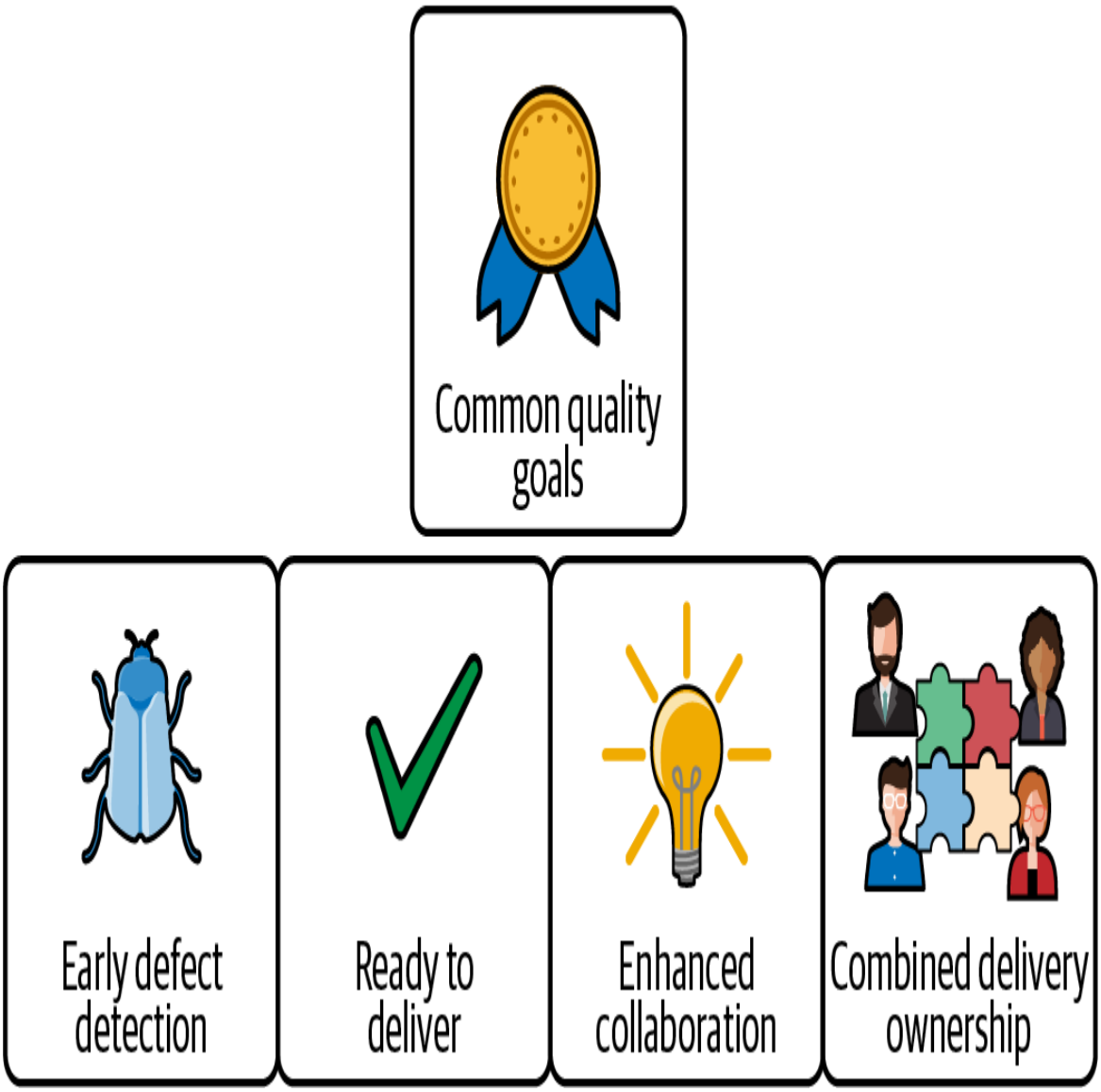


Figure 4-6. Benefits of the continuous testing process

Let's take a look at each of these in turn:

Common quality goals

> Following the continuous testing process ensures that all team members are aware of and working toward a common quality goal—in terms of both functional and cross-functional quality aspects—as their work is continuously evaluated against that goal. This is a concrete way to build quality in.

Early defect detection

Every team member gets immediate feedback on their commits, both in terms of functional and cross-functional aspects. This gives them the opportunity to fix issues while they have the relevant context, as opposed to coming back to the code a few days or weeks later.

Ready to deliver

Since the code is continuously tested, the application is always in a ready-to-deploy state for any environment.

Enhanced collaboration

It is easier to collaborate with distributed team members who are sharing their work and to keep track of which commit caused which issues, thereby avoiding accusations and limiting animosity.

Combined delivery ownership

Ownership for delivery is distributed among all team members instead of just the testing team or senior developers, as everyone is responsible for assuring their commits are ready for deployment.

If you've been working in the software industry for a while, you will surely know how hard it is to achieve some of these benefits otherwise!

Exercise

It's time to get hands-on. The guided exercise here will show you how to push the automated tests that you created as part of Chapter 3 into a VCS, set up a CI server, and integrate the automated tests with the CI server such that whenever you push a

commit to the VCS, the automated tests will be executed. You will be learning to use Git and Jenkins as part of this exercise.

Git

Originally developed in 2005 by Linus Torvalds, creator of the Linux operating system kernel, Git is the most widely used open source version control system. According to the 2021 Stack Overflow survey, 90% of respondents use Git. It's a distributed version control system, which means every team member gets a copy of the entire code base along with the history of changes. This gives teams a lot of flexibility in terms of debugging and working independently.

Setup

To begin with, you'll need somewhere to host your code base. GitHub and Bitbucket are companies that provide cloud-based offerings to host Git repositories (a repository, in simple terms, is a storage location for your code base). GitHub allows hosting public repositories for free, which makes it popular, especially among the open source community. So for this exercise, if you don't have a GitHub account already, create one now.

In your GitHub account, navigate to Your Repositories → New to create a new repository for your automated Selenium tests. Provide a name for the repository, say *FunctionalTests*, and make it a public repository. You will land on the repository setup page on successful creation. Note the URL for your repository (*https://github.com/<yourusername>/FunctionalTests.git*). The page will also give you a set of instructions to push your code to the repository using Git commands. You'll have to set up and configure Git on your machine to run them.

To do this, follow these steps:

1. Download and install Git from your command prompt using the following commands:

```
// macOS
$ brew install git
// Linux
$ sudo apt-get install git
```

If you are on Windows, download the installer from the official Git for Windows site.

2. Verify the installation by running the following command:

```
$ git --version
```

3. Whenever you make a commit, it needs to be tied to a username and email address for tracking purposes. Provide yours to Git with these commands so that it automatically attaches them when you make a commit:

```
$ git config --global user.name "yourUsername"
$ git config --global user.email "yourEmail"
```

4. Verify the configuration by running this command:

```
$ git config --global --list
```

Workflow

The workflow in Git has four stages that your code will move through, as seen in Figure 4-7. Each stage has a different purpose, as you will learn.

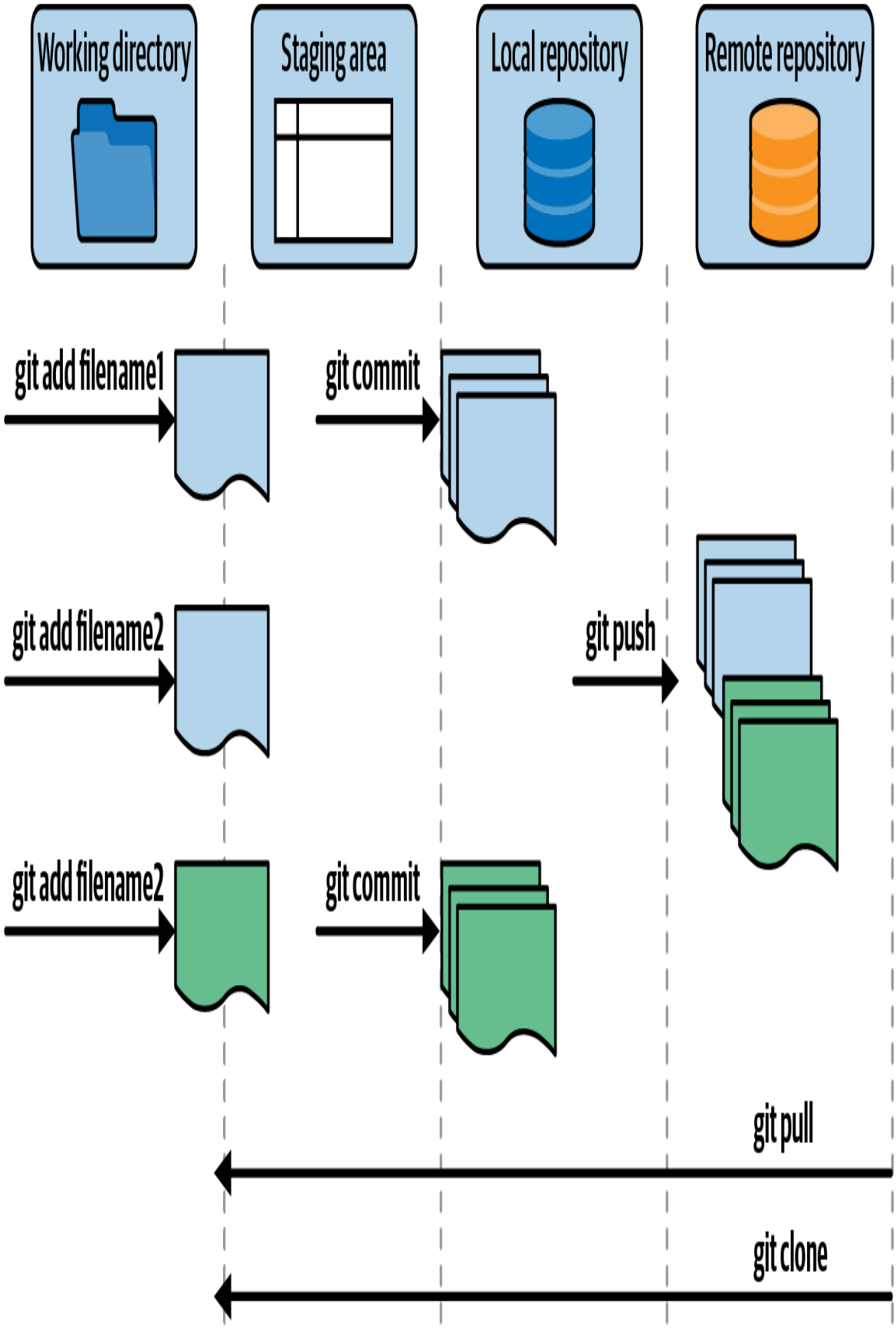

Working directory
Staging area
Local repository
Remote repository
git add filename1
git commit
git add filename2
git push
git add filename2
git commit
git pull
git clone

Figure 4-7. Git workflow with four stages

The first stage is your *working directory*, where you make changes to your test code (add new tests, fix test scripts, etc.). The second stage is the local *staging area* to which you add each small chunk of work, such as creating a page class, as you finish it. This allows you to keep track of the changes you are making so you can review and reuse them later. The third stage is your *local repository*. As mentioned earlier, Git gives everyone a copy of the entire repository along with the history on their local machine. Once you have a working test structure, you can make a commit that will move everything you've added to the staging area to your local repository. This makes it easy to revert back all the code as a single chunk when there are failures. Once you are finally done with all the required changes—in this case, when you have fully completed a test and want it to run as part of the CI pipeline—you can push it to the remote repository. The new test will be available to everyone in your team as well.

The Git commands to move the code through the different stages are shown in Figure 4-7. You can try them now step by step as follows:

1. In your terminal, go to the folder where you created your automated Selenium tests in Chapter 3. Run the following command to initialize the Git repository:

   ```
   $ cd /path/to/project/
   $ git init
   ```

   This command will set up the *.git* folder in your current working directory.

2. Add your entire test suite to the staging area by running the following command:

```
$ git add .
```

You can instead add a specific file (or directory) with `git add` *`filename`*.

3. Commit your changes to the local repository with a readable message explaining the context of the commit by executing the following command with the appropriate message text:

```
$ git commit -m "Adding functional tests"
```

You can combine steps 2 and 3 by appending the optional parameter with `-a`; i.e., **`git commit -am "`*`message`*`"`**.

4. To push your code to the public repository, you must first provide its location to your local Git. Do that by running the following command:

```
$ git remote add origin
```

`https://github.com/<`*`yourusername`*`>/FunctionalTests.git`

5. The next step is to push it to the public repository. You need to authenticate by providing your GitHub username and personal access token while pushing. A personal access token is a short-lived password mandated by GitHub for all operations from August 2021, for security reasons. To get your personal access token, go to your GitHub account, navigate to Settings → Developer Settings → “Personal access tokens,” click “Generate new token,” and fill in the required fields. Use the token when prompted after running the following command:

```
$ git push -u origin master
```

> **TIP**
>
> If you don't want to authenticate every time you interact with the public repository, you can choose to set up the SSH authentication mechanism.

6. Open your GitHub account and verify the repository.

When working with team members, you will have to pull their code to your machine from the public repository. You can do that by running the `git pull` command. If you already have a functional tests repository for your team, you can use `git clone` *`repoURL`* to get your local repository copy instead of `git init`.

Several other Git commands, such as `git merge`, `git fetch`, and `git reset`, make our lives easier. Explore them in the official documentation when needed.

Jenkins

The next step is to set up a Jenkins CI server on your local machine and integrate the automated tests from your Git repository.

> **NOTE**
>
> The intention of this part of the exercise is to give you an understanding of how continuous testing can be implemented in practice using CI/CD tools, not to teach you DevOps. Teams might engage developers with specialized DevOps skills or have a DevOps role to manage CI/CD/CT pipeline creation and maintenance work. However, it is essential for both developers and testers to be familiar with the CI/CD/CT process and its workings, as they will be interacting with this process and debugging failures firsthand. Also, from a testing perspective, it is critical to learn to adapt the CT process to the specific project needs and ensure the test stages are properly chained, as per the team's CT strategy.

Setup

Jenkins is an open source CI server. To use it, download the installation package for your OS and follow the standard installation procedure. Once installed, start the Jenkins service. On macOS, you can install and start the Jenkins service using `brew` commands as follows:

```
$ brew install jenkins-lts
$ brew services start jenkins-lts
```

After the service has started successfully, open the Jenkins web UI at *http://localhost:8080/*. The site will lead you through the following configuration activities:

1. Unlock Jenkins with a unique administrator password that was generated as part of the installation process. The web page will show you the path to the location of this password on your local machine.
2. Download and install the commonly used Jenkins plug-ins.
3. Create an administrator account. You will log in to Jenkins every time with this account.

After the initial configuration you will be taken to the Jenkins Dashboard page, as seen in Figure 4-8.



Dashboard

New Item

People

Build History

Manage Jenkins

My Views

Lockable Resources

New View

Build Queue

No builds in the queue.

Build Executor Status

1 Idle

2 Idle

add description

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job →

Set up a distributed build

Set up an agent →

Configure a cloud →

Learn more about distributed builds

Figure 4-8. Jenkins Dashboard view

> **NOTE**
>
> Although you are setting up a CI server on your local machine for this exercise, in practice the CI server will be hosted either in the cloud or on a VM in the same network so that all team members can access it.

Workflow

Now, follow these steps to set up a new pipeline for your automated tests:

1. From the Jenkins Dashboard, go to Manage Jenkins → Global Tool Configuration to configure the `JAVA_HOME` and `MAVEN_HOME` environment variables, as seen in 4-9 and 4-10. You can type the `mvn -v` command in your terminal to get both locations.



Figure 4-9. Configuring `JAVA_HOME` in Jenkins

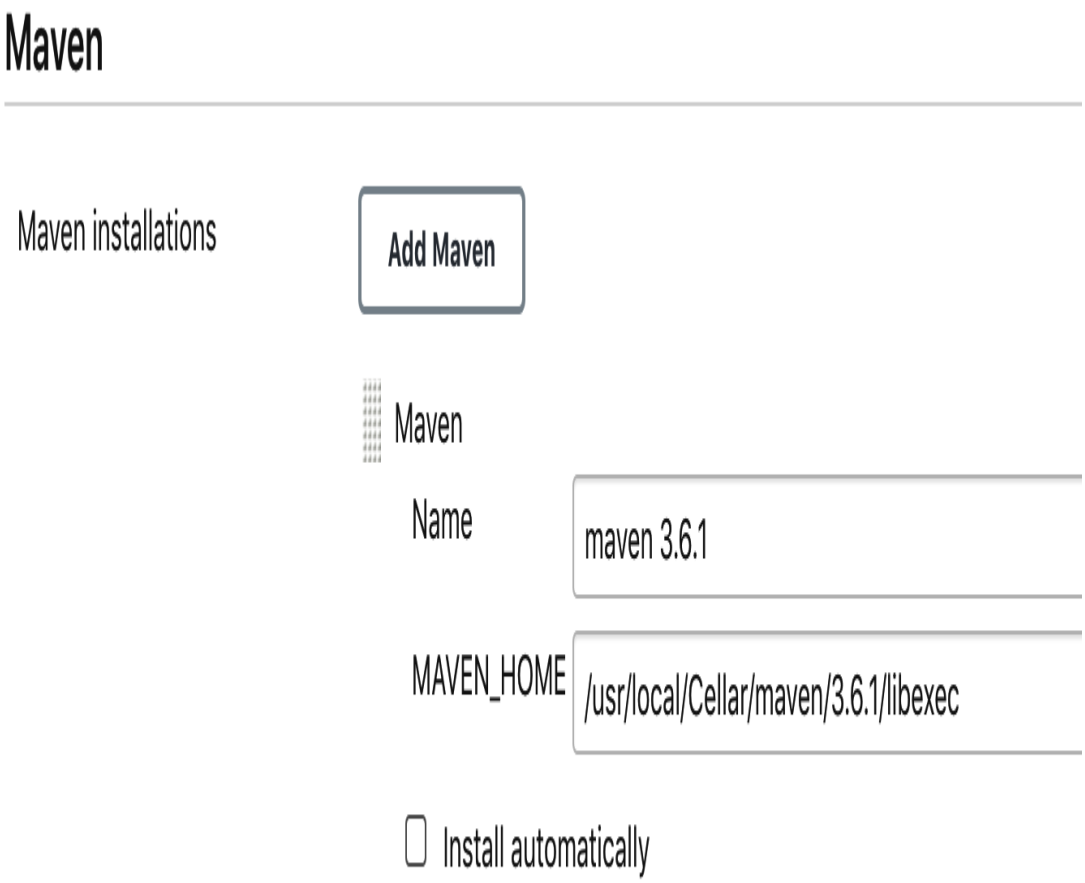


Figure 4-10. Configuring `MAVEN_HOME` in Jenkins

2. Coming back to the Dashboard view, select the New Item option in the lefthand panel to create a new pipeline. Enter a name for the pipeline, say “Functional Tests,” and choose the “Freestyle project” option. This will take you to the pipeline configuration page, as seen in Figure 4-11.

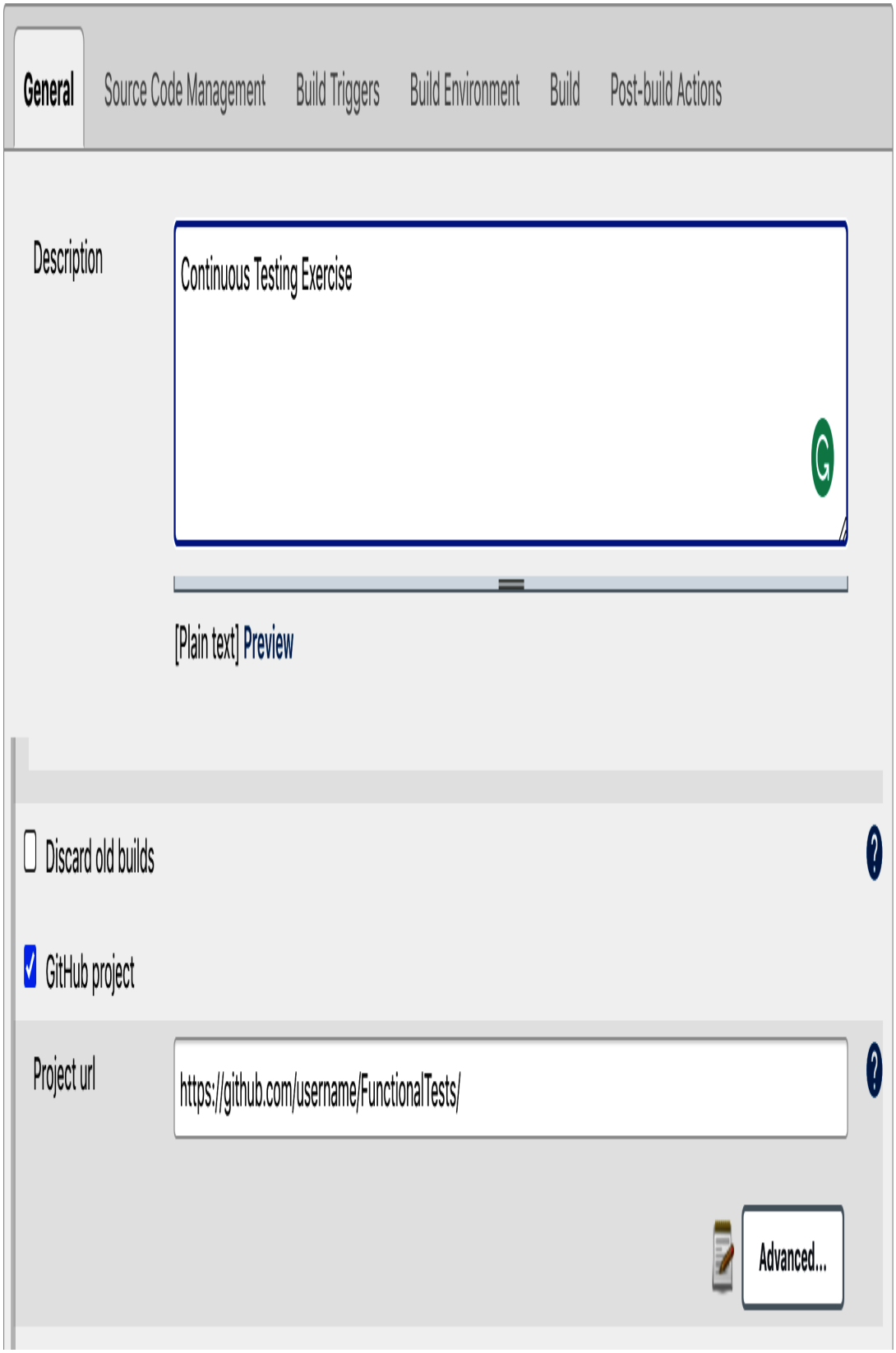


Figure 4-11. The Jenkins pipeline configuration page

3. Enter the following details to configure your pipeline:

    - On the General tab, add a pipeline description. Select “GitHub project” and enter your repository URL (without the *.git* extension).
    - On the Source Code Management tab, select Git and enter your repository URL (with the *.git* extension this time). Jenkins will use this to do `git clone`.
    - The Build Triggers tab provides a few options to configure when and how to kickstart the pipeline in an automated fashion. For example, the Poll SCM option can be used to poll the Git repository every two minutes to check for new changes and, if there are any, start the test run. The Build Periodically option can be used to schedule the test run at fixed intervals even if there are no new code changes. This can be used to configure nightly regressions. Similarly, the “GitHub hook trigger for GITScm polling” option configures a GitHub plug-in to send a trigger to Jenkins whenever there are new changes. To keep it simple, choose Poll SCM and enter this value to poll the functional tests repository every two minutes: `H/2 * * * *`.
    - Since your Selenium WebDriver functional test framework uses Maven, select the “Invoke top-level Maven targets” option on the Build tab. Choose your local Maven, which you configured in Chapter 3. In the Goals field, enter the Maven lifecycle phase that needs to be run by the pipeline: `test`. This will execute the `mvn test` command from the project directory.
    - The Post-build Actions tab is where you can chain multiple pipelines—i.e., trigger the CFR tests pipeline after the functional tests pipeline has passed and create a complete CD pipeline.[4]

4. Save and navigate to the Dashboard view. You will see the pipeline created, as seen in Figure 4-12.

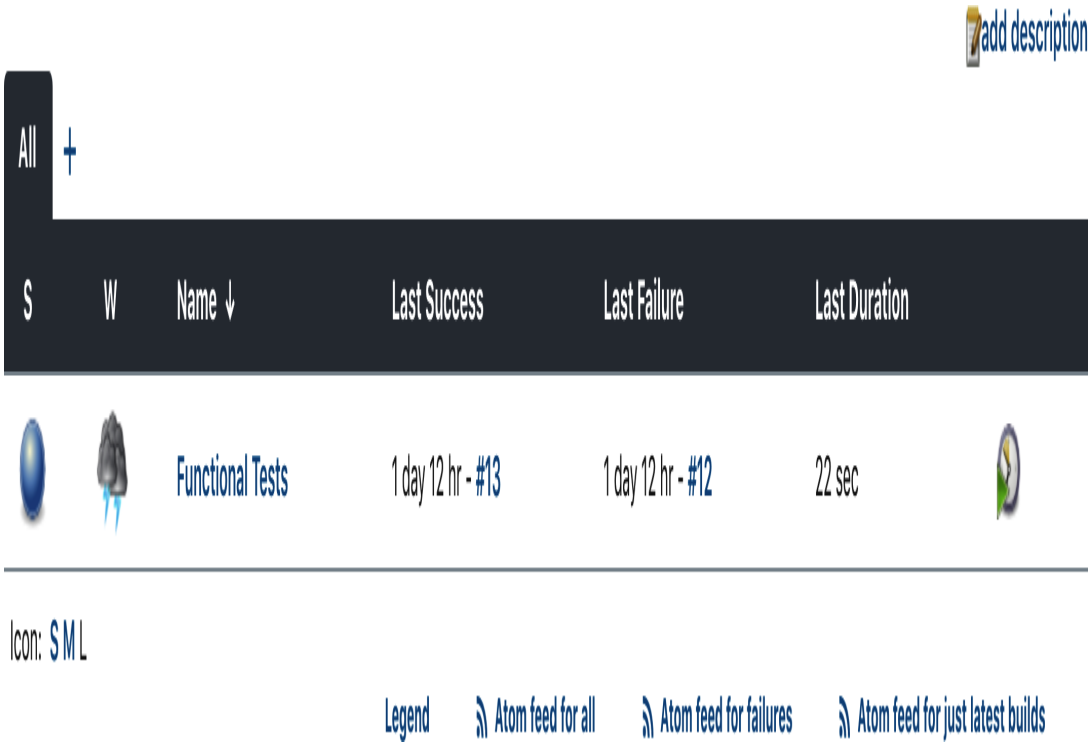


Figure 4-12. Your pipeline in the Jenkins Dashboard

5. Click the pipeline name in the Dashboard view, and on the landing page, select the Build Now option in the left panel. The pipeline will clone the repository on your local machine and execute the `mvn test` command. You can see the Chrome browser open and close as part of test execution.

6. Locate the *Workspace* folder on the same page. You'll find the local cloned copy of the code from the repository and the reports generated after the tests are run in this folder; you can use this for debugging purposes.

7. In the bottom section of the lefthand panel on the same page, select the current pipeline execution build count. You will have the option to view the console output in the left panel of the landing page. This view will show the live execution activities for debugging.

Congratulations, that completes your CI setup!

Along the same lines, you will have to add the stages of the respective tests (static code, acceptance, smoke, CFR) as per your continuous testing strategy to complete the end-to-end CD setup for the project. Ensure that the stages get triggered not only after the application code changes but also after configuration, infrastructure, and test code changes!

The Four Key Metrics

The ultimate outcome from all this effort spent on setting up your CI/CD/CT processes (and adhering to the principles and etiquette laid out earlier) is the team qualifying as an elite or high-performing team according to the four key metrics (4KM) identified by Google's DevOps Research and Assessment (DORA) team. DORA formulated the 4KM based on extensive research, and outlined how to use these metrics to quantify a software team's performance level as either elite, high, medium, or low. The book *Accelerate* by Jez Humble, Gene Kim, and Nicole Forsgren is an excellent read to learn the details of the research.

In short, the four key metrics give us a handle on measuring a team's delivery tempo and the stability of their releases. They are:

Lead time

> The time taken from code being committed to it being ready for production deployment

Deployment frequency

> The frequency at which the software is deployed to production or an app store

Mean time to restore

The time taken to restore any service outages or recover from failures

Change fail percentage

The percentage of changes released to production that require subsequent remediation, such as rollbacks to a previous version or hot fixes, or that cause a degradation in service quality

The first two metrics, lead time and deployment frequency, expose the delivery tempo of the team. They measure how quickly a team can deliver value to end users and how frequently they add value to end users. However, in the rush to deliver value to customers, the team should not compromise on the stability of the software. The last two metrics validate this. The mean time to restore and change fail percentage provide an indication of the stability of the software being released. In today's world, software failures are inevitable, and these metrics measure how easy it is to recover from these failures and how often such failures occur due to new releases. As you can see, together the 4KM give a clear picture of a software team's performance by measuring their speed, reactivity, and ability to deliver with quality and stability.

The targets for an elite team, per DORA research, are represented in Table 4-1.

Table 4-1. An elite team's fourkeymet

r
i
c
s

Metric	Target
Deployment frequency	On-demand (multiple deploys per day)
Lead time	Less than a day
Mean time to restore	Less than an hour
Change fail percentage	0–15%

As discussed earlier, one of the main benefits of having a rigorous CI/CD/CT process is that your team will be able to deliver value to customers on demand. Similarly, as you've seen, when you place automated tests in the right application layers, you can have your code tested as part of the continuous testing process and easily made ready for deployment within hours (i.e., your lead time will be less than a day). Also, with your functional and cross-functional requirements tests automated and run as part of the CT process, it should be no problem to keep your change fail percentage well within the recommended range of 0–15%. Thus, the effort you put in on this front will enable your team to earn "elite" status, per the DORA definition. DORA research also shows that elite teams contribute to an organization's success, in terms of profit, share price, customer retention, and other criteria. And when the organization does well, it takes good care of its employees, right?

Key Takeaways

Here are the key takeaways from this chapter:

- The continuous testing process validates the application quality in terms of both functional and cross-functional aspects in an automated fashion for every incremental change.
- Continuous testing relies heavily on the continuous integration process. Continuous integration and testing, in turn, enable continuous delivery of software to customers on demand.
- Continuous integration and testing processes require teams to follow strict principles and etiquette for them to be fruitful.
- Plan your continuous testing process such that you get fast feedback in multiple loops continuously.
- The benefits of continuous testing are numerous, and many of them—including establishing common quality goals across roles and teams, shared delivery ownership, and improved collaboration across distributed teams—are difficult to achieve otherwise.
- Although DevOps engineers might be responsible for CI/CD setup and maintenance, it is vital for testers on the team to devise the continuous testing strategy and ensure the feedback loops are triggered correctly. Most importantly, they should keep a keen watch over the team's CT practices to ensure the effort spent on creating and maintaining tests reaps the right benefits.
- Following rigorous CI/CD/CT processes will lead your team to become an elite team, as defined by DORA research. And an elite team contributes to the success of the entire organization!

---

1 For more on this and other commonly prescribed CI/CD industry principles, see *The DevOps Handbook* (IT Revolution Press), by Gene Kim, Jez Humble, Patrick Debois, and John Willis.

2 Jez Humble and David Farley discuss such optimization techniques at greater length in *Continuous Delivery*.

3 For further details, see Jez Humble, Gene Kim, and Nicole Forsgren's book *Accelerate* (IT Revolution Press).

4 For more information on working with pipelines, see the Jenkins documentation.

Chapter 5. Data Testing

Make or break trust with data!

Take a moment to think about the online services that you use every day. You will find that they essentially offer you one of two types of services: they sell their data to you or collect your data and process it on your behalf. For instance, ecommerce, ride-hailing, food delivery, movie booking/streaming, and online gaming applications are examples of the first category, where their core value proposition comes from the collection of data, whereas a notes application, social networking apps like Facebook, Twitter, and Instagram, blogging sites and the like thrive by accumulating your data! In both cases, data is at the center of their galaxy, and their unique functionalities, user experience design, branding, and marketing aspects revolve around it. To elaborate on this with an example, Amazon is a data business at its heart. Its collection of product information forms the epicenter of the business, and core functionalities, such as purchasing and delivering products, are built on top of it. The company's branding and marketing subtly draws attention to its data supremacy: the Amazon logo, with an arrow between the A and the z, tells the world that it has an enormous variety of data on products ranging from a–z.

Data has an unparalleled significance in any application, and when its integrity is not maintained with diligence, customers' trust in the application can quickly go downhill—and along with their trust, sales and the business will suffer. For instance, imagine you transfer money between two of your accounts via your online banking app, and although the transaction is deemed successful, the balances in both accounts fail to reflect the right amounts for a period of time. You would likely panic, and might start to question the integrity of the bank! Such reactions happen even when applications don't handle any critical data. For instance, suppose the posts you publish on a blogging site show inconsistently to a set of your colleagues, or a social networking site loses your family photos. I am sure we would all feel disappointed, as

our data is important to us, irrespective of its relative significance! Eventually, such failures will lead us to seek alternatives.

What we can derive from these examples is that data integrity has an unforgiving make-it-or-break-it power, and as a result, testing how data is stored, processed, and presented is pivotal to ensuring the success of the application. In this chapter, we will discuss the essentials of such testing. You will first be introduced to the different ways in which an application stores and processes data—specifically, databases, caches, streaming, and batch processing systems. The discussion around these systems will enable you to recognize the new test cases introduced by each of them, and especially the failure cases caused due to problems with concurrency, distributed data processing, and asynchronous communication. The latter part of the chapter has exercises that will equip you to perform automated and manual data testing using a variety of tools.

DATA TESTING AND FUNCTIONAL TESTING

One might argue that data testing will be covered as part of testing the application's functionalities, and that is partially true. However, when you think of the same functionality along the lines of data flow, you will discover new test cases, which is the focus of this chapter.

Also, testing the functionality via the UI and the APIs may not suffice all the time. You may have to test data integrity in the storage and processing systems separately to ensure the functionality is complete. And to do this, you need to learn a specific set of tools and methods. Additionally, you will see in the course of this chapter that the nature of these storage and processing systems introduces new test cases, so specialized knowledge of those data systems is required. The data testing skill covers all of this.

In other words, in order to completely test a functionality, you will require data testing skill as well.

Building Blocks

Let’s start with an introduction to a set of data storage and processing systems that are typically used in web and mobile applications to understand the testing aspects of each of them. For the purposes of this discussion, we’ll again consider the simplified ecommerce application used in Chapter 3. Figure 5-1 shows the same application architecture, this time highlighting the data systems.

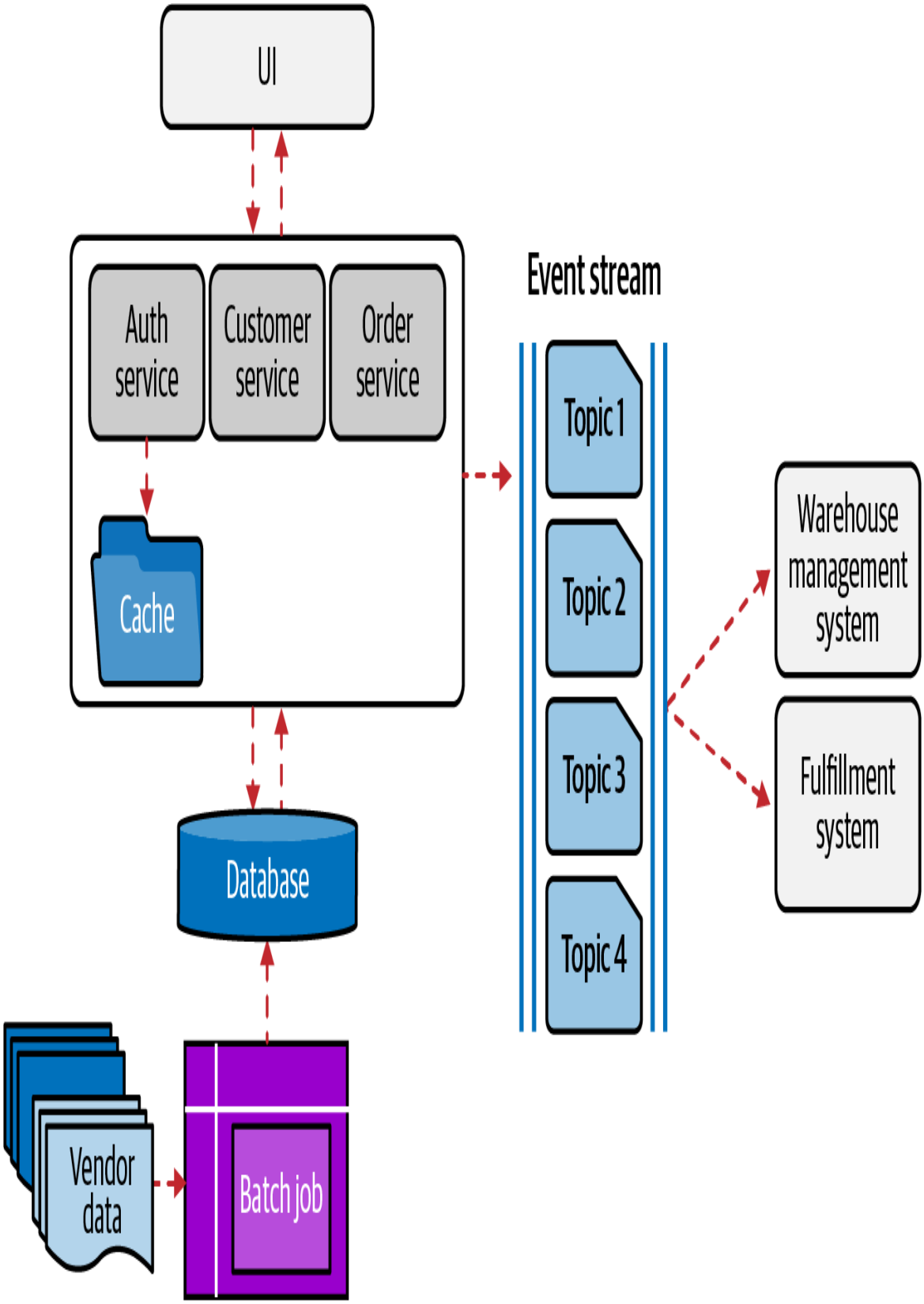

UI
Auth service
Customer service
Order service
Cache
Event stream
Topic 1
Topic 2
Topic 3
Topic 4
Warehouse management system
Fulfillment system
Database
Vendor data
Batch job

Figure 5-1. A simple ecommerce application with four data storage and processing systems

As you'll recall, the application has a UI layer, which communicates with a set of services for different kinds of business processing. The services, in turn, are connected to a centralized database, where all the application data is stored. In addition, you will notice three other data systems: a cache server, a batch processing system, and an event stream. The arrows in the figure indicate the data flow between these systems. Let's trace the data flow starting from the UI layer to clarify the distinct roles played by these data systems.

To begin with, let's say an end user is trying to log in to the application by entering their credentials in the UI. The UI layer starts by passing the credentials data to the authentication (auth) service. This service in turn passes the data to the database to check if the record matches. If the credentials match, the auth service, assuming it follows the OAuth 2.0 protocol, returns an access token to the UI and also persists it in the cache server. This is a key piece of internal data in the application, as any further action performed by the user will require this access token to prove that a valid end user is requesting the action.

Let's take this a step further and look at an example. Suppose a user tries to place an order from the UI. The UI layer constructs an order request to be sent to the order service and appends the access token in the header of the request. The order service checks with the auth service to make sure the access token is valid, and the auth service in turn queries the cache. If the access token has already expired, the cache server will have deleted it automatically, and therefore the services will return a 404 status code. On receiving this 404 response, the UI will diligently redirect the user back to the login page in an effort to protect their security.

On the other hand, when the access token is valid (i.e., hasn't expired in the cache), the auth service acknowledges this to the order service, which will then allow the order to be created in the database. The order service will also create an event with the new order information and place it in the event streaming system for the downstream systems, such

as the warehouse management and fulfillment management systems, to do their respective jobs. A point to note here is that the order service's responsibility ends with placing the event in the event streaming system, and it doesn't really care if the respective downstream systems then do their jobs correctly. The downstream systems continuously listen to the event streaming system and only consume those events that are relevant to them. For instance, if there is a customer address change event, the fulfillment management system may be interested, but the warehouse management system may not process that; however, if there is an order creation event, both systems may process it.

Meanwhile, there is a batch processing system in place to parse the different vendors' product details into the centralized database. This batch processor gets triggered automatically at the programmed time—say, once every day at midnight—and imports all the new and updated data into the database so that new products and any other changes will become visible in the application on the following day.

As we can see from this example, each of these four data systems plays a critical role in catering to a set of essential application requirements. Let's explore them individually in more detail, to uncover the unique properties that make them suitable for their respective contexts and the new test cases they expose.

Databases

Databases don't really need an introduction, as they are a prevalent component in almost all applications and are, comparatively speaking, an established data storage system. One of the reasons for this wide adoption is their capability to provide strong durability of data, as the data gets stored on a hard disk and is lost only in the event of hardware failures.

To those who are new to databases, they can be compared to jewelry boxes, where you organize different accessories in their respective compartments and access them whenever needed. Each piece of jewelry is kept safe in the box until someone transfers or replaces it. Similarly, an application's data is organized and stored meaningfully in

databases and can be queried back whenever needed. The application has the flexibility to *create* new data and to *read*, *update*, or *delete* existing data as required by the functionality (these four core operations are commonly referred to by the acronym CRUD).

Based on the way in which the data is structured, such as in tables, as JSON or XML documents, or as graphs, databases can be classified into relational, document, and graph databases, respectively.[1] Relational databases are the dominant category, and have been serving a wide range of applications' data storage requirements over the last several decades. MySQL and PostgreSQL are examples of open source relational databases; we'll look at working with a PostgreSQL DB in this chapter's exercises.

In a relational database the data is stored in table structures, with rows and columns. Each row in a table depicts a set of related information separated into columns, like in the `Customers` table in Table 5-1.

Table 5-1. An example of a table structu

c
t
u
r
e

i
n

a

r
e
l
a
ti
o
n
a
l
d
a
t
a
b
a
s
e

UUID (primary key)	Customer name (varchar 30)	Phone number (int)	Email address (varchar 254)	Shipping address (varchar 100)
019367	Alice	4567879	alice@xyz.com	8/13, Block A
045678	Bob D’arcy	0898678	bobdarcy@xyz.	23-A, Winscent

The columns can have predefined names and properties, such as their data type and maximum length. Additionally, every row is given a universally unique identifier (UUID) that serves to relate the records across multiple tables. For example, the customer list in our ecommerce application might be stored in a table like this one, with each row representing the details of a single customer, such as their name, email address, phone number, and shipping address. In this case, a unique user ID will be created as the record's unique identifier and stored along with it, which can then be used to query the customer's information. The same user ID can be used in other tables, such as an account history table, making it possible to retrieve an entire catalog of information about any given user. This definition of tables, rows, column names, unique identifiers, etc. is referred to as the *database schema*. The database schema is derived based on the application's business use case by the developers or a database administrator. The schema may also get redefined during the course of delivery as the application requirements grow. To perform these operations, a domain-specific language called the Structured Query Language (SQL, pronounced *see-quel*) is used in relational databases.

Given this information, some of the basic test cases we might want to test for are:

- Verifying the positive test case where the information gathered from the user via the UI should be stored in the DB and be appropriately related.
- Testing the boundary values based on the column data type and length of inputs. For example, when the customer name field is restricted to be less than 20 characters in the DB, the same restriction should be applied in the UI, and an appropriate error message should be displayed to the user if the length is exceeded.
- Testing with inputs that include SQL syntax. For instance, can Bob D'arcy, whose name includes an apostrophe, have his name stored

properly in the DB? Or is some cleaning logic required in the code?

- What happens to an ongoing write operation if a sudden network failure occurs? Does the data get written partially to some tables but not to all the related tables? This scenario is amplified when the operation is split across multiple services.
- How does a retry operation impact such cases?
- What is the timeout period before the application retries a database operation, and what will the user experience be like?

When we include the concurrency factor—in other words, when multiple users and systems concurrently access the database for reads and writes—we should think of a few more test cases, especially around race conditions. Here are some considerations to trigger the thought process:

- It is possible for one user's actions to clash with another's, leading to lost updates. For instance, when two users purchase the same item at the same instant, the item quantity may decrease by only one count, and not two.
- Similarly, users could see mismatching data if the application chooses to read partial updates. For example, let's say stock of some unavailable items is getting replenished, and the process first changes the item availability flag to true in one table and then updates the available item count in another table. Between these two operations, an end user could see the item as being available but with 0 quantity.
- Again, concurrency could have an unprecedented effect on shared resources. For instance, if two users buy the last available item concurrently using the cash on delivery payment option, it is possible that the item may be allocated to one user, but the invoice may be generated for the other.
- Also, concurrency imposes a limit on the database performance. Thus, performance testing with expected real-time data volumes will become a crucial test case.

A callout here is that the concurrency-related test cases are hard to simulate, and knowledge of them is mainly useful in the analysis phase so that they can be preemptively addressed during development.

Beyond concurrent accesses of a single instance, databases cater to scalability via *replication*. Replication refers to creating redundant instances of the same data. The instances are usually kept geographically separate, to improve performance for users in different locations (say, the East and West Coast of the US, or North America and Europe). In such cases, there needs to be a mechanism to keep all the replicas in sync with the latest updates. This is usually accomplished by assigning one of the replicas the role of *leader*, responsible for sending the updates to the other replicas (the *followers*). Such a situation may lead to replication lag, where it takes followers some time to get the update and reach the same state as the leader. The lag could be on the order of anywhere from a few seconds to a few minutes based on network latency, traffic to that instance, and so on. This model of reaching a consistent state after a period of time is referred to as *eventual consistency.*

> **NOTE**
>
> To explore more about other consistency models, check out Jepsen's guide, with a clickable map.

The eventual consistency model is suitable for applications such as Twitter or Facebook, where showing slightly older posts may not have a great impact on the users. However, in certain other applications, the lag could confuse users if not handled properly, and even harm trust. Let's discuss a few of the possible problems that can occur:

Reading your own writes

Suppose a user updates their profile information, then wants to confirm the changes and opens the profile page again a few seconds later. The updates might not have propagated to all the followers yet, so if the application reads from a lagging follower, they may end up

seeing their old profile information. Confused by this, they may reenter the changes. If this cycle repeats a few times, in addition to the user becoming frustrated the system could get overloaded, prolonging the replication lag.

Time traveling

Let's say a user is tracking live cricket updates on a sports website. They keep refreshing the page every few seconds to see the scores. If the website reads from multiple followers with eventual consistency, they might experience a sensation of time traveling. For example, the first time update could show the score as 116 runs in 5 overs, while on the next refresh the website could read from a lagging follower and show the score as 110 over 4.5 overs.

Inconsistent ordering

Sometimes data is chained sequentially, and it doesn't make sense if the sequence is not sustained. For example, in a conversation over a Facebook post the order in which the users write their comments needs to be maintained, but when replication lags aren't thought of and handled appropriately it is possible for a user to see a comment answering a question without seeing the question before it.

Write conflicts

To avoid single points of failure, sometimes more than one leader is assigned to manage replication. In such cases, new updates could be sent to different leaders, resulting in write conflicts. Write conflicts happen whenever a single resource is altered by many parties, such as a Google slide that gets edited at the same time by different team members. In such a case, edits to the same text could be accepted by different leaders, but when the updates are combined there will be a conflict as to which one to take as the final update.

The good news is that the solutions to these common issues are well established, and most often the databases themselves handle them

inherently. However, being aware of and vigilant about such possible issues is essential, both in application development and in testing.

To summarize the key points of this section, when testing databases consider your application's data and its variations, as well as potential problems such as network failures, concurrency clashes, and other distributed data challenges.

Caches

A *cache* is an in-memory data store where data is persisted as key/value pairs. Storing data in memory boosts performance by several orders of magnitude, as the application doesn't have to make calls to a heavyweight backend storage system such as a traditional relational database. Today's popular caching tools, such as Memcached and Redis, can store terabytes of data and provide sub-millisecond responses. However, when it comes to durability, databases have an upper hand as the data is firmly written to disk.

> **NOTE**
>
> Redis has evolved to provide many features apart from just being an in-memory caching system. It can even be configured to persist point-in-time data (snapshots) on the disk for recovery. To read more about Redis's features, check out the official documentation.

Given these pros and cons of caches, the recommended practice is generally to cache only data that is transient in nature and is frequently needed by the application. For instance, in our ecommerce application, the access tokens are cached as they are expected to live only for a short period (until the user is logged in) and are required to be frequently accessed by the application internally, to validate the authenticity of every service request. Also, in an unfortunate situation such as a cache failure, if all the users' access tokens are lost the impact is minimal: to recover, the current set of logged-in users will simply have to log out and log in again (a minor annoyance, not on par with loss of valuable

personal data or customer history). A cache fits this scenario perfectly, since there isn't a demand for solid durability like there is in a database.

An alternative and common approach is to replicate the frequently accessed application data both in the cache and a database. In such cases, the application code has to bear the responsibility of maintaining the lifecycle of the cached data: keeping it in sync with the database, purging old data, falling back to the database on cache failure, and so on. These scenarios will become test cases when data is replicated in the database and the cache. A few other test cases, in general, would be:

- The data in the cache will be configured with a time to live (TTL) value beyond which it expires. For example, the access tokens might be configured to live for 30 seconds. Beyond 30 seconds, we should ensure that the auth service generates a new token and stores it in the cache again.
- If the cache becomes a single point of failure for the application, as in the case of a cache failure causing all users to need to log out and log in again, the user redirection flow has to be tested.
- If service instances are replicated, their caches will be too, leading to distributed cache storage. You may then have to ensure the functionality still works correctly. (Most distributed cache implementations, such as Redis Cluster, support redirection to the right cache instance out of the box, and hence this is a matter of verifying the functional flow.)
- Testing application performance with maximum load will again be critical.

Batch Processing Systems

A *batch processing system* is one where a program or job is written to transform a set of input data into the desired output, not in real time, but in batches gathered over a period of time. These batch jobs can be written using frameworks and libraries like Spring Batch or Apache Spark and can run autonomously without user intervention. The input

data to the batch jobs can be in the form of files, database records, images, etc. The volume of input data can be massive, causing the job to take hours or even days to complete. Indeed, the performance of a batch job is measured in terms of how big an input file it can process and in what time, not in terms of response times like with databases or caches.

Some typical use cases for a batch processing system are report generation, bill generation, monthly payslip generation, and cleaning data before training machine learning models. A couple of observable patterns in these use cases are that the batch jobs transform unorganized or sparse data into meaningful data structures, and such transformations need not happen in real time for the application to run smoothly.

To illustrate, let's go back to the ecommerce application. When vendors want to display their new or updated product catalogs in the application, they send their latest item details as files. The files may have thousands of item records, with each item represented by its SKU, color, size, price, etc. The keys in the item records may be different across vendors, and the files may be in different formats, such as JSON or CSV, depending on the vendor's internal systems. This unorganized data, with all its variations, has to be transformed into a common structure in order to be meaningful to the application—that is, the files need to be transformed into database records so that the application can display them in the UI. A point to note here is that it is enough to reflect the updated catalog the next day or a few days later, and not necessarily in real time. So, a batch processing system fits right in.

A batch job (or several batch jobs) can be written to read the records one by one from different files, extract the right information, and transform it into database records. The job can be scheduled to run autonomously at the same time every day; say at midnight, when the site's traffic is low. In such an arrangement, the vendor files sent that day before midnight form a batch. Usually on failures batch jobs are rerun, with a provision to discard or overwrite the data created during the previous failed run.

Given the nature of batch processing systems, some of the general test cases to think of while testing them are:

- Verifying that the input files are processed in full and that the operation is not abandoned halfway through
- Handling inputs that are corrupted, such as having unexpected null values, large integers, and other anomalies
- Flagging and isolating incomplete records that cannot be transformed into the required structure
- Ensuring that retry mechanisms clean up or overwrite the failed run's data
- Verifying that the batch jobs, which can take up significant processing capacity, do not adversely affect the application's performance

Sometimes while testing you may discover that different parties are sending data in new formats, which may require your team to update the batch job. Also, different vendors may have different counts of products in certain categories, such as men's apparel, sports shoes, etc. If the quantity of products in one particular category is too high, also referred to as *data skew*, then the batch job's performance may be affected, depending on how it is programmed. So, getting several samples of inputs in advance from the relevant parties will help in testing.

Event Streams

An *event*, in its literal sense, refers to an action, while a *stream* represents an entity that is flowing, or, in other words, continuous in nature. So, *event streams* are systems where application-specific events are continuously published to a stream, from which other relevant systems in turn consume that data whenever they get time for further processing. For example, in the ecommerce application, as seen in Figure 5-2, an order event with the order details is posted to the event stream immediately when a customer places an order, and the downstream systems read the event and take respective actions to fulfill

the order. From the data flow perspective, the order data is stored in the event stream for a period of time, and the expected systems read from it until then.

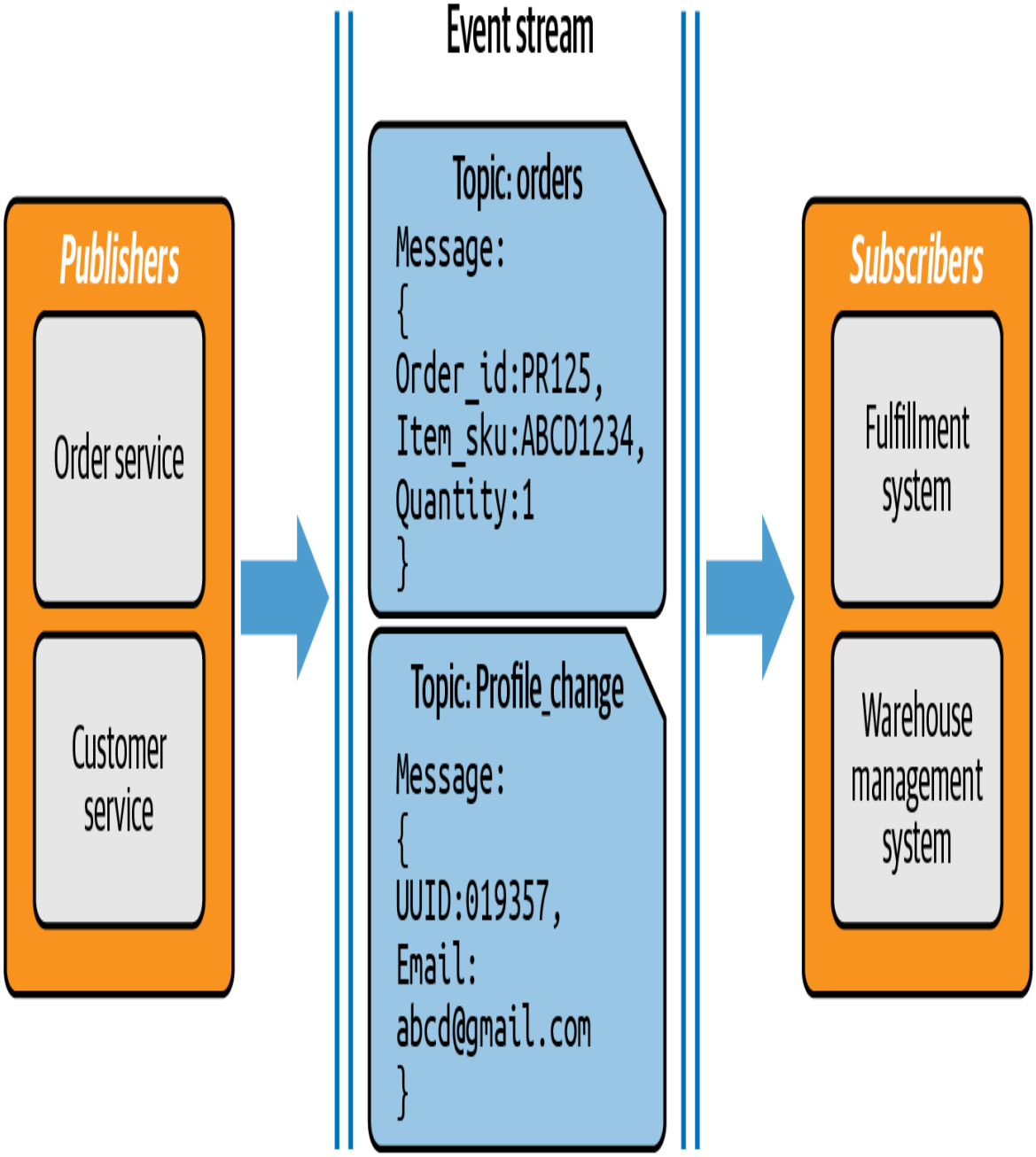


Figure 5-2. An event streaming system

Here, the order service is called the *publisher* as it publishes the events, and the downstream systems that consume the events are called *subscribers*. Every event is posted with a specific *topic* name so that the

subscribers can identify the events that are relevant to them. Some event stream systems, like Google Cloud Pub/Sub and RabbitMQ, delete each event after all its intended subscribers have consumed it. In other systems, like Apache Kafka, the events are deleted after a configured time. This retention feature enables subscribers to catch up after interim failures. Event streams also provide durability as they write the events to the disk, like databases.

Given the features of an event streaming system, it is fair to ask if a batch job wouldn't fit in its place. A batch processing system differs from an event stream processing system mainly in its time-bounded nature; that is, a batch job processes the inputs at or after some preconfigured time, whereas with an event stream processing happens in near real time. To elaborate on that term, the order service in the ecommerce application posts an event immediately after the order is created, but it doesn't require an acknowledgment from the downstream systems—that is, it's asynchronous. So, placing the order in the event stream permits the order processing to not be delayed by several hours or more, like it would be with batch processing, but the order is not synchronously processed like a web service request either. As a result, this is referred to as *near real-time* and not *real-time processing*, even though events may get consumed within a few seconds by their subscribers. This asynchronous model serves well for parallel processing and scaling. It has found considerable use these days in web and mobile applications development.

Some of the test cases to think of in an event streaming system are:

- The event structure is an agreement between the publisher and the subscriber, so whenever there is a change to the structure, the entire functional flow has to be tested again.
- Sometimes, backward compatibility to support both old and new event structures has to be tested.
- There may be a requirement to process the events in a specific sequence. For example, an item's shipment cannot be processed

until the warehouse confirms its availability. Since the event processing happens asynchronously, the flow has to be tested.

- A subscriber, on failure, should be able to catch up with new events in the right order.
- If even after several retries there are errors in processing an event, it is moved to a separate queue called the *dead letter queue*, with error details appended to aid in debugging. This flow of events to the dead letter queue needs to be tested.
- What happens when the event stream goes down? How do the publishers and subscribers handle the failure? When and how do they retry?
- Subscribers' performance could be slower than the publisher's, causing bloat in the stream. Hence, their ability to consume events in a timely fashion has to be tested.

As you can see, the different data storage and processing systems each perform a unique role in the larger application ecosystem and thereby demand specific attention when testing. That leads us to the next section, where we will look at an approach to test the four previously discussed data systems.

Data Testing Strategy

In his book *Designing Data-Intensive Applications*, Martin Kleppmann writes:

> *It would be unwise to assume that faults are rare and simply hope for the best. It is important to consider a wide range of possible faults—even fairly unlikely ones—and to artificially create such situations in your testing environment to see what happens.*

I couldn't agree with him more on that, especially when it comes to data testing. Ninety percent of data testing involves thinking about possible faults, unlike functional testing, where the thought process revolves around probable user actions in the application. This will have been

evident from the previous section, where we focused on fault-causing test cases.

With that mindset at its core, a typical data testing strategy can be visualized to comprise four branches, as depicted in Figure 5-3.

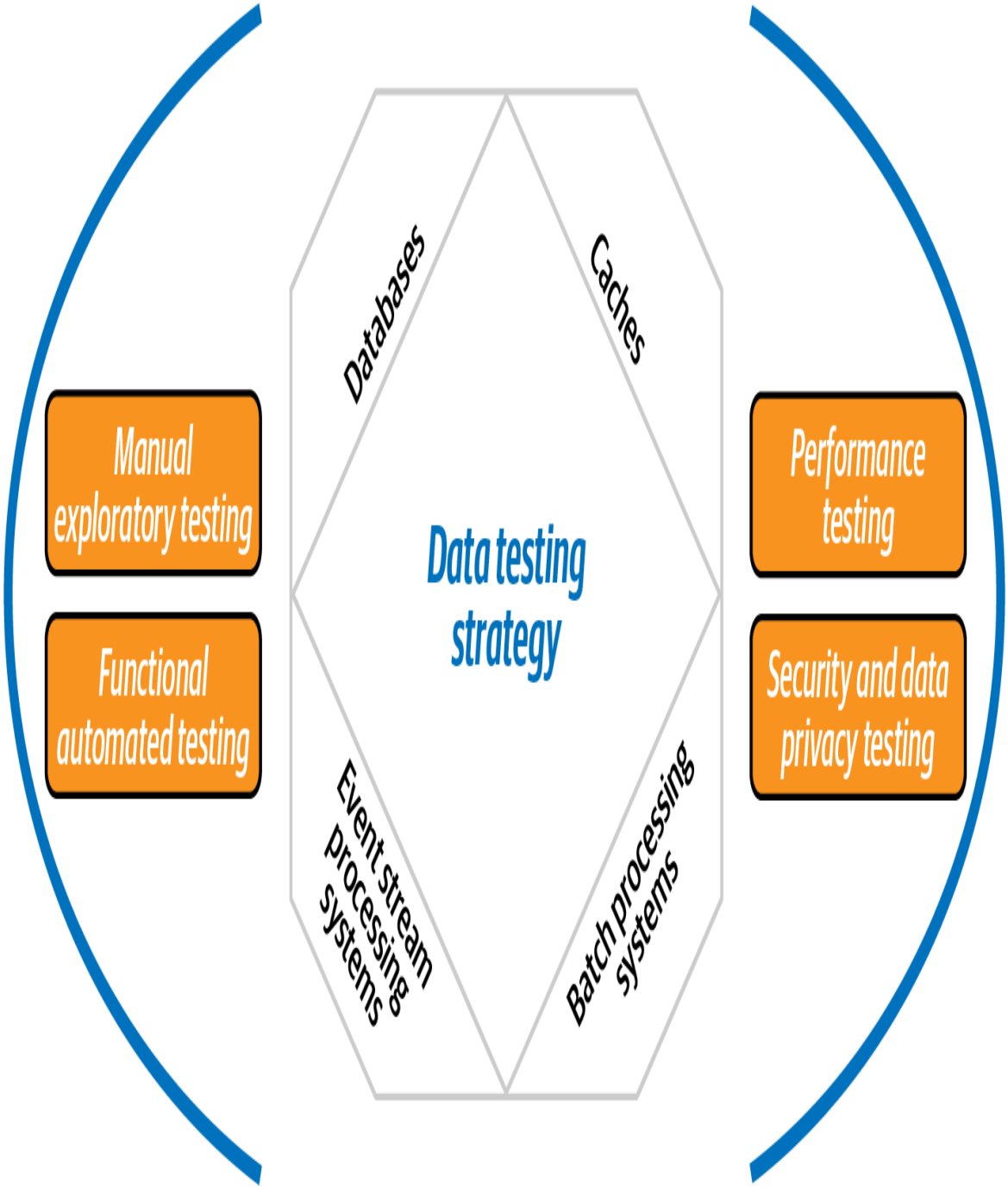


Figure 5-3. A data testing strategy

The branches are:

Manual exploratory testing

Manual exploratory testing can lead to discovering a lot of fault-causing test cases and is very important in data testing. In Chapter 2 you learned about sampling techniques, which are essential for data testing and can be applied mainly when databases and batch processing systems are involved. Also, learning about the specific properties of the data processing tools you are using (such as Apache Kafka, Redis, etc.) will help in identifying the particular aspects of each tool that are worth exploring manually.

You may need to learn additional tools for manual exploration too. For instance, SQL is an essential tool for exploratory testing of relational databases; an introduction to its use is included in this chapter's exercises.

Functional automated testing

In order to get fast feedback on data-related test cases, we need to automate them and integrate them with CI. Starting with unit or integration testing is the recommended approach for all four data systems discussed in this chapter. Some relevant tools are discussed in the following section.

Performance testing

As we have seen, the data storage and processing systems are critical components in any application, and therefore, their performance greatly affects the overall application performance. So, it is important to conduct load and stress testing on all the data storage and processing systems in the application. Backend performance testing is discussed in detail in Chapter 8.

Security and privacy

Data breaches cause huge losses for customers and result in heavy penalties for businesses. Testing for security is one of the most critical aspects of data testing and will be discussed in detail as part

of Chapter 7. Also, there are country-specific data protection laws that enforce the privacy of data by design. These regulations and testing for their compliance will be discussed in Chapter 10.

To summarize the data testing strategy, let's quickly recall a few key pointers from earlier: while testing each of these branches, consider data types and variations, concurrency, the distributed nature of the data and systems, and the possibility of network failures. Also be aware that some data-related test cases, even when tested, may not reveal the underlying bugs as they are heavily dependent on the timing of actions (e.g., the concurrency-related test cases). Remember to discuss those test cases during the analysis phase itself. Next, we'll get hands-on with some exercises.

Exercises

The exercises here introduce a few tools that are essential for database testing, such as SQL and JDBC. We will also explore Apache Kafka and Zerocode, a tool to write automated tests for verifying Kafka messages.

> **NOTE**
>
> As mentioned earlier, most of the data-related test cases should ideally get automated as part of unit integration testing during development. Here, we are discussing tools that are required for testers either in manual exploratory testing or macro-level functional automated testing.

SQL

SQL knowledge is something that you cannot live without when testing the functionalities of an application that includes a database component. You will inevitably encounter scenarios where you need to query the database and ensure the data is intact. When dealing with databases that have many tables, columns, and rows, having the necessary SQL knowledge to quickly filter the data and view what is needed for the test case will save you a ton of frustration. So, if you're not familiar with the

various facets of the SQL language, such as sorting, filtering, grouping, nesting, joining, etc., do give this exercise a try!

For this exercise, you need a relational database. If you have one ready, great! Otherwise, follow the steps in the prerequisites section to set one up.

Prerequisites

Set up a PostgreSQL database on your local machine by downloading the relevant package or installer from the official website. Once installed, start the postgres server using your respective OS-specific commands. For example, if you are a Mac user, open the Terminal and run the following commands:

1. Download PostgreSQL using **`brew install postgresql`**.
2. Start the postgres server using **`brew services start postgresql`**.
3. Open the shell client, `psql`, using the command **`psql postgres`**. The `psql` client will connect to the database server and execute the SQL queries on it. Alternatively, you could choose to use a GUI client such as pgAdmin.

> **NOTE**
>
> When you've completed the exercises, don't forget to stop the database server, for example using the command **`brew services stop postgresql`**.

Workflow

As mentioned earlier, the SQL language is used to operate upon (read, write, update, and delete data in) relational databases. The language is composed of various keywords and functions that make sorting, filtering, and joining data from multiple tables simple enough. I'll show you the set

of queries that are most frequently needed for manual database testing here.

Create

First, create a new table called `items` that stores the item details, such as SKU, color, size, and price. To do this, run the following query from your `psql` client:

```
postgres=> create table items (item_sku varchar(10), color
varchar(3), size varchar(3), price int);
```

This query uses the SQL keywords `create table` to specify the type of operation to perform, and gives a name to the table. Additionally, it elaborates on the column structure with column names, data types (`varchar` and `int`, indicating characters and integers), and a maximum length for each column. The length is specified explicitly for the three character columns and implicitly for the integer column, which will store values up to 4 bytes in size.

> **NOTE**
>
> SQL syntax is generally case insensitive. You may see the keywords written in all capital letters (e.g., `CREATE TABLE`, `VARCHAR`, `INT`). This has no effect on the processing performed, and you can use whichever style you prefer.

Insert

To populate the table with data—in our case, with different item details—run the following query:

```
postgres=> insert into items values ('ABCD0001', 'Blk', 'S',
200),
('ABCD0002', 'Yel', 'M', 200);
```

This query uses three keywords: `insert`, `into`, and `values`. The first two keywords indicate the type of the operation and point to the table in which the insert will be performed (`items`). The values to be inserted are provided within parentheses and should match the column order. You can insert as many rows as needed in the same way. If you attempt to insert data that doesn't fit within the maximum defined column lengths or match the specified data types, the `insert` query will fail. Populate your table with more items with different prices, colors, and size combinations before trying the next set of queries.

Select

The most frequent operation for database testing is the *read*. To read the data from our table, use the following command (your results will vary based on the exact item details you added):

```
postgres=> select * from items;
item_sku | color | size | price
---------+-------+------+-------
ABCD0001 | Red   | S    |   200
ABCD0002 | Blk   | S    |   200
ABCD0003 | Yel   | M    |   200
ABCD0004 | Blk   | S    |   150
ABCD0005 | Yel   | M    |   100
ABCD0005 | Blk   | S    |   120
ABCD0007 | Yel   | M    |   180
(7 rows)
```

Note the keywords `select`, `*`, and `from` in the query. The `*` wildcard symbol says to read all the rows and columns from the table. If you have to select specific columns, then you can specify the column names, separated by commas, instead of `*`.

Filtering and grouping

Most times, tables have lots of rows, and rows have *n* number of columns. You might need to filter the data that is relevant to a particular test case. The following query narrows down the results:

```
postgres=> select item_sku, size from items limit 3;
item_sku | size
---------+------
ABCD0001 | S
ABCD0002 | S
ABCD0003 | M
(3 rows)
```

Here we select just the `item_sku` and `size` columns from the `items` table, and the `limit` keyword will show the top *n* records from the results. Similarly, the keyword `where` is used to define filter criteria based on column values, as in the following example:

```
postgres=> select color from items where size='S';
color
-------
Red
Blk
Blk
Blk
(4 rows)
```

This query has filtered the item records which have a size of S and shows only the color column in those records. The results reveal that the same color is repeated in multiple rows, but it's hard to derive meaning from this initial observation. It may be useful to see a summary of the number of S-sized items in each color. To get this result, the `group by` keyword can be used as follows:

```
postgres=> select color, count(*) from items where size='S'
group by color;
color | count
-------+-------
Blk   |     3
Red   |     1
(2 rows)
```

Note that the `group by` keyword can be used without the `where` keyword as well. It essentially summarizes multiple rows based on the

defined criteria and presents each group as a single row in the query results. We can further filter the grouped results using the `having` keyword, as follows:

```
postgres=> select color, count(*) from items where size='S'
group by color
having count(*)>1;
color | count
-------+-------
Blk   |     3
(1 row)
```

This query filters the groups that have an item count greater than 1. Note that the `having` keyword can only be used along with `group by`.

You'll also have noticed the function `count(*)` in the previous `select` clauses, which does the task of adding the number of records in each group. We will look at some more SQL functions soon.

Sorting

Beyond filtering, SQL also allows the results to be sorted in ascending/descending order based on the values of a single or multiple columns using the `order by` keyword, as shown here:

```
postgres=> select item_sku, color, size from items order by
price asc;
item_sku | color | size
----------+-------+------
ABCD0005 | Yel   | M
ABCD0005 | Blk   | S
ABCD0004 | Blk   | S
ABCD0007 | Yel   | M
ABCD0001 | Red   | S
ABCD0003 | Yel   | M
ABCD0002 | Blk   | S
(7 rows)
```

A query to sort multiple columns in different orders might look like this:

```
postgres=> select * from items order by price asc, size desc;
```

Functions and operators

We saw the `count()` function in some of the earlier examples. The SQL language provides a suite of functions and operators to perform aggregations, comparisons, and other transformations. Some useful functions are `sum()`, `avg()`, `min()`, and `max()`, which can be used in the same way as `count()`. As you might expect, these find the sum, average, and minimum or maximum values. Similarly, operators like `and`, `or`, `not`, and `null` are useful in filtering values. For example, try the following query, which returns black-colored items with size S:

```
postgres=> select * from items where size='S' and color='Blk';
```

Expressions and predicates

We can also use expressions and predicates in SQL. Expressions can be mathematical formulas such as `price+100`, and predicates are logical comparisons, which may result in the values `true`, `false`, or `unknown`. For example, try the following query:

```
postgres=> select * from items where price=100+50 and color is
not NULL;
```

The SQL language will compute the result of the mathematical expression to determine the price value to filter for, and also executes a logical condition, `is not null`, on each of the records' color values to ensure they are not `null`.

Nested queries

We can also nest queries within one another if needed. The subquery can be placed anywhere inside the main query, including in the `where` clause, the `group by` clause, etc. This example query displays the

count of total items and the average price of all items. Note the nested subquery, enclosed in parentheses:

```
postgres=> select count(*), (select avg(price) from items)
from items;
 count |         avg
-------+---------------------
     7 | 164.2857142857142857
(1 row)
```

Joins

Most times, data is spread across multiple tables, and we may need to correlate them to verify a test case. To support querying multiple tables, SQL provides the `join` keyword. It lets us join two tables together based on their common attributes. To try this, create another table called `orders` with `order_id`, `item_sku`, and `quantity` columns and insert multiple rows, as seen here:

```
postgres=> create table orders (order_id varchar(10), item_sku
varchar(10),
quantity int);
postgres=> insert into orders values ('PR123', 'ABCD0001', 1),
('PR124', 'ABCD0001', 3), ('PR125', 'ABCD0001',2);
```

Now we can use the `inner join` keyword to merge the `items` and `orders` tables based on `item_sku`, the common column between the two tables. Try this query to do this:

```
postgres=> select * from orders o inner join items i on
o.item_sku=i.item_sku;
order_id | item_sku | quantity |item_sku | color | size |
price
----------+----------+----------+----------+----------+-------
+-----
PR124    | ABCD0001 |        3 | ABCD0001 | Red   | S    |
200
PR125    | ABCD0001 |        2 | ABCD0001 | Red   | S    |
200
```

```
PR123     | ABCD0001 |         1 | ABCD0001 | Red    | S     |
200
----------+----------+----------+----------+----------+-------
+-----
```

As you can see in the results, the columns from the two tables have been merged as a single row. Note, however, that only the `item_sku`s present in both the tables are included in the merge—the other items from the `items` table are not listed here. There are a couple of other specialities to observe in the query: it uses the `on` keyword to describe the condition to merge on, and it defines aliases for the table names (`o` for `orders` and `i` for `items`) for ease of use. The aliases are reused in the merge condition.

Apart from the inner join, the other commonly used join types are the *left join*, *right join*, and *full outer join*, where the query syntax remains the same and only the keyword changes. A left join takes all the rows in the table mentioned first (on the left) in the merge condition and merges them with the matching rows in the other table. When a row has no match in the other table, the columns are filled with `null` values. A right join works in the reverse order, including all rows from the second table and only matching rows from the first one. A full outer join returns all the rows in both tables, and wherever there are no matching rows, `null` values are shown in the merged results.

Such join queries can be further extended with filtering and sorting keywords to view the appropriate results.

Update and delete

The remaining two CRUD operations—update and delete queries—are presented here. If you want to update a column value and see the results, use the keywords `update` and `set`, as seen here:

```
postgres=> update items set color='BK' where color='Blk';
```

Similarly, if you would like to delete some of the records that you created for testing purposes, use the `delete` keyword as shown here:

```
postgres=> delete from items where price=180;
```

> **NOTE**
>
> The SQL language is far richer than this short introduction suggests. As mentioned earlier, the list of commands presented here was curated mainly to assist in manual database testing. If you are interested in learning more about SQL, explore O'Reilly's *SQL Pocket Guide*, by Alice Zhao.

JDBC

JDBC stands for Java Database Connectivity. It provides a set of Java APIs to connect to relational databases and execute SQL queries on them. JDBC can be combined with any of the UI or API automation test suites mentioned in the previous chapters to directly verify the data in the database. There are various database-specific JDBC drivers that can be imported into a project as a Maven dependency; for example, we can use the PostgreSQL JDBC driver to connect to the PostgreSQL DB we created earlier and verify the database records.

We'll be using the following three simple JDBC APIs in our test to connect to the DB and execute queries:

```
// To connect to the database
connection =
DriverManager.getConnection("jdbc:postgresql://host/database",
"username", "password");

// To execute a SQL query
Statement statement = connection.createStatement();
ResultSet results = statement.executeQuery(String query);

// To close the connection after use
results.close();
statement.close();
```

ABIDE BY THE PYRAMID

The DB-related test cases should be added as unit and integration tests and not as UI or API tests, for reasons discussed in Chapter 3. Also, the ideal way to create the test data required for the macro-level automated tests is to use the application APIs. This ensures that even if the database schema changes, the tests don't need to worry about it, as the application code underneath the APIs will handle the changes. The APIs themselves should rarely change, as that would hamper integration with clients.

Having said that, there could be situations where you want to verify a functionality end to end, which might require verifying the downstream systems as well. If your downstream systems are legacy systems, you may not have APIs to verify the functionality. The only choice will be to directly connect to the database in such cases. For instance, in the ecommerce application example, to verify the successful end-to-end order processing flow, we might have to first create an order in the ecommerce application and then verify the fulfillment system's DB directly, assuming it is a legacy system and there are no APIs. This exercise is specifically meant to assist you in such situations.

Setup and workflow

Let's extend the Java–Selenium WebDriver automation framework created as part of Chapter 3 to add a test to fetch an order from the `orders` table and assert on the `order_id` and `quantity` values. The steps for this are listed here:

1. Add the PostgreSQL JDBC driver as a dependency in the POM file.
2. Create a new test class file under the `tests` package called *DataVerificationTest.java*.
3. Example 5-1 shows the `DataVerificationTest` class, where the connection to the PostgreSQL DB is initiated before every test and closed after the test run. The test specifies the SQL query it intends

to execute and gets the DB records. Finally, it uses TestNG assertions to validate the data returned.

Note the JDBC URL used in the test to connect to the database. It uses *localhost* as the hostname (since the DB is on your local machine); in this example the database name is *postgres*, and you'll need to substitute in your username and password. You can run the command `\l` from the `psql` client to list the existing databases, and `\dt` to see all the tables and their owners.

Example 5-1. Test involving JDBC connection to the PostgreSQL DB

```
package tests;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;
import static org.testng.Assert.*;
import java.sql.*;

public class DataVerificationTest {

    private static Connection connection;
    private static ResultSet results;
    private static Statement statement;

    @BeforeTest
    public void initiateConnection() throws SQLException {
        connection = DriverManager.getConnection(
            "jdbc:postgresql://localhost/postgres",
            "newuser", null);
    }

    public void executeQuery(String query) throws
SQLException {
        initiateConnection();
        statement = connection.createStatement();
        results = statement.executeQuery(query);
    }

    @Test
    public void verifyOrderDetails() throws SQLException {
        executeQuery("select * from orders where
item_sku='ABCD0006'");
        System.out.println(results);
```

```
        while (results.next()){
            assertEquals(results.getString("Quantity"),
"1");
            assertEquals(results.getString("order_id"),
"PR125");
        }
    }

    @AfterTest
    public void closeConnection() throws SQLException {
        results.close();
        statement.close();
    }
}
```

4. You can run the test from the command line (using **`mvn clean test`**) or from your IDE to see the results. Remember to start the `postgres` server before the test run.

That's how simple it is. You can also choose to abstract the database connection–related methods into a separate `utils` class for reusability purposes.

Apache Kafka and Zerocode

Kafka is an open source distributed streaming platform. It enables multiple producers and consumers (or publishers and subscribers, to use the terminology from our earlier discussion of event streaming) to exchange information through a common stream. The LinkedIn team originally developed Kafka as a solution to their struggle with aggregating information from multiple systems and producing meaningful metrics; it allowed them to scale to process trillions of messages and to consume petabytes of data every day.

> **NOTE**
>
> In case you were wondering, the tool is indeed named after Franz Kafka, the famous author of several surrealistic works, including the short story "The Metamorphosis"—simply because the principal engineer who led the development efforts was a fan of his.

Let’s explore this tool a little to give you an idea of what it is and how to test it. Figure 5-4 depicts a sample Kafka system, composed of a server (the broker), producers, and consumers.

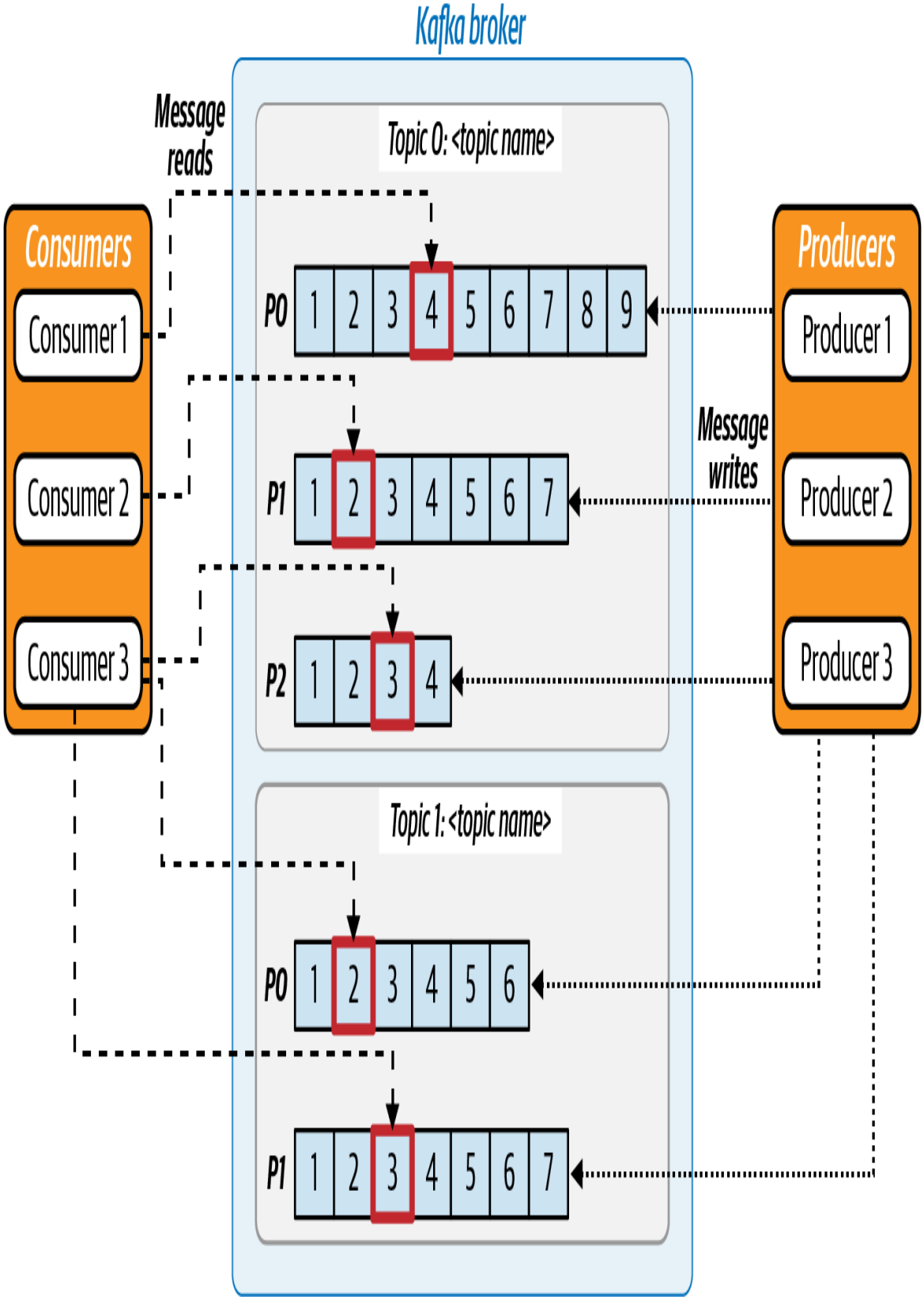


Figure 5-4. Apache Kafka workflow

To help you understand the workflow, let's take a look at some key terminology:

Messages

Events are referred to as messages in Kafka, and a message is composed of a unit of information, such as an order's details. Messages are stored on disk directly, ensuring durability.

Topics

Messages are organized under topic names. For example, in our ecommerce example, order details could be posted to the topic `orders`. There could be many messages with the same topic name, sent from different producers. This allows the aggregation of data from multiple producers and also makes it easier for the consumers to identify which messages are relevant to them.

Partitions

Messages in each topic are typically stored across multiple partitions, as seen in Figure 5-4. The messages are appended to a given partition, and therefore partitions enable the possibility of processing the messages in a given sequence if the workflow mandates it. In order to manage the distribution of messages to the right partitions, the producers append metadata called *keys* within the messages. For example, if a set of transactions are tied to a customer ID, then that is used as a key to send them to the right partition and therefore provide the ability to process the transactions in the right sequence.

The partition-related features are the way Kafka achieves its performance and scalability. Partitions can also be replicated for redundancy to prevent loss of information due to unexpected failures.

Offset

Each consumer may read from many topics, and it has to keep track of which messages it has read from each topic so that it doesn't end up reprocessing messages. For this purpose, consumers use the *offset* number placed within the messages. An offset is a

continuously growing integer created by Kafka and added as metadata to each message at the time it is produced. So, whenever there is a consumer failure, it starts from the last offset it successfully processed. Also, to onboard a new consumer, it simply needs to start from an earlier offset number to catch up. This feature is also called message replaying.

Brokers

The Kafka server, referred to as a broker, mediates between the producers and the consumers. The broker gets messages from the producers, appends offsets to the messages, and stores them on disk arranged by topic. Similarly, it responds to consumer requests, fetching the right messages from the right partitions.

Schemas

Although Kafka considers messages as just collections of bytes of data, to facilitate collaboration between producers and consumers they need to agree on a data format and structure, referred to as a *schema*. For example, we saw the schema of the `orders` topic earlier, in Figure 5-2: it's composed of `order_id`, `item_sku`, and `quantity` fields.

Kafka supports messages in JSON, XML, and Apache Avro formats, among others. The data structure of the messages cannot change without a change in both the consumer's and the producer's code. Basically, there needs to be backward and forward compatibility when there are different schema versions. You can compare these to a web service's request and response contracts. These schema versions are stored in a separate component called the *Schema Registry*, which assists in performing compatibility checks and ensures the contract between the producer and consumer is not broken when the schema evolves.

Retention

Kafka retains the messages for a short period of time before deleting them. The default setting is to persist them for seven days or until the

partition size reaches 1 GB. This value can be configured per message as well, ensuring messages with different retention needs are blessed with an appropriate lifetime.

That should be a sufficient primer to get started and see how things work firsthand.

Setup

You can follow the steps described here to install Kafka on your local machine. Since the goal here is to familiarize you with the Kafka ecosystem from a testing point of view, we'll abstract its installation details and use Docker containers.

A BRIEF INTRODUCTION TO DOCKER

Let's say you are developing an application that requires a series of tools to be installed: a PostgreSQL DB, Kafka, Nginx, and so on. A common approach to share the installation details with your new team members is to hand them a neatly written document pointing to the right versions of the software to be installed and their specific configurations. This often becomes a bottleneck, as each team member could be using a different version of operating system, face installation issues due to incompatibilities with existing tools, and so on. It may take them a few days to resolve the issues and get their machine set up. An easier approach is to package all the application-relevant setup as a *container* using Docker and distribute it to your team members. Then, they just have to install Docker and download the container, which will bring up the application with a single command.

Docker essentially isolates the infrastructure and the application software. If you run the application in a container, it is equivalent to having an isolated machine inside your host machine with application-specific software. The key advantage is that this machine is shippable, unlike your host machine. This method is especially useful when creating QA and production environments, so you can enjoy the benefits of using the exact same application binaries everywhere.

An important point to add before you install Docker is that it is free only for personal use. You may need to adhere to your company's laptop policies when installing Docker on your work laptop.

To install Kafka using Docker, follow the steps here:

1. Install Docker Desktop by getting the OS-specific binaries from the official website. After you complete the installation procedure, the Docker Desktop application will show up with a Start prompt.
2. On clicking Start, it will suggest that you run the command `docker run -d -p 80:80 docker/getting-started`. Try running

that from your terminal to ensure Docker is accessible from the command line. The command basically downloads a sample `hello-world` container and ensures that it is mapped to port 80 of the host machine.

3. You will see the `hello-world` container running in the Docker Desktop application. Stop the container once it is up by clicking the Stop button next to it.
4. I'll introduce Zerocode shortly, but for now, clone the Zerocode Docker Factory repository using the `git clone` command. (Refer to Chapter 4 for Git instructions.) This repository has all the necessary configuration files to set up Kafka and its dependencies, and will enable you to write automated tests using the Zerocode tool.
5. To finish the Kafka installation, use the `cd` command to go to the folder *zerocode-docker-factory/compose* in your terminal and run the following command:

```
$ docker-compose -f kafka-schema-registry.yml up -d
```

   Once you see the green "done" in the output of the command, run `docker ps`, which should show a bunch of containers to be up.

With that, you have Kafka and its dependencies running successfully on your machine! We can now write automated tests for producing a message and consuming it using Zerocode.

Workflow

Zerocode is an open source tool that enables writing declarative-style automated tests for REST APIs, SOAP APIs, and Kafka systems. The test cases can be created as JSON or YAML files and wired as normal JUnit tests. You can write tests to hit an API and verify the messages produced by it in Kafka, and vice versa. You can also write tests to produce new messages and verify the message structure by consuming

them. The tool's main value-add is the abstraction layer that hides all the Kafka APIs necessary to perform those operations and the serialization/deserialization code to read different types of responses.

Let's use Zerocode to push an order message with a JSON structure containing `order_id`, `item_sku`, and `quantity` values to the local Kafka broker you just created by writing declarative test cases. The message will be posted to the topic named `orders`. We'll then consume the message, just as the fulfillment system would do, and verify the order details by writing declarative test cases again. This will give you an idea of what the messages look like and the details to test for.

Here is a step-by-step walkthrough of creating tests using Zerocode:

1. Create a new Maven project in IntelliJ, say *KafkaTesting*, with the Java 1.8 JDK. Refer to Chapter 3 if you need assistance with this step.

2. Add JUnit 4 and the *zeroCode-tdd* library in your *pom.xml* file, as seen in Example 5-2.

   Example 5-2. pom.xml file for Kafka testing with Zerocode

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
              http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>KafkaTesting</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
    </properties>

   <dependencies>
        <dependency>
            <groupId>org.jsmart</groupId>
```

```
            <artifactId>zerocode-tdd</artifactId>
            <version>1.3.28</version>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

3. Create a new folder called *kafka_servers* under *src/main/resources*.
4. In that folder, add three properties files: *broker.properties*, *producer.properties*, and *consumer.properties*, with the contents shown in Example 5-3. These files specify the details about each of the broker, producer, and consumer containers running on your local machine.

Example 5-3. Kafka testing properties files

```
// broker.properties

kafka.bootstrap.servers=localhost:9092
kafka.producer.properties=kafka_servers/producer.properties
kafka.consumer.properties=kafka_servers/consumer.properties
consumer.commitSync = true
consumer.commitAsync = false
consumer.fileDumpTo= target/temp/demo.txt
consumer.showRecordsConsumed=false
consumer.maxNoOfRetryPollsOrTimeouts = 5
consumer.pollingTime = 1000
producer.key1=value1-testv ycvb

// producer.properties

client.id=zerocode-producer
key.serializer=org.apache.kafka.common.serialization.StringS
erializer
value.serializer=org.apache.kafka.common.serialization.Strin
gSerializer

// consumer.properties

group.id=consumerGroup14
```

```
key.deserializer=org.apache.kafka.common.serialization.Strin
gDeserializer
value.deserializer=org.apache.kafka.common.serialization.Str
ingDeserializer
max.poll.records=2
enable.auto.commit=false
auto.offset.reset=earliest
```

5. Now create another folder under *src/main/resources* called *test_cases*.

6. Add a new JSON file called *orderMessages.json* in the *test_cases* folder. This is where you will write your tests in JSON format.

7. Let's first write a test to produce a message with order details and assert on the metadata received as a response from the broker. Usually, we get a status value (just like with APIs), a partition number, and the topic name in the response. The *orderMessages.json* file (i.e., the producer test case) is shown in Example 5-4.

   Example 5-4. A sample producer test case in Zerocode

```
{
  "scenarioName": "Produce an order details JSON message for
the orders topic",
  "steps": [
    {
      "name": "produce order messages",
      "url": "kafka-topic:orders",
      "operation": "produce",
      "request": {
        "recordType" : "JSON",
        "records": [
          {
            "value": {
              "order_id" : "PR125",
              "item_sku" : "ABCD0006",
              "quantity" : "1"
            }
          }
        ]
      },
      "verify": {
        "status": "Ok",
```

```
        "recordMetadata": {
          "topicPartition": {
            "partition": 0,
            "topic": "orders"
          }
        }
      }
    }
  ]
}
```

8. Next, you need to wire the JSON test case to a JUnit test. To do this, create a new file called *ProducerTest.java* under *src/test/java* and add the code in Example 5-5 to it.

   Example 5-5. The `ProducerTest` class

```
// ProducerTest.java

import org.jsmart.zerocode.core.domain.JsonTestCase;
import org.jsmart.zerocode.core.domain.TargetEnv;
import org.jsmart.zerocode.core.runner.ZeroCodeUnitRunner;
import org.junit.Test;
import org.junit.runner.RunWith;

@TargetEnv("kafka_servers/broker.properties")
@RunWith(ZeroCodeUnitRunner.class)
public class ProducerTest {

    @Test
    @JsonTestCase("testCases/orderMessages.json")
    public void
verifySuccessfulCreationOfOrderDetailsMessageInBroker()
        throws Exception {

    }
}
```

   Here, the `@TargetEnv` attribute tells the test where to find the broker configuration, the `@RunWith` attribute binds Zerocode with JUnit, and the `@JsonTestCase` attribute points to the JSON file or the test case to be run as part of the test.

9. You can run the test from the IntelliJ IDE by right-clicking the green button next to the `@Test` attribute. Once it passes, you can find the

messages created in your local Kafka instance by running the following commands in your terminal:

```
// to get inside the container
$ docker exec -it compose_kafka_1 bash

// to see the records as a consumer
$ kafka-console-consumer --bootstrap-server kafka:29092
  --topic orders --from-beginning
```

Congratulations on writing your first Kafka test!

10. You can also try a consumer test to validate the message content by adding the JSON shown in Example 5-6 to the `steps` array in the *orderMessages.json* file.

Example 5-6. A consumer test using Zerocode

```
{
      "name": "consume order messages",
      "url": "kafka-topic:orders",
      "operation": "consume",
      "request": {
        "consumerLocalConfigs": {
          "recordType": "JSON",
          "commitSync": true,
          "showRecordsConsumed": true,
          "maxNoOfRetryPollsOrTimeouts": 3
        }
      },
      "assertions": {
        "size": 1,
        "records": [
          {
            "value": {
              "order_id" : "PR125",
              "item_sku" : "ABCD0006",
              "quantity" : "1"
            }
          }
        ]
```

```
    }
}
```

As part of the consumer test, we are validating the number of messages received with the topic name `orders` and the message content. Zerocode allows adding validations to other parts of the message content, such as offset, partition, keys/values, etc., in the same declarative style.

You can find more information about this style of Kafka testing in Zerocode's official documentation.

Additional Testing Tools

Apart from the tools discussed in the exercises, this section aims to shed light on a few others that are commonly used in the data testing space to give a wider perspective.

Test Containers

A prerequisite for some of the previous exercises was setting up an actual PostgreSQL database on your local machine and creating the relevant table structures so that your tests could connect to the database and verify the data. If you had access to the application database in a test environment, the tests could connect to it instead. An alternative approach would be to use the Testcontainers tool, which provides containerized throwaway database instances. It is especially useful for developers who want to run unit and integration tests on their local machines without needing to set up a proper database as a prerequisite. Even if they had a database to use, the probability that it would be polluted due to active feature development is fairly high. Testcontainers bridges this gap by bringing up a fresh database instance at the same stable state every time, a vital necessity for tests to succeed.

In order for the application code to use a Testcontainers database, the JDBC URL has to be tweaked slightly. Alternatively, APIs can be used to instantiate database instances wherever necessary. For example, to

bring up a containerized PostgreSQL database before a test run, the following lines of code can be added as part of the test setup:

```
PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
(A_sample_image);
postgres.start();
```

You can operate further on the container object within the test as required. Testcontainers provides a variety of database instances, including MySQL, PostgreSQL, Cassandra, MongoDB, and so on. It brings with it the advantage of working with JUnit and the facility to launch the database with an init script, function, or file before the test run begins.

> **NOTE**
>
> Testcontainers allows you to spin up many other kinds of containers apart from databases (Kafka and RabbitMQ containers, containers that include web browsers, and more) with a single line of code. It also provides a generic framework to import any other custom containers you may need for test runs. See the documentation for complete details.

PORTABILITY TESTING

Portability refers to the ability of an application to switch out its internal components without much rework. Database portability sometimes becomes an essential requirement in product development. For instance, let's say a product team develops an order management system (OMS). The OMS as a product is expected to be pluggable between any given ecommerce platform currently used by a business and its internal legacy systems. In such cases, database portability—in other words, being able to work with different types of databases, such as Oracle or PostgreSQL—can become a key selling point for the OMS product, as it means the business's IT team doesn't have to learn a new tool. In a case like this, the OMS development team could use Testcontainers to verify the applications functionalities against different databases as part of the unit and integration testing.

Deequ

Earlier in this chapter, we spoke about vendors sending their updated catalogs to our ecommerce application as files, and batch jobs transforming them into database records. In my experience working on retail projects, I have seen that even leading retailers in the US and Europe still have legacy systems that use COBOL and mainframes to manage their data. They send millions of records containing data about their product catalogs as files every day to update item availability, and batch jobs run overnight to insert them into the application's database. The records in the files often contain outdated or incorrect data, null values, missing keys, and many other inconsistencies. It's a nightmare scenario indeed as this data, once populated into the database, also corrupts the application. Deequ, an open source unit testing tool, can help in situations like this.

Deequ was originally built by Amazon, which continues to use it for testing the quality of data produced by its internal systems. The tool is built on top of Apache Spark, a large-scale distributed data processing

platform. As mentioned earlier, Spark, among many other things, can be used for batch processing of large-scale data. The Deequ library fits in here to unit test the data before and after batch processing. For example, we can add unit tests using Deequ to verify the expected data types, absence of null values, presence of only certain allowed values, and so on. In our ecommerce application, when the files get loaded to Spark for batch processing, these unit tests can be run against them first. As part of the test run, any records in the file that don't meet the requirements will get quarantined for further analysis. Also, a set of unit tests can be written to test the transformed data after batch processing in order to reveal errors in the batch processing job itself.

A sample test using Deequ might look like the following:

```
val verificationResult = VerificationSuite()
  .onData(data)
  .addCheck(
    Check(CheckLevel.Error, "unit testing vendor files")
      .hasSize(_ > 100000) // we expect more than a million
rows
      .isComplete("item_sku") // should never be NULL
      .isUnique("item_sku") // should not contain duplicates
      // should only contain the values "S", "M", "L", "XL"
      .isContainedIn("size", Array("S", "M", "L", "XL"))
      .isNonNegative("price") // should not contain negative
values
    )
    .run()
```

Deequ generates different metrics as a result of test runs to indicate the quality of a parameter across all the records. For example, the results may show that 90% of the `price` values are acceptable but 10% aren't, indicating that they need to be fixed. There are other related facilities provided by the tool as well, such as anomaly detection on data quality metrics, automatic suggestions for validations, and so on.

> **NOTE**
>
> TensorFlow Data Validation and Great Expectations are two other data validation tools similar to Deequ.

With that, we have completed our walkthrough of the data testing space. We covered a lot of ground here: the different types of data storage and processing systems, the new test cases they add to the functional testing portfolio, a crash course in SQL, and some hands-on exercises with a few useful tools to get you started in your own projects. The data testing skill is a much-needed skill in the industry today.

Key Takeaways

Here are the key takeaways from this chapter:

- Data is at the core of any online application these days, and application features, branding, marketing, and design aspects revolve around it. If data integrity is violated, companies face damage to their reputation and unsatisfied customers, making data integrity priceless. Therefore, data testing is one of the primary testing skills.
- The data testing skill encompasses knowledge around different data storage and processing systems, including their unique properties, the specific test cases imposed by them, and the testing methods and tools necessary to perform automated and manual exploratory data testing.
- This chapter discussed four commonly used data storage and processing systems: databases, caches, batch processing systems, and event stream processing systems. We touched upon their distinctive properties and the new test cases demanded by each of them.
- A typical data testing strategy should include manual exploratory testing, automated functional testing, performance testing, and

testing for data security and privacy. As a standard, when performing data testing you should include test cases around the data and its variations, its distributed nature, concurrency factors, and network failures.

- A fault-finding mindset is imperative for data testing, as 90% of data testing is about faulty test cases. This is in contrast to functional testing, where the thought process revolves around user actions.

---

1 For an overview of different data models and query languages, see Chapter 2 of Martin Kleppmann's *Designing Data-Intensive Applications* (O'Reilly).

Chapter 6. Visual Testing

Visual quality amplifies brand value!

The visual quality of an application shapes a customer's first impression. When the look and feel are palatable, they tend to explore the application further. Imagine a customer has to make a payment online, and the Continue button looks like it does in Figure 6-1. Do you think they will have the trust to proceed further? I very much doubt it. I would choose a competitor's website to get my job done rather than risk losing my money!

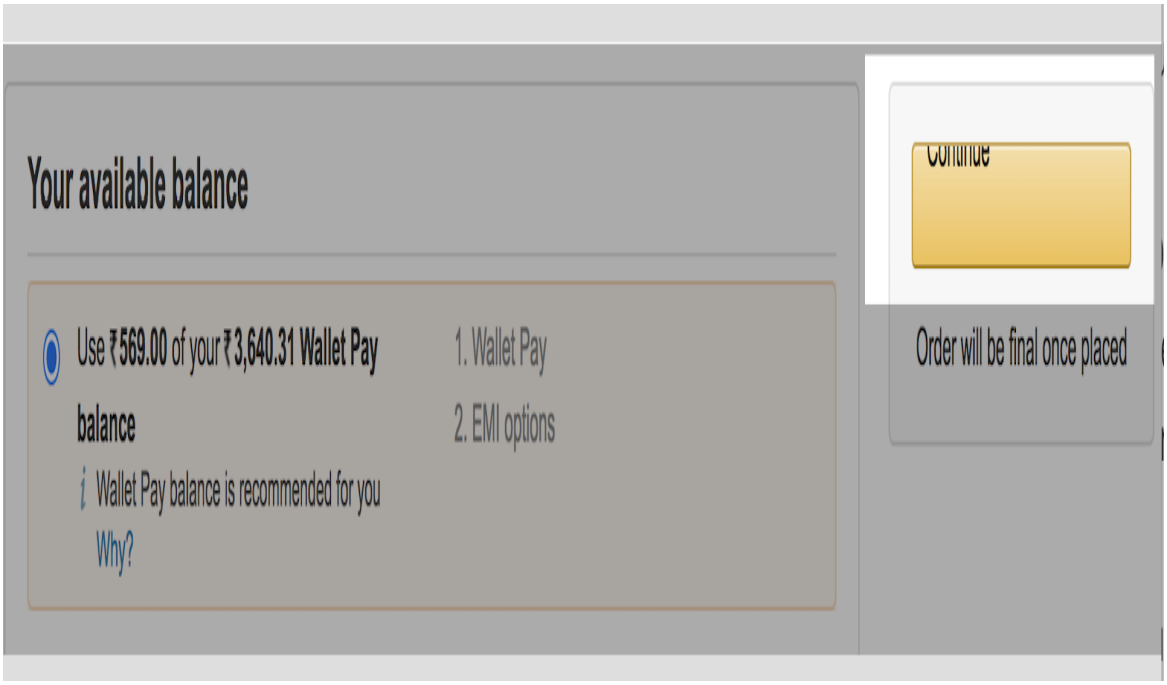


Figure 6-1. A sample payment UI where the button text is cropped

With businesses spending a lot on strategies to acquire more customers through advertisements, freebies, campaigns, and more, software teams forgetting to focus on visual quality is equivalent to building a luxurious house and forgetting to paint it. An application's visual quality is a critical factor that takes the business closer to the

customer and helps gain their affinity—and the customer's affinity directly amplifies the brand value. Visual testing is all about validating an application's visual quality, using both manual and automated testing methods.

> **NOTE**
>
> Note that validating the user experience (UX) design falls under usability testing, not visual testing, and is discussed in Chapter 10.

Visual testing involves confirming whether the application sticks to the *expected* appearance as per the design in terms of each element's size and color, the relative positioning of elements, and similar visual attributes across devices and browsers. This chapter will provide a compact overview of visual testing with a focus on mandatory project/business-specific use cases, along with practical exercises using tools like Cypress and BackstopJS. A peek at a new AI-powered automated visual testing tool, Applitools Eyes, is included as well. In addition, we will take a holistic look at the frontend testing landscape and explore how the different frontend testing types apart from visual testing cumulatively contribute to validating an application's visual quality.

Building Blocks

We'll begin with an introduction to visual testing and the different methodologies, then do a cost-benefit analysis to determine when it might become critical to a project.

Introduction to Visual Testing

Many software development teams, even today, rely heavily on manual eyeballing and UI-driven automated testing to verify the

visual quality of an application. Though this may be sufficient for some applications, it is essential to understand the trade-offs associated with the approach.

First of all, we should agree that human eyes can't notice pixel-level changes, and we can only achieve a certain level of precision with manual eyeballing. For example, it's pretty easy to miss details like curved edges on buttons, or if the logo is shifted up or down by a few pixels. In fact, a research study in 2012 found that changes in up to a fifth of an image's area can regularly go unnoticed by human viewers. This phenomenon, termed *change blindness*, has nothing to do with defects in our vision—it's purely psychological. So, you can imagine how small changes in an application could easily go unnoticed every day in manual testing. Also, let's not forget the time and effort that would be required to manually test the visual quality of the application on a multitude of browsers, devices, and screen resolution combinations. Clearly, we need some automation here.

But when it comes to automated UI-driven functional tests, although they partially contribute to validating the visual quality, it may not be enough as they do not check the "look and feel" of the elements; they simply identify an element by its locator, such as an ID or XPath, and check if it behaves functionally as expected. For example, the UI-driven tests for Figure 6-1 would have passed because the Continue button existed as per its locator with the expected label, and on clicking, it would have taken the user to the next page successfully. You can't blame the test in a case like this, as it meets its purpose of validating the end-to-end functional user flow. Another caveat with using UI-driven functional tests for visual testing is that you can't add tests to assert the presence of every element on every page of the entire application, as that would make their execution much slower and add a lot of maintenance effort.

To help us overcome these challenges, we now have mature automated visual testing tools, just like automated functional testing tools. Indeed, these tools have existed for a while and have adopted

various methodologies to perform visual testing, and they have become more stable and easier to use over time. The following are some of the techniques the existing tools employ to perform visual testing:

- Requiring us to write code to verify the CSS aspects of the elements (e.g., a test to verify if `border-width=10px`)
- Analyzing static CSS code to find browser incompatibility issues with the UI elements
- Using AI to recognize changes on the page, just like human eyes would
- Taking a screenshot of the page and comparing it pixel by pixel against an expected base screenshot

The last of these methods is the one most commonly used in visual regression testing today. For this reason, it is sometimes referred to as *screenshot testing*. Some open source tools that do this kind of visual testing are PhantomJS and BackstopJS. There are paid tools as well, such as Applitools Eyes and Functionize, which are AI-powered. After a manual comparison of the application against the UX design, you can use such tools to automate visual testing to help you catch visual bugs, just as automated functional testing helps you catch functional bugs. Over the course of iterative development, visual tests will give you continuous feedback on the application's visual quality.

An important point to note with automated visual tests is that they can become flaky in an iterative development process when you don't add them at the right stage. For example, suppose your team has decided to play the login functionality as part of two user stories: one where the bare-bones functionality is laid out and the second to finesse the functionality and look and feel. Although adding UI-driven functional tests as part of these two user stories makes sense, adding visual tests as part of the first story might not add value. So,

as part of iteration planning, include visual testing efforts clearly in only the relevant user stories.

Project/Business-Critical Use Cases

We discussed why adding automated visual tests has become important, but it may not be compulsory for all applications. A major consideration is cost. Testing costs are cumulative. In any project, first there is the cost of developing and maintaining the UI-driven functional tests, which are an absolute requirement for all applications. In addition to this, we have the cost of developing and maintaining the visual tests, even if the two types of tests can be combined into a single test suite. So, take the cue from the nature of your application to decide if automated visual testing is a must-have or a nice-to-have. For example, for an internal application that will be used only by a few admins, it's probably not worth spending time and effort creating automated visual tests; manual visual testing will be quite sufficient. However, there are a few use cases, such as the following, where automated visual testing may provide enough value to justify the cost:

- When you are developing a business-to-customer (B2C) application, the visual quality will be a critical quality attribute. Therefore, you will need continuous feedback on this aspect of the application during development. For example, while developing a global ecommerce website that has a large number of components on each page, you need continuous feedback on visual quality, just as you receive feedback on the functionality using UI-driven tests. In cases like this, unless you're just developing a rapid prototype to assess the market needs and plan to rework your application's design later, automated visual tests will help you build a stable application.
- When you have to support your application across multiple browsers, devices, and screen resolutions, automated visual

tests will help you with the massive load of regression testing.

Figure 6-2 shows web usage statistics across devices, browsers, manufacturers, OSs, and screen resolutions as of March 2022, according to gs.statcounter.com. As you can see, there are more mobile than desktop users. Chrome accounts for the largest share of the browser market, followed by Safari. Among OSs, Android, Windows, and iOS are substantial players. Testing the visual quality of your app across all these combinations could easily become a 24/7 job, and automated visual testing will save significant effort for your team.

Mobile vs. desktop vs. tablet market shares	
Mobile	56.45%
Desktop	41.15%
Tablet	2.40%

OS market share	
Android	41.56%
Windows	31.15%
iOS	16.85%
OS X	6.30%

Browser market share	
Chrome	64.53%
Safari	18.84%
Edge	4.05%
Firefox	3.40%
Samsung Internet and Opera	~5%

Device vendor market share	
Samsung	28.22%
Apple	27.57%
Xiaomi	12.24%
Huawei	6.53%
Oppo	5.25%

Screen resolution market share			
1920x1080	9.27%	360x800	5.35%
1366x768	7.32%	1536x864	4.05%

Figure 6-2. Statcounter data on global device, browser, vendor, OS, and screen resolution usage

- Usually, enterprises that own suites of applications have a centralized team that develops the UI components, and multiple teams reuse these components, often referred to as *design systems*. For example, UI components like header navigation panels with elements like “FAQs,” “Contact us,” and “Share to Social Media” will be developed by a single team and reused across the whole suite of applications. Visual testing at the component level becomes a necessity in those cases, as any flaw in these standard components will percolate through the entire suite.
- Sometimes an application is rebuilt completely to improve scalability and other quality aspects, but the expectation is to keep the user experience as-is because the customers have become familiar with it. Writing visual tests can serve as a safety net for teams working on such an application.
- Similarly, visual tests will come in handy when you do significant refactoring of an existing application. For example, improving the frontend performance might require considerable reorganization of UI components. Automated visual tests will give the team great confidence in such a scenario.
- When an application is scaled to reach audiences in different countries, localization features such as a different look and feel per region and native-language text get incorporated. These changes might affect the page layout. In such cases where multiple versions must be tested, automating the visual tests can be a big help.

In summary, you should consider factors like the impact on the customer, the type of work that needs to be done, the team’s confidence, and the amount of effort manual testing would require

when deciding whether your application needs automated visual testing. If it is required, try to balance your frontend testing strategy with minimal visual tests for only the most critical flows. The next section introduces the different types of tests that such a strategy might incorporate.

Frontend Testing Strategy

Automated visual testing can yield the right benefits only when balanced proportionally with other types of frontend tests. Knowing what the different pieces of the frontend testing jigsaw puzzle look like will help you fit them together as required by your application. You'll also notice that some of the other frontend test types contribute to visual testing themselves. You should take this into account when planning the visual testing strategy for your application. It is also crucial to understand where and how automated visual tests fit in, so that teams don't propose them as a solution to other unrelated problems. For example, obviously you don't have to add visual tests for every kind of error message that appears on the page—that's the UI unit tests' job. So, let's zoom out and take a look at the overall frontend testing strategy.

A web application's frontend code comprises three major parts: the HTML code that defines the basic page structure, the CSS code that styles the elements on the page, and the scripts that dictate the behavior of those elements. Another significant component is the browser that renders this code. Most of the newer browsers follow standards for rendering elements. As a result, frontend development frameworks can provide built-in support for various browsers. This means elements or features built using these frameworks are guaranteed to render correctly in the major browsers, but you might have to check for cross-browser compatibility issues when you use features that the frameworks have not tested across old and new browsers.

To validate the different parts of the frontend code, as with the backend code, we can use various types of micro- and macro-level tests. It is usual practice for the developers and testers to own these frontend tests collectively. Figure 6-3 shows how the different micro- and macro-level frontend tests can be employed throughout the development process to get faster feedback—in other words, the figure throws light on shift-left frontend testing implementation. Although we discussed the basics of some of these test types in Chapter 3, in this section we'll explore them from the frontend code perspective and look at how they can contribute to visual testing.

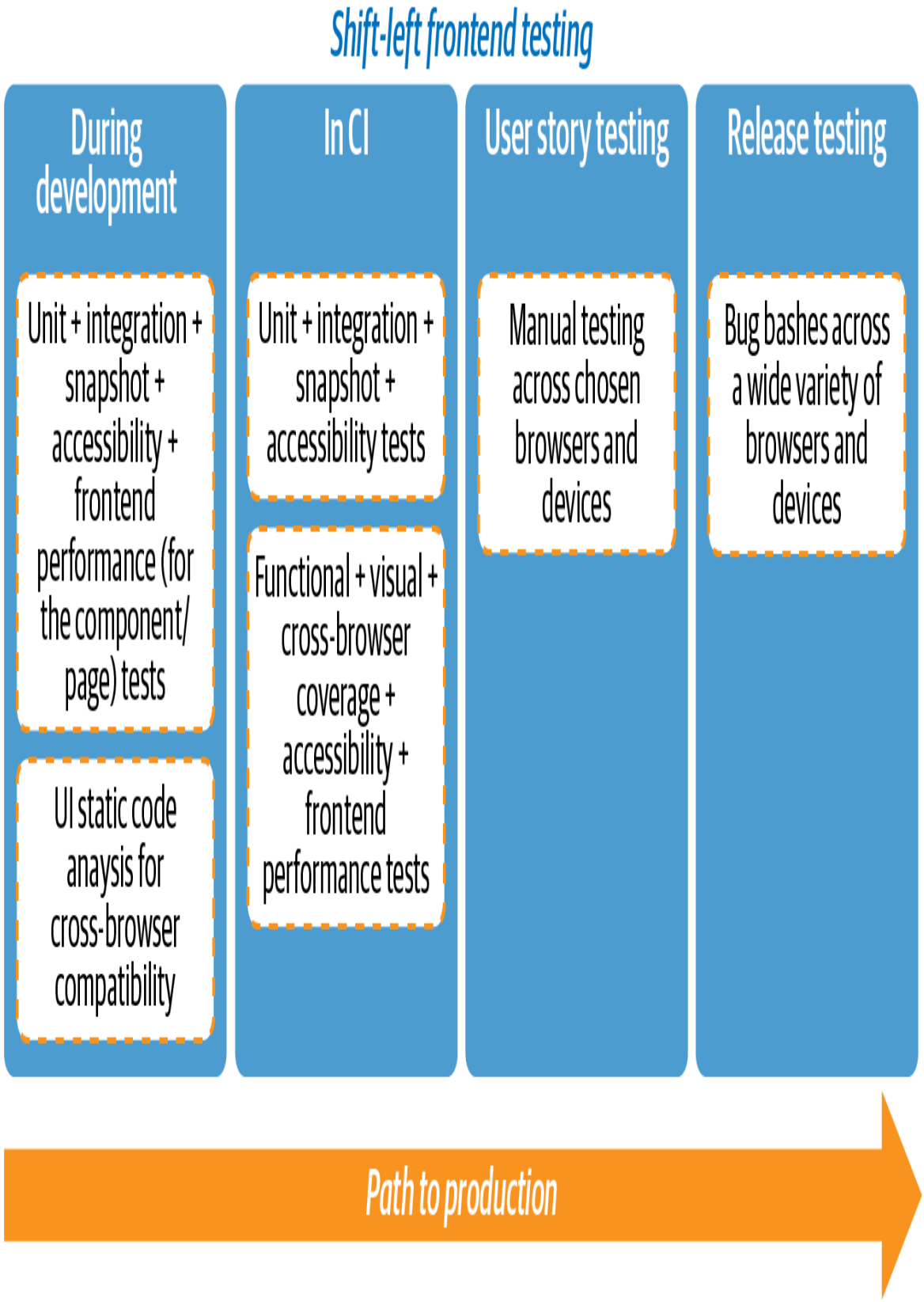


Figure 6-3. Implementation of shift-left testing in the frontend

Unit Tests

Frontend unit tests are written at the component level to assert their behavior in different states. They also partially contribute to visual testing. For example, unit tests might assert the greeting message in the title component of the page or the disabled/enabled state of a submit button. Typically, the developers write these tests when they begin development, using tools like Jest and the React Testing Library. They reside inside the development code base and provide fast feedback during the development stage.

Example 6-1 shows a sample unit test to verify a greeting message. As you can see, it fetches the `h1` heading element and asserts its text. By asserting that the element is `h1`, it offers a contribution to visual testing.

Example 6-1. Sample unit test using Jest

```
describe("Component Unit Testing", () => {
    it('displays greeting  message as a default value', () =>
{
      expect(enzymewrapper.find("h1").text()).toContain("Good
Morning!")
    })
  })
```

Integration/Component Tests

These tests are written to validate a component's functionality and the integrations between components—for example, login form behavior, as seen in Example 6-2. This sample test verifies the functionality of the entire form, not just a single component as in unit testing. Integration tests usually mock the service calls and simulate UI component state changes. In Example 6-2, the login response is mocked, and the test asserts that after a successful login the login form elements disappear from the screen. Integration tests can also help verify components with multiple child components and the integrations between them in different states.

Example 6-2. Sample integration test using Jest

```
test('User is able to login successfully', async () => {

  // mocking Login response
  jest
    .spyOn(window, 'fetch')
    .mockResolvedValue({ json: () => ({ message: 'Success' }) })
});

  render(<LoginForm />);

  const emailInput = screen.getByLabelText('Email');
  const passwordInput = screen.getByLabelText('Password');
  const submit = screen.getByRole('button');

  // enter login credentials and submit
  fireEvent.change(emailInput, { target: { value:
'testUser@mail.com' } });
  fireEvent.change(passwordInput, { target: { value:
'admin123' } });
  fireEvent.click(submit);

  // submit button should be disabled immediately
  expect(submit).toBeDisabled();

  // wait for form elements to be hidden after successful
login
  await waitFor(() => {
    expect(submit).not.toBeInTheDocument();
    expect(emailInput).not.toBeInTheDocument();
    expect(passwordInput).not.toBeInTheDocument();
    });
});
```

Developers write these tests at the end of component development and maintain them inside the development code base. Like unit tests, they help in providing fast feedback on the functional behavior during the development stage and contribute to visual testing—for example, as in the sample test shown here, by asserting the disabled state of the relevant elements after login. The same tools used for unit testing can be used for integration testing. It is also a good practice to add accessibility tests at a component level.

Snapshot Tests

Snapshot tests are intended to verify the structural aspects of individual components and component groups, contributing directly to visual testing at a micro level. These tests render the actual DOM structure of the components using test renderers and compare the results against the expected structure, stored in a reference snapshot alongside the test. Tools like Jest and react-test-renderer can be used for this purpose.

> **NOTE**
>
> Snapshot tests compare HTML code snippets. This is different from visual tests, discussed later, which compare images (screenshots) pixel by pixel.

Example 6-3 shows a sample snapshot test for verifying a `Link` component's structure using Jest.

Example 6-3. Sample snapshot test using Jest

```
import React from 'react';
import renderer from 'react-test-renderer';
import Link from '../Link.react';

it('renders correctly', () => {
  const tree = renderer
    .create(<Link page="http://www.example.com">Sample
Site</Link>)
    .toJSON();
  expect(tree).toMatchSnapshot();
});
```

For every code commit, this test will generate a new snapshot file with the DOM structure of the `Link` component, as shown in Example 6-4, and verify it against the previous snapshot of the component.

Example 6-4. Snapshot file generated by Jest

```
exports[`renders correctly`] = `
<a
  className="test"
  href="http://www.example.com"
  onMouseEnter={[Function]}
  onMouseLeave={[Function]}
>
  Sample Site
</a>
`;
```

These tests enable fast feedback on the structural aspects of components during the development phase. (In contrast, visual tests require the application to be fully functional on the local machine.) These tests become important when components are reused across multiple applications, like in design systems. Like unit and integration tests, they are written as part of the development process and stay within the development code base.

It's recommended that snapshot tests maintain a narrow focus, such as testing a single small component (like a button or header) or a slightly bigger component that won't undergo frequent changes. It is best to write them after the components are developed, for regression purposes. Otherwise, additional effort will be required to maintain them when there is even a slight change in the layout.

Functional End-to-End Tests

As discussed in Chapter 3, automated functional tests mimic a real user's actions on a website in an actual browser. These tests are written to validate complete end-to-end functional user flows, while ensuring the integration between the frontend and backend services. Unlike the tests discussed previously, automated functional tests require the application to be fully deployed and set up with appropriate test data. Although they use an actual browser, these tests only partially contribute to visual testing as they check if an element is present based on its locator but do not verify the look and feel of the element.

Visual Tests

While all of the previously discussed types of tests partially contribute to visual testing, visual tests do the heavy lifting. Like the functional tests described in the previous section, they open the application in a browser; then they compare a screenshot of each page against a base screenshot. Visual tests can be kept as a separate suite of tests or integrated into the functional test suite so they're easier to maintain. Open source tools like Cypress, Galen, BackstopJS, and others can be used for this purpose, as can paid tools like Applitools Eyes, CrossBrowserTesting, and Percy.

VISUAL VERSUS SNAPSHOT TESTING

Visual and snapshot tests may seem similar, but they operate on different levels, much like high-level functional end-to-end tests and low-level API tests. The feedback cycle of the two types of tests also differs significantly. And, as we discussed, visual tests verify the application after it is fully rendered in a browser, whereas snapshot tests give feedback on the HTML structure and hence are developer-friendly and aid with shift-left testing.

Snapshot tests work well when focused on smaller, individual components, whereas visual tests are ideal for validating the integration of multiple components in a large view, like a web page.

Cross-Browser Testing

Cross-browser testing has to be done to fulfill two important purposes: functional verification and visual quality verification across browsers. Though the application's functional flow shouldn't change much across browsers, there have been instances where discrepancies have been noted. For example, in 2020 Twitter had to fix a security issue where users' non-public information was stored in

the Firefox browser's cache. Apparently, Chrome didn't have that issue. So, testing the functional flow in different browsers should be part of your cross-browser testing strategy.

As the first step in cross-browser testing, you need to decide on the list of browsers you're going to focus on. As we saw earlier, Chrome and Safari are the most widely used browsers across the globe, and users might access your application via these browsers using different devices like desktop PCs, tablets, and phones. The application's responsiveness is therefore another criterion to consider when you are testing across browsers. A general rule of thumb is to focus on the browsers and resolutions that account for 80% of users. You can test the remaining 20% in bug bashes toward the end of release, based on priority.

So, to fulfill the purpose of getting functional feedback across browsers, given the caveats of UI-driven functional tests (which offer slower feedback and don't include visual testing), a good strategy might be to pick only the most critical functional flows and run them in your selected browsers. And to fulfill the purpose of getting feedback on visual quality, visual tests can be reused. They can provide feedback on both cross-browser compatibility and responsiveness. Choose the browsers and screen sizes used by 80% of your end users, and add visual tests for the most critical user flows. Overall, you should have a handful of functional and visual tests (you can combine them in the same tests using tools like Cypress and Applitools Eyes) to verify cross-browser compatibility and the responsiveness of the application.

If you're worried that these efforts won't suffice for your cross-browser testing needs for all application pages, frontend development tools/libraries like React, Vue.js, Bootstrap, and Tailwind have built-in cross-browser support. You can rely on these tools to ensure the visual quality of the non-critical user flows in the application. A caveat, however, is that these frameworks support

only the more recent standardized browsers, and some of their features may not be supported by older browsers.

To verify whether a given browser supports a particular feature of a development framework (and thus whether or not you should use it), you can check the support tables at *CanIUse*. For example, if you want to use the *flexbox* CSS layout in your UI, you can first check whether your target browsers support this layout. Teams can also include plug-ins like stylelint-no-unsupported-browser-features to automatically check for CSS features not supported by their target browsers based on CanIUse data. Similarly, the eslint-plugin-caniuse plug-in helps by pointing out unsupported scripting features for your target browsers. There is also another way to provide backward compatibility of JavaScript code, which is to use transpilers like Babel. They convert code written in the latest JavaScript to a version that is compatible with older browsers. Using all the aforementioned provisions, you can ensure that, by default, all the pages of your application meet your cross-browser compatibility requirements, especially in terms of their visual quality.

SHIFTING CROSS-BROWSER TESTING TO THE LEFT

Starting from the left:

- Use development libraries such as React, Vue.js, etc., that have support for standardized browsers.
- Use plug-ins like `stylelint-no-unsupported-browser-features` and tools like CanIUse to ensure the UI features are compatible with your target browsers during development.
- Have a handful of UI-driven functional tests combined with visual tests to run on a selected set of browsers and devices that cover 80% of your application's target usage.
- Conduct bug bashes frequently to test as much of the remaining 20% target usage as possible.

Frontend Performance Testing

Frontend performance testing involves checking the delay in rendering the frontend components by the browser. You can beautify your application and enhance its visual quality by adding attractive images and fancy gestures, but when the performance is not great, users are unlikely to return. It's widely recognized that downloading frontend components accounts for about 80% of page load time. As a result, balancing frontend performance and visual quality becomes very important. Tools and best practices for frontend performance testing are discussed in detail in Chapter 8, but it warrants a mention here given its relative importance.

Accessibility Testing

Web accessibility is mandated by law in many countries, and as a result, frontend code should be designed according to WCAG 2.0 requirements. Accessibility features significantly impact and generally enhance the visual quality of a website, as they emphasize having a consistent layout throughout the site, having understandable text, adequate clicking space, and so on. You will learn all about accessibility testing tools and best practices in Chapter 9.

That concludes our overview of the different frontend testing types. To summarize, the team should tailor its frontend testing strategy based on the intentions of the different types of tests and the needs of the application. The general recommendation is to have more micro-level tests (like unit tests) and fewer macro-level tests (like visual and end-to-end functional tests).

Exercises

We're now ready to start exploring automated visual testing tools. There are tools in this space that can be operated from the command line or driven by code, or you can outsource the task to software-as-a-service (SaaS) providers. Here, we'll walk through two exercises using BackstopJS and Cypress. You can add these visual tests to your CI pipelines and run them as part of every commit post-deployment, just like functional tests.

BackstopJS

BackstopJS is a popular visual testing tool with an active open source community supporting it. It comes as a Node library that is easy to integrate with CI and adopts a configuration style to write tests, meaning you don't need to add higher-level programming code. It uses *Puppeteer,* a UI automation tool, for rendering and navigating the application in Chrome and *Resemble.js* for screenshot comparison of web pages. After image comparison, BackstopJS

generates results as HTML reports. It has good support for configuring image comparison sensitivity and auto-maintenance during test failures, which we shall explore now.

This exercise will guide you through creating a visual test using BackstopJS to verify a web application across three resolutions: tablet, mobile, and normal browser.

Setup

As a prerequisite, you might need to set up Node.js and Visual Studio Code (as in the Cypress exercise in Chapter 3). Once that's done, follow these steps to install the tool and get the basic project scaffolding:

1. Create a new project folder and run the following command from your terminal to install BackstopJS:

   ```
   $ npm install -g backstopjs
   ```

   BackstopJS will be installed globally on your local machine so that you can reuse it across different projects. You will see the command has installed `chromium` (for Chrome) and `puppeteer` engines along with it.

2. Set up the default configurations and project scaffolding with the following command:

   ```
   $ backstop init
   ```

You should be able to see the default configuration file, *backstop.json,* in the project folder now. This is the file where you will add your visual tests as configurations.

Workflow

Now, take any sample public website for visual testing and follow these steps to create a test:

1. To verify the web page in three different screen resolutions, as dictated by our test case, add the *backstop.json* configuration shown in Example 6-5.

   Example 6-5. Sample test in the backstop.json config file

```
{
  "id": "backstop_demo",
  "viewports": [
    {
      "label": "browser",
      "width": 1366,
      "height": 784
    },
    {
      "label": "tablet",
      "width": 1024,
      "height": 768
    },
    {
      "name": "phone",
      "width": 320,
      "height": 480
    }
  ],
  "onBeforeScript": "puppet/onBefore.js",
  "onReadyScript": "puppet/onReady.js",
  "scenarios": [
    {
      "label": "Application Home page",
      "cookiePath":
"backstop_data/engine_scripts/cookies.json",
      "url": "<give site URL here>",
      "referenceUrl": "<give same URL here>",
      "readyEvent": "",
      "delay": 5000,
      "hideSelectors": [],
      "removeSelectors": [],
      "hoverSelector": "",
      "clickSelector": "",
      "readySelector": "",
      "postInteractionWait": 0,
```

```
      "selectors": [],
      "selectorExpansion": true,
      "expect": 0,
      "misMatchThreshold" : 0.1,
      "requireSameDimensions": true
    }
  ],
  "paths": {
    "bitmaps_reference":
"backstop_data/bitmaps_reference",
    "bitmaps_test": "backstop_data/bitmaps_test",
    "engine_scripts": "backstop_data/engine_scripts",
    "html_report": "backstop_data/html_report",
    "ci_report": "backstop_data/ci_report"
  },
  "report": ["browser"],
  "engine": "puppeteer",
  "engineOptions": {
    "args": ["--no-sandbox"]
  },
  "asyncCaptureLimit": 5,
  "asyncCompareLimit": 50,
  "debug": false,
  "debugWindow": false
}
```

Here are a few important things to note in this configuration file:

- The `viewports` array has three different screen resolutions defined for browser, tablet, and mobile users.
- The Puppeteer scripts to interact with the UI elements in Chrome are configured with the `onBeforeScript` and `onReadyScript` parameters. You can add your own scripts to define new actions too.
- The test case is defined under the `scenarios` array with parameters like `url`, `referenceURL`, `clickSelector`, `hideSelectors`, etc. You will understand the need for each parameter shortly.

- The locations to store reference screenshots and test screenshots are specified by the `bitmaps_reference` and `bitmaps_test` parameters. The location to store reports is provided by the `html_report` parameter.
- The `report` parameter is set to the value `"browser"` to enable viewing the results in the browser. When it is set to `"CI"`, it generates a report in the JUnit format.
- The parameter `engine` is used to configure the appropriate browser. By default, it is set to `"puppeteer"` and runs on headless Chrome. You can change this to `"phantomjs"` to run the tests on Firefox using older versions of BackstopJS.
- The parameter `asyncCaptureLimit`, set to `5`, will run the tests in parallel in five threads.

2. The next step is to take reference screenshots of the web page in different screen sizes for the tests to compare. BackstopJS gives you a hand here by doing this automatically. It opens the URL in the `referenceURL` parameter to take reference screenshots for all the screen sizes defined under the `viewports` array and stores them in the folder mentioned in the `bitmaps_reference` parameter. The command that does all that is:

```
$ backstop reference
```

3. The next step is to run the tests. Use the following command to do this:

```
$ backstop test
```

   BackstopJS will then verify the website indicated by the `url` parameter against the reference screenshots for all resolutions.

Once the test execution is complete, you can view the test results in the browser, as seen in Figure 6-4. I've chosen the Amazon home page as my test site, and the results show that one test passed and two tests failed—specifically, the browser test passed and the other two failed. On selecting the failed tests, I get to see the reference and actual screenshots; additionally, I can see a third image highlighting the differences between the two. Figure 6-4 shows all three screenshots. You can see the differences highlighted in the bottom half of the third image.

1 passed

2 failed

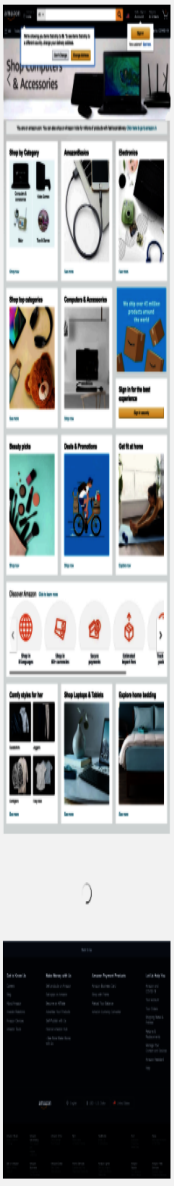
Filter tests with search...

backstop_demo_Amazon_Home_page_0_document_1_tablet.png

REFERENCE



TEST



DIFF

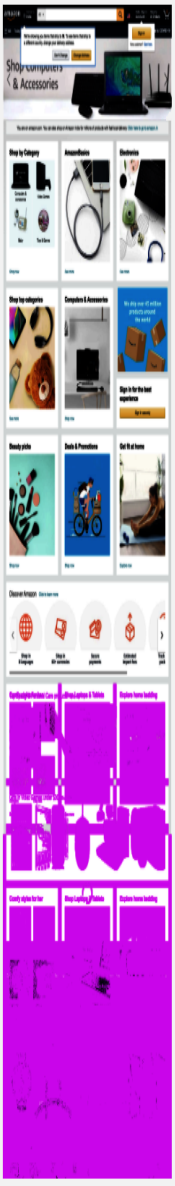


Figure 6-4. BackstopJS report on Amazon home page

The failures are because of the dynamic content on the Amazon home page. Although you would control the test data in your test environment, if you encounter such cases of dynamic content in your application, BackstopJS provides facilities to hide it during the test run. You can use the `hideSelectors` or `removeSelectors` parameters in the *backstop.json* file to hide or altogether remove the dynamic content. The selectors can be class names or ID names, as shown here:

```
"hideSelectors": [".feed-carousel-viewport"]
```

Alternatively, you could choose particular components on the screen for visual testing and omit the dynamic content using the `selectors` parameter.

Sometimes, even after removing the dynamic content, you might see the tests fail due to small pixel-level changes that do not really hamper the visual quality. In such cases, you can configure the sensitivity of the tests by using the parameter `misMatchThreshold`, which can be set to a percentage value from 0.00% to 100.00%. This can spare you test maintenance havoc.

BackstopJS assists with test maintenance, too. Suppose your application has evolved, and you have to update the reference screenshots. You can simply approve the new screenshots taken during the latest test run with the following command (of course, after manually verifying them once):

```
$ backstop approve
```

You can also enhance the test to search for a product and verify the product page using `keyPressSelectors`. Example 6-6 shows the configuration to enter search text in Amazon's search box and click the search button.

Example 6-6. Entering search text using `keyPressSelectors` in the backstop.json config file

```
"keyPressSelectors": [
  {
    "selector": "#twotabsearchtextbox",
    "keyPress": "Women's Tshirt"
  }
],
"clickSelectors": ["#nav-search-submit-button"],
```

A common use case in many projects is to compare the pages in different environments—for example, on the local machine and in the testing environment. You can do this by giving the local URL as the `url` and the test environment URL as the `referenceURL`.

When integrating with CI, you need to change the value of the `report` parameter to `"CI"` and save the output artifacts, including screenshots. It is also a good idea to archive the older screenshots so you can explore the history if required.

Cypress

We discussed the prerequisites and setup of a functional test automation framework using Cypress in Chapter 3. You can incorporate visual testing as part of the same framework using the plug-in cypress-plugin-snapshots. Just like BackstopJS, the plug-in compares screenshots and highlights the differences between them. It also provides many of the same features, such as allowing you to configure test sensitivity, select elements for comparison, and more.

Setup

To get started with the Cypress plug-in, follow these steps:

1. Run the following command to install the plug-in:

```
$ npm i cypress-plugin-snapshots -S
```

2. In the *cypress/plugins/index.js* and *cypress/support/index.js* files, add code to import the plug-in commands, as shown in Example 6-7.

Example 6-7. Cypress plug-in configuration

```
// cypress/plugins/index.js

const { initPlugin } = require('cypress-plugin-
snapshots/plugin');

module.exports = (on, config) => {
  initPlugin(on, config);
  return config;
};

// cypress/support/index.js

import 'cypress-plugin-snapshots/commands';
```

3. Cypress's configuration file is called *cypress.json*. Add the test configurations here, as seen in Example 6-8. A few parameters to note are `threshold`, which defines the test sensitivity; `auto clean-up`, which automatically deletes the unused screenshots; and `excludeFields`, which excludes an array of components from screenshot comparison.

Example 6-8. Sample test in the cypress.json config file

```
{"env": {
  "cypress-plugin-snapshots": {
    "autoCleanUp": false,
    "autopassNewSnapshots": true,
    "diffLines": 3,
    "excludeFields": [],
    "ignoreExtraArrayItems": false,
    "ignoreExtraFields": false,
    "normalizeJson": true,
    "prettier": true,
    "imageConfig": {
      "createDiffImage": true,
      "resizeDevicePixelRatio": true,
      "threshold": 0.01,
      "thresholdType": "percent"
```

```
    },
    "screenshotConfig": {
      "blackout": [],
      "capture": "fullPage",
      "clip": null,
      "disableTimersAndAnimations": true,
      "log": false,
      "scale": false,
      "timeout": 30000
    },
    "serverEnabled": true,
    "serverHost": "localhost",
    "serverPort": 2121,
    "updateSnapshots": false,
    "backgroundBlend": "difference"
  }
}}
```

Workflow

To add visual tests, the Cypress plug-in provides the method `toMatchImageSnapshot()`, which will take a screenshot of the specified component or the current page and compare it against the base screenshot. Cypress uses the screenshots from the first test run as the base/reference screenshots. Example 6-9 shows a test that opens an application URL, waits for the page to be visible, and then captures a screenshot of the entire page's content to do an image comparison.

Example 6-9. A visual test using Cypress to validate an application's home page

```
describe('Application Home page', () => {
    it('Visits the Application home page', () => {
      cy.visit('<give application URL here>')
      cy.get('#twotabsearchtextbox')
        .should('be.visible')
      cy.get('#pageContent').toMatchImageSnapshot()
    })
  })
```

If you run the test against the Amazon home page, it will fail because of the dynamic content. You can see the results in the Cypress test

runner with highlights of the image differences, as seen in Figure 6-5.

Amazon Home page > Visits the Amazon home page #0

Expected result



Actual result



Figure 6-5. Cypress Snapshot comparison results of Amazon home page

The base, reference, and comparison screenshots with highlights are stored separately in the screenshots folder, and can be saved as output artifacts in CI for debugging. The advantages of clubbing in the visual tests with functional tests are easier maintenance and reuse of test data creation scripts.

Additional Testing Tools

As mentioned earlier, there are many methods for incorporating visual testing. I'll introduce a couple of other tools you can use to add automated visual tests to your application to give you a broader idea of the visual testing space.

Applitools Eyes, an AI-Powered Tool

AI has penetrated the visual testing space, using computer vision and deep learning technologies. Computer vision is a branch of AI that enables computers to view digital media content like images and videos and gain a higher-level understanding of them. From an engineering perspective, the goal is to make computers understand and perform tasks that a human's visual system can do—for instance, viewing a web page and assessing the changes.

Applitools Eyes is one such AI-powered visual testing tool. The computer vision technology it's based on, called *Visual AI,* has cognitive abilities that allow it to analyze the structure and layout of a page, including the colors and shapes of the elements, and spot the differences just like a human eye would do. It is offered as a paid SaaS solution.

Visual AI is trained to resolve some common visual testing hurdles, such as these:

Maintenance

When tests fail due to common visual changes, it recognizes them, and on approval it autocorrects the base screenshots as needed.

Dynamic data handling

It can omit dynamic data, unlike in an exact pixel-to-pixel comparison.

Test sensitivity control

The sensitivity can be tuned to disregard minor UI changes that are of no consequence.

To set up Eyes, you need to sign up with Applitools and get a private API key to access the Eyes server hosted in their cloud. The Eyes server does the actual comparison. You will have to download the Eyes software development kit (SDK) in order to talk to the server, and configure it with your API key. The Eyes SDK provides the respective APIs to capture and send a screenshot to the Eyes server. You can use these APIs inside your existing Selenium WebDriver tests to do visual testing. (The SDK integrates many other UI development and testing tools too, such as Cypress, React Storybook, and even Appium, a mobile automated testing tool.)

The Eyes SDK APIs that you'll need to use in the tests are:

- `eyes.open(driver)` to instantiate a connection between the WebDriver instance and the Eyes server
- `eyes.checkWindow()` to check the visual quality of the page on all the prescribed devices and browsers
- `eyes.closeAsync()` to let the Eyes server know that the test is complete and to generate results

Example 6-10 shows a sample snippet of code integrating the APIs within the Selenium WebDriver tests.

Example 6-10. Applitools Eyes integration with WebDriver tests

```
// Visual checkpoint 1 after navigating to the application
home page

driver.get("<give application URL here>");
eyes.checkWindow("Application Homepage");

// Visual checkpoint 2 after clicking a button

driver.findElement(By.className("searchbutton")).click();
eyes.checkWindow("After clicking search button on home
page");
```

Applitools Eyes brings in a performance boost by taking the snapshot of the web page's DOM (instead of the screenshot) generated while running the Selenium WebDriver tests and using the snapshot to compare the web page in parallel on multiple browsers, devices, and screen resolutions hosted in the Applitools cloud. This not only yields ultra-fast performance but saves on your project's infrastructure costs, as all the devices are hosted in their cloud. The product also provides a hosted dashboard view to manage the overall workflow.

Storybook

Storybook is an open source tool that assists in UI development that is quite popular (~70K GitHub stars) in the frontend development world. It has integrations with various commonly used frontend development frameworks and libraries like React, Vue.js, and Angular. It helps you build UI components in isolation, which means developers can build the entire UI without setting up a complex application stack, creating test data, or navigating the application.

Storybook allows developers to create new components and manually verify their behavior and appearance by rendering them within the tool itself. Similarly, you can verify the different states of the component within the tool. Storybook saves the rendered components as *stories*. For each state of the component, a story is

stored; for example, a button component might be rendered in different states such as Large, Small, etc., and Storybook would store each of them as separate stories. This serves as an excellent repository for visual testing.

In fact, the tool offers visual testing out of the box through Chromatic, a hosted service that the Storybook maintainers have extended to enable automatic visual testing across multiple browsers (a free tier with limits is available). You can use Chromatic to automatically test every new story against the previous story—this is truly shifting visual testing to the left.

In organizations where a centralized UI development team produces shared components for use in design systems, without a backend to integrate, this tool is of great use.

As you've seen, there are multiple ways to do visual testing and integrate it into the development workflow. It can be integrated within the development environment with Storybook, integrated with the development process using BackstopJS, or integrated within the functional tests using Cypress and Applitools Eyes. This gives you a range of options for getting fast feedback in a way that suits your project's needs.

Perspectives: Visual Testing Challenges

One of the challenges with visual testing is choosing the tools to use. The ones mentioned here are only a small sampling; with AI and SaaS providers in the game, the choices are numerous indeed. Here are some features to look for when choosing an automated visual testing tool:

- Ease in workflow, from test creation to maintenance and CI integration.
- Robust screenshot management techniques. If you have to replace hundreds of base images for every small change, you

will pay enormous costs for visual testing. Tools that lend a hand with automatic cleanup and updating of screenshots have the upper hand.

- Test sensitivity control, so the tool can ignore minor UI changes.
- Ability to handle dynamic data.
- Ability to run across different browsers and device combinations.
- Performance while running visual tests across the different combinations of browsers and devices.

Apart from the choice of tools, the unspoken challenge is making your teammates fully buy into the idea of automated visual testing because additional effort is required to create and maintain these tests. But if you do this in the right stages and choose tools that provide easier test maintenance options, your team will soon appreciate the value. That said, also remember that not all applications need automated visual testing; before going down this path, consider your project/business-specific use case and do a cost-benefit analysis.

Key Takeaways

Here are the key takeaways from this chapter:

- Visual testing is a way to ensure the application looks the way it is supposed to, per the design. Visual quality takes the business one step closer to gaining customers' trust, which in turn amplifies the company's brand value.
- Although manual eyeballing and UI-driven functional tests can partially contribute to visual testing, they are not enough on their own as manual testing might be error-prone and functional tests do not verify the visual aspects of the application. Hence, you may need separate automated visual tests.

- Automated visual tests can provide great value, depending on the nature of the application and scope of work. In general, think about factors like customer impact, team confidence, manual effort, and type of work to determine whether your application needs automated visual tests.
- Open source tools like BackstopJS, Storybook, and Cypress offer varied features that are adequate to perform automated visual testing in projects. SaaS solutions like Applitools Eyes and Chromatic provide additional infrastructure and workflow management features for a cost.
- Apply visual tests in the right stages of the application to avoid flakiness and choose tools that can give fast, stable feedback early in the delivery cycle.
- Visual testing is just one piece of the frontend testing ecosystem. Taking advantage of the other frontend micro- and macro-level tests and devising a good frontend testing strategy can help you get faster feedback on visual quality.

Chapter 7. Security Testing

A chain is only as strong as its weakest link.

—Thomas Reid, Essays on the Intellectual Powers of Man (1786)

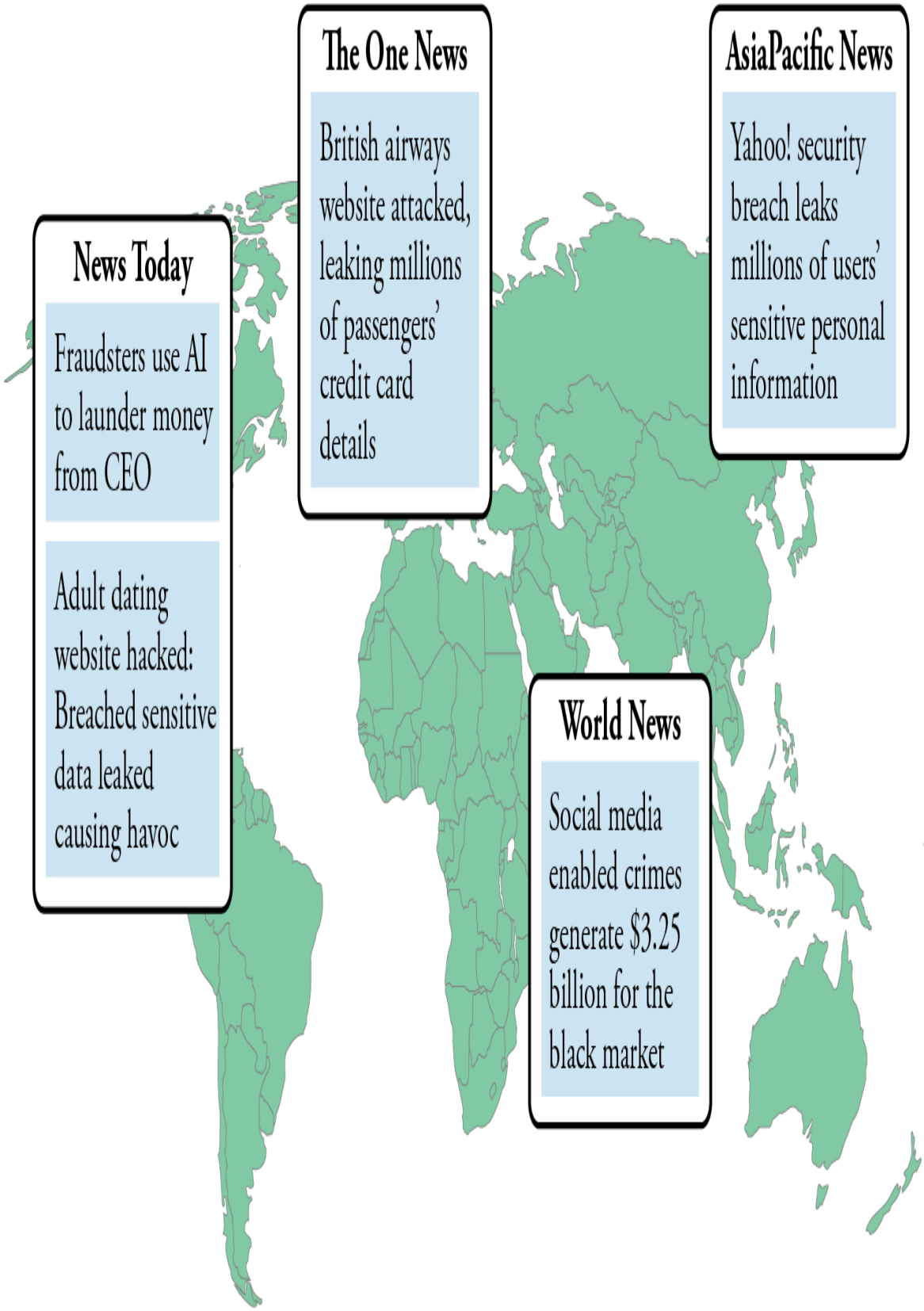


Figure 7-1. Example news headlines demonstrate that security is a global concern

We live in a world where we are more susceptible to cybercrimes than ever before—especially when we have social media accounts! *Cybercrime* is an umbrella term referring to all the illegal activities that can be performed with a computer and a network, including financial theft, theft of private assets such as sales documents and research reports, exploitation of sensitive information such as an individual's biological data, and more. Cybersecurity experts estimate that by 2025 the annual global cost of cybercrime (including both direct and indirect costs to its victims) will reach $10.5 trillion, up from an estimated $6 trillion in 2021. Social media–enabled cybercrimes account for the lion's share of this, with a 2019 study estimating global annual revenues for the perpetrators of $3.25 billion. These are unquestionably huge amounts, and sadly, the spoils may include some of our and our friends' hard-earned money!

The numbers imply that cybercrimes are far more prevalent than one might imagine. As Figure 7-1 shows, daily news reports also provide proof that cybercrimes are not restricted just to banking or social media sites, but extend to all kinds of websites—flight booking and adult dating sites get attacked too. We will discuss examples of such real-life attacks later in the chapter to get more insights. All of this raises a crucial question for software teams: what measures can we take to protect an application from such attacks?

To build a strong, secure system, the industry recommendation is to build your defenses in depth—that is, build security measures into multiple layers of the application, rather than focusing on a single outer solid layer. This is analogous to how castles were protected long ago: with a moat, then strong iron gates, armed guards in the keep, and so on. Each layer has to be strong enough to provide resistance, as each layer the intruders break through gives them access to more and more resources.

Also, remember that the security of your system is only as strong as your weakest link—be it your internal admins or hacked passwords. Thus, exploring for weak links is a crucial step in the effort to build

strong, impenetrable systems. And security testing is the primary activity that unveils such weak links!

Security testing is the skill of thinking like a hacker and looking for potential vulnerabilities, threats, and risks in the system that might open the gates to cybercrimes. Professional security or penetration (pen) testers have developed this skill over many years, and they can script different attacks on the application to expose the vulnerabilities. However, you don’t have to be a pen tester to use some good automated security testing tools and techniques in your day-to-day team activities to prevent significant security issues.

Security bugs, just like any functional software bug, are incredibly costly to fix when found in the later stages of the development cycle. Hence, to reduce the likelihood of leaving gaps in your application’s defense systems, you should practice shift-left security testing rather than waiting to engage pen testers until the end of the delivery cycle. Hence, you should practice shift-left security testing.

Indeed, the recommended best practice is to start thinking about a feature’s security aspects right from the beginning of the requirements gathering phase. For example, in a banking application the requirement to hide all account transactions from anyone other than the account holder and bank administrator is an obvious and necessary security feature. Similarly, the requirement to have a two-factor authentication feature provides an additional layer of security, much like having armed guards in addition to a stone wall protecting a castle. Incorporating security best practices throughout the analysis, development, and testing phases will help you build strong, secure systems.

This chapter will cover the basics you need to know to upskill yourself in security testing so that you can shift your security testing to the left in the delivery cycle. You will learn about real-life attacks, application vulnerabilities, and the STRIDE threat model. The chapter includes an exercise on threat modeling, a testing strategy

that implements shift-left security testing, and guided exercises on security testing tools with instructions on how to integrate them into the CI pipelines to get continuous feedback on your application's security.

> **NOTE**
>
> Becoming a professionally trained security tester is not the expected outcome of this chapter. As mentioned earlier, it takes years of practice. But that does not wash away a software team's responsibility to build secure applications. To help you meet this goal, this chapter focuses on the recommended practices, testing tools, and, most essentially, how to build a hacker-like mindset.

Building Blocks

To start thinking like a hacker, the first step is to observe different kinds of cyberattacks and understand the vulnerabilities that lead to them. This will trigger you to think about potential threats to your application and help you prevent them. So, let's start by discussing the prevalent types of cyberattacks.

COMMONLY USED SECURITY-RELATED TERMS

The following are some terms you will encounter in this chapter and in other reading about security issues:

- *Assets* are the critical entities in the application that need to be guarded by building appropriate defense mechanisms.
- A security *compromise* happens when the defense mechanisms of the system fail to protect the assets.
- *Vulnerabilities* are the potential gaps in a system that can be leveraged to compromise its security.
- *Threats* are the potential negative actions or events that can take advantage of the underlying vulnerabilities to compromise the security of the system.
- An *attack* is an unauthorized malicious action that is performed on a system with the aim of compromising its security.
- *Encryption* is a technique to scramble information in such a way that only the intended recipient who has the key to decipher the code can understand the information.
- *Hashing* is a technique to map data of any length to a fixed-size output using algorithms (sets of rules for performing calculations or other operations). The resulting output, or *hash*, can be used to verify the authenticity of the data, as even the smallest change to that data would result in a different hash being produced. Hashes are immutable—each unique input produces the same unique output—and it is practically infeasible to decrypt them to get back the original content. Hashing is thus said to be a one-way technique.

Common Cyberattacks

This section presents the most common types of cyberattacks, with real-life examples.

Web scraping

The easiest way to abuse a system is to exploit the publicly available data on the website—especially the users' personal data. A web scraping attack uses software or a script to automatically crawl a website and gather information that can potentially be used for malicious intent. Social media applications are prime targets as they can provide a lot of personal data, like the users' locations, phone numbers, and more—to an attacker, it's like putting your valuables on display in the window! The study mentioned at the beginning of the chapter estimates that personal data scraped from social media sites generates revenue of $630 million per year.

One prominent example of a web scraping attack was revealed in 2019, when 419 million Facebook user records including phone numbers were found in an unprotected database server on the internet. Though Facebook had removed the feature to display phone numbers on users' profiles by then, it appeared the information had been scraped while the feature was still available. Another example of unintentional exposure of data was reported in 2018, when Twitter found a bug in an internal tool that logged users' passwords in plain text without hashing (you never know where the weak link is!). Fortunately there was no evidence of compromise, but as a precaution the company was forced to ask its 330 million users to change their passwords.

Exposed data is always an avenue for exploitation, be it on a website or anywhere else. When you are thinking like a hacker, you should look for such exposed data throughout your application.

Brute force

If you had to guess your friend's password, how would you go about it? You might try their date of birth, favorite color, spouse's name, or a combination of such things, correct? When you extend the same trial-and-error method to include an organized list of all possible key combinations, it's called a brute-force attack.

In 2016, a brute-force attack on FriendFinder Networks' databases exposed 412 million user records with passwords and other sensitive information, such as users' sexual preferences. The usernames and passwords were reportedly hashed using the SHA-1 cryptography algorithm, but with modern brute-force techniques and computing power this failed to provide a sufficient defense.

Social engineering

Social engineering is the psychological manipulation of individuals into giving away their confidential information. You might have received phone calls from what seemed like reputable companies, where friendly customer service representatives requested your credit card details in exchange for some service—if you were taken in, you're not alone. In 2019, a UK-based energy company's CEO was manipulated by a phone call from someone who sounded like his boss (later discovered to be a trained AI program) to transfer a lump sum of $243,000 to the hacker's account!

Phishing

Phishing is a type of social engineering attack where an attacker sends a fraudulent communication (typically an email) to the victim with the intention to steal their personal data. The target may be tricked into downloading a malware attachment or clicking a link that takes them to a fake website that closely resembles a real one, where they are asked to provide login details or credit card information. It would be a surprise if you hadn't received at least one such email in today's world. In 2021, Microsoft 365 users were targeted by such a campaign: they received an email with an

attachment that supposedly contained details on a price revision, but on opening it was scripted to capture their authentication details.

Cross-site scripting

In a cross-site scripting (XSS) attack, the attackers take advantage of an unsecured website and inject code to manipulate the application's behavior. For instance, they might attempt to inject code that enables them to redirect the customers' payment details to their own servers. In 2018, British Airways was the victim of an XSS attack that exposed the credit card details of 380,000 customers. The company was fined heavily due to the lack of a proper defense system.

Ransomware

Ransomware attacks involve malware that blocks the system until a ransom is paid to the attacker. The Weather Channel went offline for an hour in 2019 due to a malicious ransomware attack on its network. Since the company had backup servers, it was able to get past the episode successfully.

Cookie forging

Cookie forging is a technique to manipulate the cookies that store user information on a website and get access to the users accounts. In 2017 Yahoo! disclosed that around 32 million user accounts had been breached after hackers gained access to the company's proprietary code and learned how to forge cookies that would allow them to gain access to accounts.

Cryptojacking

Cryptojacking has become a widespread attack these days. It is the activity of secretly mining cryptocurrencies using other people's devices without their authorization. Usually, bots are set to crawl public GitHub repositories to find infrastructure (e.g., AWS) access

keys. Once found, the instances are exploited within seconds, costing huge losses for the infrastructure owners. In 2018, Tesla Inc. was a victim of such illegal mining.

The attacks discussed in this section are only the tip of the iceberg. As seen in Figure 7-2, many more types of attacks can happen at different layers—application, infrastructure, and network. And hackers continue to create new kinds of attacks every day. It is a software team's paramount responsibility to stay ahead of the game, to protect the customers and the business. In fact, this is often a legal responsibility, as governments have developed regulations like the European Union's General Data Protection Regulation (GDPR) and Revised Payment Services Directive (PSD2) to enforce security and data privacy.

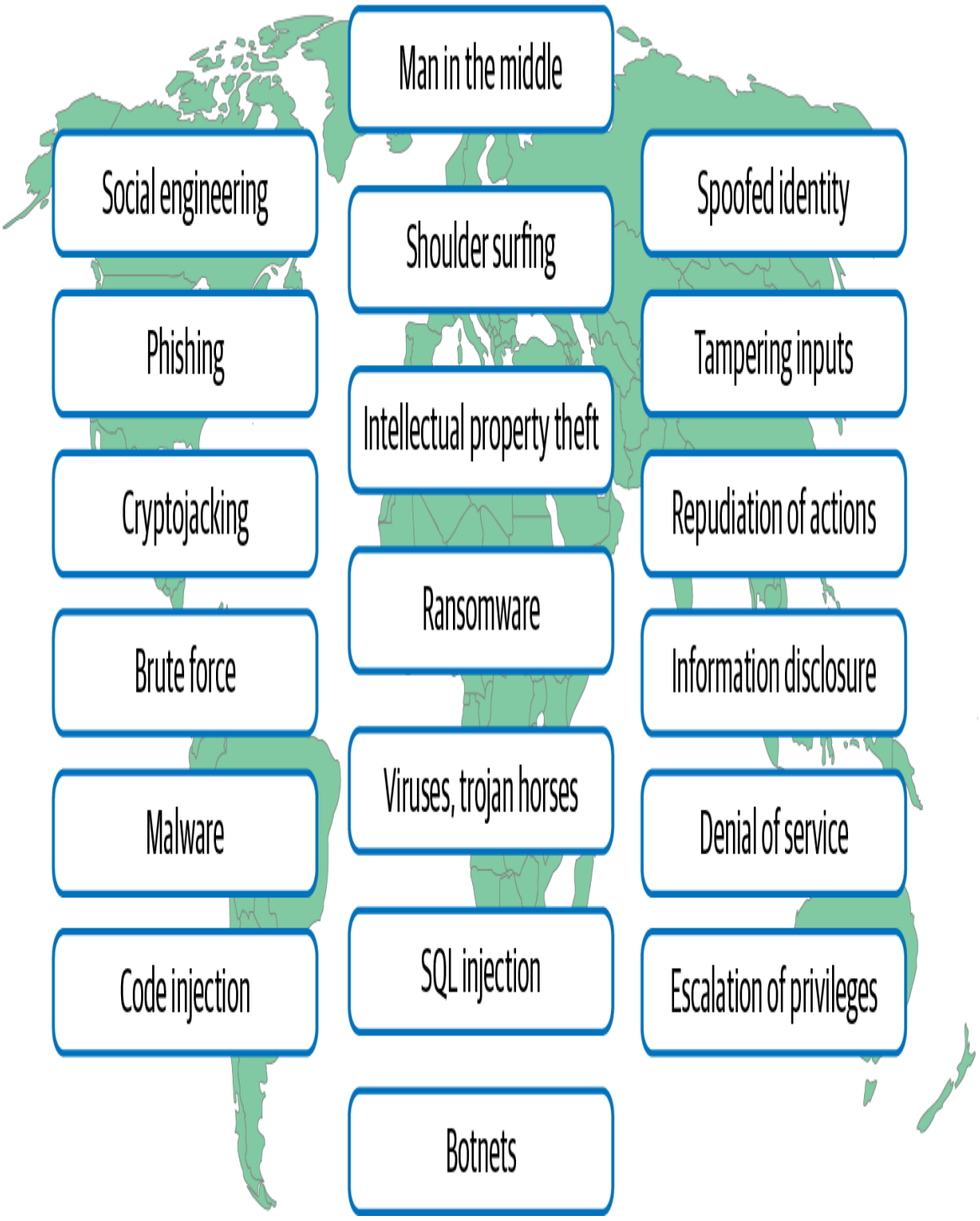


Figure 7-2. Common threats to software systems

The STRIDE Threat Model

The real-life examples we discussed in the previous section clearly illustrate the attackers' motives—they seek to hijack or abuse users' or companies' data, finances, infrastructure, access to services, or brand reputation. These are the primary assets that we need to protect in any application, and it is important not to underestimate the scale of the challenge. I once worked on a project where we designed the central security system for a bank. The number of potential security threats we brainstormed that could lead to a compromise of any of these five assets was truly eye-opening.

Similarly, when you are considering the security of your application, you need to think of all possible threats that could compromise the application's assets. To help you identify them, you can adopt a threat modeling framework called the *STRIDE* model, invented by Loren Kohnfelder and Praerit Garg from Microsoft. STRIDE is an acronym for **S**poofed identity, **T**ampering with inputs, **R**epudiation of actions, **I**nformation disclosure, **D**enial of service, and **E**scalation of privileges. You can take these one at a time and discuss all the possible threats under that category for your application.

Let's take a closer look at each of these classifications of security threats.

Spoofed identity

Spoofed identity attacks are those where a hacker assumes someone else's identity in order to access assets. Recall the social engineering example from earlier where the AI impersonated the CEO's boss to convince them to send money. Such identity thefts have become prevalent today, made possible by social engineering, phishing, malware, and shoulder surfing (spying over someone's shoulder when they're entering their personal information, such as a password or ATM pin), to name just a few.

Some of the defense mechanisms adopted to counter this type of threat are multifactor authentication, strong password

recommendations, and strong encryption while storing data and while transferring credentials.

Tampering with inputs

Tampering with inputs involves modifying something in the application (code, data, memory, etc.) in a way that violates its integrity. This is commonly done by injecting malicious code, such as a script, into the UI or other layers. For example, in the British Airways example mentioned earlier, the hacker changed the behavior of the website by injecting a script to harvest the customers' credit card details.

The defenses against this threat include adding appropriate validations (e.g., validating input fields to prevent someone from sending SQL queries), authentication and authorization mechanisms, and other standard security practices[1] while coding to avoid vulnerabilities that might result in a code injection.

Repudiation of actions

Repudiation occurs when the actions of a malicious user cannot be proven or tracked back to them. For example, a customer could deny receiving an item after delivery if there is no proof of receipt. This is a critical aspect to consider when designing a functionality, as it can result in loss of goods, reputation, and money, and sometimes legal action. To combat this threat, the application should have adequate logs and auditing mechanisms to ensure non-repudiation.

Information disclosure

Information disclosure threats involve unauthorized entities gaining access to the application's assets. As we saw in the Twitter example earlier, the employees were able to see exposed user passwords, which they were not supposed to have access to. In that case the exposure was an unintended consequence of the design, and fortunately, there were no security compromises. However, attacks to

gain unauthorized access to information are common. One popular approach is setting up malware to listen in the background and relay the information from a legitimate site to the hacker. This is referred to as a *man-in-the-middle* attack. To provide a proper defense against this kind of threat, you should build a strong authorization mechanism into your application, encrypt all secrets, and have secure transmission protocols.

Denial of service

A denial of service (DoS) attack aims to bring down the application's services, causing loss of revenue and reputational harm for the company. One way this threat could manifest is through a *distributed denial of service* (DDoS) attack, where the system is intentionally overloaded with millions of requests from multiple devices so that it becomes slow and finally fails.

The defenses against this kind of threat include adding load balancers, throttling requests per IP address, allowing only certain IP addresses, creating system backups, automatically spinning up new machines when load increases, and setting up monitoring systems to alert on sudden surges in request volumes, to list a few.

Escalation of privileges

An escalation of privileges attack happens when a malicious user is able to gain unauthorized privileges giving them elevated access. Imagine a hacker getting super admin privileges to a system! In my opinion, this is the worst kind of threat to deal with, as it can open up all kinds of risks: theft of private data, denial of service, financial loss, and so on. A best practice is to follow the principle of least privilege, granting the system's users only the minimum privileges they need to accomplish their tasks and nothing more. We can apply this principle in our individual teams too, for example by granting code-committing privileges only to developers and extending it to others only when necessary. Some useful defense techniques to counter

this threat are frequent access token refreshes, multi-signature features for authorizing transactions, storing secrets in vaults, and so on.

So, use this STRIDE model to brainstorm all the possible security threats to your application. Obviously you need to think of solutions to prevent the various types of attacks, but also think about how to contain the impact of an attack if one should happen.

Application Vulnerabilities

In learning to think like a hacker, we have discussed some of the common types of attacks, potential assets that hackers might try to abduct, and a framework to use to identify all the possible threats to your application. The next step is to get closer to the application code to learn about the different security vulnerabilities that the threats can exploit. Understanding these vulnerabilities will help you to add defensive code and test for them.

Code or SQL injection

An attacker could inject malicious commands or SQL queries to alter the behavior of a website if it is unprotected. Example 7-1 shows a query to retrieve a student's record by name.

Example 7-1. A SQL query in code that takes an input variable

```
// SQL query in the code to fetch student's record by name

SELECT * FROM Students WHERE name = '$name'
```

As you can see, the query takes an input variable, `$name`, from the user. The query is written in such a way that if a malicious user enters a SQL query to drop the entire table in place of a student's name, it will work perfectly fine, as seen in Example 7-2.

Example 7-2. The injected SQL query will drop the entire table

```
// If a malicious user inputs this as the student name in the
UI:
```

```
Name: Alice'; DROP TABLE Students; --

// The application will execute this command:
SELECT * FROM Students WHERE name = 'Alice'; DROP TABLE
Students; --'
```

Therefore, testing for proper input validation is necessary.

Cross-site scripting

As discussed earlier, cross-site scripting involves executing a script in the victim's browser that allows the attacker to take over the user's session, redirect the user to a malicious site, or even alter the code to deface the website. This type of attack is prevalent when there is no validation or proper sanitization of user input.

For example, a Twitter user posted the simple JavaScript script shown in Figure 7-3 to reveal an XSS vulnerability in the TweetDeck application. The code causes the tweet to auto-retweet itself whenever it appears on someone's timeline, popping up an alert dialog after. This could have been prevented if the app had properly validated the post's text for the presence of scripts, but since it didn't, the viewers' browsers innocently executed the script.



Figure 7-3. Tweet with XSS

Unhandled known vulnerabilities

If an application is dependent on third-party software (e.g., OSs, libraries, frameworks, tools), a vulnerability in any of these could be exploited to gain access to the system. Such vulnerabilities are often found and fixed, and patches are sent out as updates by the maintainers. However, teams might not regularly update all of their applications' vulnerable components, causing them to remain exposed. Tools can provide some assistance here. For example, GitHub's Dependabot offers to auto-update any dependencies with known vulnerabilities in the application code. Similarly, vulnerability scanning tools like Snyk and OWASP Dependency-Check help highlight vulnerable components.

Authentication and session mismanagement

Sometimes the authentication mechanisms in a website are not foolproof, leaving space for attackers to steal session tokens and exploit the privileges they gain. If you store session IDs and sensitive user data in session cookies, you need to refresh them very often and invalidate the older cookies. Also, you need to watch out for vulnerabilities like exposing session IDs in URLs, using unencrypted connections to send sensitive authentication data, and so on.

Unencrypted private data

Users often fall prey to the common vulnerability of unencrypted private data, like in the case described earlier where Facebook users' phone numbers were captured and stored in a database. You must ensure private data is not laid open without encryption in logs, databases, code repositories, project documentation, publicly hosted services, etc. Also, choose high-end cryptographic algorithms like AES, HMAC, or SHA-256, with dynamic salt and pepper techniques, to help protect data in transit and rest.[2]

Application misconfigurations

It is a frequent mistake to give blanket admin permissions to everyone using the application, for no better reason than to save on

maintenance effort. Misconfiguring the relevant permissions to users, folders, systems, etc. can result in unauthorized access and escalation of privileges to application content such as the database and admin endpoints, which can be easily abused. Teams should strictly adhere to the principle of least privilege, as discussed earlier.

Application secrets exposure

A common practice that leads to compromise is hardcoding the application secrets, such as environment credentials, superuser credentials, etc., in code and configuration files as plain text. An appropriate measure is to use secret management services such as vaults and access secrets only from there. This applies to application code, CI/CD pipelines, configuration files, and all the places you might have to access secrets.

The list in this section has highlighted a set of vulnerabilities that have to be carefully dealt with during development and testing. The Open Web Application Security Project (OWASP), a community-led non-profit organization, has identified the top 10 common vulnerabilities on the web, which you might be interested in reading as well.

Threat Modeling

Having come this far, you might be thinking about the security threats and vulnerabilities your application might be exposed to right now. In this section we will discuss a methodical approach to threat modeling—a structured way to aggregate all the potential security threats—that you can apply to your own application.

A general best practice is to perform this threat modeling exercise for each small scope of the application—for example, you might do 15 minutes of threat modeling per user story. Once you model your security threats, you can prioritize them based on impact and probability of risk, then incorporate the solutions as part of the story

or as a new feature. When you prioritize the threats, use the following general rule of thumb: *the cost of building security measures to handle a potential threat should not be higher than the value of the asset that you are trying to protect*.

For example, let's say your team is developing a blogging platform. Before building the home page, you do a 15-minute threat modeling exercise and discover a potential threat that a ransomware attack could bring down the page. The team proposes to implement a security monitoring system as a solution. They estimate that the monitoring system will cost $400K per year. Is it worth implementing this solution to protect against the ransomware threat? Maybe not, as the cost might be higher than the profit per year for the company. Also, how often does a ransomware attack happen on a blogging platform? The probability is very low. On the other hand, a threat like a code injection attack on an ecommerce website that could lead to loss of credit card details can be considered a high-impact and high-probability threat.

Once you have identified and prioritized the threats, address the solutions in the same user story or, if needed, create new "abuser" or "evil user" stories for this purpose. For example:

> *As an abusive user, I cannot inject code to redirect the content of the website.*

You can also derive security-related test cases from the threat modeling exercise and the abuser stories' acceptance criteria for development and testing purposes.

Threat modeling steps

Let's take a closer look at the framework for completing a threat modeling exercise. It is recommended to do this exercise as a team with all roles represented. Whiteboarding with colored stickies works well as you can capture your team's thoughts and categorize them quickly. In a remote world, choose tools like MURAL to conduct the

exercise. Once your team is assembled, navigate through the following milestones:

Define the feature

> The first step is to define the scope of the feature for threat modeling. Then, draw the user flows and the different types of users or actors in the system. Once this is clear, map the flow of data from one component to another. This way, you will cover user flows, actors, data flow, and the integration between components in the system.

Define the assets

> The second step is to identify the assets in the feature that need to be protected. Discuss the impact of losing each asset and capture the severity of the risk.

Black hat thinking

> Next, open the floor for black hat thinking, where you start thinking like a hacker and come up with ways to attack the application's assets. Here, the mindset of the team should be "Let's break the system!" Use the STRIDE model to structure this discussion. Allow imaginations to flow freely without debating if something is really a threat or not for now, and capture all the ideas as stickies.

Prioritize the threats and capture stories

> Analyze the probability and the potential impact of the threats you've identified, and prioritize them. Capture these as abuser stories so the team can act on them after the threat modeling brainstorming session.

Now that you know the basics, you're ready to get some firsthand experience by actually doing a sample threat modeling exercise.

Threat modeling exercise

For this exercise, suppose we have an application to manage (create/view/update/delete) orders in a retail store. The application has a web UI and backend REST services to perform business operations on the order data stored in the database. Let's get to the first step of defining the actors, data flow, and integrations between different components.

> **NOTE**
>
> This exercise is intended only to get you familiar with the threat modeling steps, not to provide an accurate threat model for an order management system.

The users of the system are:

- The store assistant who places, edits, and cancels orders
- The system administrator who manages the infrastructure, configurations, and deployments
- The customer service executive who uses the application to answer queries related to order statuses over the phone

The user flow is simple: the store assistant and the customer service executive have to log in to the application to view the latest orders list, and they are provided with options to manage the orders. Figure 7-4 shows the integration of components and data flow between the components to aid that user flow.

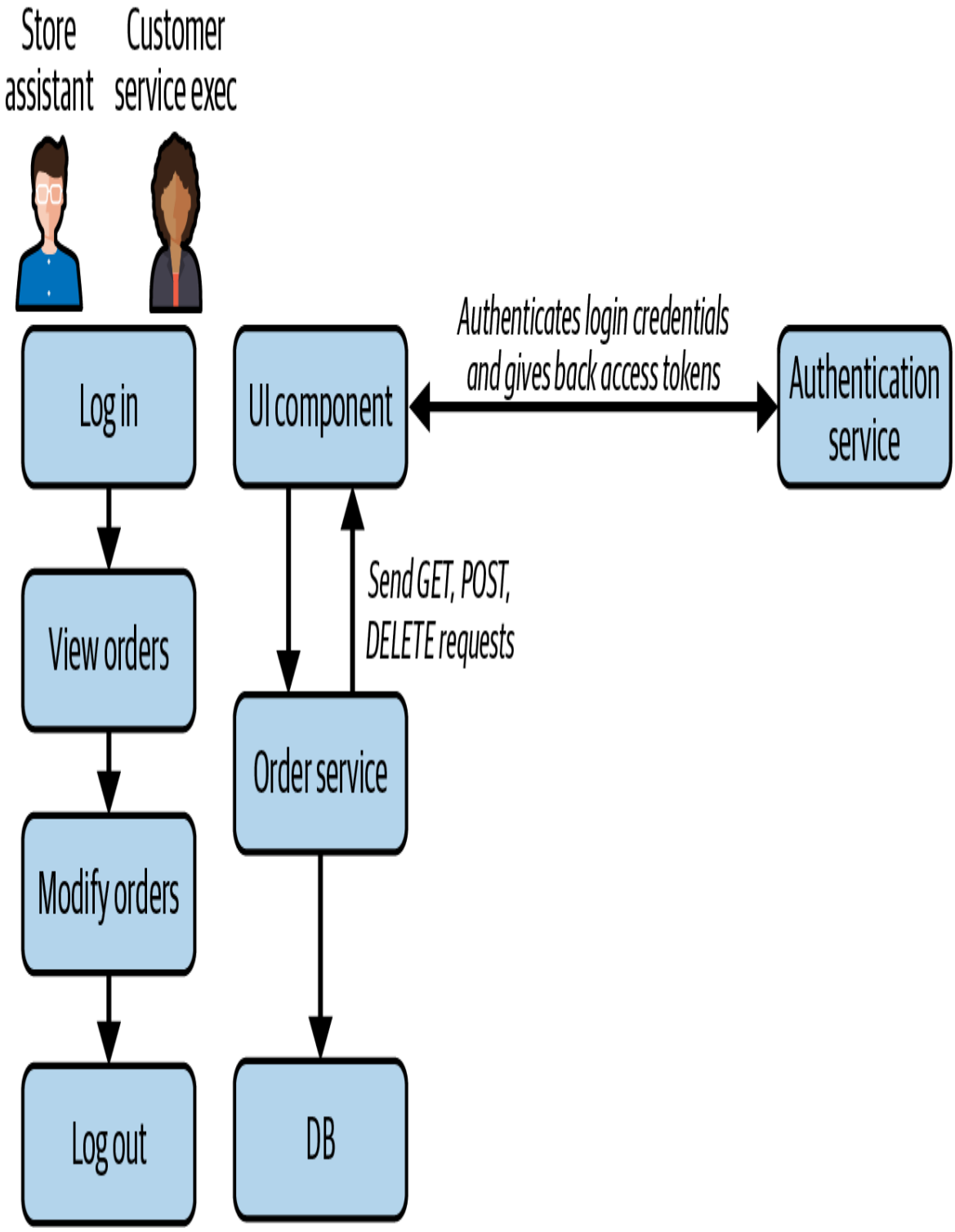


Figure 7-4. User flows and data flows for the sample order management system

Similarly, as shown in Figure 7-5, the system admin has to log in to the virtual machines (VMs) to run any scripts or configure the

infrastructure.

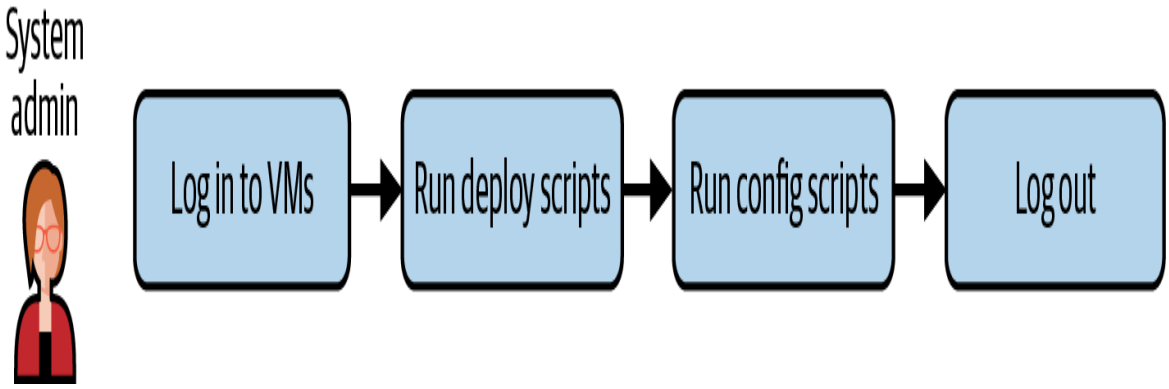


Figure 7-5. User flow of the system administrator

Let's discuss the assets that we need to protect next. Here are a few:

1. Order information is a critical asset to the business. Customers will be dissatisfied if their orders are tampered with, resulting in a loss of reputation.
2. Orders include customers' private data, such as names, phone numbers, payment details, and home addresses. Any exposure of confidential information will result in a lawsuit and cause harm to the customers, so customer details are another essential asset.
3. The database has the complete sales information of the business. A breach there will be hazardous to customers and the business, as the data could be sold on the black market or to competitors.
4. The infrastructure that hosts the application is crucial to protect as well, as any downtime will lead to failures in order transactions and lost sales.

We've finished the first two steps of threat modeling. Next is black hat thinking! Take a moment to ideate. Recall the user and data

flows and think about how the hackers could gain control of the assets. Use the STRIDE model to structure your thinking.

When you're done, compare what you've come up with to possible threats identified in Figure 7-6.

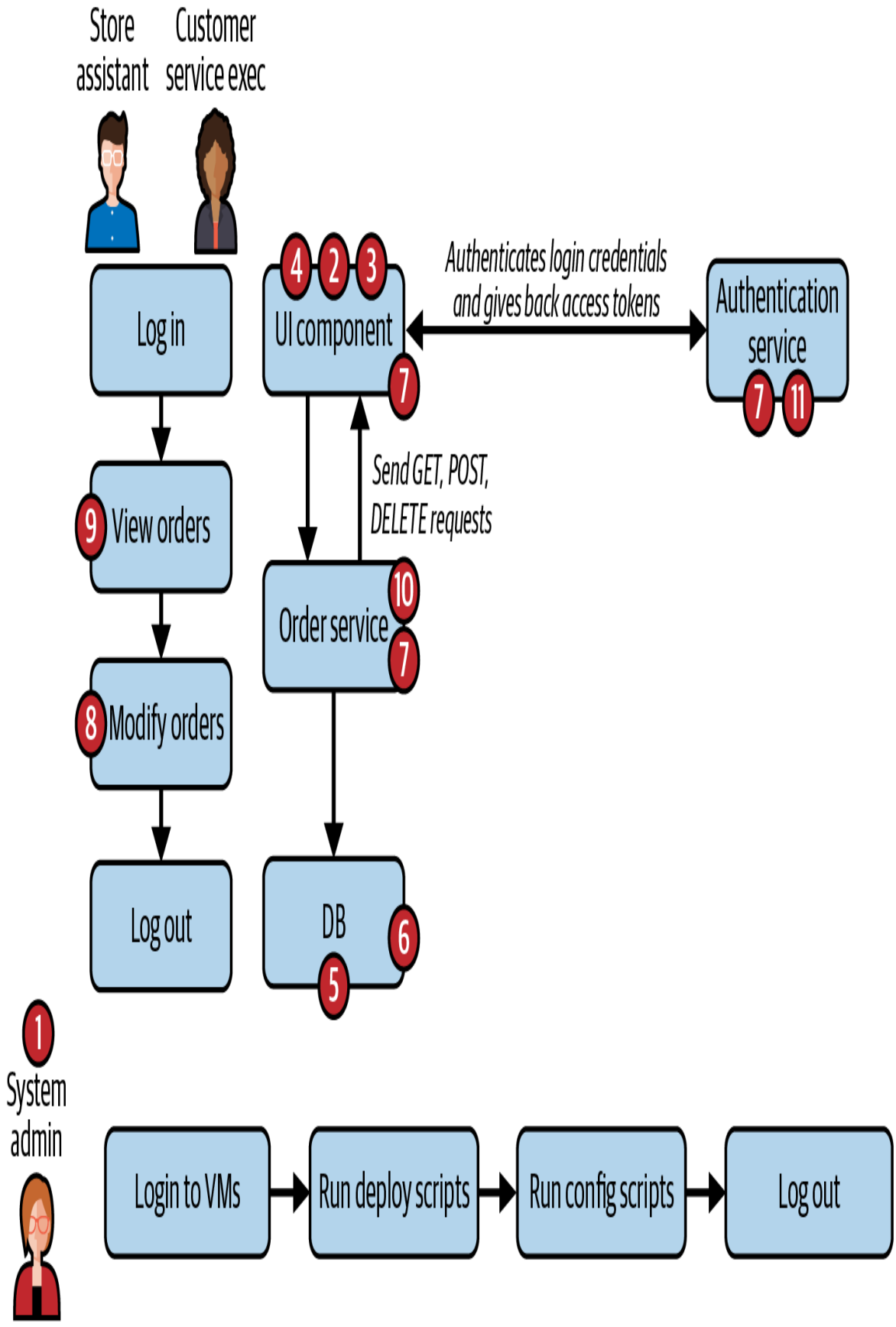

Store assistant
Customer service exec
Log in
UI component
4
2
3
7
Authenticates login credentials and gives back access tokens
Authentication service
7
11
9
View orders
Send GET, POST, DELETE requests
Order service
10
7
8
Modify orders
Log out
DB
6
5
1
System admin
Login to VMs
Run deploy scripts
Run config scripts
Log out

Figure 7-6. A map of the identified threats

Let’s take a look at each of these:

Spoofed identity

1. Social engineering tricks could be played on the system admin to get their login credentials, or just shoulder surfing or malware might do the trick. Since the system admin role has superpowers, a malicious actor could use it to bring the infrastructure down.
2. The store assistant could forget to log out, and anyone in the store could use the logged-in session to change the delivery addresses of existing orders (e.g., to their own address).

Tampering with inputs

3. The attacker could get hold of the order service endpoints from any open browser session and tamper with orders later, since the endpoints may not be protected.
4. Code injection could be used while placing an order to hijack customer payment details.

Repudiation of actions

5. The system admin, when they find out there are no logs for their actions, could create bulk orders for their family and friends by directly inserting records in the database and triggering other relevant processes.

Information disclosure

6. If the database is attacked via a back door, all the information it holds will be exposed, as the data is stored in plain text.

7. Stealing passwords from unencrypted logs or other storage would enable the attacker to tamper with order data.
8. The customer service executive doesn't have any restrictions on their operations—all they're supposed to do is relay the current order status information to customers, but they could edit orders as well. They could work with an accomplice to abuse their permissions.
9. The `/viewOrders` endpoint allows any number of records to be returned. Once compromised, this endpoint could be used to view all orders. We should at least think of reducing the blast radius.

Denial of service

10. The attacker could perform a DDoS attack and bring down the order service, leading to loss of sales.

Escalation of privileges

11. If an attacker manages to get hold of the admin credentials, they could add new users or escalate the privileges of existing users to maintain an elevated level of access to the system in the future. They could also create, modify, or delete order records without anyone noticing, as there are no logs for system admin actions.

As you can see, even in a small system with just a handful of components and users, there are many attack points. Imagine how many there will be in a real system with thousands of components and users!

The next step is to prioritize the threats and capture stories. Based on likelihood and impact of the threats identified here, we might add new security-related user and abuser stories like the following:

1. “As an abusive user, I should not be able to see customer details even if I gain access to the database.”
2. “As an abusive user, I should not be able to take advantage of open browser sessions.”
3. “As an abusive user, if I get access to the system administrator’s or customer service executive’s login credentials, I should not be able to edit orders.”
4. “As a store assistant, I should be the only person authorized to make edit requests to the order service.”
5. “As a store assistant, I should be frequently prompted to change my password to a strong password.”

As mentioned earlier, to discover all the potential threats to your application, you should perform these threat modeling exercises iteratively throughout your development cycle, keeping the scope small. You will likely uncover new threats to older features as you incrementally brainstorm threats to new features.

Security test cases from the threat model

From the threat model, you can get insights into the many ways an attack might be made on your application. And along with the abuser stories, you will get a vivid idea about the security-related test cases. After threat modeling, apply the exploratory testing mindset described in Chapter 2 to capture the security test cases for each application layer.

NOTE

Zero trust is a principle that suggests not to place your trust in any entity, be it a person or a component, even within your secure perimeter. Zero-trust architectures verify the authenticity of every request before executing the request, using an authentication and authorization protocol such as OAuth 2.0, which uses bearer tokens for verifying authenticity. You will see the test cases referring to these tokens.

Assuming a zero-trust architecture is implemented using OAuth 2.0 as a solution to address the threats discussed earlier, here are some security test cases at different application layers for the order management system example:

1. In the UI layer:
    - Verify that after the session times out, the user is prompted to log in again.
    - Verify that the user credentials are locked after a set number of failed login attempts.
    - Verify that the input fields have validation for illegitimate inputs (e.g., JavaScript code, SQL queries, etc.).
    - Verify that access tokens are expired after a short period. However, a refresh token call should be made from the UI to keep the user logged in until the session times out.
    - Verify that when logged in as the system admin or customer service executive there is no option to edit an order in the UI.
2. In the API layer:
    - Verify that reusing an expired access token causes the order service to return a 401 Unauthorized response

(though 400 is preferred to avoid revealing further information to the attacker).

- Verify that appropriate validation is performed on API parameters' values (similar to the UI input fields) and that the APIs return a 404 error if the validation checks do not pass.
- Verify that the `/editOrder` endpoint returns a 401 Unauthorized response if a system admin's or customer service executive's access token is used.

3. In the DB layer:
    - Verify that passwords are stored as hashes (with a dynamic salt) in the DB, per NIST guidelines.
    - Verify that sensitive customer details are encrypted in the DB.
4. In the application logs:
    - Verify that passwords are not logged as plain text in application logs.
    - Verify that user-sensitive information is not logged as plain text in application logs.
    - Verify that there are appropriate application logs for all actions performed on the system, including by the system admin, with timestamps.

These are just a handful of security-related test cases. You may be able to think of more! I hope this exercise has helped you understand how teams can bring security into the software development lifecycle, from threat modeling during analysis to devising solutions for potential threats and testing security aspects.

Security Testing Strategy

The practices we've discussed so far—conducting a 15-minute threat modeling exercise per user story, writing abuser stories, building security measures in layers, deriving security-related test cases, and so on—will help strengthen your defense systems significantly. One more critical step is to add mechanisms to give continuous feedback on potential vulnerabilities in the code being developed so that they can be fixed as soon as possible. This is where you need to think of shifting security testing to the left. We will discuss a security testing strategy that implements this approach here.

Figure 7-7 shows a shift-left security testing strategy across the different stages of the path to production, starting with development.

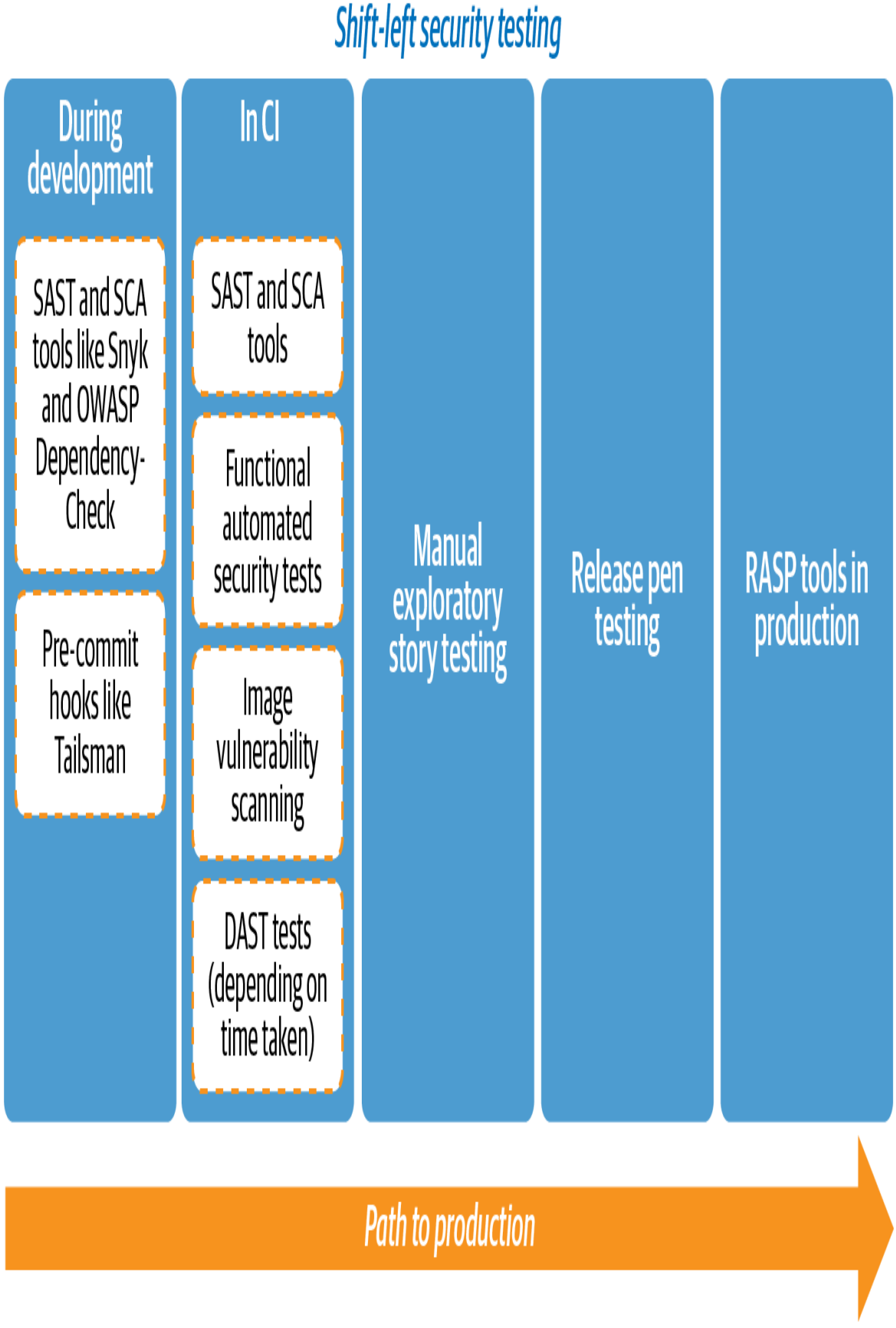

Shift-left security testing
During development
SAST and SCA tools like Snyk and OWASP Dependency-Check
Pre-commit hooks like Tailsman
In CI
SAST and SCA tools
Functional automated security tests
Image vulnerability scanning
DAST tests (depending on time taken)
Manual exploratory story testing
Release pen testing
RASP tools in production
Path to production

Figure 7-7. A shift-left security testing strategy

Let's take a look at some of the tools and techniques you can use in the different stages:

Static Application Security Testing (SAST) tools

SAST is a technique for analyzing the static application source code, byte code, and assembled code for known vulnerabilities. For example, it scans the application code for unencrypted secrets. SAST tools come in various forms, including plug-ins, libraries, and SaaS solutions (e.g., Snyk IDE plug-ins, Checkmarx SAST, Security Code Scan), and can be integrated with CI pipelines to run against every commit. SAST is a big part of shifting left, as it helps you discover issues during development.

Talisman, though not exactly a SAST tool, is specifically designed to scan the application code for secrets such as private keys, environment credentials, and so on, and it can be integrated as a pre-commit hook. This prevents secrets from even being pushed to the repository. We will take a brief look at the Snyk JetBrains IDE plug-in and Talisman later in the chapter.

Software Composition Analysis (SCA) tools

SCA is a technique that identifies vulnerabilities in the application's third-party dependencies. Especially when you are using a lot of open source libraries, these tools (e.g., OWASP Dependency-Check and Snyk) will add a lot of value. They also give feedback during development and can be integrated with CI for every commit. SCA combined with SAST will help uncover the static application code's vulnerabilities during the development phase itself. A guided exercise using OWASP Dependency-Check is included in the next section.

Functional security test automation

You can add automated tests using functional automation tools, as discussed in Chapter 3, to cover the functional security test cases. For instance, in the OMS threat modeling example from earlier, the security test case verifying that only the store assistant is authorized to edit orders can be added as a service test in the order service.

Image scanning

Containers have become a widely adopted way to package and deploy applications. Testing for vulnerabilities in the container images, if you are using them, is critical. Tools like Snyk Container, Anchore, and others can be used to do image scanning and can be integrated with CI. Docker has a built-in command, `docker scan`, to perform vulnerability scanning on Docker images. Similarly, Amazon Elastic Container Registry (ECR) offers image scanning capabilities for the images pushed to the registry. When writing infrastructure as code (e.g., with Terraform or Kubernetes), tools like Snyk IaC and `terraform-compliance` can be used to enforce security best practices as well.

Dynamic Application Security Testing (DAST)

DAST is a black-box testing technique. It finds security issues by analyzing how the application responds to specially crafted requests that mimic actual attacks. For example, DAST tools like OWASP ZAP and Burp Suite try to inject malicious scripts into the application to check for code injection vulnerabilities. They can be integrated with CI pipelines too, but depending on the application they might take a while to run, so choose the appropriate CI stage to run them in (as discussed in Chapter 4). A detailed exercise that walks through performing DAST with OWASP ZAP is provided in the following section.

NOTE

Interactive Application Security Testing (IAST) is a new technique that aims to combine SAST and DAST to analyze the application behavior during runtime. It works through software instrumentation and scans for security vulnerabilities in real time. This space is evolving; some example IAST tools are Contrast Security and Acunetix.

Manual exploratory testing

During manual exploratory testing you can derive security-related test cases from threat modeling exercises across all layers (UI, services, DB). Chrome DevTools and Postman both provide a variety of options for executing security test cases, as you'll see later in this chapter.

Penetration (pen) testing

Depending on the criticality of the application and the development team's competency with respect to security, you may choose to involve a professional security tester near the end of the delivery cycle to ensure that the application doesn't suffer from security issues later.

Runtime Application Self Protection (RASP)

Techniques like SAST and DAST, discussed earlier, help in finding vulnerabilities in the application code. But when attacks actually happen in the production environment, you need a layer of defense to prevent them from being successful. RASP is a security technique that involves monitoring the application for potential attacks in the production environment and preventing them from happening. RASP tools (e.g., Twistlock, Aqua Security) extend the traditional firewall concept further by working within the application runtime and building up knowledge of what

is and is not expected application behavior. They then listen to the application processes at runtime and automatically take protective measures, such as automatically terminating cryptomining processes, examining incoming request payloads and denying them if they are malicious, and helping prevent malware attacks. RASP tools are currently available only as paid products.

We'll see how some of these tools can be applied in practice in the coming sections.

Exercises

The exercises here will guide you through performing automated SCA using OWASP Dependency-Check and DAST using OWASP ZAP, and integrating them with CI to get continuous feedback.

OWASP Dependency-Check

As you saw earlier, a common threat is to have dependencies with vulnerabilities. OWASP Dependency-Check is an open source SCA tool that scans for known vulnerabilities in the project's libraries and external dependencies. It can be used via the command line or as a Jenkins or Maven plug-in, among other options.

Setup and Workflow

Follow these steps to set up the command-line Dependency-Check tool and run a scan on the Selenium WebDriver automation test project you created in Chapter 3:

1. To install Dependency-Check on macOS, use the following command:

```
$ brew install dependency-check
```

You can download the dependency-check ZIP file for other OSs from the official website.

2. Once installed, run the scan on the Selenium WebDriver automation project using the command shown here:

```
// On macOS

$ dependency-check --project project_name -s
project_path --prettyPrint

// On Windows (the dependency-check.bat file is inside
the bin folder
// when you unzip the folder you downloaded in the
previous step)

> dependency-check.bat --project "project_name" --scan
"project_path"
```

   This command can be integrated into your CI pipeline to make it fail if vulnerabilities are detected.

3. The command will generate an HTML scan results report in the same folder listing all the vulnerabilities found. The Selenium WebDriver project may or may not have vulnerabilities. A sample report with vulnerabilities is presented in Figure 7-8 for the purposes of illustration.

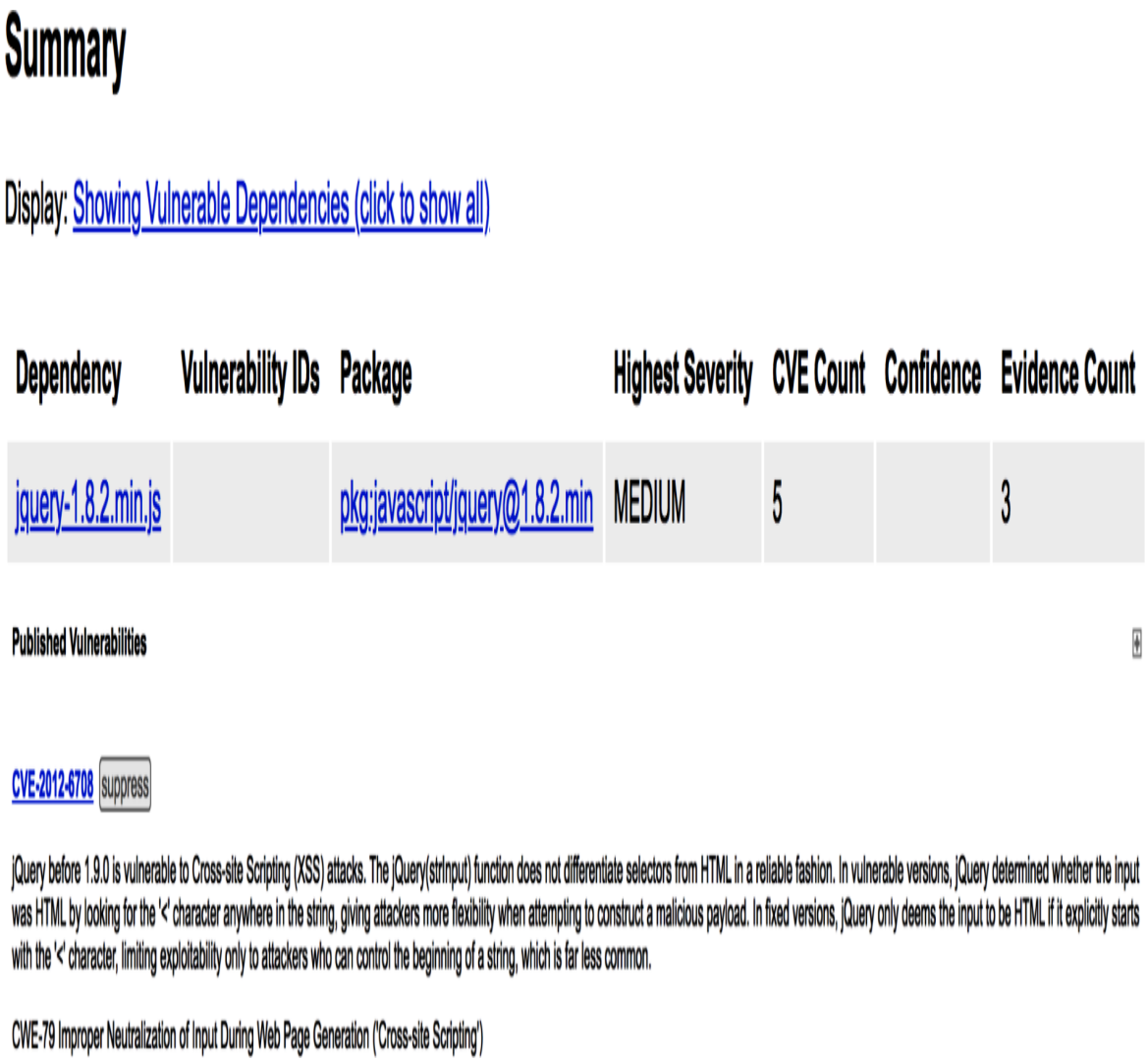


Figure 7-8. OWASP Dependency-Check scan results

As you can see, the tool has reported a vulnerability in the `jquery-1.8.2` library. It refers to the published vulnerability with ID CVE-2012-6708 and explains that “jQuery before 1.9.0 is vulnerable to Cross-site Scripting (XSS) attacks,” educating us to update the library appropriately.

OWASP ZAP

OWASP Zed Attack Proxy (ZAP) is an open source tool that does DAST on a deployed application. It uses preconfigured automated scripts as attacks on the application to expose a known set of

vulnerabilities. ZAP essentially operates as a man-in-the-middle tool between the browser and the application; it sniffs the messages exchanged between them to detect known vulnerabilities and modifies them in order to perform various attacks. This makes it easier for teams new to security fundamentals to find security issues easily. In addition, ZAP has an exhaustive list of configurations and add-ons to support multiple functionalities, enabling security professionals to add advanced scripts. There is good documentation to explore these options. ZAP can be integrated with other tools like Selenium, making it easier to run in CI as well.

Let's get hands-on with ZAP now.

Setup

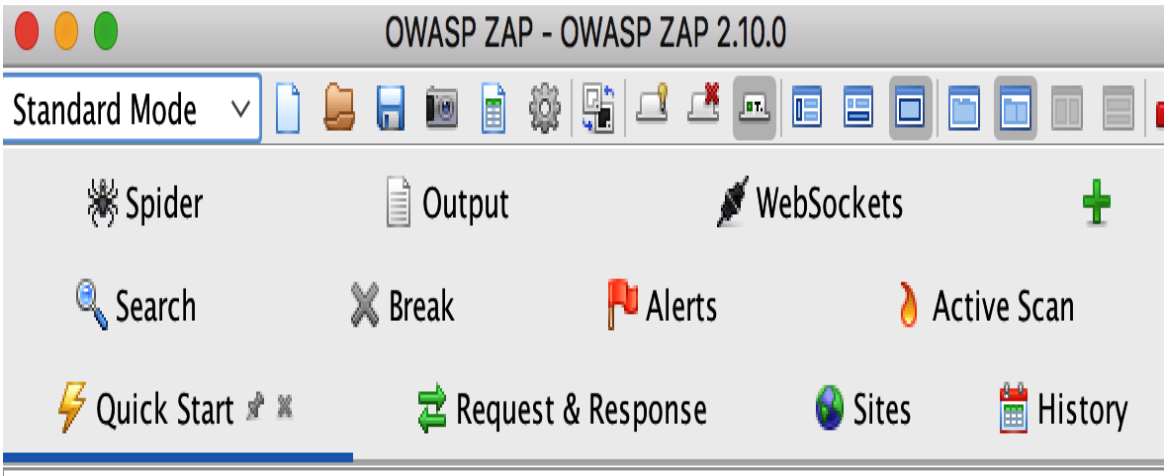
To install ZAP on macOS, use the following command:

```
$ brew install cask owasp-zap
```

Use the installation binaries from the official website for other OSs.

Workflow

Once the installation is done, you should be able to open the ZAP desktop UI, as seen in Figure 7-9. (On a Mac, you may have to give permission to open the app since it's not from the App Store.)

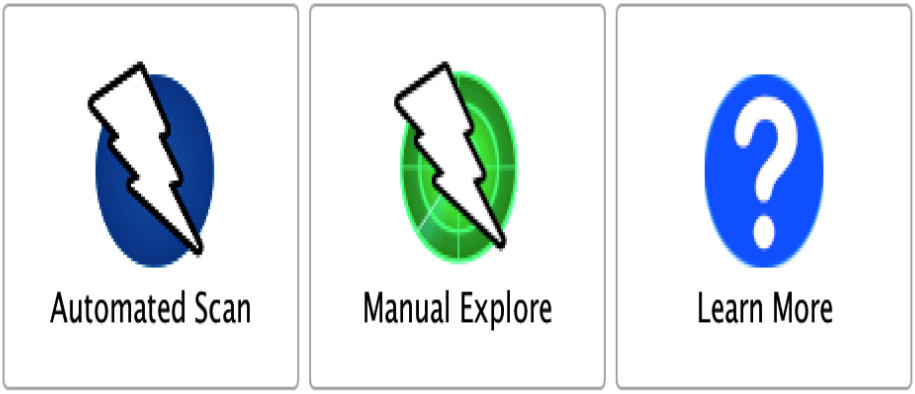


Welcome to OWASP ZAP

ZAP is an easy to use integrated penetration testing tool for finding vulnerabilities in web applications.

If you are new to ZAP then it is best to start with one of the options below.

Text



News

Announcing the First Ever ZAPCon - Call for Papers Learn More X

Figure 7-9. The ZAP Desktop UI

The first step is to educate ZAP about the internal application URLs and the UI components so that it can attack them later. You can do this in two ways: by using the Manual Explore option that can be seen in Figure 7-9, or by using the ZAP Spider.

Manual Explore

To manually explore an application, click the Manual Explore button in the ZAP Desktop UI. A new screen will open, as seen in Figure 7-10, asking you to enter the application URL.

> **WARNING**
>
> Do not use this tool on a public website. It is illegal to conduct security testing on a website without authorization.

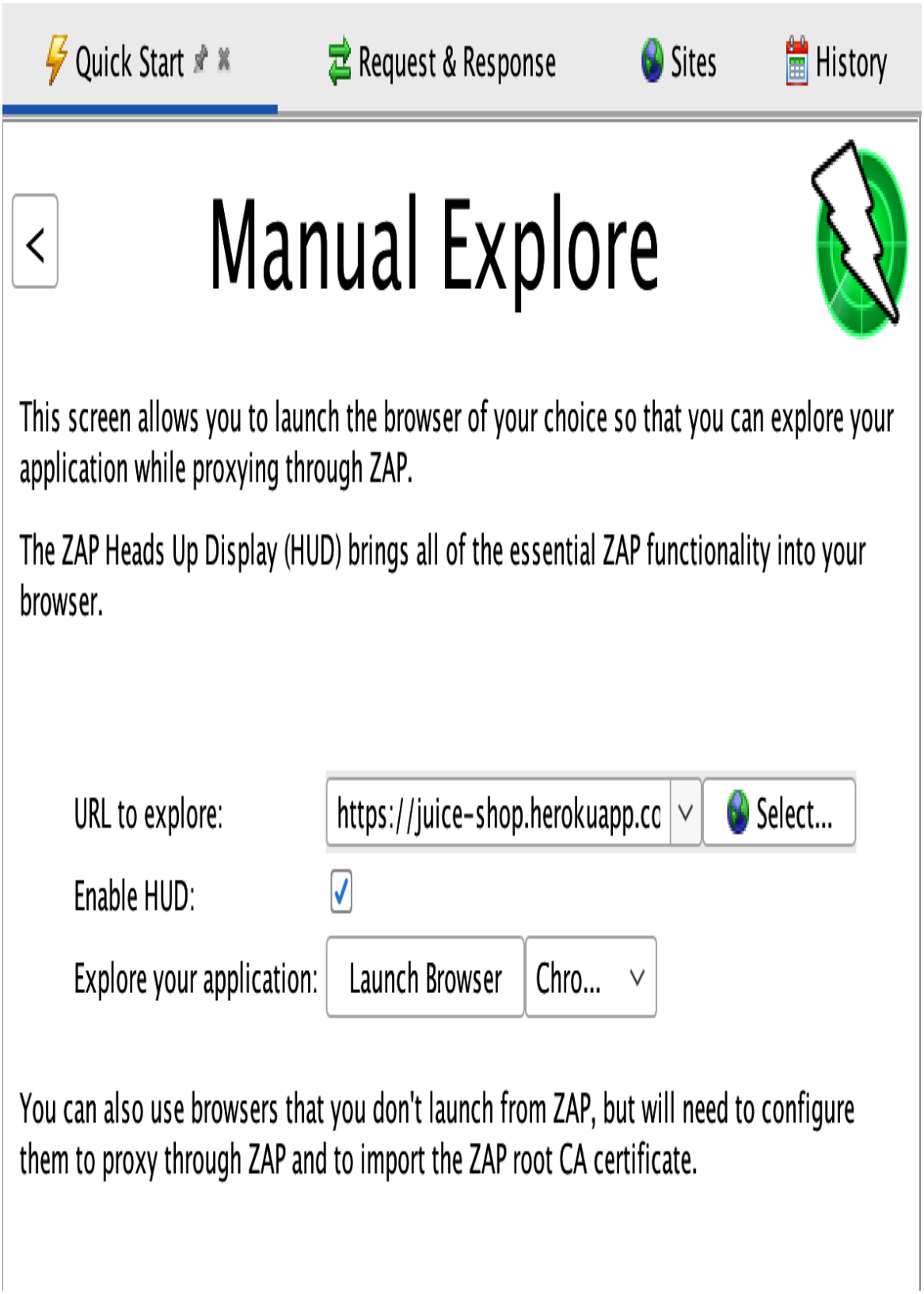


Figure 7-10. ZAP Manual Explore option

OWASP gives us a guinea pig website, the OWASP Juice Shop, that we can use to learn about security testing. I've used this URL. ZAP allows us to explore using both Firefox and Chrome; choose one. Once the browser opens the application, manually walk through the user flow once. ZAP will scan the application in the background and record the relevant details.

Notice the Enable HUD checkbox in Figure 7-10. A Heads Up Display (HUD) is a browser overlay that is displayed on top of your application. This is a provision given by ZAP to avoid the need to toggle between the desktop ZAP UI and the browser during attacks. If you have enabled it, you will see the HUD on top of the Juice Shop website, as in Figure 7-11 (the panels on the left and right).

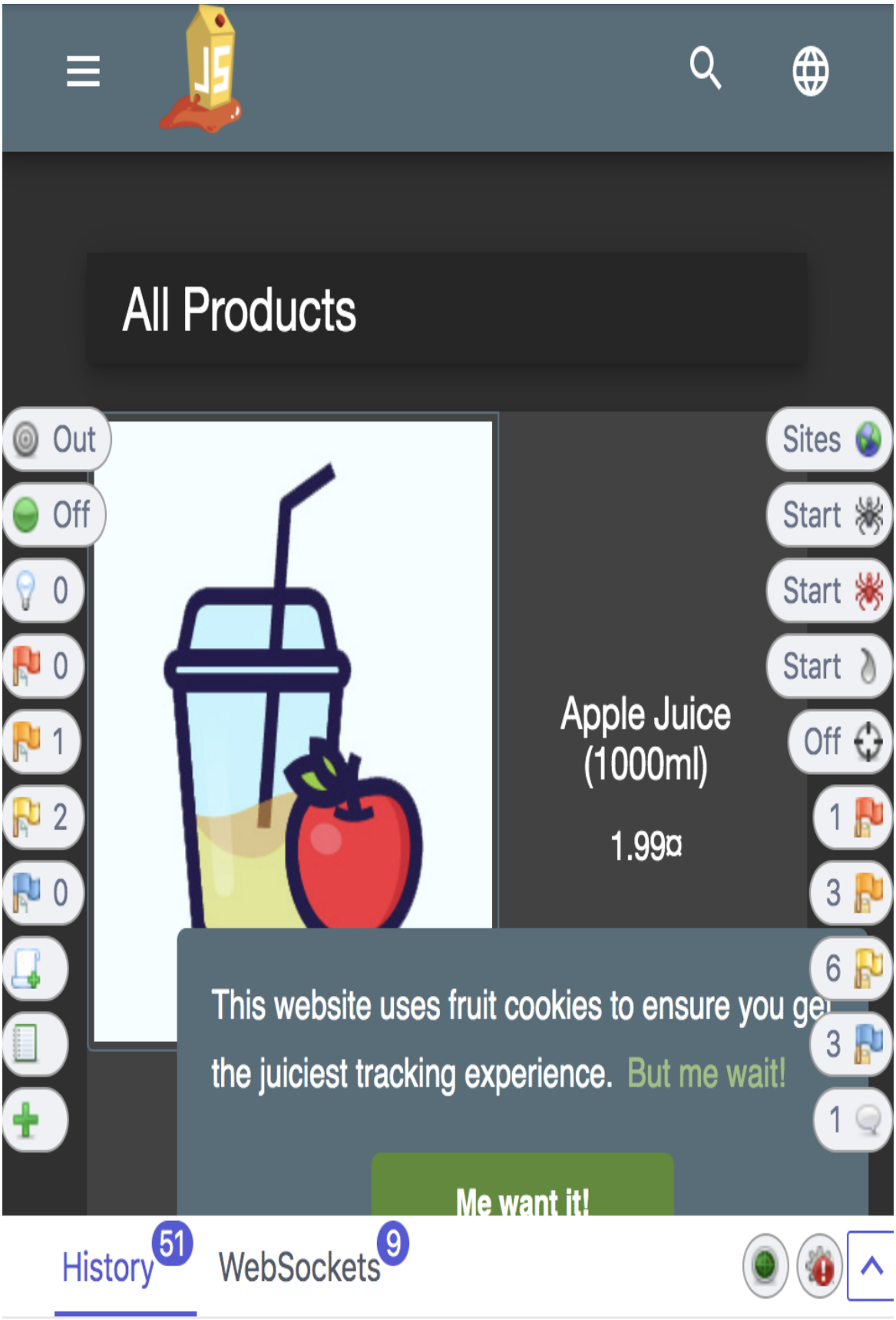

All Products
Out
Off
0
0
1
2
0
Sites
Start
Start
Start
Off
1
3
6
3
1
Apple Juice
(1000ml)
1.99¤
This website uses fruit cookies to ensure you ge
the juiciest tracking experience. But me wait!
Me want it!
History 51
WebSockets 9

Figure 7-11. The HUD on the Juice Shop website

As you explore the site manually, you'll see that the Sites icon in the right panel gathers the site tree, and the History tab at the bottom lists the URLs visited. ZAP will use these details later when it actively attacks the application.

ZAP Spider

The ZAP Spider removes the burden of manually exploring the website. It automatically crawls the site using Selenium WebDriver and gathers all the application URLs and UI components. The simple Spider (the gray icon under Sites on the right) might not be able to navigate JavaScript components, but the AJAX Spider (the red icon) can. You can use both to completely explore the application.

Scanning

As the ZAP Spider navigates through the application gathering information, it does passive scanning in the background. ZAP also does another type of scanning called an active scan, where it attacks the application! Try them now:

- Passive scanning involves reading the messages exchanged between the browser and the web application and inspecting them for vulnerabilities—the messages are not modified (i.e., attacked). This scanning is done automatically, so you will see alerts being shown in the right and left panels when the ZAP Spider is crawling. The alerts are prioritized based on severity (high, medium, and low) and grouped under red, orange, and yellow flags, as seen in Figure 7-11. On clicking those flags, you can see the details of the respective vulnerabilities. Also, the ZAP Desktop UI will have detailed logs. For example, a passive scan found a private IP address exposed in the Juice Shop website, as seen in Figure 7-12.

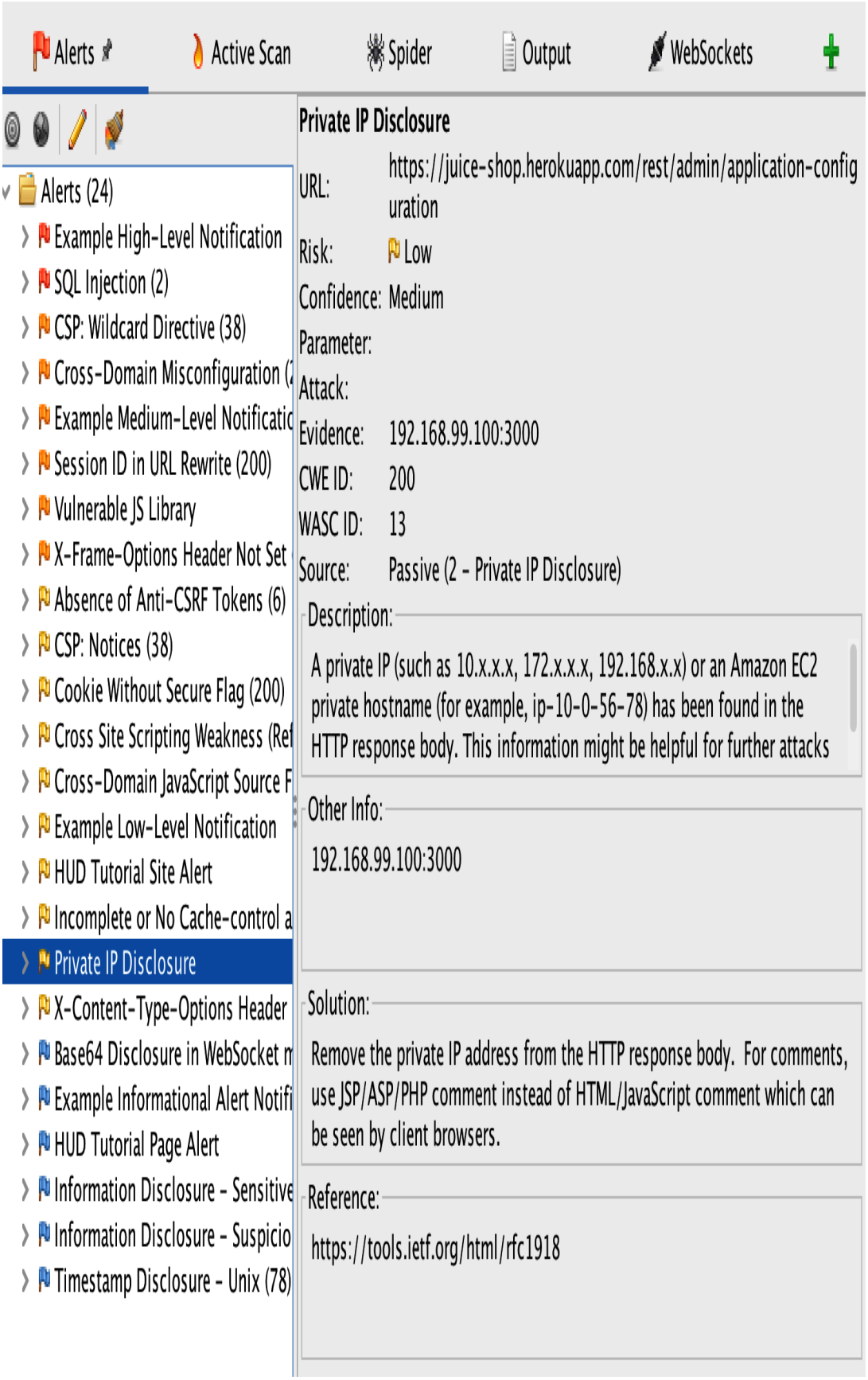

Alerts
Active Scan
Spider
Output
WebSockets
Alerts (24)
Example High-Level Notification
SQL Injection (2)
CSP: Wildcard Directive (38)
Cross-Domain Misconfiguration (
Example Medium-Level Notificatio
Session ID in URL Rewrite (200)
Vulnerable JS Library
X-Frame-Options Header Not Set
Absence of Anti-CSRF Tokens (6)
CSP: Notices (38)
Cookie Without Secure Flag (200)
Cross Site Scripting Weakness (Ref
Cross-Domain JavaScript Source F
Example Low-Level Notification
HUD Tutorial Site Alert
Incomplete or No Cache-control a
Private IP Disclosure
X-Content-Type-Options Header
Base64 Disclosure in WebSocket m
Example Informational Alert Notifi
HUD Tutorial Page Alert
Information Disclosure - Sensitive
Information Disclosure - Suspicio
Timestamp Disclosure - Unix (78)
Private IP Disclosure
URL: https://juice-shop.herokuapp.com/rest/admin/application-configuration
Risk: Low
Confidence: Medium
Parameter:
Attack:
Evidence: 192.168.99.100:3000
CWE ID: 200
WASC ID: 13
Source: Passive (2 - Private IP Disclosure)
Description:
A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks
Other Info:
192.168.99.100:3000
Solution:
Remove the private IP address from the HTTP response body. For comments, use JSP/ASP/PHP comment instead of HTML/JavaScript comment which can be seen by client browsers.
Reference:
https://tools.ietf.org/html/rfc1918

Figure 7-12. Passive scanning results in the ZAP Desktop UI

- During active scanning ZAP attacks the application by intercepting the requests, modifying them, and bouncing them back and forth, checking for known vulnerabilities like SQL injection and many more. Click on the fourth icon from the top in the right-side panel of the HUD (below the red spider) to trigger an active scan. You will immediately see ZAP navigating through the website page by page, mimicking different attacks. The scan takes a while; once it's complete, you can see the vulnerabilities found under the colored flags. Figure 7-13 shows a SQL injection vulnerability found on the Juice Shop website.

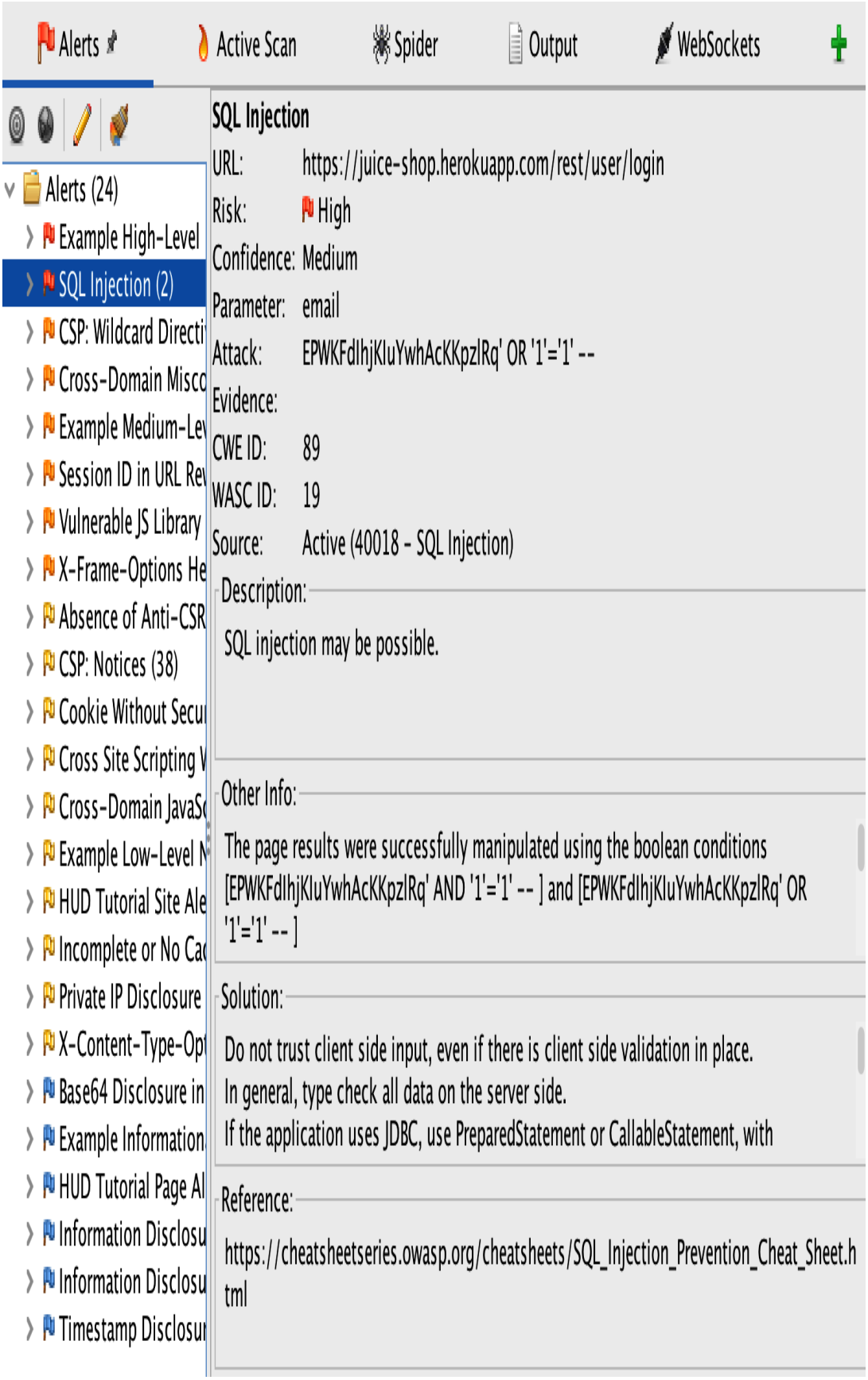

Alerts
Active Scan
Spider
Output
WebSockets
Alerts (24)
Example High-Level
SQL Injection (2)
CSP: Wildcard Directi
Cross-Domain Misco
Example Medium-Lev
Session ID in URL Rev
Vulnerable JS Library
X-Frame-Options He
Absence of Anti-CSR
CSP: Notices (38)
Cookie Without Secu
Cross Site Scripting V
Cross-Domain JavaSc
Example Low-Level N
HUD Tutorial Site Ale
Incomplete or No Cac
Private IP Disclosure
X-Content-Type-Opt
Base64 Disclosure in
Example Information
HUD Tutorial Page Al
Information Disclosu
Information Disclosu
Timestamp Disclosur
SQL Injection
URL: https://juice-shop.herokuapp.com/rest/user/login
Risk: High
Confidence: Medium
Parameter: email
Attack: EPWKFdlhjKluYwhAcKKpzlRq' OR '1'='1' --
Evidence:
CWE ID: 89
WASC ID: 19
Source: Active (40018 - SQL Injection)
Description:
SQL injection may be possible.
Other Info:
The page results were successfully manipulated using the boolean conditions [EPWKFdlhjKluYwhAcKKpzlRq' AND '1'='1' --] and [EPWKFdlhjKluYwhAcKKpzlRq' OR '1'='1' --]
Solution:
Do not trust client side input, even if there is client side validation in place.
In general, type check all data on the server side.
If the application uses JDBC, use PreparedStatement or CallableStatement, with
Reference:
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Figure 7-13. A SQL injection vulnerability found in during an active scan

That's how simple ZAP makes DAST for all software teams: just open the application in the ZAP UI, use the spiders to perform a passive scan, and trigger an active scan!

Integrating ZAP with CI

Now that ZAP has provided you with a list of all the vulnerabilities in the application, you need to analyze them and decide which ones to fix. This takes a lot of time and expertise. It's advisable to do this exercise continuously instead of leaving this work to pile up at the end—what better option than integrating with CI to get continuous feedback?

> **NOTE**
>
> As mentioned previously, the active scan can take a long time to complete, depending on your application (sometimes even hours). In that case, you can choose to integrate it with CI as a nightly regression stage or include a manual trigger that you can use for every user story.

To integrate with CI, ZAP provides APIs such as the following:

- `zap.urlopen(target)` to open the application
- `zap.spider.scan(target)` to trigger the ZAP Spider and do passive scanning
- `zap.ascan.scan(target)` to trigger an active scan
- `zap.core.alerts()` to print the results found

You can use these APIs in a simple JavaScript or Python script to perform the scans and integrate with CI using the ZAP CLI.

Alternatively, you can embed the ZAP APIs within your Selenium WebDriver functional tests and run them like typical functional tests

in the CI pipeline. WebDriver can also help ZAP to log in to the website, which it might not be able to do on its own. Example 7-3 shows a sample test that scans the application and fails if there are vulnerabilities. Note that you should add appropriate waits, depending on your application's scan time.

Example 7-3. ZAP scan as part of Selenium tests

```
@Test
public void testSecurityVulnerabilities() throws Exception {

    zapScanner = new ZAProxyScanner(ZAP_PROXYHOST,
ZAP_PROXYPORT, ZAP_APIKEY);
    login.loginAsUser();

    // Step 1- Spider the app using ZAP API
    zapSpider.spider(BASE_URL)

    // Step 2 - Enable passive scanning
    zapScanner.setEnablePassiveScan(true);

    // Step 3 -Start Active scan. Add wait methods.
    zapScanner.scan(BASE_URL);

    // Step 4 - Log the alerts and assert the count of alerts
    List<Alert> alerts =
filterAlerts(zapScanner.getAlerts());
    logAlerts(alerts);
    assertThat(alerts.size(), equalTo(0));
}
```

ZAP produces HTML reports of the vulnerabilities, as seen in Figure 7-14, that can be saved as output artifacts in CI.

High (Medium)	SQL Injection
Description	SQL injection may be possible.
URL	https://juice-shop.herokuapp.com/rest/user/login
Method	POST
Parameter	email
Attack	EPWKFdlhjKluYwhAcKKpzlRq' OR '1'='1' --
URL	https://juice-shop.herokuapp.com/rest/user/login
Method	POST
Parameter	email
Attack	VqqxCXFFxHhqClxYYvCGioKa' OR '1'='1' --
Instances	2
	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'

Figure 7-14. ZAP HTML report

Lastly, if you're using GitHub Actions for your CI/CD, you have an easy option to integrate ZAP using the predefined OWASP ZAP Baseline Scan and OWASP ZAP Full Scan Actions. As their names

suggest, they perform the ZAP scan and also add the issues to GitHub.

Beyond those discussed here, ZAP has many other helpful features that enable various kinds of exploratory security testing on the application. A few are listed here:

- ZAP can use OpenAPI specifications to do security testing on APIs.
- It has a feature called Breaks that will help you insert specific test data into a request and observe the behavior. For example, to test if the API validates the input parameters for SQL injection, you can use the Break feature.
- It allows replaying a request in the browser.
- There is on option to highlight specific hidden keywords in the HTML.
- It has a feature to disclose all hidden input fields in the application.
- There are add-ons with prewritten scripts crafted by experts which can be used to play specific types of attacks if needed.

Overall, ZAP is an excellent tool to try and can teach you many things about security.

Additional Testing Tools

A few more tools that aid in SAST and manual exploratory security testing are discussed here to give a broader perspective on security-related tools that can be adopted during the software delivery cycle.

Snyk IDE Plug-in

The Snyk JetBrains IDE plug-in combines both SCA and SAST capabilities. It is entirely free and can be used with any of the JetBrains IDEs (e.g., IntelliJ IDEA, WebStorm, PyCharm). The biggest advantage is that it is so close to development and a prominent shift left. You can trigger scans to check for vulnerabilities in both application code and their dependencies while the code is being developed. Figure 7-15 shows the results of such a scan displayed in the bottom panel of the IntelliJ IDE. You can see that Snyk has highlighted an “Information disclosure” vulnerability in the application code. It also shows remediation options to fix the vulnerabilities it finds, making it easier for developers to build security in.

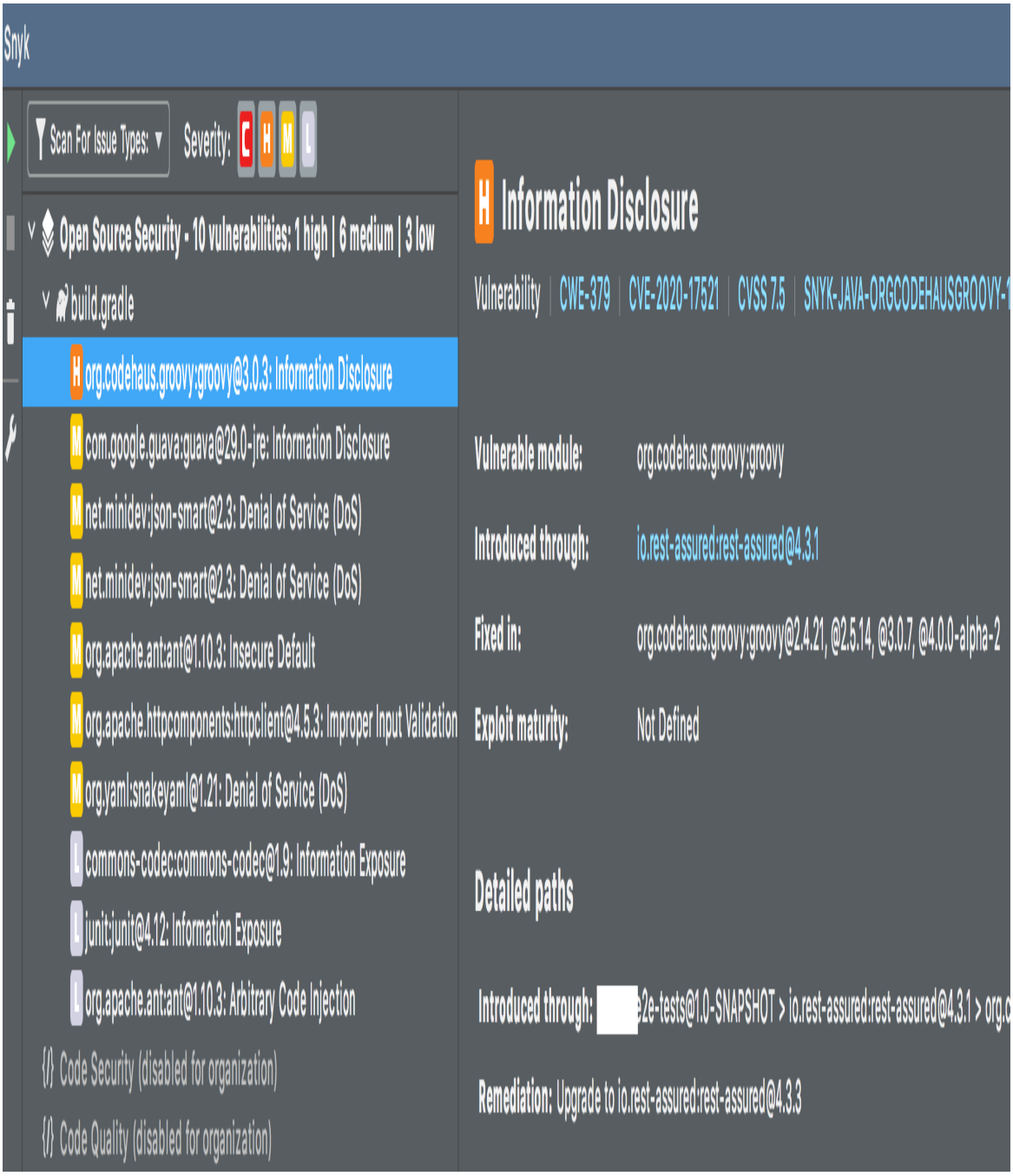


Figure 7-15. Snyk IDE plug-in example scan results

Snyk also comes as a CLI option, but only with SCA capabilities to be integrated with CI. The company also provides a suite of other security-related services as paid options.

Talisman Pre-Commit Hook

Talisman, an open source tool, scans your application code for secrets and sensitive information (such as passwords, SSH keys, tokens, etc.) when you commit to the version control system and raises alerts when it finds them. This is very helpful in preventing development teams from committing secrets by accident. You can configure it either as a pre-commit hook or a pre-push hook. Example 7-4 shows a sample scan result when trying to commit code using Git.

Example 7-4. Talisman sample scan results

```
$ git commit
Talisman Report:
+----------------+-------------------------------------------------------------
---+
|     FILE       |                                ERRORS
|
+----------------+-------------------------------------------------------------
---+
| sampleCode.pem | The filename "sampleCode.pem"
|
|                | failed checks against the
|
|                | pattern ^.+\.pem$
|
+----------------+-------------------------------------------------------------
---+
| sampleCode.pem | Expected file not to contain hex-encoded texts such as:
|
|                | awsSecretKey=
|
|                |
c99e0c79ddcf5ddb02f1274db2d973f363f4f553ab1692d8d203b4cc09692f79 |
+----------------+-------------------------------------------------------------
---+
```

Talisman has identified the presence of `awsSecretKey` in the application code. With bots crawling GitHub repositories looking for secrets, as discussed earlier, this is an important tool that you should add to your repertoire.

Chrome DevTools and Postman

For performing manual exploratory security testing around functional use cases, such as the different test cases that result from a threat modeling exercise, Chrome DevTools and Postman are pretty handy.

We discussed Postman's features at length in Chapter 2. Postman also enables you to do security-related exploratory testing such as configuring auth tokens as part of API requests, as seen in Figure 7-16. You can use this feature to test scenarios like tampered-with and expired access tokens.

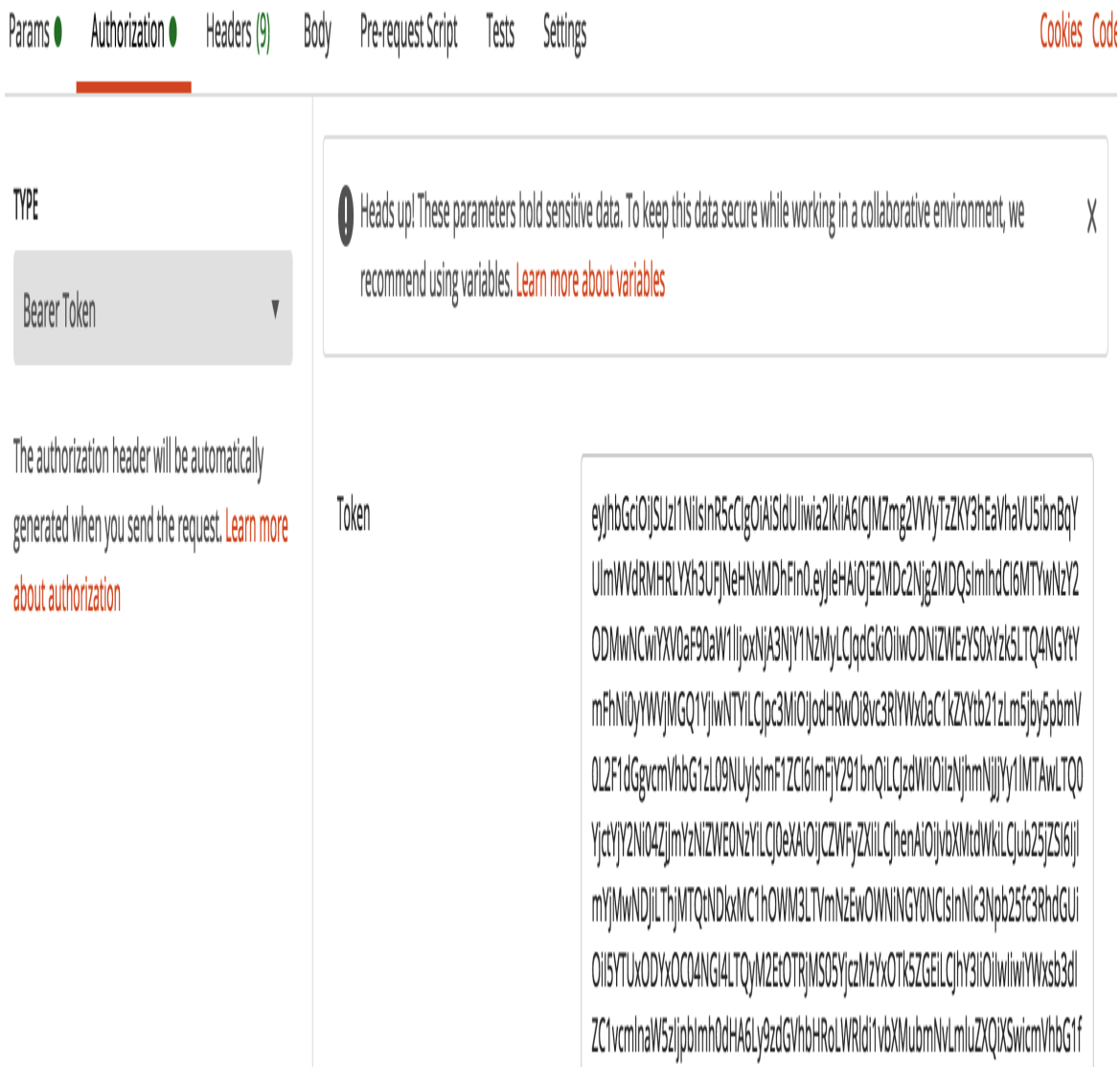


Figure 7-16. Postman access token setup

Similarly, the Security tab in Chrome DevTools, as seen in Figure 7-17, tells you whether the page is properly served over HTTPS or not. It also highlights when resources from third-party sites are not

served securely, as this can potentially lead to man-in-the-middle attacks.

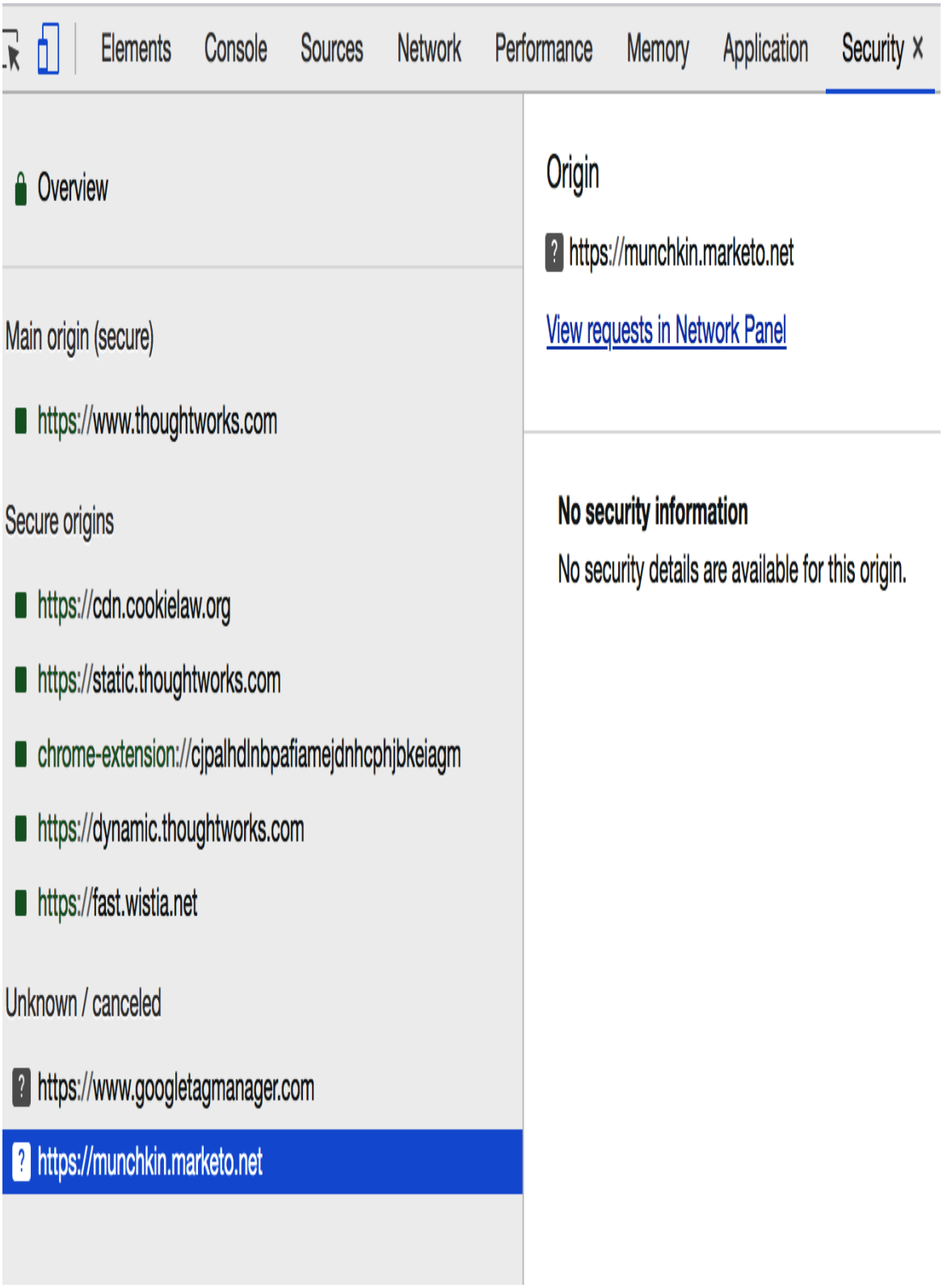


Figure 7-17. The Security tab in Chrome DevTools

With the security testing tools we've explored here, it should be clear now that security testing can be embedded within the software development cycle and needn't be the sole responsibility of expert pen testers. Shift-left security testing will help prevent your team from facing major security issues later.

Perspectives: Security Is a Habit

One of my observations from experience is that no matter how much effort we pour into the various activities discussed in this chapter trying to create secure software, unless we make security a habit, we might still leave unexpected weak links in our applications that could be exploited. In fact, several common practices in software teams could easily lead to a security compromise. For example, have you thought about the security aspects of the tools that you use to assist in development and testing? Do any of them store project data in the vendor's private cloud? Are you leaving the architecture diagram of the project with environment details in an online portal? Did you share the production system credentials with all your team members? If so, did you share them in Slack in plain text? Many such simple acts could easily result in a compromise.

Therefore, making security a habit is the only way forward. It's the same as how we look at food before eating it or notice when someone is following us. We do those checks unconsciously and naturally. Similarly, we, as software teams, should train ourselves to make security an unconscious and natural habit. We can start by questioning ourselves on a daily basis about whether any of the simple, unintentional, harmless acts we are performing could result in a security breach!

Key Takeaways

Here are the key takeaways from this chapter:

- Cybercrimes in the digital era are more prevalent than you might think, with annual revenues expected to surpass $10 trillion in just a few years.
- Real-life examples of attacks show that all sorts of digital platforms are hacked with the intention to steal money, private data, infrastructure, and so on. So, clearly, security is no longer just a “nice-to-have” feature.
- Security measures must be built into the application throughout the software development lifecycle, from analysis to testing, in order to achieve the goal of building strong, impenetrable systems.
- The STRIDE model gives a structured lens through which explore security threats for an application, which can be applied to perform threat modeling.
- Threat modeling exercises should be done frequently with the entire team for small portions of application functionality, such as a user story or a feature. Threat modeling should result in creating abuser stories and security-related test cases.
- Deploy shift-left security testing strategies using various types of automated security testing tools (SAST, SCA, DAST, etc.) , as well as manual exploratory and functional automated testing.
- With the availability of accessible automated security testing tools, software teams don’t have to wait for penetration testing as the only way to get feedback on security issues.
- Most importantly, make security a habit.

---

1 For an excellent introduction to principles and best practices for writing secure software, see Daniel Deogun, Dan Bergh Johnsson, and Daniel Sawano’s *Secure by Design* (Manning).

2 For more on this topic see Wade Trappe and Lawrence C. Washington's *Introduction to Cryptography with Coding Theory*, 3rd Edition (Pearson).

Chapter 8. Performance Testing

Time is money!

—Benjamin Franklin

We've all experienced it: sometimes our favorite websites suddenly become as slow as sloths, leaving us wondering, "Is there a problem with my internet?" Remember waiting an eternity during the Cyber Monday sales for a website to load? Or staring at the loading icon waiting for train tickets to show up when you are dying to book your Christmas vacation? Or being left hanging on the booking page for a blockbuster movie? Poor website performance in cases like these can cause us as customers to feel intense frustration.

If you want to save your application's end users from such frustrations, you have to continuously measure and work to improve its performance. This chapter aims to equip you with the essentials you need for measuring or testing web performance—specifically, we'll cover topics like performance KPIs, API performance testing, frontend performance testing, and shift-left performance testing. You will also get a chance to try both frontend and API performance testing hands-on as part of this chapter's exercises.

Because performance testing is such a large topic and must be done both on the backend and the frontend, this chapter is structured a little differently from the previous ones. You'll find all the familiar sections here, but split into two halves. We'll start by covering everything you need to know to get up to speed with backend performance testing, including exercises and additional tools. Once that's done, we'll shift our attention to frontend performance testing. A global strategy for shift-left performance testing is presented toward the end of the chapter.

Backend Performance Testing Building Blocks

Let's first take a look at why performance is so critical to a business's success. Here, we'll explore factors that affect an application's performance, key indicators of a web application's performance, and ways to measure them.

Performance, Sales, and Weekends Off Are Correlated!

We spoke about customers becoming frustrated because of application performance at the beginning of the chapter. We need to understand what that might lead to. How much can a few seconds' delay really matter? In fact, there is a quantitative indicator that provides a measure of the effect of page load times on customer behavior. The *bounce rate* is a measure of the percentage of customers that leave a website after viewing just one page.

Among all the potential factors that might increase the bounce rate, website performance has been shown to be the major contributor. Statistics published by Google (see Table 8-1) show the correlation between page load time and user bounce rates. The data confirms that with every additional second's delay, businesses lose customers to their competitors.

Table 8-1. Google statistics scorrelati

n
g

p
a
g
e

l
o
a
d

ti
m
e

a
n
d

b
o
u
n
c
e

r
a
t
e
s

Page load time	Increases the probability of bounce by
1–3 seconds	32%
1–5 seconds	90%
1–6 seconds	106%
1–10 seconds	123%

And there's more—Google's search engine optimization (SEO) algorithms rank slower websites lower, which means if your website is not performing enough, it will get pushed further down into the abyss! For its own website, Google aims for a load time of under a half-second, and it recommends 2 seconds as the maximum for acceptable website performance.

Losing customers translates to losing sales, and businesses can pay a very heavy price for performance failures. For example, in 2018, Amazon faced an estimated loss of $72–99 million when its website failed to handle the traffic for its Prime Day event. Lousy performance can also lead to a loss of reputation for the brand, especially in a world where, thanks to social media, bad reviews can spread so rapidly.

On the flip side, a slight increase in performance can result in significant improvements in sales. For instance, in 2016, the Trainline, a train operating company in the UK, reduced its average page load time by 0.3 s, and revenue increased by £8 million ($11 million) a year. Similarly, frontend-as-a-service provider Mobify observed that every 100 ms decrease in load time for its home page increased conversions at a rate that translated to an annual revenue boost of $380,000. The correlation between sales and performance makes it clear that the first step to improving sales for an online business is to look at its application's performance. This means we,

as software teams, need to build and test for performance early and frequently—i.e., shift our performance testing to the left.

One of my primary motivations to focus on website performance early is straightforward: I love my weekends and want to spend them relaxing. Since performance issues can be very costly, as you'll have observed from the earlier examples, and directly affect the brand reputation, when they occur in production software development teams usually are placed under high pressure to fix them ASAP. So, if you do not incorporate performance testing early and often during the software development lifecycle, you can expect to pay for it later by working on weekends (and long hours) to fix the performance issues that arise!

Simple Performance Goals

Performance, in simple terms, can be thought of as the ability of an application to serve large numbers of concurrent users without significant degradation in its behavior compared to when it is serving only a single user. That is, the performance must not degrade beyond a point that is acceptable to the end users. So, to test for performance, first you need to determine the expected number of peak-time users for your application, then you need to verify that the application's performance under that level of load remains acceptable.

What constitutes acceptable performance is largely dependent on the limits of human perception. According to web usability and human–computer interaction researcher Jakob Nielsen, when the response time of the site is less than about 0.1 second, the user feels the behavior is instantaneous. With response times of 0.2 to 1 second, they perceive the delay but still feel that they are in control of the navigation on the website. Beyond this, they feel the UI is sluggish and lose the sense of flow in performing their desired task. As we saw earlier, Google's research shows that with delays beyond

3 seconds you are at risk of losing the majority of your customers, and they recommend keeping the page load time at less than 2 seconds.

These are your performance goals. To achieve such good results, a lot of infrastructure tuning and code optimization needs to happen in many iterations before your application goes live—yet another reason to adopt a shift-left performance testing strategy!

Factors Affecting Application Performance

Achieving the performance goals laid out in the previous section is not that straightforward—if it were, businesses wouldn't have lost so much money due to performance issues. There are many factors in an application that affect performance, including those listed here:

Architecture design

> Architecture design plays a vital role in the performance of a website. For instance, when the responsibilities of the web services are not properly compartmented, numerous calls will have to be made to different services from the UI, delaying the response time. Similarly, when appropriate caching mechanisms are not implemented at the right levels, website performance will be affected.

Choice of tech stack

> Different layers of the application need different sets of tools. These tools may fail to work together coherently, affecting the overall performance. To give just one example of the kinds of interactions you need to consider, the choice of language (e.g., Java, Ruby, Go, Python) can have a perceptible impact on AWS Lambda cold startup time.

Code complexity

Complex or poorly written code (think complicated algorithms, long operations, missing or duplicate validations, etc.) often leads to performance issues. Consider the case where a search is done with an empty string. What would be optimal is for the search endpoint to do some simple input data validation and fail the request quickly. Failing this, the service will search the database and then return an error, delaying the response time unnecessarily.

Database choice and design

Databases play a key role in application performance. There are various types of databases, as discussed in Chapter 5. If your application requires very high performance, choosing a suitable database type and properly organizing the data inside it will be critical. For instance, storing the details of a single purchase order across multiple tables will require consolidation and delay the retrieval of the final order. Structuring the data properly with performance in mind is essential.

Network latency

The central nervous system for any application is the network. All the components in an application communicate internally via some kind of network. So, ensuring good connectivity between components is crucial, be it within the same datacenter or across multiple datacenters. Further, end users around the globe will interact with the application using their own networks (2G, 3G, 4G, WiFi). The quality of those networks is outside the control of software teams, but designing the application to cater to users with weak network connectivity is within their purview. A good UX design avoiding heavy images and substantial data transfers is important for boosting application performance for all users.

Geolocation of the application and users

If the users of your website are only from a particular region, then having the website hosted physically close to that region will reduce the number of network hops and hence the latency. For example, if the website is for European customers but is hosted in Singapore, connecting to the system will require multiple to-and-fro network hops; hosting it somewhere in Europe will improve performance for end users. Conversely, if the website intends to serve customers around the world, there should be a strategy to replicate it in different hosting locations (or use content delivery networks [CDNs]). If you use cloud infrastructure, you should remember to request machines that are physically closer to your intended customers—a common mistake is using infrastructure that is closer to the development team's location.

Infrastructure

Infrastructure is the skeleton that supports all the muscles of a system. The power of your infrastructure, in terms of CPU, memory, etc., will directly impact the system's ability to take the load. Designing infrastructure to deliver a high-performing system is an art in itself. Infrastructure engineers continuously collect the results of the performance tests as one of the parameters to plan the infrastructure needs of the application.

Third-party integrations

When there are integrations with third-party components, the application is dependent on those components' performance. Any latency in a third-party component will eventually add to the latency of the application itself. For example, as discussed in Chapter 3, a typical retail application integrates with many external services, such as vendors' product information management systems, warehouse management systems, etc., and in such cases, choosing high-performing components is vital.

During performance testing, you should consider all of these factors in order to simulate real-world test cases. For instance, you need to set up a performance testing environment that is very similar to the production environment in terms of network, infrastructure, geolocation, etc. Otherwise, you may not have an accurate measure of performance!

Key Performance Indicators

Measuring or testing an application's performance involves capturing a set of quantitative key performance indicators (KPIs). Measuring these continuously throughout the development cycle will help the team to course-correct earlier and with less effort. As a general rule, the KPIs you should monitor are:

Response time

Response time refers to the time taken by the application to answer a query by the user—for example, the exact time taken to show the results of a product search query to the customer. As we saw earlier, the expected response time for web applications is at most 3 seconds; beyond this, they risk losing the majority of customers. Note that 3 s is the delay experienced by the end user, and thus includes both the API response time and the time taken by the frontend to load the page fully.

Concurrency/throughput

Websites may be accessed by numerous users from across the globe at any given point in time. Indeed, some high-speed applications such as stock exchange sites cater to millions of transactions per second. Establishing that the application can support a given volume of users within the acceptable limits at a given point in time is referred to as measuring *concurrency*. For example, you might want to validate that the application can respond within 3 seconds to 500 concurrent users.

Although “concurrent users” is a term commonly used by businesses and software teams, when we think from the system’s perspective, it receives various requests from end users and other components, which are queued and selected for processing one after the other by parallel threads. Hence, from its perspective using the number of concurrent users as an indicator doesn’t sit well. Instead, a better indicator to measure is the *throughput*. Throughput measures the number of requests the system can support during an interval of time.

To understand this better, consider the analogy of cars crossing a very short bridge over a river. Let’s say there are four car lanes. Assuming traffic is flowing smoothly, each car may be able to cross the bridge in a few hundred milliseconds. So, in a second, the total number of cars crossing the bridge will be 30 to 40. This value of 30–40 cars per second is the throughput.

Concurrency and throughput are both helpful in server capacity planning and are often used in different contexts to make impactful decisions.

Availability

Availability is a measure of the system’s ability to respond to the end users within the same acceptable limits over a given continuous period. Typically, websites are expected to be available 24/7 except for planned maintenance. Availability is an essential criterion to test because an application might perform well for the first half-hour, but responses could degrade over time due to memory leakage, overconsumption of the infrastructure’s capacity by parallel batch jobs, and many other unpredictable reasons.

Now that we’ve discussed the KPIs, let’s take a look at how to measure them.

Types of Performance Tests

To measure KPIs, you need to specifically design your performance tests in a certain fashion. The following list describes three common types of performance tests:

Load/volume tests

As discussed earlier, concurrency or throughput is measured to validate that the application can serve the expected volume of users in an acceptable time. For instance, suppose you want the search functionality to respond within 2 seconds for a volume of 300 users. A performance test to simulate this volume of users and validate whether the application meets the expected target response time is called a *volume test* or *load test*. You may have to repeat such tests multiple times to observe consistency and measure the average to benchmark the application.

Stress tests

A commonly observed behavior is that an application's performance starts degrading as more users are stressing it. For example, it may perform within acceptable limits for X users, but beyond X users it starts to respond with delays, and finally, at X+n users, it responds with errors. You need the exact measure of these figures. This measure will be used in planning the infrastructure when scaling the application to new regions, or during events such as sales. The performance test will be designed to slowly increase the load on the application in small steps beyond the volume test limits, to determine precisely the point at which it responds with errors. This process of stressing the system to find the breaking point is called *stress testing*.

Soak tests

When the application runs with the expected volume of users for a while, there may be a degradation in response time due to

infrastructure issues, memory leakage, or other issues. Performance tests designed to keep the application under a constant volume of load for an extended period and observe the behavior are called *soak tests*.

While designing all of these tests, an important point is to keep them realistic and avoid overloading the application with extreme situations that may never occur. For instance, not all users will log in to the application at the exact same instant. A more realistic use case will be users logging in gradually, with gaps of a few milliseconds in between. This delay between the start of the test and the time when all the virtual users are considered to be connected is called the *ramp-up time*. Your test cases should include such a practical design; for example, you might plan to ramp up 100 users in 1 minute.

Furthermore, users aren't robots capable of logging in, searching for a product, and completing a purchase within milliseconds—but performance test cases might be designed that way unintentionally. In reality, users take at least a few seconds to think between actions and typically take minutes to complete a transaction like buying a product after logging in. This is called the *think time* in performance testing terms. You need to include appropriate think time in your test cases and spread the user actions apart by a few seconds or minutes. Related to think time is another concept called *pacing,* which defines the time between transactions (not user actions). In real life, users could initiate transactions again after some time. So, if you're expecting 1,000 transactions per hour during peak-hour sales, you can spread the transactions over the hour by configuring the pacing time. These three attributes must be tuned wisely to measure an application's performance realistically.

Types of Load Patterns

We spoke about the different types of performance tests used to measure KPIs in the previous section. These performance tests translate to generating different load patterns on the application, using the attributes we just discussed: the ramp-up time, think time, number of concurrent users, and pacing. We'll discuss some commonly tested load patterns in this section:

Steady ramp-up pattern

In the steady ramp-up pattern (illustrated in Figure 8-1), the users are steadily ramped up within a given period, and then the load is maintained constantly for a sustained period to measure performance. This is a very common pattern in real-world scenarios—for example, the Black Friday sales—where the users gradually but steadily come to the application and stay there for a while before dropping out steadily.

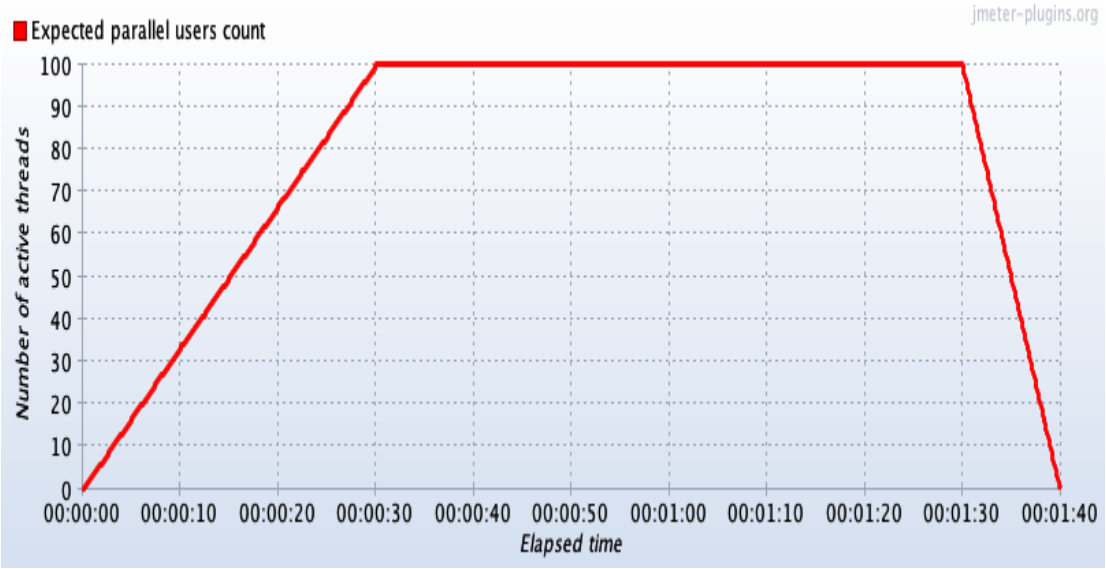


Figure 8-1. Steady ramp-up of users

Step ramp-up pattern

With the step ramp-up pattern (Figure 8-2), users are ramped up in batches periodically—for example, 100 users every 2 minutes. Observing and measuring the application's performance for each step count of users will help benchmark the application for

different loads. The step ramp-up pattern is useful in performance tuning and infrastructure capacity planning.

> **NOTE**
>
> Benchmarking is measuring the average response time from repeated runs.

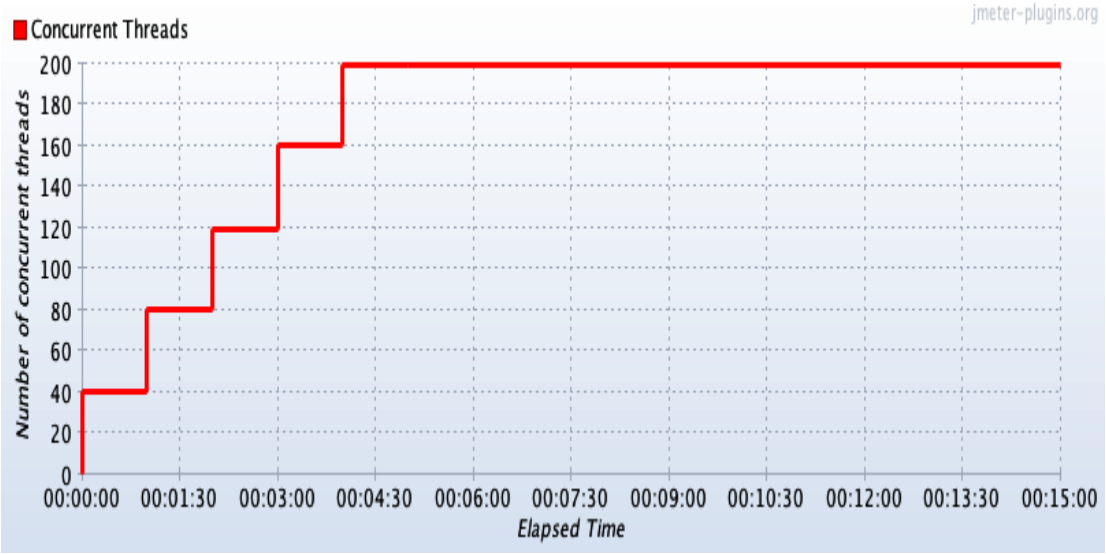


Figure 8-2. Step ramp-up of users

Peak-rest pattern

The peak-rest pattern (Figure 8-3) is when the system is ramped up to reach peak load and then ramped down to complete rest in repeated cycles. This scenario can be observed in some applications like social networking sites, where the peak comes and goes in cycles over the course of a day.

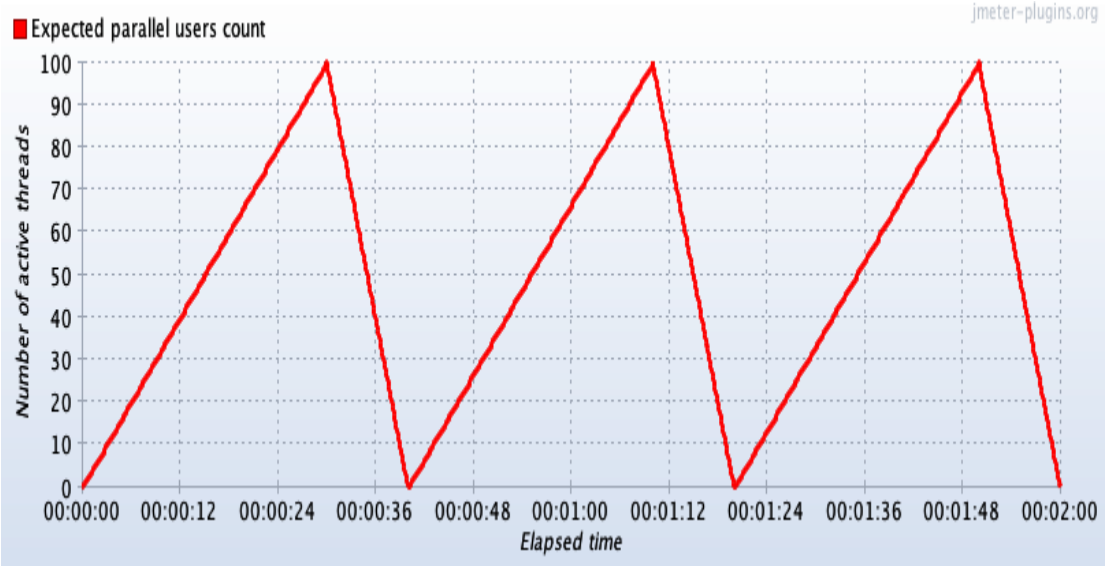


Figure 8-3. Peak-rest load pattern

Performance testing tools lend a hand in generating these patterns easily, as we shall see later in the chapter.

Performance Testing Steps

Now that we've discussed the KPIs, performance test types, and load patterns, the next step is to walk through the steps of a performance testing exercise. This will help you plan the time and capacity needed for performance testing in your project.

Step1: Define the target KPIs

The first step is defining the target KPIs based on business needs. The best way to start thinking about the target numbers is to consider them qualitatively, and then translate them into numbers.[1] For instance, qualitative thinking about performance could lead to goals such as:

- The application should be able to scale to one more new country.
- The application should perform better than its competitor X.

- The new version of the application should perform better than the previous version.

These qualitative goals naturally lead toward the next steps. If the goal is to do better than the last version of the application, you need to measure the performance of the earlier version and see if your current numbers are better. Similarly, if you know the competitor's performance numbers, you need to validate that your numbers are better than theirs.

> **NOTE**
>
> Business folks tend to call out performance numbers that may not reflect the actual usage pattern. Always derive the target KPIs from data:
>
> - If there is an existing application, analyze the production data to arrive at KPIs and load patterns.
> - If you are building a new application, ask for competitors' data.
> - If the application is completely new with no reference data, use data around country-wide internet usage, probable peak duration, etc., to work out your target KPIs.

Step 2: Define the test cases

The second step is to describe the test cases using the load patterns and the performance test type semantics. Your test cases should mandatorily cover measuring the availability, throughput, and response time of all the critical endpoints in the application. The performance test cases will subsequently reveal the test data setup needed to run the test cases. At the end of the day, you may only need a handful of performance test cases, unlike with the functional test cases.

Step 3: Prepare the performance testing environment

As mentioned earlier, the performance testing environment should be as close to the production environment as possible so that you can get realistic results. This will also help you identify any performance bottlenecks in the environment configurations.

Here is a sample checklist for achieving this goal, which you can adapt to your circumstances:

- The respective tiers/components should be deployed in a similar fashion.
- The machine configurations (number of CPUs, memory capacity, OS version, etc.) should be similar.
- The machines should be hosted in the same geolocation in the cloud.
- Network bandwidth between machines should be similar.
- Application configurations like rate limiting should be precisely the same.
- If there will be batch jobs running in the background, those should be in place. If there are emails to be sent, those systems should be in place too.
- Load balancers, if any, should be in place.
- Third-party software should be available at least in a mocked capacity.

Setting up a production-like environment for testing is often challenging due to the additional costs involved, although cloud provisions are cheaper. You may need to have a cost vs. value conversation with the business stakeholders. If you don’t win that battle, prepare to make meaningful trade-offs on certain parts of the performance environment setup and make it clear to the respective stakeholders that because of those concessions the performance numbers measured might not be foolproof.

> **NOTE**
>
> A best practice is to request that the performance testing environment be set up alongside the QA environment right at the beginning of the project so it's available when you need it.

Apart from the performance testing environment, you also need a separate machine to be the test runner—i.e., to run the performance tests. Plan to have individual test runners hosted in different geolocations (this is possible with cloud providers) to observe the respective performance behaviors with network latencies from multiple countries, if your application is intended to serve a global audience.

Step 4: Prepare the test data

Just as the performance testing environment should be as similar as possible to the production environment, the test data should be as reflective as possible of the production data. The performance numbers that you will measure will greatly depend on the test data quality, and hence this is a critical step. An ideal situation would be to use actual production data after anonymizing any sensitive user information, as it will reflect the actual database size and data complexity. However, this may not be possible due to security concerns in certain situations. In such cases, prepare test data that closely mimics the production data.

A few pointers when creating production-like data are:

- Estimate the size of the production database (e.g., 1 GB or 1 TB) and set up scripts to populate the test data. It may be necessary to clean and repopulate the test data for every test run, so having the test data creation and cleanup scripts will be crucial.

- Create a variety of test data similar to what is observed in production. Instead of “Shirt1,” “Shirt2,” etc., use actual production-like values such as “Van Heusen Olive Green V-Neck Tshirt.”
- Populate a fair share of erroneous values, like addresses with spelling mistakes, blank spaces, etc., that might represent actual user inputs.
- Have a similar distribution of data across factors like age, country, etc.
- Depending on the test cases, you may have to create a lot of unique data like unique credit card numbers, login credentials, etc., to run volume tests with concurrent users.

Yes, preparing the test data can be a tedious job! These activities need to be planned well ahead of time in the release cycle. It’s impossible to squeeze this in later as an afterthought, and if you try to do so the test data might not be of good quality, resulting in inaccurate performance numbers.

Step 5: Integrate APM tools

The next step is to integrate application performance monitoring (APM) tools (e.g., New Relic, Dynatrace, Datadog) so that you can see how the system behaved during the performance tests. These tools greatly help in debugging any performance issues. For instance, requests may fail during performance test runs due to insufficient memory in the machine, and the APM tools will expose such issues easily.

Step 6: Script and run the performance tests using tools

The last step is to script the performance test cases using tools and run them against the performance testing environment. There are many tools you can use to script and run your performance test cases with a single click and also integrate them with CI to help you

shift left. JMeter, Gatling, k6, and Apache Benchmark (ab) are some of the popular kids in this playground. In addition to these open source tools, there are also commercial cloud-hosted tools like BlazeMeter, NeoLoad, and others. Some of these tools provide simple user interfaces to configure the performance tests and don't require coding. You can get test run reports with graphs, while commercial tools even offer a dashboard view. An exercise to create test scripts using JMeter and integrate them with CI is included in the following section.

> **TIP**
>
> Performance test runs may take anywhere from a few minutes to a few hours, depending on the test. To get an idea of how long yours will take, you may want to do a dry run of the scripts with a smaller user count before starting the full-fledged test run.

Those are the six steps in performance testing—we'll apply them as part of an exercise in the next section. The key to successfully executing all the steps in your project is to plan capacity for them adequately, as mentioned earlier. While planning, also include time and capacity to collect test run reports, debug and fix performance issues, and do server capacity tuning. That will complete the entire performance testing cycle!

Exercises

Now, we'll take the example of an online library management application and navigate through the performance testing steps. For convenience, we'll keep the features of the library management application simple. It has two types of users: the admins, who can add and delete books, and customers, who can view all the books

and search for a book by its ID. The respective REST APIs are `/addBook`, `/deleteBooks`, `/books`, and `/viewBookByID`.

Step 1: Define the Target KPIs

To arrive at the target KPIs for the library application, assume we got the following data from the business and the in-house marketing team:

- They are campaigning aggressively for launch in two European cities and expect 100,000 unique users to join in the first year.
- They have a study that says users spend 10 minutes on average searching for books, viewing similar books, etc., in a single session.
- The study also said that a typical user might borrow a book twice every month on average. Hence, they expect users to access the site twice a month.
- In Europe, the users are active on the internet between 10 a.m. and 10 p.m. (12 hours) daily.

With that data, we can calculate the following:

- Total users accessing the site monthly = 100,000 users * 2 accesses per month = 200,000 monthly users
- Average users per day = 200,000 monthly users ÷ 30 days per month = 6,667 daily users. (Note that there could be more users on weekends than weekdays, but we are calculating average daily users.)
- Average users per hour = 6,667 average daily users ÷ 12 hours per day = 555 hourly users. (Similarly, there could be more hourly users at some times of the day than others, such as at midday or in the evening.)

- To allow for peaks, we can be generous and round up to 1,000 hourly users.
- Each user uses the website for a session time of 10 minutes, which is 0.166667 hours.
- Number of concurrent users = 1,000 peak hourly users * 0.166 = 166 concurrent users.
- Assuming each user makes at least 5 requests (searching for books and viewing the book list) in a 10-minute session, the system will have to support 5 * 1,000 hourly users = 5,000 requests per hour.

Based on the calculations, these are our target KPIs:

- For 166 concurrent users, the system should respond within 3 seconds.
- System throughput has to support 5,000 requests per hour.

We should get consensus with the client management team on these numbers before we proceed. We can also probe the business to think beyond the first year and check again for target numbers.

> **NOTE**
>
> This is only a sample calculation to give an idea of how to work out target KPIs. As mentioned earlier, the first place to dig is the existing application's production data or a competitor's data, which will give a more accurate picture of KPIs and load patterns.

Step 2: Define the Test Cases

Now that we know the target KPIs, we can define appropriate performance test cases based on the library application's features.

Recalling the factors we discussed earlier, the test cases for our application could include:

- Benchmark the response times for all four endpoints: `/addBook`, `/deleteBooks`, `/viewBookById`, and `/books`.
- Volume test the customer-facing endpoints with 166–200 concurrent users—i.e., the `/viewBookById` and `/books` endpoints should respond in less than 3 seconds with 166 concurrent users. (Note that 3 seconds is inclusive of frontend performance, so you will have to assert with a lower boundary value specific to your application for the endpoints.) Only the admins access the other two endpoints; hence, volume testing may not be necessary for them.
- Stress test the customer-facing endpoints with ramp-up steps of 100 users and find the breaking points.
- Validate the throughput of 5,000 requests per hour. The user flow for this test case could be to view the book list, select a book and skim its description, then go back to the book list page, select another book and read its description, and return again to the book list page—in total, making five requests per user flow. Include a think time of, say, 30 seconds between each of these actions, and assume 45 users can continue doing this user flow for an hour. Ramp up the users slowly over the first 10 minutes.
- Soak test for a continuous 12 hours to validate that the system is available to users continuously. We could reuse the above throughput test design to run for 12 hours, too, if it is successful.

Steps 3–5: Prepare the Data, Environment, and Tools

For the sake of this exercise, I have developed a sample library application and hosted it on Heroku. To complete the exercise yourself, you can create a stub (refer to "WireMock" for details on this) on your local machine for the `/books` endpoint, as shown in Example 8-1, and configure it to return 50 books. Test it once after you set it up.

> **WARNING**
>
> Conducting high-volume load tests on public APIs can be considered a DDoS attack; hence the need to create stubs for the exercise. Alternatively, the various performance testing tools (such as JMeter and Gatling) provide test sites that you can use to practice with performance testing. Refer to their respective official sites to get the test site URLs and hit the test sites only with the minimum prescribed load.

Example 8-1. /books endpoint

```
GET: /books

Response:

Status Code: 200
Body:
[
{ "id": 1,
  "name": "Man's  search for meaning",
  "author": "Victor Frankl",
  "Language": "English",
  "isbn": "ABCD1234"
},
{ "id": 2,
  "name": "Thinking Fast and Slow",
  "author": "Daniel Kahneman",
  "Language": "English",
  "isbn": "UFGH1234"
}]
```

Step 6: Script the Test Cases and Run Them Using JMeter

JMeter is a popular performance testing tool. It is entirely open source and can integrate with CI and generate beautiful graph reports. It integrates with BlazeMeter, a cloud-hosted performance analytics tool, if you want to be free from infrastructure management tasks. JMeter is based on Java, and there is a community of active developers who contribute to different valuable plug-ins. The figures in "Types of Load Patterns" were created using one of these plug-ins. There is good documentation and tutorials on many use cases for beginners too. Let's install the tool and write some test scripts for our library application.

Setup

Follow these steps to set up JMeter:

1. Download the ZIP file from the official site and install it. Make sure your local Java version is compatible. Also ensure the `JAVA_HOME` variable is set in your environment's *bash_profile*.
2. To open the JMeter GUI, run the shell script *jmeter.sh* inside the folder */apache-JMeter-version/bin* from your terminal.
3. We will be using JMeter plug-ins as well. You can download the Plugins Manager from the official site and place the JAR under */apache-JMeter-version/lib/ext*.
4. Restart JMeter. You should then see Plugins Manager on the Options menu.

Workflow

Use the steps described here to set up a basic JMeter test skeleton and add a simple test to benchmark the response time of the `/books` endpoint:

1. Create a thread group from the JMeter GUI by right-clicking Test Plan in the lefthand pane and selecting Add → Threads (Users) → Thread Group. Name the thread group *ViewBooks*. Configure the parameters as shown in Figure 8-4 (*Number of Threads = 1, Ramp-up period = 0, Loop Count = 10*) to record the response times of the endpoint 10 different times and average them.

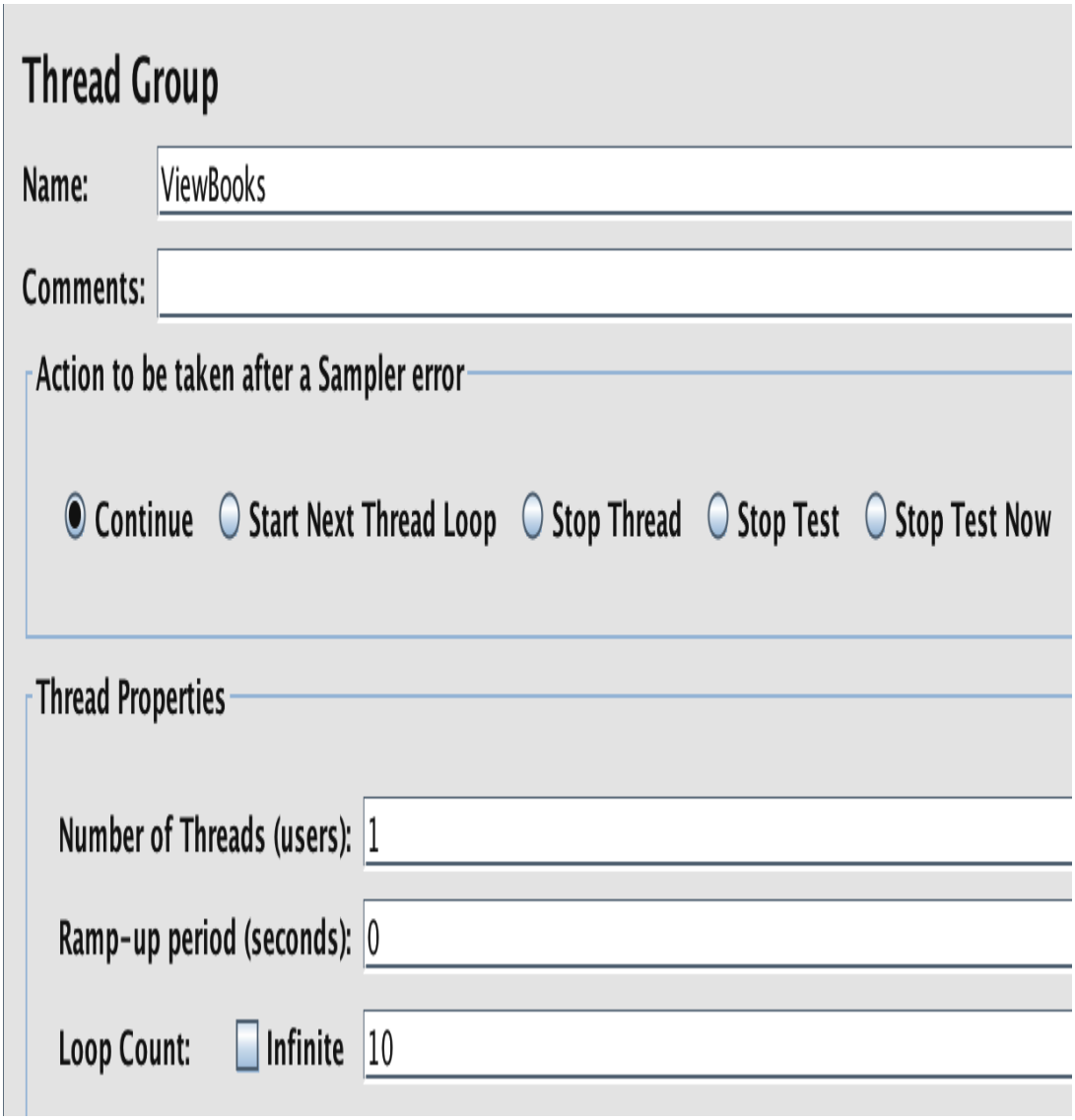


Figure 8-4. Thread group configuration to run one request 10 times

2. Add the HTTP Request sampler to configure the API's parameters. Right-click the thread group you just added in the

left pane and select Add → Sampler → HTTP Request. Enter the web server name, HTTP request type, and path (see Figure 8-5). Name the sampler *viewBooksRequest*.

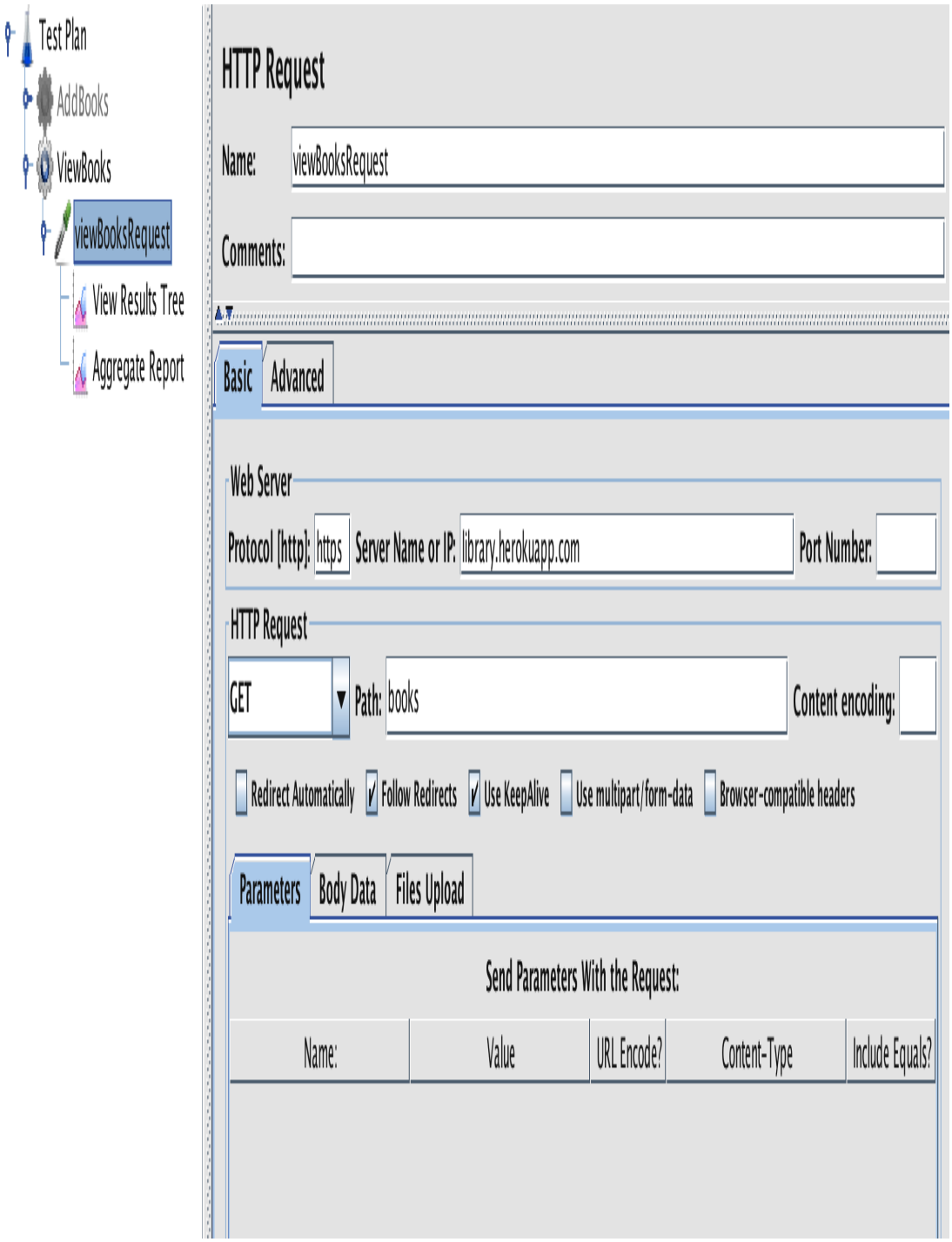


Figure 8-5. viewBooksRequest HTTP request configuration

3. Add listeners, which will record every request and response during the test run. Right-click the viewBooksRequest sampler and select Add → Listeners → View Results Tree, then repeat the process but this time select the Aggregate Report listener.
4. Save the basic test skeleton. Then, to measure the response time, click the Run button. The results will be available in the two listeners' sections.

Click View Results Tree in the lefthand panel to view the output from this listener. You will see the list of individual requests made by JMeter, with a success or failure indication for each. JMeter takes the response status code 200 to mean success; otherwise, it considers the request a failed request. One point to note is that there can be situations in your application where the service will return a 200 status code to indicate that the operation has been executed, but it may not have produced the intended results. For example, the `/addBook` endpoint could return a 200 status code for duplicate books with a message indicating it is a duplicate. In such cases, you need to add explicit assertions on the results (assertions, like listeners, are components of JMeter too). The View Results Tree view will also show request and response data on clicking each request for further debugging, as shown in Figure 8-6.

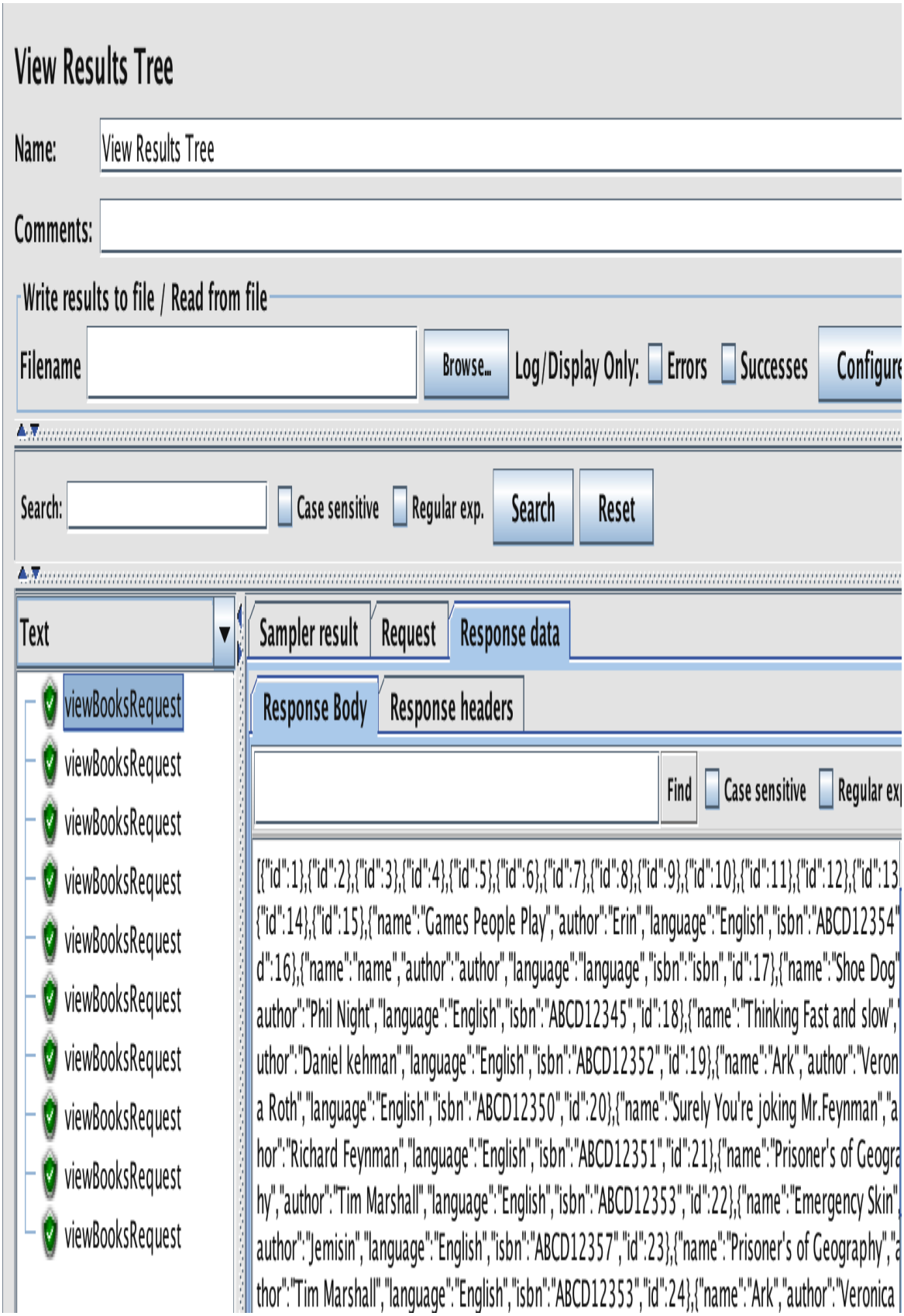
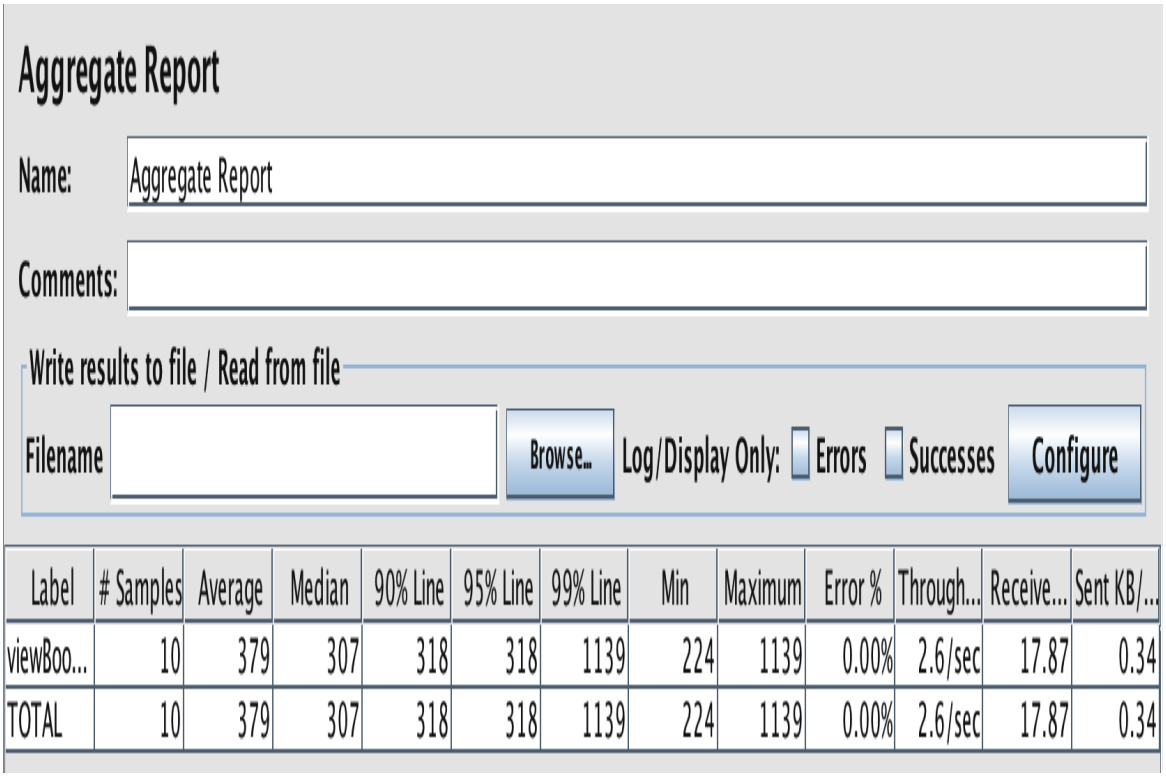

View Results Tree
Name:
View Results Tree
Comments:
Write results to file / Read from file
Filename
Browse...
Log/Display Only:
Errors
Successes
Configure
Search:
Case sensitive
Regular exp.
Search
Reset
Text
viewBooksRequest
viewBooksRequest
viewBooksRequest
viewBooksRequest
viewBooksRequest
viewBooksRequest
viewBooksRequest
viewBooksRequest
viewBooksRequest
viewBooksRequest
Sampler result
Request
Response data
Response Body
Response headers
Find
Case sensitive
[{"id":1},{"id":2},{"id":3},{"id":4},{"id":5},{"id":6},{"id":7},{"id":8},{"id":9},{"id":10},{"id":11},{"id":12},{"id":13
{"id":14},{"id":15},{"name":"Games People Play","author":"Erin","language":"English","isbn":"ABCD12354"
d":16},{"name":"name","author":"author","language":"language","isbn":"isbn","id":17},{"name":"Shoe Dog"
author":"Phil Night","language":"English","isbn":"ABCD12345","id":18},{"name":"Thinking Fast and slow",
uthor":"Daniel kehman","language":"English","isbn":"ABCD12352","id":19},{"name":"Ark","author":"Veron
a Roth","language":"English","isbn":"ABCD12350","id":20},{"name":"Surely You're joking Mr.Feynman","a
hor":"Richard Feynman","language":"English","isbn":"ABCD12351","id":21},{"name":"Prisoner's of Geogra
hy","author":"Tim Marshall","language":"English","isbn":"ABCD12353","id":22},{"name":"Emergency Skin",
author":"Jemisin","language":"English","isbn":"ABCD12357","id":23},{"name":"Prisoner's of Geography","a
thor":"Tim Marshall","language":"English","isbn":"ABCD12353","id":24},{"name":"Ark","author":"Veronica

Figure 8-6. View Results Tree listener output

Similarly, when you click Aggregate Report, you will see a table with metrics like average, median, throughput, etc. For the `/books` endpoint, the average response time for 10 samples is 379 ms (see Figure 8-7), which suggests that the best-case response time is 379 ms when the application is not under load.



Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
viewBoo...	10	379	307	318	318	1139	224	1139	0.00%	2.6/sec	17.87	0.34
TOTAL	10	379	307	318	318	1139	224	1139	0.00%	2.6/sec	17.87	0.34

Figure 8-7. Aggregate Report view for the response time of the `/books` endpoint

The next step is to perform load testing on the `/books` endpoint with 166 concurrent users and check the response time. JMeter offers many ways to configure different load patterns. Here we'll look at three simple options for configuring load on the `/books` endpoint.

As we saw earlier, the thread group is a basic JMeter element under which you can place different listeners and controllers. It can also be used to configure load parameters, such as the number of parallel threads, the length of the ramp-up period, and the number of times the test should repeat. Earlier, you configured the *ViewBooks* thread

group to run the `/books` request in a loop 10 times in order to benchmark its response time. Now, to conduct a volume test, you can change the parameters to `Number of Threads = 166, Ramp-up period = 0, Loop Count = 5`. JMeter will spin up 166 concurrent threads with no ramp-up time and loop 5 times to get the average response time.

There's also a handy plug-in that gives you access to additional types of thread groups that are useful for configuring different load patterns, such as the step ramp-up pattern. Here, I'll show you how to use the Concurrency Thread Group and Ultimate Thread Group. We'll start with the Concurrency Thread Group, which gives you a concurrency controller for volume testing:

1. Select Options → Plugins Manager. Search for "Custom Thread Groups" on the Available Plugins tab and install it.
2. Restart JMeter so the new thread group types are available.
3. Right-click Test Plan in the lefthand panel and select Add → Threads (Users) → bzm → Concurrency Thread Group.
4. Configure the load parameters as shown in Figure 8-8 (`Target Concurrency = 166, Ramp Up Time = 0.5, Hold Target Rate Time = 2`). This tells JMeter to ramp up 166 users in 30 seconds and hold each of them for 2 minutes in the system.
5. Add the HTTP Request sampler like before under this thread group, run the test, and view the results in the listeners.



Figure 8-8. Concurrency Thread Group to volume-test the `/books` endpoint

The Custom Thread Groups plug-in also provides an Ultimate Thread Group type with additional features. For example, it allows you to tailor your load pattern by configuring the initial delay before the test run, the shutdown time after test run, and more. To use an Ultimate Thread Group for volume testing:

1. Right-click Test Plan and select Add → Threads (Users) → jp@gc Ultimate Thread Group.

2. Configure the load parameters as seen in Figure 8-9 (`Start Threads Count = 166, Initial Delay = 0, Startup Time = 10, Hold Load For = 60, Shutdown Time = 10`). This instructs JMeter to spin up 166 concurrent requests within 10 seconds and hold the load for 1 minute, after which it will ramp down the users within 10 seconds. You can add more rows as appropriate to generate the peak-rest pattern here, too.
3. Add the HTTP Request sampler like before, run the test, and view the results.

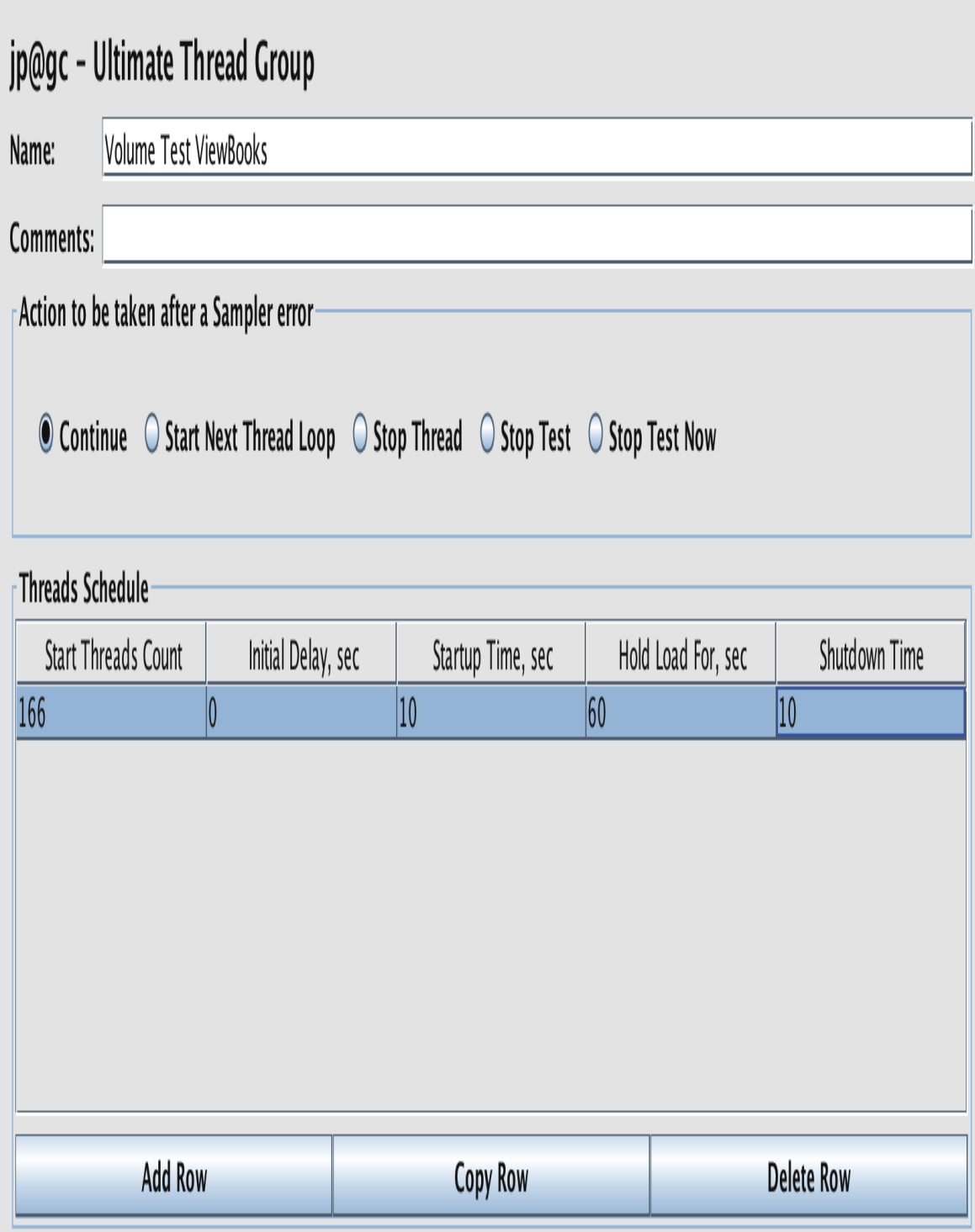
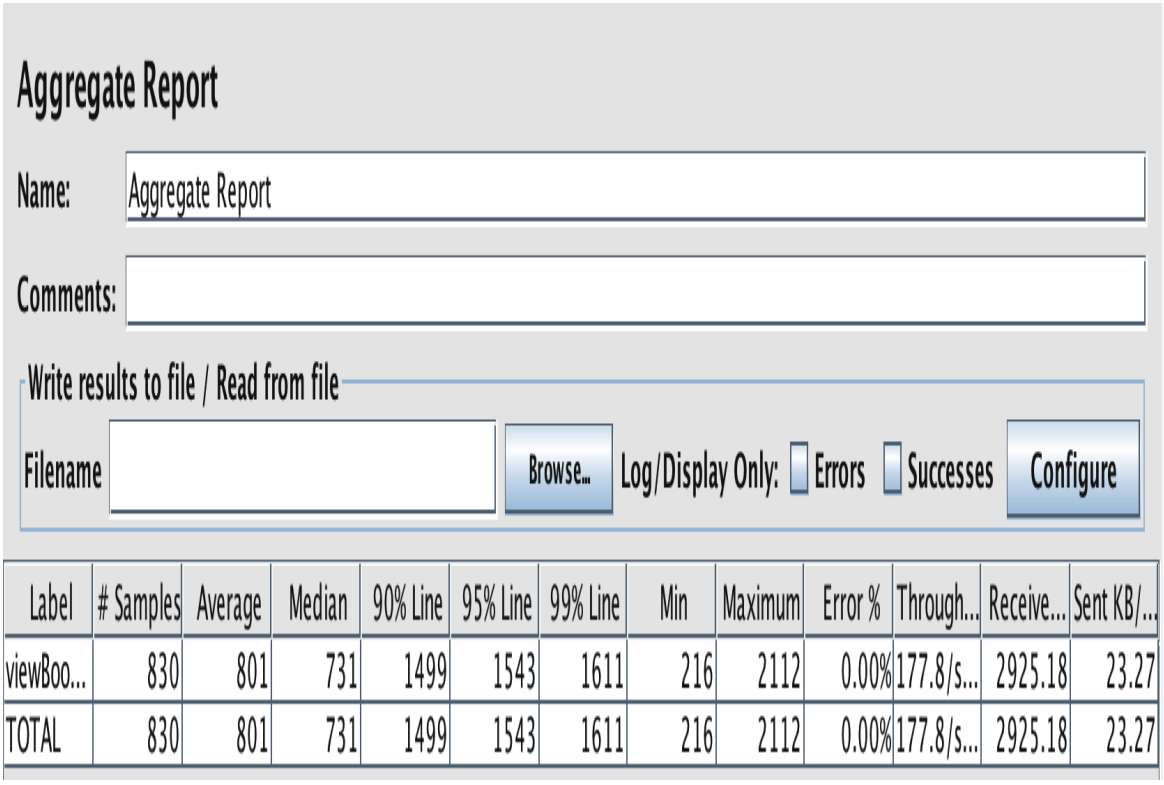


Figure 8-9. Ultimate Thread Group to volume test the `/books` endpoint

Figure 8-10 shows the results using the simple thread group option (the first option) with 166 concurrent users and 0 ramp-up time, averaged over 5 loop counts: `Average = 801 ms, 90% Line =`

`1499 ms`. In other words, 90% of 166 concurrent users get their response back in ~1.5 s, and on average, all 166 concurrent users get their response within 0.8 s. The average is lower because, as we can see from the table, the minimum time for some users to get a response was just 216 ms.



Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
viewBoo...	830	801	731	1499	1543	1611	216	2112	0.00%	177.8/s...	2925.18	23.27
TOTAL	830	801	731	1499	1543	1611	216	2112	0.00%	177.8/s...	2925.18	23.27

Figure 8-10. Volume test results for the `/books` endpoint

Designing other performance test cases

In the previous section, you explored different ways to distribute load using JMeter. This experience of using the tool for volume testing is a great start and should allow you to simulate other performance test cases like stress tests, soak tests, throughput validation, and so on. To do stress testing, you can use a Concurrency Thread Group to introduce load in steps of *x* users up to a maximum limit, running each step for a given time. The objective here is to find the load at which the response time slows down and ultimately results in errors.

To do soak testing, you can simulate constant load for an extended time using an Ultimate Thread Group. To validate hourly throughput, use the Parallel Controller plug-in to run multiple HTTP requests in parallel while pausing between requests using Timer components, e.g., for setting the think time. There is also a Constant Throughput timer, which can be used to fix the throughput at a constant value and validate whether the application performs as expected; it automatically slows down the number of requests made to the server by JMeter if it crosses the set throughput value.

There are many more components in JMeter to help model application-specific use cases. The If, Loop, and Random controllers enable you to include conditions in tests. There are also provisions to feed in user credentials, if the application requires a login, from an external source like a CSV file to perform volume testing. This is called *data-driven performance testing*. You can also use this feature of JMeter to set up test data at the beginning of the test. We'll look at an example next.

Data-driven performance testing

Let's say the `/addBook` endpoint in the library application takes a request body with the book's name, author, language, and ISBN. To create load on this endpoint, you need to add unique books with every request. You can utilize the data-driven performance testing capabilities of JMeter to do this, as follows:

1. Create a CSV file with `name`, `author`, `language`, and `isbn` as keys. JMeter refers to these keys while defining variable inputs. Add rows for 50 books. (You can do this in Google Sheets and download it as a CSV file.)
2. In JMeter, add a thread group with an HTTP Request sampler for the `/addBook` endpoint and set the Loop Count to 50.
3. To wire the CSV file to the HTTP Request sampler, right-click Thread Group and select Add → Config Element → CSV Data

Set Config. In the CSV Data Set Config window (see Figure 8-11), specify the CSV file path and the variables to read from the file.

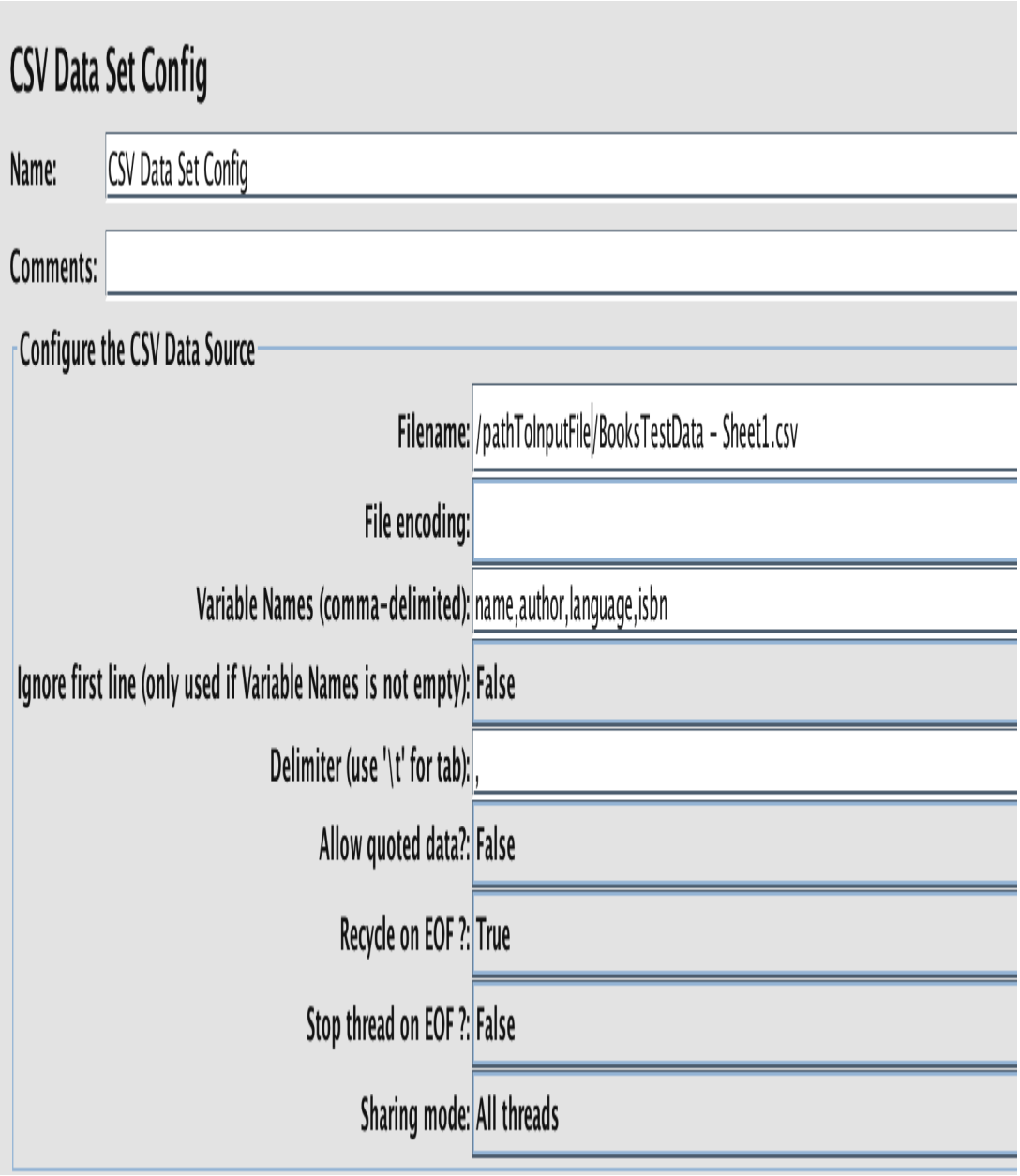


Figure 8-11. Configuring CSV dataset input for data-driven testing

4. In the HTTP request body of the `/addBook` endpoint, use the variables as `${variable_name}` as seen in Figure 8-12. These variables can be referred to using the same

`${variable_name}` notation wherever needed across JMeter tests.

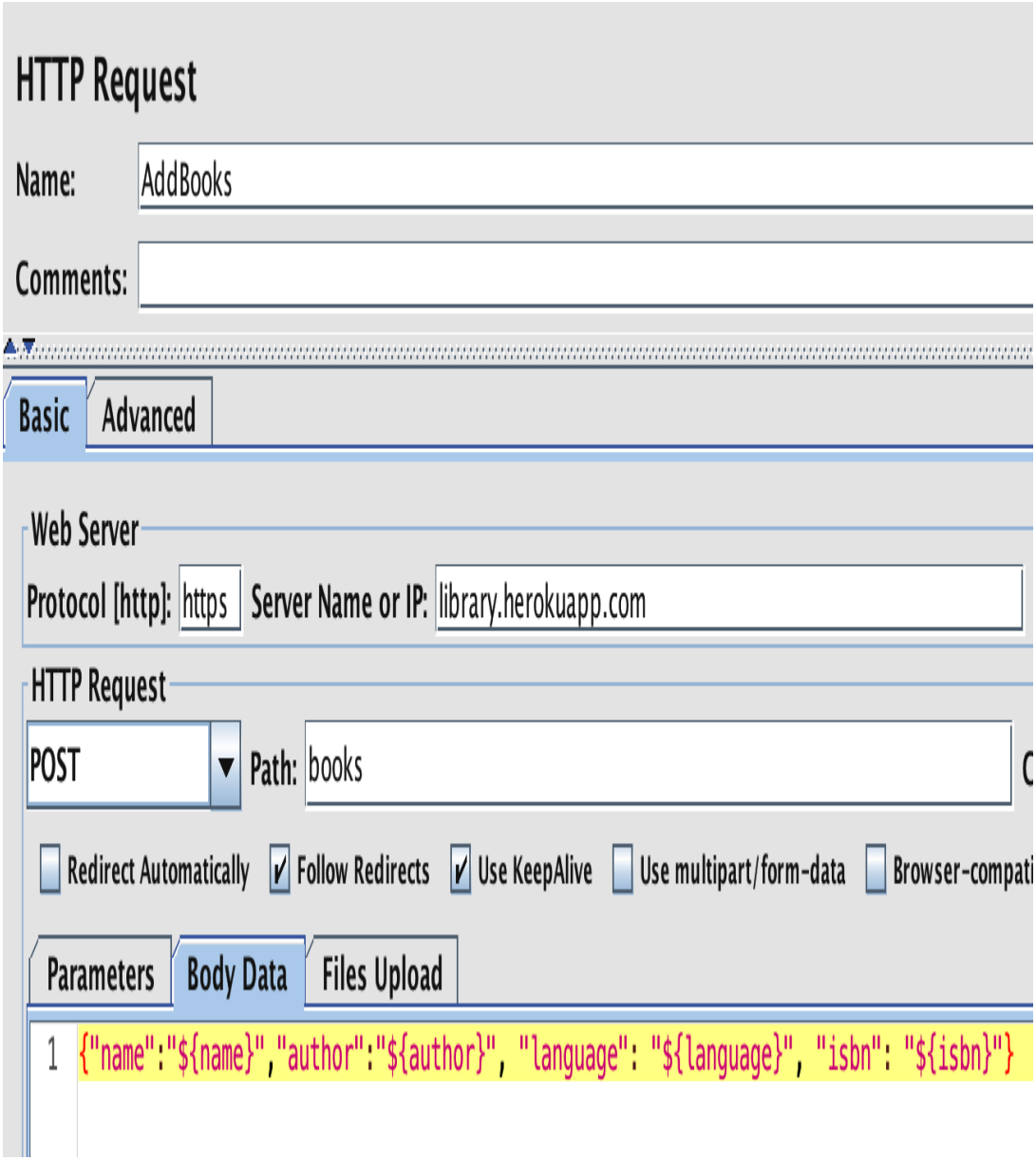


Figure 8-12. Referencing variables from the CSV file

This test can be run to create the test data before starting the performance tests.

Integrating into CI

The final step is to integrate the JMeter tests into your CI pipeline as a separate job and shift performance testing to the left. It's important to ensure that the performance tests are run in complete isolation to get the right metrics. To integrate the tests with CI, save them, locate the saved *.jmx* files, and run the following command:

```
$ jmeter -n -t <library.jmx> -l <log file> -e -o <Path to
output folder>
```

You can also configure JMeter to provide exhaustive dashboard reports as needed with further extensions.

As you can see, JMeter makes performance testing easier with a simple GUI to configure and run performance test cases.

Additional Testing Tools

There are several other performance testing tools that can help you script your performance test cases. These tools essentially provide varied handles to configure the four key parameters to design load patterns (ramp-up time, think time, number of concurrent users, and pacing). For instance, as we saw, JMeter offers a GUI, while Gatling provides a domain-specific language and Apache Benchmark (ab) uses simple command-line arguments. Let's briefly get to know Gatling and ab as well.

Gatling

Gatling provides a Scala-based DSL to configure the load pattern. It's an open source tool with the option to record user flows. The tests can be integrated with CI pipelines. If you're game to explore Scala, this is a robust tool for simulating nuanced load patterns. You can see a sample Scala script demonstrating how to induce load

with think time on our library management application's `/books` API in Example 8-2.

Example 8-2. Sample Scala script for load testing

```
package perfTest

import scala.concurrent.duration._

import io.gatling.core.Predef._
import io.gatling.http.Predef._

class BasicSimulation extends Simulation {

// Defining the HTTP request
  val httpProtocol = http
    .baseUrl("https://library.herokuapp.com/")

.acceptHeader("text/html,application/xhtml+xml,application/xm
l;q=0.9,*/*;q=0.8")
    .doNotTrackHeader("1")
    .acceptLanguageHeader("en-US,en;q=0.5")
    .acceptEncodingHeader("gzip, deflate")
    .userAgentHeader("Mozilla/5.0 (Windows NT 5.1; rv:31.0)
Gecko/20100101
        Firefox/31.0")

// Defining a single user flow with think time
  val scn = scenario("BasicSimulation")
    .exec(http("request_1")
    .get("/books"))
    .pause(5) // Think time

// Configuring load of 166 concurrent users to do the above
user flow
  setUp(
    scn.inject(atOnceUsers(166))
  ).protocols(httpProtocol)
}
```

Apache Benchmark

If you just want to quickly get some numbers on your application's performance, ab is a great choice. It's a simple open source command-line tool. If you're on a Mac, ab comes as part of the OS, so you don't even have to worry about installation. To get performance numbers for load testing the `/books` endpoint with 200 concurrent users, you can run the following command from your terminal:

```
$ ab -n 200 -c 200 https://library.herokuapp.com/books
```

The results will be as follows:

```
Concurrency Level:      200
Time taken for tests:   5.218 seconds
Complete requests:      200
Failed requests:        0
Total transferred:      1389400 bytes
HTML transferred:       1340800 bytes
Requests per second:    38.33 [#/sec] (mean)
Time per request:       5217.609 [ms] (mean)
Time per request:       26.088 [ms] (mean, across all
concurrent requests)
Transfer rate:          260.05 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:      869 2074  97.6   2064    2289
Processing:   249 1324 299.4   1303    1783
Waiting:      249 1324 299.5   1303    1781
Total:       1192 3398 354.3   3370    4027

Percentage of the requests served within a certain time
(ms)
  50%   3370
  66%   3483
  75%   3711
  80%   3776
  90%   3863
  95%   3889
```

```
 98%   4016
 99%   4022
100%   4027 (longest request)
```

You now have an idea of how different tools can assist in scripting performance test cases and measuring the server KPIs. Remember, though, that this is not the end of the performance testing cycle. If you find performance issues, you will have to debug, tune, and test again!

We have dealt with backend performance testing at some depth here, but we're not done yet. Next, we'll turn our focus to frontend performance testing!

Frontend Performance Testing Building Blocks

Though performance testing tools allow you to mimic application behavior during peak times, there is a gap between the measured performance numbers and the actual user-experienced performance. This is because the tools are not actual browsers, and they don't do all the tasks a typical browser does!

To understand this gap, let's explore a bit about browser behavior. As we saw in Chapter 6, there are three parts to the frontend code that gets rendered in the browser:

- HTML code, which is the barebones structure of the website
- CSS code, which styles the page
- Scripts to create logic on the page

A typical browser first downloads all the HTML code from the server, then downloads the stylesheet, images, etc. and begins executing the scripts, as per the sequence in the HTML. There is parallelization to an extent, such as when it is downloading images from different hosts. But the browser stops parallel processing completely when executing a script, as it is possible for the script to completely

change the way the page is made visible. Since there could be scripts at the end of the HTML, the page becomes visible for the user only when the entire document has been fully executed.

Performance testing tools don't do most of these jobs. They hit the page directly and get the HTML code, but they don't render the page while executing the performance tests. So even if you have measured the services' response time to be within milliseconds, the end user will see the page appear only after a further delay because of the additional rendering tasks that the browser does. This frontend rendering is estimated to account for 80–90% of the entire page load time—shocking, isn't it?

For example, if you navigate to the CNN home page, the browser will carry out 90 tasks before the page appears to you. Figure 8-13 shows the first 33 of these tasks. If you had been thinking optimizing the web service's response time alone would have a significant impact on website performance, here is a piece of evidence that may change that view!

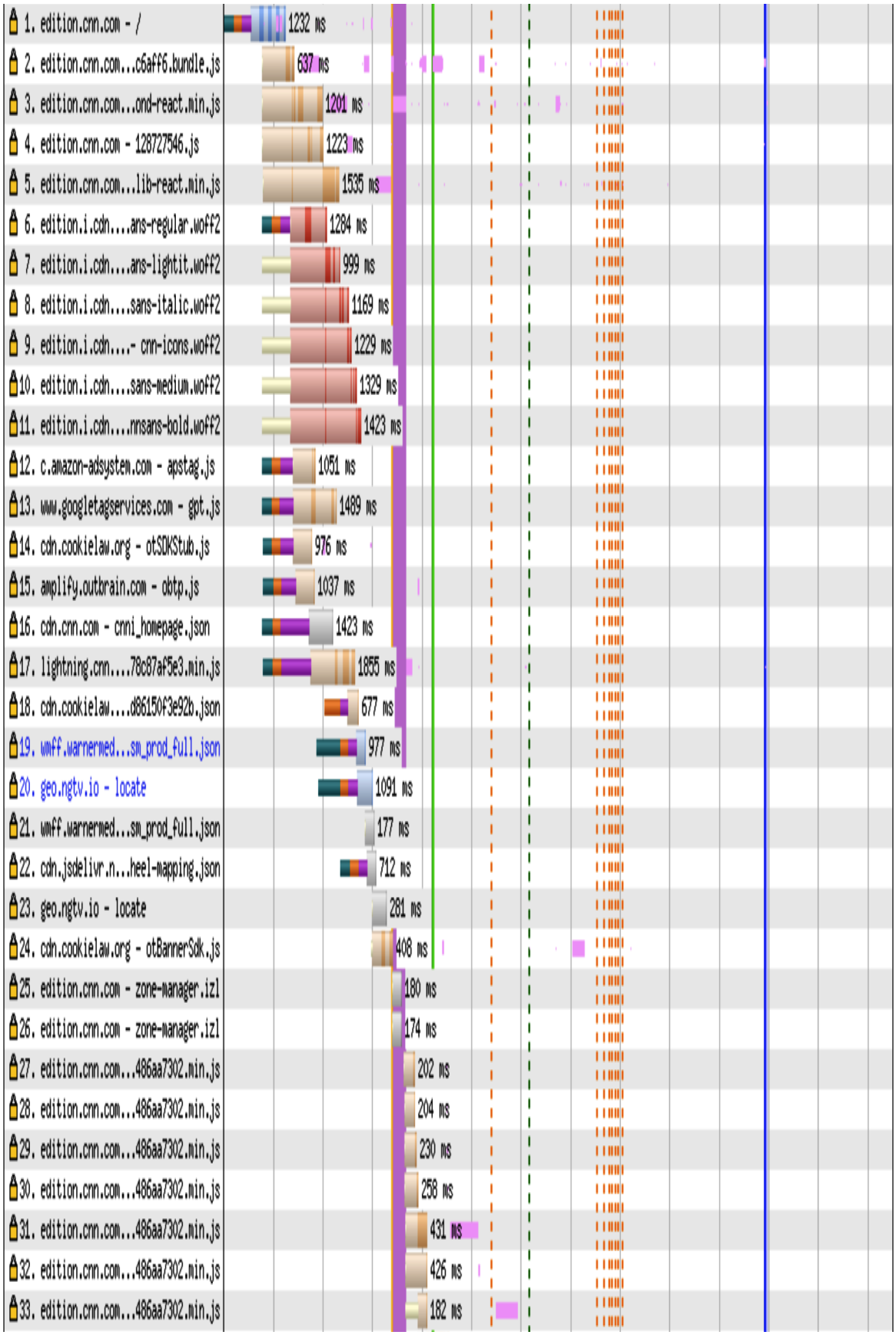

1. edition.cnn.com - / 1232 ms
2. edition.cnn.com...c6aff6.bundle.js 687 ms
3. edition.cnn.com...ond-react.min.js 1201 ms
4. edition.cnn.com - 128727546.js 1223 ms
5. edition.cnn.com...lib-react.min.js 1535 ms
6. edition.i.cdn....ans-regular.woff2 1284 ms
7. edition.i.cdn....ans-lightit.woff2 999 ms
8. edition.i.cdn....sans-italic.woff2 1169 ms
9. edition.i.cdn....- cnn-icons.woff2 1229 ms
10. edition.i.cdn....sans-medium.woff2 1329 ms
11. edition.i.cdn....nnsans-bold.woff2 1423 ms
12. c.amazon-adsystem.com - apstag.js 1051 ms
13. www.googletagservices.com - gpt.js 1489 ms
14. cdn.cookielaw.org - otSDKStub.js 976 ms
15. amplify.outbrain.com - obtp.js 1037 ms
16. cdn.cnn.com - cnni_homepage.json 1423 ms
17. lightning.cnn....78c87af5e3.min.js 1855 ms
18. cdn.cookielaw....d86150f3e92b.json 677 ms
19. wmff.warnermed...sm_prod_full.json 977 ms
20. geo.ngtv.io - locate 1091 ms
21. wmff.warnermed...sm_prod_full.json 177 ms
22. cdn.jsdelivr.n...heel-mapping.json 712 ms
23. geo.ngtv.io - locate 281 ms
24. cdn.cookielaw.org - otBannerSdk.js 408 ms
25. edition.cnn.com - zone-manager.izl 180 ms
26. edition.cnn.com - zone-manager.izl 174 ms
27. edition.cnn.com...486aa7302.min.js 202 ms
28. edition.cnn.com...486aa7302.min.js 204 ms
29. edition.cnn.com...486aa7302.min.js 230 ms
30. edition.cnn.com...486aa7302.min.js 258 ms
31. edition.cnn.com...486aa7302.min.js 431 ms
32. edition.cnn.com...486aa7302.min.js 426 ms
33. edition.cnn.com...486aa7302.min.js 182 ms

Figure 8-13. CNN frontend browser tasks during page load

However, the KPIs described and measured as part of the exercises performed earlier are still relevant and critical. They are vital for planning the system's capacity and troubleshooting performance issues. In other words, they help you to answer questions like "Will the application support a peak load of 5,000 transactions during the Black Friday sales?" But if the KPIs indicate that the peak response time for your application is ~1.5 seconds, that may not reflect the actual experience of an end user. To understand that, you must also evaluate the frontend performance metrics. That is what we will discuss in this section.

To begin with, let's understand the factors specifically affecting frontend performance and the metrics that need to be measured to quantify it. Later, you'll get hands-on with actually measuring them.

Factors Affecting Frontend Performance

There are several factors that contribute to frontend performance:

Frontend code complexity

Failing to implement best practices such as minifying the JavaScript, reducing the number of HTTP requests made per page, and implementing proper caching techniques will lead to lower performance. For instance, it takes at least a few milliseconds for the server to respond to each HTTP request, and if the page has to make many such requests, the delays will accumulate.

Content delivery networks (CDNs)

A CDN is a collection of servers hosted in multiple locations that aims to deliver web content, such as images, to users more efficiently. As we discussed earlier, the geolocation of the server and the user have an impact on application performance, due to

network latency. To reduce the network latency, the content is stored in CDNs and served from the server that is physically closest to the user. This is much simpler than replicating the application in different geolocations; however, the performance of the CDN itself will affect the page load time.

DNS lookups

It typically takes 20–120 ms for a browser to look up the IP address for a given hostname. This is known as Domain Name Service (DNS) resolution. Once this has been done the first time, the browser and the OS cache the IP address, reducing the page load time for subsequent visits. Internet service providers (ISPs) also cache the IP addresses for a while, contributing to performance improvements. However, the first-time user experience is affected by the DNS lookup time.

Network latency

The user's network bandwidth has a huge impact on the overall page load time. As we saw in Chapter 6, global usage data indicates that mobile usage trumps desktop usage today, and mobile network bandwidth can be very low at times, in both urban and rural areas. Some sites overcome this by serving a "lite" version of their website when they identify the bandwidth to be low. However, users who generally operate with low bandwidth (like 3G) tend to be used to the slowness and not to complain unless the performance is jarringly bad.

Browser caching

As well as IP addresses, the browser caches a lot of other content (images, cookies, etc.) after the first visit. Consequently, the page load time often significantly varies between the first time it's rendered and subsequent viewings. Browser caching can be made intentional via code to improve page load times.

Data transfers

> If large volumes of data get transferred to and fro between the user and the application, obviously this will affect the overall frontend performance because of network latency.

Looking at all these factors, you might think it's beyond the team's control to even think about optimizing them, leaving you wondering where to start. Many folks in the software industry have also felt the pain of dealing with this challenge. That's where the RAIL model comes in.

RAIL Model

The RAIL model is a way to structure the thought process around frontend performance. It is designed with the guiding principle of keeping the end user's experience at the core of frontend performance, and it quantifies goals for frontend performance. It can be helpful to view frontend performance through this lens and to integrate the goals as part of your testing efforts.

The RAIL model breaks down a user's experience on a website into four key areas:

Response

> Have you ever had the experience of clicking a button but not seeing any immediate visual indication that you've done so, making you wonder if you imagined clicking it in the first place? This delay is known as the *input latency*. The "response" aspect of RAIL defines the goals for input latency. When a user performs an action on a website such as clicking a button, toggling an element, selecting a checkbox, etc., RAIL prescribes that the response time for that action should be less than 100 ms; failing that, the user will sense the lag!

Animation

> Similarly, the user will perceive a lag in animation effects (e.g., loading indicators, scrolling, drag and drop, etc.) when each frame is not completed within 16 ms (the minimum to achieve a frame rate of 60 FPS).

Idle

> A general frontend design pattern is to group noncritical tasks like beaconing back analytics data, bootstrapping a comments box, and so on and perform them later, when the browser is idle. These tasks should ideally be bundled into blocks that take about 50 ms to complete, so that when the user comes back to interact, you can respond within the 100 ms window.

Load

> A high-performing website should aim to begin rendering the page within 1 second as only then will users feel they are in complete control of the navigation (as per the research mentioned earlier).

As you can see, the RAIL model guides us to think about what to test for from a frontend performance perspective. It also provides a concrete language for communication within teams, instead of expressing vague feelings like “the page seems slow”!

Frontend Performance Metrics

In practice, the high-level goals set by the RAIL model are broken down into smaller metrics in order to fine-tune debugging of performance issues. A set of standard frontend performance metrics adopted in the industry are as follows:

First contentful paint

This refers to the time taken by the browser to render the first element from the DOM (images, non-white elements, SVGs, etc.). This helps us understand how long the user has to wait to see some action on the website after opening it.

Time to interactive

This is the time taken for the page to become interactive. In the rush to make the page performant, the elements could be made visible quickly but could fail to respond to the user's actions, leading to frustration. Hence, in parallel to measuring the time taken to see the first content on the website, this metric helps us understand whether the information presented is helpful or just noise.

Largest contentful paint

This is the time it takes for the most prominent element on the web page, like a big blob of text or image, to become visible.

Cumulative layout shift

Have you ever come across sites where you started reading an article and then the page automatically shifted down as additional content loaded, making you lose track of what you were reading? It's frustrating, isn't it? This metric aims to measure the visual stability of the page and quantifies how often the user faces an unexpected change in page layout. The lower the number, the better the performance.

First input delay

Between the first contentful paint and time to interactive, when the user clicks on a link or performs any interaction with the web page, there will be a delay which is longer than the usual delay because the page is still loading. This metric gives that time delay for the first interaction.

Max potential first input delay

> This represents the worst-case scenario of the first input delay. It measures the time taken by the most prolonged task that occurs between the first contentful paint and the time to interactive to complete.

Google classifies the largest contentful paint, first input delay, and cumulative layout shift as the *core web vitals* to help the business folks understand a site's performance in simple terms. Most frontend performance testing tools capture these three metrics specifically. We can use such tools to continuously measure these metrics as part of CI and hence shift frontend performance testing to the left. We'll discuss how to do that next.

Exercises

As elicited by the RAIL model, frontend performance is all about the end user's experience. So, in order to measure frontend performance metrics for your application, you need to first define a set of test cases that will encompass all your target end users' experiences across different demographics. For example:

- Consider users with different types of devices (desktop, mobile, tablet). Also gather information on device manufacturers that are significant players in the region your application wishes to serve. This is important because each device will have its own CPU, battery, and memory capacity, which affects the end user experience.
- Consider users with varying network bandwidths: WiFi, 3G, 4G, etc. Also, be aware that average mobile and broadband speeds are different in different countries. According to World Population Review data, for example, as of 2021 Monaco had the fastest average broadband speed at 261.8 Mbps, compared

to 203.8 Mbps in the US, 102.2 Mbps in the UK, and 13.8 Mbps in Pakistan.

- Consider your target users' distribution. As the previous point suggests (though there are other factors that contribute as well), the frontend performance experienced in different geolocations will have to be tested specifically.

Plenty of research on this kind of usage data is available on the internet. Alternatively, if you have an existing live application, Google Analytics will give you the site's real-time usage information. Once you have the test cases, you can use the tools described here to measure the frontend performance metrics and also add those tests to your CI pipeline.

Let's define a sample test case to do the hands-on exercises here: "A user from Milan, who has a Samsung Galaxy S5, is accessing the Amazon home page using a 4G network connection." Now, let's see how tools like WebPageTest and Lighthouse can help with measuring frontend performance.

WebPageTest

WebPageTest is a free online tool for assessing a website's frontend performance. It's a powerful tool, as it includes provisions to choose the geolocation from which the website is being accessed and gathers the frontend performance metrics by rendering the website on real web and mobile browsers. A tool can't get much closer than this to replicating a real end user's behavior!

Workflow

Using the tool is simple, as the steps here show:

1. Enter the Amazon URL in the input field, as seen in Figure 8-14.

2. Choose the end user's location, browser type, mobile device type, and network bandwidth, as per the example test case. Refer to Figure 8-14.
3. Set the Number of Tests to Run parameter to 3. The results from just one round of evaluation may be faulty due to glitches in network bandwidth, so it's a good idea to run the test case a few times to observe the average.
4. Set the Repeat View parameter to "First View and Repeat View." This will capture the performance metrics separately for the first visit and subsequent visits. As you may recall, the metrics could vary for the subsequent visits due to caching.
5. Run the test, and view the reports with metrics.

https://www.amazon.com/

Start Test →

Test Location: Milan, Italy - EC2 (Chrome,Firefox) ∨ | Select from Map

Browser: Samsung Galaxy S5 ∨

Advanced Settings ∨

Test Settings | Advanced | Chromium | Script | Block | SPOF | Custom

Connection: 4G (9 Mbps, 170ms RTT) ∨

Desktop Browser Dimensions: default (1366x768) ∨

Number of Tests to Run
Up to 9: 3

Repeat View: (•) First View and Repeat View ( ) First View Only

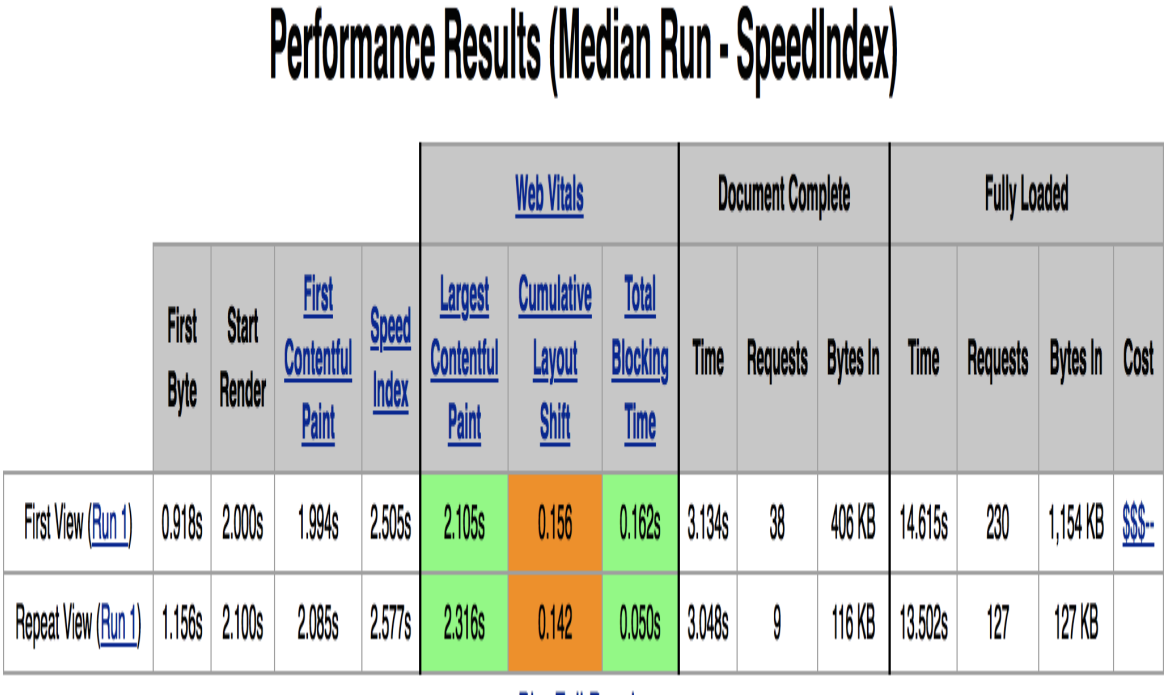
Capture Video: [✓]

Figure 8-14. WebPageTest configuration

Since WebPageTest is a free and publicly available tool, you may have to wait in the queue for a few minutes to view the report. To avoid the wait, you can choose to set it up privately in a local test environment for a fee.

The report has many valuable sections that enable detailed debugging. Each report can be retrieved using a unique ID for 30 days. Let's discuss a couple of important sections from the report generated by WebPageTest for our example test case.

The performance metrics table (see Figure 8-15) has the core web vitals for the first view and repeat view for all three test runs. To benchmark the page load time for this test case, you can take the median of the document complete time from all the runs. Notice that the first-time view's document complete time is 3.134 s and the largest contentful paint is 2.105 s, which tells us that the user experience is within acceptable limits. The fully loaded time in the table includes the time taken to load all the secondary content, i.e., tasks deferred by the load event. Although it is substantial (~14 s with 230 requests), it's unlikely to affect the end user's experience.



Performance Results (Median Run - SpeedIndex)

					Web Vitals			Document Complete			Fully Loaded			
	First Byte	Start Render	First Contentful Paint	Speed Index	Largest Contentful Paint	Cumulative Layout Shift	Total Blocking Time	Time	Requests	Bytes In	Time	Requests	Bytes In	Cost
First View (Run 1)	0.918s	2.000s	1.994s	2.505s	2.105s	0.156	0.162s	3.134s	38	406 KB	14.615s	230	1,154 KB	$$$--
Repeat View (Run 1)	1.156s	2.100s	2.085s	2.577s	2.316s	0.142	0.050s	3.048s	9	116 KB	13.502s	127	127 KB	

Figure 8-15. Performance metrics table from WebPageTest report

The waterfall view, shown in Figure 8-16, shows a colorful timeline view of how long each task—like DNS resolution, connection initiation, downloading HTML and images, runtime for scripts, etc.—takes, giving us a clue about avenues for further optimization.

Waterfall View



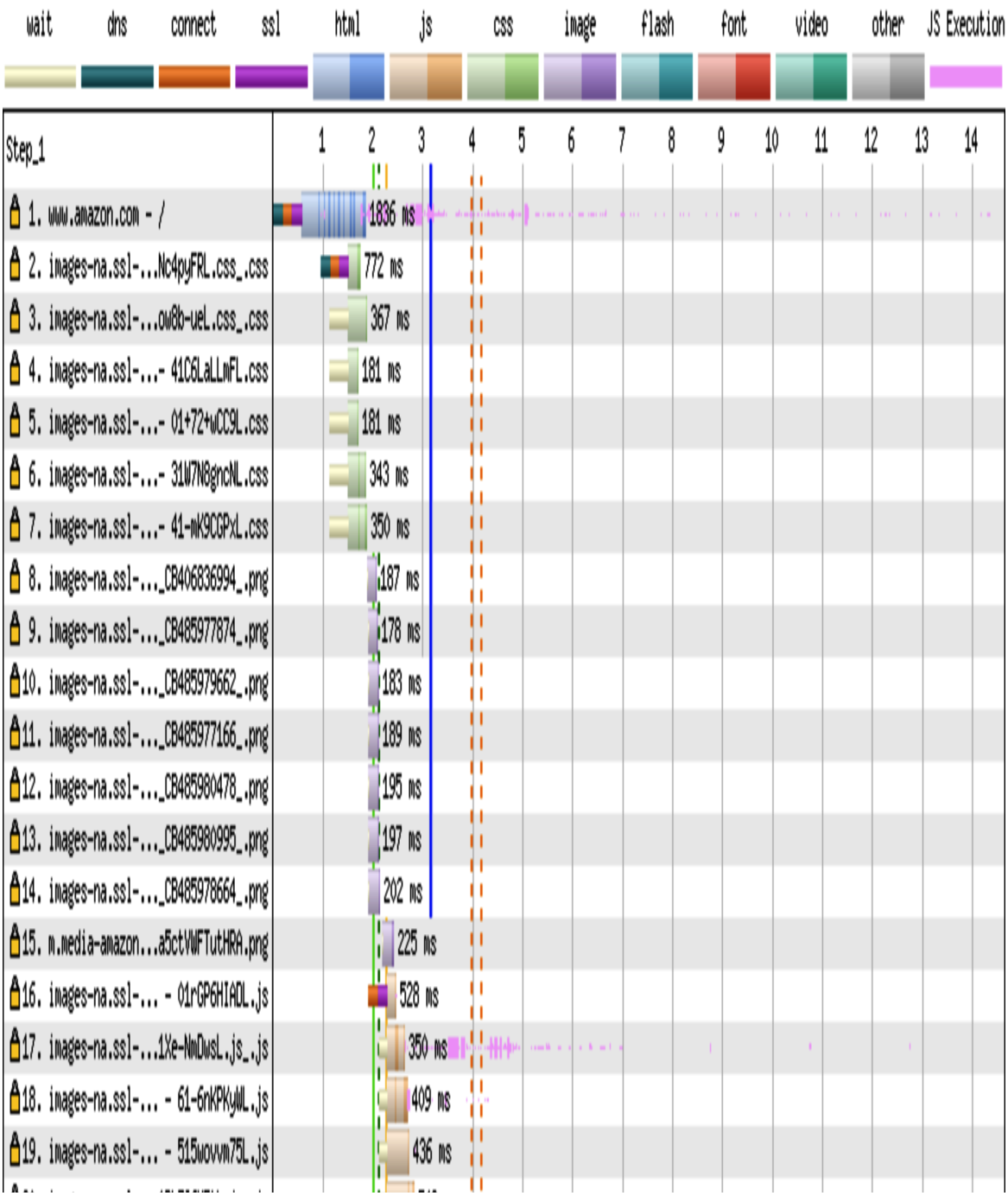


Figure 8-16. Waterfall view from WebPageTest report

WebPageTest has facilities to pass authentication details too, but note that the test credentials you provide will be visible to whomever has access to the test run report as it is publicly hosted.

WebPageTest also exposes APIs to get these reports programmatically, and there is a Node.js module variant to run the tests directly from the command line as well. These two options enable integration with CI pipelines. Both of them require an API key which is available for a fee. If you decide to purchase one, refer to Examples 8-3 and 8-4 for details on the CLI commands and API usage, respectively.

Example 8-3. WebPageTest CLI commands to install, run test cases, and view results

```
// Step 1: Install using npm

npm install webpagetest -g

// Step 2: Run a sample test case via the command line

webpagetest test http://www.example.com --key API_KEY --
location
ec2-eu-south-1:Chrome --connectivity 4G --device Samsung
Galaxy S5 --runs 3 --first --video --label
"Using WebPageTest" --timeline

// Step 3: Read test results from the report ID generated by
the above command

webpagetest results 2345678
```

Example 8-4. APIs to run WebPageTest test cases and view results

```
// Step 1: Run a sample test case using WebPageTest's API

http://www.webpagetest.org/runtest.php?
url=http%3A%2F%2Fwww.example.com&k=API_KEY&location=ec2-eu-
south-
```

```
1%3AChrome&connectivity=4G&runs=3&fvonly=1&video=1&label=Usin
g%20WebPagetest&timeline=1&f=json

// Step 2: Read test results from the report ID returned as
the response
// by the above API

http://www.webpagetest.org/jsonResult.php?test=2345678
```

Lighthouse

Lighthouse comes as part of Google Chrome, and it is also available as a Firefox extension. It audits your website along multiple dimensions, including security, accessibility, and frontend performance. The performance audit report generated by Lighthouse includes an overall score and all the detailed frontend performance metrics.

One of the advantages of Lighthouse is that it is not publicly hosted, and hence there's no queuing or wait time. Since it runs in your local browser there are no security concerns either, although that also means you can't configure the geolocation of the end user (you will be accessing the website from your actual location). You can still throttle your network and CPU and resize to different mobile browser resolutions in Chrome to simulate different test cases and obtain respective metrics, however.

Lighthouse is also available as a CLI tool, making it easier to integrate with CI and get continuous feedback. Zalando, a leading European retail chain, has stated that it reduced its frontend performance feedback time from 1 day to 15 minutes with Lighthouse CI. The tool is entirely free and open source.

Workflow

You can follow these simple steps to explore Lighthouse:

1. Open the Amazon website in Chrome.

2. Open Chrome DevTools using the shortcut Cmd-Option-J on macOS or Shift-Ctrl-J on Windows/Linux, or choose the Inspect option from Chrome's right-click menu.
3. Choose the network throttling preferences on the Network tab. Select "Slow 3G" for the example test case.
4. Choose the CPU throttling preferences on the Performance tab. The default options are 4x slowdown and 6x slowdown for middle- and low-tier mobile devices. Choose 4x slowdown for the example test case.
5. Choose the window size from the responsive drop-down. The Galaxy S5 option for the example test case is available too, as seen in Figure 8-17.
6. On the Lighthouse tab (Figure 8-17), select the Performance category and then click "Generate report."

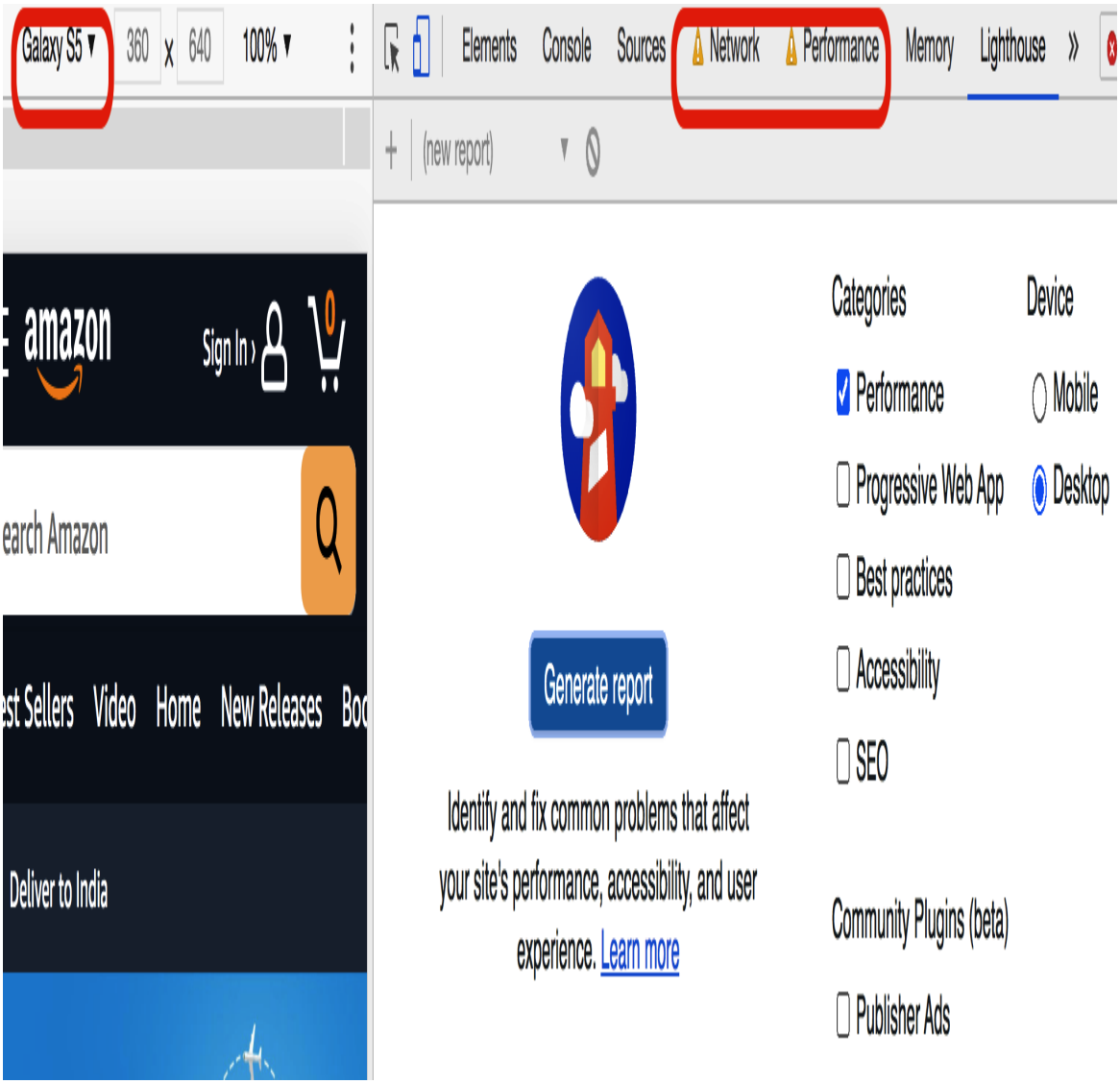


Figure 8-17. Lighthouse window with network, CPU, and resolution configurations

As seen in Figure 8-18, the results tell us that Amazon does a pretty good job. The time to interactive metric even in such skewed conditions is 3.8 s!



Performance

Metrics

First Contentful Paint	1.1 s	Time to Interactive	3.8 s
Speed Index	3.6 s	Total Blocking Time	420 ms
Largest Contentful Paint	3.8 s	Cumulative Layout Shift	0.036

Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

View Original Trace



Figure 8-18. Lighthouse performance report

You can use Lighthouse to test various frontend performance test cases as early as during development itself. To integrate with CI, you can use the Lighthouse Node.js module. To install it, run the following command from your terminal:

```
$ npm install -g lighthouse
```

To run a performance audit, run the following command:

```
$ lighthouse https://www.example.com/ --only-
categories=performance
```

You can send optional parameters along with this command to set network and CPU throttling values and select device screen sizes. The audit report will be available in the current directory. You can write a wrapper to fail the pipeline if the performance score is less than a threshold; for instance, you might want to fail the build if the score is less than 90. You can also define performance budgets (upper threshold values) for each of the web vitals using the LightWallet feature. This will assert Lighthouse's performance results against the defined threshold values for each metric and raise alerts when they are exceeded.

Another way to integrate with CI is via the cypress-audit tool. It integrates Lighthouse with Cypress, using which you can run the performance audits as part of your functional tests in the CI.

Additional Testing Tools

There are a couple of other tools that assist in different ways in measuring and debugging frontend performance. In this section we'll

explore some of the features of PageSpeed Insights and Chrome DevTools.

PageSpeed Insights

The tools we used in the preceding exercises allow us to simulate test cases like in a lab, where we set preconditions and observe results. But there could be many nuanced variations in real life due to minor differences in users' network bandwidth, device configurations, etc., which can't be predicted and measured. The only way to know how different users truly experience the website's performance is through real user monitoring (RUM) after the application has gone live. Google provides free monitoring services that record the core web vitals along with other metrics as and when users from across the globe access the live application. This data is called *field data* or *RUM data*.

The PageSpeed Insights tool attempts to give a holistic view of frontend performance by presenting the RUM data along with the lab data produced by Lighthouse, as seen in Figure 8-19. Try this tool by entering your live application URL on the PageSpeed Insights home page.

Field Data – Over the previous 28-day collection period, field data shows that this page passes the Core Web Vitals assessment.

First Contentful Paint (FCP)	1.5 s	First Input Delay (FID)	29 ms
82% / 11% / 7%		92% / 6% / 2%	
Largest Contentful Paint (LCP)	1.7 s	Cumulative Layout Shift (CLS)	0.01
86% / 8% / 6%		90% / 4% / 6%	

Show Origin Summary

Lab Data

First Contentful Paint	2.5 s	Time to Interactive	12.7 s
Speed Index	9.4 s	Total Blocking Time	370 ms
Largest Contentful Paint	3.3 s	Cumulative Layout Shift	0.121

Figure 8-19. A PageSpeed Insights field data and lab data report

PageSpeed Insights also exposes APIs to monitor and alert constantly.

Chrome DevTools

Another handy tool for frontend performance debugging is the performance profiler available on the Performance tab in Chrome DevTools. It gives detailed analysis reports around the network stack, animation frame rates, GPU consumption, memory, script run time, and more to enable the developers to save precious milliseconds. The profiler also allows you to throttle the network and CPU while debugging. And since it is embedded within the browser itself, it is developer-friendly.

Here's how it works. Suppose you want to find out how the auto-populated drop-down in your application's UI performs. You can record the action of entering text in the drop-down using the provision on the Performance tab; once the recording is stopped, the performance analysis reports will be shown in the same tab, as seen in Figure 8-20.


Disable JavaScript samples
Network: No throttling
Enable advanced paint instrumentation (slow)
CPU: No throttling
2000 ms
4000 ms
6000 ms
8000 ms
815 ms
815 ms
6815 ms
8815 ms
FPS
CPU
NET
2215 ms
2415 ms
2615 ms
2815 ms
3015 ms
3215 ms
3415 ms
3615 ms
3815 ms
4015 ms
4215 ms
Network
com.amazon.csm.csa.prod (unagi.amazon.com)
www.amazon.com/ (www.amazon.com)
render (www.amazon.com)
suggestions...
Frames
223.0 ms
203.8 ms
384.8 ms
Interactions
Ani...on
Timings
FP
FCP
DCL

Figure 8-20. A sample report from the Chrome DevTools performance profiler

With that, you should be ready to get started with end-to-end performance testing of your application. The last part is putting all of this together to form a performance testing strategy so that you can plan the required time and capacity well in advance.

Performance Testing Strategy

As mentioned a few times throughout the chapter, shifting left should be the guiding principle behind your performance testing strategy. Shifting left should start from designing the architecture in a way that befits the expected performance numbers and extend to integrating performance tests into your CI pipelines for frequent and continuous feedback, recalling how this will be profitable for the business, congenial to the end users, and favorable for your weekend plans. Figure 8-21 shows an overview of a shift-left performance testing strategy that applies the fundamentals discussed in this chapter.

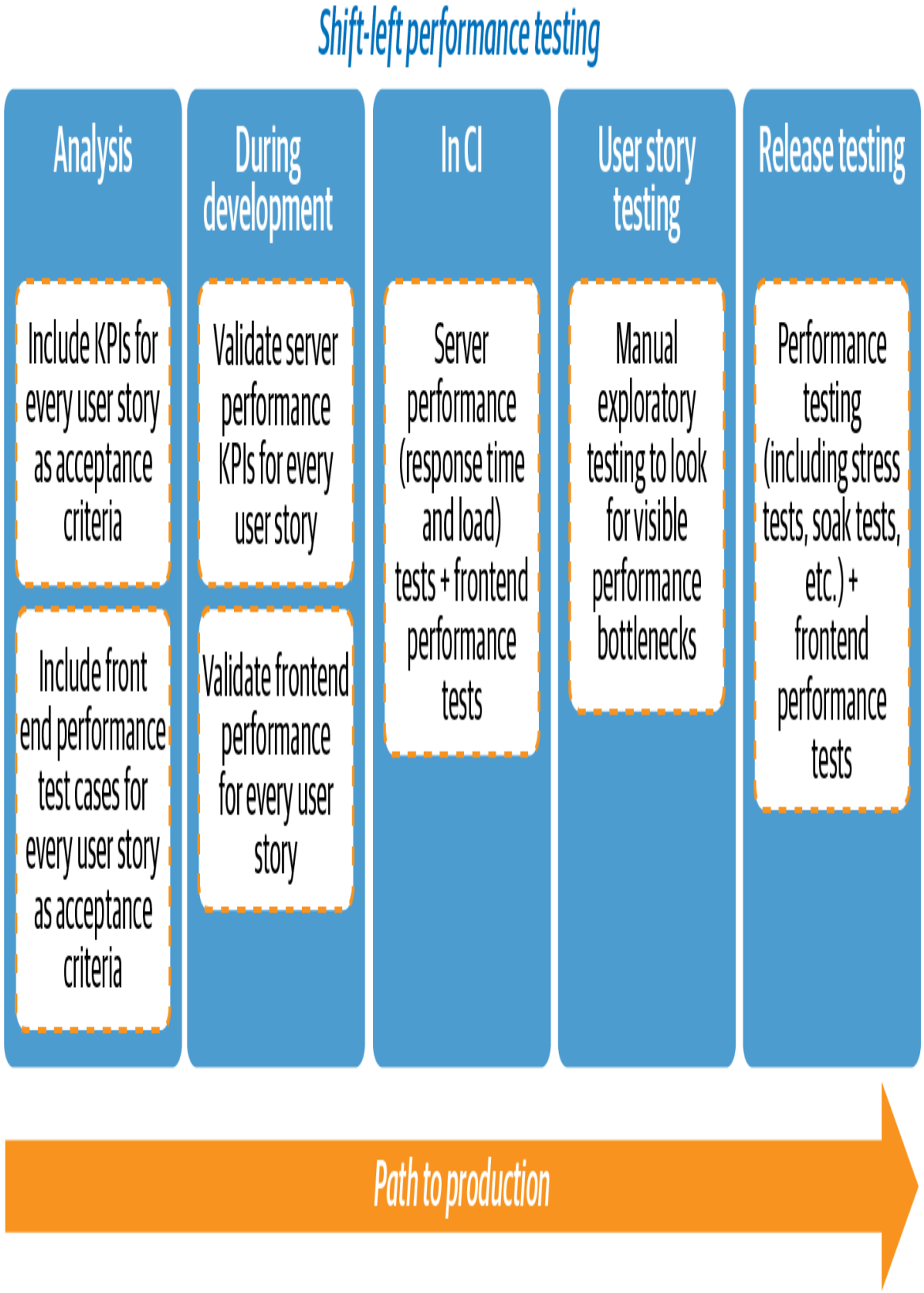


Figure 8-21. A shift-left performance testing strategy

Let’s walk through the different phases in shift-left performance testing. In the planning phase:

- Arrive at a consensus on performance KPIs with all the application stakeholders, including business, marketing, and technical folks, before the project starts. Design the architecture and choose the tech stack and other details based on these numbers.
- Get a performance testing environment set up at the beginning of the project. If it can’t be similar to the production environment, at least make sure you have an environment to begin testing in.
- Include various frontend performance test cases (like network conditions, geolocation, etc.) as part of every user story’s acceptance criteria.
- Include the expected KPIs (response time, concurrency, and availability) of the APIs as part of every user story’s acceptance criteria.

During development:

- Validate the respective server KPIs (through response time and load testing of the endpoints) for every user story.
- Validate the frontend performance test cases for every user story.

In CI:

- Run all the response time validation tests for every commit. Depending on the time taken to run the load tests, run them for every commit or as nightly regressions to catch performance issues early. This will also help you see how the performance degrades gradually as you add more features and will help in debugging later.

- Include frontend performance tests for the frequently visited pages as part of your CI pipeline.

During user story testing:

- Watch for visible performance bottlenecks during exploratory testing of different test cases.
- Ensure the performance-related acceptance criteria are met, automated, and integrated with CI before marking a user story as complete.

And finally, during the release testing phase:

- Complete the end-to-end application performance testing, including stress testing and soak testing as well as debugging activities. Before this stage, strive to get a production-like performance testing environment set up.

As you will have figured out by now, performance testing takes significant effort and can't be abruptly introduced in your release cycle as an afterthought without disrupting the timelines!

Key Takeaways

Here are the key takeaways from this chapter:

- Poor web performance can have a serious financial impact on a business. Conversely, improving performance can boost conversions and revenue significantly.
- Diverse factors, such as architecture, the performance of third-party services, network bandwidth, the user's geolocation, and more, influence an application's performance. These factors keep changing throughout the software delivery cycle and sometimes optimizing one can only be done at the expense of another, posing a tough challenge to software teams.

- Measuring the KPIs (availability, concurrency/throughput, and response time) continuously from the beginning of the software delivery cycle will help in preventing major performance issues in production.
- Several tools, such as JMeter, Gatling, and Apache Benchmark, are available to perform shift-left performance testing.
- Focusing separately on frontend performance is essential, as the frontend code is responsible for 80% or more of that average application's load time.
- Google's RAIL model provides a useful framework for defining your frontend performance metrics.
- Design your frontend performance test cases with the end user's experience in mind. Include different end user variables like network bandwidth, geolocation, and device capabilities.
- Include both API and frontend performance tests in your CI pipeline, and save your team from big-bang performance surprises!

---

1 For more on this idea, see Scott Barber's white paper "Get Performance Requirements Right—Think Like a User".

Chapter 9. Accessibility Testing

Accessibility—essential for some, useful for all.

—W3C WAI

The web is an essential aspect of our lives in so many ways—we use it to purchase everyday commodities and have them delivered to our doorsteps, to communicate with friends and family, to learn new skills and keep up with world news. I can't imagine how much more difficult it would have been to get through the Covid-19 pandemic without the connectivity, productivity, and information the web provides. Making such a vital commodity available to all users with permanent, temporary, or situational disabilities is termed *web accessibility*. This includes people who are visually impaired, elderly, have literary gaps, are driving cars, or face other challenges in accessing the web. Accessibility is a subset of usability, in web development terms. It is a subset of inclusivity in humanitarian terms.

Although the main goal is allowing people with accessibility challenges to avail themselves of the services of the web, it in fact enhances everyone's lives. I love the tagline "Essential for some, useful for all," coined by the W3C's Web Accessibility Initiative (WAI); it emphasizes how web accessibility features are useful for all users, irrespective of disabilities or other obstacles. For example, all of us prefer a clear, structured layout where the different parts of the page are easy to locate and identify and the site is easy to navigate. Similarly, having simple, understandable error messages and instructions is a fundamental need for all users, and voice-enabled applications are witnessing rapid adoption across user segments because of the ease they provide in this fast-moving world.

> **NOTE**
>
> W3C stands for World Wide Web Consortium. It is an international community led by Tim Berners-Lee, inventor of the World Wide Web, that works with member organizations and public users to establish standards for the web. The W3C WAI has set global standards for web accessibility, which we will discuss in this chapter.

Shifting the focus to a business's point of view, the disabled community can be said to form the third-largest economy globally in terms of purchasing power, as 1 in 5 of the world's population is challenged in some way. This results in a concrete business case to invest in web accessibility features.

Furthermore, an accessible web is often a legal requirement. According to the United Nations Convention on the Rights of Persons with Disabilities (UN CRPD), access to information and communications technologies, including the web, is a fundamental human right. Many countries now have legal policies for web accessibility based on that, and in recent years there has been a surge in lawsuits against companies for violation of those policies. The first case was won in 2017, when a person with visual impairments sued Winn-Dixie, a supermarket chain in the US, because the company's website did not support screen readers (although that decision has since been overturned). So, for all of these reasons and more, if you aren't already, it is time for the software development teams and businesses to start paying closer attention to web accessibility features.

This chapter will give you a broad introduction to web accessibility testing and tools. You will get an overview of accessibility personas, the ecosystem of tools and technologies, the inner workings of screen readers, and the web accessibility guidelines that are mandated by many governments around the globe. You will also learn about web development frameworks that support accessibility,

and a shift-left accessibility testing strategy. Finally, there are exercises presenting automated accessibility auditing tools that you can incorporate into your continuous testing strategy to empower your team to continuously deliver an accessible website!

> **NOTE**
>
> Mobile accessibility testing tools are covered in Chapter 11.

Building Blocks

Let's begin with getting to know accessibility user personas and their specific needs. The user persona discussion will be followed by an overview of the accessibility ecosystem and the web accessibility guidelines.

Accessibility User Personas

To recall, a user persona is a character that represents a subset of the larger audience with similar attributes. We create user personas in software projects to understand their specific needs and assimilate them throughout the software development stages, starting from design. Figure 9-1 shows a set of accessibility-specific user personas.



Figure 9-1. Accessibility user personas

These personas might be defined as follows:

- Matt, a 30-year-old business professional, has recently broken his arm. As he struggles to operate the mouse, he needs keyboard-only access to the website.
- Helen is an 80-year-old retired teacher whose color sensitivity has become poor. To access the web, she needs color contrast in the UI—i.e., distinguishable background and foreground

elements like images, links, buttons, etc. This requirement also applies to users with color blindness.

- Abbie is a teenager who has cognitive disabilities. Since she takes time to learn new things, she needs a clean web layout with proper headings, navigation bars, and consistent navigation structures to access the web. Fred (not pictured), who wants to find a nearby gas station while driving, also needs a clear layout of information to make a decision quickly.
- Connie is blind and an independent store manager. He needs text to speech and voice recognition support for accessing the web.
- Laxmi has an infant whom she carries around most of the day. She also needs speech to text in order to send texts.
- Maya is a software professional who has reduced dexterity, and she needs large text, buttons, and controls to access the web. Users with dyslexia and low vision also have this requirement.
- Philip is deaf and a cooking enthusiast. He needs captions to understand the recipe videos he enjoys watching.
- Xiao is a Chinese-speaking retail shop owner. He has been learning English for only a couple of months now. Xiao needs simple instructions and understandable content that doesn’t include jargon or complex words and sentences to access the web. Users with cognitive and learning challenges will also benefit from this feature.

Collectively, our user personas display visual (complete or partial), hearing, cognitive, and muscular challenges, as well as temporary accessibility restrictions. The goal is to enable all of them to perceive, understand, navigate, and interact equally, like any other user.

Accessibility Ecosystem

To build accessible web features, we have to understand the entire accessibility ecosystem. This encompasses the various tools and technologies (beyond just web technology) that interact and combine to deliver content to users with temporary and permanent disabilities. For example, a user persona like Connie, who is blind, uses text to speech and voice commands to interact with the web. To enable that, text reader and voice command technology cooperate. Some of our other personas need assistive devices to be integrated into the computer. So, in order for us to think of different accessibility use cases to build, we need to understand these various components and integrations, at least at a high level. Elements of the accessibility ecosystem that we should consider include:

Web development tools and practices

> Quite obviously, web development tools such as HTML, CSS, etc. should have the necessary facilities to make the web accessible. For instance, to pass on information about the elements on a page to the screen reader, there should be provisions in the web development frameworks to mention them explicitly.

User agents

> These are the tools that render the web content, such as browsers and media players. These user agents should understand that the web content is enabled with accessibility-related features and integrate with other tools, such as screen readers, to deliver the content.

Assistive technologies

> Assistive technologies are the additional devices and technologies that talk to the browser and relay information to and

from the user—for example, screen readers, alternative keyboards, switches, and more.

As you can see, the accessibility ecosystem comprises a vast set of tools and technologies. Having accessibility provisions in all these components enables all of our personas to interact with the web. Some may provide more advanced features than others, which might lead to more roundabout work being required in one area or a lack of some features for our user personas.

To ensure all these components have standardized accessibility features, the W3C WAI has established international standards for each of them, as listed here:

- *Authoring Tool Accessibility Guidelines* (ATAG) establishes the standards for content authoring tools such as HTML editors.
- *Web Content Accessibility Guidelines* (WCAG) defines web content standards and is the one we should pay attention to during development.
- *User Agent Accessibility Guidelines* (UAAG) addresses standards for web browsers and media players, including some aspects of assistive technologies.

All of these standards are detailed on the WAI site. As web development teams, we will take a deep dive into WCAG in the next section—specifically WCAG 2.0, which lists specifications for various aspects of the web content (text, images, colors, media, etc.) to make it accessible. Many countries have crafted policies for government, public, and private sectors to mandate WCAG 2.0 standards, as mentioned earlier.

Example: Screen Readers

To make sense of why WCAG 2.0 prescribes specific guidelines, we need to understand how assistive technologies work. Let's consider the example of screen readers, used by our visually challenged user personas—this is a common assistive technology whose support we should be sure to test.

As the name suggests, screen readers read aloud the content on the page for the user, who interacts with the website via a keyboard. So, as they hear the content, the user presses keyboard shortcuts such as Tab, Tab+Shift, Enter, etc., to interact with the site.

The screen reader recites the content on the page in the order of the page's *accessibility tree.* This is a DOM-like structure with the page elements, together with attributes like roles, IDs, etc., explicitly defined in a sequence representing a meaningful flow. For example, consider a booking site with To and From text input fields and a Search button on the home page. The accessibility tree will be structured to represent the "search tickets" user flow, i.e., to enter the From location first, then the To location, and then click the Search button. We can code certain elements on the web page to be hidden in the accessibility tree if needed.

To better relate to the accessibility features, it's a good idea to experience using a screen reader yourself. Google Chrome provides a browser-based screen reader as an extension. Try it out! There are also demo websites like the example booking site in Figure 9-2 that give a sense of how the visually impaired might experience the web; the content is intentionally blurred, and you can walk through making a booking using a screen reader and your keyboard.

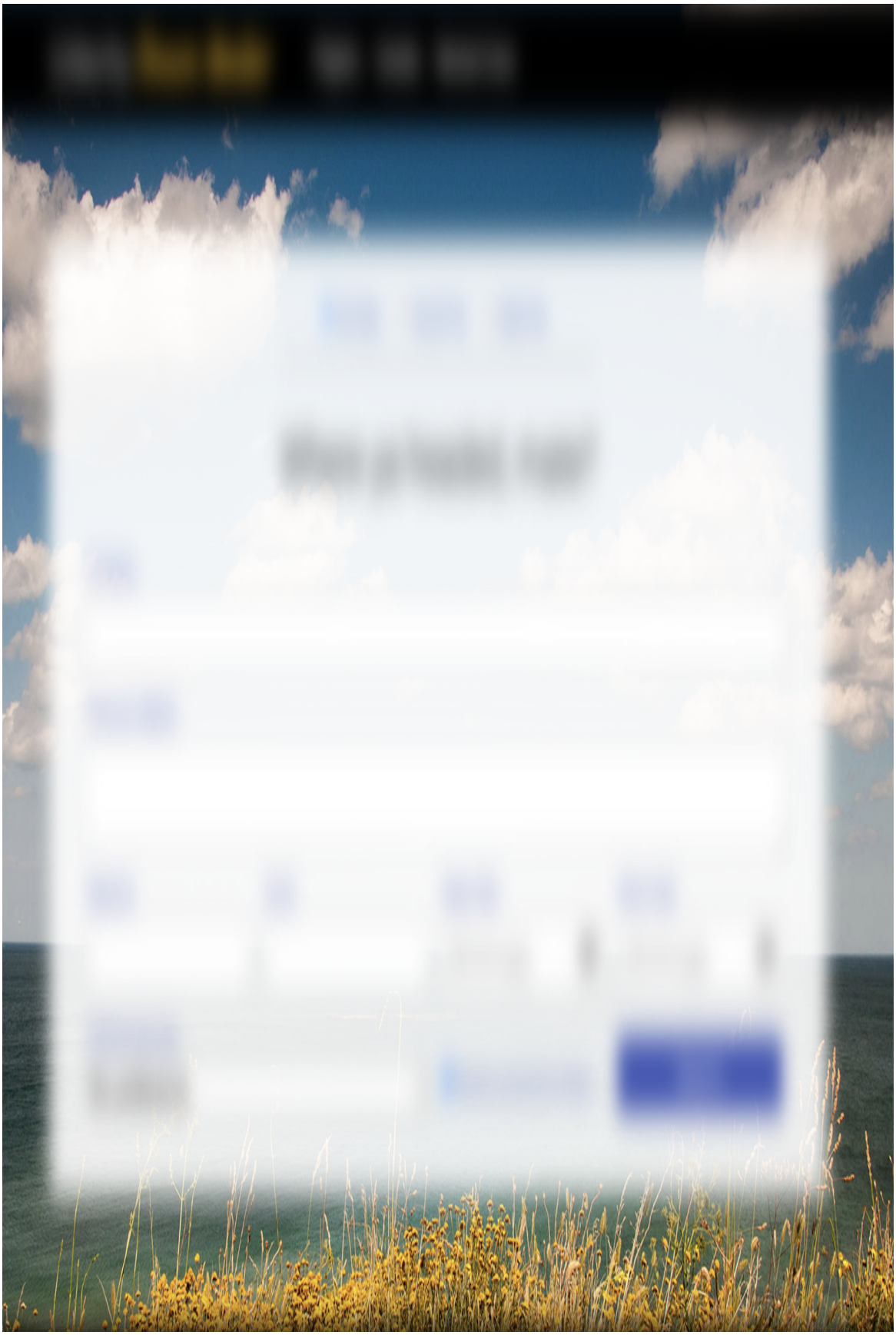


Figure 9-2. Screenshot from a demo website, intentionally blurred, to simulate screen reader experience

WCAG 2.0: Guiding Principles and Levels

If you've had a chance to try the screen reader, great! Otherwise, I hope you got an idea of how things work from the previous section. Let's explore WCAG 2.0 in detail now.

WCAG 2.0 sets four guiding principles to remember while designing web content: the content should be *perceivable*, *operable*, *understandable*, and *robust.* The standard also defines three levels of conformance based on the extent to which the content meets the success criteria defined for each of the defined guidelines:

Level A

> This is the minimum level of conformance, providing essential support without which the site is inaccessible. For example, audio or video content should have captions, all functionality should be accessible via the keyboard, and color must not be used as the only means of conveying information. Level A conformance will enable all our user personas to navigate the web.

Level AA

> This encompasses all the Level A requirements plus additional stricter requirements, such as constrained color contrast ratios across the site. Some legal policies recommend that sites achieve this level of conformance.

Level AAA

> This level subsumes all the requirements from the previous two levels and calls for additional enhanced requirements to make the web truly accessible for all users. An example requirement from this level would be to have sign language interpretations for

video content. When you seek to achieve this level of conformance, it shows the users you really care about them!

Organizations should determine the level they need to conform to based on legal requirements, but can choose to implement a higher level of conformance to serve additional users.

Level A Conformance Standards

Let's take a look at the WCAG 2.0 Level A requirements, the minimum level websites need to adhere to. We can translate these requirements directly into our accessibility test cases.

> **NOTE**
>
> The details provided here are only intended to give an overview, to help the reader understand the requirements of accessibility testing. The official documentation is available on the W3C WAI site.

Perceivable

The first principle is to make the web content available conveniently for all our user personas. Only when they can perceive the content will they be able to operate on it further. So, we should think of all the possible scenarios that could hamper this essential requirement right from the web design phase, and avoid them.

WCAG 2.0 elaborates this principle with detailed requirements for us to begin with, as follows:

- All non-text content, like images, should have alternate text that describes it to enable visually challenged users to understand the content using screen readers.
- Audio or video content should have text transcripts and captions (synchronized with the media) as alternatives, with provisions to

pause, stop, and control the volume.

- Any audio that starts playing automatically on page load should have audio control mechanisms like pause, replay, and volume controls.
- The web page's information and structure should be designed to have a hierarchy, such as a page title above all headings, appropriate page title and heading tags, and so on. This helps users with screen readers to have a meaningful flow.
- Instructions to navigate the website should not solely rely on sensory characteristics of components such as shape, color, size, visual location, orientation, or sound. For example, avoid an instruction that says "wait until the button turns green" or "wait until you hear a beep."
- Colors should not be the only way to indicate an action, prompt for a response, or distinguish elements on the screen. Make it intuitive via text for color-blind users.
- The page should have color contrast between background and foreground elements to support users with less color sensitivity. There is a fixed ratio prescribed for this.

Operable

Once the web content is made perceivable, we must consider ways to let users operate the website comfortably, such as clicking a button using keyboard shortcuts. WCAG 2.0 includes specific requirements along these lines, as follows:

- Provide keyboard-only navigation support allowing users to operate the entire website. While using keyboard navigation, the focus on the elements should be clear and have appropriate color contrast.

- Add provisions to move forward, backward, and exit an area via keyboard shortcuts—for example, keys to exit a modal window.
- Provide enough time for users to read the content entirely.
- Avoid content that flashes on the screen and has many animations, as it could cause physical reactions like seizures.
- Provide the ability to skip repetitive content.
- Hide the offscreen content for screen readers. For example, if a link appears only in a particular selection, hide it in the screen reader flow.
- Provide elaborate, meaningful text for links.

Understandable

A website may require many elements and user flows to complete an action. For example, to book an airline ticket, there may be several steps and instructions to follow. Such content and user flows should be carefully crafted to be simple and straightforward for all user personas. WCAG 2.0 once again calls out solid requirements here:

- Avoid jargon and technical terms; present simple, meaningful content instead. For example, avoid technical error messages like “034506451988 is invalid” and provide understandable text like “Incorrect date format.”
- Provide expansions and abbreviations where necessary.
- Avoid sudden changes in context (e.g., opening multiple windows), as it will affect the keyboard navigation.
- Avoid changes of context when the user has different settings, like a larger font.
- Provide clear, actionable label text for elements to help users take the right action. For example, an email address input field

should have the label Email and a sample value like *example@xyz.com*.

Robust

Finally, we should make the web content robust, supporting different types of user agents and assistive technologies. Screen readers are not the only assistive technologies; many others will require proper integration! WCAG 2.0 calls out the following requirements for this principle:

- The markup language content should follow standards like having opening and closing tags, no duplicates, unique IDs, etc., so that it is easily parsable by multiple assistive technologies.
- The name, role, and state of each element, including those generated by scripts, should be available for assistive technologies (for example, `role="checkbox"` and `aria-checked="true|false")`. Provide the updated state of elements like checkboxes to the screen reader after selection.

WAI'S ACCESSIBLE RICH INTERNET APPLICATIONS (WAI-ARIA)

We saw earlier how a screen reader reads the elements on the page and describes the actions to perform on them based on the web page's accessibility tree. Sometimes, when custom elements are developed for enriched user interaction, assistive technologies will not be able to identify the elements. Such elements must carry additional attributes indicating their type, states, and behaviors for assistive technologies to understand them. For example, a standard HTML element definition, say `<input type="checkbox">`, will be automatically translated as a checkbox, and the end user will be rightly instructed to perform a click action (click) on it. However, when a list (`<li>`) element is made to look like a checkbox using CSS, it has to be augmented with new attributes for the assistive technologies to understand it properly.

WAI-ARIA provides specifications for these attributes (e.g., `roles`, `aria-checked`, etc.) that must be adhered to during web development. These ARIA attributes are added to the accessibility tree, making it friendly for all assistive technologies.

Those are the essential Level A requirements. There is also an updated version of the standard, WCAG 2.1, which includes a few more requirements in order to better cater to a certain set of user personas; you can explore these in the official documentation if your organization chooses to comply with this version.

Accessibility Enabled Development Frameworks

To build the previously mentioned features, many development frameworks provide elaborate accessibility support. For example, React fully supports building accessible websites, often by using standard HTML techniques. Similarly, the Angular team maintains an

Angular Material library that provides a suite of reusable UI components that aim to be fully accessible. Vue.js has support to create accessible components as well. There are also automated accessibility auditing tools, as you will see in the following section, that alert if standard accessibility-related tags are missing in the HTML. So, don't worry, you have support! Your team can achieve this without much additional effort too.

Accessibility Testing Strategy

It should be evident from the preceding sections that most of the accessibility requirements have to be thought through right from the start of the project and continuously supported throughout the development process rather than retrofitted after the testing phase. For example, to conform to Level A, you should incorporate simple, consistent navigation across the site, captions for videos, meaningful error messages, color-contrasted images, etc., during product design rather than during development or testing. So, as a first step to support accessibility, teams should define the application's accessibility user personas, similar to what we discussed at the beginning of the chapter, and tailor user stories catering to each of those user personas. Then, when your team discusses the product's features, collectively validate whether the accessibility flows are included in the scope. That will be the primary step in shifting accessibility testing to the left!

Figure 9-3 shows the shift-left implementation of accessibility testing throughout the software development lifecycle. We'll dive into a few of these items in the remainder of this section and in the exercises that follow.

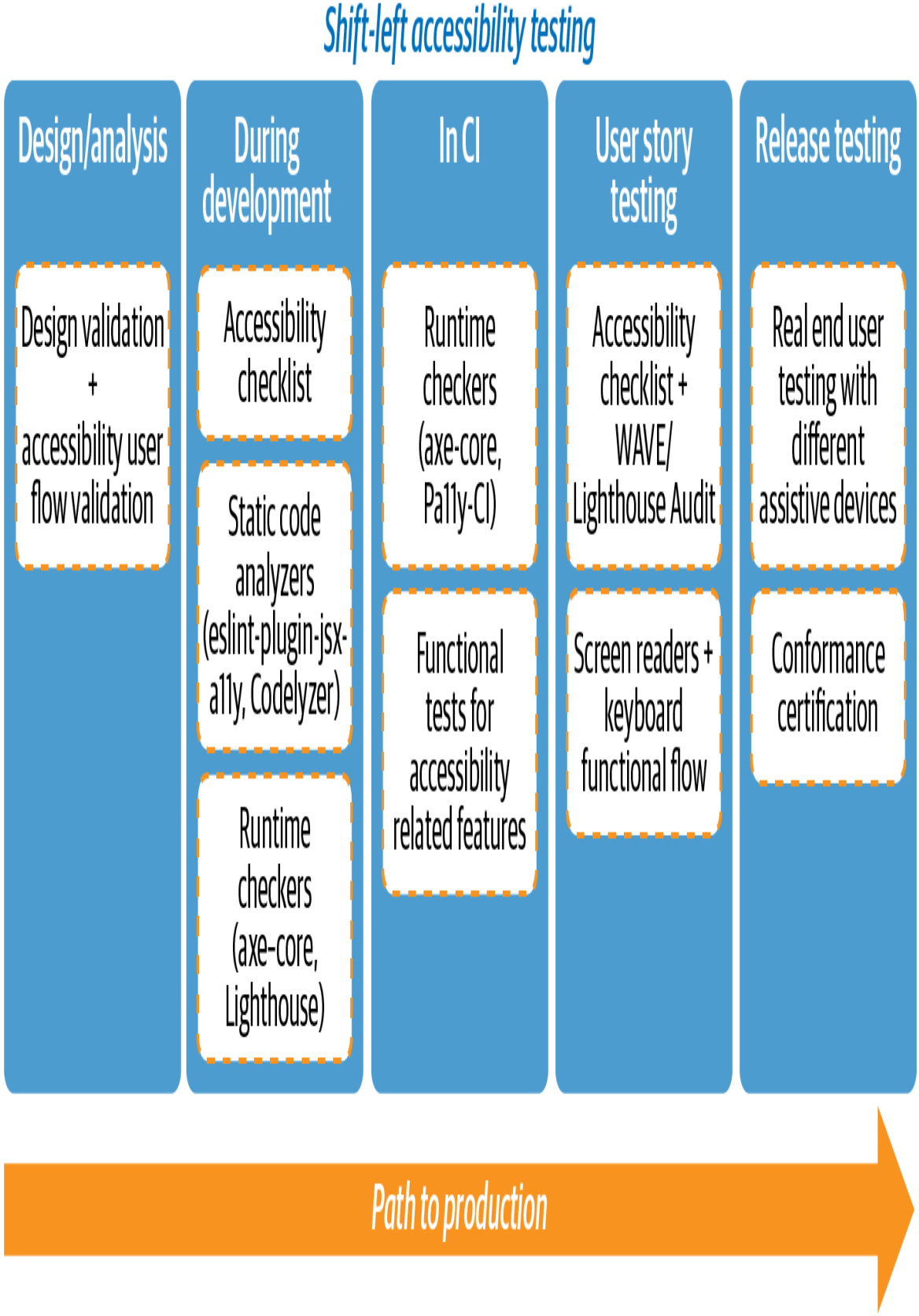

Shift-left accessibility testing
Design/analysis
Design validation + accessibility user flow validation
During development
Accessibility checklist
Static code analyzers (eslint-plugin-jsx-a11y, Codelyzer)
Runtime checkers (axe-core, Lighthouse)
In CI
Runtime checkers (axe-core, Pa11y-CI)
Functional tests for accessibility related features
User story testing
Accessibility checklist + WAVE/ Lighthouse Audit
Screen readers + keyboard functional flow
Release testing
Real end user testing with different assistive devices
Conformance certification
Path to production

Figure 9-3. A shift-left accessibility testing strategy

Accessibility Checklist in User Stories

The WCAG 2.0 guidelines include several universal requirements that span across all the pages of the website, such as adding alternate text, support for keyboard navigation, page titles, etc.. Hence, appending an accessibility checklist to all the user stories will help developers and testers go through these requirements meticulously. The following is a generic checklist you can use in your team, after appending any application-specific items.

ACCESSIBILITY CHECKLIST

- Check for the page title in the browser. When you hover over the browser tab, you can see the page title as a small widget in Chrome. This text should clearly define the context of the page within the website.
- Check the basic structure of the web page to ensure it has proper element attributes and hierarchy. Turn off the CSS and verify that the elements are listed in an order that is screen reader–friendly. Chrome has an accessibility tree view in the DevTools to show the order of elements.
- Check for keyboard-only navigation, including proper element highlighting and keyboard operations for backward, forward, and exit.
- Check that error messages, labels, links, and in general, any text on the page communicates the right intent.
- Check the page's readability upon resizing the text, either using the system preferences or browser zoom options.
- Check for readability in grayscale. For example, Mac users can enable this setting via System Preferences → Accessibility → Display → Use grayscale.
- Check the captions for video and audio content, ensuring they are meaningful and synchronized.
- Check for meaningful alternate text descriptions of images. You can verify this by turning off the image download option in your browser settings (for example, in Chrome, select Settings → Site Settings → Images → Block); the browser will then show the alt text in place of images.
- Check that the screen reader flow is meaningful and the end user is able to complete the user flow.

Automated Accessibility Auditing Tools

The accessibility checklist includes some aspects that it is only possible to verify with human intervention, such as viewing the site in grayscale, zooming in and out, checking the meaning of the alternate text, and so on. You can complement it with automated auditing tools that will scan the basic HTML structure and alert if the accessibility tags for any elements are missing. These tools can save you a lot of time and effort by giving instantaneous feedback on missing tags during the development phase itself.

They come in the form of static code analyzers and runtime accessibility checkers. eslint-plugin-jsx-a11y is a linting tool for React; it's an ESLint plug-in that enforces several accessibility standards directly in your JSX. Similarly, Codelyzer has linting rules for accessibility standards in TypeScript, HTML, CSS, and Angular source code. These tools will give feedback as you're developing, while the runtime checkers (such as axe-core, Pa11y CI, and Lighthouse CI, which are discussed later in the chapter) will give feedback on the actual web page post-development. They can be run in the local development machine to get faster feedback on the pages developed as part of every user story and also as part of CI for continuous testing.

In addition to using tools such as these, you can add functional flows that cater to accessibility needs, such as having a separate transcript section below any video/audio or a set of meaningful error messages and instructions, as automated micro- and macro-level functional tests.

Manual Testing

Manual testing is critical in validating website accessibility. As mentioned previously, the automated tools only check the HTML structure, and the checklist only includes the mandatory items that are common across all the pages. There will be many items left to handle apart from these as part of manual testing—for example, verifying the functional flow with a screen reader and a keyboard. So, as part of manual testing, you can focus on different scopes in different stages, such as the following:

User story testing

As part of user story testing, ensure the checklist works appropriately on all the pages to which the user story extends. You can couple this with the web accessibility evaluation tool *WAVE*, a free online service by WebAIM. It helps find accessibility issues on the web page, as per WCAG 2.0 guidelines. Although the checks do not verify all of the WCAG 2.0 standard's requirements, it highlights issues visually on the web page in a browser, which can help you notice some accessibility issues that you may not have been aware of. Alternatively, you can use Lighthouse, which (as we saw in the last chapter) is part of Chrome DevTools, to get accessibility audits on the web page. Both of these tools are discussed later in this chapter.

Feature testing

With user story–level testing, you will cover the bulk of the accessibility testing effort. But when a feature is complete, you need to do another round of manual testing to ensure that end users can complete the user flow with only keyboard access and that screen readers can navigate the functionality as expected. This level of feature testing will help identify any lack of coherence in the application's end-to-end navigation.

To test keyboard-only navigation, use the Tab and Tab+Shift keys to move forward and backward through the website, the Enter key to select, and the up and down arrow keys for drop-down

selections. While doing this, make sure the focus is on the right element and that element is clearly highlighted. For testing the screen reader flow, you can use the Chrome extension mentioned earlier.

With these checks, you will finish the end-to-end accessibility testing of features.

Release testing

Finally, when all the release features are completed, it is recommended to do testing with actual end users, including people with disabilities. Since different users may have various assistive devices, this will give you real-time feedback before the product goes for a final evaluation for conformance certification. UserTesting.com is a remote testing service where you can request people with disabilities as testers for your site.

Conformance certification

Once the website is ready, experts in WCAG standards do the conformance evaluation before it goes live. This is not a centralized unit; organizations may have in-house experts or hire consultants to do the final assessment once the product is ready for certification.

Since every single element on the page requires changes to make it accessible, shifting left is the only way to rescue your team from the uphill task of fixing all the accessibility issues at the end of the development cycle.

Exercises

I mentioned some automated accessibility auditing tools in the previous discussion. You can try some of these tools as part of the following exercises.

> **NOTE**
>
> Accessibility auditing tools help confirm that the HTML structure is intact and alert if it isn't. For example, they check that all HTML tags are closed, that all images have an alt text attribute, that all form elements have labels, that all element IDs are unique, and so on. They're handy for performing a preliminary scan and providing fast feedback; however, they don't eliminate the need for manual testing.

WAVE

WAVE is an online accessibility evaluation tool that you can use to check a web page for compliance with accessibility standards. It has provisions to ascertain the structure of the page without CSS, and it flags issues to do with things like the color contrast of elements, `lang` attributes, and so on. WAVE is simple and free to use.

Workflow

To run an audit using the WAVE tool:

1. Open the WAVE website.
2. Enter your application URL in the "Web page address" box. Alternatively, you can use the WAI's inaccessible demo website, which has intentionally been made inaccessible for learning purposes.
3. Click the arrow to run the audit.

Figure 9-4 shows the summary of the audit results for the WAI's demo site. The tool has identified 3 structural elements and 6 features, and has flagged 37 errors and alerts and 2 contrast errors. You can see failure, success, and alert icons next to the respective web page elements on the right.


Summary
Summary
Details
Reference
Structure
Contrast
37
Errors
2
Contrast Errors
37
Alerts
6
Features
3
Structural Elements
0
ARIA
View details ›
CITYLIGHTS
*Red dot with a white let
crescent as well as the sun. This
that says 'CITYLIGHTS' which is t
Finally, the slogan of the portal,
turquoise green handwriting sty
top banner.*
Traffic: Construction w

Figure 9-4. WAVE audit report on the WAI's inaccessible demo website

Clicking the Details tab next to Summary will show the error details, as seen in Figure 9-5.

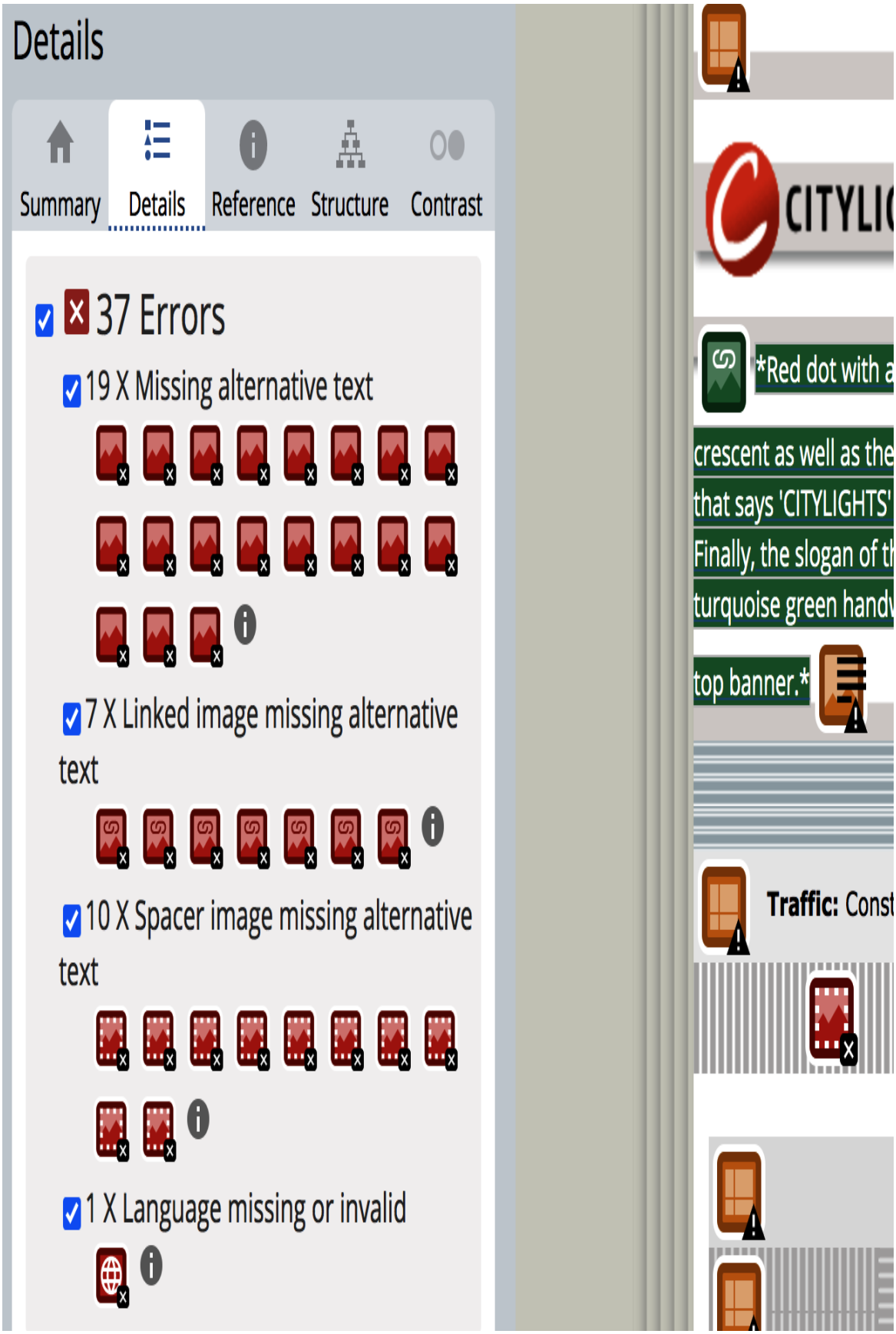

Details
Summary
Details
Reference
Structure
Contrast
37 Errors
19 X Missing alternative text
7 X Linked image missing alternative text
10 X Spacer image missing alternative text
1 X Language missing or invalid
*Red dot with a
crescent as well as the
that says 'CITYLIGHTS'
Finally, the slogan of th
turquoise green handw
top banner.*
Traffic: Const

Figure 9-5. WAVE error details display

The demo page has 19 images without alternative text, 7 images that are links without alternative text, 10 spacer images without alternative text, and 1 missing or invalid language attribute. The different icon styles on the Details tab can be matched to the icons on the web page for easier identification and debugging.

Next, to see the page's structure, you can turn off CSS styles using the control present above the summary section. The Structure tab will then show the analysis of the page structure. As you can see in Figure 9-6, when the styles are off, the text overlaps on the page and is clumsy. The web page is also missing proper hierarchies and Header, Navigation, and Main sections, qualifying it as inaccessible.

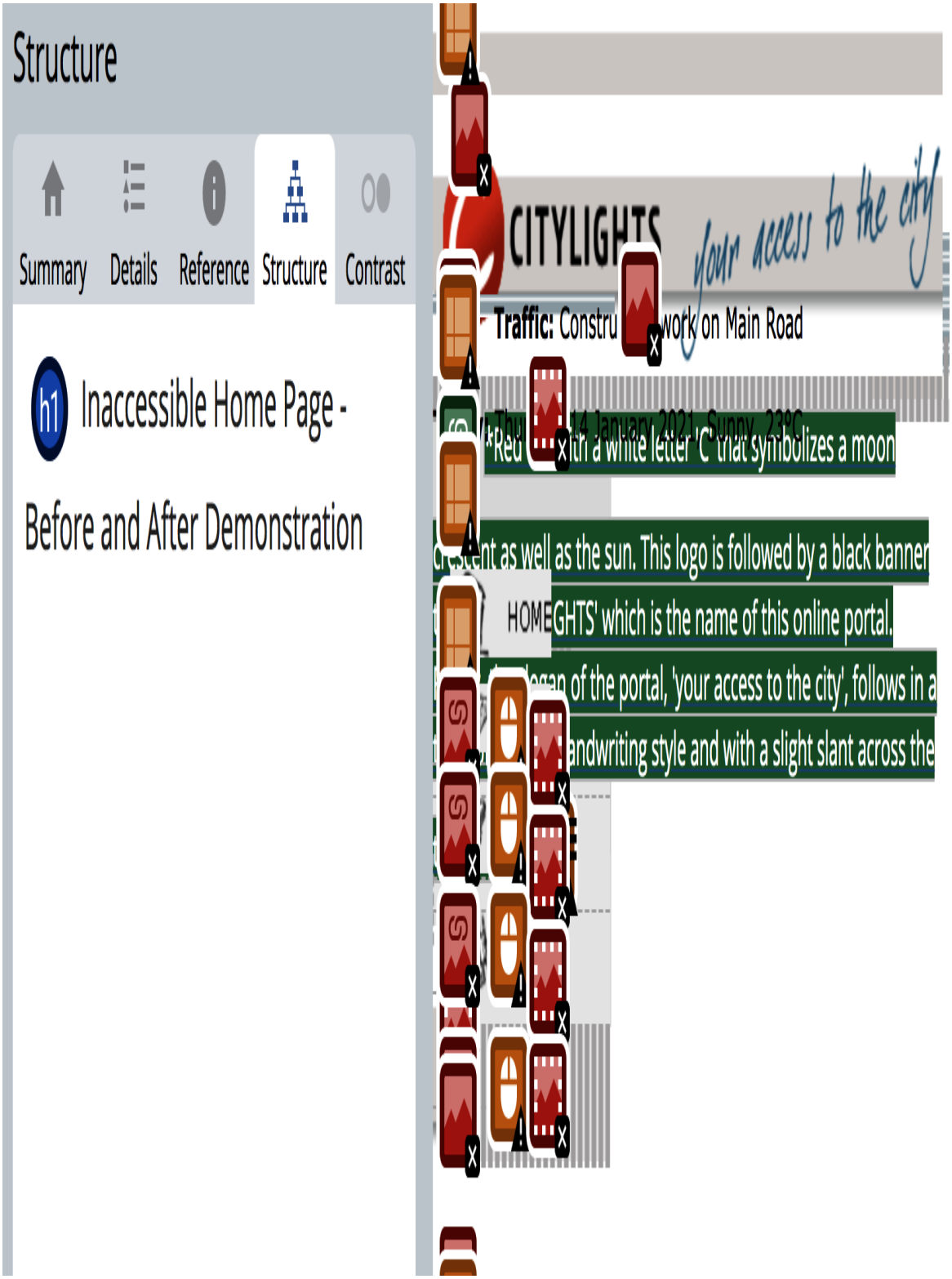
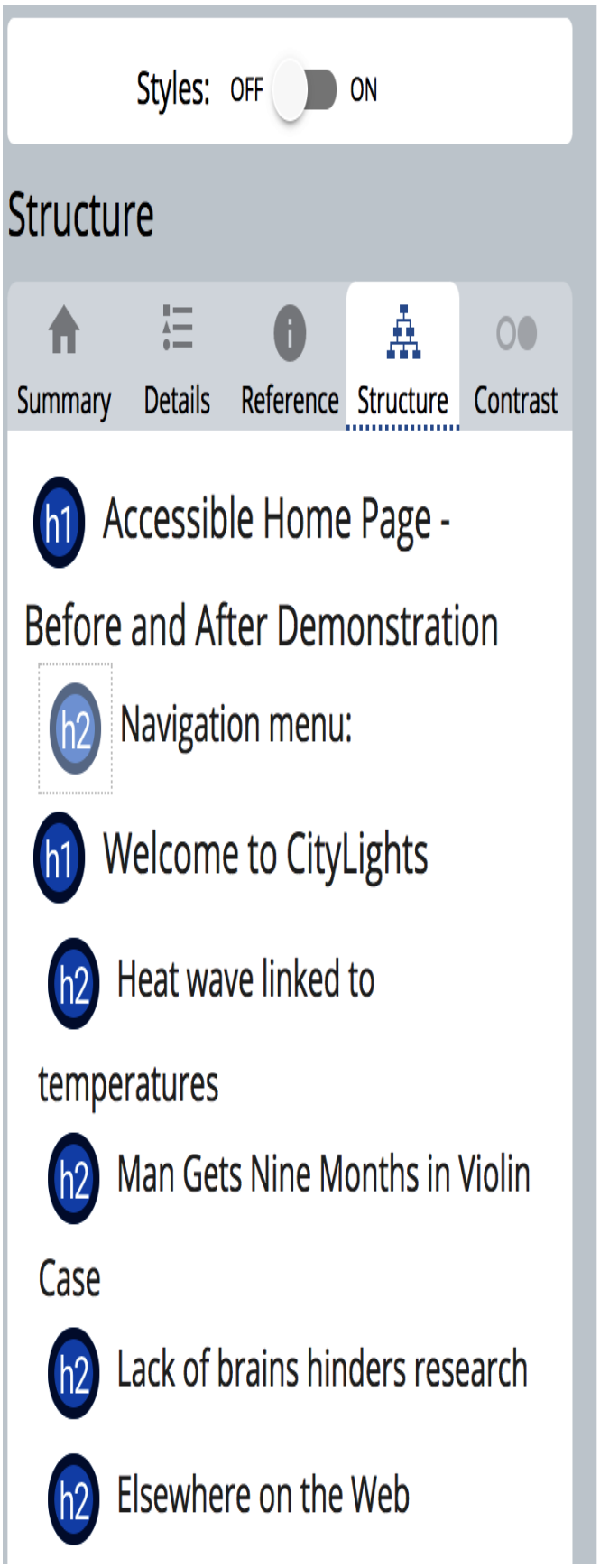


Figure 9-6. WAVE page structure analysis with styles off

Now try the WAI's accessible demo site, which is an accessible version of the same website. You can see the page structure is properly designed with a hierarchy, as shown in Figure 9-7.



- Skip to content (within demo page)
- Skip to navigation (within demo pag

CITYLIGHTS your

Explore Site by Topic: Quick Menu

Traffic: Construction work on Main Road

Today: Thursday 14 January 2021, Sunny

Navigation menu:

- Home
- News
- Tickets
- Survey

Figure 9-7. WAVE page structure analysis for the WAI's accessible demo website

In an accessibility-enabled site such as this, you can easily verify the sequence of elements and content and check that the navigation is as expected given the hierarchy displayed.

Lighthouse

If your application is not publicly accessible, you may not be able to use WAVE. An alternative is to use the Lighthouse tool from Google, which enables you to audit the accessibility of a website using your local Chrome browser.

Workflow

Try these simple steps to see how Lighthouse works:

1. Open the WAI's inaccessible demo website in Chrome.
2. Open Chrome DevTools using the shortcut Cmd-Option-I on macOS or Shift-Ctrl-J on Windows/Linux.
3. On the Lighthouse tab, select the Accessibility category, as shown in Figure 9-8, and click "Generate report."


Web Accessibility initiative
ter Demonstration
eb Content Accessibility Guidelines (WCAG)
News
Tickets
Survey
Template
Home Page
Report
Show Annotations
Home Page
Report
your access to the city
on Main Road
Today: Thursday 1
elcome to CityLights
tylights is the new portal for visitors and residents. Find out wha
(new report)
Generate report
Identify and fix common problems that affect your site's performance, accessibility, and user experience. Learn more
Categories
Performance
Progressive Web App
Best practices
Accessibility
SEO
Community Plugins(beta)
Publisher Ads

Figure 9-8. Using Lighthouse in Chrome DevTools to generate an accessibility report

Lighthouse's accessibility audit report will shortly be available in the same panel, as seen in Figure 9-9.





Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Only a subset of accessibility issues can be automatically detected so manual testing is also encouraged.

Contrast — These are opportunities to improve the legibility of your content.

▲ Background and foreground colors do not have a sufficient contrast ratio. ^

Low-contrast text is difficult or impossible for many users to read. Learn more.

Failing Elements

Free Penguins

```
<b>Free Penguins</b>
```

More City Parks

```
<b>More City Parks</b>
```

Internationalization and localization — These are opportunities to improve the interpretation of your content by users in different locales.

▲ `<html>` element does not have a `[lang]` attribute ˅

Names and labels — These are opportunities to improve the semantics of the controls in your application. This may enhance the experience for users of assistive technology, like a screen reader.

▲ Image elements do not have `[alt]` attributes ˅

Figure 9-9. Lighthouse accessibility audit report for the WAI's inaccessible demo website

As you can see, it reports similar issues to those identified by WAVE (two contrast issues, a missing `lang` attribute, etc.). Also, to help with debugging, the actual lines of code containing the errors are presented, and there are educational links to support the developers in fixing them. Lighthouse provides an overall score at the top of the report to give a sense of how good or bad the page is, and there is a checklist at the bottom of the report ("Additional items to manually check") to convey what the audit did not cover. This list also has educational links to guide in performing manual verification.

Lighthouse Node Module

The Lighthouse Node module does similar auditing to the version that is part of Chrome DevTools, but can be executed from the command line. It can therefore be used to integrate with CI and offers more flexibility in how the runs are configured and reported.

Workflow

To run Lighthouse's accessibility audits from the command line, follow these steps:

1. Assuming you have Node.js installed already, use the following command to install Lighthouse:

```
$ npm i -g lighthouse
```

2. Run the audit using this command:

```
$ lighthouse --chrome-flags="--headless" URL
```

The report is generated as an HTML file by default, as seen in Figure 9-10, in the same working directory.





Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Only a subset of accessibility issues can be automatically detected so manual testing is also encouraged.

Contrast — These are opportunities to improve the legibility of your content.

- Background and foreground colors do not have a sufficient contrast ratio.

Internationalization and localization — These are opportunities to improve the interpretation of your content by users in different locales.

- `<html>` element does not have a `[lang]` attribute

Figure 9-10. Lighthouse CLI report for the WAI's inaccessible demo website

You can add this HTML file as an output artifact in the CI pipeline for easy verification. Also, you can make the build fail if the scores are below a threshold with a wrapper build task.

Additional Testing Tools

A couple of other auditing tools that are frequently used for accessibility testing are Pa11y CI and axe-core. Let's look at what they offer.

Pa11y CI Node Module

Pa11y CI is a command-line tool that comes as a Node module. It runs accessibility audits against one or more URLs and reports issues, similar to what we saw with the WAVE and Lighthouse tools. To test multiple web pages using this tool, include the URLs in the `urls` section of the config file, as seen in Example 9-1. Alternatively, you can pass in an XML sitemap on the command line with the `--sitemap` option. You can add Pa11y CI to the project build stage in CI, or even as a separate stage in the pipeline.

> **NOTE**
>
> Accessibility is sometimes abbreviated to *a11y*, which stands for "a–[11 letters in between]–y."

Example 9-1. Pa11y CI's config file with before and after WAI demo sites

```
{
    "defaults": {
        "timeout": 1000,
        "viewport": {
```

```
            "width": 320,
            "height": 480
        }
    },
    "urls": [
        "https://www.w3.org/WAI/demos/bad/after/home.html",
        "https://www.w3.org/WAI/demos/bad/before/home.html"
    ]
}
```

The tool comes with options to set a threshold for errors and warnings up to which the CI build can pass, define viewport sizes to perform audits, and define the time to wait for the page to load.

Axe-core

The GitHub documentation for axe-core states that it can find, on average, 57% of WCAG issues automatically. It works with many browsers, including Microsoft Edge, Google Chrome, Firefox, Safari, and IE. The tool has many extensions built on top of it. For example, for Java Selenium WebDriver integration, you can add axe-core as a Maven dependency. Similarly, for Cypress, you have the `cypress-axe` Node module. There are also `vue-axe` and `react-axe` libraries to add tests in the frontend.

Basically, axe-core provides APIs to perform accessibility auditing on web pages, which can be added as part of functional tests. For instance, the `run()` API runs the accessibility audit on the current page and throws assertion errors on failures. So, just like other element assertions on the page, you can add this additional line to assess the accessibility of the page inside the functional tests.

To summarize, we have seen standalone tools that can be run as part of project build scripts or as a separate stage in CI and tools that can be integrated as part of a functional test suite. Whichever option you choose, it is best to strategize the continuous testing process such that your team gets feedback early in the development cycle.

All the effort you put in will pay off as you'll be able to ensure that you provide comfortable access to the application to Matt, Fred, Helen, Laxmi, Connie, Xiao, Abbie, Maya, Philip, and also yourself!

Perspectives: Accessibility as a Culture

Although we have discussed accessibility in the context of web applications throughout this chapter, these concepts don't only apply to websites. Accessibility is a culture and requires a shift in mindset. When we adopt this mindset, we start asking ourselves questions like: If I send an email with only images and no alternative text summary, will it be accessible for everyone? If I use a small font size in my presentation slides, will everyone be able to read them? Can I choose simple, clear messages over erudite vernacular so that everyone can understand? I am sure all of us know the answers to these questions!

Key Takeaways

Here are the key takeaways from this chapter:

- Web accessibility features are essential for some, but useful for all.
- The disabled community represents the third-largest economy globally in terms of purchasing power, which makes accessibility a strong business case.
- Many governments have policies to mandate accessibility, so it is also a legal requirement.
- The W3C WAI has come up with a set of Web Content Accessibility Guidelines that software development teams must follow. Check for the latest version of the guidelines at the time of your project development.

- The accessibility ecosystem is comprehensive, encompassing tools and technologies beyond just those of the web. It’s a good idea to try out at least one assistive technology, such as a screen reader, to help you think of good accessibility-enabled features for your application.
- We should integrate accessibility within the software development lifecycle from the beginning stages, as retrofitting it at the end will be a nightmare.
- Many web development frameworks have built-in support for accessibility features.
- Accessibility testing can be shifted to the left with accessibility checklists and automated static and runtime auditing tools such as Codelyzer, Pa11y CI, Lighthouse, WAVE, and axe-core.
- You can reach out to organizations that provide user testing services, including users with disabilities, to get real-time feedback on your application.

Chapter 10. Cross-Functional Requirements Testing

When we understand CFRs is when we truly understand quality!

Businesses often think of hundreds of functional requirements, looking to add value for customers and gain revenue. These functional requirements constitute the core business services offered to the customers—for instance, the feature to book a ride with a ride-hailing app or make a payment with an internet banking facility. However, just implementing these functional requirements is not enough to guarantee success. Imagine you want to book a ride, and you have to wait five minutes to see the list of available options. You could probably hail a taxi in that time, so why bother to use the app? Or maybe the app does its job functionally well, but it takes several steps to book a ride. The complexity would be frustrating, and you would likely look for a more user-friendly alternative sooner or later. Likewise, if you found out the app was exposing your personal details, you would certainly get rid of it. These are just a few examples of why businesses and software teams need to focus on cross-functional requirements (CFRs). They make the application complete and, most importantly, imbue high quality.

CFRs are features of the application that have to be built into every functional feature. For example, a couple of CFRs for the ride-hailing app could be that the app should respond to users within *x* seconds, users should be able to perform any action within *n* steps, and the app should transmit and store the users' details securely. Only when CFRs are built and tested thoroughly across all features will any app have a chance of becoming a strong competitor in the market.

CROSS-FUNCTIONAL VERSUS NON-FUNCTIONAL REQUIREMENTS

You will often hear CFRs referred to as *non-functional requirements* (NFRs). I, along with many others in the software industry, prefer the term *cross-functional*, as it emphasizes that the requirements are spread across the application and need to be built and tested as part of every user story and feature. Also, calling them "non-functional" may cause the false impression that they are nonessential, which goes totally against the goal we want to achieve—building a high-quality application!

If you are used to thinking of software requirements as functional or non-functional, you will see some "functional" features like authentication and authorization being referred to as cross-functional features in this chapter. This is because they are spread across the application. For instance, we have to verify the authenticity of every service request and respond with only the relevant information as per the requestor's access levels/permissions every time.

Although we have been discussing CFRs in the previous chapters (while considering topics such as performance, security, accessibility, and visual and data testing), this chapter is specifically focused on bringing attention to the full range of these requirements. In the process, we shall take a broader view of CFRs, and discuss an overall CFR testing strategy that can cater to providing continuous feedback for the team. We will also discuss some essential testing methodologies and tools to assist in implementing the testing strategy.

Building Blocks

In Chapter 1, we discussed how software quality is envisioned differently by the business and customers and how both parties demand a long list of quality attributes. These attributes essentially translate into the standard CFRs for any given application. Table 10-1 lists 30 common CFRs, with definitions and examples of each.

Table 10-1. Simple definitions of a lon

g

li
s
t
o
f
C
F
R
s

CFR	Simple definition
Accessibility	Ability of the system to enable user personas with disabilities to access the application seamlessly, as discussed in Chapter 9, such as through support for screen reader integration.
Archivability	Ability of the system to store and retrieve the history of application events and transactions as needed, such as storing a user's online purchase order history.
Auditability	Ability of the system to track the business events and states of an application through logs, database entries, etc. As explained in Chapter 7, this feature helps defend against the threat of repudiation.
Authentication	Ability of the system to allow only authenticated users to access the application's services in all layers. For example, a simple login feature.
Authorization	Ability of the system to restrict access to the application's services based on permissions, such as restricting access to view account details to only certain bank employees.
Availability	Ability of the system to provide the application's services for a defined period or threshold, as discussed in Chapter 8.
Compatibility	Ability of two or more systems to work in tandem without disrupting one another. For instance, the ability of the application to work with an earlier version of the same service (known as backward compatibility).
Compliance	Adherence of the system to legal requirements and industry

	standards, such as WCAG 2.0.
Configurability	Ability of the system to configure the behavior of the application with variables, such as the ability to configure the types of multifactor authentication.
Consistency	Ability of the system to produce consistent results in distributed environments without loss of information, such as being able to show comments in a social media post in the right order irrespective of the end user's geolocation.
Extensibility	Ability of the system to plug in new features, such as being able to add a new type of payment method to the application.
Installability	Ability of the system to be installed on supported platforms, such as OSs and browsers.
Interoperability	Ability of the system to interact with applications that operate on multiple technologies and platforms. For example, an employee management system that integrates with insurance systems, payroll management products, performance assessment systems, and so on.
Localization/internationalization	Ability to scale the application to different regions with a different user experience, if necessary, and language translations. For example, *amazon.de* is localized for German-speaking users. This CFR is also commonly referred to as l10n/i18n for the same reasons as a11y (see the note in Chapter 9).
Maintainability	Ability of the application to be easily maintained in the long run, with readable code, tests, etc. An example is creating meaningful method names.
Monitoring	Ability of the system to collect data about its activities and alert when predefined errors are encountered or when acceptable metrics go out of bounds. For instance, alerting when the server is down.
Observability	Ability of the system to analyze the information gathered by monitoring systems in order to debug and gain insights on application behavior, for example to understand how each feature is utilized during peak days, weeks, and so on.
Performance	Ability of the system to respond on time to the user's requests even at times of peak load. For example, ride availability should be presented to the users in *x* seconds, even under peak load.
Portability	Ability of the application to be shipped to new environments, such as integrating with new database types and cloud providers.

Privacy	Ability of the system to protect private and sensitive user data, such as encrypting credit card details while storing them in the database.
Recoverability	Ability of the system to recover from system outages, for example by having automatic data backup mechanisms.
Reliability	Ability of the system to tolerate errors and continuously maintain the services and data with precision. For example, applications usually incorporate retry mechanisms to handle network and other transient failures.
Reporting	Ability of the system to present meaningful reports to the business and end users based on the events collected. For example, Amazon lets users create order history reports.
Resilience	Ability of the system to handle errors and downtime. For example, load-balancing solutions may be put in place so that requests are sent only to servers that are online.
Reusability	Ability of the system to reuse application code and services as needed to implement new features; for example, reusing design components across multiple suites of enterprise applications.
Scalability	Ability of the system to handle expansion to new regions, more users, etc. For example, most cloud providers have options to enable an auto-scaling feature, which ensures that additional computational resources are added when there is a heavy load.
Security	Ability of the system to curb vulnerabilities and defend against potential attacks, using the tools and methods discussed in Chapter 7.
Supportability	Ability of the system to support new developers onboarding to teams and new users onboarding to the application code. An example is automating the code base and test suite setup steps.
Testability	Ability of the system to simulate different test cases and experiment with the application. For example, creating mocks for third-party services in order to simulate different test cases and test the integrations.
Usability	Ability of the system to provide a user experience that is intuitive, meaningful, and easy. For example, having a consistent navigation layout with a header panel.

This is not meant to be an exhaustive list; there may be others. Collectively, the CFRs, or *-ilities*, as they are sometimes called, define the *executional* and *evolutionary* qualities of the application. Executional qualities refer to the behavior of the application during runtime, such as availability, authentication, monitoring, and others. Evolutionary qualities, like maintainability, scalability, extensibility, etc., refer to the quality of the static application code. When executional qualities are not embedded in the application, the end users and the business will witness the impact firsthand. When evolutionary qualities are not addressed, software teams take the blow first, and this soon becomes an issue for the business. For instance, end users get frustrated when the system is unavailable, and when the code is unmaintainable team members get frustrated, leading to productivity loss. In order to avoid such frustrations, teams should establish a set of CFRs for the application right at the beginning of development and continuously test for these throughout the delivery cycle, just like functional requirements.

To lend a hand there, we'll discuss an overall CFR testing strategy now.

CFR Testing Strategy

Let's discuss the FURPS model, a model used for classifying all software requirements,[1] to begin with. We will be using this model to establish a high-level CFR testing strategy. FURPS stands for functionality, usability, reliability, performance, and supportability. The themes can be elaborated as follows:

Functionality

> This category of software requirements can be experienced as user flows in the application, such as the login flow, ride availability flow, and booking flow.

Usability

This category represents the set of requirements that affect the user experience, such as the visual quality, browser compatibility, accessibility, ease of use, and so on.

Reliability

These requirements contribute to making the application consistent, fault tolerant, and recoverable.

Performance

These requirements relate to the backend KPIs and frontend performance metrics, as discussed in Chapter 8.

Supportability

This category includes all the evolutionary code qualities, such as maintainability, testability, secure code, and so on.

The CFRs in Table 10-1 can be visualized along the same lines. For example, accessibility, as discussed in Chapter 9, is manifested through functional features, such as adding transcripts for videos, and also via the application design. So, the testing approach for accessibility will comprise methods and tools used for testing the functionality as well as usability. Similarly, one way to incorporate security is to add authentication-related functional features such as user login and to imbue security-related practices into the static code.

This section presents testing strategies for each of these five themes, as depicted in Figure 10-1. To formulate your project-specific CFR testing strategy, decompose the different aspects of the CFRs and adopt the appropriate methods and tools based on your project's priorities.

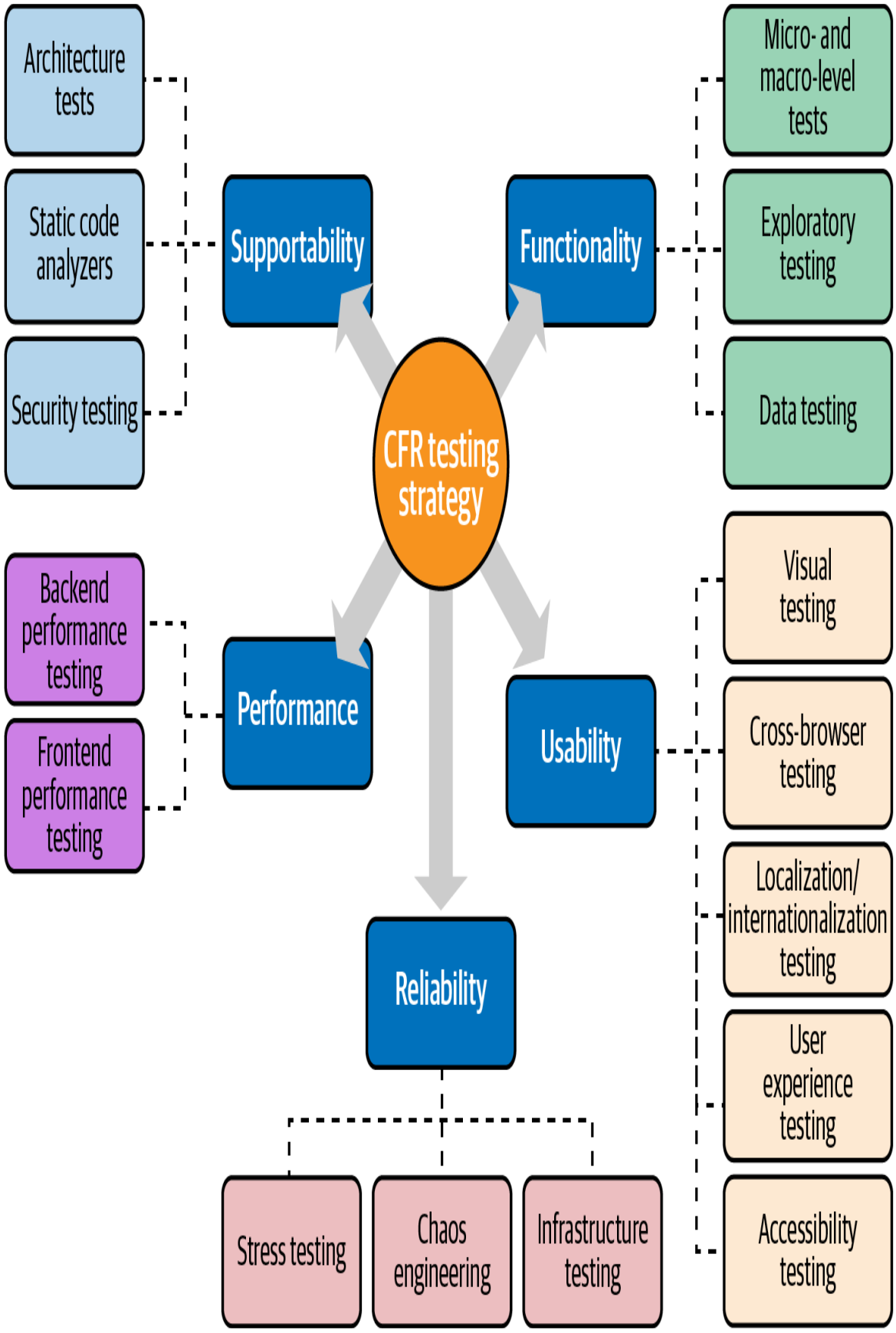

Architecture tests
Static code analyzers
Security testing
Supportability
Functionality
Micro- and macro-level tests
Exploratory testing
Data testing
CFR testing strategy
Backend performance testing
Frontend performance testing
Performance
Usability
Visual testing
Cross-browser testing
Localization/ internationalization testing
User experience testing
Accessibility testing
Reliability
Stress testing
Chaos engineering
Infrastructure testing

Figure 10-1. A CFR testing strategy, decomposed across the five themes

Functionality

To test the functional aspects of the CFRs, the manual exploratory and automated functional testing tools and methods discussed in Chapters 2 and 3 can be employed at different application layers. To reiterate, tools like Postman, Selenium WebDriver, REST Assured, and JUnit can be used to automate these functional aspects and get continuous feedback. Additionally, the data testing tools and methods discussed in Chapter 5 will be essential for testing them.

A special callout for testing compliance-related functional features, such as the strong customer authentication feature that is part of PSD2 or GDPR-related functional features, is that it is crucial to gather the right information on those regulations and to involve the legal team in both the testing and the requirements gathering phase. For quick reference, a section on regulatory requirements is included as part of the next section of the chapter (see "Compliance Testing").

Usability

In order to approach usability testing methodically, we can deconstruct usability into a few aspects, such as visual quality, cross-browser compatibility, localization/internationalization, user experience design, and accessibility. We discussed tools and approaches for testing visual quality and cross-browser compatibility in Chapter 6, and accessibility testing in Chapter 9. Let's discuss the testing approaches for the remaining aspects here:

Localization/internationalization testing

> Localization testing can be addressed in a few ways. If the UI skin varies for different locales, you should perform visual testing. If the skin doesn't vary but only the language and features like date and money formats change, you can rely on unit testing and

manual testing. For instance, you can add unit tests to compare the locales' string files for missing keys other than date and money format validations. However, when the language changes, sometimes the length of the text is affected, which can result in changes to the UI layout. In those cases, you can again adopt visual testing practices.

The manual testing of language-specific text needs to follow a certain sequence of steps to avoid duplication of efforts. The first step is to get meaningful text for all the elements, messages, etc., from a person who knows the language, followed by approval from the product owner or any business-approved representative. Next is to document the right text in each user story to enable development and manual story testing. Most often, when these steps are omitted the developers are left with no other option than to fill in the text using an online translation service, which results in double testing efforts before and after getting the approved strings.

It is important to bear in mind that putting off such localization testing until just before the release poses the risk of encountering a broken UI layout late in the delivery cycle as there is a possibility of the translated text not fitting the elements' layout, as mentioned previously.

NOTE

UI-driven functional tests should not be used to verify all the text in the application, as that would make them very slow. Use these tests only to verify the functional flows in the different locales, if they are different. In such cases, you can reuse the UI-driven functional tests by parameterizing the strings used in assertions and element identifiers, provided your overall test strategy adheres to the test pyramid.

User experience

The user experience encompasses all the design-related aspects of the application, such as how intuitive the user flows are, how many clicks it takes the user to get the information they need, whether the icons convey the right meaning, whether the application's color palette is to the end users' liking, and so on. Such aspects are researched during the beginning of the project and incorporated into the design. For instance, in a retail mobile project I worked on, we found that people from Italy preferred to see bright colors, like vibrant red, and designed the application with such a color palette.

As a general practice, you should include user experience aspects in manual exploratory testing for every user story. The Nielson and Norman group has done extensive research on user experience design and collated a list of 10 usability heuristics to follow, which can be incorporated into testing. Most often, the product owners and UX designers pair on such testing. There are also tools like UserZoom and Optimal Workshop that can be used to conduct UX tests on the design prototypes with real end users. I have seen such tests, when conducted periodically during the delivery cycle with different end user groups, result in significantly enhancing the design.

A/B testing is another way to get real-time feedback on the user experience in production. Though it's called testing, this is really more experimentation: it involves presenting different UX designs of the same feature as prototypes to two different end user groups in production and collecting data on the users' behaviors to enable the product team to decide on the final design. For example, a simple experiment would be to understand if a Sale button has the highest likelihood of being clicked if it's red or blue. In such an experiment, the red and blue buttons are presented to different user groups in production, and usage data

is collected and analyzed over a set period. This kind of experiment may require data science capabilities, and usually a team of product owners, data scientists, developers, and user experience designers work together on such experiments.

Reliability

From Table 10-1, the CFRs that contribute to the application's reliability are recoverability, resilience, auditability, archivability, reporting, monitoring, observability, and consistency. Many aspects of reliability, such as error handling, retry mechanisms, fallback mechanisms for single points of failure, measures to ensure consistency of data, and integrations with third-party tools for monitoring, observability, and reporting services, can be experienced as user flows. Functional testing approaches, as discussed in Chapters 2 and 3, can be deployed here. Apart from these, the other testing methods that contribute to reliability testing are the following:

Chaos Engineering

Chaos Engineering is a way to unearth inherent flaws in the application that might lead to system outages, failures, and other disasters, making the application unreliable. Usually, this method uncovers the unknown unknowns and is immensely helpful in large-scale systems. Chaos Engineering is discussed in detail in the next section of the chapter.

Infrastructure testing

Infrastructure is one of the many important parts of an application that contribute to reliability and recoverability. If it goes down, everything goes down. Additionally, the infrastructure layer has to be wired appropriately in order to support auto-scaling, alerting/monitoring, load balancing, and archiving capabilities. Although focused testing at the infrastructure layer hasn't become prevalent yet, it is gaining traction due to the increased

need for businesses to scale widely. We will discuss this topic in detail in the next section of the chapter as well.

Performance

We discussed the importance of performance and a selection of tools and metrics for both frontend and backend performance testing in Chapter 8. To reiterate, some of the key metrics are availability, response time, and concurrency, and tools that can help with performance testing include JMeter, WebPageTest, and Lighthouse. An additional point to note is that performance testing caters to fulfilling the scalability requirements by identifying the system breakdown threshold, and thereby contributes to enhancing the reliability of the application too.

Supportability

Supportability refers to all the evolutionary code qualities, such as compatibility, configurability, extensibility, installability, interoperability, portability, maintainability, reusability, security, and testability. Some of their functional manifestations, such as the configurability of functional features, compatibility with required protocols, installability in appropriate operating systems, interoperability features, etc., can be tested using the functional testing approaches discussed earlier in the book, with proper environment setup and stubs. Other approaches to test supportability include:

Architecture tests

Architecture tests are added to assert a set of architectural characteristics, such as verifying that the right classes are under the right packages (thereby ensuring reusability). These automated tests provide feedback to the team in the event of deviations from the essential architectural characteristics that were designed to cater to CFRs such as reusability, portability,

maintainability, and so on. We'll discuss some tools that can be used to write such tests in "Architecture Testing".

Static code analyzers

Many tools do static code analysis and provide useful feedback that serves to enhance maintainability. For example, Checkstyle ensures that the team sticks to a common coding style. PMD is a tool that reports issues such as unused variables, empty catch blocks, duplicate code, etc. It also allows the team to add custom rules specific to the project's standards. ESLint is a similar tool for checking JavaScript code for possible style and code errors, and SonarQube is a widely adopted tool that helps in assessing code coverage and scanning for vulnerabilities. In earlier chapters, we also discussed other static code analyzers that examine the code to ensure it is secure and accessible.

Using these methods and tools, you can shift your CFR testing to the left. As discussed in Chapter 4, these CFRs can be continuously tested along with functional tests as part of CI, enabling the team to get continuous feedback on all quality dimensions and thereby continuously deliver high-quality software to their customers!

Other CFR Testing Methods

To help you meet the goal of shifting CFR testing to the left and being able to do continuous delivery, several of the CFR testing methods introduced in the previous section, such as Chaos Engineering, architecture testing, and infrastructure testing, are discussed at greater length here. You can also read about a set of commonly implemented regulatory requirements toward the end of the section, which will support your compliance testing efforts.

> **NOTE**
>
> The title of this section emphasizes the fact that we have already discussed various CFR testing methods and tools, in Chapter 5 through Chapter 9.

Chaos Engineering

Application reliability is one of the critical CFRs, as any service outage results directly in a loss for the business. A Gartner study in 2014 estimated that the cost of downtime ranges from $140k–$540k per hour for some businesses, and I wouldn't be surprised if the cost is even higher in 2022. In recognition of the importance of reliability, established products in the market such as the Amazon Web Services strive to achieve an uptime of 99.999%, i.e., a cumulative downtime of just 5 minutes and 15 seconds per year.

Some of the factors that can lead to downtime are bugs in the application, single points of failure in the architecture, network issues, hardware failures, unexpected high loads of traffic, and issues with third-party services on which the application is dependent. Most of these factors are considered while designing the architecture, and teams do take relevant preventive measures during development too. For example, the exponential back-off method is widely adopted to handle service downtime: it prescribes that the request frequency to a down service be exponentially decreased so that it gets breathing time to recover quickly. Likewise, the blue/green deployment model is frequently implemented in order to avoid downtime during system updates; it works by having two identical production instances, where one is live and the other is used for upgrading, then switched to be the live instance. Apart from methods like these, teams handle downtime preemptively in several ways, such as having replicas to share high load, auto-scaling infrastructure, proper error handling of inputs, and so on. Yet despite all these efforts, large-scale distributed systems pose discrete

challenges to the reliability of the application in the form of convoluted workflows, multiple-layer dependencies, third-party failures, downstream systems errors, and so on, which cannot be easily foreseen and eventually lead to downtime.

Let’s consider a hypothetical example. A 50-member team worked on a large-scale distributed application and set up two instances separately to cater to customers in the US and UK. They configured each instance to redirect to the other when one went down and built functional capabilities into the application to handle requests from both regions. The team tested the functionality and the redirection flow. They also checked the performance of their application under load. However, when the UK instance went down due to technical issues at the same time as a peak sale was happening in the US and all the UK requests were redirected to the US instance, the application ended up throwing errors to all the users. The root cause was later spotted in one of the third-party downstream systems with a constraint on requests per hour, which started throwing errors when the rate limit was exceeded. Practically speaking, this is one of those edge cases that is hard to pinpoint. The team had done their due diligence, but in reality it’s hard for anyone to know all the nitty-gritty details in such large-scale distributed systems!

That hypothetical team was not alone in having this experience. Netflix, too, had troubling experiences when its service became cloud-native: the cloud instances faced unplanned outages due to various issues, resulting in losses and extended working hours for the engineers. They took that up as a challenge, deliberately mimicking the failures and solving the issues in their application one by one until it became entirely resilient to such unplanned outages. To achieve this goal they designed a tool called Chaos Monkey, which brought down one random instance of a cluster every day during working hours, with the engineers implementing safety measures appropriately. This approach ensured every one of their system’s inherent unpredicted flaws were addressed, ultimately

making it both resilient and reliable. Based on this success, they further evolved and crystallized the practice, calling it *Chaos Engineering*.

A formal definition from the book *Chaos Engineering* by Nora Jones and Casey Rosenthal (O'Reilly) is as follows:

> *Chaos Engineering is the discipline of experimenting on a distributed system in order to build confidence in the system's capability to withstand turbulent conditions in production.*

In other words, it involves conducting experiments on the application, simulating errors, outages, and other unexpected scenarios, and observing the application's behavior. This practice is becoming widely adopted in the software industry, and many companies have evolved it further to suit their needs. From the collective learnings of the industry, the fundamental characteristics of Chaos Engineering can be described as follows:

- It's more about experimentation than testing—i.e., it doesn't involve verifying the expected behaviors of the system when faced with unknown issues, but rather observing the behavior of the system in unexpected situations and gaining insights.
- The purpose of experimenting is to gain confidence in the system's reliability and resilience. You can choose *not* to experiment when you have enough confidence in your system's ability to handle unknown turbulence.
- It's particularly beneficial when you are developing a large-scale distributed system.
- Chaos Engineering is not the responsibility of one particular role, such as the DevOps engineers or testers. It is a team activity where all stakeholders work together to design the experiment, conduct it, and debug the behavior.

Perhaps if they had conducted such chaos experiments, that 50-member team would have spotted the rate limiting issue earlier and avoided the disastrous outage!

Chaos experiment

If you're planning to orchestrate chaos experiments, the Netflix team recommends conducting them directly in production, as the real-life variables are extremely tough to simulate in a test environment. They also recommend designing capabilities to pause the experiments and revert the system back to normal. There are several tools today that assist in scripting chaos experiments, such as the Chaos Toolkit and ChaosBlade, so you don't have to manually bring things down and up in production.

To perform a chaos experiment, first develop a hypothesis that might challenge the reliability of the application, along with a cross-functional team. Next, define a steady-state hypothesis, which is the predicted application behavior during the experiment. Script the experiment using your tool of choice, and run it in production. If the tool alerts you that the experiment has failed—i.e., the steady-state hypothesis has not been met—your cross-functional team can jump into action.

To give you a glimpse of how one of the chaos experiment tools work, Example 10-1 shows a simple experiment setup using the Chaos Toolkit, an open source tool written in Python. The experiment is scripted to simulate a technical issue (here, by deleting a configuration file in the current instance of the application) and to check whether an alternative instance of the application is still up and running.

Example 10-1. Chaos experiment to simulate a technical issue and observe the application behavior

```
{
    "version": "1.0.0",
    "title": "Application should still be up if there are
technical issues",
```

```
    "description": "When a particular config file is missing,
application should
                    still be up from another instance",
    "contributions": {
        "reliability": "high",
        "availability": "high"
    },
    "steady-state-hypothesis": {
        "title": "Application is up and running",
        "probes": [
            {
                "type": "probe",
                "name": "homepage-must-respond-ok",
                "tolerance": 200,
                "provider": {
                    "type": "http",
                    "timeout": 2,
                    "url":"https://www.example.com/"
                }
            }
        ]
    },
    "method": [
        {
            "type": "action",
            "name": "file-be-gone",
            "provider": {
                "type": "python",
                "module": "os",
                "func": "remove",
                "arguments": {
                    "path": "/path/config-file"
                }
            },
            "pauses": {
                "after": 1
            }
        }
    ]
}
```

The script begins by describing the intent of the experiment and tagging it as a test case contributing to high reliability and availability. It then elaborates the steady-state hypothesis and the method to

trigger the technical issue. The method uses the tool's capability to delete a file from a given path and pauses for 1 second before the tool attempts to verify the steady-state hypothesis, using one of the tool's probes to hit the application URL and checking whether it returns a 200 status code within 2 seconds. This experiment can be run from the command line and observed.

Suppose we run the experiment and it fails. On further investigation, we find that the application takes 4 seconds to respond instead of the expected 2 seconds, due to hurdles in rerouting to the alternative instance. That will be a valuable insight gained from this simple chaos experiment.

The Chaos Toolkit provides many other APIs to conduct varied types of experiments, and it's easy to configure them as JSON files like the one shown here. It can generate HTML reports at the end of the experiments too.

Architecture Testing

At the beginning of any project, a list of functional and cross-functional requirements is mulled over and drawn up, and a conducive architecture design is laid out. For example, let's say that in order to ensure maintainability and reusability, the architecture design proposes that the application have separate layers. Also, since performance is a priority, caching mechanisms are placed in the right layers. But there's a universal law that may disrupt these well-thought-through design decisions, known as Conway's law. In his paper "How Do Committees Invent?" Melvin Conway states that the team structures, particularly the communication paths between people, inevitably influence the final product design. The individual teams working on smaller portions of a large system will inadvertently optimize their respective portions without factoring in the big-picture needs. For example, a team may choose to prioritize performance over reusability and bypass layers, resulting in a need

for reworking later. This is where architecture tests, when rightly placed to guard the essential architectural characteristics, come in useful: they provide feedback to teams as and when they deviate from the big picture.

Tools like ArchUnit for Java and NetArchTest for .NET, among others, can be used to write such tests. For example, you can introduce architecture tests to check for cyclic dependencies so that maintainability is continuously preserved, or to make sure the packages are independent so that they are reusable. ArchUnit tests are similar to JUnit tests and can be run as part of a CI pipeline. Example 10-2 shows an ArchUnit test to assert that all the classes in the order management service reside in the `oms` package, to ensure reusability. So, whenever there is a need to include a class outside the responsibilities of the order management service, the team will be pushed to discuss the big picture and decide where it fits in appropriately.

Example 10-2. An ArchUnit test to assert reusability

```
@Test
public void order_classes_must_reside_in_oms_package() {

classes().that().haveNameMatching("*order*").should().residеI
nAPackage("..oms..")
                .as("order classes should reside in the
package '..oms..'")
                .check(classes);
}
```

Similarly, JDepend is a tool that produces metrics on design quality in terms of extensibility, maintainability, and reusability. It does a static code analysis on all the Java classes and gives different design scores for a given Java package. (NDepend is a parallel tool in the .NET world.) JDepend uses the number of abstract classes and interfaces in a package as a measure of its extensibility, checks for dependencies on external packages and raises alerts when there are unwanted dependencies, and checks for package cyclic

dependencies. JDepend tests can be written as JUnit tests and integrated with CI for continuous feedback on architecture quality.

Example 10-3 shows a JDepend test that checks if packages A and B depend on each other, introducing cyclic dependencies and thereby hampering reusability.

Example 10-3. A JDepend test to assert on package cyclic dependencies

```
import java.io.*;
import java.util.*;
import junit.framework.*;

public class PackageDependencyCycleTest extends TestCase {
    private JDepend jdepend;

    protected void setUp() throws IOException {
        jdepend = new JDepend();
        jdepend.addDirectory("/path/to/project/A/classes");
        jdepend.addDirectory("/path/to/project/B/classes");
    }

    public void testAllPackages() {
        Collection packages = jdepend.analyze();
        assertEquals("Cycles exist",
                     false, jdepend.containsCycles());
    }
}
```

Along the same lines, tests can be written to verify that a package has only the expected dependencies or has no dependencies at all. This way, the teams can get feedback continuously whenever there is an undesired change to the critical architecture characteristics.

Infrastructure Testing

The term *infrastructure* at a high level refers to the computational resources (e.g., machines, VMs, containers), network structures (e.g., VPNs, DNS entries, proxies, gateways), and storage resources (AWS S3, SQL Server, secrets management systems, etc.)

necessary to support the smooth functioning of the application. Infrastructure testing involves testing the setup and configuration of these resources. This is an emerging area in testing.

The need for infrastructure testing mainly arises from the growing demand to scale applications. This demand is mainly because businesses, once successful, want to quickly expand their online services to newer regions and start serving larger numbers of customers. To enable such quick scaling, they should have the ability to replicate the existing end-to-end application stack, including the infrastructure setup, within a short time—if possible automatically, with a single click. Although most software teams have an established automated process to test, bundle, and deploy the application to any environment with one single click, they don't always have the same capability on the infrastructure side, which introduces a gap in their ability to scale rapidly. This is where the practice of *Infrastructure as Code* (IaC) becomes very useful.

The term IaC refers to the practice of designing the infrastructure setup and configuration as reusable code, just like application code, to power continuous delivery and scalability. For instance, code is written to spin up a cloud instance with 3 GB of memory using the cloud provider's APIs, set up application-specific load balancer rules, and set up a firewall. These features need to be tested so that the same code can be used to spin up new infrastructure instances whenever there is high load or to enable expansion to new regions.

Terraform by HashiCorp is a widely adopted open source tool for scripting infrastructure code using a declarative coding style. It enables the code to work across multiple cloud providers. Here are some things to keep in mind when testing infrastructure code written using Terraform in various stages of the path to production:

- Terraform provides the `terraform validate` command to check for syntax errors in the Terraform code, which can be applied as early as in the development stage.

- TFLint, a linting plug-in for Terraform, can help in analyzing the static infrastructure code for deprecated syntax, deviation from best practices such as naming conventions, etc. TFLint can also check if the specified image types are offered by popular cloud providers such as AWS, Azure, etc.
- During the incremental development process, Terraform can compare the latest code changes against the existing environment state and present a preview of the changes as a safety measure before execution. For example, if the code changes result in the unintentional deletion of the database, the preview feature will save the day! The command for this is `terraform plan`. You can also write automated tests against the output of this command to verify certain aspects, like security policy compliance.
- The next step is to deploy the infrastructure code to create actual cloud instances and verify whether the instances have the intended infrastructure resources—for example, if the instance is running within the private subnet and has the required disk space. These test cases can be automated using tools like Terratest, AWSSpec, Inspec, and Kitchen-Terraform, and added to CI.
- Then comes the end-to-end testing of the infrastructure components. We need to check if the components can interact with each other in the expected way—for instance, if the web server can make a call to the application services. This testing, in a way, gets covered when the deployment of the application code is successful and the functional tests run smoothly. But it's also possible to catch such issues before deploying the application by writing infrastructure tests using a combination of the tools mentioned earlier.

Kief Morris, author of the book *Infrastructure as Code* (O'Reilly), suggests that the distribution of the infrastructure tests in the various

layers may form a diamond pattern instead of a pyramid. This is because unit tests for low-level declarative code, such as that in Terraform, may not be of much use and are recommended to be kept to a minimum. So, depending on the nature of the infrastructure code, we should choose to add relevant tests in the appropriate layers.

Apart from the functional end-to-end testing, the other aspects to be tested when it comes to infrastructure are:

Scalability

> We should test that instances auto-scale based on load and verify that the application features work smoothly after scaling.

Security

> Infrastructure security is a critical aspect to test. Tools like Snyk IaC check for potential vulnerabilities in infrastructure code during development. Some of the security test cases, like checking for unexpected open ports, appropriate public- and private-facing instances, and so on, can be tested manually and by writing automated infrastructure tests.

Compliance

> Sometimes, the infrastructure code needs to adhere to policies and compliance features. For instance, to be PCI DSS–compliant (PCI DSS is discussed as part of the next section), appropriate firewalls should be set up. To check for compliance rules, HashiCorp offers Sentinel for enterprises.
>
> An open source tool to check compliance features in Terraform is terraform-compliance. The tool is based on Python and provides a behavior-driven development layer just like Cucumber to write tests. The tool runs the tests against the output of the `terraform plan` command instead of the actual instance.

Operability

> All other operational features, such as log archiving for auditability, monitoring tool integration, automated maintenance features, and so on, need to be tested as well.

Depending on the complexity and nature of the infrastructure code, you can choose to write automated tests for these cases and integrate them with CI. Many of the tools for automated infrastructure testing are still evolving and require coding skills beyond just one language. For instance, Terratest uses GoLang, `terraform-compliance` uses Python, and AWSSpec uses Ruby. Additionally, many automated testing tools may require real infrastructure to be up and running, incurring costs. Given these constraints, you can craft an infrastructure testing strategy specific to your application's needs.

Compliance Testing

Two commonly implemented regulations on the web are the GDPR and WCAG 2.0. We discussed WCAG 2.0 at length in Chapter 9. Here, we will briefly explore the GDPR, then take a quick look at some payment-related regulations that you should be aware of.

> **NOTE**
>
> This section intends only to serve as a brief introduction to these regulatory requirements. Software teams are recommended to engage with legal advisors to get the details specific to their application and domain.

General Data Protection Regulation (GDPR)

The GDPR primarily aims to protect the private data of EU citizens. If you aim to sell goods to EU citizens, then your website will be subject to GDPR compliance. Similarly, if a school in the US allows admissions to EU citizens via its website, then it has to abide by GDPR requirements. Noncompliance may result in heavy penalties, as high as 4% of the company's annual revenue.

> **NOTE**
>
> Different countries also have their own data protection and privacy laws. As of April 2022, 71% of countries across the globe reportedly had proper legislation in place for data protection and privacy. For example, Canada has the Consumer Privacy Protection Act (CPPA) and the UK has its own version of the GDPR (post-Brexit).

According to the GDPR, *private data* is any information that, on its own or combined with other information, can be used to identify a living individual. The individual's racial or ethnic origin, religious or philosophical beliefs, political opinions, sexual orientation, genetic data, biometric data, past or present criminal convictions, etc., are classified as *sensitive personal data*, which needs to be carefully protected. Even many online identifiers like IP addresses, MAC addresses, mobile device IDs, cookies, user account IDs, and other system-generated data that can be used to identify a living individual come under GDPR protection. In order to protect the data, GDPR recommends that development teams implement Privacy by Design principles (the Privacy by Design framework, developed by Dr. Ann Cavoukian, lays out seven foundational principles that focus on preventing any privacy-invasive events).

Some of the technical measures that you can implement are the protection of data at rest using dynamic salts and hashing techniques, encryption of data in transit, adhering to the principle of least privilege, pseudonymization, anonymization of data, and other general data security measures discussed as part of Chapter 7.

The GDPR also protects users' rights to control their data in various ways, like the following:

Right to be informed

> You need to let the application's users know how their personal data is used. This is typically handled through the site's privacy policy.

Right of access

> Users have the right to request their stored personal records.

Right to be forgotten

> Users can request that the site owners delete their personal data when there is no compelling reason for its continued processing.

Right to restrict processing

> Users can prohibit the processing of their personal data. The website can still store the data but no longer process it.

Right to rectification

> Users can correct incomplete or inaccurate information on the website.

Right to portability

> Users can obtain and reuse their personal data.

Right to object

> Users can object to their personal information being used for marketing, research, and statistics.

Rights related to automatic decision making

> Users must be asked for their consent to use their profiles for automated decision making, like profiling.

Most of these requirements can be tested using functional testing approaches. For instance, you can add automated micro- and macro-level tests to assert that there is no implicit opt-in, verify that personal data is stored only after obtaining the user's consent, and check that personal information is not stored in application logs. The security testing concepts and mindset discussed in Chapter 7 will fit right in here.

PCI DSS and PSD2

If your application deals with credit card payments (which most retail websites do) or provides payment services to the EU region, there are two regulations that will come into play:

Payment Card Industry Data Security Standard (PCI DSS)

> PCI DSS is a global standard defined by the PCI Security Standards Council to protect online card transactions. It applies to any entity that stores, processes, or transmits cardholder data. This means, in general, that it applies to all sites that take credit card details—even donation sites. PCI DSS is not a legal requirement, but a mandatory standard expected by banks and merchants for card transactions. There are fines for noncompliance, as per the respective contracts between the company and the payment processor. Companies can usually validate their compliance through a self-assessment questionnaire.
>
> PCI DSS provides 12 guidelines to make credit card transactions secure in an application, such as encrypting the transmission, having a firewall, updating anti-virus software, etc. So, when you are testing, you should think of scenarios to protect the card details, such as masking the card details in the UI and in all storage locations, implementing restrictions on accessing the card data, avoiding storing card details in logs, and so on. A threat modeling exercise, as discussed in Chapter 7, will come in handy here.

Payment Services Directive (PSD2)

PSD was the first implemented payment services directive in the EU region, which aimed to prevent online payment crimes. It also intended to increase competition in the payment industry, to prevent banks from monopolizing payment services. PSD2 is an overhaul of the original PSD standard. Compliance is mandated by law in the EU region for all payment services providers. If you are building an application that provides payment services to customers in the EU region, you should pay attention to PSD2 regulations.

PSD2 mainly focuses on strong customer authentication (SCA) features and on extending the reach of PSD2 within and beyond the EU region—for example, it calls for compliance if even one leg of the transaction involves an EU member state. To be PSD2-compliant, options for businesses are to choose an already compliant payment services provider like Stripe or PayPal, or to build the SCA features for payment services into their applications. In simple terms, SCA can be equated to multifactor authentication. The European Commission defines SCA as an authentication mechanism that uses at least two of the following three verification elements:

- The user's unique knowledge of something, like a password
- The user's unique possession of something, like a debit or credit card or mobile device
- The user's unique biometric identifiers, such as their face, voice, or fingerprints

Such features have to be tested thoroughly to ensure they are PSD2-compliant.

To summarize, the first step in compliance testing is to develop a thorough understanding of the legislation. Then, the respective CFR testing approaches as discussed in the strategy section (covering the five themes) can be employed appropriately to holistically test them. Once the application is tested and ready, the legal team or an authorized entity will get involved for compliance certification. Only on successful certification can the cycle of compliance testing be considered complete.

And with that, you are equipped to test the long list of CFRs that your application might require to thrive and to enable your team to do continuous delivery by shifting CFR testing to the left.

Perspectives: Evolvability and the Test of Time!

We have discussed how to harness quality by testing the application's functional and cross-functional requirements. However, it is important at this juncture to understand that software requirements are not set in stone at the beginning of the project. As established previously, software requirements change continuously along with market needs; such change is inevitable. Also inevitable is that the new requirements almost always threaten the existing implementation when not shepherded wisely. For instance, a team member may hastily override encryption in order to improve performance and thereby entirely compromise the security of the application.

This is the core premise of the book *Building Evolutionary Architectures* by Neal Ford, Rebecca Parsons, and Patrick Kua (O'Reilly), in which the authors prescribe a new CFR: *evolvability*, which is the ability of the system to preserve the existing architecture characteristics (e.g., layered architecture, data persistence methods, encryption at rest and transit) that facilitate a given set of functional and cross-functional requirements, while incorporating new changes. In order to achieve evolvability, they recommend the implantation of

appropriate guard rails for the essential architectural characteristics that may be non-tradeable, which will provide the teams with instantaneous feedback whenever they deviate. These guard rails can be in the form of automated tests around each of the functional and cross-functional requirements (such as performance tests, security scans, accessibility audit results, architecture tests, and micro- and macro-level functional tests) as well as code coverage metrics, static code analyzer metrics and so on. This ensemble of tests and metrics, collectively referred to as *fitness functions*, will guide the teams in making incremental changes without compromising the existing implementation, and in the process create an evolutionary architecture.

To consolidate the views presented here, all the functional and cross-functional testing methods and tools that we have been learning all throughout the book, including what is bundled in this chapter, collectively aid in building an evolutionary architecture that stands the test of time, in addition to imbuing high quality today!

Key Takeaways

Here are the key takeaways from this chapter:

- Cross-functional requirements, commonly called non-functional requirements, are as essential as functional requirements for the application's success. Functional and cross-functional requirements together make the application a high-quality product.
- CFRs mainly define the executional and evolutionary qualities of the application.
- CFRs apply to a wide breadth of the application's features and hence have to be developed and tested as part of each user story. Having a CFRs checklist as part of every user story can be a way to ensure the completion of CFR testing.

- The FURPS model abstracts themes out of all the software requirements. The CFRs can be seen to manifest themselves along those themes.
- The chapter provides a testing strategy for each of the five themes in the FURPS model, which can be used to put together a project-specific CFR testing strategy. This testing strategy should pay attention to every CFR individually, based on the project's needs.
- Shift your CFR testing to the left by automating these tests and integrating them with CI.
- Chaos Engineering is an experimentation method to unveil the inherent flaws in the application that might make it unreliable. The experiments should be iteratively conducted together as a team.
- Tools like ArchUnit and JDepend help to assert the architectural characteristics of the application in order to sustain some of the evolutionary code qualities.
- Infrastructure testing is an emerging area in testing. It is necessary in cases where you need to scale the application quickly. Automated infrastructure testing tools are still evolving and can incur additional costs in terms of knowledge ramp-up and actual infrastructure test deployments.
- The GDPR and WCAG 2.0 are commonly implemented regulations in web applications. In order to test for compliance, we may be required to know these regulations thoroughly, essentially by learning from a legal team.
- The functional and cross-functional tests become fitness functions and not only help teams in delivering high-quality software today, but also assist in creating evolutionary architectures that will stand the test of time.

1 Developed at Hewlett-Packard and originally described by Robert Grady in his book *Practical Software Metrics for Project Management and Process Improvement* (Prentice-Hall).

Chapter 11. Mobile Testing

Imagine a day without your mobile!

Ever since the advent of smartphones, mobile devices have become like an additional limb to many of us. They have brought sophistication to our lives by delivering everyday services to our doorsteps with a touch and a swipe. I can't think of a single other object that serves the multitude of purposes that a smartphone does. We use them to shop for groceries, outfits, home appliances, and other basic necessities. We use them to read books, watch movies, and play games to entertain ourselves. Smartphones facilitate banking, paying bills, and organizing our calendars. Even more, they bring us a sense of psychological safety as we know that help is only a call away!

With all of these benefits and uses, it's no wonder there are 6.6 billion smartphone users across the globe. What may be surprising, though, is that there are well over 8 billion mobile subscriptions—significantly more than there are people on the planet! The scale of adoption is staggering, but studies correlate these numbers with extensive usage too. For example, one recent study found that Americans, on average, check their phones 344 times a day, or every 4 minutes. Similarly, an average smartphone user across the globe reportedly accesses 10 apps per day and 30 apps every month. And this extensive smartphone usage is not confined to any specific age group: 18- to 24-year-olds are estimated to spend 93.5 hours each month on smartphones versus 62.7 hours for 45- to 54-year-olds and 42.1 hours for people in the 65+ age bracket (or roughly 3 hours, 2 hours, and 1.5 hours a day, respectively). Such is the impact of smartphones on all of us.

With all of this usage, it should come as no surprise that as of 2021 there were 5.7 million apps available in the leading app stores,

Google Play and Apple's App Store—and the number of mobile apps is only going to continue to grow in the years to come, as they are proving to be lucrative for businesses. In 2020 alone, mobile apps generated revenue of over $318 billion worldwide, and that is projected to rise to more than $613 billion by 2025.

Why are all of these numbers important? Because we, as software developers and testers, are going to be developing and testing these mobile apps, and it is a clear call for action to hone our mobile skills. The aim of this chapter is to give insights into the mobile testing mindset and tools. If you're wondering how mobile testing is different from web testing, this chapter will answer that question. It introduces the overall mobile landscape and the specifics of mobile versus web testing. You will learn a strategy to fully test the mobile layer, including automated functional testing as well as performance, security, accessibility, visual, and CFR testing. Additionally, this chapter has guided exercises to get you up to speed so that you're mobile project–ready!

Building Blocks

To begin with, let's take a look at the overall mobile landscape, the challenges it throws at us, and the specifics that require our attention while testing mobile apps.

Introduction to the Mobile Landscape

As visualized in Figure 11-1, there are three main areas to consider when approaching the mobile landscape: the devices, the apps themselves, and the network. Let's take a look at each in turn.



Figure 11-1. The mobile landscape

Devices

As part of their evolution, mobile devices have come to vary in several dimensions from one another. It is critical to understand these dimensions in order to decide which devices to use for testing. As a general rule, you should aim to provide testing coverage for at least 85% of the target devices. Here is a list of distinct device dimensions you should factor in when deciding on your testing strategy:

Screen size

Mobile devices include tablets as well as smartphones. There are well over a billion tablet users worldwide, and that is not a tiny number to discard! With all the different form factors and device models, it should come as no surprise that the screen sizes of tablets and smartphones vary significantly. What's more, the screen size differs within the same device depending on its orientation (that is, whether it is held in landscape or portrait viewing mode), and modern handsets enable viewing multiple apps in a split-screen fashion, further subdividing the available screen space.

Screen size has a major impact on the overall end user experience, which makes designing, developing, and testing for different screen sizes critical in the mobile landscape. For example, on smaller screens the user might have to scroll to view the entire page, whereas on larger screens there may be a lot of empty space—neither of which may be a welcoming user experience.

Pixel density

A pixel is a small square area on the screen carrying a piece of the information that is displayed, and pixel density is the number of distinct pixels that can fit into a square inch of the screen. The greater the density, the better the viewing experience. Not only do mobile devices differ in their screen sizes, but devices with the same screen size can have different pixel densities. Based on

their pixel densities, the devices are categorized as low, medium, high, extra high, extra-extra high, or extra-extra-extra high density. You[1] This dimension of mobile devices particularly affects image rendering, as images automatically get resized on certain devices to accommodate the screen size, resulting in blurring or distortion. Hence, images must be specifically designed and programmed for respective pixel densities, and this must be included in testing.

> **NOTE**
>
> A screen's *resolution* is an indication of the number of pixels that it can display horizontally and vertically. For example, a screen with a resolution of 1,024 x 768 can display 1,024 pixels horizontally and 768 vertically when the device is in landscape orientation.

Operating system

Just as the desktop world has Windows, macOS, Linux, and other operating systems, the mobile world has Android, iOS, Windows Mobile, Symbian, KaiOS, and so on. As you can guess, Android and iOS take the top two places, accounting for ~99% of mobile OS usage across the globe. But it doesn't stop there! There are many versions of each OS that are still officially supported and used by many with older phones. This is referred to as *fragmentation*. For example, as of 2020, Android 6.0 was still the second most widely used Android version, despite being released in 2015, while Android 9.0 took first place. So, your testing scope should include different OS versions, too, as some versions may not support certain features or might handle them differently.

Hardware

The hardware configurations of mobile devices—RAM, CPU, battery capacity, local storage capacity, etc.—are another dimension that varies from model to model. Hardware impacts app performance in terms of parallel processing, swiftness in rendering the app, and the overall user experience of the app. Especially when your app depends upon built-in device capabilities such as the GPS, camera, microphone, touchscreen, and other hardware sensors, the end user's experience will vary based on the hardware's inherent capabilities.

This dimension is significant as it may even impact the core functionality provided by the app. For instance, a mobile app designed to gather survivor information during disasters such as tsunamis or cyclones cannot rely on volunteers possessing a high-end camera and must take care not to draw too much battery power. Depending on its intended usage, your app may need to be designed, developed, and tested with stringent hardware conditions in mind.

Device manufacturer

There are several device manufacturers in the market these days, such as Oppo, Samsung, Xiaomi, LG, Motorola, Google, Apple, and so on. Some of these manufacturers have their own custom Android versions, such as the Cyanogen OS, Oxygen OS, and Hydrogen OS. Also, each device manufacturer follows their own hardware design, such as providing a central hardware home button or back button. These nuances must be accounted for while developing and testing mobile apps.

Clearly, as you can see, considerations related to the devices themselves pose a heap of challenges for software teams to manage. Next, let's look at the problem through the app lens.

App

A key specialty of mobile applications is the varied suite of interactions they enable. In addition to the standard click and type supported by web applications, you can swipe, touch, long press, zoom in and out, pinch and zoom, press and drag, rotate, and more in a mobile app! These gestures and interactions are a large part of what makes mobile usage more attractive and personalized. A subset of these interactions may be made commonly available across the app, such as swiping left to right on any page to display the menu, or swiping upwards from the bottom of the screen to bring up additional features of the current functionality. Such common interactions become cross-functional requirements across the app, which need to be designed, built, and tested as part of every user story. The possibility of enhanced interactions, however, gets constrained based on the app type. While as end users we might have never considered these distinctions between mobile apps, as software teams we need to know the different app types so that we can approach testing accordingly. Currently, the following four types of mobile apps get adopted frequently:

Native

> Native apps are developed mainly to work on a single mobile OS, such as Android or iOS. The benefits of choosing to develop native apps are that they can provide excellent performance, offer access to the device hardware and all the OS features and APIs (including gestures), can work offline, and have a consistent and harmonious look and feel. Android native apps are typically written in Java or Kotlin, and iOS apps are developed with Objective C or Swift. They are distributed via Google Play and the Apple App Store, respectively. Each of these distribution platforms has compliance guidelines and approval processes after app submission, so there may be a delay before the app is made available to the public. While that may not be problematic, even urgent bug fixes go through the approval process, delaying their release! Another significant downside is the development

cost, as a separate native app has to be developed for each of the target operating systems.

Mobile web

Mobile web apps are websites that are accessed using mobile web browsers. They have the advantages of being OS-independent, free of installation procedures, and not requiring local storage space for installation. They also are not dependent on app stores for approval or distribution. Furthermore, these apps can be developed with the usual web development technologies (such as HTML5 and CSS), so there's no need to learn mobile OS–specific languages. The disadvantages are that they don't have access to OS features such as the phone book, camera, etc., and they can't work offline. As a result, the user experience is very limited.

Hybrid

Hybrid apps bring the best from both the native and web worlds. A hybrid app is developed using standard web development technologies such as HTML, JavaScript, or CSS and is then wrapped in a native container that provides access to OS-related APIs. React Native, Ionic, Apache Cordova, and Flutter are some popular hybrid app development frameworks that enable this, even allowing the same app to work across multiple OSs. Hybrid apps need to be submitted to an app store for distribution, but the web elements of the app can be hosted on a server and fetched over the network. As a result, updating these parts of the app can be done easily without going through the store's approval process. However, this will constrain offline viewing of the app, so teams usually store a minimum set of selected content locally on the device to enable offline viewing. Overall, the hybrid approach eases development and reduces development costs. The trade-off, though, is performance, where native apps fare better. Also, since these apps are commonly built to work across different

OSs, there may be an unintended side effect of alienating some end users, who are accustomed to using their apps in a certain way in their preferred OS.

Progressive web

Progressive web apps (PWAs) are advanced versions of mobile web apps. Users can install them on their devices via a URL, and they take up very little storage space. Even though they are web apps, they can provide push notifications, can work offline, and have access to OS features, making the experience similar to a native app. The performance provided by PWAs is also on par with native apps, and since they are web apps they can work across OSs and browsers. What's more, all these capabilities can be achieved with cheaper development costs compared to native and hybrid apps. Given these benefits, PWAs have become the go-to choice for businesses these days. Twitter replaced its mobile web app with a progressive web app in 2017 and has seen a 20% decrease in bounce rate, 75% increase in tweets sent, and 65% increase in pages viewed per session!

As you will have gathered, the type of app you choose to build will dictate the testing scope in terms of testing for offline versus online behavior, OS-specific feature support, app update behavior, interactions, and so on. With that, let's get to the last major consideration: the network.

Network

People across the planet do not have equal access to high-speed networks. While low bandwidth is often a problem in remote locations, even within urban areas network connectivity is not consistent. So, when your app depends on network connectivity, you need to support different mobile network types (such as 2G, 3G, and 4G), apart from WiFi and being completely offline. You will have to test how your app handles scenarios such as network timeouts, error

displays on network oscillations (e.g., switching between 4G and 3G), offline activity, launch performance with various network types, and so on. You might even have to design the app with such constraints in mind to begin with. For example, Facebook released the Facebook Lite app mainly to address network bandwidth issues. It can work with 2G and generally provide seamless operation in unstable network connections.

Looking at the mobile landscape through these three lenses should have given you some idea of the additional complexities that need to be dealt with in mobile testing. To give you a better understanding of the mobile testing scope, next we'll dig a bit deeper into the architecture of a mobile app.

Mobile App Architecture

We discussed the architecture of a typical web application in Chapter 2. As you'll recall, it has a web UI that receives the user requests and services that process the requests by collaborating with the DB layer. A mobile app's architecture is not very different. As shown in Figure 11-2, the mobile UI replaces the web UI layer, and the rest tends to be more or less the same.

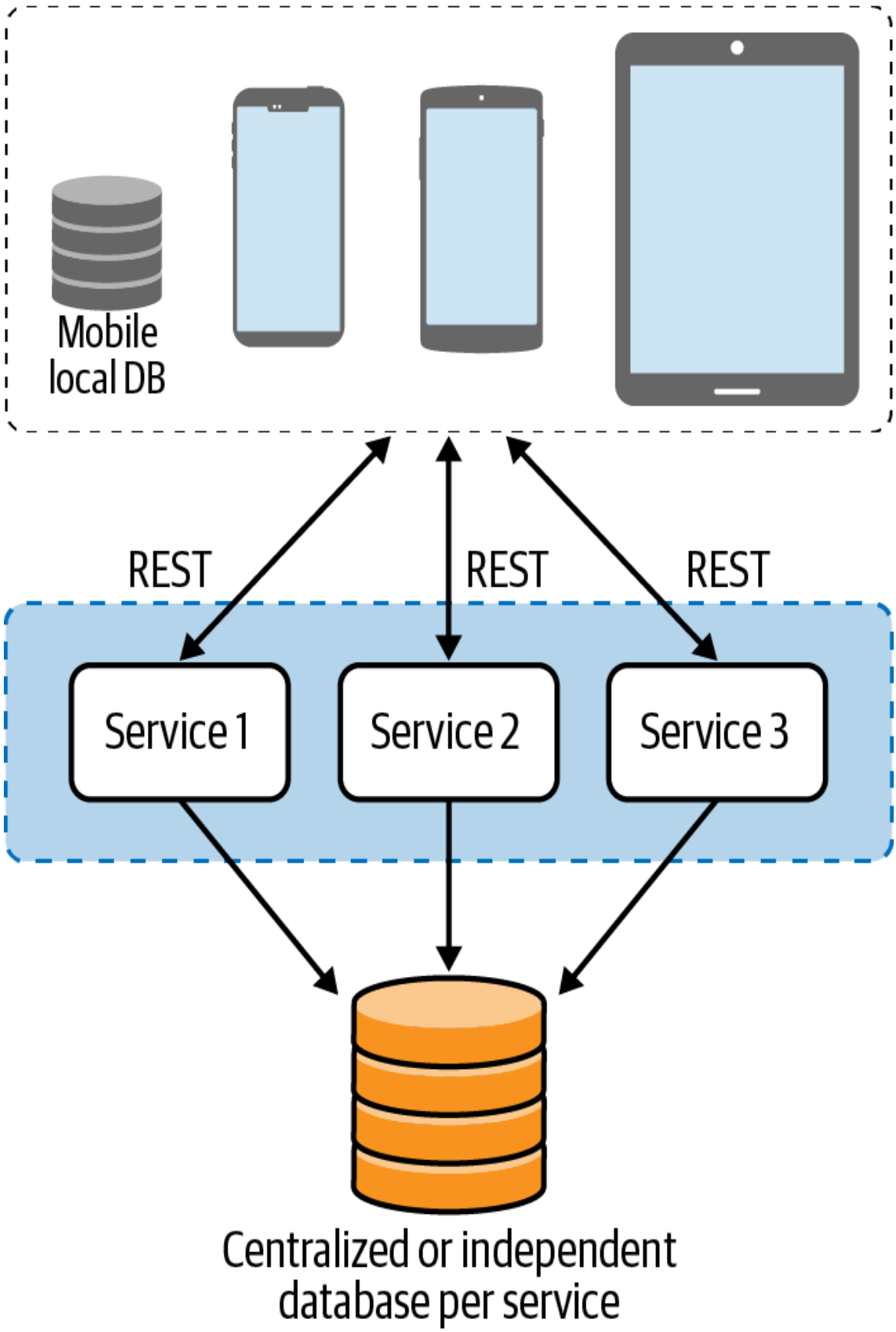

Mobile
local DB
REST
REST
REST
Service 1
Service 2
Service 3
Centralized or independent
database per service

Figure 11-2. Mobile application architecture

Note the extra component, the local DB, in the mobile layer. This is where native and hybrid apps store selected data, such as the username, profile picture, last fetched content, etc., to support offline behavior and accelerate app rendering time. The rest of the flow is similar to that of a web app, where the mobile app calls services over the network and completes users' requests. So, the testing approach to the services and the database layer remain the same: you do micro- and macro-level testing and write unit, integration, and API tests. You'll also test for services' performance, security, legal compliance, and other CFRs. Added to this, you should perform mobile UI–specific testing, paying attention to its inherent complexities. These specific focus areas in mobile UI testing are what we will discuss next.

> **NOTE**
>
> To get a deeper understanding of the interior components of a mobile UI layer, explore the Android architecture guide by Google.

Mobile Testing Strategy

The very first item to consider in your mobile testing strategy is the list of devices you will be testing against. The goal is to provide testing coverage for 85% of the target customer segment for every user story, as testing on all permutations of the devices (screen sizes, OSs, hardware, etc.), as mentioned earlier, is unlikely to be a viable option. For example, testing on all Android versions and devices from every manufacturer would be time-consuming and expensive. Especially in an iterative development process such as Agile or Scrum, having to test on tens of devices to validate one user story would adversely impact your team's delivery tempo. So, narrowing down the list of test devices is crucial. Here is a set of

questions you should find answers to in order to filter the devices to meet the goal of 85% coverage, as illustrated in Figure 11-3:

- What are the target customer segments for the business? For example, a high-end apparel business may target the affluent, and therefore lower-end handsets can be set aside.
- Which specific markets/countries the business is trying to expand to, and what are the top OSs and vendors in those markets? For example, the apparel business might want to focus on European cities, and Samsung and Apple are the top two vendors in Europe. This narrows down the device list a bit further, as you can assume there is a high possibility that well-off users will be using flagship devices from each of these brands.
- If the business already has an online presence, what does the device-specific usage look like? For example, their existing web app could be accessed most from iPhones and Samsung tablets.
- What is the network bandwidth range available in the target markets? For example, the average mobile network speed in Europe is estimated to be about 54 Mbps, which demands inclusion of 4G-enabled devices. The network criteria become especially important when dealing with lower-end phones.

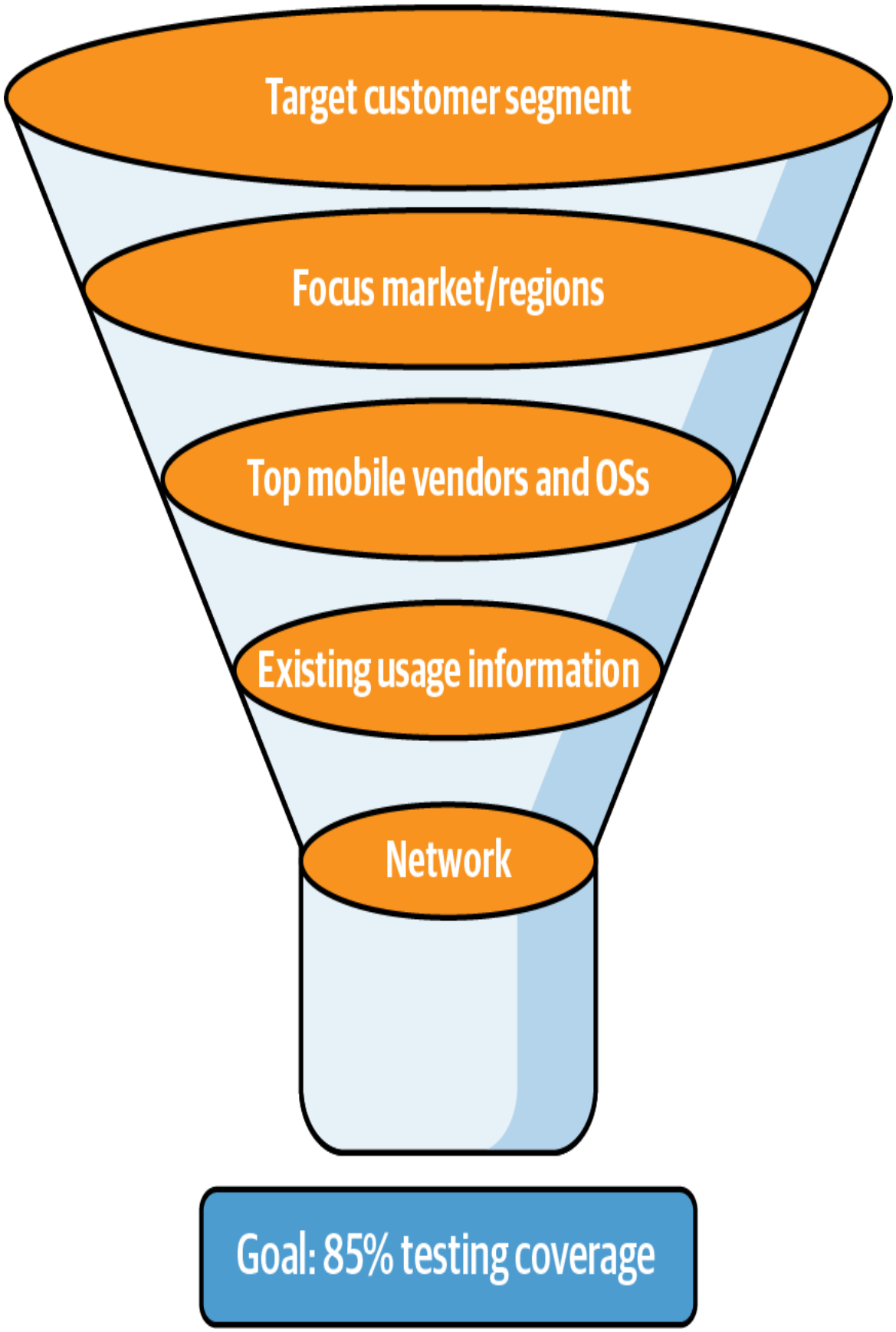

Target customer segment
Focus market/regions
Top mobile vendors and OSs
Existing usage information
Network
Goal: 85% testing coverage

Figure 11-3. Mobile devices filtering strategy

Once you get the answers to these questions from the business or from a product representative, you can select three or four handsets that have the appropriate characteristics, and perhaps a few more nice-to-have devices to check out during your regular team bug bashes.

> **NOTE**
>
> Selecting the devices for testing is an activity that needs to be done at the beginning of the project itself. Once you choose your devices, you should do a cost analysis on whether to buy them or subscribe to a cloud-hosted device provider such as the AWS Device Farm, Firebase Test Lab, Xamarin Test Cloud, Perfecto, or Sauce Labs. Such providers enable you to run automated tests on their hosted real devices, but you might find the interactions to be slower in such cases.

Figure 11-4 shows a testing strategy for the mobile UI layer. As you can see, the testing methods are similar to those that we have discussed throughout the book up to this point. Next, we'll discuss how the different methods lend a hand to navigate through the complexities posed by the mobile landscape.

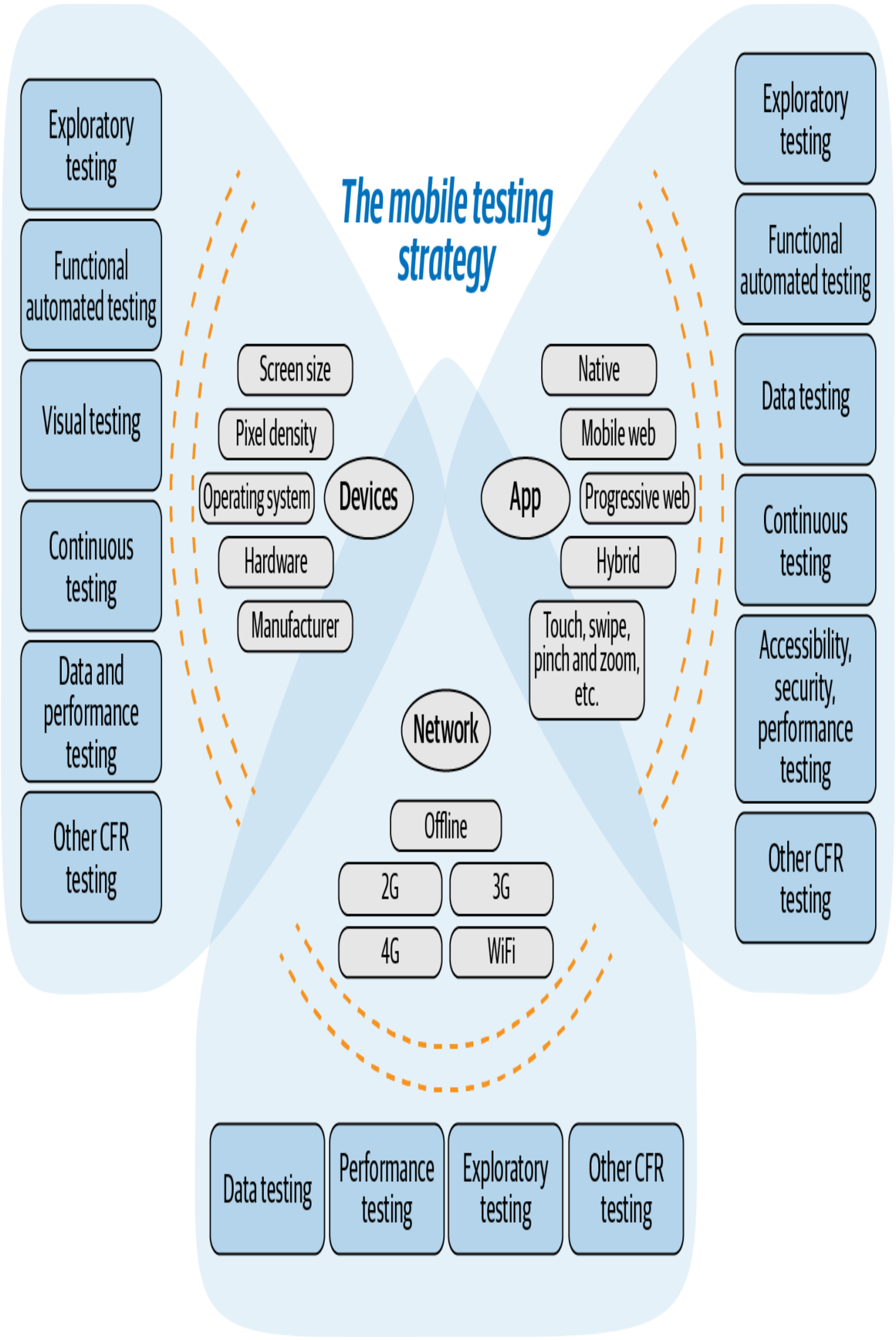

The mobile testing strategy
Exploratory testing
Functional automated testing
Visual testing
Continuous testing
Data and performance testing
Other CFR testing
Devices
Screen size
Pixel density
Operating system
Hardware
Manufacturer
App
Native
Mobile web
Progressive web
Hybrid
Touch, swipe, pinch and zoom, etc.
Network
Offline
2G
3G
4G
WiFi
Exploratory testing
Functional automated testing
Data testing
Continuous testing
Accessibility, security, performance testing
Other CFR testing
Data testing
Performance testing
Exploratory testing
Other CFR testing

Figure 11-4. A mobile testing strategy

Manual Exploratory Testing

Exploratory testing becomes even more important in the mobile landscape due to the myriad of device, app, and network combinations for test cases. The exploratory testing techniques and strategy elaborated in Chapter 2 will be of assistance here as well, and applying them through these three lenses will enable comprehensive inspection of the app behavior. The Chrome DevTools option to view websites in any given resolution, as we saw in Chapter 8, can be used to facilitate exploration of mobile web apps. To explore other app types, you can either purchase the devices that you've decided to use for testing or use emulators/simulators.

Emulators and simulators are programs that create a virtual device environment on a computer. For example, Android Studio provides Android emulators that mimic the exact hardware and software configurations of real devices such as the Google Nexus 4, Samsung Galaxy 5, Moto G, and many others. Similarly, iOS provides simulators for iPhones and iPads. We will look at the setup of an Android emulator in the exercises section later in this chapter. Although they may be sufficient to do sanity testing of the app, emulators and simulators have limitations in mimicking some hardware features, including certain touch gestures, sensor integrations, and more. Personally, I find testing with these tools insufficient to call an app version ready for release, and the industry agrees—both Apple and Google recommend testing on real devices before release. You may find them useful during test development to swiftly check the code, but in general you should use emulators and simulators only in the absence of real devices or for quick sanity checking .

> **TIP**
>
> In an Agile testing environment, you can easily shift the device testing to the left. For example, the developer can use the most challenging resolution (LDPI or MDPI) during development. During dev-box testing, the business analyst, QA engineer, and developer can each test on one device version. And for regression purposes, the automated tests can run on a set of chosen devices in CI.

Functional Automated Testing

Testing of the functional behavior of the app and the interactions can be automated using unit and UI-driven functional end-to-end tests. Just like functional tests for web apps, they need to be plugged into the CI pipeline to get continuous feedback. The end-to-end tests can also be used to ensure that the app features work reliably across a set of target devices by running them either as smoke tests or nightly regressions. Appium and Espresso are a couple of widely used tools for writing UI-driven end-to-end tests for Android, while Appium and XCUITest are widely used when developing for iOS. Espresso and XCUITest can be used only for native apps, but Appium can actually be used to automate all three app types (native, hybrid, and mobile web apps). You'll learn how to use this tool to create UI-driven end-to-end tests later in this chapter.

Data Testing

When it comes to mobile apps, data can be stored in various layers for different purposes. As shown previously in Figure 11-2, there is a local mobile DB, a common database, and the local device storage to store and retrieve data. It is essential to understand the data flow to all of these storages and include them in your testing, as discussed in Chapter 5. To give a few examples of data flow in mobile apps, a social networking app like Facebook could store the most recent posts in the local mobile DB so that when the network

conditions are unstable, it can be used to quickly render the app. In such use cases, you may need to think of the user experience with respect to showing stale information, the volume of data that can be stored in the local DB, keeping the local DB in sync, and so on. Also, users may access the app from multiple devices, such as a phone, a tablet, and a desktop web browser. So, data syncing to all the devices' DBs needs to be tested.

When it comes to the common database, there needs to be a trace of the different transactions made from different devices and a mechanism to update the appropriate data without conflicts. For example, a user could save calendar events from different mobile devices and sync them when the network is available. This has to be tracked and updated in the common database. Such scenarios can be automated as micro- and macro-level functional tests too. In summary, the two-way data sync between the common database and the local mobile DB across devices, bound by the network conditions, may be an essential aspect to test when it comes to data testing in mobile apps.

In addition, when there are functionalities that require storing files in and retrieving them from local device storage, these need to be tested as well. You'll need to consider boundary conditions for both internal and external device storage (when the storage is full or unavailable) and the limitations of the OS in processing different file formats. Overall, when thinking about data testing it's a good idea to view your mobile app through all three lenses!

Visual Testing

Visual testing will be covered when you do device testing manually. As mentioned earlier, you can shift your device testing to the left as well. You can also automate visual testing for different screen sizes (which you'll have picked during device selection) using Appium and Applitools Eyes for regression purposes. Applitools Eyes, as

discussed in Chapter 6, is a paid service that employs AI to automate the visual testing of mobile apps, while Appium is open source. An exercise to automate visual tests using Appium is included later in this chapter. You can choose between the two tools based on the factors mentioned in Chapter 6.

Security Testing

In Chapter 7, we discussed the security testing mindset and the essentials to look for from a security perspective while testing a functionality. This can be carried forward to mobile testing as well. For example, considerations during testing should include encryption and secure storage of users' sensitive data, strong authentication mechanisms, appropriate permissions to access other apps on the phone, etc.

We also discussed security testing tools in Chapter 7 that automatically scan the static application code for security vulnerabilities and inject known attacks to detect vulnerabilities in the application. As mentioned, those tools also work at the services layer and can be utilized in that layer in mobile applications. An automated static and dynamic security scanning tool specifically for the mobile UI layer (Android/iOS/Windows) is provided by an open source tool called the Mobile Security Framework (MobSF). GitLab, a popular DevOps platform, provides SAST for mobile apps powered by MobSF too. We'll walk through using MobSF and another automated security scanning tool later in this chapter.

Beyond relying on automated security scanning tools, it is essential for software teams to know the OWASP top 10 risks for mobile apps and address those risks during development. Depending on the skill level of the team, engaging pen testers post-development may also be necessary.

Finally, as mobile app security testing is a fairly niche area at the time of writing, I recommend keeping tabs on the collective wisdom

of the OWASP community through its Mobile Security Testing Guide.

Performance Testing

From Chapter 8, you know how to set up automated load/stress/soak tests for your services. Continue to measure and monitor them.

When it comes to the mobile UI layer, performance can mean a few different things than in a web UI, since mobile apps typically operate in a resource-constrained environment with limited and shared CPU, memory, battery power, and network conditions. To consolidate the mobile performance testing approach, ensure two aspects:

1. The app should not monopolize or deplete any of the device's critical resources, such as the CPU, memory, and battery power.
2. The app should respond quickly to end user actions.

To test point 1, you can use profiler tools in the respective OSs—for example, the Android Profiler in Android Studio and XCode Instruments for iOS. Validations on resource consumption can be added as automated unit tests using the respective tools and integrated with CI pipelines for continuous performance testing. Appium, too, provides an API to get similar performance data for Android apps, as you'll see later in this chapter.

While testing for point 2, pay particular attention to elements such as the app launch time, or the time taken from clicking the app icon to the time it opens. This should be less than 5 seconds. Similarly, the response to any kind of action inside the app should be under 3 s, or the bounce rate increases. Delays, however, are impacted significantly by network bandwidth where there are calls to services. You can simulate different network conditions in your emulator or simulator to measure the app's response time. Another part of in-app performance testing is stress testing. To stress the app, you can trigger multiple actions rapidly, such as touching several buttons, zooming in and out, sending requests, navigating across pages, etc.,

to see if the app crashes. Android comes with an automated tool called Monkey that conducts automated stress testing; we will explore that tool later in this chapter.

In summary, as with many of the other types of testing, when thinking about performance testing of mobile apps you should consider them through all three lenses.

Accessibility Testing

The W3C WAI provides detailed guidelines on how to apply WCAG 2.0 to mobile applications. The mobile accessibility guidelines follow the same four key principles: the app should be perceivable, operable, understandable, and robust. Some of the features to test for include being able to zoom in and out, readability on small screen sizes, color contrast between elements, reasonable click space for the buttons, consistent layout throughout the app, placing the right elements within the view area without the user needing to scroll, and so on. Both iOS and Android provide quite a few tools to test accessibility, as listed in the following subsections, although automated accessibility testing tools are currently limited.

iOS

iOS provides the following tools for accessibility testing:

- The VoiceOver screen reader is available both on physical devices and in iOS simulators. This can be used for end-to-end testing of user flows.
- The XCode Accessibility Inspector is available in iOS simulators to inspect elements to check if they have accessibility-related attributes appropriately set. This can be used for debugging purposes.

Android

Android has more elaborate support for shift-left accessibility testing. Starting from the left, the tools are:

- Android Studio, a development environment, can be configured to show lint warnings for various accessibility issues during development.
- Espresso (and Robolectric, prior to version 4.5) is a native Android app UI test automation tool with the facility to scan each view of the application for accessibility provisions. These tests can be integrated with an existing Espresso test suite and run in CI.
- TalkBack is the built-in screen reader in Android. It can be used for end-to-end testing of user flows.
- Accessibility Scanner is a tool that audits mobile apps for accessibility issues. It can be used during the user story manual testing phase.

Additionally, Android has a Switch Access feature that enables the use of external assistive devices for interacting with apps (known as switches), and it supports BrailleBack for connecting a braille display to the device and Voice Access to control an Android device with spoken commands. During app submission, the Google Play store even provides pre-launch accessibility audit reports to teams.

CFR Testing

The CFRs discussed in Chapter 10 remain relevant in mobile testing—for example, auditability, portability, reliability, compatibility, etc. In addition to security and accessibility, CFRs that you should explicitly focus on while testing the mobile UI include:

Usability

> If you think about it, a mobile device is a very personal object. As mentioned earlier, most adults spend two to three hours a day on

their phones, and these devices have become like extra limbs to many of us. Consequently, usability is a major concern. End users might be convinced to download an app based on fanfare, but they're only likely to use it continuously if they are able to personalize it to their liking. For example, the end user could be left- or right-handed, have a habit of multitasking with multiple apps open, use apps while driving, be multilingual, prefer one type of interaction over another, and so on. All of these factors have to be thought through while testing from a usability perspective. Of course, it may not be possible to cater to all 7.7+ billion unique individuals' needs; rather, the point is to tune our mindset to specifically consider these usability aspects while testing mobile apps. The usability testing approach from Chapter 10 can be applied here as well. On top of it, an important recommendation is to do a preliminary study on end user behaviors in your target market/regions. Google provides assistance there: the Think with Google site provides detailed infographics on mobile user behavior for several countries in addition to other key mobile-related reports.

Interruptions

This is a mobile-specific flavor of the reliability CFR. Since mobile devices are used for diverse purposes, including messaging and phone calls, it is a given that any app flow could be interrupted by external distractions. Typical user behavior is to keep the current app in the background while attending to such distractions, such as incoming phone calls or important chat notifications. Once they've been dealt with, the app flow is resumed.

So, while performing mobile testing, it's important to account for interrupting behaviors. What happens to the current request when the app is suddenly parked in the background? What happens with authentication when the app is paused, then resumed from the background? What happens with an ongoing

request when the app is killed/closed abruptly? What happens if the device runs out of battery power during the app workflow? Remember that this is a CFR and needs to be tested across the app.

Installability and upgradability

App installation on different devices and OSs from the respective app stores is an essential aspect of mobile testing. Installation requires a certain amount of local storage space on the device. Also, during installation, the app may ask the users for relevant permissions to access device hardware or other apps (camera, microphone, contacts, photo gallery, location services, etc.). These installation scenarios need to be tested, including failure cases such as lack of sufficient storage space, denial of app permissions, and incompatibility with the device's OS version. Also, app upgrade testing should ensure existing flows are not broken. For example, changes to local database structures, if any, shouldn't affect existing features. Additionally, the user should not be logged out of the app after the upgrade. Remember to test the case of upgrading from older versions of the app and not just from the latest app version. If upgrades require newer app permissions, then those need to be tested as well.

As installation and upgrades depend on network conditions, include various network-related scenarios as well. Similarly, ensure uninstallation of the app works perfectly.

Monitoring

Unlike with web applications, app crashes are pretty common in the mobile world, and therefore monitoring is a critical CFR when it comes to mobile apps. Sometimes the conditions that cause an app to crash are hard to reproduce, and monitoring tools (Firebase Crashlytics, Dynatrace, New Relic, etc.) can be key to

understanding the issue. Thus, even during the development phase, you should integrate these tools into the test environment so that it is easier to debug app crashes.

Note that testing of some of these CFRs, such as successful installation and upgrades on your target devices and OSs or behavior upon interruption, can be automated using functional micro- and macro-level tests to get continuous feedback—recall the continuous testing strategy from Chapter 4 here.

That brings us to the tail end of the mobile testing strategy. Next, we'll get hands-on with building test suites using some of the tools mentioned in this section.

Exercises

The exercises here will guide you through setting up a Java–Appium framework for creating UI functional and visual tests. I've chosen Appium because, as mentioned previously, it can support all three types of mobile apps (native, web, and hybrid) and can work across multiple OSs.

Appium

Appium is an open source tool supported by a vibrant community. As a cross-platform automation tool, Appium bundles OS-specific automation frameworks such as XCUITest (provided by Apple for iOS) and UiAutomator (provided by Google for Android) under a common set of wrapper APIs—the WebDriver APIs we met in Chapter 3. For example, Appium uses the `DesiredCapabilities` object to instantiate the `driver` object to interact with the app. Also, the `findElements(By.id)`, `click()`, `isElementPresent()`, and other APIs remain the same. As a result, the learning curve is very light if you are comfortable with automated testing using

Selenium WebDriver. Also, just like WebDriver, Appium is language-independent. This means that you can write tests in the programming language of your choice, such as Ruby, Python, Java, JavaScript, etc., with the respective client libraries.

Appium has announced its next major release, 2.0, which redefines how the Appium server, automation drivers, and plug-ins need to be installed. For example, the OS-specific automation drivers were bundled within Appium 1.x, whereas in 2.x they must be installed separately. It is in the beta stage at the time of writing and is expected to be released in 2022. Since it's the way forward for Appium, we will be using the 2.x beta version for this exercise. We'll be using Android, but since Appium's APIs are the same across OSs you should be able to apply similar steps to write tests for iOS apps too.

APPIUM, AN RPA TOOL!

Robotic process automation (RPA) is a hot topic in the industry these days. It's seen as a way to reduce the burden of mundane process-related manual tasks and accelerate operational efficiency by automating the business processes end-to-end. In other words, it refers to the automation of typical business processes such as capturing data in a spreadsheet, entering it in an internal tool, triggering a job to process the data, verifying that it appears online, and so on.

Appium 1.x, although primarily used for mobile app automation, supports Windows and Mac desktop app automation with the relevant drivers. Also, it doesn't require the application under test to be developed by the team creating the tests in order to interact with it (i.e., it doesn't need the source code). So, along with its Selenium WebDriver capabilities, Appium 1.x can be used as an RPA tool! Hopefully this support will be maintained in Appium 2.x.

Let's get started!

Prerequisites

The prerequisites are similar to those of the other automation tools we discussed in Chapter 3, so check if you have them before installing them. You will need:

- Node.js, to set up the Appium server
- The latest version of Java (we'll be using Appium's Java client library for this exercise)
- An IDE such as IntelliJ
- Maven

Android emulator

Once you have successfully set up all the prerequisites, create an Android emulator to run your Appium test on by following these steps:

1. Download and set up Android Studio. This brings with it the required Android SDK and tools.
2. In Android Studio, select More Actions → AVD Manager. (AVD stands for Android Virtual Device.)
3. Click Create Virtual Device to see a list of existing hardware profiles for tablets, phones, Wear OS devices, and more. Choose the Phone category and select a profile—say, Pixel 2, 5.0 inches—then click Next.
4. Select an Android OS version, such as Android 8.0. You'll be given the option to download it if you don't have the requested version.
5. Provide a name for the emulator on the next screen—say, "Oreo"—and click Finish. Your Android 8.0 Pixel 2 emulator

should now appear in the list of available virtual devices.

6. Click the play/run button to start the emulator.

For this exercise, you can download the demo Android app *ApiDemos-debug.apk* from Appium's GitHub repository. Install the app by dragging and dropping it inside the emulator, then open it up and take a look around to get familiar with it.

Appium 2.0 setup

To set up Appium, follow these steps:

1. Run the following command to install Appium v2.0:

```
$ npm install -g appium@next
```

> **NOTE**
>
> Note that this step is subject to change after the official release.

2. Set up the UiAutomator2 driver by running the following command:

```
$ appium driver install uiautomator2
```

> **NOTE**
>
> For iOS you'll need the XCUITest driver, which can be installed using the command **`appium driver install xcuitest`**.

3. Start the Appium server using the following command:

```
$ appium server -ka 800 -pa /wd/hub
```

4. Download Appium Inspector, a GUI tool that allows us to find the element locators in a mobile app.

Workflow

As mentioned earlier, Appium uses the `DesiredCapabilities` object to instantiate a connection with the mobile app. In Appium Inspector, you can configure this object via the GUI and connect with the app to inspect the elements. Try inspecting the demo Android app by following these steps:

1. Open the inspector and provide the Desired Capabilities values shown in Figure 11-5. Save them so that you can reuse them later.

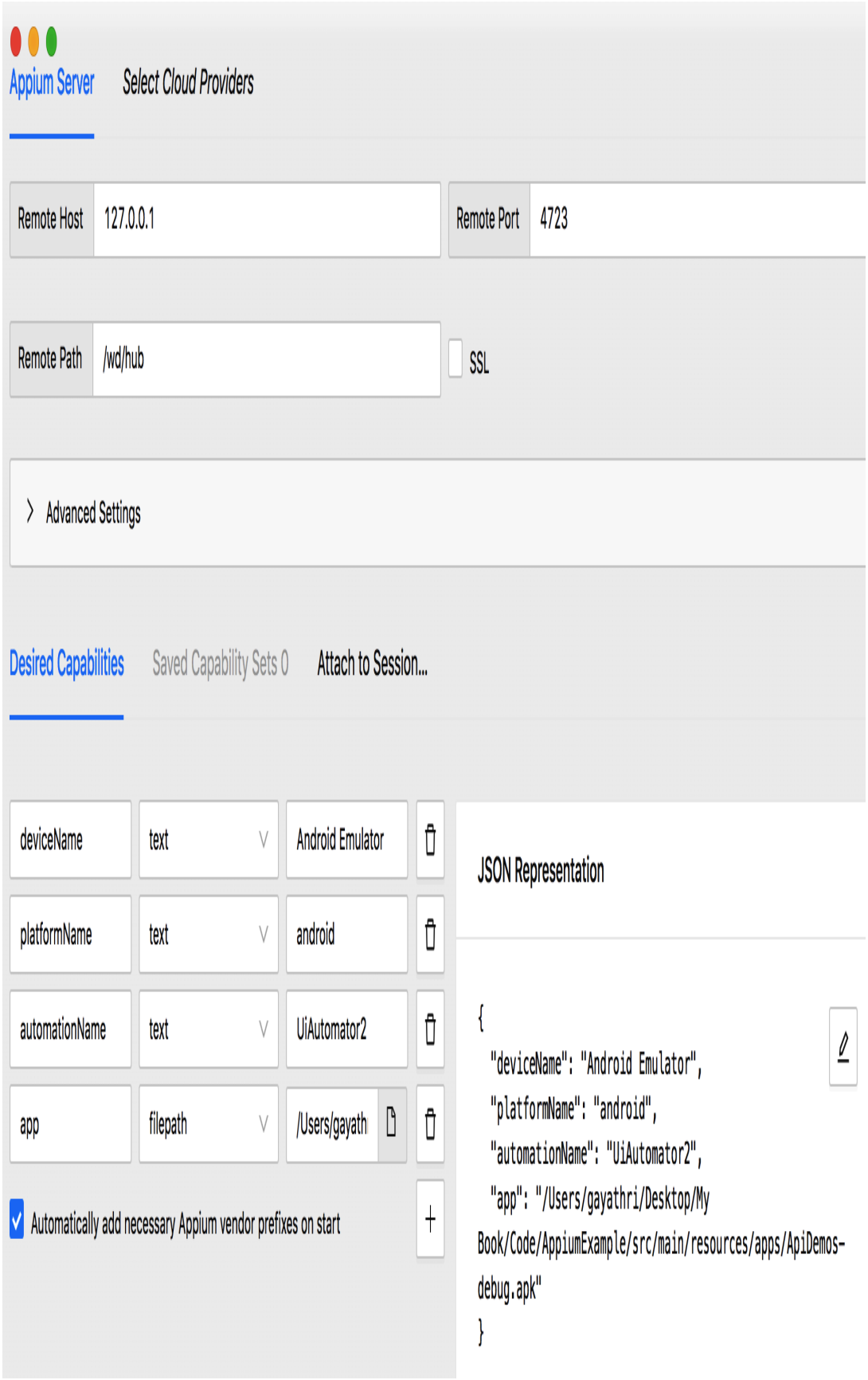

Appium Server
Select Cloud Providers
Remote Host
127.0.0.1
Remote Port
4723
Remote Path
/wd/hub
SSL
Advanced Settings
Desired Capabilities
Saved Capability Sets 0
Attach to Session...
deviceName
text
Android Emulator
platformName
text
android
automationName
text
UiAutomator2
app
filepath
/Users/gayath
Automatically add necessary Appium vendor prefixes on start
JSON Representation
{
"deviceName": "Android Emulator",
"platformName": "android",
"automationName": "UiAutomator2",
"app": "/Users/gayathri/Desktop/My Book/Code/AppiumExample/src/main/resources/apps/ApiDemos-debug.apk"
}

Figure 11-5. Desired Capabilities to connect to the demo app

2. Click Start Session. This should open the demo app within the inspector window.
3. Click the Select Elements icon (the third icon in the top bar), and hover over the app to inspect the elements. You will see the elements getting highlighted as you hover over them. Select one, and the element's properties will be displayed in the righthand panel, as seen in Figure 11-6.

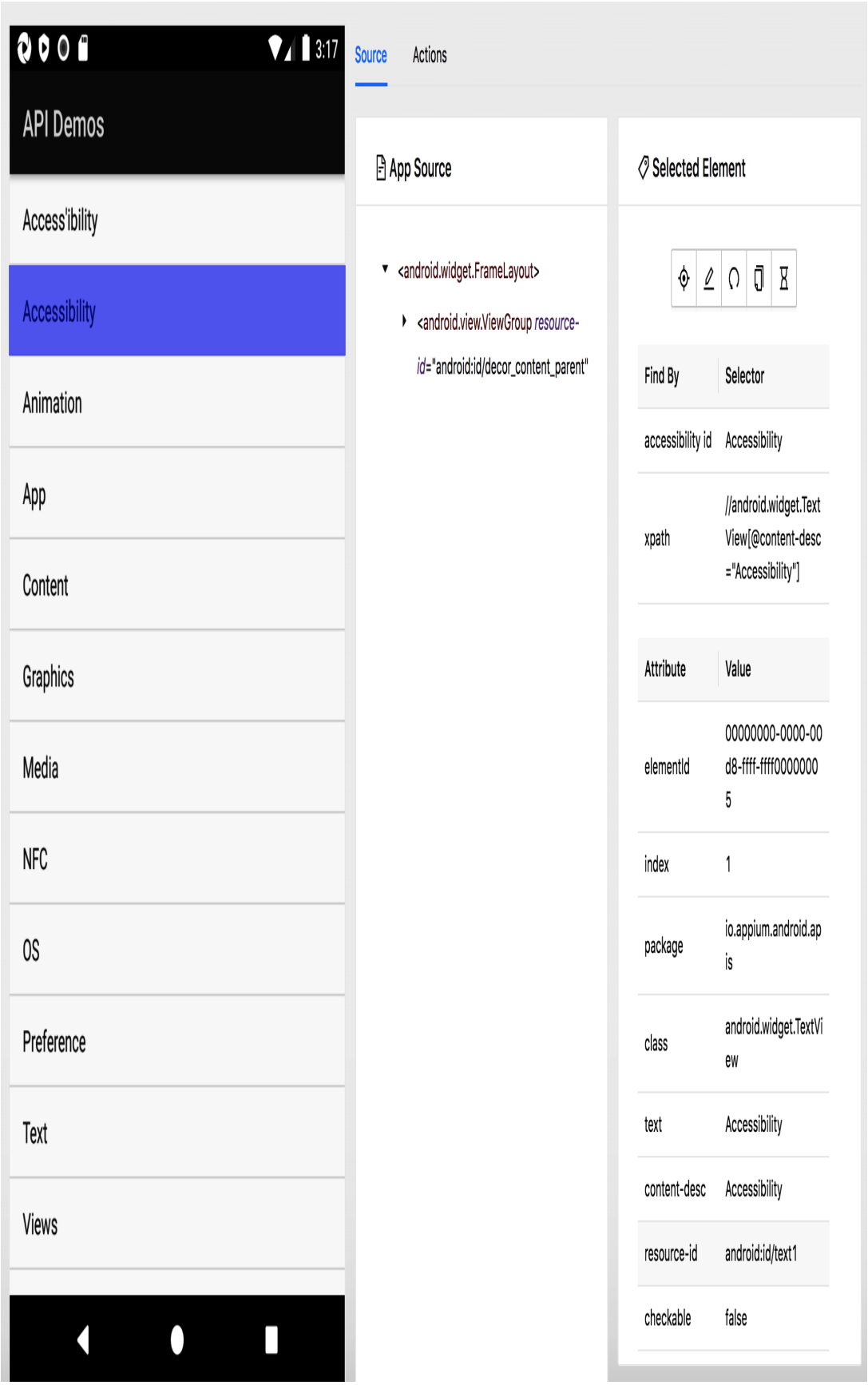

3:17
API Demos
Access'ibility
Accessibility
Animation
App
Content
Graphics
Media
NFC
OS
Preference
Text
Views
Source
Actions
App Source
<android.widget.FrameLayout>
<android.view.ViewGroup resource-id="android:id/decor_content_parent"
Selected Element
Find By
Selector
accessibility id
Accessibility
xpath
//android.widget.TextView[@content-desc="Accessibility"]
Attribute
Value
elementId
00000000-0000-00d8-ffff-ffff00000005
index
1
package
io.appium.android.apis
class
android.widget.TextView
text
Accessibility
content-desc
Accessibility
resource-id
android:id/text1
checkable
false

Figure 11-6. Inspecting elements in Appium Inspector

The panel on the right has the element's attributes, such as `resource-id`, `class`, `text`, etc., that can be used as element locators. Note the `package` value, `io.appium.android.apis`. You will need that to write tests. You can also send commands such as Tap, Send Keys, and Clear to selected elements by pressing the buttons in the righthand panel. This is useful for debugging, for example to check if a Tap on an element takes you to the next page as intended.

Sounds easy, right? The UI test framework setup is simple as well.

The Java–Appium framework setup is similar to the Java–Selenium framework setup discussed in Chapter 3. You can use Maven and TestNG along with the Appium client library. You can also adopt the Page Object Model for mobile UI tests.

Let's take a simple test case to automate: open the demo app and verify the second element's text, which is "Accessibility," on the home page. Follow these steps:

1. Open IntelliJ and create a new Maven project called AppiumExample.
2. Add an Appium Java dependency and a TestNG dependency to your *pom.xml* file. (Refer to Chapter 3 if you need a refresher on these two steps.)
3. Create a new folder called *apps* under */src/main/resources* and copy-paste the demo app file, *ApiDemos-debug.apk*, here.
4. Create a `pages` package under */src/main/java*. Create `tests` and `base` packages under */src/test/java*. Your page classes go under the `pages` package, and test classes go under the `tests` package. The setup classes go under the `base` package.

5. The `Base` class contains the Appium setup and teardown methods with `DesiredCapabilities` configured with the app package name, app path, emulator name, device name, platform name, and automation framework name, as seen in Example 11-1.

Example 11-1. The `Base` class with Appium setup

```
// src/test/java/base/Base.java

package base;

import io.appium.java_client.MobileElement;
import io.appium.java_client.android.AndroidDriver;
import io.appium.java_client.remote.MobileCapabilityType;
import org.openqa.selenium.remote.DesiredCapabilities;
import org.testng.annotations.*;
import java.io.File;

public class Base {
    protected AndroidDriver<MobileElement> driver;

    @BeforeMethod
    public void setUp(){
        File appDir = new
File("src/main/resources/apps");
        File app = new File(appDir, "ApiDemos-
debug.apk");

        DesiredCapabilities capabilities = new
DesiredCapabilities();

capabilities.setCapability(MobileCapabilityType.DEVICE_NA
ME,
                        "Android Emulator");

capabilities.setCapability(MobileCapabilityType.PLATFORM_
NAME,
                        "android");

capabilities.setCapability(MobileCapabilityType.AUTOMATIO
N_NAME,
                        "UiAutomator2");
```

```
capabilities.setCapability(MobileCapabilityType.APP,
                           app.getAbsolutePath());
        capabilities.setCapability("avd", "Oreo");
        capabilities.setCapability("appPackage",
"io.appium.android.apis");
        driver = new AndroidDriver<MobileElement>
(capabilities);
    }

    @AfterMethod
    public void tearDown(){
        driver.quit();
    }
}
```

6. Example 11-2 shows the `HomePage` class, with a method that gets the text of the second element on the home page identified by its `id`. The `id` provided here is the value of the `resource-id` attribute.

Example 11-2. The `HomePage` class with element locator and actions

```
// src/main/java/pages/HomePage.java

package pages;

import io.appium.java_client.MobileElement;
import io.appium.java_client.android.AndroidDriver;
import org.openqa.selenium.By;

public class HomePage{

    private AndroidDriver<MobileElement> driver;
    private By textItem = By.id("android:id/text1");

    public HomePage(AndroidDriver<MobileElement> driver)
{
        this.driver = driver;
    }

    public String getFirstTextItem(){
       return
driver.findElements(textItem).get(1).getText();
```

```
    }
}
```

7. Example 11-3 shows the `HomePageTest` class, with a test that opens the app and asserts the text of the second element on the home page using TestNG.

   Example 11-3. The `HomePageTest` class with the first test

```
// src/test/java/tests/HomePageTest.java

package tests;

import base.Base;
import org.testng.Assert;
import org.testng.annotations.Test;
import pages.HomePage;

public class HomePageTest extends Base {

    @Test
    public void verifyFirstTextItemOnHomePage() throws
Exception {
        HomePage homePage = new HomePage(driver);
        Assert.assertEquals(homePage.getFirstTextItem(),
"Accessibility");
    }
}
```

8. You can run the test from the IDE or by running the command `mvn clean test` in your terminal. You'll see the test executing in the emulator (if the emulator is not open, Appium will open it and then run the test). When you run tests from the command line, you can also get HTML reports under */target/surefire-reports/*.

You can add the tests to your CI pipelines to do continuous testing of your mobile app. If you are in need of additional APIs to automate your app's test cases, such as tapping, scrolling, or swiping, refer to Appium's official documentation.

Appium Visual Testing Plug-in

The Appium 2.0 visual testing plug-in uses OpenCV, an open source image processing tool, to perform image comparison. Compared to Applitools Eyes, which we looked at in Chapter 6, this plug-in has limited capabilities. For example, Applitools Eyes can scroll down the entire page and do a visual comparison without additional programming, whereas with Appium we need to write code to take multiple screenshots and stitch them together before passing the result for image comparison. However, the Appium plug-in is open source and therefore allows us to add at least a minimal set of visual tests alongside the Appium functional test suite without additional cost.

Let's add some visual tests to the same UI test we created earlier.

Setup

To set up the plug-in, follow these steps:

1. Install OpenCV by running the following command:

```
$ npm install -g opencv4nodejs
```

2. Install the Appium visual testing plug-in using the following command:

```
$ appium plugin install images
```

3. Start the Appium server with this command:

```
$ appium server -ka 800 --use-plugins=images -pa
/wd/hub
```

Workflow

The Appium plug-in, `images`, provides two APIs for visual testing. One is to do image comparison between the baseline image and the actual image, as seen here:

```
SimilarityMatchingResult result =
    driver.getImagesSimilarity(baselineImg, actualScreen,
options);
```

And the other is to get a comparison score from the `result` object, which can be used to fail the test if it is less than a threshold value, as seen here:

```
result.getScore() < 0.99
```

The comparison score ranges from 0–1. The ideal expected value is 1, but due to subtle variations you might not always get the ideal score; you can use the threshold value as a way to control the test sensitivity as appropriate to your project's needs.

With these two APIs, the visual testing workflow is simple: you create a set of base screenshots of the app's screens, compare them against the current versions of the app's screenshots, and fail the test if the score is less than the threshold. Example 11-4 shows the previously created Appium UI test with additional visual assertions. The example code also creates base screenshots on the first run of the test by default. Note that you'll need to create a `BasePage` class and add the respective setup code there.

Example 11-4. Automated visual testing with the Appium 2.0 plug-in

```
// src/main/java/pages/BasePage.java

package pages;

import io.appium.java_client.MobileElement;
import
io.appium.java_client.imagecomparison.SimilarityMatchingOptio
ns;
import
```

```
io.appium.java_client.imagecomparison.SimilarityMatchingResul
t;
import org.openqa.selenium.OutputType;
import io.appium.java_client.android.AndroidDriver;
import java.io.File;
import org.apache.commons.io.FileUtils;

public class BasePage {
    private File baselineDir = new
File("src/main/resources/baseline_screenshots");

    public void checkVisualQuality(String screen_name,
        AndroidDriver<MobileElement> driver) throws Exception
{
        File baselineImg = new File(baselineDir, screen_name
+ ".png");
        File actualScreen =
driver.getScreenshotAs(OutputType.FILE);

        if (baselineImg.exists()) {
            SimilarityMatchingOptions options = new
SimilarityMatchingOptions();
            options.withEnabledVisualization();
            SimilarityMatchingResult result =

driver.getImagesSimilarity(baselineImg, actualScreen,
options);

            if (result.getScore() < 0.99) {
                File imageDiff = new
File("src/main/resources/baseline_screenshots"
                                        + "FAIL_" +
screen_name + ".png");
                result.storeVisualization(imageDiff);
                throw new Exception("Visual quality
hampered");
            }
        } else {
            FileUtils.copyFile(actualScreen, baselineImg);
        }
    }
}

// src/test/java/tests/HomePageTest.java
```

```
public class HomePageTest extends Base {

    @Test
    public void verifyFirstTextItemOnHomePage() throws
Exception {
        HomePage homePage = new HomePage(driver);
        Assert.assertEquals(homePage.getFirstTextItem(),
"Accessibility");
        BasePage basePage = new BasePage();
        basePage.checkVisualQuality("home_page", driver);
    }
}
```

The plug-in provides an API, `result.storeVisualization()`, to view the differences between the two images when a test fails. To see how the tests work, first, run `mvn clean test` from the command line. This will create a baseline screenshot of the home page under */src/main/resources/baseline_screenshots*. Now, when you rerun the test, it should pass as there were no changes to the app. To make the test fail, you can give a different *.png* file as the baseline and run it again. You will see a new image, as shown in Figure 11-7, generated under the same *baseline_screenshots* folder, highlighting the images' differences.


API Demos
API Demos

Figure 11-7. Visual test failure results

So far, we've discussed a step-by-step approach to add automated UI tests to validate a mobile app's functional behavior and visual quality using Appium. These are the two most commonly practiced types of mobile testing. Next, we'll explore some additional tools that assist in other types of mobile testing.

Additional Testing Tools

In this section, you will get to know some tools that can be used to conduct performance, security, accessibility, and data testing. Although some of these testing types are not currently widely practiced in the mobile space, it is advisable to start applying them wherever appropriate, for the same reasons for which they should be adopted in a web context.

> **NOTE**
>
> The illustrations in this section mostly show the tools' workflow with Android. However, the same workflow can be applied for iOS as well. In cases where there are parallel tools for iOS, they are highlighted in the relevant section of the earlier mobile testing strategy discussion.

Android Studio's Database Inspector

If you want to explore the local database of the mobile app, Android Studio provides a Database Inspector tool that provides easy GUI access. Like any other database client, you can use the tool's interface to add/edit/delete data and verify the app's behavior.

To use the Database Inspector:

1. Select More Actions → Profile or Debug APK from Android Studio, and choose the app's .apk file. Note that this file must

have the debug option enabled in order for you to use this tool.

2. Select View → Tools Window → App Inspection. This will open the App Inspection panel at the bottom of the screen.
3. Run your app in an emulator from Android Studio using the green run button.
4. The Database Inspector will now be open inside the App Inspection panel. Note that the demo app doesn't have a local DB; Figure 11-8 shows a different app's local database, just as an example.

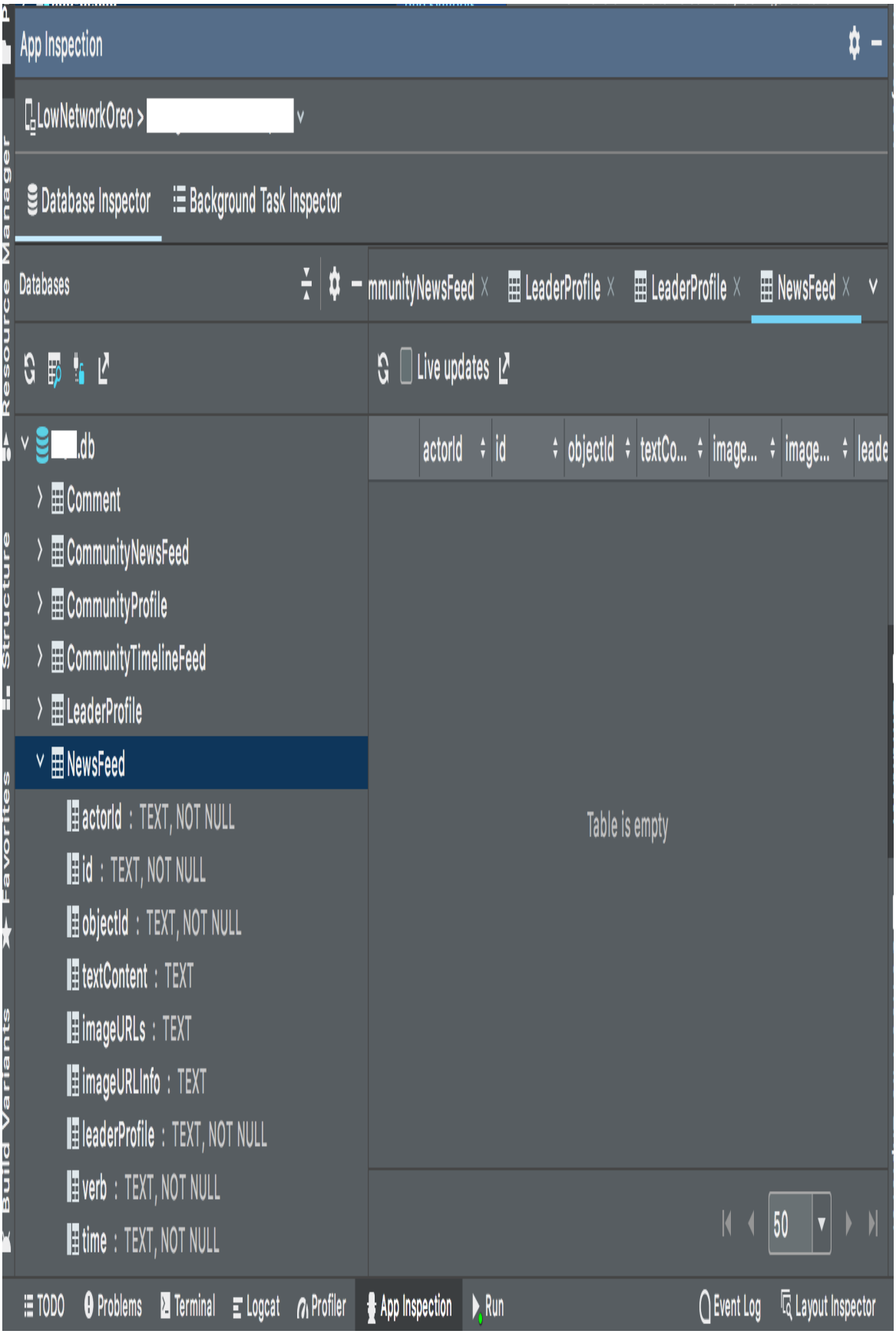

App Inspection
LowNetworkOreo >
Database Inspector
Background Task Inspector
Databases
mmunityNewsFeed X
LeaderProfile X
LeaderProfile X
NewsFeed X
Live updates
.db
Comment
CommunityNewsFeed
CommunityProfile
CommunityTimelineFeed
LeaderProfile
NewsFeed
actorId : TEXT, NOT NULL
id : TEXT, NOT NULL
objectId : TEXT, NOT NULL
textContent : TEXT
imageURLs : TEXT
imageURLInfo : TEXT
leaderProfile : TEXT, NOT NULL
verb : TEXT, NOT NULL
time : TEXT, NOT NULL
actorId
id
objectId
textCo...
image...
image...
leade
Table is empty
50
TODO
Problems
Terminal
Logcat
Profiler
App Inspection
Run
Event Log
Layout Inspector
Resource Manager
Structure
Favorites
Build Variants

Figure 11-8. Android Studio's Database Inspector tool view

In general, you can use it to ensure that the expected data is stored to enable offline access to the app and to verify that sensitive information is not stored without encryption.

Performance Testing Tools

In "Performance Testing", we discussed different aspects of performance that need to be tested with mobile apps. Here we will explore three tools that can help you with that.

Monkey

Monkey can be considered a Chaos Engineering equivalent for Android apps. It performs random sequences of actions such as touches, keypresses, clicks, and other gestures in the app UI and reports app crashes, if any. Monkey comes as a simple command-line tool. If you have Android Studio installed already, you can stress test the demo Android app either on an emulator or a physical device with the following command:

```
$ adb shell monkey -p "io.appium.android.apis" -v 2000
```

This command sends 2,000 different events to the app. You can watch the execution on the emulator/device as well. When there are unhandled exceptions or the app fails to respond, Monkey pauses execution and reports those issues. You can customize the stress test to perform specific events by passing appropriate optional parameters in the command, as listed in the documentation.

Extended controls: Network throttler

Another aspect of performance testing we discussed earlier was testing the app's performance under various network conditions. Android emulators enable simulation of different network types, such

as GSM, GPRS, Edge, LTE, etc. You can also throttle bandwidth further by setting the signal strength to Good, Moderate, Poor, Great, etc. To try this, click the ”More options” button in the emulator’s side panel and select Cellular from the settings panel. You will see the throttling options shown in Figure 11-9. Note the additional controls apart from network type and signal strength, such as data status, voice status, etc. You can use these to customize the network further, as required by your test case.

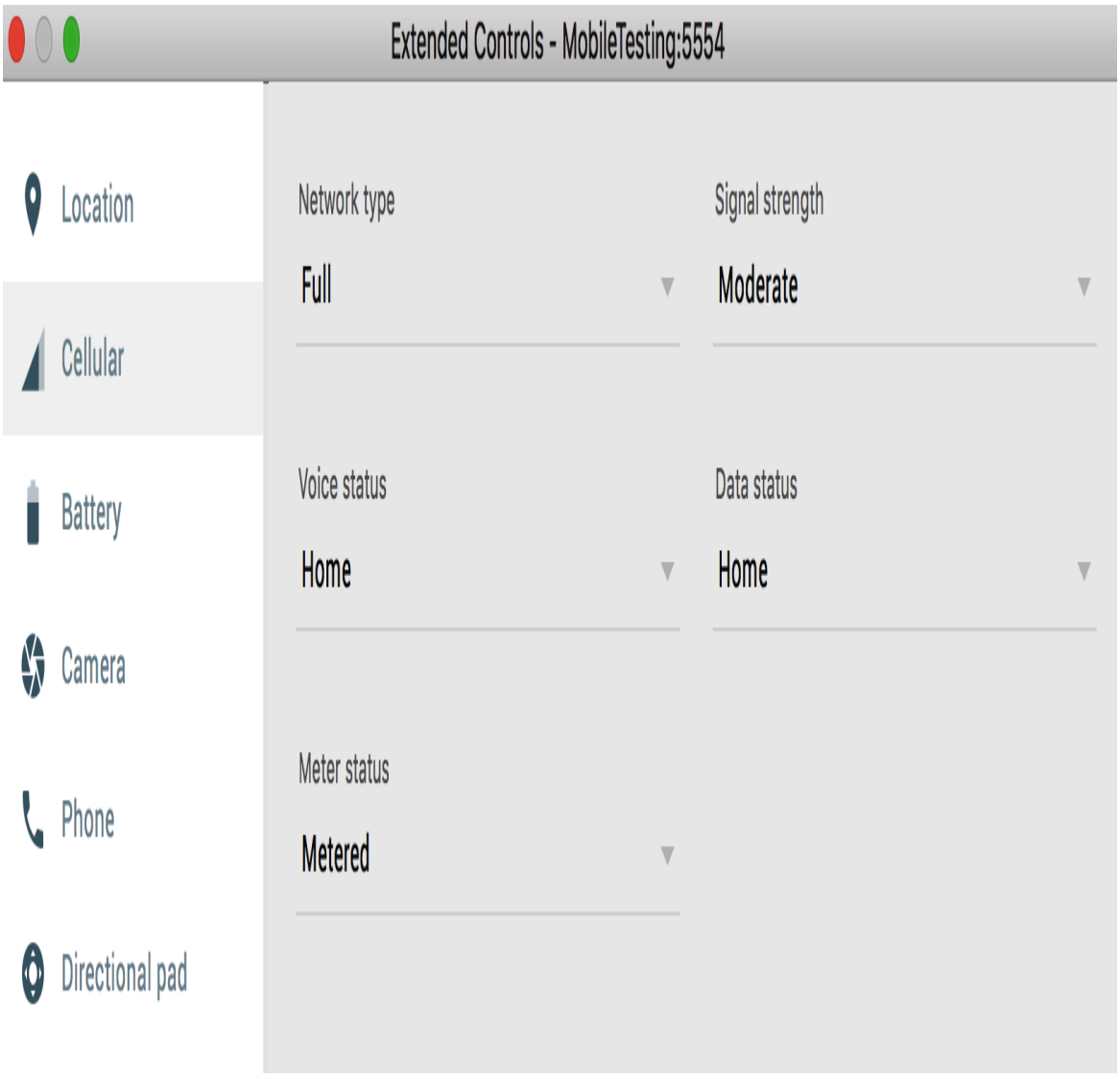


Figure 11-9. Network throttling options in Android Emulator

Appium’s performance API

Appium provides an API to gauge an Android app's performance in terms of memory, CPU, battery, and network consumption. You can add automated performance tests alongside the UI tests using this API, as shown here:

```
driver.getPerformanceData("package_name", "perf_type",
timeout);
```

where *package_name* is the app's package name, which you used for automation earlier; *perf_type* refers to the system state that you want to monitor, such as CPU, network, etc.; and *timeout* is the number of seconds that the API will poll for performance data before throwing an error. The exact value for the *perf_type* parameter can be obtained from another Appium API, `getSupportedPerformanceDataTypes()`. Currently, there are four supported values: `cpuinfo`, `memoryinfo`, `batteryinfo`, and `networkinfo`.

> **NOTE**
>
> Internally, this API is built on top of Android's `dumpsys` command-line tool, which outputs diagnostics of the system's services. Hence, it can be used only with Android apps.

Using this API, we can add a performance test, get the performance numbers at different points in the user flow being executed by the UI test, and assert that the numbers are within a certain threshold. For example, we can assert the memory consumption against a threshold after a complex operation in the app. Example 11-5 shows the memory consumption data just after opening the demo Android app.

Example 11-5. Demo app's memory consumption output using Appium's performance API

```
driver.getPerformanceData("io.appium.android.apis","memoryinf
o", 10);

// Output
[[totalPrivateDirty, nativePrivateDirty, dalvikPrivateDirty,
eglPrivateDirty, glPrivateDirty, totalPss, nativePss,
dalvikPss, eglPss, glPss, nativeHeapAllocatedSize,
nativeHeapSize], [11432, 4708, 1692, null, null, 20807, 4926,
1717, null, null, 12648, 14336]]
```

For details on interpreting these output values and adding appropriate app-specific assertions, refer to the dumpsys documentation.

Security Testing Tools

We will look at two tools for automated security testing in this section: MobSF and Qark.

MobSF

As mentioned earlier in this chapter, the Mobile Security Framework is an open source tool that does automated static and dynamic analysis of Android, iOS, and Windows apps. It also helps with malware analysis. To try MobSF, follow these steps:

1. Download and install Docker Desktop, if you haven't done this yet, and open the application. (You don't need to know much about using Docker to try this. A word of caution, however: check your company's policies on Docker installation on your work laptop, as it is free only for personal use.)
2. Get the MobSF Docker container by running the following command:

```
$ docker run -it -p 8000:8000
  opensecurity/mobile-security-framework-mobsf:latest
```

3. This will set up MobSF on your local machine. Open *http://0.0.0.0:8000* to view the MobSF web page.
4. Upload the demo Android app's APK to this web page. Alternatively, you can use the InsecureBankv2 APK, intentionally created with vulnerabilities for learning purposes.
5. MobSF will scan the app and display the results on the same local web page, as seen in Figure 11-10, highlighting the vulnerabilities with their severity.

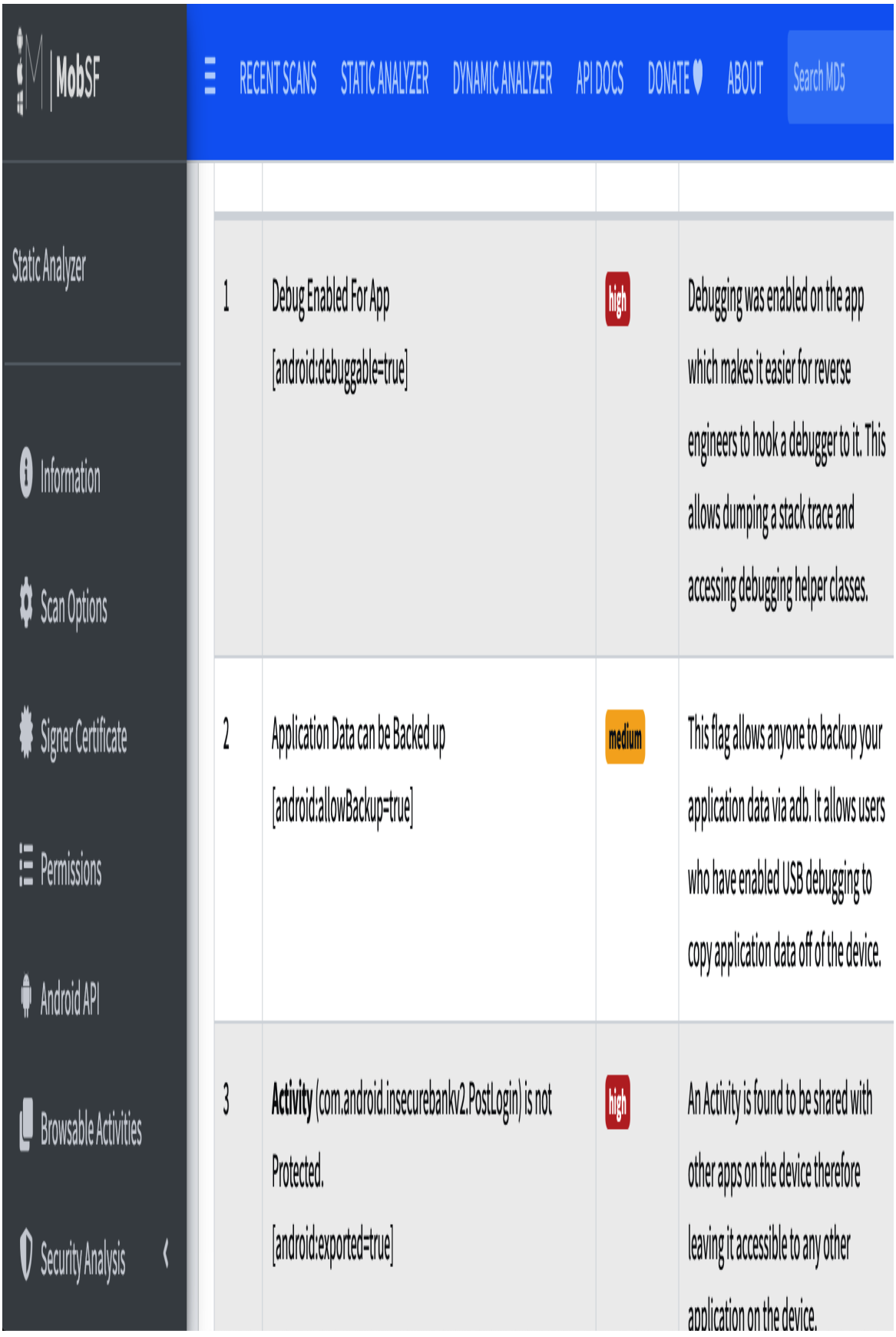

MobSF
RECENT SCANS
STATIC ANALYZER
DYNAMIC ANALYZER
API DOCS
DONATE
ABOUT
Search MD5
Static Analyzer
Information
Scan Options
Signer Certificate
Permissions
Android API
Browsable Activities
Security Analysis
1
Debug Enabled For App
[android:debuggable=true]
high
Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.
2
Application Data can be Backed up
[android:allowBackup=true]
medium
This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
3
Activity (com.android.insecurebankv2.PostLogin) is not Protected.
[android:exported=true]
high
An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.

Figure 11-10. MobSF scan results

You can also integrate the tool with CI pipelines as per the documentation to automatically scan the code on check-in.

Qark

Qark is another open source security scanning tool for Android apps. It can scan the source code or the APKs. Qark is a Python-based tool. You can install it with the `pip` package manager using this command:

```
$ pip install qark
```

and run the security scan against your APK using this command:

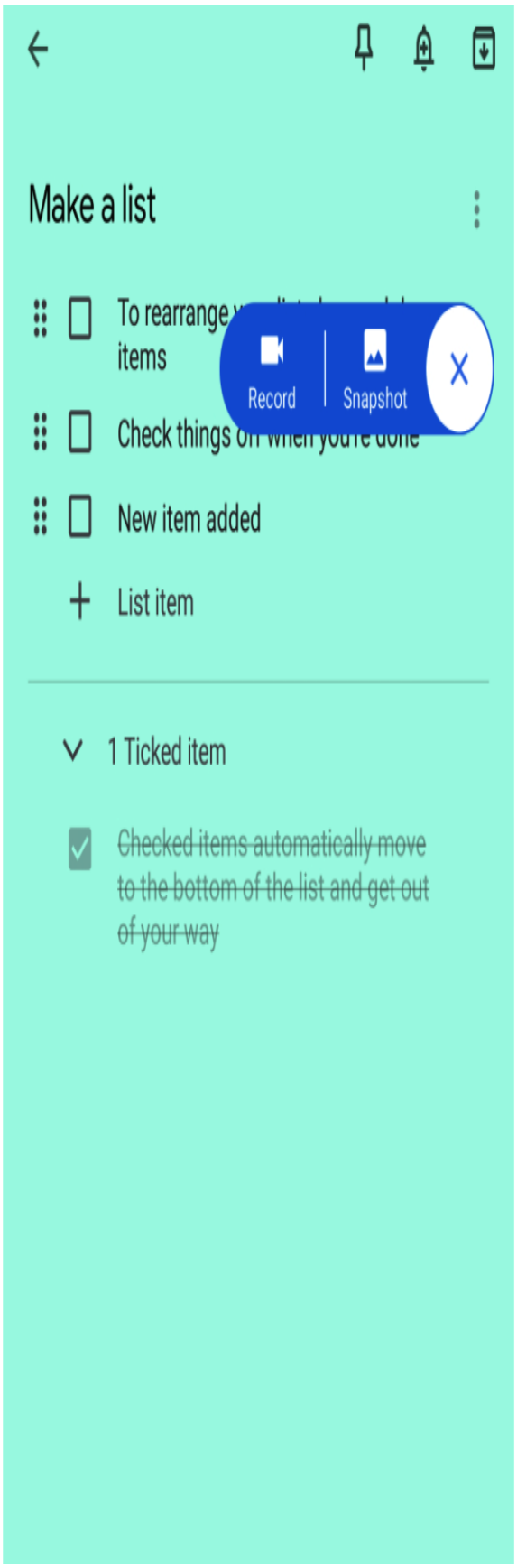
```
$ qark --apk ~/path/to/apk --report-type html
```

This will generate an HTML report file highlighting the vulnerabilities.

As mentioned in Chapter 7, automated security scanning tools such as these help software development teams to shift security testing to the left. However, depending on the security testing skill of the team and the context of the app, you might still need to engage professional security testers toward the end of development.

Accessibility Scanner

Accessibility Scanner is an accessibility scanning tool for Android apps available from Google Play. Once you've installed it on your device and granted it the necessary permissions, you can launch the app you want to test and tap the blue checkmark button to start the accessibility scan. It will then show the option to record the app's flow, as seen in Figure 11-11.


Make a list
Record
Snapshot
New item added
List item
1 Ticked item
Checked items automatically move to the bottom of the list and get out of your way

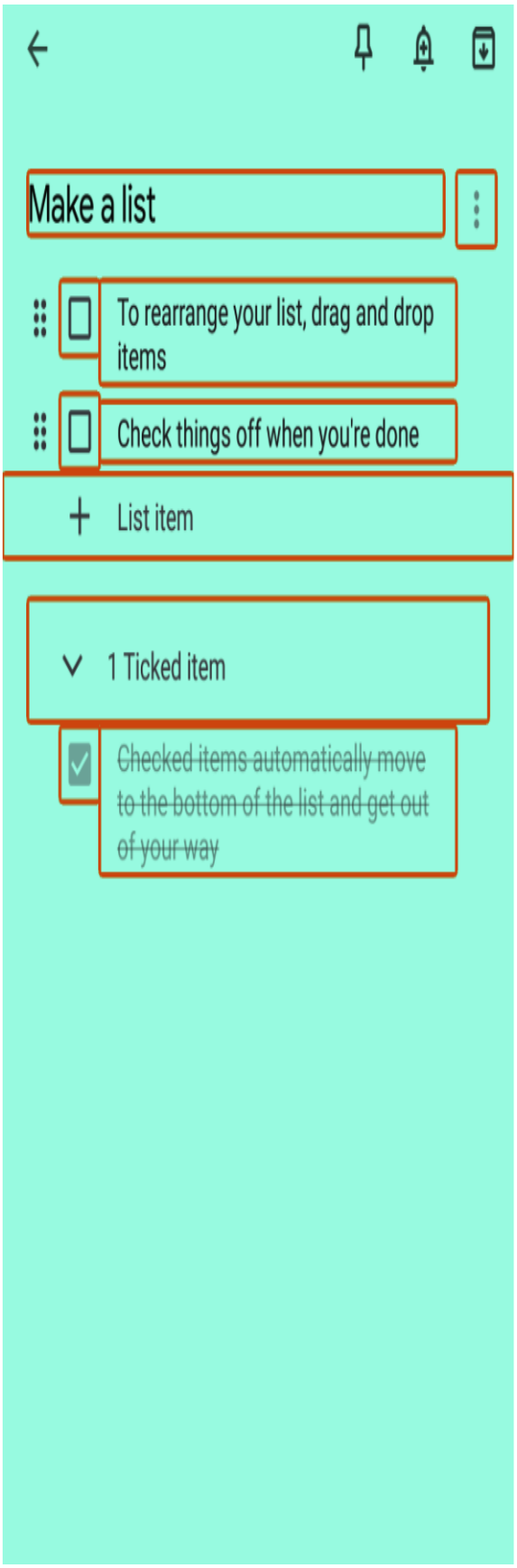

Make a list
To rearrange your list, drag and drop items
Check things off when you're done
List item
1 Ticked item
Checked items automatically move to the bottom of the list and get out of your way

Figure 11-11. Accessibility Scanner app results

Try it now—open any app on the Android device and start recording. Navigate through the app, and once you're done, stop recording by clicking on the blue checkmark again. The Accessibility Scanner app will then show the accessibility results for each screen you navigated through by highlighting the elements whose accessibility provisions can be improved, as seen in Figure 11-11. Clicking the highlighted text shows a description of the issues. This app can be used as early as in the development phase to check for missing accessibility features.

And with that, you deserve a big congratulations, as you have successfully navigated through a wide breadth of tools for testing various functional and cross-functional mobile app requirements!

Perspectives: The Mobile Test Pyramid

Having explored various automated mobile testing tools throughout this chapter, we need to see how they come together and if they can conform to the usual test pyramid. Let's quickly discuss.

As mentioned earlier, a mobile app's basic architecture is similar to a web app's: the app calls services, which in turn talk to the DB. So, the tests in the bottom layers will continue to adhere to the test pyramid. As you'll recall, the test pyramid recommends having a fat layer of micro-level tests and a thin layer of macro-level tests. However, there have been debates on whether it is possible to achieve the pyramid shape when it comes to the mobile layer. Some say that in their experience the mobile test pyramid instead takes an inverted form, with a fat layer of UI tests with extensive manual testing and a thin layer of unit tests. Their experiences mainly vary because of two factors: the restricted scope of unit and UI tests in the mobile layer and the app's context.

To elaborate further, when it comes to unit tests in the mobile layer, as usual, they validate small pieces of functionality delivered by a class or method. However, for functionalities that depend on the OS's APIs, unit tests may not be written to validate the API behavior, as it is understood that the OS provider will have tested the API. This imposes a need for end-to-end testing, be it manual or automated, to ensure that the OS's APIs work as expected within the app's context across different device types. Additionally, hardware-specific features such as integrations with the camera, sensors, and so on and usability-related features such as ease of scrolling, advanced gestures, etc. cannot be thoroughly tested either as part of unit or UI tests. These aspects mandate manual testing efforts.

Given this restricted scope of unit and UI tests, the shape of the mobile app's test pyramid is dictated by the characteristics of the app. For apps that contain extensive functional logic and have few dependencies on external factors such as the device's hardware and the OS's APIs, the mobile test pyramid will continue to look similar to the traditional test pyramid as there will be more lower-level tests that cover the functional logic. On the other hand, for apps that have limited functional logic but heavy dependencies on external factors, you might have an inverted test pyramid. In such cases, you will have to plan for additional testing capacity and strive to achieve a balance between writing more automated UI tests and performing exhaustive manual regression testing.

Key Takeaways

Here are the key takeaways from this chapter:

- With individuals' growing dependency on the mobile sphere and the profit it brings to businesses, we can expect to witness a continuing uptrend in mobile app development in the years to come. So, as software developers and testers, we should prepare ourselves by acquiring relevant mobile testing skills.

- Mobile testing differs from web testing in many regards. This chapter introduced the nuances of the mobile landscape by viewing it through three different lenses, focusing on considerations to do with the devices, the apps, and the network. The diversity in each of these areas poses a significant challenge to software development teams in all phases of the development lifecycle, including design, development, and testing.
- Your mobile app testing strategy should incorporate the fundamentals of full stack testing, such as a heavy focus on micro- and macro-level testing, shift-left testing practices, and testing for various quality dimensions such as security, performance, and accessibility.
- The exercises and discussion of additional testing tools provided a detailed overview of a variety of functional and cross-functional manual and automated tools that can be put to use for mobile testing.
- The shape of the mobile test pyramid is sometimes inverted, depending on the app's characteristics and features. This is an important aspect of mobile testing that should be considered early so that appropriate testing capacity can be planned.

---

1 may also see the designations LDPI, MDPI, HDPI, XHDPI, XXHDPI, and XXXHDPI, where DPI refers to dots per inch, one way of measuring pixel density.

Chapter 12. Moving Beyond in Testing

Practitioners follow directions; experts understand principles!

So far, we've discussed all the testing skills a software professional should possess in order to successfully deliver high-quality web and mobile applications. We've established that testing is a broad and growing space that has evolved over decades to include new processes, tools, and methodologies. While there are 10 different lenses that testing skills can be viewed through today (the 10 full stack testing skills outlined in the previous chapters), tomorrow there could be more. However, even in such a dynamic environment, the foundational principles in testing will remain unchanged, irrespective of the technology or the domain. Understanding these first principles in testing will provide you with the framework and knowledge you need to succeed, regardless of how the testing space continues to grow in the future.

In this chapter, I will provide a brief overview of these first principles in testing and their critical benefits, and take a look at how the existing tools and team practices have evolved based on these principles. We will also explore how an individual's soft skills add to their technical skills in contributing to their team's overall success in delivering high-quality software.

First Principles in Testing

Figure 12-1 shows the seven first principles in testing. In the following subsections, we'll dive into each one in turn.

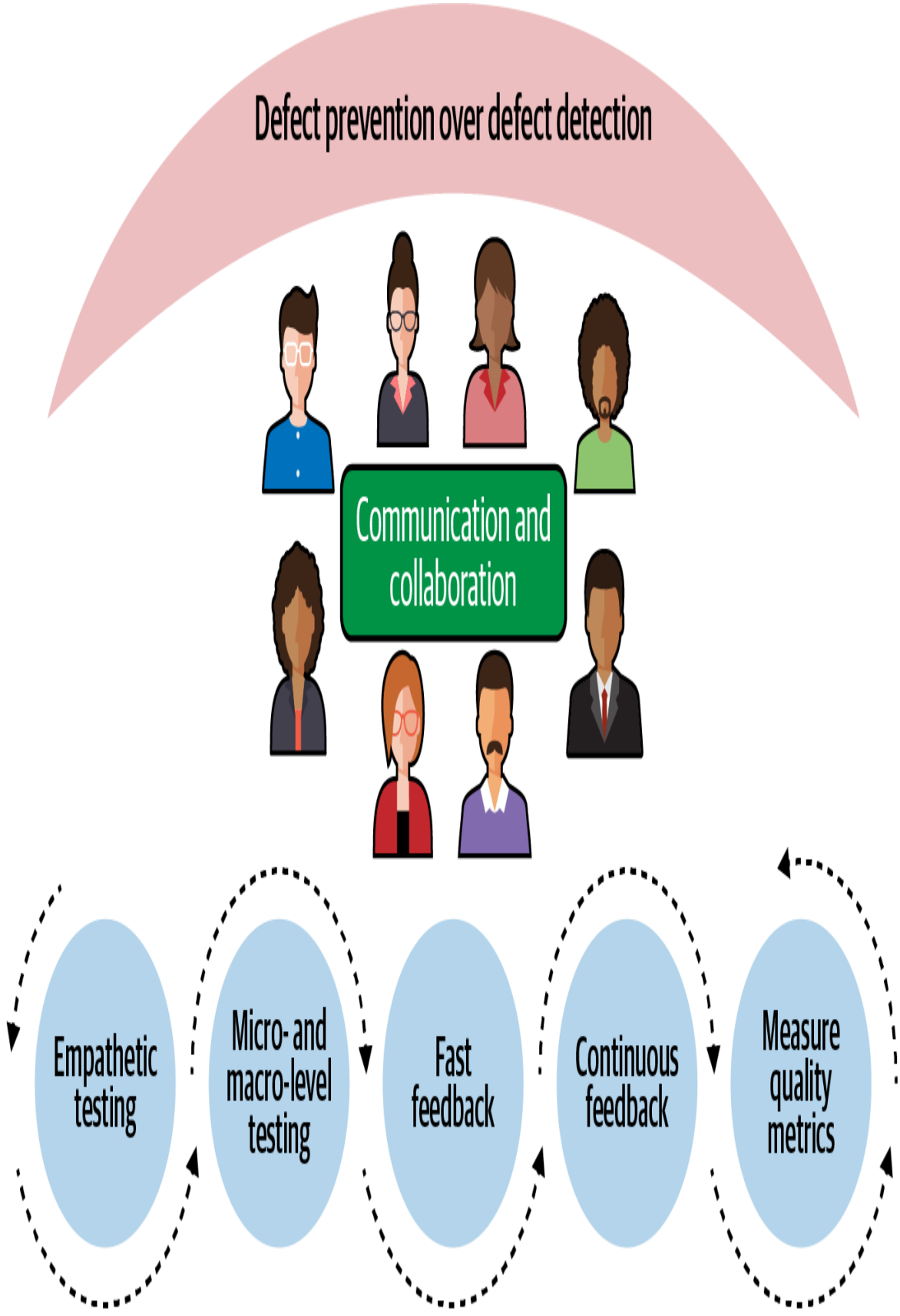

Defect prevention over defect detection
Communication and collaboration
Empathetic testing
Micro- and macro-level testing
Fast feedback
Continuous feedback
Measure quality metrics

Figure 12-1. First principles in testing

Defect Prevention over Defect Detection

Even though testing mainly aims to find issues in the application, defect prevention should be considered the core of testing. An obvious reason why we should avoid defects is the cost of fixing them. We can compare such fixes to plastering over a crack and touching up the paint on an otherwise seamlessly painted wall—sometimes the newly painted patch doesn't blend in well, and we have to paint the whole wall all over again! Similarly, software defects could lead to significant architectural changes, requiring a huge amount of rework and cost. Hence, this core principle suggests adopting practices, tools, and methods that will allow defects to be prevented from occurring in the first place, rather than detected and repaired.

Some practices in today's software world that aim to fulfill this principle are:

- Iteration planning meetings (IPMs), which are conducted at the beginning of an iteration or sprint to discuss the user stories in detail. An IPM is an open space for teams to brainstorm about missing integrations and edge cases in user stories.
- The three amigos process, where the business representatives, developers, and testers mull over each feature thoroughly during the analysis phase. The process aims to collect all three roles' perspectives so that integrations, edge cases, and other business requirements don't get overlooked.
- Similarly, the user story kickoff aims to repeat the three amigos process just before the user story's development begins. It is also a common practice in shift-left testing for the testers to capture and discuss the test cases during story kickoffs, for the same reasons.

- Architecture decision records (ADRs) and test strategies are discussed and documented to steer the team toward the project's collective quality goals.
- During development, test-driven development (TDD) triggers the thought processes around edge cases for even small pieces of code.
- Pair programming is another development practice aimed to prevent bad code leading to defects and to avoid missing edge cases.
- Similarly, linting software catches coding smells even while developers are creating them.

As you can see, there are many practices that focus on defect prevention. This approach can be applied to any new domain, like data, even when the roles in the team are different.

Empathetic Testing

Testing is all about placing oneself in the role of the end user, at the end of the day. When we are wearing the testing hat, we should operate with the end user's interests at heart, overcoming the day-to-day distractions of critical business needs and technical implementation details. We cannot restrict ourselves to just verifying the user story's acceptance criteria and moving on; we have to explore the application in the way that a typical end user will consume it. Consequently, understanding and empathizing with the different end user personas that an application is targeted to is vital before starting to test. Often, teams make trade-offs on end user needs based on factors like the complexity of development and timelines. However, when playing the role of a tester, we should bring the end user's perspective to the forefront and negotiate the trade-offs with their best interests in mind. Even though we work with

our teams day in and day out, while testing we should put our end users first.

Micro- and Macro-Level Testing

As discussed in Chapter 1, testing must be done at both the micro and macro levels to deliver high-quality software. To recall, micro-level testing involves drilling down to a small piece of functionality and testing that in detail—for example, testing order total calculation with various boundary conditions (negative prices, long decimals, etc.). Macro-level testing uses a broader lens to cover the functional flows, data propagation between modules, integrations between components, and so on. For example, macro-level tests might focus on testing the order creation flow, third-party integrations, UI flows, failures in order creation, and so on.

We saw an elaborate automated functional testing strategy with various types of micro- and macro-level tests in Chapter 3. In summary, unit, integration, and contract tests focus on micro-level testing, while API tests, UI functional tests, visual tests, and so on focus on macro-level testing. We also discussed how an imbalance in the distribution of micro- and macro-level testing can delay feedback, leading to a compromise in quality.

Another critical consequence of such an imbalance will be the detection of unanticipated issues in production. This is because, in general, teams that focus only on macro-level testing overlook details. For example, they will have tested macro-level scenarios such as the order getting created successfully and a failure in order creation due to item unavailability. But when the item prices in production are negative or have an unexpected number of decimals, order creation might fail, leading to defects. So, zooming in and out constantly to ensure you get both the big picture and the details while testing is essential.

Fast Feedback

This principle is about finding defects early so that the defect fixing cycle and, as a result, the release cycle can be faster. There is a notable correlation between the time it takes to fix a defect and how late it is found. When a feature is in development, the developer has all the required context on the code and can easily understand bugs' root causes and quickly fix them. But when the developer moves on to other features and the code base continues to grow every day with refactoring, that context is lost, and debugging the root cause becomes a longer and costlier process.

Furthermore, the defect tracking cycle contributes significantly to the delay in the defect fixing cycle. For instance, imagine a high-priority bug found two weeks after feature development. Time and effort will have to be expended on creating bug cards, triaging them, tracking them in iterations, and finding the right developer with time to fix them. These tasks can take days, if not weeks! And after all these delays, in certain worst-case scenarios you may find that it is impossible to fix the defect without a major refactoring due to the development that happened in the interim period, thereby delaying the release timeline. This is the ultimate cost to pay for a defect, and also the main reason why you should focus on creating faster feedback cycles in your teams.

So, how early should a piece of code be tested to give fast feedback? Shift-left testing is all about faster feedback, and we have seen how to implement this in each of the earlier chapters. To recollect some of the team practices that yield faster feedback discussed in Chapter 2 through Chapter 4, you can implement dev-box testing (running automated tests as early as on the developer's machine) and the test pyramid. Also, user story sign-offs by product owners (POs) or business representatives and showcases to all stakeholders upon completion of every sprint will get the team faster feedback on missing business cases.

In a nutshell, testing to give faster feedback is equivalent to harvesting at the right time. When the timing is delayed, you have to settle for a lower-quality harvest.

Continuous Feedback

Fast feedback should be backed by continuous feedback. It is not enough to just test a feature once and then leave it idle until release. You have to continue regression testing to get feedback on whether the current features and their integrations are still intact as the team continues development of new features and refactoring of existing code. Such continuous feedback mechanisms help in catching issues when they are still relatively small, which prevents disruption of release timelines. Continuous feedback also places the team in the sweet spot of being able to do continuous delivery!

As discussed in Chapter 4, the predominant way to get continuous feedback is by implementing continuous testing practices. To highlight a couple of key pointers on continuous testing, run all the micro- and macro-level functional tests and the CFR tests for every commit as part of your CI pipeline. This will provide continuous feedback on all quality dimensions and thereby equip the team to do continuous delivery.

Measuring Quality Metrics

Anything that is measured tends to improve! The purpose of having KPIs in any field is to track these indicators and iteratively improve them by taking the right steps. So, when we are trying to achieve high-quality results, we should measure quality as well. That said, when disproportionate emphasis is placed on metrics, team members tend to find ways to fool the metrics and forget that ultimate purpose. Thus, the metrics should be deployed wisely to steer the team toward common quality goals.

Here are some quality metrics that will benefit the team when tracked regularly:

Defects caught by automated tests in all layers

Automated tests create a safety net for the team, and when a majority of defects are found in the early stages, teams feel more confident in making new changes. This metric also reflects the strength of the safety net.

Time from commit to deployment

As we saw earlier, faster feedback is critical to making progress. When the developer commits, the new changes should immediately be tested by the automated tests in the CI pipeline and deployed to the QA environment to kick-start exploratory testing. I have seen teams where the CI pipelines take a long time to generate a green build due to unstable tests and environment issues, delaying feedback and resulting in productivity loss.

Number of automated deployments to testing environments

This and the previous metric will show how quickly and successfully the team is able to make new changes. Ideally, you want the team to be set up with a good safety net that will enable fast and stable deployments. If you find that the number of automated deployments to the testing environments is low, due to infrastructure, test, or other failures, this is a sign that your feedback cycle needs improvement.

Regression defects caught during user story testing

Regression defects caught during the user story testing phase indicate missing business use cases or missing automated tests. For example, automated tests in CI will miss a refactoring of a SQL query to use `equals` instead of `like` if the input data to the test was designed to match both. As we discussed in Chapter 3,

when regression defects are found during user story testing it can be a symptom of teams following antipatterns in automated testing. Hence, they should immediately reflect on the root causes of such defects and improve their processes regularly.

Automation coverage based on the severity of test cases

Keep a detailed record of your automation coverage with the goal of having no backlog. Tracking this will help you plan your iterations in advance to cover the backlog, if any.

Production defects and their severity

Tracking production defects shows you the bigger picture of missing business use cases, missing configuration, data mismatches, and any other issues that the team may have overlooked. Identify their root causes and automate tests around them. Also, build a living test strategy and keep evolving it as the application and team venture into new ground.

Usability scores with end users

Collect feedback from end users on the overall user experience during the development phase itself. This will help you enhance UX design–related metrics (e.g., maximum number of clicks to obtain information, text versus icons, and more).

Failures due to infrastructure issues

Track infrastructure issues like services being down intermittently in testing environments, problems in the CI pipelines, mismatches in the testing and development environments' configurations, etc. Sometimes the infrastructure code may need technical debt to be paid down to make it scalable and stable.

Metrics around cross-functional aspects

Measure the performance KPIs consistently and show the results to your teams. Include statistics around automated security test cases and vulnerabilities found during the automated scans as part of your iteration showcases. Similarly, include automation coverage of your project-specific CFR test cases and present relevant metrics (cross-browser testing coverage, chaos engineering results, localization testing coverage, etc.).

Many of the metrics mentioned here tie back into the four key metrics discussed in Chapter 4, which measure quality in terms of the stability of the code and delivery tempo of the team. For instance, recall that one of the four key metrics, *lead time* (the time from code being committed to it being ready for production deployment), is expected to be less than a day for an elite team. When there is a good safety net of automation coverage, the team can make such rapid changes confidently.

Similarly, the *deployment frequency* metric needs to be "on demand" for an elite team. When we measure the time taken from commit to deployment and the number of deployments in a day to testing environments, we get a sense of the team's delivery tempo. Production defects will tell us about the *change fail percentage* (percentage of changes released to production that fail), which should be 0–15% for a high performer. When these metrics are tracked and discussed consistently, the team inherently chases the goal of high-quality software.

Communication and Collaboration Are Key to Quality

Testing cannot be done as a siloed activity. For testing to add value, there must be proper communication about business requirements, domain knowledge, technical implementation, environment details, and so on. This requires consistent collaboration and interaction

between all roles within a project team. Communication could be via Agile ceremonies like stand-ups, story kickoffs, IPMs, dev-box testing, and proper documentation like story cards, ADRs, test strategies, test coverage reports, and the like. While we cannot expect the communication to be synchronous in today's world, with distributed teams working in different time zones, we should work to ensure handovers go smoothly through proper documentation and asynchronous mediums like video recordings and emails.

To summarize, following these seven first principles will guide software teams in developing effective testing strategies even as they venture into new areas of the technology space. I have applied these principles myself in projects with new technology stacks and in unfamiliar domains and have seen them consistently produce high-quality results.

Soft Skills Aid in Building a Quality-First Mindset

At this point, it is essential to highlight once again that several aspects of software development—design, analysis, development, infrastructure, etc.—contribute to producing high-quality software. Testing for quality is one of these aspects, and a critical one at that. It is therefore necessary for all team members to work together toward the goal of achieving high quality. No one person can completely own quality, and neither can any single person *not* own quality—it's not dissimilar to how a relay team cannot win a race even if one runner slows. And soft skills play a crucial role in building a quality-first mindset in the team. If you are a tester by profession or are responsible for testing at work, here is a list of soft skills I would like to throw light on that will help in building a collaborative quality-first mindset within your team:

Ability to drive outcomes

> With every role in the team focused on driving quality outcomes, it's collectively set up to produce high-quality results. For

example, designing an intuitive user journey is owned by the UX role, envisioning a customer-friendly product is owned by the PO/business representative, and guaranteeing a good architecture and robust code is the responsibility of the developers. Along the same lines, the testers primarily should own the testing-related activities and drive the team to incorporate them into their day-to-day practices. For example, they are responsible for ensuring the team adopts defect prevention practices and tools, corroborating that the continuous testing practices are followed through on, tracking the automation coverage and getting it completed as part of every user story, and other practices described throughout the book.

Collaboration

Inculcating the mindset that quality is the team's responsibility can only happen through strong collaboration with all team members and clients or business stakeholders. If we are rigid in our ideas and apathetic about reaching out to other team members, we will not achieve high-quality results. For example, owning the test strategy collaboratively with the developers will go a long way toward meeting this goal, just as collaborating with the business representatives to discover missing test cases will help significantly with defect prevention.

Effective communication

Sometimes, the way we communicate makes all the difference between a task being successfully completed or not. Effective communication also means choosing a suitable medium and the right time to communicate. In particular, there should be regular and clear communication from the testers to the team regarding the overall product quality and what is required to achieve the desired level of quality.

Prioritization

Testing can become a never-ending activity if it is not prioritized efficiently. Sometimes, what seems like a small task from a development point of view calls for a disproportionate unplanned testing effort, throwing schedules into disarray. To avoid such situations, testers should prioritize the list of testing activities per user story well ahead of time and ensure the efforts they will require are accommodated within the iteration's capacity. This will pave the way for the team to successfully deliver features without sacrificing quality.

Stakeholder management

A project's stakeholders include clients, managers, teammates, tech leads, and anyone else who can change the required course of action. We have to manage the stakeholders' expectations about quality consistently. Clients could be expecting the automation coverage to be 100%, which may not be a realistic goal, and managers might be more interested in meeting release timelines than in quality. Managing and helping to shape these expectations up front through collaboration, effective communication, and prioritization will lead to collective success.

Coaching/mentoring

Onboarding new members is common in teams, and we can't expect the newcomers to know all the team's practices and tools at the outset. However, in keeping with the idea that quality is the team's responsibility, every single team member should be on the same page regarding the testing practices and tools. Therefore, in our capacity as testers we (along with all the other roles) should pair up with new team members to share our knowledge on these subjects and help them ramp up quickly.

Also, bear in mind that mentoring or coaching is an activity that goes beyond the initial project onboarding. It should result in continuous learning and improvement for the mentee/coachee,

especially in improving their soft skills, so that they can operate as quality champions themselves in the team.

Influence

Influence is important, especially when working with large teams and new clients. Without it, even if we lay out a wise testing strategy, it may not be implemented across the board in the way we would like. Influence is key in gaining support for the testing strategy, and for convincing business stakeholders to invest in new testing tools and practices. Of course, there is no set recipe for building influence, but being able to produce high-quality outcomes consistently, along with showcasing the previously discussed six soft skills, should go a long way toward this goal!

Soft skills can be harder to master than technical skills, and they require diligent practice day in and day out. But as you work toward proficiency in these skills, you may discover to your surprise that you are already quite good at some of them, and if you utilize them appropriately you will find that they are highly beneficial for your and your team's collective success.

Conclusion

We have come to the end of an extensive exploration of the testing skills needed to deliver high-quality web and mobile applications. At this juncture, it is my responsibility to point out that testing is a continuous learning journey. As you actively practice all that we have discussed here, you will continue to gain more insights. Also, as I pointed out at the outset, testing is a rapidly evolving field with new tools, processes, and best practices appearing all the time. This rapid growth may seem overwhelming at times. If so, take a step back and remember that all of these new developments fundamentally cater to one of the first principles, and learning how

and where they fit in is only a small step away. In the end, a simple blend of the full stack testing skills with your soft skills will place you well on your way to efficiently delivering high-quality software!

With that, we have also reached the tail end of the book. There is a bonus chapter after this, which discusses four emerging technologies and some of the testing aspects specific to them. It is meant to be a quick and breezy read with the intention to get the reader thinking beyond the realm of web and mobile applications.

While you decide on whether to venture there, I would like to thank you for treading this long trail with me. It shows your commitment to delivering high-quality software, which is truly commendable! I hope the book has given you effective guidance on learning new testing skills and thrown light on contemporary testing practices that you can put to good use at work. Until we meet again in our testing journey, all the very best, and thank you for giving me the opportunity to travel with you through this book! :)

Chapter 13. Introduction to Testing in Emerging Technologies

Technology's rapid changes can be exhilarating and dizzying at the same time!

Technology has taken giant leaps forward in the last decade. Many of the things we saw as kids in sci-fi movies are in front of us today—surveillance drones, fingerprint logins, smart assistants, fully immersive video games, and the list goes on. We hear so many buzzwords: AI, ML, human-centered AI, blockchain, AR, VR, MR, bots, and more! It is a challenge even to absorb them all at once. One way to assimilate this vast spread of technologies is to group them into themes, such as the following:

Human-like interactions

For a long time, all we had to interact with computers was a mouse and a keyboard. In today's world, these interactions have expanded to include touch, voice, gestures, and more. Fitbit and Alexa are here, interacting with us—and more precisely, talking with us!

Augmented intelligence

Technology is used to augment human intelligence, making our lives much easier. Smart assistants, personalized recommendations, and chatbots are a few examples of how technology has changed our lives irrevocably.

Platforms as standards

The current trend in technology is for data, services, infrastructure, and more to be abstracted to form technology platforms for reusability and scalability purposes. This enables continuous innovation of new products in alignment with market needs. So-called *super apps* like Uber, WeChat, Grab, and Gojek use platforms as their foundation.

Connected things

Let's stop thinking about humans for a moment. *Things* are now connected over the internet too! We live in a world where our phones, watches, and coffee machines all talk to each other.

Thoughtworks' *Seismic Shifts* podcast and *Looking Glass* report present detailed overviews of technological advancements if you want to explore further.

Though many of these technologies haven't become mainstream yet and hence skills for testing them are not must-haves, it is wise to be prepared before the wave hits. This chapter aims to give a brief introduction to four emerging technologies—AI/ML, AR/VR, blockchain, and IoT—and discuss the testing aspects involved with each of them. As may be obvious, each of these topics deserves a book in itself, and this chapter only aims to provide a preliminary look at what these technologies are and where they are headed.

Artificial Intelligence and Machine Learning

Artificial intelligence (AI) is a subfield in computer science that aims to use machines to perform tasks that are typically performed by humans, simulating human intelligence. *Strong AI*, in particular, is a theoretical construct that can do anything a human can. AI is exercised through machine learning (ML), another subfield in computer science based around the idea that computers can be

programmed to learn from experience rather than being explicitly programmed to perform a task in a set way.

The terms AI and ML are often used interchangeably. To draw a distinction, any program that shows human-like behavior can be called AI, but unless its behaviors are automatically learned from experience—i.e., from historical data—it's not machine learning. This will become much clearer shortly, when we talk about the machine learning programming approach.

Introduction to Machine Learning

Typically, when we develop an application, we code a sequence of instructions for the computer to execute—at least, this is how we have known computers to work so far. But to hear that computers can learn from their experience without being explicitly programmed is extremely intriguing. To demystify what this means, let's consider an example: a social media app's abusive content filter. This will help us to understand more precisely the difference between traditional programming and machine learning approaches.

To build an abusive content filter the conventional programming way, we would start by listing the criteria that identify content as abusive, code them as rules, and remove posts that trigger them. For example, we might write code to check for a list of keywords such as *suicide*, *sex*, *trigger warning*, etc. Similarly, we might check for known exploiters' user IDs, mark the content as abusive, and skip the feeds automatically.

But is that enough? When we code a rule to mark a list of words as abusive, abusers quickly introduce new words to bypass it. Similarly, when existing accounts are restricted, the abusive users set up new accounts to send content from. In such a problem space, where rules themselves are nondeterministic, writing a foolproof solution using the traditional programming approach is quite challenging. This is where machine learning extends a helping hand.

With the ML programming approach, as seen in Figure 13-1, we feed a huge amount of historic data labeled as either abusive or nonabusive into a machine learning model. This is called *training* the model. The model is fundamentally a mathematical algorithm, and it learns the differences between the two content types from the data. This, in a way, is similar to how the human brain learns. We get to see many apples of different sizes, shapes, and colors from different angles over the years, and we become adept at spotting an apple. Similarly, we also learn to spot the difference between an apple and an orange.

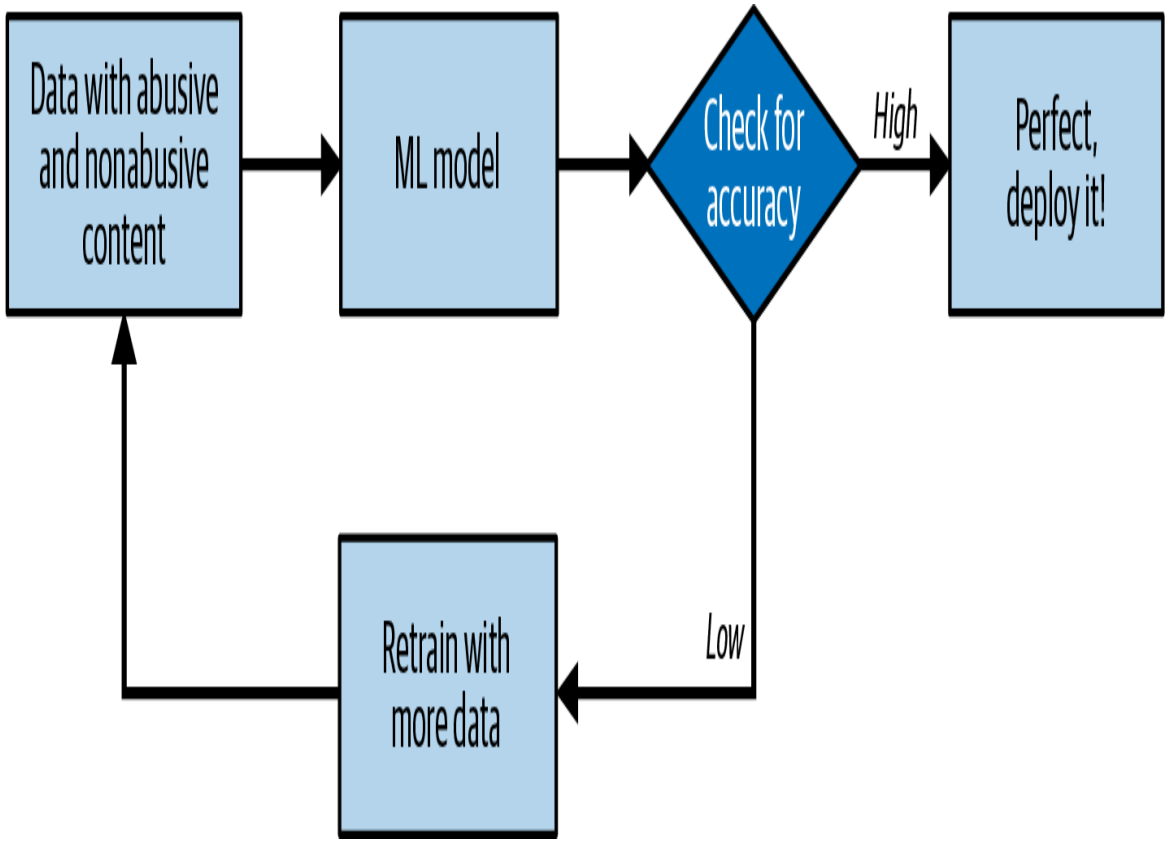


Figure 13-1. The ML way of programming an abusive content filter

Once the model is trained, it can tell us if a new post is abusive or not. It may not get all the answers right at the beginning, just like a human child wouldn't. We have to continuously train it with a more diverse set of data and keep evaluating the model's accuracy by testing with unlabeled data. The unlabeled data used for testing the

model's accuracy is called the *test set*, and the data used for training it is called the *training set*. Once the accuracy of the model is high enough, it is deployed to production. The model is also continuously trained with new content from production to ensure it can catch new words and variations.

> **NOTE**
>
> Machine learning with labeled data is called *supervised* learning. ML algorithms can also be trained with unlabeled data, in which case they try to learn patterns automatically from the data presented to them. This type of learning is referred to as *unsupervised* learning.

To summarize, the ML programming workflow starts with collecting a lot of data, tagging it appropriately, splitting it into a training set and a test set, using the training set to train the ML model, evaluating the model's efficiency with the test set, deploying, and continuing the training. Some popular machine learning frameworks that help with these tasks are scikit-learn, PyTorch, and TensorFlow. Machine learning has found applications in areas like medicine, banking, social media, and more, and is being continuously explored in newer domains as well. We'll touch upon the testing aspects next.

Testing ML Applications

Most ML applications adopt a typical service-based architecture with the ML component integrated into the services. In the content filter example, a service-based flow could be as follows: when a user creates a new post, the UI sends it to a content service to check with the model whether it's abusive or not. If the model identifies the content to be abusive, the content service instructs the UI to hide the content. So, along with the usual approach to testing a typical service-oriented architecture, we should include the following aspects to cover testing of the entire application:

Validating training data

The data that is fed to the model largely dictates the quality of the model. If the data quality is poor, the model quality will be poor. So, focusing on input data quality is critical for ML applications. Because we need a huge amount of data to train the model, it may be procured from various sources, such as public databases, scraping from public websites, user inputs from different websites, and even system logs. This usually leaves us with data in various forms and shapes—basically, a cluttered and chaotic mess. In our example, our data source was historical posts on social media. As well as text, these posts could contain images, videos, GIFs, comments, tags, and so on. Some of these could in turn be of different sizes, file formats, color gradients, and more. If we feed such inconsistent data to the model, it is hard for the model to focus on the features of the content that make it abusive, such as the keywords, and to learn the distinctions accurately.

So, the usual practice is to clean the input data, eliminate the noise, transform it into a standardized format, and then feed it to the model for training. This cleansing and transformation logic has to be tested thoroughly. A couple of basic test cases to give an idea of how this could work might be:

- When input data comes with different scales—for instance, numerical data could range from decimal values to exponentially large numbers—logic to clean the data and transform it to a uniform scale should be tested.
- When input data can contain null or empty values, they must be either replaced with default values or eliminated during the cleaning stage.

In general, data has a lot of domain-specific aspects that must be explicitly tested. For instance, social media posts might have a

set character limit, which has to be validated while checking the input data quality. Typically, teams also write unit tests for cleansing and transformation logic to automate some of these test cases.

Validating model quality

The model's quality is measured in terms of various metrics, such as error rates, accuracy, confusion matrices, precision, and recall. There are methods to calculate each of these. In our example we might use precision and recall, as described here:

- *Precision*, as the name suggests, refers to the model's ability to correctly predict a result (that is, the number of true positives out of the total number of true and false positives). For example, if the model identifies 100 posts as abusive and 99 are actually abusive, its precision index is 0.99.
- *Recall*, on the other hand, is the metric that tells us how many of the actual abusive posts were identified correctly by the model (that is, the number of true positives out of the total number of true positives and false negatives). If the model correctly identified 99 posts as abusive out of a total of 110 total abusive posts, its recall index is 0.90.

The ML frameworks mentioned earlier have built-in features to calculate these types of metrics, and we can write tests to fail the CI pipeline based on these metrics whenever a new model is checked in. MLflow is an open source tool that you can use to view the model performance for every version of the model.

Validating model bias

Poor-quality data is one thing to deal with, but bias in the model makes it worse. Recently, Twitter's image cropping ML algorithm faced public criticism as it preferred white individuals' faces over black individuals while cropping, leading to the company

abandoning the automatic cropping approach. Such biases percolate to the model from the input data. If the input data has a large sample set representing a particular demography, the model will be biased toward that demography. It is therefore critical to test both the input data and the ML model for biases. Facets is an open source tool that can help with this, by allowing us to visualize patterns in the input data.

Validating integrations

Integrations between the three layers—specifically the data and model layers and the model and API layers—have to be tested using the regular contract and integration testing approaches.

Focusing on these aspects should enable us to do continuous delivery as well. The discipline of Continuous Delivery for Machine learning (CD4ML) is discussed in detail by some of my colleagues in their article on Martin Fowler's website.

Blockchain

Sir John Hargrave and Evan Karnoupakis give a simple one-line definition of blockchain in their report *What Is Blockchain*: "Blockchain is the *Internet of Money*." If we consider money to be anything of value, such as stocks, bonds, reward points, etc., and the internet to be a platform for sharing information freely with our peers, then blockchain can be understood as a platform for sharing anything of value.

The name is derived from the way it works. Whenever a transaction (exchange of value) is made, a block is created with that transaction's data, chained to the previous transaction. By "chained," I mean that every block has a hash of the previous block's content, creating a chain of blocks, as seen in Figure 13-2.

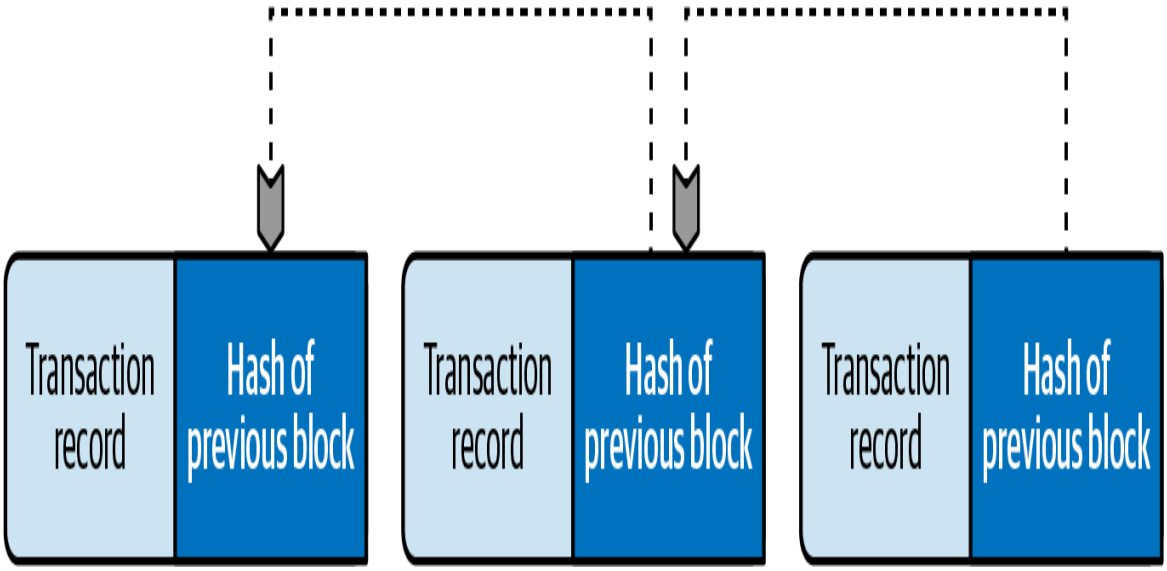


Figure 13-2. A chain of blocks with transaction data

This is how blockchain brings security in. If someone alters the content of a block, the next block's hash will not match, and hence the chain will be broken. We can therefore say that the transactions are immutable: new blocks can be added to the chain, but the existing blocks can never be altered. Typically, high-end cryptography algorithms like SHA-256 are used for hashing, making blockchain impenetrable to hackers.

What was the philosophy behind creating such an impenetrable system? In 2008, an anonymous author writing under the pseudonym Satoshi Nakamoto released a whitepaper, "Bitcoin: A Peer-to-Peer Electronic Cash System", that talked about a new concept called *digital money*, or *e-cash,* that could be transferred between parties without the involvement of a centralized agent, like a bank, in the middle. The thinking was simple: people work hard to earn money, and they should have control over it: money, for the people, and by the people! The thrilled dev community quickly set to work implementing the whitepaper, which evolved into the blockchain technology of today. To draw your attention to a couple of key points here, blockchain evolved to aid decentralization and promote peer-to-peer transactions. Security was required to be a part of it, as the technology intended to deal with money.

Introduction to Blockchain Concepts

Now, we will discuss the building blocks of blockchain to get an idea of how testing can be implemented:

Decentralized ledgers

A *ledger* is a repository holding all the accounting data (the inflow and outflow of a transaction). Blockchain uses decentralized ledgers; i.e., the ledger is not owned by one person but by all participants. Any party intending to make a transaction will get a copy of the ledger. The advantage is that it is trustworthy, since no one person can manipulate the records. However, this incurs an additional cost of keeping all the ledgers in sync at all times.

Nodes

A node is any computer or server that participates in the blockchain network. Nodes can belong to a single individual or a group of individuals. Each node stores a copy of the decentralized ledger. When there is a new transaction, they each update their copy of the blockchain. As seen in Figure 13-3, the nodes communicate with each other to keep the ledgers in sync. This process relies on something called *distributed ledger technology* (DLT).

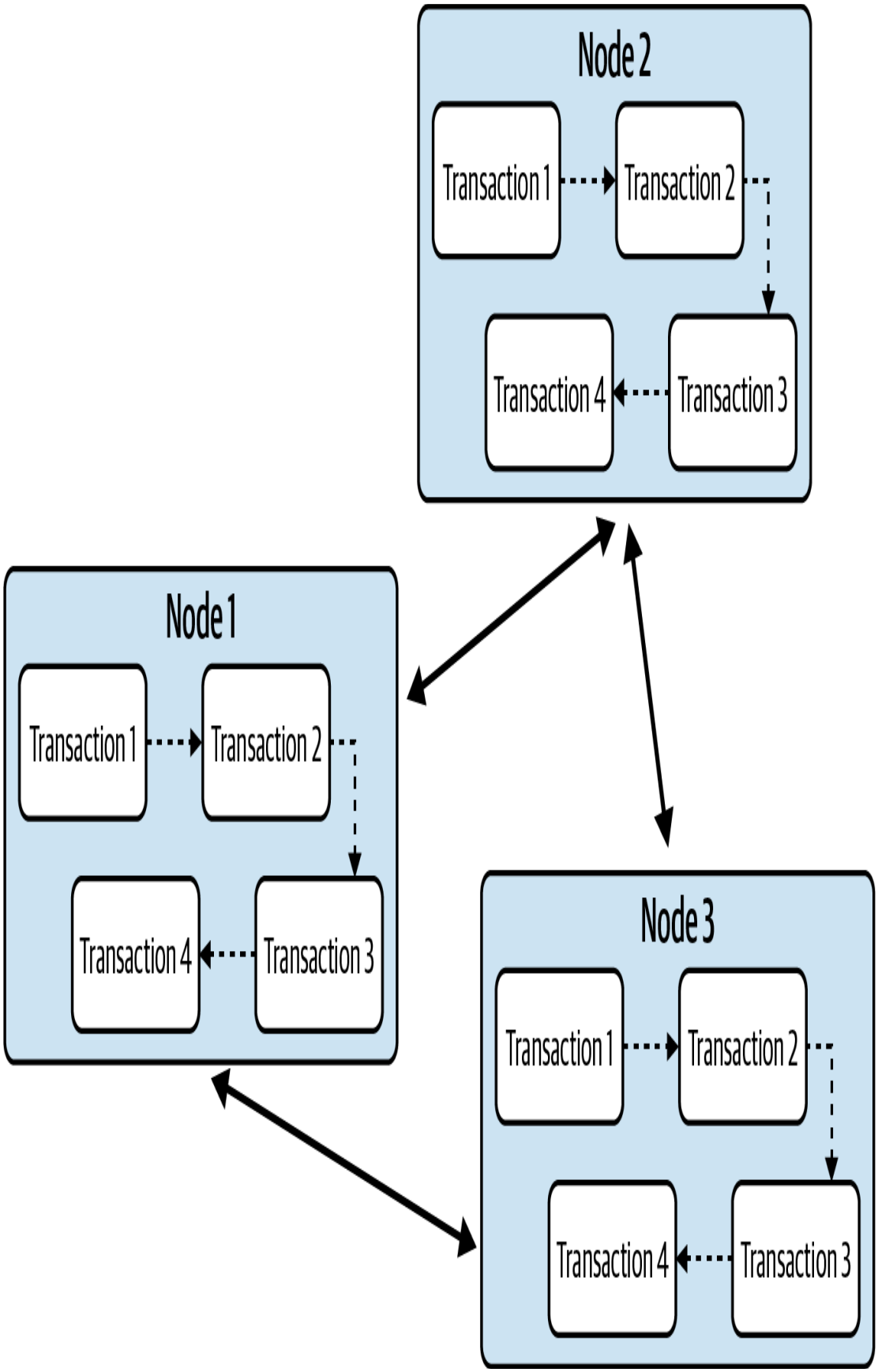

Node 2
Transaction 1
Transaction 2
Transaction 4
Transaction 3
Node 1
Transaction 1
Transaction 2
Transaction 4
Transaction 3
Node 3
Transaction 1
Transaction 2
Transaction 4
Transaction 3

Figure 13-3. Distributed ledger technology with nodes each holding a separate copy of the blockchain

Consensus

In blockchain, we have decentralized ledgers with the accounting data and nodes providing the needed infrastructure to hold them. A bank is a centralized authority that can add or delete customers' transactions after verifying their integrity, but in blockchain all the nodes are equal participants—so who can add new transactions to the chain? This is where *consensus* comes into play.

Consensus is the process by which nodes collectively agree to add a transaction. To get this done programmatically, we have consensus algorithms like *Proof of Work* and *Proof of Stake*. In the Proof of Work algorithm, an extremely complex mathematical problem is given to the nodes to solve. The first node that gets the correct answer is given the authority to add the new block. The other nodes in the network verify the integrity of the new block before adding it. When a node adds a new block, it is rewarded with digital currency (a process referred to as *mining*). The downside of this algorithm is the need for large amounts of computational power to solve the complex mathematical problems. With the Proof of Stake algorithm, nodes have mining power proportional to the amount of digital currency they control. The downside here is that the nodes with the largest stakes continue to get the privilege and get richer.

Smart contracts

A banking system has a set of established rules and conditions to execute a transaction successfully. For example, before approving a housing loan, the bank first verifies your salary, account balance, the housing documents, etc. In blockchain, the logic required to complete a transaction is written as a *smart*

contract. Every node gets a copy of the smart contract as well. The advantages of this approach are that it enables paperless transactions, eliminates commissions for go-betweens, and makes it easy for the respective parties to complete a transaction independently.

Those are the building blocks of the blockchain. To bring it all together and get an idea of the overall workflow, let's take a look at an example. Suppose Alice wants to buy some tomatoes from Bob for 10 Ethereum (a popular cryptocurrency). Alice initiates a transaction and transfers the money. The smart contract holds the money from Alice until Bob delivers the tomatoes. As proof of delivery, Bob might produce a QR code to be scanned. When Alice scans the QR code, the transaction completes, and the smart contract transfers the money to Bob. If Bob fails to deliver the tomatoes, the smart contract returns the money to Alice after a set period. In the meantime, the nodes on the network compete to solve the mathematical problem and gain the right to add a new block for this transaction. The winning node also gathers the transaction records from the smart contract to be added to the block. Once the block is added, it is synced with all the other nodes.

The blockchain development frameworks that enable all of this include Ethereum, HyperLedger Fabric, and Stellar. OpenZeppelin and Solidity are used to write smart contracts. MetaMask is a wallet to hold digital currency (specifically, Ethereum).

Testing Blockchain Applications

Having discussed the overall workflow in blockchain technology, here are a few focus areas for testing:

Functional testing

The first step when testing any application is validating the end-to-end functional flows, like in the tomato purchasing example.

The functional logic is written in the smart contracts, so we need to look for loopholes in that. The test cases to validate the smart contracts can be added as unit tests too.

API testing

Most often, there are APIs on top of the blockchain, connected to the frontend. We should focus on the standard API layer testing: functionalities, integrations between modules, contract versioning, error handling, retries, etc.

Security testing

There are a lot of security aspects involved here that need to be tested, from account creation and authorization mechanisms for the individuals involved in a transaction to exchange of currencies, account balance maintenance, checking for illegitimate transactions, and the cryptography aspects like hashing the blocks.

Performance testing

Given that the transactions rely on the availability of nodes and consensus algorithms, the time it takes to complete a transaction can be longer than with a standard web technology. Hence, transaction performance and the functional behavior to handle the delays must be tested.

BLOCKCHAIN-SPECIFIC TESTING

Most applications use existing blockchain networks such as Ethereum to deploy their smart contracts, and hence you may not need to test the blockchain's features in particular. However, if this does become necessary, some aspects to look out for are:

Addition of transactions

Every transaction should be recorded without loss of information. This is the most critical requirement in the blockchain. The blocks should be correctly chained and synced with all the other nodes.

Block size

The transactions are clubbed into the same block until the block size reaches an upper limit (in the Bitcoin network, for example, the original block size was 1 MB). We need to test if a new block is created when the block size reaches its limits.

Chain size

As the number of transactions grows, the chain becomes very large. We need to check the application performance with such large chain sizes.

Node testing

Nodes are foundational blocks in the blockchain. Nodes should be able to participate in consensus and be synced with the latest data all the time. New nodes should be able to join the network seamlessly.

Resiliency

When nodes become available after a short outage, they should be able to smoothly integrate back into the network without disrupting the application's functionality. If no nodes are available for a while, the application needs to handle the outage gracefully.

Collisions

There could be situations where more than one node has solved the mathematical problem and they are fighting for the right to add a new transaction. We need to test for such collision scenarios.

Data corruption

A Byzantine node is a node that misbehaves in a decentralized system. When that happens, the data across the nodes could get corrupted. There are proven ways to handle such scenarios, and that behavior needs to be tested.

Tools like Ethereum Tester and Populus are useful for testing Ethereum-based blockchain applications, and bitcoinj and testnet help to test Bitcoin transactions.

As you can see, blockchain technology has significant advantages in terms of strong security, fully digitalized transactions, elimination of middlemen, and combating monopoly. However, there are also some disadvantages that make it harder to adopt this technology. For example, blockchain requires large amounts of computational and

electrical power to solve the complex mathematical problems and synchronize all the ledger data, and because of the consensus algorithms and intermittent availability of nodes, it may take a long time to complete a transaction. Visa reportedly handles about 1,700 transactions per second, whereas it may take 10 minutes for a single blockchain transaction to be confirmed. Accordingly, performance is a major bottleneck.

Internet of Things

The Internet of Things (IoT) is the technology that connects the physical world to the digital world. It enables devices ("things") around us to gain intelligence and start communicating with each other and with us over the internet. This intelligence also enables devices to react autonomously to changes in their surroundings without human intervention. For example, smart thermostats adapt to atmospheric conditions such as humidity and set the right temperature based on the user's preferences. The IoT has proven itself as a solution to both small and large-scale needs. A famous example in the domestic sector is the smart home solution; the global value of the smart home market is expected to surpass $53 billion in 2022. At the other end of the spectrum are smart city IoT solutions that strive to enhance residents' overall quality of life by improving infrastructure, air quality, transportation, energy consumption, and more.

IoT devices are commonly provisioned with three features: a sensor, an actuator, and a communication medium. Sensors detect physical states, like temperature, pulse rate, motion, etc. The actuator triggers changes to the current environment, like raising an alarm when smoke is detected or opening and shutting valves to control temperature. Communication mediums, such as digital displays and voice, help the IoT devices present information to the user.

Building an end-to-end IoT solution requires both hardware and software skills. A software component is embedded inside the hardware to control its functionalities and to relay information to the users. Another software component resides outside the hardware, and aggregates and analyzes the data sent from multiple devices to take collective actions. For example, to read the user's pulse rate using a fitness device, the software inside the device triggers the hardware sensor to measure the count and relay it to the digital display. The software also sends this information to the cloud, where a service performs pattern analysis on the pulse rate information, sleep cycles, etc., and instructs the embedded software to raise an alarm when anomalies are detected.

For all of these technologies to work together cohesively—sensors, networks, communication and routing protocols, data processors, end user applications, the cloud, and more—a lot of end-to-end integration is required. A closer look at the IoT's five-layer architecture will help you understand these integrations more clearly.

Introduction to the IoT's Five-Layer Architecture

There are varying views on defining the number of layers in an IoT architecture—three, four, or five. The five-layer architecture, as seen in Figure 13-4, gives a broader and deeper view of the technologies involved in building an end-to-end IoT application. We will take a look at each of these layers briefly in order to understand the testing aspects in them.

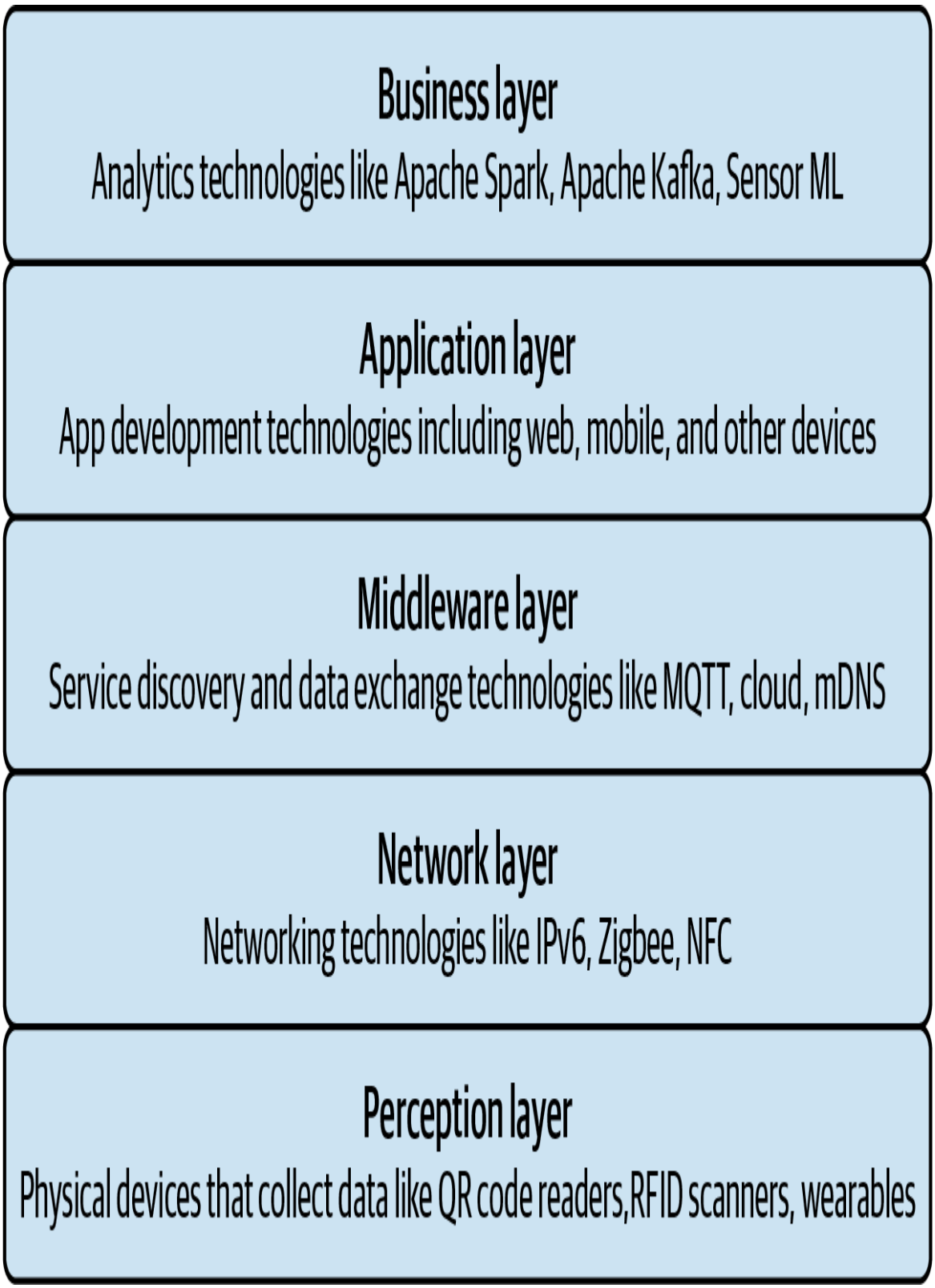


Figure 13-4. IoT five-layer architecture

Let’s briefly take a look at each of these layers in order to understand the testing aspects involved in each of them:

Perception layer

This is the bottommost layer, where the hardware reads information from the physical world and transfers it to the following layers. Hardware can be categorized as passive, semi-passive, or active, depending on its support for unidirectional or bidirectional communication. For example, QR code scanners fall under the passive category as they can only communicate one way, and the range of communication is limited. This is sufficient for scenarios like shipment tracking. Also, note that passive components don’t come with the power capacity to perform computations. Active components can receive and transmit data, and they are equipped with the requisite power capacity. Some examples are smart actuators that do mechanical tasks, wearables with embedded sensors, GPS radios, etc. They can communicate over longer ranges too.

Network layer

The physical devices have to be identified on the internet for other devices to communicate with them. IPv4 and IPv6 are popular networking protocols that provide unique IP addresses to devices (IPv6 is preferred). To efficiently transfer data to the destination address, the devices use routing protocols like the Routing Protocol for Low-Power and Lossy Networks (RPL). They use standard communication technologies like WiFi, Zigbee, near-field communication (NFC), and Bluetooth to transmit and receive information.

Middleware layer

IoT applications should be able to access the physical devices using their names or addresses to request their services (such as reading the room temperature, the user’s heart rate, etc.) without

knowing the underlying infrastructure details. The middleware layer helps with such service discovery. It also handles extracting data from the physical devices and communicating information back to the users. This is the core of the IoT solution. Service discovery protocols like Avahi and Bonjour and data exchange mechanisms like the Constrained Application Protocol (CoAP) and Message Queuing Telemetry Transport (MQTT) are widely used.

Application layer

This layer enables end users to access the desired services via a simple interface like a web or mobile app without knowing how service requests are processed in the underlying layers. The application layer includes the logic to aggregate, process, and store information from multiple devices.

Business layer

This layer analyzes the information gathered from the hardware, services, etc. to improve the application's services. Big data technologies like Apache Spark and Apache Kafka are used to analyze the vast amounts of data received from different IoT devices. This layer is mainly for the internal administrative folks and not the end users.

IoT platforms like AWS IoT and IBM Watson combine many of these capabilities to ease IoT development.

Testing IoT Applications

Some specifics to focus on while testing IoT solutions are as follows:

Hardware/software integration

The end-to-end functionality of any IoT application mainly relies on proper hardware and software integration, and that needs to

be tested with various edge cases. For instance, the heartbeat monitoring app in a smartwatch needs to display the correct heartbeat count as recorded by the sensor, and when there are issues in recording the heartbeat, the software should handle errors appropriately. These integration test cases should be tested after new installations and hardware or software upgrades. Furthermore, the usual hardware constraints in terms of memory and battery must be accounted for while testing functional features.

Network

Network connectivity between devices and with the cloud is an important aspect of IoT solutions that should be tested properly. Some devices can support multiple communication protocols, such as WiFi and Bluetooth, and those capabilities must be tested independently.

Interoperability

Interoperability in an IoT solution refers to the ability of different devices to exchange information with one another, even though they may follow different standards and protocols. For example, in a smart transportation IoT solution, traffic sensory devices, accident detection services, and automatic routing systems must be able to exchange information seamlessly, even though each could operate individually with different sets of technologies and protocols. Interoperability truly unlocks the potential of IoT, but the integrations must be carefully tested.

Security and privacy

Some communication protocols, like Z-Wave, may not always be secure, so there is a need to employ additional lightweight security mechanisms such as IPsec to prevent attacks. Furthermore, the data collected and stored in the cloud must be

private by design. Not only is it unethical to store individuals' biometric and other private data without their consent, but (as we saw in Chapter 10) there are legal requirements relating to the storage of personal information, and we have to test for compliance.

Performance

Performance is an important quality aspect in IoT solutions, as there could be many devices talking to each other and relaying information back to the aggregator services. So, we need answers to questions like how quickly the hardware responds to the software commands, what the overall response time for a service is (like getting the pulse rate), and what the data collection performance is like when there are many devices in the network (like in a smart city).

Usability

Usability is critical, especially in the domestic sector—for example, with devices like smartwatches and smart TVs. With such devices, there may be many aspects to test. Smartwatches respond to wrist movements, have displays of different sizes, may be operated with different buttons and gestures, have alert systems consisting of vibrations and sound notifications, can be worn on the right or the left wrist, and more. Onboarding the users to a device's capabilities is also an essential part of the product. Testing the ensemble of usability aspects is critical to the product's success.

From my own experience of working on a smart coffee machine, I can testify that testing IoT solutions is incredibly complex due to the varied combinations of devices and their internal states. To help manage the multitude of states and device combinations and derive

test cases, I formulated a testing framework called the IoT Testing Atlas, which you may be interested in exploring further!

Augmented Reality and Virtual Reality

Augmented reality (AR) is a technology that superimposes graphics, texts, images, and other sensory information onto the real-world environment and presents it to users to enhance their overall experience. It was initially invented to assist jet plane fighters; the pilots had to attack targets with precision while flying, and AR presented the necessary details on their frontal displays to help them focus on both tasks simultaneously. One of the latest examples of AR is the heads-up display (HUD) developed by Mercedes, which casts information like maps and acceptable speed limits onto the vehicle's windshield.

These days, we have games that use smart glasses, HUDs, and various mobile and handheld AR devices to give us the AR experience. You may have heard of or tried wearable smart displays from makers like Google, Vuzix, Epson, and Nreal. However, AR-enabled smartphones are the closest to us. Both Android and iOS are equipped with the needed tools and frameworks, like ARCore, ARKit, and Unity's AR Foundation, to enable this technology, and there are many phones with AR-compatible hardware (such as the Pixel 5, Nokia 8, Moto G, etc.).

Whereas AR augments your real-world surroundings, *virtual reality* (VR) transports the user to a simulated virtual world. In addition to popular applications like gaming, this technology is beneficial for simulating hazardous environments such as fires or air attacks and training the fighters to deal with them. VR has also gained popularity in the commercial space, where customers are offered product customization features like designing the interiors of their new house and real-time product experiences like virtual dressing rooms.

VR experiences require head-mounted display (HMD) devices for a fully immersive experience. Some popular ones in the market include the Oculus Quest, Oculus Go, HTC VIVE, and Sony PlayStation VR. Once again, smartphones with solutions like Google Cardboard offer a more accessible and economical option.

In addition to AR and VR we also have *mixed reality* (MR), which is a combination of AR and VR that allows users to interact with digital content in 3D. The Pokémon Go game is an example of MR. Similarly, *eXtended Reality* (XR) is where AR, VR, and MR devices integrate with other devices, like home appliances, sensors, etc. The AR, VR, MR, and XR space is expanding, and is definitely one to watch out for.

Testing AR/VR Applications

AR and VR tech is fascinating and gives the users an exhilarating experience—but it's also incredibly complex. Developing and testing these products requires expertise in a wide range of areas, from biology (human perception of images, mechanics of image formation by the eye, depth perception, etc.) to spatial mathematics, head-mounted display technologies, and so much more. There are development platforms like Unity that have abstracted these complexities to a certain extent for our ease. Also, there have been many improvements in the quality and performance of HMDs over the years.

That said, there are still not enough testing tools or an established testing approach in this area. Testing is customized contextually. A recent development by Thoughtworks is the functional test automation tool for Unity called Arium. Arium is open source and is available as a Unity package. Let's briefly look at a few concepts from Unity to understand how to test them using this tool.[1]

A *scene* in Unity represents a game environment. Typically, each level in a game is called a scene. Each scene has its own collection

of objects. A *GameObject* is an element in the scene. It could be a prop, like a ball, or a player. These objects' abilities can be programmed by attaching *components* to them (a component is any feature or functionality that is linked to a GameObject). Unity provides many built-in components for fundamental needs like casting light, collisions, and so on. For instance, we could attach a light component to a GameObject to define the lighting on that object. Every GameObject also has a default *Transform* component which represents its position, size, and rotation.

Arium provides the following functions to enable functional test automation of Unity apps:

- `_arium.FindGameObject("Ball")` to find a `gameObject` by name
- `_arium.GetComponent<name_of_component>(<name_of_gameObject>)` to retrieve components from the `gameObject`, which can then be validated
- `_arium.PerformAction(new UnityPointerClick(), "<name_of_gameObject>")` to perform actions on `gameObject`s to navigate

Arium can also be extended to perform usability testing, experiential and immersive testing, performance testing, and compatibility testing of XR applications.

That's a short and crisp read on emerging technologies and some of the testing aspects to consider. These trends will keep evolving, and may become mainstream sooner than we expect. So, let's continue to watch this space!

---

1 For a much more complete introduction, see Casey Hardman's *Game Programming with Unity and C#: A Complete Beginner's Guide* (Apress).

Index

A

A/B testing, Usability

acceptance testing stage, continuous testing, Continuous Testing Strategy

accessibility

- Accessibility Scanner, Android
- alternate text, Perceivable
- Android Studio and, Android
- assistive technologies, Accessibility Ecosystem
- ATAG (Authoring Tool Accessibility Guidelines), Accessibility Ecosystem
- audio control, Perceivable
- captions, Perceivable
- colors, Perceivable
- Espresso and, Android
- keyboard navigation, Operable
- legal requirements, Accessibility Testing
- mobile testing strategy, iOS
- operability, Operable
- page hierarchy, Perceivable

perceivability, Perceivable

robustness, Robust

screen readers, Example: Screen Readers-Example: Screen Readers

transcripts, Perceivable

UAAG (User Agent Accessibility Guidelines), Accessibility Ecosystem

understandability, Understandable

user agents, Accessibility Ecosystem

user personas, Accessibility User Personas-Accessibility User Personas

WCAG (Web Content Accessibility Guidelines), Accessibility Ecosystem

web development tools and practices, Accessibility Ecosystem

XCode Accessibility Inspector, iOS

accessibility enabled development frameworks, Accessibility Enabled Development Frameworks

Accessibility Scanner, Android, Accessibility Scanner

accessibility testing, Ten Full Stack Testing Skills, Accessibility Testing

Axe-core, Axe-core-Axe-core

exercises

Lighthouse, Lighthouse-Workflow

Lighthouse Node module, Lighthouse Node Module-Workflow

WAVE, WAVE-Workflow

Pa11y CI Node Module, Pa11y CI Node Module

strategies, Accessibility Testing Strategy

automated auditing tools, Automated Accessibility Auditing Tools

checklists, Accessibility Checklist in User Stories-Accessibility Checklist in User Stories

manual testing, Manual Testing-Manual Testing

visual testing, Accessibility Testing

accessibility tree, screen readers and, Example: Screen Readers

ADRs (architecture decision records), Defect Prevention over Defect Detection

Agile development

dev-box testing, Repeat Exploratory Testing in Phases

shift-left testing and, Shift-Left Testing

AI (artificial intelligence), Artificial Intelligence and Machine Learning

Applitools Eyes visual testing tool, Applitools Eyes, an AI-Powered Tool-Applitools Eyes, an AI-Powered Tool

Visual AI, Applitools Eyes, an AI-Powered Tool

alternate text, accessibility, Perceivable

Android

accessibility testing, Android

Database Inspector, Android Studio's Database Inspector-Android Studio's Database Inspector

emulators, Manual Exploratory Testing, Android emulator

Android Studio, accessbility and, Android

antipatterns in automated functional testing

cupcake, The cupcake

ice cream cone, The ice cream cone

Apache Benchmark, Step 6: Script and run the performance tests using tools, Apache Benchmark

Apache JMeter, Automated Functional Testing

Apache Spark, Batch Processing Systems

API (application programming interface)

RESTful, API Testing

Selenium WebDriver, Selenium WebDriver-Selenium WebDriver

API testing, API Testing

blockchain apps, Testing Blockchain Applications

discovery paths, API Testing

Postman, Postman-Postman

WireMock, WireMock-WireMock

APM (application performance management) tools, Step 5: Integrate APM tools

Appium, Appium

Android emulator, Android emulator

Appium 2.0 setup, Appium 2.0 setup

Java-Appium framework, Workflow

performance API, Appium's performance API-Appium's performance API

RPA (robotic process automation) and, Appium

visual testing plugin, Appium Visual Testing Plug-in-Workflow

workflow, Workflow-Workflow

application architecture, manual exploratory testing and, Understand the Application

application layer, IoT (Internet of Things), Introduction to the IoT's Five-Layer Architecture

application misconfiguration, Application misconfigurations

application performance monitoring (APM) tools (see APM (application performance monitoring))

application vulnerabilities

authentication, Authentication and session mismanagement

code injection, Code or SQL injection-Code or SQL injection

known vulnerabilities, unhandled, Unhandled known vulnerabilities

misconfiguration, Application misconfigurations

secrets exposure, Application secrets exposure-Application secrets exposure

session management, Authentication and session mismanagement

SQL injection, Code or SQL injection-Code or SQL injection

unencrypted data, Unencrypted private data

XSS (cross-site scripting), Cross-site scripting

applications

mobile, App

architecture, Mobile App Architecture-Mobile App Architecture

hybrid applications, App

mobile web, App

native applications, App

PWAs (progressive web apps), App

secrets exposure, Application secrets exposure-Application secrets exposure

Applitools Eyes, Applitools Eyes, an AI-Powered Tool-Applitools Eyes, an AI-Powered Tool, Visual Testing

Appvance, Test authoring

AR (augmented reality), Augmented Reality and Virtual Reality

application testing, Testing AR/VR Applications-Testing AR/VR Applications

architecture decision records (ADRs), Defect Prevention over Defect Detection

architecture design, performance and, Factors Affecting Application Performance

architecture testing, CFR testing, Architecture Testing-Architecture Testing

ArchUnit, Architecture Testing

artificial intelligence (AI) (see AI (artificial intelligence))

assets, Building Blocks

assistive technologies , Accessibility Ecosystem

ATAG (Authoring Tool Accessibility Guidelines), Accessibility Ecosystem

attacks (see cyberattacks)

audio, accessibility, Perceivable

augmented intelligence, Introduction to Testing in Emerging Technologies

augmented reality (AR) (see See AR (augmented reality))

auth service, access token and, Building Blocks

authentication

- application vulnerabilities and, Authentication and session mismanagement
- functionalities and, Cross-functional aspects
- GitHub, Workflow

Authoring Tool Accessibility Guidelines (ATAG), Accessibility Ecosystem

authorization, functionalities and, Cross-functional aspects

automated functional testing, Ten Full Stack Testing Skills, Automated Functional Testing, Automated Functional Testing

Strategy

AI/ML tools

test authoring, Test authoring

test governance tools, Test governance

test maintenance, Test maintenance

test report analysis, Test report analysis

antipatterns

cupcake, The cupcake

ice cream cone, The ice cream cone

code coverage percentage, 100% Automation Coverage!-100% Automation Coverage!

exercises, Exercises

service tests, Service Tests-Setup and workflow

UI functional tests, UI Functional Tests-Setup and workflow

unit tests, Unit Tests-Setup and workflow

implementing, Building Blocks

Karate, Karate

macro test types, Introduction to Micro and Macro Test Types

contract tests, Contract tests

end-to-end tests, End-to-end tests

integration tests, Integration tests

service tests, Service tests-Service tests

UI functional tests, UI functional tests-UI functional tests

unit tests, Unit tests-Unit tests

micro test types, Introduction to Micro and Macro Test Types

contract tests, Contract tests

end-to-end tests, End-to-end tests

integration tests, Integration tests

service tests, Service tests-Service tests

UI functional tests, UI functional tests-UI functional tests

unit tests, Unit tests-Unit tests

Pact, Pact-Pact

tracking automation test coverage, Automated Functional Testing Strategy

automated testing, shift-left testing and, Shift-Left Testing

AutoTester, Automated Functional Testing

Avahi, Introduction to the IoT's Five-Layer Architecture

Axe-core, Axe-core-Axe-core

B

B2C (business-to-customer) applications, visual testing and, Project/Business-Critical Use Cases

backend performance testing, Performance, Sales, and Weekends Off Are Correlated!-Performance, Sales, and Weekends Off Are Correlated!

performance goals, Simple Performance Goals

BackstopJS

backstop.json config file, Workflow

Node.js and, Setup

Puppeteer, BackstopJS

scripts, Workflow

viewports array, Workflow

Resemble.js, BackstopJS

setup, Setup-Setup

Visual Studio Code and, Setup

workflow, Workflow-Workflow

bandwidth throttling, Extended controls: Network throttler

batch processing, Batch Processing Systems-Batch Processing Systems

BDD (behavior-driven development), Setup and workflow

benchmarking, Types of Load Patterns

Bitbucket, Setup

blockchain, Blockchain

API testing and, Testing Blockchain Applications

consensus, Introduction to Blockchain Concepts-Introduction to Blockchain Concepts

functional testing and, Testing Blockchain Applications

ledgers, Introduction to Blockchain Concepts

nodes, Introduction to Blockchain Concepts

performance testing and, Testing Blockchain Applications

security and, Blockchain

security testing and, Testing Blockchain Applications

smart contracts, Introduction to Blockchain Concepts

blockchain-specific testing, Testing Blockchain Applications-Testing Blockchain Applications

Bonjour, Introduction to the IoT's Five-Layer Architecture

bounce rate, Performance, Sales, and Weekends Off Are Correlated!

boundaries, value testing, Boundary value analysis-Boundary value analysis

boundary value analysis, Boundary value analysis-Boundary value analysis

broken builds, pushing to, Principles and Etiquette

browsers

caching, performance testing and, Factors Affecting Frontend Performance

cross-browser testing, Cross-Browser Testing

framework support, Cross-Browser Testing

web UI testing, Browsers-Browsers

BrowserStack, Browsers

brute force attacks, Brute force

bug bashes, Cross-Browser Testing

Bug Magnet, Bug Magnet-Bug Magnet

builds, broken, Principles and Etiquette

business layer, IoT (Internet of Things), Introduction to the IoT's Five-Layer Architecture

business priorities, manual exploratory testing and, Understand the Application

C

caches, Caches-Caches

captions, accessibility, Perceivable

cause-effect graphing, Cause-effect graphing

CD (continuous delivery), Continuous Testing

- automated deployment, The CI/CT/CD Process
- versus CD (continuous deployment), The CI/CT/CD Process

CD (continuous deployment) versus CD (continuous delivery), The CI/CT/CD Process

CDNs (content delivery networks), Factors Affecting Frontend Performance

CFR testing, Cross-Functional Requirements Testing

- architecture testing, Architecture Testing-Architecture Testing
- chaos engineering, Chaos Engineering-Chaos experiment
- compliance testing
  - GDPR (General Data Protection Regulation), General Data Protection Regulation (GDPR)-General Data Protection Regulation (GDPR)
  - PCI DSS (Payment Card Industry Data Security Standard), PCI DSS and PSD2

PSD2 (Payment Services Directive), PCI DSS and PSD2-PCI DSS and PSD2

infrastructure testing, Infrastructure Testing

compliance, Infrastructure Testing

end-to-end testing, Infrastructure Testing

IaC (Infrastructure as Code, Infrastructure Testing

operability, Infrastructure Testing

security, Infrastructure Testing

Terraform, Infrastructure Testing

TFLint, Infrastructure Testing

mobile testing strategy, CFR Testing-CFR Testing

strategies, CFR Testing Strategy

functionality, CFR Testing Strategy, Functionality

performance, CFR Testing Strategy, Performance

reliability, CFR Testing Strategy, Reliability-Reliability

supportability, CFR Testing Strategy, Supportability-Supportability

usability, CFR Testing Strategy, Usability-Usability

CFRs (cross-functional requirements), Ten Full Stack Testing Skills, Cross-Functional Requirements Testing

definitions, Building Blocks-Building Blocks

versus non-functional requirements, Cross-Functional Requirements Testing

change blindness, Introduction to Visual Testing

chaos engineering, Reliability

CFR testing, Chaos Engineering-Chaos experiment

Chromatic, Storybook

Chrome DevTools, Chrome DevTools and Postman-Chrome DevTools and Postman, Chrome DevTools-Chrome DevTools

cookies, Chrome DevTools

first-time users, Chrome DevTools

number of requests from page, Chrome DevTools

page errors, Chrome DevTools

service down behaviors, Chrome DevTools

UI and API integration, Chrome DevTools

UI behavior, slow networks, Chrome DevTools

Chrome, cross-browser testing, Cross-Browser Testing

ChromeDriver executable, Setup and workflow

CI (continuous integration), Continuous Testing, Introduction to Continuous Integration-Introduction to Continuous Integration

description, Introduction to Continuous Integration

JMeter, Integrating into CI

versus continuous testing, Continuous Testing Strategy

CI server, The CI/CT/CD Process

build and test stage, The CI/CT/CD Process

commits, The CI/CT/CD Process

CI/CD (Continuous Integration/Continuous Delivery), shift-left testing and, Shift-Left Testing

CI/CT/CD process

etiquette, Principles and Etiquette-Principles and Etiquette

principles, Principles and Etiquette-Principles and Etiquette

VCS (version control system), The CI/CT/CD Process

cloud-hosted testing platforms, Browsers

coaching, soft skills, Soft Skills Aid in Building a Quality-First Mindset

CoAP (Constrained Application Protocol), Introduction to the IoT's Five-Layer Architecture

code complexity, performance and, Factors Affecting Application Performance

code injection, Code or SQL injection

collaboration

continuous testing and, Benefits

first principles and, Communication and Collaboration Are Key to Quality

soft skills and, Soft Skills Aid in Building a Quality-First Mindset

colors, accessibility, Perceivable

commenting out failing tests, Principles and Etiquette

commits

frequency, Principles and Etiquette

Git VCS, The CI/CT/CD Process

self-tested code, Principles and Etiquette

communication

first principles and, Communication and Collaboration Are Key to Quality

soft skills and, Soft Skills Aid in Building a Quality-First Mindset

compliance testing, CFR testing

GDPR (General Data Protection Regulation), General Data Protection Regulation (GDPR)-General Data Protection Regulation (GDPR)

PCIDSS (Payment Card Industry Data Security Standard), PCI DSS and PSD2

PSD2 (Payment Services Directive), PCI DSS and PSD2-PCI DSS and PSD2

compromises, security, Building Blocks

configuration, application misconfiguration, Application misconfigurations

conformance certification, accessibility, Manual Testing

connected things, Introduction to Testing in Emerging Technologies

consensus, blockchain and, Introduction to Blockchain Concepts-Introduction to Blockchain Concepts

consistency models, Databases

Constrained Application Protocol (CoAP), Introduction to the IoT's Five-Layer Architecture

containers, Testcontainers, Test Containers-Test Containers

content delivery networks (CDNs), Factors Affecting Frontend Performance

continuous delivery (CD) (see CD (continuous delivery))

continuous integration (CI) (see CI (continuous integration))

Continuous Integration Certification Test, Principles and Etiquette

continuous testing, Ten Full Stack Testing Skills

- acceptance stage, Continuous Testing Strategy
- build-test stage, Continuous Testing Strategy, Continuous Testing Strategy
- change fail percentage, The Four Key Metrics
- collaboration and, Benefits
- common quality goals, Benefits
- delivery ownership, Benefits
- deploy stage, Continuous Testing Strategy
- deployment and, Benefits
- deployment frequency, The Four Key Metrics
- early defect detection, Benefits
- exercises
  - Git, Git-Workflow
  - Jenkins, Jenkins
- functional testing stage, Continuous Testing Strategy
- lead time, The Four Key Metrics
- mean time to restore, The Four Key Metrics

metrics, The Four Key Metrics-The Four Key Metrics

nightly regression stage, Continuous Testing Strategy

smoke testing, Continuous Testing Strategy

strategies, Continuous Testing Strategy-Benefits

versus CI (continuous integration), Continuous Testing Strategy

contract tests, Contract tests

cookie forging, Cookie forging

criteria-specific sampling, Sampling

cross-browser testing

from the left, Cross-Browser Testing

functional feedback, Cross-Browser Testing, Cross-Browser Testing

visual testing, Cross-Browser Testing-Cross-Browser Testing

cross-functional requirements (CFRS) (see CFRs (cross-functional requirements))

cross-functional requirements testing (see CFR testing)

cross-site scripting (XSS), Cross-site scripting

CRUD operations, Databases

cryptojacking, Cryptojacking

CSS (Cascading Style Sheets), testing and, Introduction to Visual Testing

CT (continuous testing), Continuous Testing

Cucumber, Karate

cupcake antipattern, The cupcake

customer impact, visual testing and, Project/Business-Critical Use Cases

cyberattacks

- brute force, Brute force
- cookie forging, Cookie forging
- cryptojacking, Cryptojacking
- phishing, Phishing
- ransomware, Ransomware
- social engineering, Social engineering
- web scraping, Web scraping
- XSS (cross-site scripting), Cross-site scripting

cybercrime, Security Testing-Security Testing

Cypress, Cypress-Workflow

D

DAST (Dynamic Application Security Testing), Security Testing Strategy

data skew, Batch Processing Systems

data testing, Ten Full Stack Testing Skills, Data Testing

- batch processing, Batch Processing Systems-Batch Processing Systems
- caches, Caches-Caches
- databases, Databases

boundary values, Databases

concurrency, Databases

order consistency, Databases

reading writes, Databases

relational databases, Databases

replication, Databases

schema, Databases

SQL, Databases

test cases, Databases

time traveling, Databases

write conflicts, Databases

Deequ, Deequ-Deequ

event streams, Event Streams

exercises

JDBC, JDBC-Setup and workflow

Kafka, Apache Kafka and Zerocode-Workflow

SQL, SQL-Update and delete

Zerocode, Apache Kafka and Zerocode-Workflow

functional testing and, Data Testing

mobile testing strategy, Data Testing-Data Testing

pyramid and, JDBC

strategies, Data Testing Strategy

functional automated testing, Data Testing Strategy

manual exploratory testing, Data Testing Strategy

performance testing and, Data Testing Strategy

security and privacy, Data Testing Strategy

Testcontainers, Test Containers-Test Containers

data transfers, performance testing and, Factors Affecting Frontend Performance

data-driven performance testing

JMeter, Data-driven performance testing

Database Inspector, Android Studio's Database Inspector-Android Studio's Database Inspector

DB (databases), Introduction to Micro and Macro Test Types, Databases, Databases

boundary values, Databases

concurrency, Databases

CRUD operations, Databases

ordering consistency, Databases

performance and, Factors Affecting Application Performance

relational databases, Databases

UUIDs, Databases

replication, Databases

scalability, Databases

schema, Databases

test cases, Databases

time traveling, Databases

write conflicts, Databases

writes, reading, Databases

DDoS (distributed denial of service) attack, Denial of service

dead letter queue, Event Streams

decision table, Decision table-Decision table

Deequ, Deequ-Deequ

delivery ownership, continuous testing and, Benefits

deployment

continuous testing and, Benefits

frequency, Continuous Testing, The Four Key Metrics

design systems, Project/Business-Critical Use Cases

design, shift-left testing and, Shift-Left Testing

dev-box testing, Repeat Exploratory Testing in Phases

development, shift-left testing and, Shift-Left Testing

devices

IoT (Internet of Things), Internet of Things

mobile

device manufacturer, Devices

hardware, Devices

operating system, Devices

pixel density, Devices

screen resolution, Devices

screen size, Devices

digitalization, Introduction to Full Stack Testing

DNS (Domain Name Service) lookups, Factors Affecting Frontend Performance

Docker, Setup

domains, manual exploratory testing and, Understand the Application

DoS (denial of service) attack, Denial of service

drivers, Selenium WebDriver, Selenium WebDriver

Dynamic Application Security Testing (DAST), Security Testing Strategy

E

early defect detection, continuous testing, Benefits

ecommerce UI, Introduction to Micro and Macro Test Types

edge case, Manual Exploratory Testing

empathetic testing, Empathetic Testing

emulators, Manual Exploratory Testing

Android, Android emulator

encryption, Building Blocks

unencrypted data and, Unencrypted private data

end-to-end tests, End-to-end tests

equivalence class partitioning, Equivalence class partitioning

error guessing method, Error guessing method-Error guessing method

error handling, Failures and error handling

escalation of privileges, Escalation of privileges, Threat modeling exercise

Espresso, accessbility and, Android

event streams, Event Streams

    Apache Kafka, Event Streams

    Google Cloud Pub/Sub, Event Streams

    near real-time, Event Streams

    publisher, Event Streams

    RabbitMQ, Event Streams

    subscribers, Event Streams

    topics, Event Streams

events, Event Streams

eventual consistency, Databases

explicit wait strategy, Selenium WebDriver

exploratory testing, Manual Exploratory Testing

    (see also manual exploratory testing)

    frameworks, Exploratory Testing Frameworks

        boundary value analysis, Boundary value analysis-Boundary value analysis

cause-effect graphing, Cause-effect graphing

decision table, Decision table-Decision table

equivalence class partitioning, Equivalence class partitioning

error guessing method, Error guessing method-Error guessing method

pairwise testing, Pairwise testing-Pairwise testing

sampling, Sampling-Sampling

state transition, State transition-State transition

monkey testing, Understand the Application

expressions, SQL, Expressions and predicates

Extreme Programming (XP), Shift-Left Testing

F

failure screenshots, Setup and workflow

failures, Failures and error handling

owning, Principles and Etiquette

feature testing, accessibility, Manual Testing

features, Manual Exploratory Testing

feedback, first principles of testing, Fast Feedback, Continuous Feedback

first principles

collaboration, Communication and Collaboration Are Key to Quality

communication, Communication and Collaboration Are Key to Quality

continuous feedback, Continuous Feedback

defects, prevention over detection, Defect Prevention over Defect Detection-Defect Prevention over Defect Detection

empathetic testing, Empathetic Testing

fast feedback, Fast Feedback

macro-level testing, Micro- and Macro-Level Testing

metrics, Measuring Quality Metrics-Measuring Quality Metrics

micro-level testing, Micro- and Macro-Level Testing

Flipkart, Introduction to Full Stack Testing

fluent wait strategy, Selenium WebDriver

FORTRAN, testing and, Automated Functional Testing

FriendFinder attack, Brute force

frontend performance testing, Frontend Performance Testing Building Blocks-Frontend Performance Testing Building Blocks

browser caching, Factors Affecting Frontend Performance

CDNs (content delivery networks), Factors Affecting Frontend Performance

code complexity, Factors Affecting Frontend Performance

data transfers, Factors Affecting Frontend Performance

DNS lookups, Factors Affecting Frontend Performance

exercises, Exercises

Lighthouse, Lighthouse-Workflow

WebPageTest, WebPageTest-Workflow

macro-level tests, Frontend Testing Strategy

metrics, Frontend Performance Metrics-Frontend Performance Metrics

micro-level tests, Frontend Testing Strategy

network latency, Factors Affecting Frontend Performance

visual testing, Frontend Testing Strategy, Frontend Performance Testing

accessibility testing, Accessibility Testing

cross-browser tests, Cross-Browser Testing-Cross-Browser Testing

frontend performance testing, Frontend Performance Testing

functional end-to-end tests, Functional End-to-End Tests

integration/component tests, Integration/Component Tests-Integration/Component Tests

snapshot tests, Snapshot Tests -Snapshot Tests

unit tests, Unit Tests

visual tests, Visual Tests

full outer joins, Joins

functional automated testing

data testing and, Data Testing Strategy

mobile testing strategy, Functional Automated Testing

functional end-to-end tests, visual testing and, Functional End-to-End Tests

functional feedback, cross-browser testing, Cross-Browser Testing

functional test automation, Security Testing Strategy

functional testing

- automated functional testing, Ten Full Stack Testing Skills
- blockchain apps, Testing Blockchain Applications
- data testing and, Data Testing

functional testing stage, continuous testing, Continuous Testing Strategy

functionalities

- cross-functional aspects, Cross-functional aspects-Cross-functional aspects
- discovery paths, Functional user flows, Cross-functional aspects
- error handling, Failures and error handling
- failures, Failures and error handling
- functional user flow, Functional user flows-Functional user flows
- UI (user interface)
  - look and feel, The UI look and feel

functionality

- CFR testing, Functionality
- definition, Manual Exploratory Testing

Functionize, Test authoring

functions, SQL, Functions and operators

G

Gatling, Step 6: Script and run the performance tests using tools

GDPR (General Data Protection Regulation), General Data Protection Regulation (GDPR)-General Data Protection Regulation (GDPR)

geolocation, performance and, Factors Affecting Application Performance

Gherkin statements, Karate

Git, Git

Git VCS system, The CI/CT/CD Process

GitHub, Setup

    authentication, Workflow

    repositories, Setup

Google Cardboard, Augmented Reality and Virtual Reality

Gradle, Maven

graphing, cause-effect, Cause-effect graphing

H

hang fail percentage, Continuous Testing

hardware, mobile devices, Devices

hashing, Building Blocks

honeycomb test shape, Automated Functional Testing Strategy

HTC VIVE, Augmented Reality and Virtual Reality

HTML, snapshot tests, Snapshot Tests

human-like interaction, Introduction to Testing in Emerging Technologies

hybrid applications, App

I

IaC (Infrastructure as Code), Infrastructure Testing

IAST (Interactive Application Security Testing), Security Testing Strategy

ice cream cone antipattern, The ice cream cone

image scanning, Security Testing Strategy

implicit wait strategy, Selenium WebDriver

influence, soft skills and, Soft Skills Aid in Building a Quality-First Mindset

information disclosure, Information disclosure

Infrastructure as Code (IaC), Infrastructure Testing

infrastructure testing, Reliability

infrastructure testing, CFR testing, Infrastructure Testing

- compliance, Infrastructure Testing
- end-to-end testing, Infrastructure Testing
- IaC (Infrastructure as Code), Infrastructure Testing
- operability, Infrastructure Testing
- security, Infrastructure Testing

Terraform, Infrastructure Testing

TFLint, Infrastructure Testing

infrastructure, performance and, Factors Affecting Application Performance

input tampering, Tampering with inputs

integration tests, Integration tests

integration/component tests, visual testing, Integration/Component Tests-Integration/Component Tests

IntelliJ, Maven project, Setup and workflow

Interactive Application Security Testing (IAST), Security Testing Strategy

internationalization, usability testing and, Usability

Internet of Things (IoT) (see IoT (Internet of Things))

iOS accessibility testing, iOS

IoT (Internet of Things), Internet of Things

application layer, Introduction to the IoT's Five-Layer Architecture

application testing

hardware/software integration, Testing IoT Applications

interoperability, Testing IoT Applications

network connectivity, Testing IoT Applications

performance, Testing IoT Applications

privacy, Testing IoT Applications

security, Testing IoT Applications

usability, Testing IoT Applications

business layer, Introduction to the IoT's Five-Layer Architecture

devices, Internet of Things

middleware layer, Introduction to the IoT's Five-Layer Architecture

network layer, Introduction to the IoT's Five-Layer Architecture

perception layer, Introduction to the IoT's Five-Layer Architecture

IPMs (iteration planning meetings), Shift-Left Testing, Defect Prevention over Defect Detection

J

Java-Appium framework, Workflow

Java-REST Assured Framework, Java–REST Assured Framework-Setup and workflow

Java-Selenium WebDriver, JDBC

Maven, Maven-Maven

Page Object Model, Page Object Model-Page Object Model

prerequisites, Prerequisites

Selenium WebDriver, Selenium WebDriver

components, Selenium WebDriver

setup, Setup and workflow, Setup and workflow

TestNG, TestNG

JavaScript, backward-compatibility, Cross-Browser Testing

JavaScript-Cypress Framework, JavaScript–Cypress Framework

- Cypress, Cypress-Cypress
- prerequisites, Prerequisites
- setup and workflow, Setup and workflow-Setup and workflow

JDBC (Java Database Connectivity), JDBC-Apache Kafka and Zerocode

Jenkins

- build triggers, Workflow
- dashboard, Setup
- setup, Setup-Setup
- workflow, Workflow-Workflow

Jest, Snapshot Tests

Jira, Automated Functional Testing Strategy

JMeter, Automated Functional Testing, Step 6: Script and run the performance tests using tools, Step 6: Script the Test Cases and Run Them Using JMeter

- Aggregate Report, Workflow
- CI integration, Integrating into CI
- data-driven performance testing, Data-driven performance testing
- GUI, thread group, Workflow
- listeners, Workflow

load testing, Workflow

performance test case design, Designing other performance test cases-Designing other performance test cases

setup, Setup

soak tests, Designing other performance test cases

View Results Tree view, Workflow

workflow, Workflow-Workflow

joins, Joins

JUnit, Unit tests, JUnit-Setup and workflow

Spring Data JPA, Integration tests

K

k6, Step 6: Script and run the performance tests using tools

Kafka

brokers, Apache Kafka and Zerocode

installation, with Docker, Setup

messages, Apache Kafka and Zerocode

offset, Apache Kafka and Zerocode

partitions, Apache Kafka and Zerocode

retention, Apache Kafka and Zerocode

schemas, Apache Kafka and Zerocode

setup, Setup-Setup

topics, Apache Kafka and Zerocode

Zerocode and, Workflow

Karate, Karate

keyboard navigation, accessibility, Operable

KPIs (key performance indicators), Key Performance Indicators-Types of Performance Tests

target, Step 1: Define the Target KPIs-Step 1: Define the Target KPIs

test cases, Step 2: Define the Test Cases

L

lead time, Continuous Testing

continuous testing, The Four Key Metrics

left joins, Joins

libraries, Selenium WebDriver, Selenium WebDriver

Lighthouse, Lighthouse-Workflow

Lighthouse accessibility evaluation tool, Lighthouse-Workflow

Lighthouse Node Module accessibility evaluation tool, Lighthouse Node Module-Workflow

load patterns, performance testing and

peak-rest pattern, Types of Load Patterns

steady ramp-up pattern, Types of Load Patterns

step ramp-up pattern, Types of Load Patterns

load testing, Scala, Gatling

load/volume tests, Types of Performance Tests

localization, usability testing and, Usability

M

machine learning (ML) (see ML (machine learning))

macro test types, Introduction to Micro and Macro Test Types

- contract tests, Contract tests
- end-to-end tests, End-to-end tests
- integration tests, Integration tests
- service tests, Service tests-Service tests
- test pyramid and, Automated Functional Testing Strategy
- UI functional tests, UI functional tests-UI functional tests
- unit tests, Unit tests-Unit tests

macro-level testing, Micro- and Macro-Level Testing

- frontend testing strategy, Frontend Testing Strategy

manual exploratory testing, Ten Full Stack Testing Skills, Manual Exploratory Testing, Security Testing Strategy

- (see also exploratory testing)
- application and, Understand the Application
  - application architecture, Understand the Application
  - business priorities, Understand the Application
  - configuration and, Understand the Application
  - domain, Understand the Application
  - infrastructure, Understand the Application

user personas, Understand the Application

data testing and, Data Testing Strategy

exercises

API testing, API Testing-WireMock

web UI testing, Web UI Testing-Chrome DevTools

in parts, Explore in Parts-Explore in Parts

mobile testing strategy, Manual Exploratory Testing-Manual Exploratory Testing

repeating, phases, Repeat Exploratory Testing in Phases-Repeat Exploratory Testing in Phases

manual testing, accessibility testing, Manual Testing

conformance certification testing, Manual Testing

feature testing, Manual Testing

release testing, Manual Testing

user story testing, Manual Testing

Maven, Maven-Maven

mean time to restore, Continuous Testing

continuous testing and, The Four Key Metrics

mentoring, soft skills, Soft Skills Aid in Building a Quality-First Mindset

Mercury Interactive, Automated Functional Testing

Message Queuing Telemetry Transport (MQTT), Introduction to the IoT's Five-Layer Architecture

metrics, first principles of testing, Measuring Quality Metrics-Measuring Quality Metrics

micro test types, Introduction to Micro and Macro Test Types

- contract tests, Contract tests
- end-to-end tests, End-to-end tests
- integration tests, Integration tests
- service tests, Service tests-Service tests
- test pyramid and, Automated Functional Testing Strategy
- UI functional tests, UI functional tests-UI functional tests
- unit tests, Unit tests-Unit tests

micro-level testing, Micro- and Macro-Level Testing

- frontend testing strategy, Frontend Testing Strategy

middleware layer, IoT (Internet of Things), Introduction to the IoT's Five-Layer Architecture

ML (machine learning), Introduction to Machine Learning

- application testing
  - integration validation, Testing ML Applications
  - model bias validation, Testing ML Applications
  - model quality validation, Testing ML Applications
  - training data validation, Testing ML Applications-Testing ML Applications
- model training, Introduction to Machine Learning
- test set, Introduction to Machine Learning

training set, Introduction to Machine Learning

mobile application architecture, Mobile App Architecture-Mobile App Architecture

mobile landscape, Introduction to the Mobile Landscape

applications, App

hybrid, App

mobile web, App

native, App

PWAs (progressive web apps), App

devices

device manufacturer, Devices

hardware, Devices

operating system, Devices

pixel density, Devices

screen size, Devices

network, Network

mobile networks, Network

mobile test pyramid, Perspectives: The Mobile Test Pyramid

mobile testing, Ten Full Stack Testing Skills, Mobile Testing

Accessibility Scanner, Accessibility Scanner

Database Inspector, Android Studio's Database Inspector-Android Studio's Database Inspector

exercises

Appium, Appium-Workflow

Appium visual testing plug-in, Appium Visual Testing Plug-in-Workflow

mobile landscape, Introduction to the Mobile Landscape

devices, Devices-Devices

performance testing tools

Appium performance API, Appium's performance API-Appium's performance API

MobSF, MobSF-MobSF

Monkey, Monkey

network throttler, Extended controls: Network throttler

Qark, Qark

strategies, Mobile Testing Strategy-Manual Exploratory Testing

accessibility testing, Accessibility Testing-Android

CFR testing, CFR Testing-CFR Testing

data testing, Data Testing-Data Testing

functional automated testing, Functional Automated Testing

manual exploratory testing, Manual Exploratory Testing-Manual Exploratory Testing

performance testing, Performance Testing-Performance Testing

security testing, Security Testing-Security Testing

visual testing, Visual Testing

mobile web applications, App

MobSF (Mobile Security Framework), Security Testing, MobSF-MobSF

monkey testing, Understand the Application

Monkey testing tool, Monkey

MQTT (Message Queuing Telemetry Transport), Introduction to the IoT's Five-Layer Architecture

multiple-user flows, Functional user flows

mutation testing, code coverage and, 100% Automation Coverage!

N

native applications, App

near real-time, Event Streams

nested queries in SQL, Nested queries

NetArchTest, Architecture Testing

network latency, performance testing, Factors Affecting Application Performance, Factors Affecting Frontend Performance

network layer, IoT (Internet of Things), Introduction to the IoT's Five-Layer Architecture

network throttling, mobile testing, Extended controls: Network throttler

networks, mobile, Network

NFRs (non-functional requirements), Ten Full Stack Testing Skills

    versus CFRs (cross-functional requirements), Cross-Functional Requirements Testing

nightly regression stage, continuous testing, Continuous Testing Strategy

nodes, blockchain, Introduction to Blockchain Concepts

non-functional requirements (NFRs), Ten Full Stack Testing Skills

NUnit, Unit tests

O

OAuth 2.0, Building Blocks

Oculus Go, Augmented Reality and Virtual Reality

Oculus Quest, Augmented Reality and Virtual Reality

operability, accessibility, Operable

operating systems, mobile devices, Devices

operators, SQL, Functions and operators

outcomes, soft skills, Soft Skills Aid in Building a Quality-First Mindset

OWASP Dependency-Check, OWASP Dependency-Check-Setup and Workflow

OWASP ZAP (Zed Attack Proxy)

- CI integration, Integrating ZAP with CI-Integrating ZAP with CI
- scanning, Scanning-Scanning
- setup, Setup
- workflow, Workflow-Workflow
- ZAP spider, ZAP Spider

P

Pa11y CI Node Module, Pa11y CI Node Module

Pact, Pact-Pact

Page Object Model, Page Object Model-Page Object Model

PageSpeed Insights, PageSpeed Insights-PageSpeed Insights

pairwise testing, Pairwise testing-Pairwise testing

partitioning, equivalence class, Equivalence class partitioning

Payment Card Industry Data Security Standard (PCI DSS), PCI DSS and PSD2

Payment Services Directive (PSD2), PCI DSS and PSD2-PCI DSS and PSD2

PCI DSS (Payment Card Industry Data Security Standard (PCI DSS), PCI DSS and PSD2

peak-rest load pattern, Types of Load Patterns

penetration (pen) testing, Security Testing Strategy

perceivability, accessibility, Perceivable

perception layer, IoT (Internet of Things), Introduction to the IoT's Five-Layer Architecture

performance

- architecture design and, Factors Affecting Application Performance
- CFR testing, CFR Testing Strategy, Performance
- code complexity and, Factors Affecting Application Performance

databases and, Factors Affecting Application Performance

geolocation and, Factors Affecting Application Performance

infrastructure and, Factors Affecting Application Performance

network latency and, Factors Affecting Application Performance

tech stack and, Factors Affecting Application Performance

third-party components and, Factors Affecting Application Performance

performance testing, Ten Full Stack Testing Skills, Performance Testing

Apache Benchmark, Apache Benchmark

backend performance testing, Performance, Sales, and Weekends Off Are Correlated!-Performance, Sales, and Weekends Off Are Correlated!

performance goals, Simple Performance Goals

blockchain apps, Testing Blockchain Applications

Chrome DevTools, Chrome DevTools-Chrome DevTools

data testing and, Data Testing Strategy

exercises

data prep, Steps 3–5: Prepare the Data, Environment, and Tools-Step 6: Script the Test Cases and Run Them Using JMeter

environment prep, Steps 3–5: Prepare the Data, Environment, and Tools-Step 6: Script the Test Cases and Run Them Using JMeter

target KPIs, Step 1: Define the Target KPIs-Step 1: Define the Target KPIs

test case scripting, Step 6: Script the Test Cases and Run Them Using JMeter-Integrating into CI

test cases, Step 2: Define the Test Cases

test cases, JMeter and, Step 6: Script the Test Cases and Run Them Using JMeter-Integrating into CI

tool prep, Steps 3–5: Prepare the Data, Environment, and Tools-Step 6: Script the Test Cases and Run Them Using JMeter

frontend, Frontend Performance Testing Building Blocks-Frontend Performance Testing Building Blocks

browser caching, Factors Affecting Frontend Performance

CDNs (content delivery networks), Factors Affecting Frontend Performance

code complexity, Factors Affecting Frontend Performance

data transfsers, Factors Affecting Frontend Performance

DNS lookups, Factors Affecting Frontend Performance

network latency, Factors Affecting Frontend Performance

Gatling, Gatling

goals, Simple Performance Goals

IoT (Internet of Things) applications, Testing IoT Applications

KPIs (key performance indicators)

load/volume tests, Types of Performance Tests

soak tests, Types of Performance Tests

stress tests, Types of Performance Tests

length of time, Step 6: Script and run the performance tests using tools

load patterns

peak-rest pattern, Types of Load Patterns

steady ramp-up pattern, Types of Load Patterns

step ramp-up pattern, Types of Load Patterns

mobile apps

Appium performance API, Appium's performance API-Appium's performance API

Monkey, Monkey

network throttler, Extended controls: Network throttler

mobile testing strategy, Performance Testing-Performance Testing

Android, Android

PageSpeed Insights, PageSpeed Insights-PageSpeed Insights

RAIL model, RAIL Model

shift-left testing

development phase, Performance Testing Strategy

in CI, Performance Testing Strategy

planning phase, Performance Testing Strategy

release testing phase, Performance Testing Strategy

user story testing phase, Performance Testing Strategy

steps

APM tools, Step 5: Integrate APM tools

environment prep, Step 3: Prepare the performance testing environment-Step 3: Prepare the performance testing environment

scripting, Step 6: Script and run the performance tests using tools

target KPIs, Step1: Define the target KPIs

test cases, Step 2: Define the test cases

test data, Step 4: Prepare the test data

tools, Step 6: Script and run the performance tests using tools

strategies, Performance Testing Strategy-Performance Testing Strategy

phishing attacks, Phishing

pixel density, mobile devices, Devices

platforms as standards, Introduction to Testing in Emerging Technologies

POM (Project Object Model)

XML file, Maven-Maven

portability testing, Test Containers

Postman, Chrome DevTools and Postman-Chrome DevTools and Postman

Postman API testing tool, Postman-Postman

predicates, SQL, Expressions and predicates

prioritization, soft skills and, Soft Skills Aid in Building a Quality-First Mindset

privacy

    data testing and, Data Testing Strategy

    functionalities and, Cross-functional aspects

progressive web apps (PWAs), App

Project Object Model (POM), Maven

    (see also POM (Project Object Model))

PSD2 (Payment Services Directive), PCI DSS and PSD2-PCI DSS and PSD2

Puppeteer, BackstopJS

PWAs (progressive web apps), App

Q

Qark, Qark

queries, nested, Nested queries

QuickTest, Automated Functional Testing

R

RAIL model, frontend performance and

    animation, RAIL Model

    idle, RAIL Model

load, RAIL Model

response, RAIL Model

random sampling, Sampling

ransomware, Ransomware

RASP (Runtime Application Self Protection), Security Testing Strategy

rate limiting, Understand the Application

react-test-renderer, Snapshot Tests

relational databases, Databases, Databases, Databases

release testing, accessibility, Manual Testing

reliability, CFR testing, CFR Testing Strategy, Reliability, Reliability

repeat flows, Functional user flows

replication, Databases

report analysis tools, Test report analysis

ReportPortal, Test report analysis

repudiation, Repudiation of actions

requirements analysis, shift-left testing and, Shift-Left Testing, Shift-Left Testing

Resemble.js, BackstopJS

RESTful APIs, API Testing

RESTful services, Introduction to Micro and Macro Test Types

right joins, Joins

robustness, accessibility, Robust

Runtime Application Self Protection (RASP), Security Testing Strategy

RXVP tool, automated testing, Automated Functional Testing

S

SaaS (software-as-a-service), Visual AI, Applitools Eyes, an AI-Powered Tool

Safari, cross-browser testing and, Cross-Browser Testing

sampling, Sampling-Sampling

- criteria-specific, Sampling
- random sampling, Sampling

SAST (Static Application Security Testing), Security Testing Strategy, Security Testing

SCA (Software Composition Analysis) tools, Security Testing Strategy

Scala script, load testing, Gatling

scalability, databases, Databases

SCCS (Source Code Control System), The CI/CT/CD Process

screen readers, Example: Screen Readers-Example: Screen Readers

- TalkBack, Android
- VoiceOver, iOS

screen resolution, Devices

screen size, mobile devices, Devices

screenshots, failure screenshots, Setup and workflow
security
as habit, Perspectives: Security Is a Habit
assets, Building Blocks
attacks, Building Blocks
blockchain and, Blockchain
compromises, Building Blocks
cyberattacks
brute force, Brute force
cookie forging, Cookie forging
cryptojacking, Cryptojacking
phishing, Phishing
ransomware, Ransomware
social engineering, Social engineering
web scraping, Web scraping
XSS (cross-site scripting), Cross-site scripting
data testing and, Data Testing Strategy
encryption, Building Blocks
functionalities and, Cross-functional aspects
hashing, Building Blocks
threats, Building Blocks
vulnerabilities, Building Blocks

security test cases, Security test cases from the threat model-Security test cases from the threat model

security testing, Ten Full Stack Testing Skills, Security Testing

application vulnerabilities

authentication, Authentication and session mismanagement

code injection, Code or SQL injection-Code or SQL injection

known vulnerabilities, unhandled, Unhandled known vulnerabilities

misconfiguration, Application misconfigurations

secrets exposure, Application secrets exposure-Application secrets exposure

session management, Authentication and session mismanagement

SQL injection, Code or SQL injection-Code or SQL injection

unencrypted data, Unencrypted private data

XSS (cross-site scripting), Cross-site scripting

blockchain apps, Testing Blockchain Applications

exercises

OWASP Dependency-Check, OWASP Dependency-Check-Setup and Workflow

OWASP ZAP, OWASP ZAP-Integrating ZAP with CI

IoT (Internet of Things) applications, Testing IoT Applications

mobile apps

Accessibility Scanner, Accessibility Scanner

MobSF, MobSF-MobSF

Qark, Qark

mobile testing strategy, Security Testing-Security Testing

strategies

DAST (Dynamic Application Security Testing), Security Testing Strategy

functional test automation, Security Testing Strategy

IAST (Interactive Application Security Testing), Security Testing Strategy

image scanning, Security Testing Strategy

manual exploratory testing, Security Testing Strategy

penetration (pen) testing, Security Testing Strategy

RASP (Runtime Application Self Protection), Security Testing Strategy

SAST (Static Application Security Testing), Security Testing Strategy

SCA (Software Composition Analysis) tools, Security Testing Strategy

Synk IDE plug-in, Snyk IDE Plug-in

threat modeling, The STRIDE Threat Model

DDoS (distributed denial of service), Denial of service

DoS (denial of service), Denial of service

escalation of privileges, Escalation of privileges

information disclosure, Information disclosure

input tampering, Tampering with inputs

repudiation, Repudiation of actions

spoofed identity, Spoofed identity

Selenium, Automated Functional Testing

Selenium Grid, Setup and workflow

Selenium WebDriver, Selenium WebDriver

Actions class, Selenium WebDriver

APIs, Selenium WebDriver-Selenium WebDriver

components, Selenium WebDriver

explicit wait strategy, Selenium WebDriver

fluent wait strategy, Selenium WebDriver

implicit wait strategy, Selenium WebDriver

relative locators, Selenium WebDriver

SEO (search engine optimization) ,performance testing, Performance, Sales, and Weekends Off Are Correlated!

service tests, Service tests-Service tests

Java-REST Assured Framework, Java–REST Assured Framework-Setup and workflow

session management, Authentication and session mismanagement

shift-left testing, Shift-Left Testing

Agile development and, Shift-Left Testing

analysis phase, Shift-Left Testing

automated testing and, Shift-Left Testing

CI/CD and, Shift-Left Testing

performance testing

development phase, Performance Testing Strategy

in CI, Performance Testing Strategy

planning phase, Performance Testing Strategy

release testing phase, Performance Testing Strategy

user story testing phase, Performance Testing Strategy

story kickoff, Shift-Left Testing

three amigos process, Shift-Left Testing

simulators, Manual Exploratory Testing

smoke testing, continuous testing, Continuous Testing Strategy

Snapdeal, Introduction to Full Stack Testing

snapshot tests, visual testing and, Snapshot Tests -Snapshot Tests

Snyk JetBrains IDE plugin, Security Testing Strategy

soak tests, Types of Performance Tests

JMeter, Designing other performance test cases

social engineering, Social engineering

soft skills, Soft Skills Aid in Building a Quality-First Mindset-Soft Skills Aid in Building a Quality-First Mindset

Software Composition Analysis (SCA) (see SCA (Software Composition Analysis) tools)

Sony Playstation VR, Augmented Reality and Virtual Reality

Source Code Control System (SCCS), The CI/CT/CD Process

spoofing, Threat modeling exercise

Spring Batch, Batch Processing Systems

Spring Data JPA, Integration tests

SQL (Structured Query Language), Databases, SQL, Filtering and grouping-Filtering and grouping

- creating tables, Create-Create
- deletes, Update and delete
- expressions, Expressions and predicates
- functions, Functions and operators
- joins, Joins
- null values, Joins
- operators, Functions and operators
- populating tables, Insert
- predicates, Expressions and predicates
- prerequisites, Prerequisites
- queries, Filtering and grouping
  - nested queries, Nested queries
- reads, Select
- sorting, Sorting
- updates, Update and delete

SQL injection, Code or SQL injection-Code or SQL injection

stakeholders, soft skills and, Soft Skills Aid in Building a Quality-First Mindset

state transition, State transition-State transition

Static Application Security Testing (SAST) tools, Security Testing Strategy

steady-ramp up load pattern, Types of Load Patterns

step-ramp up load pattern, Types of Load Patterns

story kickoff, Shift-Left Testing

Storybook, Storybook-Storybook

streams, Event Streams

stress tests, Types of Performance Tests

STRIDE model, threats and, The STRIDE Threat Model

stubs, Contract tests, Contract tests

supportability, CFR testing, CFR Testing Strategy

    architecture tests, Supportability

    static code analyzer, Supportability

Synk IDE plug-in, Snyk IDE Plug-in

T

Talisman, Security Testing Strategy, Talisman Pre-Commit Hook-Talisman Pre-Commit Hook

TalkBack, Android

tech stack, performance and, Factors Affecting Application Performance

technologies

augmented intelligence, Introduction to Testing in Emerging Technologies

connected things, Introduction to Testing in Emerging Technologies

human-like interaction, Introduction to Testing in Emerging Technologies

platforms as standards, Introduction to Testing in Emerging Technologies

Terraform, Infrastructure Testing

test authoring tools, Test authoring

test case, Manual Exploratory Testing

test environment hygiene, Perspectives: Test Environment Hygiene

autonomous teams, Perspectives: Test Environment Hygiene

data hygiene, Perspectives: Test Environment Hygiene

deployment and, Perspectives: Test Environment Hygiene

shared versus dedicated, Perspectives: Test Environment Hygiene

third-party services, Perspectives: Test Environment Hygiene

test governance tools, Test governance

test maintenance tools, Test maintenance

test pyramid , Automated Functional Testing Strategy

service-oriented web application, Automated Functional Testing Strategy

test report analysis tools, Test report analysis

test runners, TestNG

test trophy test shape, Automated Functional Testing Strategy

Test.ai, Test authoring

Testcontainers, Test Containers-Test Containers

TestCraft, Test authoring

Testim, Test authoring

testing, Full Stack Testing for High Quality

ADRs (architecture decision records), Defect Prevention over Defect Detection

blockchain-specific, Testing Blockchain Applications-Testing Blockchain Applications

cloud-hosted platforms, Browsers

first principles

collaboration, Communication and Collaboration Are Key to Quality

communication, Communication and Collaboration Are Key to Quality

continuous feedback, Continuous Feedback

defects prevention over detection, Defect Prevention over Defect Detection-Defect Prevention over Defect Detection

empathetic testing, Empathetic Testing

fast feedback, Fast Feedback

macro-level testing, Micro- and Macro-Level Testing

metrics, Measuring Quality Metrics-Measuring Quality Metrics

micro-level testing, Micro- and Macro-Level Testing

IPMs (iteration planning meetings), Defect Prevention over Defect Detection

portability testing, Test Containers

shift-left testing, Shift-Left Testing

shift-left testing and, Shift-Left Testing

three amigos process, Defect Prevention over Defect Detection

user story kickoff, Defect Prevention over Defect Detection

testing skills, Ten Full Stack Testing Skills

accessibility testing, Ten Full Stack Testing Skills

automated functional testing, Ten Full Stack Testing Skills

continuous testing, Ten Full Stack Testing Skills

cross-functional requirements testing, Ten Full Stack Testing Skills

data testing, Ten Full Stack Testing Skills

manual exploratory testing, Ten Full Stack Testing Skills

mobile testing, Ten Full Stack Testing Skills

performance testing, Ten Full Stack Testing Skills

security testing, Ten Full Stack Testing Skills

visual testing, Ten Full Stack Testing Skills

TestNG, Unit tests, TestNG, Setup and workflow

TestRail, Automated Functional Testing Strategy

TFLint, Infrastructure Testing

third-party components, performance and, Factors Affecting Application Performance

threat modeling, The STRIDE Threat Model, Threat Modeling

- assets, Threat modeling steps
- black hat thinking, Threat modeling steps
- DDoS (distributed denial of service), Denial of service
- DoS (denial of service), Denial of service
- escalation of privileges, Escalation of privileges, Threat modeling exercise
- exercise, Threat modeling exercise-Threat modeling exercise
- features, defining, Threat modeling steps
- information disclosure, Information disclosure, Threat modeling exercise
- input tampering, Tampering with inputs, Threat modeling exercise
- prioritization, Threat modeling steps
- repudiation, Repudiation of actions, Threat modeling exercise
- spoofed identity, Spoofed identity, Threat modeling exercise
- steps, Threat modeling steps
- STRIDE model, The STRIDE Threat Model
- test cases, Security test cases from the threat model-Security test cases from the threat model

threats, security, Building Blocks

three amigos process, Shift-Left Testing, Defect Prevention over Defect Detection

transcripts, accessibility, Perceivable

U

UAAG (User Agent Accessibility Guidelines), Accessibility Ecosystem

UAT environment, The CI/CT/CD Process

UI (user interface)

- ecommerce UI, Introduction to Micro and Macro Test Types
- look and feel, The UI look and feel

UI functional tests, UI functional tests-UI functional tests

- Java-Selenium WebDriver
  - Maven, Maven-Maven
  - Page Object Model, Page Object Model-Page Object Model
  - prerequisites, Prerequisites
  - Selenium WebDriver, Selenium WebDriver-Selenium WebDriver
  - setup, Setup and workflow-Setup and workflow
  - TestNG, TestNG
- JavaScript-Cypress, JavaScript–Cypress Framework
  - Cypress, Cypress-Cypress
  - prerequisites, Prerequisites

setup and workflow, Setup and workflow-Setup and workflow

UI layer, Building Blocks

auth service, Building Blocks

UI-driven automated testing, Introduction to Visual Testing

UiAutomator, Appium

UN CRPD (United Nations Convention on the Rights of Persons with Disabilities), Accessibility Testing

understandability, accessibility, Understandable

unencrypted data, application vulnerabilities and, Unencrypted private data

unit tests, Unit tests-Unit tests

JUnit, JUnit-Setup and workflow

visual testing, Unit Tests

United Nations Convention on the Rights of Persons with Disabilities (UN CRPD), Accessibility Testing

Unity, Testing AR/VR Applications

usability testing

CFR testing, CFR Testing Strategy

internationalization, Usability

localization, Usability

user experience, Usability

IoT (Internet of Things) applications, Testing IoT Applications

UX (user experience) and, Visual Testing

User Agent Accessibility Guidelines (UAAG), Accessibility Ecosystem

user flow, Manual Exploratory Testing

user personas, accessibility, Accessibility User Personas-Accessibility User Personas

user personas, manual exploratory testing and, Understand the Application

user story kickoff, Defect Prevention over Defect Detection

UX (user experience), Shift-Left Testing

CFR testing, Usability

usability testing and, Visual Testing

V

VCS (version control system), The CI/CT/CD Process

benefits, The CI/CT/CD Process

Git, The CI/CT/CD Process

version control system (VCS) (see VCS (version control sysem))

virtual reality (VR) (see VR (virtual reality))

Visual AI, Applitools Eyes, an AI-Powered Tool

visual testing, Ten Full Stack Testing Skills, Visual Testing

Applitools Eyes, Applitools Eyes, an AI-Powered Tool, Applitools Eyes, an AI-Powered Tool

challenges, Perspectives: Visual Testing Challenges

change blindness, Introduction to Visual Testing

component level, Project/Business-Critical Use Cases

exercises

BackstopJS, BackstopJS-Workflow

Cypress, Cypress-Workflow

frontend testing strategy, Frontend Testing Strategy

accessibility testing, Accessibility Testing

cross-browser testing, Cross-Browser Testing-Cross-Browser Testing

frontend performance testing, Frontend Performance Testing

functional end-to-end tests, Functional End-to-End Tests

integration/component tests, Integration/Component Tests-Integration/Component Tests

snapshot tests, Snapshot Tests -Snapshot Tests

unit tests, Unit Tests

visual tests, Visual Tests

mobile testing strategy, Visual Testing

project/business-critical use cases, Project/Business-Critical Use Cases-Project/Business-Critical Use Cases

Storybook, Storybook-Storybook

tool selection tips, Perspectives: Visual Testing Challenges

versus snapshot testing, Visual Tests

VoiceOver screen reader, iOS

VR (virtual reality), Augmented Reality and Virtual Reality

application testing, Testing AR/VR Applications-Testing AR/VR Applications

Google Cardboard, Augmented Reality and Virtual Reality

HMD (head-mounted display), Augmented Reality and Virtual Reality

HTC VIVE, Augmented Reality and Virtual Reality

Oculus Go, Augmented Reality and Virtual Reality

Oculus Quest, Augmented Reality and Virtual Reality

Sony Playstation VR, Augmented Reality and Virtual Reality

vulnerabilities, security, Building Blocks

unhandled, Unhandled known vulnerabilities

W

W3C (World Wide Web Consortium), Accessibility Testing

WAI (Web Accessibility Initiative), Accessibility Testing

WAI-ARIA (WAI's Accessible Rich Internet Applications), Robust

WAVE accessibility evaluation tool, WAVE-Workflow

WCAG (Web Content Accessibility Guidelines), Accessibility Ecosystem

guiding principles, WCAG 2.0: Guiding Principles and Levels

Level A, WCAG 2.0: Guiding Principles and Levels

requirements, Level A Conformance Standards-Robust

Level AA, WCAG 2.0: Guiding Principles and Levels

Level AAA, WCAG 2.0: Guiding Principles and Levels

Web Accessibility Initiative (WAI), Accessibility Testing

Web Content Accessibility Guidelines (WCAG) (see WCAG (Web Content Accessibility Guidelines))

web scraping, Web scraping

web services, API Testing

web UI testing

browsers, Browsers-Browsers

Bug Magnet, Bug Magnet-Bug Magnet

Chrome DevTools, Chrome DevTools-Chrome DevTools

WebPageTest, WebPageTest-Workflow

WireMock API testing tool, WireMock-WireMock

X

XCode Accessibility Inspector, iOS

XCUITest, Appium

XP (Extreme Programming), Shift-Left Testing

XSS (cross-site scripting), Cross-site scripting, Cross-site scripting

Z

Zerocode, Apache Kafka and Zerocode

test creation, Workflow-Workflow

About the Author

Gayathri Mohan is a passionate technology leader with expertise across multiple software development roles and technical and industrial domains. Gayathri has proven her mettle by successfully managing large quality assurance (QA) teams for clients at Thoughtworks, where she is now principal consultant at Thoughtworks. While working as the company's global QA SME, she defined career pathways and the desired skill development structure for QAs at Thoughtworks. As office tech principal, Gayathri cultivated local tech communities, organized technical events and developed thought leadership across technical themes.

Gayathri is also coauthor of *Perspectives of Agile Software Testing*, released by Thoughtworks on Selenium's 10th anniversary.

Colophon

The animal on the cover of *Full Stack Testing* is a lowland streaked tenrec (*Hemicentetes semispinosus*). These small insectivorous mammals are one of many species of tenrecs found on the island of Madagascar. Lowland streaked tenrecs are typically found in scrubland, tropical lowland rainforests, agricultural land, and even some rural gardens on the eastern side of the island.

Lowland streaked tenrecs are easily identified by their long, pointed black snouts and small, tailless bodies striped with black and yellow quills. A crest of yellow spines covers the back of their necks. Their barbed quills are detachable and can be used as a defense mechanism; tenrecs also use the quills to communicate by rubbing them together, producing a high-pitched sound. Fully grown lowland streaked tenrecs are about five to seven inches long and weigh between four and ten ounces.

Lowland streaked tenrecs are social and gather in groups of up to 20. They dig connected burrows for nesting and forage for earthworms and insects individually or in small groups. In the winter, they go into torpor, a state of reduced body temperature and decreased metabolism. Females are only fertile for one year and are reproductively active at 25 days old, making them the only species of tenrec that can breed in the same season in which they were born. Lowland streaked tenrecs are classified as a species of least concern by the IUCN due to their widespread distribution, high abundance, and high tolerance to areas with large numbers of humans. Many of the animals on O’Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on a black and white engraving from *English Cyclopedia*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag’s Ubuntu Mono.